ngspice-46 — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

October 2026

Contents

ngspice-46 — Full Change Report 3
1. The OSDI ABI, version 0.4 → 0.7 . . . 3
2. The OSDI runtime — `src/osdi/` . . . 4
3. Analyses — `src/spicelib/analysis/` . . . 8
4. Frontend — `src/frontend/` . . . 9
5. Parser and device registry — `src/spicelib/` . . . 22
6. Maths — `src/maths/` . . . 47
7. Build system . . . 49
8. Per-enhancement index . . . 49

ngspice-46 — Full Change Report

Every modification applied to ngspice-46 in this project, with its reason. The baseline is the pristine ngspice-46 source tree (as vendored in the project's `original/` snapshot, which already contains OpenVAF-Reloaded's stock OSDI support); the current state is the tree committed in this repository under `ngspice-46/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document openvaf_changes_full-report.md covers the compiler side.

> Maintenance note: this report is updated whenever an enhancement touches ngspice sources. If you are reading it alongside a newer tree, the per-enhancement index at the end tells you the last change it covers.

Scope summary. Two new source files and 51 modified ones carry all the functional changes (~2,100 substantive diff lines). A further ~100 `Makefile.in` files and `config.h.in` differ only because the autotools were regenerated with a current libtool (E-77); they contain no hand-written changes and are not listed individually. Everything below is behavior, interface, or diagnostics.

---

1. The OSDI ABI, version 0.4 → 0.7

`src/osdi/osdi.h` is the authoritative C contract between ngspice and the `.osdi` objects `openvaf-r` produces. The project extended it six times — three times additively (new exported symbols old loaders simply never look up), three times with genuine version bumps (layout changes):

Change	Kind	Enhancement	Why
`OSDI_ABSDELAY_COUNTS` / `OSDI_ABSDELAY_INFOS` exports	additive symbols	E-1	`absdelay()` needs simulator-side waveform history and synthetic delay-equation rows; the descriptors tell ngspice which nodes/offsets each delay slot uses
`OSDI_LAST_CROSSING_COUNTS` / `OSDI_LAST_CROSSING_INFOS` exports	additive symbols	E-6	`last_crossing()` needs waveform history and crossing interpolation — same additive pattern as absdelay
`OSDI_TERM_SHORT_COUNTS` / `OSDI_TERM_SHORT_INFOS` exports	additive symbols	E-401	`V(a,b) <+ 0` between two TERMINALS cannot be served by node collapsing (terminals are simulator-allocated), so the compiler emits a real 0 V source and lists the branch; the simulator drops its current when the netlist already ties those terminals to one node, which is what stops the redundant equation being a singular row
`OsdiNode.nodeset` field	ABI 0.5 (node stride change)	E-45	Verilog-A net initializers (`electrical a = 5.0;`) become solver initial-guess nodesets; every node record carries the hint
`num_ac_stim_src` / `ac_stim_sources` / `load_ac_stim` descriptor tail	ABI 0.6 (descriptor grows)	E-51	full `ac_stim()` support: a Verilog-A module can *be* the AC stimulus, so the descriptor must enumerate its AC right-hand-side sources

Change	Kind	Enhancement	Why
`load_noise` destination stride 1 → 2 (signed `( flat, react)` power pairs)	ABI 0.7	E-54	correct noise physics: operating-point-dependent factors and `ddt()`-shaped (jω) noise need a complex amplitude per source, `re + jω·im`, so coherent same-named sources (LRM 4.6.4) sum with correct sign and frequency shape

Two additive flag conventions ride on the existing `eval()` interface (not version bumps):

- `EVAL_FLAG_IS_FINAL_STEP` (bit $1 \ll 21$) in the eval *input* flags — `@(final_step)` firing (E-53);
- eval *return* flags for `$finish`/`$stop` requests and `$discontinuity`-driven step rejection (E-55).

Pairing consequence: this ngspice requires OSDI $\geq$ 0.7 and rejects older objects with a clear version message; `.osdi` files from older compilers must be recompiled (E-54).

---

2. The OSDI runtime — `src/osdi/`

`osdi.h` — **`PARA_KIND_*`** unsigned shifts

The parameter-kind flag macros were `PARA_KIND_MASK (3 << 30)` and `PARA_KIND_OPVAR (2 << 30)` — both shift into the sign bit of a signed `int`, which is undefined behavior (a UBSan build flags it on every OSDI load, at `osdiinit.c:41`). The shifts are now unsigned (`3u << 30`, `2u << 30`, and the two zero/one companions for consistency). The bit patterns are unchanged (`0xC0000000` / `0x80000000`), so the OSDI ABI is byte-identical, and they now match the `u32` constants openvaf-r emits in `metadata/osdi_0_4.rs`. Found by the 2026-07 ASan/UBSan pass.

`osdiaccept.c` — new file (E-55, E-1, E-6)

The accepted-timepoint hook (`DEVaccept`) OSDI previously lacked. It advances the `absdelay/last_crossing` waveform histories only at *accepted* points (so rejected timesteps don't corrupt the delay lines) and reads the per-attempt latched `$finish`/`$stop` request flags at the one boundary where honoring them is safe. Why: acting on `$stop` mid-Newton-iteration was treated as a step failure and ground the timestep into a rejection loop; `$finish` was ignored outright.

`osdidefs.h` (104 diff lines)

- `OsdiExtraInstData` grows the per-instance fields behind every runtime feature: `dt/temp` offsets with given-flags (instance temperature), `eval_flags`, absdelay waveform-history rows and pre-allocated KLU/SPARSE matrix-pointer sets for the delay-equation rows (re-pointed on every DC↔AC transition, mirroring the regular Jacobian handling), and the last_crossing history (E-1, E-6, E-55).
- `ALIGN` macro renamed `OSDI_ALIGN`: the macOS SDK's `<sys/param.h>` defines an unrelated `ALIGN`, producing a redefinition warning in every OSDI translation unit (E-77).

`osdiitf.h` / **`osdiext.h`**

The registry-entry structure gains the absdelay/last_crossing descriptor pointers, and `OSDIpendingRequests()` is exported so the analyses can ask "did any device request `$finish`/`$stop` at this accepted point?" (E-1, E-6, E-55). It also gains the terminal-short pointers and the `OsdiTermShortInfo` record (E-401).

`osdiregistry.c` (68 diff lines)

- Loads the absdelay/last_crossing descriptor arrays from each `.osdi` and threads them into the registry entries (E-1, E-6).
- Loads the terminal-short arrays the same way (E-401).
- Requires OSDI $\geq$ 0.7 (E-54).
- Descriptor-count hardening (E-228): `load_object_file` strided/indexed the descriptor arrays by the counts the `.osdi` itself declares (`OSDI_NUM_DESCRIPTORS`, and per-descriptor `num_params`/`num_opvars`/`num_terminals`/...) with no bounds, so a corrupt, truncated, or ABI-mismatched object that still `dlopen`s but declares an out-of-range count read past the array (the inner loop dereferences each `param_opvar` entry's `name[]` pointers) → SIGSEGV during `pre_osdi`. Two generous ceilings (`OSDI_MAX_DESCRIPTORS 4096`, `OSDI_MAX_DESC_COUNT 1<<20`, far above any real model) now reject such a file with a clean diagnostic — the outer count before it sizes the registry / strides the array, and each descriptor's own count fields in the loop before they index anything (a single load-time choke point protecting all downstream consumers). Targets the egregious/out-of-range count (the truncated / byte-corrupted / ABI-drift signature — an ABI mismatch reads a pointer's bits as a count → huge); a count that lies within the plausible range is an internally inconsistent binary indistinguishable from a valid one and is out of scope, as is a hostile object (a `.osdi` is native code whose loading executes it).

`osdiinit.c` (8 diff lines)

- Registers the `DEVaccept` hook (E-55).
- Publishes the synthetic instance-temperature parameter under both spellings, `dt` and the conventional `dtemp` every built-in device uses; `n1 a b mod dtemp=10` used to fail with *"unknown parameter"* while the underlying plumbing was exact (E-80).

`osdiload.c` (441 diff lines — the largest OSDI change)

- Stamps the absdelay synthetic rows (`y_synth`, `z`) for DC, AC and transient, driving the delay from the recorded history (E-1).
- Evaluates `last_crossing` from the recorded waveform with linear interpolation between the bracketing timepoints (E-6).
- Passes the final-step flag through to `eval()` (E-53).
- Latches eval-return flags per timepoint attempt (`point_eval_flags`, OR-ed across Newton iterations, reset per attempt) so `$finish`/`$stop` under solution-dependent conditions survive to the accepted-point boundary (E-55).
- Folds the stride-2 signed noise-power pairs (`fac·|fac|` semantics) when transferring `load_noise` results (E-54, E-42).

`osdisetup.c` (178 diff lines)

- Creates the synthetic delay-equation unknowns and matrix entries for absdelay (E-1).
- Applies the OSDI nodesets (`OsdiNode.nodeset`) as solver initial guesses (E-45).
- Terminal shorts (E-401): `collapse_nodes` documents that terminals can never be collapsed (ngspice allocates them, not OSDI) and skips such a pair — which left `V(a,b) <+ 0` between two terminals connecting NOTHING, a silent open circuit where the model wrote a short. The compiler now emits a real 0 V source there and lists the branch in `OSDI_TERM_SHORT_INFOS`; setup drops that branch's current unknown (`drop_node`, the same renumbering an ordinary collapse-to-ground does) whenever the two terminals do not resolve to two DISTINCT CONNECTED circuit nodes. Without that half the equation is redundant wherever the netlist already ties the terminals together — the normal wiring of a self-heating model — and a redundant 0 V source is a singular row.

- A `$fatal`/`$finish` raised during setup (models rejecting their configuration by design — HiSIM's `\$port_connected` guards) used to surface as ngspice's baffling *"impossible error — can't occur"*; it now reads *"a Verilog-A device rejected its configuration during setup"* (E-56).
- An internal OSDI node that appears in no Jacobian entry — a net structurally decoupled from the matrix, e.g. an explicit `ground gnd` reference whose branch contribution `V(p,gnd) <+ …` drops the `gnd` column — used to be allocated its own solver row. That all-zero row/column is benign under Sparse (it decouples to `V=0`) but makes the KLU matrix structurally singular, so KLU returned a wrong DC answer (`groundcontrib`: `v(p)=0` not `1.5`; `hierbranch`: branch currents `0`). Such nodes are now tied to ground (node 0) at setup, matching how OpenVAF already treats them and fixing both solvers (E-116).

`osdiacld.c` (89 diff lines)

- AC loading gains the `ac_stim` right-hand-side injection: the partitioned AC-stimulus source array is loaded through `load_ac_stim` and added to the AC RHS with exact magnitude and phase, `$mfactor` applied linearly (deterministic signals scale with multiplicity) (E-51, E-26).
- Complex (jω-shaped) noise coupling entries participate in the AC matrix (E-54).

`osdinoise.c` (93 diff lines)

- Per-instance grouping of same-named noise sources: coherent summation of complex amplitudes `(a + jω·b)·T` before squaring, per LRM 4.6.4 — same-named sources correlate (including exact anti-phase cancellation), differently-named ones stay independent (E-42).
- Reads the stride-2 signed pairs of ABI 0.7 (E-54).

`osditrunc.c` (34 diff lines)

- Honors `$discontinuity(n)`'s next-step clamp: a negative `bound_step` sentinel from the model limits the step after the event (E-24).
- Honors the step-rejection return flag: when an event fired inside the step, `OSDItrunc` requests `delta/8` (with a `20·CKTdelmin` termination floor) so the integrator bisects onto the discontinuity instead of extrapolating across it (E-55).

`osdiparam.c`, **`osdisetup.c`**, **`osdiregistry.c`**, **`osdiinit.c`**, **`osdidefs.h`** — the collapse-owner map

A terminal collapsed onto an internal node (`V(d,di) <+ 0`, the `rd = 0` default path of essentially every compact model) reported a terminal current of exactly 0.0: the model writes its current into `di`'s residual and terminal `d`'s slot stays zero, while `OSDIask` read only `descr->nodes[t].resist_residual_off`. With `rd` and `rs` both zero every terminal read 0.0 and the reported currents summed to zero, so a Kirchhoff check passed on `0 = 0`. The solution was never affected — `osdiload.c` stamps the whole group into one matrix row, which is why nothing a user plots ever moved.

`OSDIask` now sums the terminal's whole collapse group, resistive residual plus (in a transient) the integrated reactive residual, striding the state cursor over non-members exactly as `osdiload.c` does. The grouping cannot be recovered after setup — `write_node_mapping` replaces the local mapping with global node numbers, and two terminals on one net share a global number while ground additionally collects grounded terminals, nodes collapsed to ground, E-116's decoupled internal nodes and E-401's dropped term-short flow nodes. It is therefore snapshotted the moment `collapse_nodes` returns, into one `uint32_t` per descriptor node trailing `OsdiExtraInstData` (`osdi_collapse_owner`), holding `t + 1` for "owned by terminal t" so that the `calloc`ed zero means "setup has not run" (E-416).

osdiparam.c, **osdisetup.c**, **osdiinit.c**, **osdiregistry.c**, **osdidefs.h**, **cktsens.c**, **breakp2.c** — the collapse decided once, re-decided later

OSDItemp re-runs setup_instance on every temperature update, so a model can re-decide its node collapse — but nothing derived from the first decision can be rebuilt there (the node mapping, the matrix element pointers, the state layout, E-416's owner map). The file's own // TODO check that there are no changes in node collapse? is now answered: OSDItemp clears collapsed[], re-decides, compares against a snapshot taken in OSDIsetup, and restores the snapshot unconditionally before the error check, because the matrix implements the snapshot. Clearing first is the whole fix — OpenVAF's collapse callback only ever STORES TRUE, so without it a collapse that stopped happening is invisible, which is the direction the defect takes. The snapshot is one bool per collapsible pair trailing E-416's owner map in the same instance block.

Three consumers acted on the mismatch. cktsens.c perturbed a collapse-selecting parameter and differenced two loads that shared one matrix element: the branch's +g and −g cancelled on the same diagonal and what survived was eps*E/(Y*delta^2) — roundoff whose magnitude tracks 1/rd and whose sign moves with unrelated parameters (2.5289e−02 printed where the truth is 2.2676e−04). It is now reported as 0 with a warning naming instance and parameter; it cannot be computed correctly, since DEVsetup runs only at the base value and re-running it would allocate nodes and trip cktsens's own CKTlastNode guard. .dc temp ran a whole sweep on one topology — exactly 0 A sweeping into a collapse, a plausible 2x error sweeping out of one — and now warns once per instance; a hard error would mean propagating five bare CKTtemp(ckt); returns in dctrcurv.c and changing error handling for every device model in the tree. breakp2.c stopped skipping the per-terminal expansion at exactly two terminals, where @dev[i_p] had been a length-1 scalar while @dev[i_d] was a waveform at three; the bare @dev[i] is kept beside it and synthesized names are de-duplicated against explicit .save entries (E-417).

osdisetup.c, **outitf.c**, **com_measure2.c** — three things nobody checked

absdelay() and last_crossing() are the only analog operators whose OUTPUT row the compiler deliberately leaves empty: ngspice fills it from its own history buffer, through matrix elements OSDIsetup allocates later. E-116's decoupled-node scan can only see coupling the COMPILER declared, so it found the output node in no Jacobian entry, tied it to ground, and eval() read CKTrhsOld[0] -- exactly 0.0, forever. The value was right only when the model also CONTRIBUTED it, because that is what put the node into a Jacobian entry; transition() shares the same lowering and worked only because its output feeds the slew residual. Those nodes are now marked used before the scan decides. E-415 had looked for this in the compiler and reverted when the descriptor came out byte-identical -- correctly, since nothing was wrong there. No OpenVAF change and no ABI change: the descriptor already carries the infos and osdiregistry.c already parses them.

.save validated nothing. Nothing between settrace and the per-point read looked a name up, and getSpecial's caller discards INPaName's E_NODEV/E_BADPARM, so a misspelled device, a bogus parameter or an unexpanded wildcard produced a registered vector that stayed 0 long in silence. The check now calls INPaName itself -- the same routine the per-point read uses -- and a hierarchical spelling is REWRITTEN rather than rejected, so E-410's @x1.r1[i] resolves while the display name stays the user's. Unresolvable entries are warned about and still added, because dropping them would change what the plot contains.

meas ... when interpolated a crossing-free interval. The initialisation counted a crossing in the FIRST interval -- which runs from the operating point to the first timepoint and so straddles thresholds for reasons unrelated to the waveform -- and the evaluation, which deliberately starts one sample in, applied that leftover count to a later interval, dividing by a difference that was exactly zero or one ULP: −inf, 1.15292e+05 s in a 3 us run, negative times. The count is no longer taken, every interpolation is gated on the interval actually bracketing the target (mirroring measure_at), and a .dc sweep restart resets to sample 0 so the previous sweep's last point is never a bracketing endpoint (E-418).

`pz` filled the simulator-stamped row nowhere. Because those rows are the simulator's to fill, every load path has to fill them -- `osdiload.c` does for dc/tran and `osdiacld.c` for ac, but `osdipzld.c` did for neither, so the row was identically zero, the matrix was singular at every trial `s`, every trial looked like a root, and pz blamed the netlist: "the input signal is shorted on the way to the output". This predates the fix above for a model that CONTRIBUTES the value (its row was already live), and marking the nodes used would have widened it to every observed-only model. The AC stamp cannot be reused: `e^-s*td` overflows across pz's own search range and a transport delay has infinitely many roots, so there is no finite pole-zero set to find. `absdelay` is therefore stamped as the zero-delay wire -- the same linearization the operating point uses -- and warned about once per instance; `last_crossing` is stamped exactly, with no warning, its small-signal sensitivity being zero. All three cases now report the delay-free pole to the digit, including the two that aborted before E-418 (E-418).

`osdi/Makefile.am`

Adds `osdiaccept.c` to the build (E-55).

---

3. Analyses — `src/spicelib/analysis/`

`dctran.c` (46 diff lines)

- Advances OSDI delay/crossing histories via the accept path and skips history corruption on rejected steps (E-1, E-6).
- Issues the dedicated `@(final_step)` evaluation at the converged end of a successful transient (E-53).
- Honors accepted-point `$finish` (ends the analysis cleanly, firing `final_step` at the requesting point), `$stop` (pauses resumably), and aborts with a clear error on `$fatal`'s `E_PANIC` instead of retrying it as nonconvergence (E-55).

`dcop.c` (9) and **`acan.c`** (10)

- Fire `@(final_step)` at their successful end (an operating point fires both step events — a single point is first *and* last) (E-53).
- An AC job's operating-point phase carries the analysis name bit (only the name — adding the reactive `CALC_*` bits would wrongly enable integration at an op), so `analysis("ac")` holds through the whole AC run per LRM 4.6.1 and `@(initial_step("ac"))` fires in AC (E-53).

`dctrcurv.c` (150) + **`include/ngspice/trcvdefs.h`** (13)

- Generic **`.dc @inst[param]`** sweeps (a new sweep code): the DC sweep previously hardcoded Vsource/Isource/Resistor/temperature, so sweeping any other device parameter was a fatal error. The generic sweep resolves `@inst[param]` through the device's own parameter tables, refreshes per point via `DEVtemperature` (for OSDI exactly the `alter` path), nests with other sweep variables, and restores the original value afterwards (E-62).
- Fires `final_step` at the last sweep point; honors a `$finish` raised mid-sweep (E-53, E-55).

`noisean.c` (37)

- A singular AC matrix during noise analysis crashed ngspice with a SIGABRT: the code ignored `NIacIter`'s return and the adjoint solve asserted on the unfactored matrix. It now aborts cleanly ("AC solution failed at … Hz") (E-56).
- Honors deferred `$finish`/`$stop` raised at the noise operating point and fires `final_step` on success (E-55, E-53).

span.c (31) + **cktspdum.c** (14)

- 1-port S-parameter analyses enabled: the "we need at least two ports" error was over-strict (a 1-port is a plain reflection measurement; the matrix machinery is N-general) — and it was hiding the `cadjoint` crash fixed in `dense.c` (E-64).
- **Rbase** published into the sp plot from port 1's `z0`, making the Touchstone writers work with no manual `let Rbase = 50` incantation (E-64).
- Stock NaN fixed in **donoise** noise-parameter extraction: the uncorrelated noise conductance `Gu` is analytically zero for fully-correlated single-source topologies, and floating-point rounding could land `sqrt(Ycor² + Gu/Rn)`'s argument at $-10^{-18} \to$ NaN for the noise figure. Clamped to the physical range ≥ 0 — found by parity testing against an OSDI resistor that produced the exact textbook NF where the built-in returned NaN (E-63).

cktdisto.c (16)

- `.disto` silently reported zero distortion for OSDI devices — the distortion kernel needs higher-order Taylor coefficients the OSDI ABI cannot provide (first derivatives only), and such devices were skipped without a word. A prominent warning now names each affected OSDI device type (E-62).

---

4. Frontend — **src/frontend/**

plotting/pyplot.c (new) + **com_pyplot.c** (new) + **plotit.c**, **commands.c** (E-94)

A new interactive command, **pyplot**, plots simulated vectors with matplotlib — a Python counterpart to `gnuplot`. `ft_pyplot()` (`plotting/pyplot.c`) mirrors `ft_gnuplot()`: it writes a `<file>.data` table and a `<file>.py` matplotlib script and `system()`s `python3` (synchronously for a PNG, backgrounded for an interactive window). A new `"pyplot"` device arm in `plotit()` routes to it, so it accepts the same plot expressions as `plot`/`gnuplot`; the `com_pyplot()` wrapper mirrors `com_gnuplot()`, and `pyplot` is registered in both command tables. Two `set` variables tune it: `pyplot_terminal=png` renders headless (Agg) to a PNG, and `pyplot_python` picks the interpreter (default `python3`). Purely additive (E-94). The output file base name is optional (E-95): `com_pyplot()` treats the first word as a file name only when it is not a plot expression (no `(`, and not a vector by `vec_get()`), otherwise it defaults to `pyplot` — so `pyplot v(out)` plots directly (the command's minimum argument count was lowered from 2 to 1). `ft_pyplot()` also grew stacked subplots (`set pyplot_subplots=N` → `plt.subplots(nrows, 1, sharex=True)`, `N` traces per panel; `vs` stays the x-axis, so it is not a panel separator) and matplotlib style sheets (`set pyplot_style=<name>`, `dark` → `dark_background`, guarded) (E-98). The headless `pyplot_terminal` was extended from PNG-only to the vector export formats `svg` and `pdf` (`set pyplot_terminal=svg/pdf` → `fig.savefig(<file>.<fmt>)`, matplotlib picking the writer from the extension), and a figure size control was added (`set pyplot_figsize="W,H"` → `plt.subplots(..., figsize=(W, H))`; quote the value so ngspice keeps the comma) (E-99). A **-eye** flag turns `pyplot` into a one-line eye-diagram renderer over the E-207 `eye` analysis: `com_pyplot()` detects the `-eye` marker, runs `com_eye()` on the trailing tokens (which folds the waveform and leaves `eye_wave`/`eye_t` + the scalar metrics in a fresh current `eye` plot), then a new **ft_pyplot_eye()** reads those back and writes a `hist2d`-based persistence eye script (`LogNorm`, `turbo`, empty cells masked; threshold + eye-centre lines, eye-height/width double-arrows, a metric title), launched by the same `system(python …)` mechanism as `ft_pyplot()` and honouring the same `pyplot_terminal`/`pyplot_python`/`pyplot_backend`/`pyplot_figsize`/`pyplot_style` settings. `ft_pyplot()` and the gnuplot/asciiplot back-ends are untouched (E-208).

Four further groups of additions. Appearance controls — seven `set` variables (`pyplot_grid`, `pyplot_legend`, `pyplot_markers`, `pyplot_axhline`, `pyplot_axvline`, `pyplot_dpi`, `pyplot_transparent`) threaded into `ft_pyplot()`, all additive so a control-free plot is byte-identical to before (E-296). A **-fft** mode — `ft_pyplot()`'s `bool hist` became an `int mode` (`LINE`/`HIST`/`FFT`); `-fft` reuses the time-

domain data table and the emitted script resamples onto a uniform grid (`np.interp`, because transient data is adaptively sampled) before `np.fft.rfft`, scaled by `2/sum(window)` so a tone reads back its amplitude (`pyplot_fft_window`/`_db`/`_points`/`_logf`) (E-297). Complex-aware AC views — a new `ft_pyplot_ac()` renderer for `–bode` (mag dB / phase deg vs log-f, stacked, `np.unwrap`), `–nyquist` (imag vs real) and `–polar` (mag at phase), emitting a `<vec–index> <freq> <re> <im>` table that KEEPS the imaginary part an ordinary `pyplot` discards (E-298). And finishing touches — cross-run overlay (`pyplot tran1.v(out) tran2.v(out)`) now sizes the data table by the longest scale so a finer second run is not truncated, plus a `pyplot_cursor` hover crosshair (matplotlib's built-in `Cursor`, window-only) (E-299). A `pyplot_mplcursors` variable then selects the `mplcursors` package (data cursors that snap to a trace) as the cursor backend, turning the cursor on by itself, with a try/except fallback to the built-in `Cursor` if the package is absent where the emitted script runs (E-300). The cursor gating was then consolidated so `pyplot_cursor` is the single master switch (off by default, the only enable); `pyplot_mplcursors` only selects the backend when the cursor is on (E-301).

`postcoms.c` (414 diff lines) + **`postcoms.h`** + **`commands.c`** (16)

The Touchstone I/O subsystem (E-64, E-72):

- **`wrsnp`** (with `wrs2p` dispatching to the same handler): Touchstone v1 for any port count — the classic 2-port `S11 S21 S12 S22` column order preserved byte-identically, N $\geq$ 3 in the spec's row-major layout with at most four complex pairs per line;
- writer options `wrsnp <file> [ri|ma|db] [s|y|z] [hz|khz|mhz|ghz]`, any combination in any order: MA/DB formats, Y/Z parameters (normalized to Rbase, `Y·R` / `Z/R`, per the v1 spec), frequency units — the option line reflects every choice;
- **`rdsnp`**: reads any Touchstone v1 file into a new plot with a Hz `frequency` scale and complex vectors matching the `.sp` plot's conventions (MA/DB converted back, Y/Z de-normalized, the 2-port column order handled, `Rbase` published so imports round-trip) — measured data diffs against simulation in one `let` expression.

Why: E-63 showed the S-parameter *analysis* was exact but its results could not leave ngspice in the industry-standard format (`wrs2p` demanded a never-created `Rbase` vector), and nothing could bring VNA data in.

`outitf.c` (16)

- Integer operating-point variables recorded per point: `getSpecial` masked with `IF_REAL` only, so an integer opvar saved with `.save @n1[flag]` produced garbage in transient vectors (E-32).
- The plot-memory guard's message printed `%Id` — a Windows-only printf length modifier that is undefined behavior elsewhere and rendered the message as the numberless `"memory required (Id Bytes)"`. Now `%zu`: real byte counts on every platform (E-77).

`resource.c` (27)

`ft_ckspace()` — the *"approaching max data size"* warning (RSS > 95% of RSS + free RAM) — now honors **`set no_mem_check`** (the same opt-out the fatal output-size estimate in `outitf.c` respects; one variable governs both memory guards) and warns once per excursion above the threshold, re-arming below 90%, instead of repeating on every check through a long analysis (E-81).

`display.c` (1)

`#undef NODEV` before the local macro definition — the macOS SDK's `<sys/param.h>` defines an unrelated `NODEV` (E-77).

inpcom.c, **parse.c**, **vectors.c**, **device.c** + **analysis/dctrcurv.c** — bracketed names (E-408)

Three defects on the path from a name written in a deck to the object it denotes, none of which reported anything.

A leading zero split a node in two. n[1], n[01] and n[001] were three distinct nodes — print all listed all three — while the vector lookup canonicalised the index before resolving it, so all three probes returned the n[1] value. With a single node the same mismatch is worse: a netlist that built m[02] was unreachable by every spelling, its own included, because the lookup only ever asked for m[2]. Enhancement-221's own header settles the question (*"the scalar names produced use the same bracket form, so a bus a[0:1] and an explicit a[0] denote the same node"*), so inp_canon_bus_index_token() canonicalises the index in the netlist, negative indices included; it is a byte-identical no-op when the spelling is already canonical.

@dev[param] could not name a bracketed parameter. show lists a bus device's terminal currents i_a[0]..i_a[3] (E-394) and its array parameter elements ap[0]..ap[2] with correct values, and the instance line can set them, but four separate splitters each cut the name at the inner] and reported *"no such parameter i_a[0."*. All four had to change because each serves a different command: PPlex (parse.c) lexes the token at all, vec_get (vectors.c) serves print and let, com_alter (device.c) serves alter, and DCTfindInstParam (dctrcurv.c) serves E-62's .dc @inst[param] sweep, which had been aborting with a fatal error. The E-269 wildcard alias @*[[param]] works *because* of the old first-] split, so a name beginning with [keeps it deliberately.

A bus range expanded everywhere except where the answer is read. [lo:hi] expanded on instance lines, R/C node lists, subcircuit calls and port lists, but not on .save/.print/.plot (the card produced nothing) or .ic/.nodeset (warned and ignored). Those cards were skipped by design: inp_expand_buses admits lines carrying bare node fields, and an output card writes v(a[0:3]). inp_bus_rewrite_wrapped() emits one copy of the whole token per index, so the wrapper and any =value ride along; a spaced .ic v(a[0:1]) = 0.25 absorbs its value into the token first. A token naming two ranges has no unambiguous expansion and stays literal.

com_sweep.c + **spiceif.c** — a wildcard sweep knob is put back (E-409)

sweep @*[wavelength_nm] ... ran correctly and then printed

```
Error: no such device or model name
PPerror: syntax error in line segment
   @*[wavelength_nm]
near
      wavelength_nm]
```

The name looks mangled, which is what made the message alarming, but nothing was wrong with the parameter: the expression lexer's specials set contains *, so @*[p] cannot lex as one token and the parser reports the leftover text.

The message was the harmless part. sw_read_knob() is how E-385 captures the nominal it puts back at the end of a sweep, and it parses the knob name as an expression — the call that cannot succeed for a wildcard. The read reported failure, and E-385's deliberate all-or-nothing rule then skipped restoring everything: a wildcard sweep left every target at its last swept value, and with -vs it took a concrete co-knob down with it (@dev1[wavelength] left at 3.0 although it restores perfectly on its own).

One number could not fix it. A wildcard sets every matching model or instance to one value, but the values it overwrites can all differ, so undoing it needs one reading per target. if_saveparam_wildcard() / if_restoreparam_wildcard() walk exactly the same targets in exactly the same order as the existing if_setparam_wildcard{,_instance} — same device-type loop, same model and instance chains, same parmlookup(..., inout=set) predicate — so index *i* of the saved array names the same target on the way out as on the way in, reading through the parameter's askable twin and the same doask/doset

pair a plain `alter`/`altermod` uses. Saving stays all-or-nothing (a set-only or vector-valued parameter refuses the whole capture), and the replay is pinned to the circuit the readings came from, so a `.param` co-knob that re-sources the deck cannot have them replayed onto a different one.

`spiceif.c`, **`dctrcurv.c`**, **`com_sweep.c`**, **`gens.c`** — a subcircuit device without its type letter (E-410)

A resistor inside a subcircuit is `@r.x1.r1[resistance]`, while the node beside it is plain `x1.m`. The obvious spelling `@x1.r1[resistance]` named nothing.

The letter is not an accessor convention — `r.x1.r1` is the device's real name. ngspice flattens a subcircuit by rewriting its cards back into the deck and re-parsing them as ordinary element lines, and the parser takes the device type from the FIRST CHARACTER of the card (`inppas2.c`), so the flattened refdes has to keep a type letter in front: the card `x1.r1 a m 1k` would otherwise be re-read as another subcircuit call. Two details confirm it: `translate_inst_name()` (`subckt.c`) exempts `x` devices, whose names already start with the right letter, so `x2` inside `x1` is plain `x1.x2`; and NODES, having no type, keep plain hierarchical paths.

The reconstruction needs no search. The letter prepended is literally the leaf name's own first character, and a device's name must already begin with its type letter, so `x1.r1` can only mean `r.x1.r1` — at any depth, and for OSDI devices alike (`x1.x2.nd1` -> `n.x1.x2.nd1`). `if_find_instance_hier()` is consulted only after the exact lookups fail, so every name that resolves today is unaffected.

Following E-408's lesson, every consumer is routed through it rather than just `print`: `finddev()` (print/let/alter/altermod), `finddev_special()`, `DCTfindInstParam()` for E-62's `.dc @inst[param]` sweeps, and `sw_fp_bind()` for the `sweep` knob. For `sweep` the old behaviour was not even an error — the knob silently failed to bind and the three points came out identical.

`show` needed a different answer: its grammar takes the query's first character as the device-TYPE selector and uses `:`/`#`, not `.`, as the subcircuit delimiter, so `x1.r1` parsed as *type x* and could never match a resistor. `dgen_hier_match()` is consulted alongside that grammar as a whole-word alternative and can only add a match. A first attempt widened the `strcmp` inside the decomposition instead; it was measured as DEAD CODE (the query bails at the type check long before) and reverted.

an option matched by substring, and three called unknown (E-451)

Two findings in option handling, both from asking the question E-450 had just answered for `savecurrents`: which OTHER options are decided by searching the line for a word?

`inp.c` -- **`seed=`** and **`cshunt=`** were found by substring. `eval_opt()` located them with a bare `strstr` over the option line, so any option whose NAME MERELY ENDED in the watched text was taken as that option:

```
.options seed=7        sunif -> 4.3854057230e-01     (correct)
.options myseed=7                4.3854057230e-01     identical
.options noseed=7                4.3854057230e-01     the spelling that reads "off"
.options xseed=7                 4.3854057230e-01
   baseline, no seed set:        6.8709117128e-01
```

and for `cshunt` the answer moves by SIX ORDERS OF MAGNITUDE -- a node reading `1.0000000000e+00` reads `6.9209970972e-07` under `.options nocshunt=1e-6`, and the same for `mycshunt=` and `xcshunt=`. Each of these ALSO prints `Warning: unknown option 'nocshunt'`, which makes it worse rather than better: the user is told the option was not recognised, and it changes the answer anyway. For a Monte Carlo run the `seed` case means a mistyped option name silently changes the random sequence while being reported as ignored. Matched as whole tokens now, on E-450's boundary rule; `seedinfo` too, so `noseedinfo` no longer switches it on and `seed=` no longer sees the `seed` inside it.

NOT a defect, though it reads like one: the loop is `for (card = deck; ...)` with no `.options` test, which looks as though any deck line carrying `seed=` would hijack the generator. It does not -- the function's own comment says *"Input is the option deck (already sorted for .option)"*, and measurement agrees: a `.param seed=7`, a model parameter named `seed`, and a comment containing `seed=` all leave the sequence at baseline.

`spiceif.c` -- three options took effect while being reported unknown. Asking which flagged names demonstrably change a run: `scale=2` doubles `@m1[w]` (1e-6 -> 2e-6), `rseries=100` moves a transient, and **`autostop`** truncates one from 567 rows to 2 while being reported as a name ngspice does not know. That is the case E-447 fixed for `savecurrents`, `seed` and `numdgt`, with three names it did not cover, and its reasoning applies unchanged -- a warning that fires on a setting the run then honours teaches the user to ignore the check E-438 added. `scale` and `rseries` are read out of the deck's own option cards, so like `seed` they never reach the `.options` parameter table the check consults. **`scalm`** is flagged too and is deliberately NOT registered: it could not be shown to change anything, and this list is for options that demonstrably take effect.

savecurrents could be requested but never declined (E-450)

`inp.c` -- a bare substring search decided the option. Whether `.options savecurrents` was in force came down to `strstr(options->line, "savecurrents")`, so EVERY option line merely CONTAINING the word switched it on, whatever the line actually said:

```
.options savecurrents            ON     (correct)
.options savecurrents=0          ON     <- says off, does on
.options savecurrents=false      ON
.options nosavecurrents          ON     <- ngspice's own no<option> convention
.options mysavecurrentsxyz       ON     <- any identifier containing the word
```

`nosavecurrents` is the one that matters, because it is not an invented spelling: `noacct`, `noinit`, `nomod` and `nopage` are all real options, so a `no` prefix is exactly what a user reaches for. Once on there was no way back -- the feature could be requested but never declined.

Silent in every case, because nothing goes wrong when it fires: a deck that quietly saves every terminal current still simulates correctly, it just carries vectors the user asked not to have. On a large deck that is the difference between a small rawfile and a very large one, and on an OSDI device it is one vector per terminal -- which is how it surfaced, a user noticing `i_<terminal>` vectors they had not asked for.

The option is also reachable ONLY from an option CARD -- the deck, an `.included` file or a `.lib` section -- and not from `spinit`, `.spiceinit`, `set` or a `.control` `option`, because `inp_savecurrents()` is a netlist pre-pass run before the shell-variable machinery matters. That is the same "too late" ordering E-411 recorded and the reason E-449 reads its own option from the option lists. It narrows where the wrong answer can come from, and makes it stickier once there.

The line is now read as TOKENS: a token belongs to the family when it is exactly `savecurrents` or begins `savecurrents_` (the declared variants being `savecurrents_bsim3`, `savecurrents_bsim4`, `savecurrents_mos1`), so an unrelated identifier no longer matches; a `no` prefix or a `0`/`false`/`no`/`off` value turns it off; the later card wins, as a repeated `.param` or `.options` value already does. WHICH card is returned is deliberately unchanged -- the caller re-searches that one line for the `_bsim3`/`_bsim4`/`_mos1` names to choose the MOS current set, so the FIRST enabling card is returned exactly as the old first-match did, and splitting the family across two cards behaves as it always has (E-399: no wider than the evidence).

autobus across a subcircuit boundary (E-449)

E-444 lets one node name stand for a whole Verilog-A bus port, and does it in `INP2N` because that is where the model -- and so the bus width -- is known. Inside a `.subckt` that is too late.

subckt.c, **inp.c** -- the bus bits went to nodes that went nowhere. A definition declaring a[0:4] has the five formals a[0]..a[4], so a device line writing the bare a matched none of them. Flattening turned it into the local node x1.a and INP2N expanded THAT, leaving x1.a[0]..x1.a[4] -- five floating nodes with the device wired to them instead of to the ports, the five bits all reading one identical voltage and the device contributing nothing.

Nothing was reported, because every terminal DID receive a node, so E-402's under-connected warning had nothing to say. Turning the option ON therefore REMOVED the diagnostic the same deck gets with it off -- autobus OFF warns "4 of the 6 terminals ... are not connected", autobus ON says nothing at all and rc=0. That is exactly the shape E-445 recorded when autobus indexed ground: switching a feature on removing an existing warning.

The fix is at the SUBSTITUTION, not at device parse. A token that is not a formal, but for which formals token[i] exist, expands to those formals' actuals; it needs no model, only the .subckt line, so it works at flattening time. The option is not reachable from there in the obvious way: cp_getvar("autobus") is false during flattening, because inp_subcktexpand() runs before inp_dodeck() publishes the options, AND the .option cards are no longer in the deck either, having been split into the option lists -- a deck scan finds nothing. inp.c reads them from those lists and passes the answer in, the same way it already reads scale a few lines above the expansion call. Restricted to OSDI instance lines, so a bare a on an R/C/L line inside the same subcircuit stays the ordinary local node it always was.

Bit order comes from the declaration, not the table. The bits are emitted by ASCENDING BIT INDEX, because a compiled model always orders a bus port's terminals that way whatever direction the Verilog-A declared -- inout [4:0] a still yields terminals a[0]..a[4], measured rather than assumed -- and the instance line is positional. A DESCENDING .subckt declaration therefore binds in reverse, which is E-411's rule (the written order decides) applied one level up; a model declared [4:0] still matches its own flat form.

The other subcircuit form was already correct and is unchanged. .subckt mysub a b declares two SCALAR formals, so both tokens on the device line ARE formals, flattening substitutes them normally, and INP2N expands the caller's own names: the subcircuit has two pins and the bits live in the caller's scope as p[0], p[1], q[0]... That was verified here against a fully-written-out reference for one instance and for two instances with different actuals. The two forms are different interfaces -- one passes a bus through as a unit, the other exposes one pin per bit -- and both are pinned.

Out of scope, deliberately: writing ONE BIT where the whole bus is expected (N1 a[0] b inside a subcircuit whose port is a[0:4]) still creates floating n0[0]..n0[4] silently. a[0] is a formal, so flattening resolves it to the actual n0, and by INP2N nothing distinguishes it from a legitimate top-level line whose bus base happens to be called n0. It is BYTE-IDENTICAL before and after this change; E-445 guarded the two spellings it could recognise, this one is not recognisable at either end, and a heuristic for it belongs in its own investigation (E-399: a fix must be no wider than its evidence). Bus shorthand on a subcircuit INSTANCE line (Xi a b inner) is likewise unsupported, but fails loudly with "too few nodes" rather than mis-simulating.

a node named after a constant (E-448)

ngspice keeps a permanent const plot of twelve vectors — c, e, i, pi, kelvin, boltz, echarge, planck, TRUE, FALSE, yes, no — and vec_get() falls back to it when the current plot has no such vector. That fallback is how a bare pi resolves and it stays. It also answered a node of that name with the constant, in three distinct ways.

evaluate.c — **v(X)** answered with the speed of light. v(X) asks for a node voltage and a constant is never one, but the name was resolved before the context was considered, so a node that had been renamed or mistyped was answered rather than refused: v(c) gave 2.9979245800e+08 where v(nosuch) correctly gave "no such vector". The other eleven names are more dangerous than the speed of light because they look like results — v(no) is a flat 0.0, and v(i), v(yes) and v(TRUE) a flat 1.0, values

indistinguishable from a grounded node or a real unity signal. It defeated an existing guard: E-431 refuses a `sweep -output` expression that never resolves, and that refusal works — but the constant walked through it because the name *did* resolve, so `-output vo=v(c)` was drawn as a flat three-point curve while `-output vo=v(nosuch)` was refused. In a sweep the output IS a curve plotted against the knob, so a flat line reads as a legitimate finding. A node context no longer accepts a constant-plot binding. The related forms `v(c,0)`, `vm/vp/vr/vi/vdb(c)`, `i(c)` and `mag(v(c))` were checked and already refused — `v` is the only entry in the function table with a null handler, which is why it alone had the hole.

`evaluate.c`, **`parse.c`** — a bus bit named **`c[0]`** was unreachable. `.option autobus` (E-444) builds the node names `c[0]`..`c[4]` for a Verilog-A bus port called `c`. Those nodes were created correctly and the circuit solved correctly around them, yet none could be named: `display` listed `c[0]` and `print all` printed it as `5.0000000000e-01`, while `print c[0]` answered "indexing a scalar (c)" and `print v(c[0])` answered "bad v() syntax". E-444's own five-bit ladder, with the bus renamed from `a` to `c`, printed nothing at all. E-224 already preferred the literal name `a[0]` over "index 0 of a vector `a`", but only when the base was an UNRESOLVED zero-length placeholder — and for all twelve constant names a vector called `c` always exists, so the literal path was never tried. The gate is now the literal vector's own existence.

The silent one needs no constant. The same gate failed whenever the base resolved to a LONGER vector, which in a transient run is every node. A circuit holding both a scalar node `q` and a bus bit `q[0]` answered `print q[0]` with **`7.5000000000e-01`** — a scalar, node q's value at t=0 — instead of the bus bit's 59-row waveform, with no diagnostic. In an `op` run the same expression errors instead, because every vector is length-1, so the failure mode flipped with the analysis type. This is what argues the fix is about bracketed names generally rather than about the constant plot.

`vectors.c` — a bare name still resolves to the constant, but not in silence. With the two above fixed, `v(c)` and `c[0]` can no longer reach the fallback at all; what remains is a BARE `c`, genuinely ambiguous since the constant plot exists precisely so bare `pi` works. It still resolves to the constant and now says so, but only when the name also exists as a vector somewhere else — so every ordinary use of a constant stays silent.

One rule, four places. The bracketed-name rule was re-derived in `checkvalid()` and `PP_mksnode()` in `parse.c` and in `op_ind()` and `apply_func()` in `evaluate.c`, each with the same too-narrow gate. E-408 recorded what happens when copies of one rule drift apart. It is now a single `e448_literal_index()`, declared in `evaluate.h` so `parse.c` shares it. The three places that turn a double index into the `[%d]` text -- the helper, `op_ind()`'s ambiguity warning and `apply_func()`'s reconstruction for the `v()` form -- round through E-274's `idx_floor()` rather than a bare `(int)` cast, which is undefined behaviour for `1e308`/`inf`/`NaN`. The helper and `apply_func()` build the SAME name, one to validate it and one to resolve it, so rounding them differently would mean the name checked is not the name looked up. Sixteen index expressions give identical values and exit codes before and after.

A deliberate change of meaning: where BOTH a vector `x` and a vector literally named `x[0]` exist, `x[0]` now means the literal one. That is the only case where a working expression changes meaning, and the reading it replaces is the silent-wrong-answer case above; indexing there needs a temporary and the warning says so. Ordinary indexing is untouched — nothing is named `myvec[3]` — and E-433's quoting still works.

One change made and withdrawn. A `checkvalid()` hook was added so `v(c)` would report once rather than once per sweep point, on the evidence that a 21-point sweep printed 22 errors where `-output v(nosuch)` printed one. That evidence was an artifact of the probe: the two paths emit different wordings ("no such vector" vs "is not available or has zero length") and the filter matched only the first. Counted properly both emit 23 lines — pre-existing `sweep` behaviour for any unresolved output. Reverted.

the guard that covered one spelling of a bad value (E-447)

Ten degenerate or invalid inputs accepted in silence. What makes them one enhancement is that in every case a working guard already sat a few lines away, covering a DIFFERENT spelling of the same mistake.

cktsopt.c — a negative **gmin** wrecked the operating point in silence. A diode's 0.63 V drop became -0.001 V at `gmin=-1` and -1e-9 at -1e6, rc=0 with nothing printed, and the error grows smoothly so there is no threshold at which it announces itself. Every sibling was already guarded -- reltol=0, abstol=-1, vntol=-1, chgtol=-1, trtol=-1, maxord=0/99 and temp=-300 all warn, most from E-426. `gmin` and `gshunt` were the two holes. Zero stays legal ("no gmin").

restemp.c — **scale=0** silently shorted a resistor. A resistance WRITTEN as 0 warned and was clamped, while the same zero reached through `scale=0` was silent AND unclamped (v(nb) read exactly 0.0), and `scale=-1` surfaced only as a downstream "singular matrix". `scale` multiplies the instance's own resistance, so a non-positive one is a dead short or an active element; it falls back to 1 with a warning now.

vsrcpar.c, **isrcpar.c** — **trrandom** with a TYPE outside 1..4 was a silently DEAD source. 0, 5, 9, -1 and 100 all gave rms 0.0 with rc=0; 1-4 give proper noise. A typo'd type number removed the stimulus from the circuit without a word.

diosetup.c — an invalid diode **level** was accepted. `level=2` -- a level the model knows about but does not implement -- was refused with a Fatal error, while `level=99` and `level=-1` were accepted silently and behaved as level 1. The check tested one specific value instead of a range.

cktsopt.c — **cshunt** set from **.control** was silently ignored. The option works by adding a capacitor to every voltage node at setup: `eval_opt()` scans the deck's own `.option` cards and publishes `cshunt_value` for `INPpas4()`. The `.control` path stores a value nothing reads. The card form is BIT-IDENTICAL to writing the capacitor by hand (6.9209970943e-07), and this was the only card-only option of nineteen tested. Behaviour kept, silence removed -- and the notice fires only when no card supplied the value, so correct decks stay quiet.

vsrcask.c, **isrcask.c** — **show** claimed a source had EVERY waveform. All eight share one coefficient array and all eight queries answered from it, so a source declared only `sin(0 2 3k)` reported 0/2/3000 EIGHT times -- under sin and under each of pulse, exp, pwl, sffm, am, trnoise, trrandom. A `dc`-only source correctly showed – for all eight, so the code separated "none" from "some" but not WHICH. Inactive waveforms now report as EMPTY rather than as an error, so `show` renders a bare – instead of a column of `<<NAN, error>>`.

spiceif.c — three real **.options** keywords called unknown. `savecurrents`, `seed` and `numdgt`; savecurrents demonstrably works in the very run that calls it unknown (without it `@r1[i]` is a scalar, with it a transient waveform). E-446 downgraded this class because 176 of 179 `cp_getvar` names are shell variables -- that reasoning does not cover these three.

commands.c, **isrc.c**/**isrcpar.c** — two diagnostics that named the wrong thing. `snload`'s help said one file while the command required two, so the documented form answered "too few args". And `pwl( ... r=)` on a current source failed with the generic "unknown parameter (r)"; `r`/`td` are voltage-source-only (ISRC's pwl evaluator is a different, older implementation), so they are now declared purely to refuse them with the reason. Scope: a precise diagnostic, NOT feature parity -- porting repeat/delay means replacing ISRC's pwl evaluator and belongs in its own enhancement.

THREE reported defects were NOT. (a) **m=0** — E-426 examined exactly this and left it silent on purpose: it is the "disable this instance" idiom, its comment says so AT THE SITE and its suite asserts the silence. (b) **@r[resistance]** reporting the nominal value — E-426 settled that convention too and documents `1/@r1[conductance]` as the effective route (1200 ohm at 227 C in its own suite); `@c` and `@l` DO fold temperature in, so the three devices disagree, but that is a settled convention rather than a defect to flip

inside a set of guard fixes. Both were caught by E-426's suite failing in regression -- the mechanism working. (c) A misspelled `option method=bogus` looked silently ignored; ngspice prints "setAnalysisParm(options) ci_curOpt: unsupported integration method" at the end of the run, and the pre-E-447 binary prints it too. My probe's message filter missed it. Unchanged.

what the netlist wrote, quietly discarded (E-446)

Six places where something written explicitly in the deck was thrown away or reinterpreted with nothing printed. Three share one root cause.

ZERO USED AS THE "ARGUMENT NOT SUPPLIED" SENTINEL. `vsrcload.c` and `isrcload.c` decided whether an optional waveform argument had been given by testing `coeffs[n] != 0.0`, so a legitimately written 0 was indistinguishable from an omitted argument.

`vsrcload.c`, **`isrcload.c`** — an explicit **`TD1=0`** on an EXP source became the timestep. `V1 a 0 exp(0 1 0 1m 5m 1m)` starts one step late; every printed row carried the exact analytic value of the PREVIOUS timepoint to ten decimals. Because the substituted value IS the timestep, the answer depended on the timestep -- err -3.7e-03 at a 10 us step, -4.0e-02 (~4 %) at 100 us -- and writing TD1 as `1e-30` instead of `0` made it exact. Both source types affected; SIN/PULSE/PWL/SFFM were exact throughout, which pinned it to EXP. Delays are now selected on whether the argument was SUPPLIED; the time constants keep a zero guard because they are divisors; omitted-argument defaults are untouched.

`isrcload.c`, **`vsrcload.c`** — PULSE **`PW=0`** and **`PER=0`** meant different things to a V and an I source. With PW=0 the voltage source emitted no pulse and the current source one that never ended; with PER=0 they disagreed the other way. A zero pulse WIDTH is now the degenerate pulse it asks for and a zero PERIOD means "do not repeat", on both. A zero rise/fall time still falls back to the timestep (documented SPICE, and what vsrc already did).

`vsrcpar.c`, **`isrcpar.c`** — a PWL list with an odd token count. One point is left without a value, and a value was invented -- differently by each source: `pwl(0 0 1m 1 2m)` on a V source was BYTE-IDENTICAL to `pwl(0 0 1m 1 2m 0)`, ramping the stimulus to zero at a time never assigned one, while the I source ate the stray token AS a value and held it. Two different guesses at one ambiguous input, so it is refused. The neighbouring checks already warned about non-increasing and duplicate times; this was the gap between them.

`inp2dot.c` — a third **`.dc`** sweep source was neither run nor refused. `.dc V1 0 2 1 V2 0 2 1 V3 0 2 1` produced 9 rows, not 27, with V3 pinned at its DC value. Proven by making V3 dominant (R3 = 1 ohm against R1=R2=1k) and watching every value stay in the 1e-3 range. Fixed with it: surplus `.ac` arguments were dropped in silence -- any number, numeric or not, gave byte-identical output -- while `.tran` and `.dc` both refuse what they cannot use.

`ptfuncs.c` — the sign of a negative base. The default evaluated `pow(fabs(x), y)`: `(-2)**3` came out +8 and `(-2)**1` came out +2, so odd exponents were wrong and even ones coincided. Everything else disagreed -- `pwr(-2,3)` is -8, a Verilog-A model's own `pow(-2,3)` is -8, and both the LTspice and HSPICE branches preserve the sign; only PSPICE mode agreed, where `|x|^y` is that dialect's documented PWR convention. E-399's rule applies: an expression must not mean different things depending on whether the netlist or the model computed it. A negative base with an INTEGER exponent now carries its sign; with a non-integer exponent there is no real result and the historical magnitude is kept rather than a NaN, which would poison the Jacobian (E-256/E-440's reasoning). Positive bases untouched.

`capask.c` — **`@c[capacitance]`** folded in **`m=`** while `@r[resistance]` reported the written value, so one accessor idiom meant two things by device type. The capacitor now reports what its instance was given; `capload.c` applies `m` to the matrix stamps itself, so the simulation does not move.

One reported defect was NOT one. An AM source with a 5th argument emits a constant -- correct, because ngspice's AM is `AM(VO VMO VMA FM FC TD PHASEM PHASEC)` and the 5th argument is the

CARRIER FREQUENCY, so `am(1 0 100 1k 0)` sets FC=0 and leaves the offset. Against the right argument order AM matches analytic to 1e-12. Unchanged.

nine silent failures made loud (E-445)

Unrelated in mechanism, identical in shape: the simulator completed, returned 0 and printed nothing. Each fix is paired below with the sibling that was always handled correctly — that pairing is the argument, because it shows the guard already existed nearby and one path was missing it.

`fourier.c` — a bare **`.four`** card segfaulted. `dotcards.c` warns "no nodes given" and declines to save anything, but does not DROP the card, so the empty command still reached `fourier()`, which read the fundamental frequency straight off a NULL wordlist. `.four` was the only one of 25 zero-argument dotcards that died; `.print` and `.plot` are handled a few lines above it, print the identical message, and were always safe. Needed transient results to exist, and survived subcircuit flattening.

`fourier.c` — an overflowing fundamental was accepted. `.four 1e400` overflows to `+INF` and produced a full report: THD 201.971 % with every harmonic frequency printed `inf`, against 1.91095e-11 % for the valid case. The surrounding validation was already thorough — `0`, `-1000`, `abc`, a literal `inf`, a literal `nan`, and even `1e-400` (which UNDERFLOWS to 0.0 and is caught by the `<= 0.0` test) were all refused. Overflow was the one hole.

`inp2r.c`, **`inp2c.c`**, **`inp2l.c`** — a comma silently changed a device value. `R1 in nb 1,5k` built a 5 kOhm resistor: `,` is a token separator, so the line parsed as the value 1 followed by a stray unlabeled `5k`, and an unlabeled trailing number OVERWROTE the value already read. That overwrite is how `R1 a b rmod 2k` legitimately works — but there the value position held a NAME, so nothing had been read. Decisive evidence: every other separator obeys ngspice's documented "ignore trailing text" rule (`1;5`, `1:5`, `1_000` → 1; `1.2.3` → 1.2), and only the comma reached the overwrite.

`inpcom.c` — an over-wide array instance collapsed to ONE device. Past E-441's 8192 cap the range failed to parse and the line was emitted unchanged, as a single device named `r[0:8192]`: `R[0:8191]` → 8192 devices (correct), `R[0:8192]` → 1 device (9x wrong), `R[0:19999]` → 21x wrong, all silent. The identical range in a NODE field already produced E-443's warning; only the instance-NAME position was quiet, because E-441 made `inp_expand_buses` skip an element line's first token so the malformed check never saw it. A lone `R[2]` is still an ordinary instance name — a scalar bit is not a list.

`inp2n.c` — **`.option autobus`** indexed tokens that cannot carry an index. `N1 0 b bd` produced five ordinary FLOATING nodes `0[0]`..`0[4]` instead of ground, and `N1 a[0] b bd` produced `a[0][0]`. With `b` driven at 1 V the correct current is -1.9375e-3; the shorthand gave 5.4e-20 — the device contributing nothing, silently. The same line with the option OFF is reported by E-402's under-connected warning, so turning the feature on removed a diagnostic the feature-off path already had. Only BUS ports are checked: a scalar port is never indexed, so `0` or `a[0]` there stays legal.

`com_sweep.c` — a failed sweep point published the PREVIOUS solution. E-438 counted these and said so, but the recorded VALUE was whatever the read-back returned — and a failed run leaves the earlier plot in place. Three different priors gave three different "results" (0.5 / 0.8 / 0.2), so this was the old solution being republished, not a constant. On a plot it is a flat shelf that looks physical, and `wrdata` wrote the rows unmarked. They are now `NaN`, which keeps the array shape — every output stays aligned against the sweep scale, exactly why E-438 declined to drop the points — while plotting as a gap. `montecarlo` was the guarded sibling all along: same cause, same binary, already excluding failed samples.

`subckt.c` — a legal 20-deep hierarchy was called infinite recursion. `MAXNEST` bounded nesting at 21 and reported *infinite subckt recursion* on exhaustion, which a strictly non-recursive chain does not have; confirmed a DEPTH cap, not a name-length one (same boundary for short and long names). Now 256: a finite hierarchy costs exactly its own depth in passes, so the headroom is only paid by a pathological deck, and runaway recursion is still bounded by the existing million-instantiation cap. The message no

longer asserts a cause it cannot distinguish.

cktsopt.c — an iteration limit below the solver floor was discarded in silence. `NIiter` raises any limit below 100 to 100. E-426 found that floor, judged it deliberate and upstream, and left it alone; that judgement stands. What was missing is that nobody was told: the value is stored, `option` echoes it back, and a circuit needing 13 Newton iterations still took 13 under `itl1=3` with both fallbacks disabled. The floor stays; setting a limit below it now says so, and only for a limit given explicitly — the shipped `itl2=50` and `itl4=10` are themselves below it.

spiceif.c — four working **.four** options were reported unknown. `.option fourgridsize=10` moves the reported grid from 200 to 10 AND printed "unknown option". A warning that fires on a setting the run honours teaches the reader to ignore E-438's check. `nfreqs`, `nperiods`, `polydegree` and `fourgridsize` are registered. Scope is deliberately those four: of the 179 `cp_getvar` names in the sources 176 are flagged, but most are shell variables whose documented home is `set` in `.control`.

One reported defect that was not one. An OSDI integer parameter declared `from [1:inf)` takes `n=0.5` as 1 while refusing `n=0.4`, which looks like a range check running after rounding. E-399 established that a netlist value assigned to an integer parameter must round half away from zero, to match what Verilog-A does for the same conversion inside a model. `0.5` becomes `1`, and `1` is inside the range. Rejecting it would contradict E-399 and refuse LRM-conformant decks — unchanged.

inp2n.c, **spiceif.c** — **.option autobus** (E-444)

A Verilog-A `inout [0:4] a` compiles to five OSDI terminals named `a[0]`..`a[4]`, so a netlist had to spell all five out. With the opt-in option, one token per PORT is enough: `N1 a b busdev` expands to `N1 a[0] a[1] a[2] a[3] a[4] b busdev`, and `a[2]` elsewhere binds to the same node.

The information was already in hand. `INP2N` resolves the model BEFORE it binds nodes, and `dev->termNames[]` holds exactly those names — E-402 was already reading that table to REPORT which terminals a short line left unconnected. Terminals are grouped into ports (consecutive `base[i]` sharing a base = one bus port, a bracket-free name = one scalar port) and a line supplying one token per port is expanded. The bit indices are copied from the model's own terminal name, so a port declared `[4:1]` expands to `a[1]`..`a[4]` rather than an assumed `0..n-1`.

Opt-in because a short instance line already means something: it leaves trailing terminals unconnected, the `\$port_connected` idiom BSIMSOI, BSIM-CMG/IMG/BULK, BSIM6 and PSP-HV rely on. Without the option nothing changes. Even with it the readings barely overlap — expansion needs the token count to equal the PORT count, and an all-scalar model has port count == terminal count, so a short line there can never be mistaken for the shorthand. The option name is registered with E-438's `.options` checker so it is not reported as unknown.

Expansion needs the model, so it happens at device-parse time, not netlist-read time; the consequence is that it does not show up in `listing e` the way E-441's array instances do.

inpcom.c — bracketed index lists (E-443)

`base[lo:hi]` (E-221 for nodes, E-441 for instance names) is joined by an explicit list and by the two written together: `a[1,3,5]`, `R[1,3,5]`, `a[0:1,7]`. Only the SOURCE of the indices is new, so every reading those enhancements established carries over — a list in a node field binds consecutive terminals, a list on an instance name makes one card per element, and a node list on an array instance is taken in step with it. Indices are used in WRITTEN order, neither sorted nor deduplicated, because the order is what binds nodes to terminals; a descending list therefore has no direction to get wrong, which is why E-411's reversed-binding warning still fires for `a[1:0]` and stays quiet for `a[3,1]`.

A lone **a[2]** is deliberately NOT a list. It is a scalar bus bit and stays a node name — E-221's contract is that `a[0:1]` and an explicit `a[0]` denote the same node. That single condition is what makes the change additive: the only spellings that now mean something are ones that previously meant nothing.

The parse is split into `inp_bus_indices()` (the bracket contents) and `inp_bus_index_parse()` (the base around them), and four consumers share it: `inp_expand_bus_token`, `inp_expand_array_instances`, `inp_bus_rewrite_wrapped` (so `.save v(a[1,3,5])` expands, its old range-specific parse REMOVED rather than left beside the shared one) and the malformed-list diagnostic.

That diagnostic exists because a malformed list is not harmless: `R2 a[1,] 0 1k` re-tokenises the line and ngspice built a resistor with NO VALUE from it, warning only "Value of resistor r2 is too small, set to 1e-12" — a silent miswire in the new syntax. A bracket group that looks like an index list and is not one now says so, on a test tight enough that `mem[addr]` and `a[2]` stay silent.

`inp.c`, **`commands.c`** — **`listing tree`** (E-442)

A new `listing` form that draws the subcircuit hierarchy rather than the flat device list. `listing e` answers "what is actually simulated" — one line per device, structure only implicit in the dotted names — which is the wrong shape for "what does this design contain". Each `X` is labelled with the subcircuit it instantiates and drawn where it is USED, so the same subcircuit appears once per instantiation, and a summary reports instances, devices and depth. `listing t` is accepted alongside the existing `l`/`p`/`d`/`e`/`r`.

The walk is over `ci_origdeck`, the deck as read with `.subckt` blocks intact, because the expanded deck no longer records which subcircuit an instance came from. A useful consequence: array instances (E-441) are already expanded there, so `X[0:2]` shows as three instances.

Two details the walk has to get right. The subcircuit name is NOT "the last token before the parameters": numparam rewrites `X1 in 0 sub PARAMS: rv=2k` into `x1 in 0 sub 2k` before this runs, so the marker is gone and counting tokens picks `2k`. Resolving against the collected definitions — the last token that names a `.subckt` — is unambiguous where counting is not. And a `.control` block's commands begin with a letter exactly like element lines, so without skipping those blocks `option` and `listing` were drawn and counted as devices, and the last-child detection broke, leaving the final top-level entry with a tee where it needed a corner.

No "defined but never instantiated" report: ngspice comments such a definition out of the deck (`*subckt spare p n`) long before this runs, so it could never fire. `(undefined)` and `(recursive)` are defensive labels — both conditions abort the deck during expansion, so this command is not reached with either.

`inpcom.c`, **`parse.c`**, **`vectors.c`**, **`device.c`** — array instances (E-441)

A range on an instance NAME now means an array of devices: `R[0:3] a b r=1k` is four resistors in parallel, `N[0:3] a[0:3] b model` gives each element its own bit, and `N[0:3] a[0:3] a[1:4] model` builds a chain.

The design turns on one fact: the instance name selects the reading. Enhancement-221 already gave a range in a NODE field a different and equally useful meaning — one device with a wide port (`X1 bus[0:3] sub`) — and decks depend on it, so the two cannot both apply to a line. A scalar name keeps E-221's in-place expansion; a range on the name switches to one card per element with node ranges indexed *in step*. Nothing that parses today changes meaning, since an instance name carrying `[lo:hi]` was not a usable device name before.

`inp_expand_array_instances()` runs immediately before `inp_expand_buses()` and consumes the ranges belonging to the array reading, so whatever survives is E-221's by definition. Both passes read a range through one shared parser, `inp_bus_range_parse()`, so they cannot disagree about what a range is — and an XSPICE port group `[d1 d2]` has no base, is refused by that parser, and is therefore invisible to both.

Refusals: a width mismatch (`N[0:3] a[0:1] b`) is named and the deck is rejected — an earlier revision printed the error and let the line through, whereupon ngspice built one resistor literally named `r[0:3]` and completed the run with a circuit nobody described; and an XSPICE A-device array is refused,

because an A-device already uses brackets for port groups so its name is ambiguous by construction. Separately, E-221 no longer bus-expands the FIRST token of an element line: it is a name, never a node list, and expanding it buried the real complaint under a message about a device called `r[3]`.

Addressing the elements needed the accessor to cope with a bracketed NAME. The lexer in `parse.c` ended the token at the first `]`, and the splits in `vectors.c` and `device.c` took everything before the first `[` as the device name, so `@r[2][resistance]` looked for a device `r` with a parameter `2` — the element was visible to `show` but not addressable. The split turned out to live in FIVE places: `dctrcurv.c` (`.dc` failed fatally, card and command alike) and `outitf.c`'s `parseSpecial` (a `save` that matches nothing loses the WHOLE plot, so `save @r[1][i]` ended the run with "no data saved ... analysis not run") were both found by verifying the feature across subcircuit hierarchy and the `.control` surface after the first round. `ft_accessor_param_start()` is the shared rule and is deliberately narrow: a bracket group holding nothing but an integer AND immediately followed by another `[` belongs to the name, which leaves `@nd1[i_a[0]]` (E-408) and `@*[[param]]` (E-269) alone and keeps a lone `@r[2]` meaning device `r`, parameter 2.

Also fixed here: `inp_rem_levels()` reads `p->line->level`, i.e. into the cards, so the scope tree must be freed before the deck. Of `inp_readall`'s rejection sites the `inp_poly_2g6_compat` one had that order and the `inp_vdmos_model` one had it reversed; the new path was written by copying the latter and crashed with `EXC_BAD_ACCESS` at offset 8 (the `level` member) the first time it was taken. Both are now in the correct order — the vdmos site had been a use-after-free waiting for its first caller.

`inpcom.c` — a descending bus range binds terminals in reverse (E-411)

`nd1 d[3:0] 0 mm` against a model declaring `inout [3:0] a` compiles clean, simulates clean, and connects the terminals backwards. Both sides say `[3:0]`, which is exactly why it reads as correct.

Two conventions disagree. The COMPILER declares a bus port's terminals in ASCENDING bit order whatever direction the declaration uses -- `hir_def`'s item_tree lowering sorts the endpoints and loops upward, *"declare from lsb to msb (ascending), matching natural bit order; direction of the original [msb:lsb] only affects range checks"* -- so `inout [3:0] a` and `inout [0:3] a` give the same positional terminal list. The NETLIST does honour direction: E-221 expands `d[3:0]` to `d[3] d[2] d[1] d[0]`. Since the instance line's Nth node binds to the model's Nth terminal, a descending node list reverses the mapping.

Measured with one conductance per bit (a[k] -> (k+1) mS, so the current at a node identifies the bit it landed on): wired `d[0:3]` the nodes read 1, 2, 3, 4 mS; wired `d[3:0]` they read 4, 3, 2, 1 -- and the model's own declaration direction changes nothing, the `[3:0]` and `[0:3]` rows being identical.

`inp_expand_bus_token()` now reports whether the range ran msb-first, and `inp_expand_buses()` warns on element instance lines only. The scope came from scanning what the shipped suite actually does with descending buses rather than from taste: a `.subckt` descending PORT list is a deliberate interface choice that E-221 documents and `busnodes_examples` tests; output and IC cards bind nothing; and a range too wide to expand never binds at all (E-338's guard), so warning there would be false. The opt-out is `set nobusdirwarn` in `.spiceinit` -- a `set` inside `.control` is too late, because bus ranges expand while the netlist is read, and the message says so.

`osdi/osdiload.c` — an operating point that changed with frequency (E-412)

An OSDI operating-point variable read after `.ac` returned the SMALL-SIGNAL solution rather than the bias, and returned a different value at every frequency: an opvar defined as nothing but `vbias = V(a, c)` gave 0.4939733 after `op` (the true bias, from solving the circuit's KCL cubic by hand) but 0.4823410 / 0.4819028 / 0.0473587 after `.ac` at 1 k / 10 k / 1 meg.

An operating point cannot depend on frequency, which is what separates this from a tolerance argument -- the value is identical at `reltol` 1e-3, 1e-6 and 1e-10, and a multi-point sweep leaves the LAST point's.

CAUSE: E-53 fires `@(final_step)` by issuing one dedicated evaluation per instance when an analysis completes, at `CKTrhsOld` -- which after a frequency sweep holds the small-signal solution. The model is genuinely re-evaluated, not mis-read: opvars chosen to tell those apart confirmed it (`3V+1` and `V^2` both matched the ac solution to nine digits, while a constant opvar stayed 42). Built-ins are unaffected, having no such post-analysis evaluation.

NOT WRONG, and established before fixing anything: the analyses themselves. A bias-dependent `white_noise(kf*V^2)` source integrated to `onoise_spectrum = 1.342573180366e-01`, matching the dc-bias prediction exactly and not the ac-bias one (7.21e-02). This is a READBACK defect; it is also not sticky, any following `op`/`dc`/`tran` restoring the correct value.

The evaluation cannot simply be skipped -- it is the only thing that fires `@(final_step)`, and `@(final_step("ac"))` and the noise variant are supported and tested by `finalstep_examples`. So `OSDIfinalStep` snapshots the instance data around the call and restores it, for `MODEAC` and `MODEACNOISE` only: the event bodies still run and their `$strobe` output stands, while what the evaluation wrote is discarded -- correct, since those results are by design never loaded into the matrix or RHS. DC, DC-sweep and transient are deliberately not snapshotted, their final solution being a real operating point.

WHY IT SURVIVED: with a purely resistive device the dc and ac solutions coincide, so the natural small test case cannot see it at all.

`breakp2.c` + `osdi/osdiparam.c` — savecurrents for a multi-terminal OSDI device (E-413)

`.options savecurrents` on a four-terminal compact model produced `@nd1[i] : current, real, 0 long` -- a vector that exists and holds nothing, with no warning -- while a built-in BJT in the same deck produced four filled waveforms (`@q1[ic]`, `@q1[ie]`, `@q1[ib]`, `@q1[is]`).

CAUSE: `inp_savecurrents()` is a TEXTUAL pre-pass over the deck. It runs long before any `.osdi` is loaded, so it cannot know a compact model's terminal names and emits the same bare `@dev[i]` that R/C/L use -- its own comment (E-394) says exactly that. E-394 defines the bare `i` alias only for TWO-terminal devices, where it is unambiguous; for anything wider the save named a parameter that does not exist, so the vector was registered and never written.

The values were never wrong, only uncapturable: read as scalars after `op` the terminal currents were exact (1.000e-3 / 4.000e-3 / 1.200e-2 / -1.700e-2, summing to zero by KCL) and an explicit `.save @nd1[i_d]` always worked. What was missing was the waveform, which is what savecurrents exists to provide.

Since the names cannot be known during a textual pre-pass, the expansion moves to where they can: `ft_getSaves()` runs at analysis start with the circuit set up, so a bare `@dev[i]` belonging to an OSDI instance that does not define it is expanded there into one entry per terminal, via a new `OSDIterminalNames()`. The two-terminal case is deliberately untouched -- `@dev[i]` is real there and is byte-identical to the pre-413 binary -- and built-ins never enter the path.

---

5. Parser and device registry — **`src/spicelib/`**

`parser/inpgtok.c` (10)

A **`.model`** card override silently dropped its first parameter when the type name and the opening parenthesis had no space between them (`modname devtype(p1=v1 …)`): `INPgetNetTok`'s scan did not treat `(` as a token boundary, so the type token swallowed the paren and the first parameter name. Found while verifying event-function counters whose model-card overrides mysteriously kept defaults (E-8).

`parser/inpgmod.c` (5)

`find_model_parameter()` dereferenced `*(device->numModelParms)`, which is NULL for built-ins that take no model cards (VCVS, CCCS, ...) — any *referenced* `.model m vcvs()` card segfaulted, with no OSDI involved at all (an ordinary MOS instance sufficed). This was the true root of the long-documented "module named like a built-in crashes" gotcha. One NULL guard; both shapes now produce clean, located errors (E-76).

`parser/numparse.c` — out-of-range int test

`ft_numparse` decides whether a parsed number is integer-valued with the round-trip `(double)(int)val == val`. Casting a `double` that lies outside `int` range to `int` is undefined behavior, reached by any control-language literal `>= 2^31` (e.g. `let x = 3e9`) — a UBSan build flags it at `numparse.c:166`. The value is now range-checked (`val >= -2147483648.0 && val < 2147483648.0`) before the cast; a value outside `int` range is, by definition, not int-representable, so the answer is unchanged for every in-range input (boundary-tested at `INT_MIN`, `INT_MAX`, and `±(2^31)`). Found by the 2026-07 ASan/UBSan pass.

`snp2va.c` — **`lstsq_real`** heap overflow on a tiny Touchstone file

`lstsq_real` (the Householder-QR least-squares primitive behind `pre_snp`'s vector fit) documented itself as requiring an overdetermined system (`m>=n`), but its back-substitution read `b[i]` and `A[i*n+i]` for every unknown `i` up to `n-1`, past the `m`-row buffers when `m<n`. `pre_snp` on a Touchstone file with very few frequency points ($\leq$3) drives the fit underdetermined — the stacked system has fewer rows than poles, and the buffers shrink to a `malloc(0)` — so the back-substitution read off the end: a heap-buffer-overflow (ASan: `READ of size 8 … 7 bytes after a 1-byte region`), reachable from an ordinary coarse `.sNp` measurement. The back-substitution now leaves any unknown with no constraining row (`i>=m`) at zero instead of reading past the matrix; for the normal `m>=n` path the guard never fires and the fit is bit-identical (verified: a 21-point file fits to 2 poles, rms 1.77e-05, before and after). Found by the 2026-07 ASan/UBSan pass (heavy-deck sweep, via a degenerate `pre_snp` input).

`snp2va.c` — unbounded filename-derived port count → heap corruption (E-227)

`parse_touchstone` infers the port count `N` from the file extension `.sNp` (`N = atoi(dot+2)`) with no upper bound. A filename such as `.s2147483647p` (`N = INT_MAX`) slips past the parse — the record stride `rec = 1 + 2*N*N` and the per-record inner-loop bound `N*N` share the same wrapped `int`, so `N*N` overflows to a small/zero value and the token layout stays self-consistent — but `N` is stored in `out->N` and then used to size the downstream vector-fit allocations (`malloc(nf*N*N*sizeof(cplx))`, `malloc(N*N*sizeof(cplx))`). The overflowing allocation and the writes that follow corrupt the heap (the program typically aborts later on an unrelated `malloc`, so it presents as flaky). The brute-force fallback already caps port inference at 512; the filename path now honors the same bound — `if (N > 512) N = 0;` falls back to inferring `N` from the data. Valid files (`.s2p`, ...) are far below the cap and unaffected. Found by the 2026-07 Touchstone-parser fuzzing pass (`rdsnp` was clean over 4,500 iters; `pre_snp` hit this on 103/3,600). Verify: `examples/snpfuzz_examples/verify_snpfuzz.py`.

`parser/inp2n.c` — an OSDI instance line with too FEW nodes (E-402)

The node count was bounded on one side only: `numnodes > *dev->terms` errors with *"too many nodes connected to instance"*, while too FEW was never asked about. The binding loop then wrote `GENnode(fast)[i] = -1` for each unsupplied terminal and said nothing, so a typo silently produced a different circuit — a 4-terminal device given 3 nodes still simulates, with the omitted pin dangling and every branch touching it dead (`v(g)` 1.333 fully connected vs 2.000 mistyped, rc=0, empty diagnostic stream). Every built-in rejects the same mistake.

Not fixed by rejecting it. That −1 is the SAME sentinel `osdisetup.c` reads as "this terminal is deliberately unconnected" (`terminals[i] == -1` ? `connected_terminals = i`) — the LRM optional-terminal feature that 32 corpus models query through `\$port_connected` (BSIMSOI, BSIM-CMG/IMG/BULK, BSIM6, PSP-HV). A dangling node is not a substitute either: electrically it matches an omitted pin, but it reports `\$port_connected == 1` where the omission reports 0.

So the lower bound WARNS, naming each absent terminal from `dev->termNames`, and states the consequence — including that an omitted terminal is not grounded, since that is the natural assumption and it is wrong (grounding the pin moves the measured node by 67%). Semantics unchanged; regression 322/322 with the warning firing zero times across the suite.

`parser/inp2n.c`, `frontend/spiceif.c` — `.option silentports=ground` (E-482)

E-481 turned the warning above off. What it could not do was make the deck run, and this is the other half.

Ten of the twelve corpus models that declare an optional pin tie it off with a POTENTIAL contribution (`Temp(t) <+ 0.0`), which puts nothing into a node ngspice allocated itself, so the private node `<inst>#<term>` has nothing driving it and the operating point dies on `singular matrix`. E-402's answer was *write `0` for the pin*, and a schematic front end that hides the pin cannot write anything — on the E-402 reproducer, silencing removed five warning lines and left all six singular-matrix reports, quiet and still broken.

`silentports_mode()` replaces E-481's `silentports_enabled()` and returns three states. It is read ONCE per instance line into `sp_mode`, which decides both the message and the binding — reading the variable twice would let a `.control` block that changes it mid-parse warn about a terminal it then grounds, or the reverse. `SP_OFF` warns and leaves the terminal dangling (unset — E-402, unchanged); `SP_QUIET` — the bare card, `=dangle`, `=quiet`, and the on-words `1/true/yes/on` — silences the report and leaves it dangling, which is E-481's behaviour preserved exactly; `SP_GROUND` — `=ground` alone — binds it. In `SP_GROUND` the binding loop no longer writes `GENnode(fast)[i] = -1` for an unsupplied terminal; it hands `INPtermInsert` a `copy("0")` and calls `bindNode`, exactly as for a token the line supplied. `INPpas2` inserts ground into the terminal symbol table with a hard-coded `char *groundname = "0"` before walking any device card, so `"0"` always resolves to the existing ground node and never creates one — checked with the OSDI card placed FIRST in the deck, and from inside a subcircuit, where it binds the GLOBAL ground.

Ground is asked for by name because it makes the model read `$port_connected() == 1` and build the branches it would otherwise skip: a different circuit from the one the netlist describes. A word the user typed is the right gate for that; the bare card is not, which is why the bare card keeps E-481's meaning rather than acquiring this one.

The binding has to happen here and not in **`osdisetup.c`**. That file reads the −1 sentinel to decide how many terminals were connected and passes the count to `setup_instance` as `connected_terminals`, which is what the model reads back through `$port_connected`. Bind in `INP2N` and everything follows on its own: the count is right, `$port_connected` reports 1, no private node is created, and no collapse machinery is involved. Grounding later would have to undo a decision instead of never making it.

The query order is load-bearing. `silentports_mode()` asks `cp_getvar` for the CP_STRING FIRST, then CP_REAL for `=1/=0`, and CP_BOOL LAST — where it now means only what it is being asked, the bare card with no value. E-467 gave `cp_getvar` a CP_BOOL COERCION that answers TRUE for any value which is not an off-word: exactly right for a two-state option, which is what E-481 relied on, and fatal for a three-state one, because it swallows every value word and reports them all as a plain "on". Measured with the BOOL query still first: `silentports=quiet` GROUNDED the terminal, and so did `silentports=bananna`. An unrecognised word is named once per distinct spelling and falls back to the DEFAULT rather than to either ON state — a typo must not be what silently drops a diagnostic or changes a circuit — following ngspice's own handling of a bad enumerated value (`.options`

`method=banana` reports *unsupported integration method* and continues).

`spiceif.c` keeps the name registered in BOTH `if_is_option()`'s list and the type dispatch beside it; only the comment changes, since the dispatch answers what TYPE the name publishes and the VALUE is decided in `INP2N`.

`analysis/dcpss.c`, **`frontend/com_qpss.c`** — harmonic-balance convergence (E-483)

`QPSShb` was called with its convergence bound and iteration cap compiled in — `QPSShb(ckt, f1, f2, K1, K2, 0, 0, 60, 1e-10, ...)`. `tol` is an ABSOLUTE bound on the residual norm, so what is reachable depends on the circuit: the diode two-tone deck in `distoexact_examples` settles at `2.3e-15`, while an FET amplifier carrying tens of milliamps floors around `1e-8` and cannot satisfy `1e-10` however many iterations it is given.

The way it failed was the defect. The Newton iteration reached a residual of `9.482e-09` at iteration 4 — nine orders down, quadratic — then sat on that number flat to seven digits for the remaining 55. Having "failed" the level, the E-135 source-stepping ladder halved its step and walked back to `lambda = 0`, 1022 Newton iterations in all, and returned a bare `E_ITERLIM`. A good answer found in five iterations was discarded after minutes of work — six to seven at `hb 4 4`, which is indistinguishable from a hang.

`com_qpss.c` gains `qpss_knob()`, which reads `qpss_tol` and `qpss_maxiter` the way `qpss_verbose` beside it is read. Both published types are asked, because E-454's lesson is that the SPELLING decides the type (`=1e-8` a CP_REAL, `= 1e-8` a CP_STRING), and the parse is `strtod` rather than `atoi` so `qpss_maxiter=2e2` means 200 and not 2 — E-478's trap. A present-but-unusable value is reported and the default kept.

`dcpss.c` gains the stall test inside `QPSShb`'s Newton loop: `QP_STALL_RUN` (4) consecutive iterates improving the residual by less than `QP_STALL_RATIO` (0.1%) is a stall, and a stalled residual is accepted as the answer only if it sits `QP_STALL_ACCEPT` (1e6) below the residual the level opened with. A stall at a residual that never came down is still a failure and falls through to the ladder — it reaches that verdict in a few iterations instead of `maxiter`. `stall_accept` is cleared whenever the ordinary `tol` test carries a level, so the closing message reports which test actually did the work.

Acceptance does not change the answer, which is the load-bearing claim: against the same circuit converging normally under a loosened bound, the fundamentals are BIT-IDENTICAL and the third-order products agree to 1.1e-05 relative, about 0.0001 dB on OIP3.

`analysis/dcpss.c`, **`frontend/com_qpss.c`** — believing an accepted residual (E-484)

E-483 made the bound reachable and taught the Newton loop to recognise a stall. It left a hole: the STALL path reported what it had settled for, the ORDINARY `tol` path reported nothing. A deck that loosened the bound far enough had its solution accepted with a clean `converged in 3 iterations`.

On a two-tone FET amplifier, `set qpss_tol=1e-1` at K=4 accepts a residual that came down by only ~100x and the third-order products land 6 dB out — 27.818 dBm against the 33.976 dBm the trustworthy K=2 run gives. That is worse than the failure E-483 fixed: a refusal is honest, a wrong number is not, and E-483 had handed users a knob whose obvious misuse was silent corruption.

The reduction is now reported on the ordinary path too, naming the residual, the reduction, the bound that permitted the early stop and both remedies. The run is not aborted and the spectrum still prints, because the user may know exactly what they are doing.

`QP_LOWRED_WARN` is deliberately a DIFFERENT constant from `QP_STALL_ACCEPT`: one decides whether to accept a stalled residual at all, the other whether to believe an accepted one, and conflating them would tie a diagnostic to a control decision. Its value is calibrated on measurement — 101x puts OIP3 6.16 dB out and is warned, 55345x agrees with the K=2 answer to 0.033 dB and is not — with suite

checks holding the bar from both sides, since a warning that fires on a good answer is one people learn to ignore (E-445's note).

It does not fix **`hb 4 4`**. The Newton step degrades as harmonics are added (5.3e8x reduction at K=2, 7.8e4x at K=3, 1.5e2x at K=4), and drive amplitude, `beta`, `is`, `alpha`, `cgs`/`cgd`, the passive network, the topology and the DFT oversampling were each ruled out by measurement; `pss_csolve` pivots and `tmalloc` is `calloc`. `qp_build_matrix`'s harmonic-domain Jacobian remains the open question.

`com_qpss.c` also takes a `const char *->char *` qualifier fix: E-483's `qpss_knob()` declared its name parameter const while `cp_getvar` takes a non-const pointer, two discards-qualifiers warnings in a tree kept at zero.

`xspice/cm/cmutil.c`, five **`icm/analog/`** models, **`inp2dot.c`**, **`inpcom.c`**, **`inp.c`**, **`com_measure2.c`** — guards that used the bad value (E-485)

Eight sites of one shape: the code establishes that its input is unusable, says so, and then answers from it.

`cm_climit_fcn` had its bail-out COMMENTED OUT. The five lines restoring a safe output sat inside `/* ... */`, so the helper detected the crossed linear range, printed, and fell through. They could not be uncommented: they assign the LOCALS and `return`, while the out-parameters are written at the end of the function, so an early return would have left `*out_final` uninitialised — very likely why they were disabled rather than repaired. The repair fixes the INPUT: `limit_range` is a smoothing half-width, so clamping it to half the limit span leaves the thresholds coincident (hard limiting at the declared bounds) and every downstream branch valid. `ilimit` goes from 24.48 to 0.437 against rails of ±1; the message from 26 copies per `op` to one (a report-once flag, since this is a raw `printf` with no instance context — E-480's INIT gate is not reachable here); and its text no longer names CLIMIT in a deck that has none.

`limit`, **`int`**, **`d_dt`** never computed **`linear_range`**. An oversized `limit_range` carried them to 24.5057, 95.04 and 24.25 against limits of ±1, and to 249999.75 at `limit_range=1e6` — unbounded, linear in the parameter, silent, and all three declare those limits as MANDATORY parameters. Same clamp, reported at INIT. E-468's comment at `limit/cfunc.mod:149` says it added its checks "as the CLIMIT sibling already does"; it ported two ideas and not CLIMIT's own guard.

`pwl`'s monotonicity guard ended in **`break`** — leaving only the checking loop, after which the table was built from the rejected data and `x=[0 2 1]` answered 5.5 for an input of 0.5. It cannot `return` in place: `x`/`y` are `STATIC_VAR`-owned, so freeing double-frees and not freeing leaves a half-built table. The test moves beside the `size_error` check — before allocation, reading the parameter directly, refusing every evaluation — with both messages now INIT-gated.

`hyst` and **`slew`** had no checks. An inverted `in_low`/`in_high` pair and a `hyst` wider than the span left the block dead at 0.0; a negative `rise_slope` drove the output to −2.0 and a zero slope disabled limiting. Reported once at INIT and repaired to the nearest sane reading. `slew`'s report is emitted from the INIT block, not beside the repair, which sits in the `MIF_TRAN` branch where INIT has passed — the shape E-480 had to move out of LIMIT.

`dot_sens` validated nothing while `dot_ac` in the same file rejects the same arguments by name; a reversed range swept a FABRICATED decade, 1e6 → 1e7 ascending. **`dot_disto`** refused them while reporting a DEVICE-parameter fault it does not have, identically for two faults. Both now share `inp_sweep_args_ok()`, which names the card and the argument.

`.include` and **`source`** accepted a DIRECTORY: `fopen()` on a directory succeeds on macOS, the BSDs and glibc, so the include contributed nothing and the deck solved a different circuit (v(out) 1.0 instead of 0.5) in silence. Both now test `S_ISREG`, the property their existing checks assume; `.lib` was already guarded.

com_measure2.c clamped a negative FROM and then printed the window the user asked for. The clamp moves to where m_from is what the report prints, and is announced.

NOT changed, deliberately: sweep's negative step is corrected on purpose (com_sweep.c:2119, "fix an obvious sign slip"); XSPICE Limits: are enforced; and .nodeset on an OSDI internal node really is ignored.

frontend/device.c, **frontend/outitf.c**, **parser/inp2r.c** — three names the simulator would not look up (E-493)

Round 53 found the same shape three times: a name the user wrote was not looked up where it was written, and what came back described something else.

showmod <MODEL NAME> COULD NOT FIND THE MODEL. The device generator's grammar reads a bare word as an INSTANCE name, and only a #-prefixed one as a MODEL name. So with .model dm d used by D1, showmod d1 printed the model and showmod #dm printed the model, while showmod dm answered "No matching instances or models" -- of a model that plainly exists. The command's own help calls its argument "models", and its write sibling takes the model name directly (altermod dm is=1e-12 works), so the one command dedicated to models was the one that could not be handed one. It affected OSDI models identically, the defect being in the name grammar rather than in any device.

RETRY RATHER THAN REINTERPRET. The bare-name-is-an-instance reading runs first and unchanged, so showmod d1 -- and any name that is both a device and a model -- behaves exactly as before. Only when NOTHING matched is each bare word retried with the # the grammar wants, and a name that is neither ends at the same message it always did.

A SAVED NAME THAT MATCHED NOTHING WAS DROPPED IN SILENCE. Pass 1 of the output setup walks each .save/.probe item against the analysis's own vector names and marks the ones it places; anything it failed to match simply stayed unmarked and the run continued without it. So .save v(n) v(nosuch) recorded v(n), dropped the typo and said nothing: the analysis succeeded and the vector the user asked for was merely absent. .probe v(nosuch) reaches the same path, which is why a mistyped NODE there was silent while a mistyped SOURCE in the same card is reported by the measure-source pass ("Could not find the instance line for ..."), and why Enhancement-418 already said it for the @dev[param] spelling ("no such device, so this vector will stay empty"). The plain node spelling was the one route left quiet.

It WARNS rather than refuses: an absent vector is not a wrong answer, and a deck that saves a node it does not always build is a real idiom. The check honours savesused[] first -- so the all/allv keywords and items belonging to another analysis are never reported -- and falls back to matching the name itself under save all, which .probe turns on and where Pass 1 never runs.

A RESISTOR MODEL NAMED **r** WAS UNREACHABLE. INP2R excluded the token r from being a model name outright. That exclusion is necessary for R1 a b r=1k, where r is the keyword that writes the resistance and must not be read as a model, but it also locked out a model actually CALLED r: .model r r rsh=1k with R1 a 0 r l=1u w=1u bound neither model nor value, so the device fell through to the default and came out as 1 mOhm with "resistance too low or not given" -- a message about the symptom, when the cause is that the model named on the line was never looked up. Every other name works, including every other resistor keyword (rsh, l, w, tc1, tc2, temp, dtemp, m, scale, ac, dc, noisy, short, narrow); only this one letter was unreachable.

The two spellings are distinguishable by what FOLLOWS the token: the keyword form is r=, a model name is not. Asking that instead keeps r=1k meaning what it always did and makes a model called r reachable. A deck with no such model is unaffected -- INPlookMod fails and the existing else branch restores the line and builds the default model, exactly as for any other unrecognised token.

Deliberately unchanged: showmod by device name, by #model and bare, and show, including a name that is both a device and a model; altermod <model>, which already worked; .save all, .save allv

and `.probe alli`, and any item belonging to another analysis; `@dev[param]` saves, which E-418 already speaks for; and `r=1k`, `r = 1k`, `R=2k`, a plain value, `r=1k tc1=0`, a value with `m=`, and `r=4k` winning when a model named `r` also exists.

`parser/inp2{e,g,s}.c`, **`parser/inppas3.c`**, **`analysis/cktop.c`**, **`KLU/klusmp.c`** — a node named only in a control position (E-492)

`E`, `G` and `S` take a controlling node PAIR, and those two names were bound exactly the way the output pair is -- INPtermInsert, which CREATES the node. So a typo simply invented a node and the run continued against it.

For `E` and `G` the invented node has no path to ground, the matrix goes singular, and the user is told "singular matrix: check node nosuch" -- a node they never wrote, reported as a fault in their circuit.

`S` IS WORSE. A switch only READS its control voltage to decide open or closed and stamps nothing for it, so the matrix stays non-singular, the solve succeeds, and the answer is silently wrong: `S1 a b ctl 0 sw` gives v(b) = 0.999001 while `S1 a b nosuch 0 sw` gives 9.99999e-07 -- a factor of a million from one mistyped character, rc = 0, with no diagnostic at all. A phantom reference is dangerous exactly where it is READ but not STAMPED.

Every other route already answered this question. `.ic` and `.nodeset` report "IC on non-existent node - %s, ignored"; `F`, `H`, `W` and a B-source's `i()` report "unknown controlling source"; and all thirteen output constructs name a vector that does not exist. `warn_physics` even validates the switch's own `ron`, `roff` and `vh`. Only the controlling-node pair skipped it, under any option.

The mechanism is Enhancement-429's, unchanged. E-429 added `CKTnode::devRef` -- "was this node ever named by a device, or made by the simulator?" -- so that a `.tf` card may legitimately precede the devices defining its nodes while a typo is still caught. A control reference is the same kind of reference: it names a node without making it real, so it no longer sets `devRef`, and `CKTnodePhantom()` answers the question as it stands. Marking is left to INPtermInsert for every other position, so a control node that IS connected somewhere -- including one that is the source's own output, `E1 out 0 out 0 2` -- stays marked and is never reported. The check runs in PASS 3 for the same reason `.ic`'s does: only once every device card has been read is "did anything connect to this?" answerable.

It REFUSES rather than warns. `E` and `G` already fail on their own; a switch does not, and would otherwise carry on and answer from a node that is not in the circuit -- the detect-announce-then-use-it-anyway shape Enhancement-485 had to undo eight times in one round.

CKTop blamed Verilog-A for faults that were not Verilog-A's. E-378 added the abort message and E-399 narrowed its entry to the `E_PANIC` checks, arguing the test is exact because E_PANIC (1) and E_ITERLIM (103) are distinct values. That is true -- but the invariant the MESSAGE relies on is "E_PANIC means a Verilog-A `$fatal`", and E_PANIC has around ten producers: cktsetup's missing model and device lists, osdiparam, osdiload, osdisetup, dcpss, cktpzstr, ifeval's bad node type and CIDER's twosolve. Measured: `.option klu` with a netlist whose matrix holds only current sources -- no Verilog-A device anywhere in the deck -- printed "a Verilog-A device raised `$fatal`" and pointed at an OSDI(fatal) message that does not exist, through op, dc, ac and tran alike, since each calls CKTop for its operating point. `CKTvaFatalRaised` is now set only where a `$fatal` is actually detected, and cleared on entry to CKTop so a previous analysis's fatal cannot be inherited.

KLU printed nine lines for one condition. PreOrder's own comment says an empty matrix is legitimate -- "XSPICE pure digital circuits produce empty KLU matrix" -- and it returns success for one. But all three sites printed "Error (...): KLU Matrix is empty" on every call, and Solve then added "KLUnumeric object is NULL" and "KLUsymbolic object is NULL", which are CONSEQUENCES of the same empty matrix rather than separate faults. Re-entered per Newton iteration, one such circuit produced nine or more lines, none of which named what had actually happened. It now says once, in words, what the state is.

Deliberately unchanged: a control node connected anywhere else -- the source's own output, one defined later in the deck, ground, or one passed through a subcircuit port; `F`, `H`, `W` and B-source `i()`, which already named their missing controlling source; a real Verilog-A `$fatal`, still named as such and still pointing at its OSDI(fatal) line; and an ordinary circuit under either solver, or a mixed-signal KLU run, which get no empty-matrix note.

`misc/printnum.c`, **`evt/evtprint.c`**, **`parser/ptfuncs.c`**, and seven more — an unbounded number used as a length, and four wrong ones (E-491)

Round 51 found ten defects sharing one shape: a number the deck supplies, used without being measured against what it was about to control.

THE CRASHES. `set numdgt` is the user's print precision and nothing bounded it. `printnum()` formatted with `sprintf(buf, "%.*e", cp_numdgt, num)` into caller buffers of BSIZE_SP (512); `evtprint.c` formatted into a `char[100]`. `%.*e` needs n+9 bytes, and both thresholds land exactly there: `set numdgt=510` with `print` aborts, `set numdgt=94` with `eprint` traps, both from a plain batch deck with no interactivity. `numdgt=94` is an ordinary "give me lots of digits" value.

printnum()'s OWN COMMENT had recorded the hazard -- "It can cause buffer overruns. The size of buf is unknown, so cp_numdgt can be large enough to cause sprintf() to write past the end of the array" -- without bounding it. The safe DSTRING sibling `printnum_ds()` sits directly below and cannot overflow, which is precisely why `fourier`, `wrdata`, `write`, `display` and `diff` were unaffected and only the `print` family crashed. printnum() now takes the buffer's size and CLAMPS the precision to what fits rather than truncating: snprintf alone would stop mid-number and hand the reader a value that is not the one computed, while beyond ~17 significant digits the extra places are zero padding, so a clamped column is the same number, just narrower -- and it says so once. evtprint uses the same clamp, so the two printing paths cannot disagree about how wide a number may be. Both sites trace by `git -S` to `4f29ffad`, the vanilla UPSTREAM import.

THE WRONG NUMBERS. `PTdivide` added `PTfudge_factor` to EVERY divisor rather than only a zero one, and that factor is `gmin * 1e-20` -- not even a fixed perturbation, but one that scaled with an unrelated convergence option. One over Boltzmann's constant came out 42% low under `.option gmin=1e-3` and 88% low under `gmin=1e-2`; `1/0` returned 1e26, 1e32 or 1e50 depending on gmin alone. A deck's arithmetic must not move because the user reached for a convergence aid, so the nudge now applies only to an exact zero and uses a fixed epsilon -- a non-zero divisor is used as written, `1/0` is the same number in every deck, and the value keeps the default gmin's 1e-32 so nothing already correct changes.

The B-source reduced trig arguments with `x - (int)(x/2pi)*2pi`. That cast is undefined above 2^31*2pi (~1.35e10), and `sin(1e20)` returned +0.9993 where the answer is -0.6453 -- wrong sign, wrong magnitude, no diagnostic. Both OTHER evaluators in this simulator were already right: numparam and a Verilog-A model's own sin() each match libm exactly, making the B-source the SOLE OUTLIER. That is the divergence Enhancement-399 forbids, and ptfuncs.c's own comment quotes the rule. libm reduces correctly for every finite double, so the macro is gone.

THE SILENT REFUSALS. A `sens` filter matching no parameter produced no plot and said nothing, returning OK and leaving the PREVIOUS analysis current for a following `print` -- and the trailing words on a `sens` line ARE filter patterns, so `sens v(b) r8` is a legitimate restriction that works, which made a mistyped filter indistinguishable from one that did its job. The interactive `meas` enforced its analysis keyword for INTERVAL measurements -- Enhancement-468 restored that -- and ignored it for POINT ones, so `meas tran m FIND v(n) AT=0.5` read a DC sweep as a transient; the check now runs once for every measurement type, in the command only, since the `.meas` card already gated correctly. `s_xfer` blamed the SOLVER for a model error: an all-zero denominator went NaN and was reported as "Dynamic gmin stepping failed", while a numerator longer than the denominator was detected, announced once per evaluation (1238 times over 300 steps), and the run then returned rc=0 with the device contributing nothing -- both knowable from the parameters before any solve. `printf("oops ")` reached users on

STDOUT from two `default:` arms in inpptree.c, one of them the very printer the "internal check of parse tree ... failed" diagnostic uses to SHOW the offending expression, so that message rendered as the word "oops" -- reachable from `.param p={ln(0)}`. `show <dev> : <bogus>` printed `?????????` where `print`, `alter`, `altermod`, `wrdata` and `sweep` all name the parameter. And a duplicate user `.func` was silently last-wins while shadowing a BUILTIN warned (E-467), so two includes each defining a helper displaced one another unannounced.

ONE COMMENT CORRECTED RATHER THAN OBEYED. Enhancement-440's comment claimed the singular-value routes "clamp to HUGE" and that clamping "keeps the Jacobian finite, which is what lets NIiter reach for gmin or source stepping". Neither is what the code does: `PTeval` in ifeval.c treats a result equal to HUGE as an ERROR FLAG, returning E_PARMVAL and aborting -- the opposite of continuing -- and the routes were never uniform (`sqrt(-1)` and `pwr(0,-1)` abort, `log(0)` returns -1e99 and runs on, and PTdivide no longer reaches HUGE at all). Recorded rather than unified: making `log(0)` abort, or `pow(0,-1)` not abort, would each change the answer a working deck gets today. What was wrong was the description, and a reader trusting it would have drawn exactly the wrong conclusion about which of these keeps a simulation alive.

Deliberately unchanged: ordinary precisions (6, 12, 17 digits format exactly as before, and only a value that cannot fit its field is narrowed); `1/0` still returns a large finite number so the solve continues; interval measurements keep E-468's behaviour; a `sens` run with no filter or with one that matches; and `.func` still resolves to the last definition -- only the silence is gone.

`osdi/osdiinit.c` — an operating-point variable whose name the simulator also supplies (E-505)

ngspice writes its own instance parameters — `m`, `temp`, `dtemp`, `dt` — into the instance lookup table before anything the model declared, so a model with an operating-point variable of one of those names computes it on every evaluation and can never read it back. `@n1[temp]` returned the ambient temperature and `@n1[m]` the multiplier. The name is legal Verilog-A and nothing said a word, in the compiler or the loader.

It is named at load time rather than refused: the rest of the model works, and refusing would break a model that runs today over a name it never reads.

Operating-point variables only. A model may legitimately declare `m` or `dtemp` as its own instance PARAMETER — that is what E-394's `has_m` exists for, a CMC-style model scales by its own `m`, and the loader routes the deck's value into it, so nothing is shadowed. The first version of this warning fired for both and `limguard_examples` caught it — that suite asserts the parameter case is silent, and `dropguard_examples` now pins it from the other side.

The three compiler-side halves of this enhancement are in the openvaf-r report.

`osdi/osditrunc.c`, **`osdi/osdidefs.h`** — a model may not demand an unbounded step count (E-504)

Two ways for a Verilog-A model to hang the transient indefinitely — over 150 s with no output, no error, and no "timestep too small".

`$bound_step(1e-18)` is a perfectly legal positive request and was taken literally. The "timestep too small" test compares against `CKTdelmin`, which for a 12 ns analysis is ~5e-20 — far *below* the 1e-18 being asked for — so nothing ever fired. The step was not too small for the solver; it was too small to finish. No clamp value makes 1.2e10 steps work, so the rule is stated in the only terms that bound the run: a model may not force more than **`E504_MAX_MODEL_STEPS`** (10^6) steps across the analysis window. Beyond that the bound is clamped and the instance is named once. A model wanting genuinely fine resolution is unaffected, and ngspice's own adaptive stepping is untouched — this bounds only what a DEVICE may demand.

`$discontinuity(0)` outside any conditional pinned the timestep to the last accepted delta, which could then never grow, so once the retry had cut it the transient crawled for the rest of the analysis. E-55 already

made the *retry* edge-triggered for exactly this reason; the cap needed the same floor. `$discontinuity(-1)` means "no discontinuity", is fine, and stays untouched and silent.

The companion compiler-side fixes — `transition`'s negative rise time, a non-positive `$bound_step`, a negative noise power and `idtmod`'s zero modulus — are in the openvaf-r report. The noise sign is deliberately NOT fixed here: by the time the power reaches `osdinoise.c` the contribution factor has been folded into it as `fac*|fac|`, and E-42 uses that sign to sum same-named sources coherently.

Verified by `examples/rtdomain_examples/verify_rtdomain.py` — 16 checks under both linear solvers, 11 of which fail on the shipped binaries.

`spicelib/analysis/cktdojob.c`, **`frontend/com_sweep.c`** — setup reuse for decks containing a built-in semiconductor (E-503)

E-471 reuses the matrix between sweep points instead of tearing the circuit down and rebuilding it, and offered that only to circuits built entirely from types whose topology is fixed *unconditionally* — the linear elements and sources — plus OSDI, whose collapse is re-decided and compared on every `CKTtemp`. Every other device type refused reuse outright, so one transistor anywhere disabled it for the whole circuit: 3.46x at 300 sections, 5.28x at 600, 10.09x at 1200.

That is a per-type gate for a per-parameter hazard. A built-in semiconductor decides its node collapse in `DEVsetup`, from a knowable set of parameters and from nothing else — BJT `rc/rb/re/rco`, diode `rs/rsw/vp/tt`, JFET `rd/rs`, and the Mos1/2/3/6/9 family `rd/rs/rsh/nrd/nrs`. Each row was read off the device's own setup routine by auditing every `CKTmkVolt` call and the condition guarding it, not from the parameter documentation, which does not say which parameters build nodes; `rco` gates a fourth BJT node through its `…Given` flag rather than its value and was missed on the first pass, which is what the audit caught.

The sweep now declares the parameters it varies, and reuse is allowed for these types when no declared name builds a node: 2.28x / 3.25x / 6.69x recovered at 300 / 600 / 1200 sections. The remaining gap to the all-linear time is the transistor's own evaluation, which no setup reuse can remove.

The absence of a declaration is what keeps it safe. `sw_request_reuse()` CLEARS the declaration rather than leaving whatever the last caller wrote, so a caller that cannot enumerate what it varies inherits nothing and keeps E-471's stricter gate: Monte Carlo binds its draws through the deck, and a `.param` knob reaches a model parameter through an expression this code cannot see — either could be feeding the very `rc` that decides a collapse. The match is whole-token, so a sweep of `rsh` is not read as a sweep of `rs`, and a type absent from both tables is still refused.

Where the declaration lives is part of the fix. It is a file-static in `cktdojob.c`, not a field on `CKTcircuit`. The first version put it in the struct, which grew it by 256 bytes and shifted every field after it — and made the `argguard` suite fail about two runs in three, its `warn_physics` count coming back 1, 2 or 3 at random for the same deck. The logic was not the cause: 256 bytes of padding alone, with none of this code, reproduces it. Something else in ngspice reads that struct by a layout it did not get from the current header. That is a real latent defect, it is recorded as an open finding, and this enhancement must not be what makes it reachable; held outside the struct the layout is byte-for-byte unchanged and `argguard` is stable ten runs out of ten.

Verified by `examples/reusedev_examples/verify_reusedev.py` — 16 checks under both linear solvers, 8 of which fail on the shipped binary. Every reuse check compares the swept curve against `.option reusesetup=0` and requires bit-identical results, having first asserted that the curve actually moves: a comparison of two flat lines proves nothing, and E-471's own notes record a naive version that silently froze a collapse and drew one.

frontend/parser/numparse.c, **frontend/com_{envelope,emir,eye,qpss,reduce,hbosc,compose,rfstab}.c** — the guard that refuses the wrong value and admits NaN (E-502)

Round 60 swept the commands that produce a report — `emir`, `eye`, `reduce`, `envelope`, `qpss`, `hbosc`, `stb`, `rfstab`, `compose` — and found one guard written the same way in six of them:

```
if (x <= 0.0) { refuse; }
```

Every comparison with NaN is false. That test refuses `0`, refuses a negative number, and lets NaN straight through — one `!` away from correct.

A one-token SIGSEGV. `envelope <node> nan <tstop>` crashed five runs out of five with `EXC_BAD_ACCESS address=0x10` in `SMPmatSize`. `fc` is NaN, so `T = 1/fc` is NaN, so the settling transient is built as `tran nan nan 0 nan` — which ngspice refuses. The refusal was never checked, the matrix was never built, and `EFanalysis` dereferenced it one call later. Both halves are fixed: the argument is refused at the front, and `envelope` now checks that its own internal transient ran. That second check matters beyond this bug — any refused transient leaves `CKTmatrix` NULL, and a command must never use the result of an internal analysis without asking whether it happened. That is E-438's shape, where a failed sample kept the previous plot and was counted as a pass.

A limit that is never exceeded. `emir jmax nan` reported *"0 segments over Jmax"* with every segment `ok`, on a grid with two genuine violations. `reduce nan` reported *"26 nodes -> 26 nodes (1.0x)"* and wrote a file called `reduced.sp` that reduced nothing. `qpss v(o) nan 1.1e6` printed a 25-row spectrum in which every frequency, magnitude and phase was `nan`, and exited 0. `eye -ui nan` reported an eye height of 0 — a fully closed serial link. Each of those guards proves its own intent by working: `jmax 0`, `reduce -1`, `-ui 0` and `qpss` with two equal tones were all correctly refused. This is E-501's NaN spec bound one command over: a limit used only in a comparison is not a strict limit when it is NaN, it is *no limit at all*.

Two returned a different answer rather than a blank. `eye -tstart nan` was neither refused nor blank — the "skip samples before tstart" test is also a comparison, so NaN never skipped, the startup transient the flag exists to exclude was folded into the eye, and RMS jitter came back 9.86e-13 against 1.50e-15, 660x, with no diagnostic. `qpss ... maxorder nan` silently became `order <= 1`, dropping every intermodulation product — the entire reason that command exists — with only a quiet `order <= 1` in a banner. Both reached their clamp through an undefined `double->int` conversion, as did `emir top nan` (landing on 1) and `compose lin=1e12`, which landed on `INT_MAX` and really allocated 17 GB: the vector was genuine, `v[2147483000]` read back correctly.

`emir` could not tell a given width from a default. It computes current density from `@r[w]`, which answers `1e-5` for a resistor whose deck line never gave a width — so the segment most likely to be the oversight was analysed as a comfortable 10 um conductor and reported `ok`, while the header of `com_emir.c` documents that such segments are *skipped*. The inverse was also true: `w=0` and `w=-0.5u` were skipped and reported as *"no width given"*, the one thing they were not. `emir` now asks the instance (`RESwidthGiven`, reading resistor internals as `spicelib/analysis/rcreduce.c` already does for `reduce`) and reports the two reasons separately.

The idiom was already in the tree, and three commands that got it right were left alone: `hbosc` refuses `K nan` because its test is `K >= 1`; `compose` refuses NaN on every parameter it takes; `stb` delegates its frequency spec to `ac`. The guards now share `ft_argpos`, `ft_argfinite` and `ft_argcount`, added next to `ft_numparse` itself. They take the raw token so the diagnostic names what the user typed, and they parse with `ft_numparse` rather than `strtod`, because these are SPICE numbers and `thick 0.5u` / `-ui 0.5n` / `1meg` are how the documentation writes them — E-501 shipped a guard built on `strtod`, it refused `dynamic 20u`, and the aging suite caught it. `ft_argcount` checks the value before the cast, because converting a NaN or an out-of-range double to `int` is undefined.

Verified by `examples/nanguard_examples/verify_nanguard.py` — 34 checks under both linear solvers, 23 of which fail on the shipped binary.

frontend/com_aging.c, **frontend/com_sweep.c**, **frontend/com_optimize.c**, **frontend/runcoms2.c** — aged state across an internal reset, and what the loop commands claim (E-501)

The aged device came back fresh. `aging` (E-157) writes an accumulated dose into each device's `age` parameter with `alter`, then leaves the circuit standing so the analyses that follow see the degraded part. That dose lives only in the running circuit — it has no deck representation, because it was never in the deck. So every command that re-sources the deck between evaluations threw it away on the first one and ran the rest of the loop on a fresh circuit, in silence.

That is `wcd`, `highsigma` and `optimize -center`: the three commands whose entire purpose is to characterise a part were the three that could not characterise an *aged* one. On the suite's deck a 95-year dose moves the worst-case distance from 8.66 sigma to 1.81 sigma — a part that sits comfortably inside its limit when new and barely inside it at end of life — and `wcd` printed 8.66 for both, with nothing in its output to say which circuit it had measured.

`montecarlo` escaped, but only by accident: E-346's fast path arms whenever a random value binds — and prints *"no per-sample reset"* when it does — so the usual Monte Carlo never re-sourced at all. Its fallback path re-sources like the others.

`aging` already issues its writes as ordinary `alter` commands, so it now records them as it writes them, and the loop commands replay that record after their internal `reset`. The record is per-run — `aging` clears it before writing — which preserves E-157's stated property that a dose is an *absolute* target and calling it twice is idempotent. A `reset` the user types must still mean "the deck as written", or there is no way back to a fresh device short of reloading the file; so the internal resets are marked as internal, `com_rset` drops the record when it is not marked, and only the marked ones replay.

`aging` accepted targets it could not age to. Its numbers went through a bare `atof()` and its guard was `t_target <= 0`. Every comparison with NaN is false, so `aging nan` walked through it and wrote NaN doses into every device while reporting them aged; `aging abc` aged to 0 s; and `dynamic`'s stop time was never checked to be a number at all, so `aging 1e8 dynamic verbose` consumed `verbose` as the stop time and then complained about `agerate`, a token the user had never typed. The guards are now written `!(t > 0.0) || !finite(t)`, and the stop time is checked before it is consumed so the message names what was actually typed. This is the shape E-497 fixed in `setseed` and `disto`.

A spec bound that is never violated. A limit is only ever used in a comparison, so a NaN limit is not a strict limit — it is *no limit at all*, indistinguishable from omitting the flag. `montecarlo ... -max nan` reported `yield : 100.000%` with a Wilson interval. `montecarlo -spec -max/-min`, `wcd -max/-min/-tol/-step`, `highsigma -max/-min/-scale` and `optimize -spec -max/-min` are now parsed the way the rest of each command's numbers already were — with `ft_numparse`, not `strtod`: the first version of this guard used `strtod` end to end and refused `dynamic 20u`, the documented spelling in E-157's own example, which the existing `aging` suite caught. A negative bound stays legal, which is why the spec readers are not E-499's `-tol` reader; that one refuses negatives, correctly for a tolerance and wrongly for a limit.

A yield with nothing behind it. A yield is a statement about variation. A deck with no random parameter — or with one the metric does not depend on — produced 50 identical samples and reported `100.000% (50/50)` with a 95% CI of `[92.865%, 100.000%]`: an interval that looks tight precisely because every sample was the same sample. E-495 warns when an individual `agauss(...)` call is degenerate; this is the whole-run version, and it catches what no per-call check can.

`optimize` published the score but not the answer. `optimize -center` published `dcenter_yield` and `dcenter_cpk` — what it *scored* — but never the centred knob value, the thing the command exists to find. A `-dparam` is a numparam symbol that never becomes a vector, so the shipped `dcenter_demo.cir` asked for it as one and printed `Warning from checkvalid: vector xc is not available` on every run since the example shipped. Knob values are now published as `optimize_<name>`, and the demo prints `optimize_xc = 5.0022` — the design centre its own comments predict.

`aging param` aimed somewhere odd. `aging 1e8 param w` wrote 4.095e8 into a MOSFET's *width* — a transistor 409,500 km across — reported *"1 device aged"*, and exited 0, because `aging` selects devices that *have* the named parameter and a width qualifies. The name stays the user's to choose, since models spell their aging state differently, so this warns and proceeds; but when the same device also exposes a plain `age`, choosing something else is far more likely a slip than an intent.

Verified by `examples/agestate_examples/verify_agestate.py` — 30 checks under both linear solvers, 19 of which fail on the shipped binary.

`frontend/com_dl.c`, **`frontend/inp.c`**, **`frontend/com_presnp.c`**, **`osdi/`** — **`pre_osdi -va`** compiles Verilog-A and loads it in one step (E-500)

A deck that ships its own models needed a separate build before it could run: compile each `.va` by hand, then point `pre_osdi` at the objects. `pre_osdi -va rmod.va gmod.va` now invokes openvaf-r on each source and loads what it produces, in one command.

Where the objects go. Into an `osdi/` directory beside the netlist, not beside the working directory and not next to each source. It keeps a model directory clean when a deck carries a dozen sources, and it makes an input/output collision structurally impossible: E-452 recorded `openvaf-r m.va -o m.va` destroying the source and exiting 0, and an object written to `osdi/<stem>.osdi` cannot name the input whatever the input was called. The `.va` argument resolves against the netlist too, exactly as `load_osdi( )` already resolves the object it is handed. That was a bug in the first version of this change — the object went beside the netlist while the source was looked up beside the working directory, so the command worked from one directory and not from another. The two halves of one command must not disagree about what a relative path means.

Which compiler. `osdi_find_openvaf()` is `pre_snp`'s existing lookup, exported rather than reimplemented, so both generators share one policy: the `openvaf` ngspice variable, then `$OPENVAF`, then `$SPICE_LIB_DIR/openvaf-r`, then `PATH`. Searching `PATH` alone would have been the obvious implementation and the wrong one — for anyone running the shipped bundle the compiler sits in `$SPICE_LIB_DIR` and is frequently nowhere else.

Recompiling is the default; **`.option osdicache`** opts out. The instinct is to cache by default, and it is wrong here: a `.va` timestamp records whether the SOURCE changed and says nothing about the COMPILER having changed, so while openvaf-r is itself under development a cached object is one built by a version that may no longer exist. That is the shape of E-453, whose cache key omitted its own codegen settings. The option is read from the deck's options cards, not through `cp_getvar` — `pre_` commands run before the circuit is set up and before any option is published, the trap E-464 recorded for `autobus` — and is matched as a whole word rather than with `strstr`, because E-451 shipped an option whose name was decided by a substring search. Every spelling that means off means off.

`-f` forces the rebuild, not just the reload. E-229 added the flag for an edit -> recompile -> re-source loop; under `-va` the compile is part of that loop, so honouring `-f` for the load alone reloaded the very object the author was trying to replace — the one case the flag exists for. This was the second defect in the first version and the sharper one: with `.option osdicache` set, an edited `.va` silently kept answering with the old model. The suite edits a source and checks the answer moves from `0.5` to `0.3333` only when `-f` is given.

The staleness test is strictly newer, never **`>=`**. `st_mtime` is one-second granular on POSIX and two-second on FAT, so an edit and a re-run inside one tick would otherwise load the object built from the previous text. A tie costs one needless recompile; the other way costs a wrong answer — and that choice is also what makes a coarse filesystem clock harmless rather than dangerous.

Portability. `stat`/`st_mtime` come from `<sys/stat.h>`, which `inpcom.c` has used on Linux, macOS and Windows for years. `mkdir` is the one call that differs and is guarded on `_WIN32` — defined by MSVC, MinGW, MinGW-w64 and clang-on-Windows — where `_mkdir()` comes from `<direct.h>`.

Mixing. `-va` only changes how a `.va` ARGUMENT becomes a path; anything else still goes straight to `load_osdi`, so the two mechanisms converge on one loader. Every arrangement works: either order on separate lines, either order on one line, `-f -va` beside a pre-built object, and the same model arriving by both routes at once (reported as already loaded, E-229's existing behaviour, not an error). `-va` on a line with no `.va` argument is a no-op.

Refusals name the cause. A missing source is *no such Verilog-A source* rather than `exit 512`; one that will not compile shows the compiler's own diagnostics plus all four ways to supply a compiler; and a read-only directory is not fatal — the objects are written beside the netlist and the fallback is announced, so a deck in a shared PDK or a CI checkout still runs.

Unchanged: plain `pre_osdi file.osdi`.

`maths/KLU/klusmp.c`, **`frontend/com_optimize.c`**, **`frontend/com_sweep.c`** — loop-command arguments, verdicts, and a KLU refactor kind (E-499)

Round 58 re-probed the `sweep` / `optimize` fast path after E-498 fixed the transient side. The reuse arithmetic was clean this time. What was wrong was everything around it — the arguments the commands accept and the verdict they print — and, underneath both, one genuine memory bug that setup reuse made reachable.

A complex refactor handed a real factorisation. `klu_z_refactor` refills an existing factorisation *in place*, walking the L and U index arrays that `klu_z_factor` built. Handed a real `Numeric` object it walks half-sized arrays with complex strides: it reads and writes past their ends, and a later `klu_free_numeric` frees whatever it scribbled. The malloc abort names object `0x3ff0000000000000` — the IEEE-754 bit pattern of the double 1.0, a matrix *value* being freed as a pointer. The mismatch is reachable because a `Numeric` outlives the analysis that built it: every AC/SP/NOISE run is preceded by a real operating point, and E-471's setup reuse keeps the matrix standing between sweep points, so the second and later points arrived holding the operating point's real factorisation. Under `.option klu`, `sweep -analysis ac` and `-analysis noise` returned 0.0 at every reused point, `optimize -analysis ac` fitted a parameter 10x wrong while reporting *converged*, and `-analysis sp` crashed on 9 of 10 runs. SPARSE was never affected, and neither was any analysis that stays real — `op`, `dc`, `tran`, `tf`, `pz` and `disto` were correct throughout, which is what kept the scope narrow enough to miss. The matrix now records which kind of factorisation its `Numeric` holds and a kind change forces a full factorisation; both directions are guarded, because a real refactor of a complex `Numeric` is wrong the same way round.

Four arguments parsed by a weaker parser than the one in the same file. `com_optimize.c` has always had `optnum()` — built on `ft_numparse`, so it reads SPICE suffixes *and* scientific notation — and uses it for the bounds, `-target` and `-spec`. Its integer options did not: `-maxiter`, `-samples` and `-swarmsize` called `atoi()` and `-tol` called `atof()`, which stop at the first character they cannot use. So **`1k`** meant 1000 in a bound and 1 in **`-maxiter`** on the same command line: `-maxiter 1k` and `-maxiter 2e2` ran one or two iterations and returned 1495 where `-maxiter 1000` returns exactly 1500, and `-samples 2e2` ran 2 Monte-Carlo samples while design centering still printed a yield and a Wilson confidence interval computed from them. `abc` was 0 in all four, in silence. `sweep lin 2e2` (200 points) and `montecarlo 2e2` (200 samples) were already right — and `sweep` had grown a strict count parser in E-478 for exactly this reason. All four now parse strictly and refuse a value they cannot use, because a count or a tolerance *is* the request. `-seed` had the matching hole: E-497 taught the `setseed` command to refuse `3.7`, but the option spelling truncated it without a word, and `-seed 0` / `-seed -3` left the run unseeded and not reproducible while the author had asked in writing for a fixed seed. All four `-seed` sites now apply the command's rule.

”converged” when nothing was optimised. The word describes the search stopping, not the answer being the one the author wanted. A parameter the objective cannot depend on — or a name that does not resolve at all — ends after three evaluations with the starting value handed back as the fit (1 k, 5 k and 9 k returned 1000, 5000 and 9000, each labelled converged); `-target x 0.4 0` printed `sum-sq residual`

= 0, the most convincing number the command can produce, for a fit that never happened; and an answer pressed against a search bound was not distinguished from a minimum. The objective's spread across the run, and the answer's position relative to its bounds, were both available and unused. Said now, as E-438 already says it for failed evaluations and as QPSS-HB says *STALLED above tol — accepted*.

A knob that names nothing. `sw_kind()` falls through to `SW_ALTER` for any name it does not recognise, and the comment on that fallback already describes what follows: `alter` reports "no such device", the sweep runs anyway over a knob that never moves, and the user gets a full set of points, `rc = 0`, and a plottable flat curve whose x-axis is named after the typo. E-435 removed that shape for a subcircuit-local model name and E-488 for `temp`; this removes it for every remaining spelling, exactly as E-431 did for an unresolved `–output`. A malformed `list` value did the same on a smaller scale — `list 1k inf 2k 3k` ran one point and called `inf`, `2k` and `3k` unrecognised, though `list 1k 2k 3k` proves the last two are read without complaint. The list check is deliberately narrow, firing only for a token that reads as a number and is still unusable, because the token after a list is very often the next knob.

`–overlay` discarded the timepoints the solver chose. It resampled onto a uniform grid of the same point count as the longest run. A transient places its points where the waveform moves, so keeping the count and throwing away the placement is exactly backwards: the overlay of an RC driven by a 0.2 us pulse reported a peak of `0.244` where every one of its own runs, and a standalone run, said `0.402` — 39% low, from the same 121 points. It now uses the union of the runs' own timepoints, bounded to a small multiple of the old size, and says which grid it used. Its per-point vector names were formatted with `%g`, six significant digits, so `list 1000.0001 1000.0002 1000.0003` produced three vectors all called **`vn_1000`** of which only one was addressable; they now carry the shortest text that reads back as the exact value, as E-494 settled for the same class of defect.

Two warnings that contradicted each other. A point whose analysis did not converge is recorded as `NaN` (E-445) and *was also* counted as an unresolved output, so one run said both *recorded as NaN, not as results* (true) and *those entries are zero* (false) about the same points. An output is now counted as unresolved only where the analysis converged.

Unchanged: a real fit is not annotated; every legitimate knob spelling (bare device, `@dev[param]`, `temp`, deck `.param`), a good `list` and a two-knob sweep still parse; a transient sweep and the reuse tally are untouched, so the KLU guard cannot be mistaken for disabling refactoring; and a legal `–seed` still pins the run.

`devices/vsrc/vsrctemp.c`, **`osdi/osdisetup.c`** — per-run state left stale by the setup-reuse fast path (E-498)

E-471 lets `sweep`, `optimize` and `montecarlo –warm` keep a circuit standing between points: `CKTdoJob` skips `CKTunsetup`/`CKTsetup` and re-runs only `CKTtemp`. That reasoning was about topology, and for topology it is right — node collapse is re-decided on every `CKTtemp`, and reuse is offered only to device types whose topology is fixed. What was never asked is a different question: *which state does `DEVsetup` initialise that is not topology at all?* For almost every device the answer is none. For two of them it is a value the transient run walks forward and expects to find reset at time zero.

A voltage source stopped scheduling breakpoints entirely. `VSRCbreak_time` is the source's own breakpoint cursor: `VSRCaccept` arms the next edge only when `CKTtime >= VSRCbreak_time`, and `vsrcset.c` seeds it to `–1.0` so the first accepted point of a run arms the first edge. Under reuse that line never ran again and the instance carried the previous run's break time — a value at or past that run's TSTOP — so the test was false at time zero and stayed false for the whole run. The consequence is not a perturbation: the source armed no breakpoints at all, and the stepper walked straight over every edge. A reused point landed on 0 of 5 corners of a piecewise-linear pulse where a standalone run landed on all five (81 native timepoints against 103).

Measured against a standalone run of the identical circuit, `maximum(v(n))` — a grid-independent quantity, so this is a different answer and not a different sampling — was +16%, +44% and -10% across a five-

point sweep, and 106% at other spacings, in silence. A 0.3 ppm change of the swept value was enough, because the schedule is either armed or it is not. Only a sweep's *first* point still had a correct schedule, so the same resistance returned two different answers depending on the direction of travel: `R = 1000` gave `0.40217511` sweeping up and `0.33963131` sweeping down.

`optimize` inherits it through E-472. Fitting R so that `maximum(v(n))` reaches `0.20` gave `2501.777336` with reuse off — which puts the peak at `0.2000000003` — against **`2156.690117`** with it on, giving `0.2264`, 13% out, in 106 evaluations rather than 67. Both printed *converged*.

This contradicts E-471's own escape-hatch comment, which promises that reuse *changes nothing a user can see except the time* and moves *a value by a few ulp*. The magnitude is fourteen orders of magnitude past that, and the cause it names is refuted as well: KLU and Sparse, two entirely different orderings and factorisations, produce bit-identical deviations.

OSDI's **`last_crossing`** cache. `crossing_time[]` is per-run state too, and `osdiaccept.c` states its contract outright — the cache is left unchanged when no qualifying crossing is found, so `V(z)` keeps reporting the last crossing per the LRM, and it *starts at 0.0 before any crossing has been observed*. Only `OSDIsetup` seeded it, so a reused point began holding the previous point's crossing: a 100 kHz sine over 40 us made `last_crossing(V(in),1)` read `3e-05` at time zero of points 2 to 5 instead of `0`. Its sibling analog operator was already right — `absdelay` re-seeds `delay_hist[k][0]` in `osdiload.c` on `is_init_tran`, the first transient call of each run rather than at setup — and that is the pattern this now follows.

The fix. Both are re-armed in the device's `DEVtemperature` method, `VSRCtemp` and `OSDItemp`. `CKTtemp` runs once per job on both paths, reuse and rebuild — the reuse branch calls it explicitly, to re-decide OSDI node collapse — and does not run on a `resume`, because `CKTdoJob`'s reset branch is skipped there, so a continued transient keeps the schedule it was paused with. Setup keeps its own seed, now as the allocation-time default rather than the only place the value is ever set.

Why the existing suites stayed green. `reusesetup_examples` and `reuseloops_examples` contain no PULSE or PWL source between them — they exercise `tran 10u 1m`, but only with sources that were provably unaffected. The two shipped examples that do combine `sweep`, `tran` and a PULSE both use `-overlay`, whose resampling onto a common grid damps the error below their tolerances. Both moved when this was fixed: `sweepwave_demo`'s `vout_2000[10]` went from `2.441425e-02` to the correct `2.440859e-02`.

Found here, deliberately not fixed here. Under KLU, `sweep -analysis ac` (and `noise`) with the reuse fast path returns `0.0` for the reused points and crashes outright in roughly one run in ten (SIGTRAP). It is non-deterministic — byte-identical decks give different answers run to run — which points at memory rather than arithmetic. It reproduces on the shipped binary, is absent with `reusesetup=0`, absent under Sparse, and absent from a manual `alter` plus `ac` loop, so it belongs to the reuse path, but it is a different defect in a different layer and needs its own investigation. It is reported rather than quietly tested around; the two controls that would touch it are asserted under Sparse.

`analysis/dsetparm.c`, **`analysis/nsetparm.c`**, **`maths/misc/randnumb.c`**, **`frontend/inpcom.c`** + five XSPICE models — constraints the documentation states and the code did not check (E-497)

Round 56 tried a technique worth keeping: mine the manual for stated numeric constraints, then ask of each whether anything enforces it. Three of the five findings came out of one `pdftotext ngspice-manual.pdf | grep -E "must be|should be"`.

THE **`disto`** TWO-TONE RATIO WAS UNVALIDATED. The manual is explicit -- f2overf1 *"should be a real number between (and not equal to) 0.0 and 1.0"* -- and the ratio is not decorative, because it sets the second tone. Measured on a reactive circuit, the 2F1-F2 product read 1.695 at ratio 0, 1.630 at 1, 1.477 at 1.5 and 1.580 at -0.5, against 1.711 for a legal 0.5. At ratio 1 F2 equals F1, so the plot ngspice still labels *"IM: f1-f2"* holds a product at DC; at a negative ratio the second tone sits at a negative frequency. All

accepted in silence, and the numbers look ordinary either way.

Both neighbouring cases in the SAME switch already test their value and return `E_PARMVAL`; `D_F2OVRF1` was a bare `job->Df2ovrF1 = value->rValue;`. Refused rather than clamped: the ratio IS the experiment being asked for, and any substitute would answer a different question without saying so.

The guard is NARROWER than the manual, and deliberately so. The first version took the manual at its word and refused everything outside (0,1); E-255's suite caught it in the very next regression. That suite measures the two-tone IM3 at f1 = 1.0 GHz and f2 = 1.3 GHz and proves the result machine-exact against an independent QPSS harmonic-balance engine, so F2 above F1 leaves all three products at distinct nonzero frequencies and is well posed -- keeping F2 below F1 is a convention, and a working verified deck is better evidence than the sentence in the manual. Refused now: only `<= 0` (the second tone at DC or a negative frequency) and `== 1` (F1-F2 would be DC and 2F1-F2 would be F1, so the plots would not hold intermodulation products at all).

`setseed` SILENTLY TRUNCATED A FRACTIONAL SEED. `%d` stops at the first character it cannot use, so `2.5` scanned as 2. Every other bad spelling is named (*"Cannot use 0 / -3 / abc as seed!"*) and the sibling command `repeat` names a fractional count outright (*"bad repeat argument 3.7"*), so this was the one way to be wrong quietly.

`s_xfer` INDEXED **`int_ic`** BY THE WRONG ARRAY'S SIZE. The initialisation loop reads `PARAM(int_ic[den_size - 2 - i])` -- `den_size` being `PARAM_SIZE(den_coeff)` -- and nothing anywhere consults **`PARAM_SIZE(int_ic)`**, though the manual states int_ic "must be of size one less as the array of values specified for den_coeff". Too short and the initial conditions the array does not reach read as zero, taking v(out) at 10 us from 7.00005 to 4.99995e-05; too long and the surplus was never looked at.

THE OSCILLATOR FAMILY WENT SILENTLY DEAD. `sine`, `square`, `triangle` and `oneshot` each announced a control/frequency array mismatch and returned WITHOUT SETTING AN OUTPUT, on every evaluation: rc=0, the source held at zero all run, and 2025 message blocks over a 2 ms transient -- about a million for a 1 s run. This is precisely the shape E-491 fixed in `s_xfer`'s `cfunc.mod` and explained there (*"say it once and stop, the way file_source does when its file will not open"*); four models were left doing the old thing. `d_osc` and `d_pwm` perform the same check inside their `INIT` block and so already say it once -- deliberately untouched.

A DUPLICATE **`.param`** WAS THE ONLY ONE OF ITS FAMILY TO PASS IN SILENCE. `.func` has been reported since E-491, `.model` and `.subckt` long before that, `.param` not at all -- and it resolves the OTHER WAY from two of them, taking the LAST definition where `.model`/`.subckt` keep the FIRST. So two included files that each set `vdd` agree or disagree purely by include ORDER, and a `.param` written in the deck is silently displaced by a library included after it; both measured. Which value wins is not changed -- decks depend on last-wins -- it is only made audible, scoped to TOP-LEVEL cards by the same `.subckt` nesting count that already governs the `.func` scan, because a subcircuit's own parameters are legitimately redefined once per instance.

Also corrected, with no demonstrated consequence: `dsetparm.c`'s `D_STOP` and `nsetparm.c`'s `N_STOP` each reset the start frequency field when it was the stop frequency that was refused. Both return `E_PARMVAL` before either field is read, so nothing observable followed -- but the line said the opposite of what it did.

Deliberately unchanged: every meaningful disto ratio -- 0.001 to 0.999 AND above 1, including E-255's 1.3 -- and single-tone `disto`, plus the neighbouring points/start/stop checks; `setseed` with an integer, with surrounding blanks, and its existing messages; `s_xfer` with a correct `int_ic`, with none at all, and E-491's num>den guard firing first; a matching `cntl_array`/`freq_array` on all three oscillators; a single `.param`, two names on one card, a subcircuit's own parameters (three shapes), and the `.func`/`.model`/`.subckt` messages.

frontend/dotcards.c, **frontend/breakp2.c**, **frontend/outitf.c** + three headers — a plot keyword is not a signal name (E-496)

Reported from use, not from a hunt. With `.option saveused` in the deck,

```
pyplot v(a[0]) xlabel 'something'
```

printed *"save 'xlabel': nothing of that name is in this analysis, so no such vector is produced"*. `xlabel` is the plot command's own grammar; the author never asked for a vector of that name, and the message tells them one is missing.

WHAT WAS WRONG. E-469's `.option saveused` reads the control block before it runs and saves every vector it believes is mentioned there. Its bare-word scan took EVERY argument of an output command that was not a number, a redirection or an expression, with no knowledge of those commands' own keywords:

written	collected as vector names
`plot v(a) xlabel 'x' ylabel 'y' title 'T'`	`xlabel`, `ylabel`, `title`
`plot v(a) vs v(b)`	`vs`
`meas tran m1 FIND v(b) AT 50u`	`tran`, `m1`, `find`, `at`

All 22 plot keywords, `hardcopy` likewise, and `meas` worst of all, since it names its analysis, its result and its function as bare words.

THE ANSWER WAS NEVER AFFECTED. E-469 over-collects deliberately -- its own comment says under-saving would cost the answer -- and a save matching nothing produces nothing: the vector the author asked for comes back bit-identical with zero or three stray keywords collected beside it. The names were gathered in SILENCE until E-493 added the unmatched-save warning, which exposed this flaw rather than causing it.

THE FIX IS IN TWO PARTS, AND THE ORDER OF IMPORTANCE IS THE REVERSE OF THE OBVIOUS ONE.

(1) An INFERRED save is never reported. E-493's warning exists to catch a name the AUTHOR wrote and got wrong -- `.save v(nosuch)`. A name `saveused` guessed on their behalf is not that. Registration now records which it is (`db_auto` on `struct dbcomm`, carried into `save_info.autosaved`), and the warning skips the inferred ones. The flag is set only around `ft_saveused()`'s own `com_save()` call, so `.save`, `save`, `.probe` and `savecurrents` are untouched and still report an unmatched name exactly as before. This single change covers every keyword of every command, including any that part 2 fails to enumerate.

(2) The grammar is skipped by the scan. The plot family (`plot`, `pyplot`, `gnuplot`, `hardcopy`) now knows its own keywords, and `meas`/`measure` contribute no bare words at all -- the vectors they read arrive as `v(...)`/`i(...)`, which the reference scan already takes from every line whatever command it belongs to. This stops the pointless work, but on its own it would be the E-487 trap: a hand-maintained list that silently rots as keywords are added, and whose failures are invisible. It mirrors `CT_PLOTKEYWORDS` in `cpitf.c`, which cannot be reused because that table is built for tab completion and populated only in an interactive session. A keyword missing from the copy costs nothing but the old noise for that one word -- which is exactly why part 1, not part 2, answers the report.

Deliberately unchanged: everything about WHAT IS SAVED, pinned by comparing every value against the same run without the option -- a plain plot, a plot carrying keywords, `plot ... vs ...`, `meas` then plot, and `let r = ...` then `plot r` are identical either way; `all` still means everything; `wrdata`'s leading file name is still not a vector; an explicit `.save` still makes `saveused` stand aside entirely; a node GENUINELY NAMED `title` or `vs` keeps the value it has without the option, so the keyword skip

cannot cost a real vector; and `.save v(nosuch)`, `.probe v(nosuch)` and `.save v(nosuch)` beside `.option saveused` all still warn.

`parser/inpgval.c`, **`parser/inpgmod.c`** — a value nothing checked had been read (E-507)

`INPgetValue`'s `IF_REALVEC` path tests `INPevaluate`'s `error` and refuses the value. Its `IF_REAL` and `IF_INTEGER` paths took the same `error` and threw it away, so a token the parser cannot read became 0 and was applied as if the deck had asked for zero.

numparam makes that ordinary rather than exotic: it substitutes the text of a `{...}` expression, and `{1/0}` substitutes `inf`. `.model nm nmos level=1 vto=0.5 kp={1/0}` therefore built a transistor with `kp = 0`, conducting 1e-12 against 1.25e-4 — exit code 0, no diagnostic. The same value written `inf` on that card is refused, and so is `{1/0}` on an instance line, as a built-in device's value, in an analysis argument and through `altermod`: the model card was one path out of six, and the conversion to zero happened *before* the range check, so a `from` clause could not catch it either.

`INPgetValue` cannot simply return NULL for these types the way its vector path does — none of its call sites test for NULL, so that would trade a wrong number for a null dereference. The failure is recorded (`INPlastValueError`) and the model-card path asks for it, that being the one caller with no other way to find out. It reports and keeps the parameter's default, which is what the surrounding `.model` conventions already do for an unknown or duplicated parameter.

`parser/inpptree.c` — an ordinary leaf was not in the release switch (E-507)

`PT_PLACEHOLDER` is node type 0 and a normal leaf (`inpptree.h`: *"for i(something)"*), but it was missing from the expression-release switch, so freeing one fell to the `default` arm and printed *"Internal error: unhandled parse-tree node type 0 while releasing an expression"*. That reached users on an error path, where a partially built tree is released — `B1 a 0 v={z}` with `.param z={1/0}` — announcing an internal fault ahead of the real, correct diagnostic. Like every other leaf it has no child to release.

`frontend/outitf.c` — **`save`** called an operating-point variable a missing parameter (E-507)

`INPaName` both finds a name and asks the device for its value, and any failure of the second half arrived here as the first half's message. An operating-point variable is registered `IF_ASK` like any other askable parameter, so the name resolves — but at `save` time no analysis has run, so the ask fails and the user was told *"device has no parameter 'gv'"* about a name the device does have and that `print` and `meas` both resolve. The netlist `.save` form reported the same case correctly, so the two spellings of one request disagreed. Only `E_BADPARM` produces that message now.

The compiler-side halves of this enhancement are in the openvaf-r report.

`parser/inpgmod.c`, **`analysis/dctrcurv.c`**, **`numparam/xpressn.c`**, **`frontend/rawfile.c`** — seven ways a decision made once was never revisited (E-495)

Round 55 probed MOSFET model binning, which no earlier round had touched, and found it soft in three ways. The shape it exposed -- a decision taken once and never asked again -- then led into the DC sweep, which had the same defect twice more.

THE BINNING TOLERANCE WAS ABSOLUTE, AND THE VALUES ARE METRES. `is_equal()` tested `fabs(a - b) < 1e-9`, which on a channel length is a slop of one nanometre applied whatever the geometry. A device up to 1 nm outside EVERY declared bin was silently placed in one -- `l=31n` bound to a bin reaching 30n, while `31.1n` was refused, so the edge really was the fixed constant. Because the magnitude does not scale, as a FRACTION of the device it grows without bound as processes shrink: 0.03% of a 3 um width, but 5% of a 20 nm channel. The comparison is now relative (1e-12).

ADJACENT BINS OVERLAPPED, AND **.model** ORDER DECIDED. `in_range()` states its rule in its own comment as `min <= value < max`, and then also accepted `is_equal(value, max)`, closing the interval so a device on a shared boundary matched both neighbours. Reversing two `.model` cards moved `l=1.999u` -- a length unambiguously inside the LOWER bin -- into the upper one and changed i(V1) by 2.95x on an otherwise identical deck. Selection now runs the strict half-open rule first and the closed reading only where the strict rule matched nothing; that second pass is what still admits a device sitting exactly on the TOP bin's `lmax`, and because it never runs when the strict rule found a bin, nothing that selects a bin today can move.

AN OSDI MODEL COULD NOT BE BINNED AT ALL. The binnable set was eleven hardcoded built-in names, so a Verilog-A model written exactly as a BSIM PDK writes one died with *"Unable to find definition of model nv"* -- for a model defined twice, the symptom rather than the cause, and the same shape as the resistor named `r` in E-493. OSDI types are now asked through the predicate E-323 already provides, and the four bin limits are consumed on the card the way `level` is.

A DEGENERATE DISTRIBUTION SPEC WAS SILENTLY FLATTENED. `agauss`/`gauss` are right to refuse a variation or sigma that is zero or negative, but did it without a word, and the result does not look like a failure: every draw returns the nominal, so a Monte Carlo over a parameter that never moves reports a yield of 100% where the real spec gave 12% -- `agauss(1000,100,3)` against `agauss(1000, 100,0)`, one character apart, silent under every option including `warn_physics`. Now audible once per run per function; the behaviour is unchanged because a deck may legitimately zero a variation. `unif`, `aunif` and `limit` are deliberately untouched -- a negative variation there describes the same symmetric interval as its absolute value.

A **.dc** SWEEP NEVER REVISITED WHAT SETUP DECIDED. E-471's own comment says `.dc` "sets the circuit up once and walks its points inside the analysis", and that a reused setup freezes the topology so "the sweep quietly draws a flat line". It gave the **sweep** command the machinery to notice and rebuild; `.dc` never got it. So a swept `l`/`w` leaving the bin the device was PARSED into kept the old bin and every point past the boundary used the wrong model (2.9x out), and a swept parameter changing an OSDI node collapse kept the matrix built for the old topology and returned a FLAT LINE -- -1.000e-03 at rs = 0, 500 and 1000, where the answers are -1e-3, -6.67e-4 and -5e-4.

Both siblings are correct: `alter` re-selects the bin through `if_set_binned_model()`, and `sweep` runs a whole job per point. Rebuilding a matrix mid-analysis is far wider than this evidence supports and the correct command already exists, so **.dc** now REFUSES the point it cannot compute and names `sweep` -- E-485's rule that a wrong answer is worse than a refusal. The detection is exact and free when nothing moves: the collapse is read from the flag `OSDItemp` ALREADY sets on every parameter write (E-417) and that `CKTdoJob` already consumes -- `DCTsetInstParam` calls `DEVtemperature` and simply never asked -- and the bin is checked by reading the four limits off the model the device is bound to. The refusal deliberately does NOT borrow E-427's message about a value "the device refused ... also refused by `alter`", which would be false on both counts.

THE ASCII RAWFILE WAS ONE DIGIT SHORT OF A DOUBLE. `#define DEFPREC 15` is the precision of a `%.*e`, so it emitted SIXTEEN significant digits where seventeen are needed before an IEEE754 double reads back as itself; `write` then `load` changed values in the last ulp while the BINARY format, which stores the bits, was exact. Over 200000 random doubles `%.15e` fails to round-trip 51390 of them and `%.16e` none. The comment calling 15 "(max)" was wrong twice.

Deliberately unchanged: bin selection that works today, in either card order, including the top bin's `lmax`; `alter` and `sweep`; `unif`/`aunif`/`limit` and a healthy `agauss`; a value the device really refuses; an ordinary source sweep, an `m` multiplier sweep, and a `.dc` sweep of a single UNBINNED model; the binary rawfile and an explicit `rawfileprec`.

parser/ptfuncs.c, **frontend/inpcom.c** — two ways a B source expression disagreed with every other value path (E-494)

Round 54 compared the B source expression path against the paths that read the same number on the same deck. It answered differently twice: once about the sign of a value, once about the value itself.

x/0 LOST ITS SIGN. E-491 removed a gmin-derived fudge factor from `PTdivide` and replaced it with a fixed epsilon, writing the nudge as `arg2 = (arg1 >= 0.0) ? PTDIV_EPS : -PTDIV_EPS;`. The intent was to keep the sign of the numerator; the effect was the opposite, because a negative numerator was then divided by a NEGATIVE epsilon and the quotient came out positive either way. `B0 b0 0 v=v(p)/0` with v(p) = -3 returned +3e+32 where it should return -3e+32, and the constant spellings `-1/0`, `(0-2)/0`, `-1.0/0.0` and `-1/(1-1)` were all positive too, as was `E0 b0 0 vol='-1/0'`. Every case E-491 measured had a positive numerator, which is why its own suite did not see this.

A divisor of exactly zero carries no sign to recover, so the real choice is which side zero is approached from, and it has to be made once rather than per operand. The epsilon is now unconditionally positive -- zero from above -- so the quotient keeps the sign of the numerator, which is what `lim(x/eps)` as `eps` tends to zero from above gives, and what the comment E-491 left in place already described. Everything else E-491 established is pinned unchanged: the magnitude of `1/0` is still 1e32, an ordinary non-zero divisor is still used exactly as written, and `1/1.38064852e-23` still does not move when gmin does.

NUMERIC LITERALS WERE ROUNDED TO 11 SIGNIFICANT DIGITS. Every B source line is rewritten before parsing by `inp_modify_exp()`, which re-emitted each numeric literal it found with `"%18.10e"`. So `B0 b0 0 v=1.2345678901234567` reached the parser as `1.2345678901e+00`, a relative error of 1.9e-11, while the *same literal* written on an `R`, `C` or `V` card, in a `.param`, or as an OSDI `.model` parameter kept all seventeen digits. The B source was the only path in the simulator that could not carry a double.

It was not merely a printout, because the rounding reached the arithmetic: `B0 b0 0 v=tan(1.5707963267948966)` returned -1.96e11 against libm's 1.633123935319537e+16. The tangent was right; its argument was not.

MORE DIGITS WOULD NOT HAVE FIXED IT. `INPevaluate()` reads this text back on the way in and accumulates the mantissa by hand as `mantis = 10 * mantis + digit`, which loses a bit once the accumulator passes 2^53. Emitting more digits than the value needs is therefore not free: a fixed `"%.17e"` turns the sixteen digits of `0.7853981633974483` into eighteen and brings it back 1 ulp out, when the user's own text would have survived intact. The new `inp_num_text()` emits the shortest decimal text that reads back as the same double, trying 15, 16 then 17 significant digits and keeping the first that round-trips. Seventeen digits always suffice for an IEEE754 double, so a finite value cannot fall out of the loop unmatched.

Deliberately not widened. `INPevaluate`'s own scaling still costs 1-2 ulp on a few values, but a `V` source shows it identically on the same literals, so the residue is shared by every value path and is a wider question than the one measured here; the suite pins parity and a 2 ulp bound rather than bit-exactness, against an 11-digit rounding that had put those same values 1.0e5-1.6e5 ulp out. And `mkb()` in `inpp-tree.c` folds a constant `/` with a raw division and no zero guard, bypassing `PTdivide` entirely, but probed from `B`, `E` and `G` sources in constant, parenthesised, nested and mixed forms it never produced an infinity -- every constant division still arrives through `PTdivide` with the corrected sign -- so the branch appears unreachable from user input and was left alone rather than fixed past the evidence.

Deliberately unchanged: the sign and magnitude of a positive `x/0` and its gmin independence; ordinary divisors (`2/4`, `-2/4`, `2/-4`, `-2/-4`, `0/5`, `1/1e-30`); the SPICE suffixes `100p`, `5MEG`, `1k`, a leading-dot `.5` and exponent forms; `pwl` B sources, which `inp_bsource_compat()` excludes from the rewrite; current B sources, node references mixed with literals, and the value surviving a transient; and `E`/`G` expressions, which never passed through `inp_modify_exp()` at all, since only `b` lines do, and were already exact.

parser/inp2n.c, **frontend/subckt.c** — mixing shorthand and written-out bits on a bus port (E-490)

`.option autobus` (Enhancement-444) lets one netlist token stand for a whole Verilog-A bus port, and it fires on a token count equal to the PORT count. Positional binding covers the other complete form, a count equal to the TERMINAL count. A line that leaves one port in shorthand while writing another port's bits out is NEITHER, so it fell through both and bound positionally against the flat terminal list. For `N1 a b[0:2] bmix` against `inout [0:4] a; inout [0:2] b`, the token `a` went to `a[0]`, `b[0]` to `a[1]`, `b[1]` to `a[2]` -- measured on a model whose eight bits each carry a different conductance, every node sat one or more terminals off and every current was wrong.

The only thing said was Enhancement-402's warning naming the terminals left over at the tail: the symptom, not the cause. A user who supplies the two nodes it asks for still has a circuit wired entirely wrong, and now with no warning at all. Enhancement-445's diagnostic -- "already carries an index, so it cannot be expanded as the bus port" -- exists for exactly this mistake, but sits INSIDE the port-count check. The guard and the failure were one `if` apart.

Nothing here is ambiguous, so the fix does not guess. It walks the ports left to right and lets each token declare which form it is in: a bare name on a bus port is shorthand for that port's bits; a token already carrying an index -- or ground, which E-445 established can never be indexed -- means that port was written out, so take one token per bit. `N1 a 0 0 0 bmix` reads correctly under the same rule. The rewrite is accepted only when the walk consumes EXACTLY the tokens the line has, and the walk is bounded by that count as well, because `gettok_instance` cannot tell where the node tokens end: without the bound, a port claiming more bits than the line has left would swallow the model name and report against it.

Where no reading exists, it refuses. If the counts do not reconcile then the bits written do not match the width the model declares, and nothing can repair that. The refusal happens where the port and both counts are still in hand to say so, naming the port, its width, the tokens written, and the two counts that would have worked. This replaces a wrong answer, not a working deck; warning and binding anyway is the detect-announce-then-use-it-regardless shape Enhancement-485 had to undo eight times in one round.

One rule, one reader. Deciding the mixed form needs to know whether a token already carries a bit index, and that is spelling-dependent: `a[0]` always, `a_0_` only while `.option autobus=kicad` is on. `subckt.c` already answered it for `.subckt` formals, and a second copy in `inp2n.c` would have been free to disagree about the KiCad spelling -- the two-readers-of-one-rule shape Enhancement-454 had to repair in this same option -- so the rule moved to `INPbusTokenIndexed`, beside `INPbusBitSuffix`, and `subckt.c` now delegates to it.

Two smaller fixes in the same option. `.option autobus=1` reported a style that does not exist: the on-word list held `true`, `yes` and `on` but not `1`, the mirror of the `0` in the off-word list. The comment on the early return explains how -- "bare flag, or `=1`: a NUMBER, not a string" -- which is true of `set autobus=1` but not of a deck `.option` card, which publishes a string. The feature worked; the message was simply wrong. And Enhancement-445's own message named a terminal where a port was meant, reporting a port declared `[0:2]` as "the bus port 'b[0]'", an index the user never wrote and could not act on.

Unchanged by design: with `.option autobus` off nothing moves, the whole path being gated on it; an all-shorthand line and an all-explicit line take the paths they always did; a short line with no shorthand token -- every bit written out, trailing terminals omitted -- keeps E-402's warning and the `$port_connected` idiom it protects; and E-445's two refusals still fire on a full-width line.

Measured and needing no fix, in the survey that preceded this change: ports of unequal width, descending `[4:0]` and non-zero-based `[1:5]` ranges, a 16-bit port, three buses of sizes 2/3/4, bus/scalar/bus interleaving, one token feeding two ports (which superposes exactly), uppercase tokens, `.option autobus` placed after the instance, two instances in parallel, `m=` multipliers, all four subcircuit shapes including reversed formals, `autobus=kicad` on unequal widths, and too-many-nodes as a hard error. A

`[0:0]` port connects correctly in both spellings; its node is named `a` rather than `a[0]`, which is inherent -- at a full token count a width-1 bus is indistinguishable from a scalar.

`frontend/com_sweep.c` — **`sweep temp`** swept nothing (E-488)

`sweep` resolves a knob as a `.model` parameter, an instance/device parameter, or a symbolic deck `.param`. The GLOBAL circuit temperature is none of those, so a bare `temp` fell through to the instance/device branch, `alter temp=...` found no such device, and the sweep ran to completion over a knob that never moved -- a full set of points, rc = 0, and a perfectly plottable FLAT curve, which reads as "temperature has no effect on this circuit". Enhancement-431 removed that shape for an unresolved `-output`, and Enhancement-435's own comment in this file describes it for subcircuit-local model names; `temp` was a third instance, and the one users are most likely to meet, because every OTHER route already worked (`.option temp=`, `set temp=`, `alter @dev[temp]=`, `sweep @#*[temp]`).

The obvious implementation does not work. Writing `ckt->CKTtemp` -- what `.dc temp` does -- left the curve flat, because CKTdoJob opens with `ckt->CKTtemp = task->TSKtemp`. `.dc` survives that only because its entire sweep runs INSIDE one CKTdoJob; `sweep` runs a fresh analysis command per point, so the write was discarded before the next point was solved. The TASK is what has to move, so the knob is applied by issuing `option temp=` -- how the frontend already moves it, and the same shape as the `alter`/`altermod` this file uses for every other knob. It also carries the value through the guarded OPT_TEMP funnel in cktsopt.c instead of around it, inheriting Enhancement-426's absolute-zero refusal and Enhancement-440's sanity check rather than copying them.

Deliberately NOT identical to **`dc temp`**. Over a node collapse that moves with temperature, `dc temp` returns 0.0 where a static `op` at the same temperature returns 0.5 -- it holds one setup for the whole sweep and never rebuilds (round-24, still open, NOT addressed here). `sweep` runs a fresh analysis per point and Enhancement-471's logic rebuilds where the collapse moves, so it is right. The suite pins all three series against the static `op` and asserts that `dc temp` DISAGREES, so a later pass that "fixes" sweep into agreement fails rather than quietly adopting the wrong answer.

Strictly additive: a deck `.param temp` is tested FIRST and still wins, so a deck defining and referencing its own `temp` sweeps exactly as before. Also matched to the oracle: an unphysical range is refused up front and sweeps nothing (left to the funnel alone, bad points are ignored one at a time and the axis still carries the value, so a row LABELLED -600 C held the answer for whatever temperature was in force), and the axis carries SV_TEMP so `plot` labels it as `dc temp`'s does.

Not fixed: sweep's other unresolvable knobs -- a bogus name, a missing device, a missing parameter -- still warn and then sweep flat. Only `temp` names something that exists.

`analysis/dcpss.c`, **`com_hbosc.c`**, **`com_hb.c`**, **`typesdef.c`** — RF results that were printed and then lost (E-487)

An RF result that is only PRINTED cannot be plotted, printed again, `wrdata`'d, diffed against a reference, or read by a script. Six of the eight RF entry points published a nutmeg plot; `hbosc` and `phasenoise` printed a table and stored nothing, leaving the session with hbosc's own startup transient as the current plot (`hbosc` runs a `tran` internally to seed the oscillation, so `setplot` after a successful run reported `Current tran1`).

One signature extended, its sibling not. Enhancement-209 added `struct hbspectrum *out` to `HBanalyze` so `com_hb` could publish its spectrum as vectors. `HBOSCanalyze` computes the same `(2K+1)*N` two-sided spectrum in the same layout and never got the parameter — the same shape E-486 spent its length on. It now takes it, on the same ownership contract, and `PhaseNoiseAnalyze` gains a matching `struct pnspectrum *out` that collects the curve beside the table it was already printing.

One publisher, not two. `com_hb`'s `hb_publish()` was `static`; copying it into `com_hbosc.c` is how the two drifted apart to begin with. It is now `hb_publish_spectrum()`, shared and parameterised, so the

driven and autonomous cases cannot diverge again. `hbosc` additionally publishes `oscfreq`, because for an autonomous circuit the frequency is part of the ANSWER rather than an input.

The plot NAME is where this gets subtle. `ft_plotabbrev()` matches by SUBSTRING, so `"hbosc"` hit the `hb` pattern and the oscillator spectrum came out named `hb1` — indistinguishable from a driven run in the same session — while `"phasenoise"` hit `noise` and collided with `.noise`. Both now have entries ahead of the pattern that shadows them, which is the rule E-383 wrote into that table. `plotorder_examples` asserts that rule for "every name in the tree", but its NAMES table is HAND-MAINTAINED and was passing over these two vacuously; it now knows about them, and mis-ordering the `hbosc` entry makes it fail with `hbosc->hb (want hbosc)`.

Types. `loadpull` created every vector `SV_NOTYPE` including two with a real type available; `pout_dbm` and `gain_db` are dB and now carry `SV_DB`. `gamma_re`/`gamma_im`, `pae`, `eff` and `stb`'s `loopgain` are deliberately left untyped — they are dimensionless ratios and the enum has no member for that.

Not changed: `loadpull` leaves its intermediate transients in the session, 61 plots for `-n 9` and 333 for `-n 21`. That is a real problem, but discarding plots a user may want to inspect is a behaviour change on a shipped command, so it is recorded rather than made silently.

`analysis/cktsens.c`, **`xspice/enh/enhtrans.c`**, **`xspice/mif/mifmpara.c`**, eleven **`icm/`** code models — a guard one sibling had and the other did not (E-486)

Twelve sites of one shape: the check exists somewhere in the tree, and is simply absent next door. Often the sibling is in the same directory; once it is ten lines away in the same function; once it is the same file's own history.

`table2D` segfaulted on a file that declares its own size. The data-row loop is driven by the FILE — `while (*cThisPtr)` — and wrote `table_data[lLineCount - 1]` with no upper bound, while every *other* dimension of that same file was checked: the x row, the y row, the width of each individual data row, even a premature EOF inside the comment block. A file declaring 3 y values and supplying 5 data rows indexed past the allocation and crashed — `EXC_BAD_ACCESS` at address 0 in `cm_table2D` — rc 139, no diagnostic, from a twelve-line table file. Too FEW rows was the mirror: the shortfall stayed as `calloc`'s zeros, the table went to `sf_eno2_set` as though complete, and a probe in the missing region returned `0.0`, a physically plausible "no current". E-247 had already worked in this exact file ("fix OOB READ + interpolation UB on degenerate/too-small tables"), addressing the read side and the too-small case and leaving the WRITE and the too-many case. The sibling `table3D` refuses the truncated file outright, so the two disagreed about one input and only one of them crashed. Both directions are now bounded, and `table3D` — bounded by its `iz`/`iy` loops, hence never crashing — now warns about the surplus it had been dropping in silence.

`count_steps()` signalled an error with a value used as a count. It is declared to return a POINT COUNT, and E-362's overflow guard signalled failure with `return(E_PARMVAL)`. `E_PARMVAL` is 11, and the sole caller assigned the result straight to `nfreqs` with no error test, so an "impossible" sweep ran exactly eleven points — and because that return happened before `*stepsize = s`, the step was never written and every frequency after the first collapsed to zero. Beneath it sat two silent repairs of values the USER STATED: a DEC/OCT sweep starting at 0 Hz had `low` rewritten to `1e-3`, and a stop at or below the start had `high` rewritten to a decade above it. Both rewrote only the LOCAL copy, so the count came from the repaired bounds while the sweep still ran from `job->start_freq` — which is why `.sens ac dec 5 0 1meg` printed a full table of 0 Hz rows and `.sens ac dec 5 1k 1k` swept a full decade past the stop asked for (6 rows where `.ac` gives 1). The arithmetic predicts each observed count exactly: 6 for `dec`, 3 for `oct`, 5 for `lin`. `.ac`, `.noise`, `.disto` and `.sp` all get both cases right, so the rules are now `.ac`'s own (`acan.c`); errors leave the function out of band in an `int *errp`, and `*stepsize` is written on every path.

Ten more of the same shape. `d_state`'s `cm_set_indices()` presets its indices to 0 and returned FALSE — "no error" — when NOTHING matched, so a transition naming a state the file never defines ran ROW

0 silently (proved by construction: row 0 of `0s 0s` holds the outputs at 0, row 0 of `1z 1z` holds them at 1). `file_source`, the file-driven sibling of `pwl`, had no monotonicity check at all, so a time column that steps backwards, repeats, or starts below zero silently produced a different waveform. `xfer`'s `table` path refuses a negative or out-of-order frequency while its `file` path, ten lines below in the same function, validated nothing. `hyst`'s `input_domain` was unbounded where `pwl`/`pwlts` declare `[1e-12 0.5]`, letting an `input_domain` of `1e6` drive the output to 250000 against declared limits of 0 and

1. `core` called `PARAM_SIZE` exactly once, on `H_array`, and indexed `B` with it. `mlin`, `cpline` and `cpmlin` had no limits on any geometry — a negative microstrip length returned 0.5011 against 0.4992 for the same line at +1e-2, a plausible answer from a geometry that cannot exist. `poly()` reported each of its two faults with the OTHER one's message. And `d_state`, `d_process`, `real_gain` and `oneshot` were the only models of twenty-plus with no declared limit on their delays.

Deliberately unchanged, with the built-in devices as the oracle. NEGATIVE capacitance and inductance are not defects: the built-in `C` accepts `-1u` and produces exactly the sign-inverted response the XSPICE model produces, and the built-in `L` diverges the same way (7.5e+288 against 1.8e+285, both ending in the same timestep abort). Only `c/l` of ZERO is fixed — a real division by zero that had surfaced as "Timestep too small; cause unrecorded". `mlin`'s `rho = 0` was first classified as an ordinary perfect-conductor idealisation; measuring it returned a bare NaN, so it is strictly positive after all, while `t`, `tand` and `d` at zero stay legal. `xfer`'s duplicate frequencies stay legal because the `table` path allows them — the fix gives the file path *that* rule, not a stricter one. `oneshot` is bounded at `[0 -]` rather than the digital `[1e-12 -]`: it is an analog model, only a negative delay was measured to do harm, and a zero rise delay works.

`git -S` puts both headline defects at `4f29ffad`, the vanilla upstream import — pre-existing upstream defects, not regressions from this tree.

device noise routines — a nominal temperature added to a difference (E-403)

`res/resnoise.c`, `bjt/bjtnoise.c`, `dio/dionoise.c`, `vbic/vbicnoise.c`, `hicum2/hicum2noise.c` all computed the thermal-noise temperature delta as

```
dtemp = inst->REStemp - ckt->CKTtemp + (model->REStnom - CONSTCtoK);
```

`NevalSrcInstanceTemp` evaluates `4*k*(CKTtemp + param2)*G`, so `param2` is a DELTA. The third term adds the NOMINAL temperature expressed in CELSIUS (27 by default) to that difference, so an instance sitting at ambient got +27 K: `rd x 0 1k temp=27` read 4.4% more noise voltage (9% power) than the identical `rd x 0 1k`. Instance temperatures are stored in Kelvin, so the first two terms were already the whole answer; the third is dropped. `temp=100` and `dtemp=73` -- the same temperature written two ways -- now agree exactly, at the analytic ratio.

Surfaced as `sens` poisoning every later `noise` in a session: `sens` restores parameter VALUES but leaves the `...tempGiven` flag set, which flips the noise routine into the branch above. Correcting the arithmetic fixes both, since an instance at ambient then contributes a zero delta whether or not the flag is set.

`devices/dev.c` (51)

- Duplicate device-type registration warned and skipped: `osdi_add_device()` appended every descriptor unconditionally, and the model-card lookup scans front-to-back — so a module name duplicated across two loaded libraries silently resolved to the *first* registration (loading an updated model library gave the stale device with no hint), and a module named like a built-in corrupted model creation. Registration now checks the table case-insensitively and skips duplicates loudly (E-76).
- **`pre_osdi`** path dedup: loading an already-loaded file notes it and skips (E-76, E-81).

- **`pre_osdi -f`** force reload (E-229): the three dedup gates (path, `dlopen` handle, device-name) plus `dlopen`'s own path cache meant a recompiled `.osdi` never took effect without restarting ngspice. A `-f`/`--force` flag (parsed in `com_osdi`, `frontend/com_dl.c`) now force-reloads: `load_osdi( path, force)` stages a byte-for-byte copy of the recompiled file under a fresh unique path and loads *that* (a new path is always re-read, sidestepping the cache; no `dlclose`, which on macOS does not reliably unmap and where overwriting a mapped file faults the process), `removes` the staged copy once mapped, and `osdi_add_device(…, replace)` swaps the registered device to the new descriptor in place at its existing table index. The old `SPICEdev` + mapping are left resident so a circuit still built against the prior model stays valid. Without `-f` the re-load is still skipped (now with a hint pointing at `-f`).

`osdi/osdiparam.c` (E-93)

Setting a fixed (**`localparam`**) OSDI parameter from the netlist used to be swallowed silently: openvaf exports every `localparam` as a parameter, its "given" flag is forced false, and the written value is overwritten by the parameter-init default — so `.model … N=8` on a frozen structural width parameter changed nothing, with no feedback. `OSDIparam`/`OSDImParam` now test the new `PARA_FLAG_FIXED` descriptor flag (set by openvaf, a free bit of the parameter `flags` field) and emit *"parameter 'N' is a fixed (localparam) value and cannot be set from the netlist; ignored"* instead of silently dropping it (E-93).

---

6. Maths — `src/maths/`

`dense/dense.c` (11 substantive lines; the file also carries a whitespace/line-ending normalization from editing)

`cadjoint()` had no 1×1 base case: its cofactor loop allocated 0×0 and then negative-sized minors, killing ngspice with `malloc: can't allocate -8 bytes` — the crash that had been hiding behind the "at least two ports" guard in `span.c`. The adjugate of `[a]` is `[1]` (the empty minor's determinant is 1 by convention), making `cinverse` of a 1×1 equal `1/a`; a 100 Ω load on a 50 Ω port now writes a proper `.s1p` with S11 = 1/3 exactly (E-64).

`sparse/spfactor.c` (6)

The Markowitz-product overflow guards compare a `double` against `LARGEST_LONG_INTEGER` (= `LONG_MAX`), which is not exactly representable as a double; the conversion is now explicit — identical semantics, intentional and warning-free (E-77).

`KLU/klu_multiply.c` (16)

`SMPmultiply`'s KLU path passed `NULL` internal↔external ordering maps to `KLU_matrix_vector_multiply`, which dereferenced them unconditionally (`&NULL[n]` = `0x14` for `n = 5`) and segfaulted — so `.option linesearch` crashed under `.option klu`. The line search is the only caller of `SMPmultiply` under KLU, so this path had never been exercised. A NULL map now means the identity ordering (`ext = i + 1`, since the KLU CSR is 0-based and the RHS/Solution vectors are 1-based), making the KLU matrix-vector product — and hence the E-111 line-search residual merit — correct under KLU, with a merit sequence numerically identical to Sparse 1.3 (E-112).

`KLU/klusmp.c` (SMPcaSolve)

`SMPcaSolve` is the complex adjoint (transposed) solve used by noise and S-parameters. Its Sparse branch calls `spSolveTransposed`, but the KLU branch called the *non-transposed* `klu_z_solve` — silently wrong for any asymmetric matrix (every circuit with a transistor or controlled source), which is why `.noise` was disabled under KLU. It now calls `klu_z_tsolve` (transposed), so KLU noise matches

Sparse exactly (including OSDI device models). This is the core of E-113, which also removes the KLU refusal guards in `noisean.c` / `pzan.c` (single-ended pole-zero runs under KLU). Balanced-output pole-zero was Sparse-only at first but E-172 closed that too (a KLU `SMPcAddCol` branch plus a union-pattern reservation in `CKTpzSetup`), so no analysis is Sparse-only under KLU any more.

`spicelib/analysis/cktsens.c`

Sensitivity builds an auxiliary perturbation matrix `delta_Y` (holding $\partial Y/\partial p$) via `SMPnewMatrix`, which allocates a plain Sparse 1.3 matrix (`delta_Y->CKTkluMODE = 0`, no `SMPkluMatrix`). It is only *multiplied* against the solution (`SMPmultiply` → `spMultiply`), never factored, and `DEVbindCSC` always binds into `ckt->CKTmatrix`, not `delta_Y` — so it is correctly Sparse in every case. But two KLU-only setup blocks were gated on the main matrix's `CKTkluMODE`, so under `.option klu` they ran and dereferenced the NULL `delta_Y->SMPkluMatrix` — a segfault on every DC/AC `.sens` deck. The blocks now gate on `delta_Y->CKTkluMODE` (its own flag, `0`), keeping `delta_Y` Sparse under KLU exactly as it already is under the default solver, while the main `Y` matrix stays KLU. DC and AC sensitivity now match Sparse exactly (E-114).

A second, independent defect: the analysis altered the circuit it measured. Each derivative is taken by perturbing a parameter, reloading, and writing the original value back — which restores the *number* but not the model's given state, since every device setter marks its parameter as supplied and no device API offers an un-set. For a model whose behaviour is selected by whether a parameter was given rather than by its value, the model is left reinterpreted. The BJT is the sharp case: `ibe`/`ibc` default to 0 and ungiven, and `bjttemp.c` keys off `BJTBEsatCurGiven && BJTBCsatCurGiven` — ungiven, the junction saturation currents fall back to `is`; given, they are taken literally, and `sens` had just made them given with the value 0. `BJTBEtSatCur` became 0, a transistor with no saturation current at all, and every later analysis in the session solved the dead device: 12.8% wrong on a single stage, 101% on a differential pair, with no diagnostic and cured only by `reset`. It was not a tolerance artifact (bit-identical at reltol 1e-3 and 1e-12), not a stale initial guess (`.nodeset` did not change it, and `sens` run first corrupted just the same), and of twenty commands driven against the same probe — `alter`, `altermod`, `sweep`, `montecarlo`, `optimize`, `tf`, `pz`, `disto`, `noise`, `hb`, `pss`, `sp` — `sens` was the only one that did it. `sens_save_models` / `sens_restore_models` now snapshot every model struct before the perturbation loop (device-agnostically, via `DEVmodSize` and `ckt->CKThead[]`), restore them after it and on the error paths, and re-run `CKTtemp()` so each instance's derived values are rebuilt from the restored models (E-440).

`spicelib/analysis/distoan.c`

Distortion is a complex analysis (Volterra series solved at the harmonic / intermodulation frequencies via `NIdIter` → `SMPcSolve`), but `distoan.c` had no KLU code at all: under `.option klu` the KLU matrix stayed in *real* mode, the complex solves ran against an unconverted matrix, and every harmonic came back zero — a silent wrong answer. The fix mirrors `acan.c`: before the solve loop it converts the matrix to complex (`DEVbindCSCComplex` per device + `KLUmatrixIsComplex`), and on exit converts back to real (`DEVbindCSCComplexToReal`) so a later real analysis in the same session is unaffected. KLU distortion now matches Sparse bit-for-bit (single-tone, two-tone intermodulation, and OSDI models) (E-115).

`misc/equality.c` (`AlmostEqualUlps`)

`AlmostEqualUlps` is ngspice's ULP-distance float comparison — the primitive behind breakpoint matching, `.measure` triggers, transient truncation, PSS, and source timing (18 files). It reinterprets each `double` as an `int64_t`, maps the sign, and compared them with `intDiff = llabs(aInt - bInt)`. That signed subtraction overflows **`int64_t`** when the two keys straddle zero, and `llabs(INT64_MIN)` is itself undefined — surfaced by a UBSan build. It is not merely theoretical UB: `AlmostEqualUlps(-2.0, +2.0)` returned TRUE (declaring −2 and +2 "almost equal"), because those two keys differ by exactly `INT64_MIN`, whose `llabs` stays negative and slips under `maxUlps`. The difference is now taken in **`uint64_t`** as larger-minus-smaller, where the wraparound is well defined and the true magnitude

of any two-`int64_t` difference always fits; the comparison against `maxUlps` is unsigned. Behavior is identical wherever the old code did not overflow, and correct (`FALSE`) where it did. Found by the 2026-07 ASan/UBSan pass.

---

7. Build system

`xspice/icm/makedefs.in` (4)

The XSPICE codemodel (`.cm`) link rule hardcoded the pre-macOS-10.3 idiom `-bundle -flat_namespace -undefined suppress`, deprecated loudly by the modern linker on all 14 codemodel links (CI macOS builds included). Now `-bundle -undefined dynamic_lookup` — the modern spelling for plugins whose host symbols resolve at load time (E-77).

Autotools regeneration (the ~100 **`Makefile.in`** files + **`config.h.in`**)

Regenerated by `./autogen.sh` with libtool 2.5.4 during the zero-warning work — a build tree whose `configure` predates libtool 2.4.7 selects the deprecated darwin `-undefined suppress` flag whenever `MACOSX_DEPLOYMENT_TARGET` is unset. No hand-written content (E-77).

---

8. Per-enhancement index

Every enhancement that touched ngspice, oldest first:

E-1 *osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c, osdisetup.c, dctran.c (+accept path)*

`absdelay()`: synthetic delay rows + waveform history

E-6 *osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c*

`last_crossing()`: history + interpolated crossings

E-8 *parser/inpgtok.c*

`.model` card first-parameter drop fix

E-24 *osditrunc.c*

`\$discontinuity` next-step clamp

E-25 *osdi callbacks (get_simparams)*

expose `analysis_name` + `simulator` simparams

E-32 *outitf.c*

integer opvars recorded per point

E-42 *osdinoise.c*

coherent same-named noise grouping (LRM 4.6.4)

E-45 *osdi.h (ABI 0.5), osdisetup.c*

Verilog-A nodesets applied to the solver

E-51 *osdi.h (ABI 0.6), osdiacld.c*

full `ac_stim` AC-RHS injection

E-53 *dctran.c, dcop.c, dctrcurv.c, acan.c, noisean.c, osdiload.c*

`@(final_step)` + analysis-name phase bits

E-54 *osdi.h (ABI 0.7), osdinoise.c, osdiload.c, osdiacld.c, osdiregistry.c*

node-free complex noise factors, stride-2 pairs

E-55 *osdiaccept.c (new), osdiload.c, osditrunc.c, osdiinit.c, dctran.c, dctrcurv.c, osdiitf.h, Makefile.am*

`\$finish`/`\$stop`/`\$fatal` honored; `\$discontinuity` step rejection

E-56 *noisean.c, osdisetup.c*

noise SIGABRT fix; setup-rejection diagnostics

E-62 *dctrcurv.c, trcvdefs.h, cktdisto.c*

generic `.dc @inst[param]` sweeps; `.disto` warning

E-63 *span.c*

`donoise` NaN clamp (stock defect, found by parity)

E-64 *span.c, cktspdum.c, dense.c, postcoms.c/.h, commands.c*

Touchstone export, auto-`Rbase`, 1-port `.sp`, `cadjoint` 1×1

E-72 *postcoms.c/.h, commands.c*

Touchstone round 2: MA/DB/Y/Z/units + `rdsnp` reader

E-76 *dev.c, inpgmod.c*

duplicate-registration warning, load dedup, stock `.model` segfault fix

E-77 *osdidefs.h, display.c, outitf.c, spfactor.c, makedefs.in (+autotools regen)*

zero-warning build, 33 → 0

E-80 *osdiinit.c*

`dtemp` instance-parameter alias

E-81 *resource.c, dev.c*

memory-warning opt-out + once-per-excursion; reload-note hint

E-93 *osdi.h, osdiparam.c*

warn (not silently ignore) when a netlist sets a `PARA_FLAG_FIXED` (`localparam`) OSDI parameter

E-94 *plotting/pyplot.c (new), com_pyplot.c (new), plotit.c, commands.c*

new `pyplot` command — plot vectors with matplotlib (a Python `gnuplot`)

E-95 *com_pyplot.c, commands.c*

make the `pyplot` output file name optional (defaults to `pyplot`)

E-98 *plotting/pyplot.c*

`pyplot` stacked subplots (`pyplot_subplots=N`) + matplotlib style sheets (`pyplot_style`)

E-99 *plotting/pyplot.c*

`pyplot` vector export formats (`pyplot_terminal=svg/pdf`) + figure size (`pyplot_figsize`)

E-110 *optdefs.h, tskdefs.h, cktsopt.c, cktntask.c*

`.option errpreset=conservative|moderate|liberal` — one knob for a coordinated tolerance/robustness set; explicit options override regardless of order (`moderate` = historical defaults)

E-111 *optdefs.h, tskdefs.h, cktdefs.h, cktsopt.c, cktdojob.c, cktntask.c, cktdest.c, niiter.c*

`.option linesearch` — globalized (damped) Newton via Armijo backtracking on a new KCL-residual merit ‖F‖ = ‖G·x−b‖ ; result-neutral, off by default

E-112 *maths/KLU/klu_multiply.c*

KLU support for `.option linesearch` — `SMPmultiply`'s KLU path passed NULL ordering maps that `klu_matrix_vector_multiply` dereferenced (SIGSEGV); NULL now means identity ordering. Line search verified merit-identical KLU vs Sparse

E-113 *maths/KLU/klusmp.c, spicelib/analysis/noisean.c, pzan.c*

KLU support for noise + single-ended pole-zero — `SMPcaSolve`'s adjoint KLU branch used the non-transposed solve (silently wrong noise on asymmetric matrices); now `klu_z_tsolve`. Guards removed; balanced-output pz keeps a targeted guard

E-114 *spicelib/analysis/cktsens.c*

KLU support for sensitivity — the auxiliary perturbation matrix `delta_Y` is Sparse, but two KLU setup blocks gated on the *main* matrix's flag dereferenced its NULL `SMPkluMatrix` (segfault on every DC/AC `.sens`); now gated on `delta_Y`'s own flag. DC/AC `.sens` match Sparse exactly

E-115 *spicelib/analysis/distoan.c*

KLU support for distortion — `.disto` is a complex analysis but `distoan.c` had no KLU code, so under KLU the matrix stayed real and every harmonic came back zero (silent wrong answer); now converts real↔complex around the solve loop like `acan.c`. Matches Sparse bit-for-bit

E-116 *osdi/osdisetup.c*

KLU wrong-DC fix — an OSDI internal node with no Jacobian entry (a decoupled `ground` reference) was given an all-zero solver row that made the KLU matrix structurally singular; now tied to ground. Fixes the `groundcontrib` and `hierbranch` KLU_XFAILs

E-117 *configure.ac (+`configure`), spicelib/analysis/dcpss.c*

Periodic steady state (PSS) productionized — built by default (was experimental `--enable-pss`, so `.pss` was unimplemented in shipped builds); ~230 lines of shooting-loop trace routed through `set ngdebug` (232 → 31 lines by default); fail-fast KLU guard (PSS hangs under KLU) directing `.pss` to `.option sparse`

E-118 *spicelib/analysis/dcpss.c*

PSS runs under KLU — the shooting loop hung under KLU (`klu_refactor`'s reused pivots inflated the truncation error into a ~20M-step timestep explosion); forcing a full re-factor (`NISHOULDREORDER`) each PSS step makes KLU converge to the same result as Sparse. E-117's Sparse-only guard removed

E-119 *pssdefs.h, spicelib/analysis/dcpss.c*

Retain the PSS periodic operating point — PSS sampled the node voltages per period for its DFT then freed them, and never captured device states; now the voltages and `CKTstate0` states are captured per sample and retained on the `PSSan` job (with frequency + dims) as the substrate for the periodic small-signal suite (PAC/pnoise/PXF). Self-checks that the retained samples reproduce the fundamental

E-120 *spicelib/analysis/dcpss.c*

Periodic small-signal Jacobian harmonics — walk the retained operating point, re-linearize each sample (CKTload MODEINITSMSIG) and stamp G+jC (CKTacLoad ω=1), read the osc-node diagonal → g(t), c(t), DFT to harmonics. The blocks the PAC conversion matrix is built from. (Fixed a complex-mode-not-set bug where C(t) accumulated across samples.) Verified: RC gives G=1/R1, C=C1, no harmonics

E-121 *spicelib/analysis/dcpss.c*

PAC conversion-matrix engine — extend E-120 from the osc diagonal to every matrix nonzero, complex-DFT to harmonics G_k, C_k, assemble the (2M+1)N harmonic conversion matrix H_{nm}=G_{n−m}+jω_m·C_{n−m}, inject a unit current at the osc node in sideband 0 and solve by dense complex LU (pss_csolve), report the sideband conversion gains. The numerical heart of PAC/pnoise/PXF. Verified: linear RC gives sideband-0 = AC driving-point |Z|(f0/2) = 303.3 Ω with the ±1 sidebands at floating-point zero (no spurious conversion)

E-122 *include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c*

.pac command (periodic AC) — the user-facing analysis on the E-121 engine. .pac Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff <dec|oct|lin> Npts Fstart Fstop runs PSS then sweeps the input frequency, solving the conversion matrix at each point and emitting the 0-th-sideband node responses as a complex PAC Analysis plot (print/plot/wrdata). Reuses the PSS analysis via a PSSdoPAC flag; engine factored into pac_extract_harmonics (once) + pac_solve_at (per freq). Verified: swept sideband-0 |b(f)| == analytic AC driving-point |Z(f)| to 1.6e−7 across 10 kHz–1 MHz

E-123 *include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c*

finish .pac — (1) source-referenced stimulus: capture the netlist AC-source RHS from CKTacLoad as the sideband-0 stimulus B_0 (a periodic-AC transfer/conversion gain), unit-current-at-osc-node fallback; (2) multi-sideband output: optional trailing maxsideband Ksb emits every conversion sideband f_in+k·f0 (k=−Ksb..Ksb) as its own vector — sideband 0 keeps the node name, others <node>_usb<k>/<node>_lsb<k> via IFnewUid. Verified: RC with V1 AC 1 gives sideband-0 = low-pass transfer 1/√(1+(2πfRC)²) (0.998→0.157) with b_usb1/b_lsb1 ~0 (no conversion)

E-124 *include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c*

.pnoise command (periodic noise) — fold each device's noise through the conversion matrix. pac_solve_adjoint solves H^T Ψ = e_{out,0}; pnoise_sweep loads the sideband-k adjoint into CKTrhs/CKTirhs and calls the existing device noise routines (NevalSrc, OSDI load_noise) once per sideband, accumulating Σ_k S·|ΔΨ_k|² — reuses every device noise model via a minimal local NOISEAN context. .pnoise <pss> OutNode InSrc <dec|oct|lin> Np Fstart Fstop; emits onoise_spectrum/inoise_spectrum. Verified: linear RC reduces exactly to .noise (4kTR/(1+(2πfRC)²), matches the .noise reference to every digit)

E-125 *include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c*

.pxf command (periodic transfer function) — the adjoint of PAC, completing the PSS→PAC→Pnoise→PXF suite. pxf_sweep solves H^T Ψ = e_{out,0} per frequency and dots each sideband block with the AC-source pattern B0 → xf_k=Σ_j Ψ_k(j)·B0(j), the input→output transfer at each sideband. .pxf <pss> OutNode <dec|oct|lin> Np Fstart Fstop [maxsb]; emits xf + xf_usb<k>/xf_lsb<k>. Verified: by the identity (H⁻¹B)_out=(H⁻ᵀe_out)ᵀB the sideband-0 transfer is bit-identical to the PAC response (0.998→0.157 low-pass), conversion sidebands ~2e−16

E-126 *include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c*

cyclostationary noise — `.pnoise ... cyclo` evaluates each device's noise PSD `S(t)` at every PSS sample's bias (`CKTload` per sample) and folds it through the time-domain adjoint transfer `A_s(j)=Σ_k Ψ_k(j)e^{j2πks/P}`, averaging: `onoise(f)=(1/P)Σ_s S(t_s)·|ΔA_s|²`. Reuses the device noise routines. Verified: (1) reduces exactly to `.noise` on the linear RC (bias-independent thermal → Parseval); (2) a flicker resistor carrying the RC current gives `onoise·f = R1²·KF·⟨I²⟩ = 4.88e−10` (uses the period-average `⟨I²⟩`, matched to 5 digits)

E-127 *include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktntask,cktdojob,cktop,cktdest}.c, maths/ni/niiter.c*

pseudo-transient continuation — `.option ptcont` embeds `f(x)=0` in a backward-Euler pseudo-transient `f(x)+Gps·(x−x_prev)=0` and marches `dτ` small→large (`Gps`→`0`). `Gps` diagonal via the gmin path; the `Gps·x_prev` RHS coupling (added in `NIiter`) makes each step a stable-trajectory move, not a static gmin step. New `pseudo_transient` in the `CKTop` cascade, off by default. Verified under KLU+Sparse: result-neutral on normal circuits; on a stiff no-limiting exp, reaches the correct DC `0.837922 V` where plain Newton lands on a spurious `70.5 V` (21/21)

E-128 *include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktntask,cktdojob}.c, spicelib/analysis/dctran.c*

LTE-based dynamic integration-order control — `.option dynorder` selects the Gear order per step from the local-truncation-error limit instead of the stock 1↔2 toggle, so orders 3–6 (already coded in `NIcomCof` but never used) are actually exercised. The selector compares the raw LTE-limited step (uncapped `CKTtrunc`) at the current order and its ±1 neighbours and moves with hysteresis (1.2×), a settling hold after each change (let the BDF divided differences rebuild), and an order-dependent step-growth cap (2× at order ≤3 → 1.3× at 6). Off by default; bounded by `maxord`; inert at the default `maxord=2`. Stiff-transient guards (post-breakpoint order hold + leaky-bucket rejection-rate order drop) keep it from collapsing the step on a violent slew. Verified under KLU+Sparse: RC decay 3–5× fewer steps at matched accuracy with monotone error; smooth RLC ringdown 8.9× fewer steps AND more accurate (0.13 % vs 0.34 %); nonlinear rectifier matches stock to 5 sig figs; the transistor-level µA741 ±5 V slew completes under dynorder and matches fixed Gear-2

E-129 *frontend/outitf.c*

sweep progress bar — the throttled `Reference value` status line (redrawn in place every 0.25 s during a sweep) now carries a live progress bar + percentage, e.g. `Reference value : 5.91926e−04 [================== ] 74%`. `outp_progress_frac()` computes the 0–1 fraction per analysis: transient from `(CKTtime−TSTART)/(TSTOP−TSTART)`, AC/noise from the frequency's linear/log position in the band, DC from the accepted-point count over the nested step-count product; analyses with no span (op, ...) keep the plain line. Fixed-width (no stale chars), stdout status-line only (never in the rawfile/`wrdata`), same `!ft_norefprint && !cp_background` gating. Verified 22/22: printed % matches the analytic sweep fraction to 0.5 % across tran/AC/DC/noise; op shows no bar

E-130 *frontend/com_optimize.{c,h}, commands.c, options.c, include/ngspice/fteext.h, spicelib/analysis/cktdojob.c, frontend/outitf.c*

built-in Nelder-Mead optimizer — `optimize −param <name> <init> <lo> <hi> [−param ...] −analysis <cmd> −minimize <expr> [−maxiter N] [−tol T] [−verbose]` varies device/`alter` parameters, re-runs an analysis, and minimizes a scalar objective expression via a derivative-free downhill simplex in normalized [0,1] space (scale-invariant across orders-of-magnitude params). Sub-commands dispatched synchronously through `cp_coms` (not the deferring re-entrant `cp_evloop`); the hundreds of inner analyses are silenced via a new `ft_`

optimizing flag gating the banner/row-count/E-129 progress bar at source (–verbose to show). Verified 9/9 against analytic optima: DC divider R1→2333.3 Ω, AC low-pass R1→2756.6 Ω, 2-D compound objective R1=3k/R2=2k exactly

E-131 *frontend/com_checkpoint.{c,h}, commands.c, com_commands.h, Makefile.am, spicelib/analysis/dctran.c, include/ngspice/cktdefs.h*

transient checkpoint / restart — new savestate <file> / loadstate <file> commands serialize the full transient integration state (solution vector CKTrhsOld, device state history CKTstates[], time/step/order/mode, pending breakpoints) to a binary file and resume it, including in a fresh process — so a long run survives a crash, splits across sessions, or moves between machines (stock ngspice could only resume in memory). DCtran() gains a CKTcheckpoint-gated branch that opens a fresh output plot (no live plot to 666-relink across a reload), initializes the XSPICE breakpoint markers, and fixes up the breakpoint list before jumping into the stepping loop; the rhs length is keyed off SMPmatSize+1 (not CKTmaxEqNum). A stored signature rejects a mismatched circuit; Sparse-solver only (KLU rejected with a clear message). Verified 19/19: resumed waveform bit-identical to an uninterrupted run for RC-step/pulse/diode built-ins and ~2e-7 for a compiled OSDI diode, across a separate process

E-132 *spicelib/analysis/dcpss.c, spicelib/parser/inp2dot.c, spicelib/analysis/psssetp.c, include/ngspice/pssdefs.h*

periodic S-parameters (.psp) — small-signal scattering parameters around a PSS operating point, including conversion between the input frequency and its sidebands f_in+k·f0 (mixers / switched circuits, where a static-DC .sp cannot see the conversion). Sits on the PSS→conversion-matrix suite (E-117–126): after PSS, psp_sweep excites each RF port (portnum/z0, the .sp framework) by driving its branch source (V=1, like .sp's VSRCspupdate) through the shared (2M+1)N conversion matrix, reads per-sideband port waves in the same Kurosawa power-wave convention, and forms S^(k)=B^(k)·A^−1 (dense-complex cinverse/cmultiply). pac_solve_at's matrix assembly factored into a reusable pac_build_matrix; PSSdoPSP flag + psp PSS param + dot_psp card. Because S=B·A^−1 is excitation-basis-invariant, sideband 0 reduces exactly to .sp for a time-invariant network. Runs under both linear solvers (the conversion matrix is a standalone dense LU; PSS runs under both since E-118). Verified 8/8: sideband-0 matches .sp to ~1e-16 for 1/2/3-port resistive + reactive networks (magnitude and phase) incl. OSDI Verilog-A devices (G + reactive ddt stamps), conversion sidebands correctly ~0

E-133 *frontend/com_qpss.{c,h}, commands.c, com_commands.h, Makefile.am*

quasi-periodic (two-tone) steady state (qpss) — qpss <expr> <f1> <f2> [periods] [max-order] computes the two-tone steady-state spectrum: every mixing product k1·f1+k2·f2 including third-order intermodulation (IM3 at 2f1-f2 / 2f2-f1) that single-tone AC/PSS cannot show. For commensurate tones (common beat fb=gcd(f1,f2)) it runs an ordinary transient over a few beat periods to reach steady state, then evaluates the Fourier coefficient directly at each exact intermod frequency (a direct DFT, exact — no FFT-bin rounding) and labels it by the 2-D index (k1,k2). Deliberately not a slow beat-frequency shooting PSS; a front-end command driving a transient, so solver-independent and works with built-in and OSDI devices. Verified 11/11: analytic fundamentals/IM3/3f of a cubic, no even-order products, the 3:1 IP3 slope law (fund ×2, IM3 ×8 per 2× drive), beat-frequency derivation, and an OSDI cubic matching the built-in

E-134 *spicelib/analysis/dcpss.c, frontend/com_hb.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h*

Harmonic Balance (hb <f0> <K>) — solves the periodic steady state in the frequency domain by Newton (each node a truncated Fourier series), instead of integrating in time; the real analysis behind ngspice's unimplemented WITH_HB stub. Residual F_k = I_R,k(V) + [dq/dt]_k − Is_k = 0 with the E-121 (2K+1)N conversion matrix as the exact Jacobian. Per iteration: inverse-DFT V→v(t_s); drive DC+AC device loads at those voltages for the resistive current I_R = G*v

– `b` (companion `b` saved before acLoad, so it's the ACTUAL current not the tangent `G*v`), and `G(t)/C(t)`; DFT; dense complex Newton (`pss_csolve`). Nonlinear reactive with NO charge extraction: `dq/dt = C(v)*v'`, so the reactive current is the conversion matrix's `jwC` term applied to V. Solver-independent (KLU + Sparse): the dense complex Newton is HB's own; the sparse solver only *reads* `G(t)/C(t)` off the device matrix, so `hb_extract` carries the same `#ifdef KLU` complex-CSC binding (`DEVbindCSCComplex`) as the PAC extraction, and `com_hb` copies the task's KLU mode before building so a bare `hb` honours `.option klu` — verified bit-identical under both. Built-in + OSDI. Verified 8/8 vs transient/`fourier`, quadratic convergence: nonlinear R (analytic 3rd harmonic), R+C, a real diode rectifier (junction limiting), OSDI varactor `Q(v)` 2nd harmonic, KLU==Sparse parity

E-135 *spicelib/analysis/dcpss.c, examples/hb_examples/verify_hb.py*

HB source-stepping continuation — makes E-134 Harmonic Balance robust on strongly-driven circuits where a cold full-strength Newton diverges (`|F|`→`1e69`). `HBanalyze`'s Newton loop is wrapped in an adaptive homotopy: every independent source scaled by `λ: 0`→`1`, each level warm-started from the last converged `V`. The residual becomes `F = I_R + I_C − λ·Is`; on level success `V` is checkpointed and `dλ` grows (×1.7), on failure (Newton exhausted, residual non-finite, or singular Jacobian) `V` is restored and `dλ` halves; `dλ<1e−5` → reported as no reachable steady state. First level is full strength (`dλ=1`) so easy circuits converge at `λ=1` immediately, bit-identical to the plain solve. Automatic (no new syntax); `set hb_verbose` shows the λ ramp, summary reports iterations + continuation steps. Verified 9/9: a strongly-driven diode rectifier (5 V into 20 Ω, sharp `IS=1e−14`) diverges cold but converges in 3 continuation steps, matching transient `fourier` to <0.1% for DC/f0/2f0; the 8 existing HB checks stay bit-identical

E-136 *spicelib/analysis/dcpss.c, frontend/com_qpss.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpss_hb.py*

two-tone Harmonic Balance (`qpss <expr> <f1> <f2> hb [K1] [K2]`) — the TRUE, incommensurate-capable quasi-periodic steady state, a frequency-domain HB engine alongside the E-133 transient `qpss` (which is unchanged, commensurate-only). Each node is a 2-D Fourier series `v(t)=ΣΣ V_{k1,k2} e^{j(k1ω1+k2ω2)t}`; `QPSShb` samples devices on a 2-D phase grid `(θ1,θ2)` — time never appears, so incommensurate tones (no beat period) just work — and 2-D DFTs to the conversion matrix `H_{(n),(m)}=G_{n−m}+jω_m·C_{n−m}` (`qp_build_matrix`), Newton-solved by `pss_csolve` with the E-135 source stepping. Sources captured by an oversampled least-squares APFT (`Γ^H Γ) Is = Γ^H b` (a square Vandermonde is catastrophically ill-conditioned past a few harmonics). Reactive needs NO charge extraction (`dq/dt=C(v)v'`); the converged operating point is retained (`qpss_hb_saved`) for `qpac` (E-137). Solver-independent (KLU + Sparse) — same `#ifdef KLU` complex-CSC binding as PAC/HB. Built-in + OSDI. Verified 7/7: analytic cubic `|IM3(2,−1)|/|3rd(3,0)|=3`, even products ~0, 3:1 IP3 slope, incommensurate ?2 tones (E-133 cannot), HB==transient (commensurate), KLU==Sparse; E-133 `verify_qpss.py` stays 11/11

E-137 *spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpac.py*

two-tone small-signal QPAC (`qpac <f_in>`) — completes the quasi-periodic gap: the two-tone analogue of PAC. Run after `qpss … hb`, `QPACanalyze` injects a small signal at `f_in` around the retained QPSS operating point (`qpss_hb_saved`) and reports the response at every sideband `f_in+k1·f1+k2·f2`, mixing it through the SAME 2-D conversion matrix `H_{(n),(m)}=G_{n−m}+jω_m·C_{n−m}` (`qp_build_matrix` at `f_in`) that the QPSS Newton used as its Jacobian. Stimulus placed in the `(0,0)` sideband — a netlist `AC`-source RHS `B0` (captured bias-independent at the op-point) or a unit-current fallback — one dense `pss_csolve`. Adds no device evaluation → solver-independent (KLU + Sparse) by construction. Verified 7/7: reduce-to-AC (pump→0 ? direct `(0,0)` = plain `.ac` response = R, sidebands vanish), v^2-pump conversion ratio `|(1,1)|/|(2,0)|=2`, equal-tone symmetry, clean no-op-point error, KLU==Sparse; E-136 (7/7) + E-133

(11/11) unaffected

E-138 *spicelib/analysis/dcpss.c, frontend/com_qpnoise.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py*

two-tone small-signal QPnoise (`qpnoise <output_node> <f_in>`) — quasi-periodic noise, the two-tone analogue of pnoise (E-124), on the retained `qpss … hb` operating point. Reports output + input-referred noise density at `f_in`, folding every device's noise over all sidebands `f_in+k1·f1+k2·f2` (mixer/PA noise conversion invisible to a static `.noise`). `QPnoiseAnalyze` biases the devices at the QPSS operating point, then `qp_solve_adjoint` transposes the 2-D conversion matrix (`qp_build_matrix`) and solves `Hᵀ Ψ = e_{out,(0,0)}` — one adjoint gives the transimpedance from every (node, sideband) to the output; each device's `DEVnoise` computes `S·|Ψ|²` (reading the transfer from `CKTrhs/CKTirhs`) and the sum over all Nh harmonics is the folded onoise; input-referred via the QPAC gain. Reads only retained data → solver-independent (KLU + Sparse) by construction. With no pump the conversion matrix is block-diagonal so only `(0,0)` survives ? reduces to `.noise`. Verified 6/6: reduce-to-noise (pump→0 ? onoise == plain `.noise` == `4kTR` exactly), conversion active under pump, `inoise=onoise/gain²`, clean no-op-point error, KLU==Sparse; E-133 (11/11) + E-136 (7/7) + E-137 (7/7) unaffected

E-139 *spicelib/analysis/dcpss.c, frontend/com_qpnoise.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py*

cyclostationary QPnoise (`qpnoise <output_node> <f_in> cyclo`) — upgrades E-138 to a time-varying device PSD. Under a two-tone pump a device's bias swings over the period (a diode's shot noise `2qI_D(t)` spikes when it conducts), so `QPnoiseAnalyze` gains a `cyclo` branch using the identity `onoise = (1/P)·Σ_s S(t_s)·|A_s|²`, where `A_s(j) = Σ_{(k1,k2)} Ψ_{(k1,k2)}(j)·e^{j2π(k1 s1/P1+k2 s2/P2)}` is the inverse 2-D DFT of the adjoint transfers (the time-domain transimpedance at phase sample s). Each sample re-biases the devices at the retained op-point (`qp_synth`, `P1/P2` now stored in `qp_harm`) with per-sample junction settling (the E-134 `MODEINITFLOAT` walk — else a limited diode reports a stale bias and the "cyclostationary" result collapses onto the stationary one), evaluates `S(t_s)`, folds through `A_s`, and averages. By Parseval reduces to the stationary sum (and `.noise`) when `S` is constant. Verified 10/10 (6 E-138 + 4 cyclo): cyclo reduce-to-noise, Parseval (bias-independent thermal PSD ? cyclo==stationary even under pump), a hard-pumped diode where cyclo differs from stationary by ~8× (switching-mixer cyclostationary enhancement, visible only with the settling), cyclo KLU==Sparse; E-133/136/137 unaffected

E-140 *spicelib/analysis/dcpss.c, frontend/com_hbosc.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/phasenoise_examples/verify_phasenoise.py*

oscillator phase noise (`hbosc <oscnode> <K> [fguess] [tstab]` + `phasenoise <fstart> <fstop> [pts]`) — closes the periodic/phase-noise gap with the phase-noise piece. `HBOSC-analyze` is an autonomous harmonic balance: an oscillator has no source, so `F(V)=I_R+[dq/dt]=0` is solved for the harmonics AND the unknown oscillation frequency `ω0`. The conversion matrix `dF/dV=H` is singular (right null space = phase mode `u_k=jk V_k`, since the oscillator's phase is free), so Newton runs on the bordered system `[H, ∂F/∂ω0; u*ᵀ, 0]` (nonsingular by bordering, `∂F/∂ω0=I_C/ω0`, gauge row `u*`), transient-seeded (`hbosc` runs a short transient from the deck's `.ic`, reads amplitude + zero-crossing frequency), converging quadratically (LC osc: 4 iters to `|F|=3e-12`; inductors fit the `G+jωC` matrix via branch-current MNA). `PhaseNoiseAnalyze` builds the adjoint of `H` at OFFSET `df` with the unit at the carrier sideband (m=1) and folds device noise (`DEVnoise`); as `df→0` the limit-cycle matrix goes singular through the phase mode so the transimpedance diverges as `1/df` → folded noise `1/df²`, normalized to carrier power `2|V1|²` → `L(df)` in dBc/Hz. Verified 8/8 on an LC oscillator: autonomous HB converges to `f0`+describing-function amplitude, `L(df)` has the −20 dB/dec skirt near carrier flattening into the noise floor at a physical level (≈−147 dBc/Hz `@1kHz`), thermal scaling **L**∝**T** (2×T → +3 dB exactly, pinning the absolute), clean no-op-point error, KLU==Sparse; pnoise (9/9) + qpnoise (10/10) unaffected

E-141 *spicelib/analysis/dcpss.c, frontend/com_qpxf.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpxf.py*

two-tone small-signal QPXF (`qpxf <output_node> <f_in>`) — the quasi-periodic transfer function, the ADJOINT of QPAC (E-137), completing the two-tone small-signal suite (QPSS→QPAC→QPnoise→QPXF ≡ PSS→PAC→pnoise→PXF). `QPXFanalyze` runs one adjoint solve `H^T Ψ = e_{out,(0,0)}` (`qp_solve_adjoint`, reused from QPnoise E-138) and dots each sideband block of `Ψ` with the netlist AC-source pattern `B0` (the QPAC stimulus, captured at the op-point) → the transfer `H_{(k1,k2)} = Σ_j Ψ_{(k1,k2),j}·B0_j` from an input at every sideband `f_in+k1·f1+k2·f2` to the output, all from ONE adjoint. By the reciprocity identity `(H^-1 B)_out=(H^-T e_out)^T B` the sideband-`(0,0)` transfer is bit-identical to the QPAC response (the PXF↔PAC cross-check, E-125). Reads only retained data → solver-independent (KLU + Sparse). Verified 6/6: reciprocity (QPXF(0,0)==QPAC(0,0) bit-identical), conversion-sideband reciprocity, reduce-to-XF (pump→0 ⍰ `(0,0)`=plain transfer=R, sidebands vanish), clean no-op-point error, KLU==Sparse; QPSS 11/11 + QPSS-HB 7/7 + QPAC 7/7 + QPnoise 10/10 unaffected

E-142 *spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, com_qpnoise.c, com_qpxf.c, commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpss_sweep.py*

input-frequency sweep for the two-tone small-signal analyses. `qpac/qpnoise/qpxf` gain a `<dec|oct|lin> N fstart fstop` sweep of `f_in` that emits a plottable ngspice plot (conversion gain / noise figure / image-rejection curves vs frequency), matching how `.ac/.pnoise/.pxf` sweep. `QPACsweep/QPnoiseSweep/QPXFsweep` step `f_in` and reuse the exact single-frequency solve per point: `qpac` records per-node `|(0,0) response|`, `qpnoise` records `onoise_spectrum/inoise_spectrum`, `qpxf` records `xf` (in-band `(0,0)`) + `xf_conv` (total conversion √Σ`|H_{sb≠0}|²`). The analysis fills plain arrays; the front-end builds the plot with a `frequency` scale via the nutmeg vector API (`plot_alloc/dvec_alloc/vec_new`) — the correct layer for a front-end command (the analysis OUTp framework dereferences a live analysis job's `JOBtype`, which a front-end command lacks → was crashing). Shared helpers `qp_steptype/qp_sweep_maxpts/qp_emit_plot`. Verified 5/5: swept value at 0.3G == single-frequency `qpac/qpxf/qpnoise` (machine precision), reactive roll-off vs `f_in`, correct point count; single-frequency QPAC 7/7 + QPnoise 10/10 + QPXF 6/6 unaffected

E-143 *frontend/com_optimize.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optimize_lsq_demo.cir*

gradient least-squares curve fitting for **`optimize`**. The built-in optimizer (E-130, derivative-free Nelder-Mead on a scalar `-minimize`) gains a least-squares mode: one or more weighted `-target <expr> <value> [<weight>]` measurements, optionally spread over several `-analysis` stages (each `-analysis` opens a stage; following `-targets` attach to it, so a single fit can combine e.g. a DC operating point AND an AC response), are fitted with Levenberg-Marquardt — a forward/backward finite-difference Jacobian `J`, normal equations `A=J^T J`, `g=J^T r`, damped solve `(A+λ·diag(A))δ=−g` via a small partial-pivot Gauss solver, standard λ up/down loop. Exploiting the sum-of-squares structure it converges in far fewer analysis runs than the simplex (27 vs 67 evals on a two-target RC fit). `-method nm|lm` overrides the auto choice (LS→lm, scalar→nm); the scalar `-minimize`/Nelder-Mead path is unchanged; `-minimize` and `-target` are mutually exclusive. Parameters are still applied in place with `alter` (no re-source) and the search runs in normalized [0,1] space. All changes in `com_optimize.c` (+ one help string); no new files, no ABI change. Verified 23/23 (E-130 checks [1]–[5] + E-143 [6]–[10]): 1-param/3-target/3-stage RC magnitude fit, 2-param DC+AC multi-analysis fit (R1=3k,R2=2k), LM-fewer-evals-than-NM, OSDI/Verilog-A diode extraction (recover both `is`→1e-14 and `n`→1.2 from two measured I-V points by weighted least squares), and input validation (lm-without-target, minimize+target, multi-analysis+scalar all rejected)

E-144 *frontend/com_optimize.c, frontend/inp.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optimize_dparam_demo.cir*

optimize tunes symbolic **.param** values via a new knob kind –dparam. –param remains an alter target (device/instance, changed in place); –dparam names a netlist .param symbol (e.g. .param w=1u, R1={500*k}) which — being expanded at parse time — is changed with alterparam <name>=<value> + a reset that re-sources the deck (re-evaluating .param expressions and re-stamping device values). Per parameter a kind (OPT_ALTER / OPT_DECKPARAM) is tracked; when any –dparam is present opt_eval applies ALL deck params first, re-sources ONCE, THEN applies the in-place alter params (reset rebuilds from the deck and would otherwise wipe an earlier alter — this ordering is what lets the two kinds mix; verified on a joint .param+device fit). No –dparam ? the E-130/-143 fast path is untouched (no reset, no perf hit). The two re-source banners (Reset re-loads circuit … and Circuit: … in inp.c) are gated by the existing ft_optimizing flag so hundreds of inner re-sources are silent (only the final leave-at-optimum run shows); ft_optimizing re-asserted after each reset. Also fixed 5 pre-existing –Wsign-conversion warnings in inp.c's *ng_script_with_params handler (argc/size unsigned). Closes the last E-143 follow-up. No new files, no ABI change. Verified 31/31 (E-130/-143 [1]–[10] + E-144 [11]–[15]): scalar .param fit (rtop→2333.3), mixed –dparam+–param joint fit (rtop=3k,R2=2k), .param in expression (k→6), least-squares –dparam (LM), quiet re-source (≤1 banner); AC + OSDI-device-value .param verified manually

E-145 *frontend/com_optimize.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optresm.va, examples/optimize_examples/optimize_mparam_demo.cir*

optimize tunes **.model**-card parameters via a third knob kind –mparam. –param remains an alter instance target and –dparam a symbolic .param (re-source); –mparam <@model[param]> names a .model-card parameter (e.g. @dmod[is], @rmod[r]) — which is NOT alter-reachable and NOT .dc-sweepable (only sources/resistors/instance params are) — and is changed in place with altermod <name>=<value>, no re-source (as cheap per eval as –param, unlike –dparam). A new kind value OPT_MODELPARAM; in opt_eval the deck params are re-sourced first (E-144), then the in-place knobs: alter for instances, altermod for models. @model[param] is passed straight to altermod (which accepts that accessor form), mirroring how –param @m1[w] emits alter @m1[w]=…. Circuits with only –param/–mparam keep the E-130/-143 fast path (no reset). All three knob kinds mix in one run. Change confined to com_optimize.c (+1 help line); no new source files, no inp.c change, no ABI change. Verified 39/39 (E-130/-143/-144 [1]–[15] + E-145 [16]–[20]): OSDI model-param fit (@rmod[r] →3k), built-in diode model-param fit (@dmod[is] →1.22e-14), determined model+instance joint fit (r=3k,R2=2k), –mparam does 0 re-sources (in-place fast path), and all three kinds (–dparam+–mparam+–param) coexist and converge

E-146 *frontend/com_sweep.c (new), frontend/com_sweep.h (new), frontend/commands.c, frontend/com_commands.h, frontend/inp.c, frontend/Makefile.am, examples/sweep_examples/**

universal **sweep** command + **.sweep** card. Sweeps ANY circuit knob, generalizing .dc (which steps only sources/resistors/instance params). sw_kind auto-detects the knob: @X[y] with ft_sim->findModel(ckt,X) non-NULL ? model param (altermod), else instance (alter); a bare name with nupa_get_param(name,&found) found ? .param (alterparam+reset), else device (alter) — so model params and **.param**s, impossible with **.dc**, are now sweepable. sweep <knob> (<start> <stop> <step> | lin|dec|oct N a b | list …) [–analysis <cmd>] [–output <name>=<expr> …]: sets the knob, runs the inner analysis (default op) at each point, evaluates each output (last value; default = all node voltages like .dc), and emits a summary plot named sweep with the knob values as the scale (nutmeg vector API, front-end layer per E-142). The per-point analysis plots are retained (tran1,tran2,…). .sweep card: inp.c collects a top-level .sweep line into the post-parse control list as a sweep command; a static sweep_active re-entrancy guard stops a .param re-source (reset re-runs the .sweep card) from recursing. New source files registered in commands.c/com_commands.h/Makefile.am. Verified 11/11 (verify_sweep.py): sweep R1 reproduces built-in .dc R1 bit-for-bit, model-param (@rmod[r]) + .param (rtop) sweeps vs analytic divider, auto-detection routing, .sweep card == command form (no recursion), AC named-

output vs analytic |H(1kHz)|, transient settled value, lin/list/step point counts, multi-output

E-149 *maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/numparam/spicenum.c, frontend/commands.c, examples/lhs_examples/**

Latin-Hypercube Monte Carlo sampling (mcsample), closing the low-discrepancy-sampling gap vs commercial tools. A stratified sampler in randnumb.c holds the mode/N/sample-index/dimension-counter/seed; each random dimension's stratum permutation (Fisher–Yates over 0..N−1) + per-sample jitter are generated lazily from a self-contained splitmix64 seeded by (seed, dimension) — reproducible and order-independent. mc_sample_uniform() = (perm[d][i]+jitter[d][i])/N; mc_sample_gauss() maps it through inv_normal_cdf (Acklam probit, rel err < 1.15e-9) so Gaussian tails stratify too. Sample boundary: spicenum.c's nupa_signal(NUPADECKCOPY) edge (once per reset re-source, guarded by the existing firstsignalS latch) calls mc_sample_advance() (step sample, rewind dimension) before that pass's .param draws. Draw hook (xpressn.c): agauss/gauss take one mc_sample_gauss() and aunif/unif/limit one 2·mc_sample_uniform()−1 when LHS is active, else the unchanged gauss1()/drand() (LHS-off is byte-identical to before). Command com_mcsample (randnumb.c, registered in commands.c both tables) parses lhs <N> [seed <s>] / random / off. Front-end only, so solver-independent. Verified 5/5 under both solvers (verify_lhs.py): stratification (each of 48 strata hit once vs random's ~32/48), two-param multi-dimension, ~130x lower Var(sample−mean) at N=32 (both converge to the same mean), seed reproducibility, analytic mean/sigma correctness

E-150 *frontend/com_sweep.c, frontend/commands.c, maths/misc/randnumb.c, include/ngspice/randnumb.h, examples/_setup.py, examples/highsigma_examples/**

high-sigma rare-event estimation (highsigma), the last statistical ROW versus a commercial simulator -- 4-6 sigma failure probabilities (SRAM / standard-cell yield) that plain MC cannot reach (1e-7..1e-9 needs 1e7..1e9 runs). Scaled-sigma importance sampling: a new MC_MODE_SSS in the E-149 sampler draws each Gaussian .param from the lambda-inflated normal (z = lambda*gauss1()) and accumulates that draw's log likelihood-ratio log(lambda) − (z^2/2)(1 − 1/lambda^2) into sss_logw; mc_sample_weight() = exp(sss_logw) reweights the sample so the estimator is unbiased for the nominal probability. Uniform .params are bounded so SSS leaves them uninflated (weight 1). Direction-free -- no gradient / sensitivity / most-probable-failure-point. mc_sss_config(N,lambda,seed) seeds the global PRNG for reproducibility; mc_sss_off() reverts. Command com_highsigma lives in com_sweep.c (reuses its sw_run_cmd reset/analysis runner and sw_eval_expr), registered in commands.c: highsigma <N> [−scale <lambda>] [−seed <s>] [−analysis <cmd>] −metric <expr> [−max <hi>] [−min <lo>] loops N samples (reset -> analysis -> eval metric -> spec test -> read weight), reports P(fail), relative error, equivalent one-sided sigma-to-fail (-Phi^-1(P)) and failure count, and leaves them in the highsigma_* vectors/vars. The spec is −max/−min values (not a >/< inside the expr, which a control command reads as an I/O redirect). Front-end only, solver-independent; the verify is heavy (thousands of re-sources) so examples/_setup.py marks highsigma SPARSE_ONLY. Verified vs analytic Phi(-beta) (verify_highsigma.py): beta=4 -> sigma 4.000, P 3.16e-5 (true 3.17e-5); deep tail beta=5 (P 2.87e-7) recovered from 6000 runs where plain MC expects ~0.0017 failures; two-sided spec ~doubles P; seed reproducibility exact; two independent Gaussians combine as N(.,sqrt(s1^2+s2^2))

E-151 *maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/com_sweep.c, frontend/commands.c, examples/_setup.py, examples/yield_examples/**

process/mismatch correlations + packaged Monte Carlo yield -- closes the last two partial (warn) statistical rows vs a commercial simulator. (1) CORRELATIONS: mc_corr_config(k, matrix) Cholesky-factors a k x k correlation matrix (row-major, unit diagonal; rejects a non-positive-definite one) into lower-triangular corr_L; mc_corr_component(i) lazily draws the k underlying z's once per sample THROUGH mc_sample_gauss() (so LHS stratification / SSS weighting apply), computes y = L*z, caches it, and returns y[i]. The cache is reset in mc_sample_advance(

) which now runs its reset in EVERY mode (so correlation works under plain MC, not just LHS/SSS). New `.param` function `mvnorm(i)` (frontend/numparam/xpressn.c: added to fmathS, XFU_MVNORM enum in fmathS order, and the dispatch) returns the i-th correlated standard normal; with no matrix registered it degrades to an independent draw. `com_mccorr` (`mccorr <k> <m11>. .<mkk> | off`) in randnumb.c. A reusable `mc_lhs_config(N,seed)` was factored out of com_mcsample. (2) YIELD: `com_montecarlo` in com_sweep.c (reuses sw_run_cmd / sw_eval_expr), registered in commands.c: `montecarlo <N> [-lhs] [-seed <s>] [-analysis <cmd>] (-spec <metric> [-max <hi>] [-min <lo>])...` runs N samples (reset -> analysis -> eval each spec), counts a sample PASS only if every spec is within limits, and reports the yield with a Wilson 95% score interval and per-spec violation counts, leaving `montecarlo_yield/montecarlo_npass/montecarlo_n`. `-lhs` uses mc_lhs_config for a lower-variance estimate; corners are the ordinary `.lib` selection. Front-end only, solver-independent; heavy deck so examples/_setup.py marks `yield` SPARSE_ONLY. Verified (verify_yield.py, Sparse): mccorr rho=+0.7/-0.6 -> empirical corr 0.711/-0.614 with correct means/sigmas; mvnorm without mccorr -> corr ~0; non-PD matrix rejected; single two-sided spec yield 0.8660 (true P(|Z|<1.5)=0.8664); two independent specs multiply (0.7508 vs 0.7506); -lhs unbiased and ~800x lower yield-estimate variance; positive correlation raises the joint yield (0.75 -> 0.82). Demo: a matched divider yields ~100% process-correlated (rho=0.9) vs ~74% independent

E-152 *include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, include/ngspice/smpdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, maths/ni/niinit.c, maths/KLU/klusmp.c, examples/klu_tuning_examples/**

KLU matrix reordering + scaling controls. The KLU direct solver ran on the compiled-in `klu_defaults` (AMD ordering, `max` row scaling, BTF on) with no way to change them, and its one exposed knob `klu_memgrow_factor` was broken (`task->TSKkluMemGrowFactor = (val->rValue == 1.2)` stored a boolean, not the value). E-152 exposes three new `.options` -- `klu_ordering=amd|colamd`, `klu_scale=none|sum|max`, `klu_btf=on|off` -- as IF_STRING options (friendly names, mapped to KLU's integer codes 0/1, 0/1/2, 1/0 in the cktsopt.c handlers) and fixes the memgrow assignment. Plumbing mirrors the existing memgrow path exactly: new OPT_KLU_ORDERING/SCALE/BTF enums (optdefs.h); TSKklu*/CKTklu* int fields on the task (tskdefs.h), circuit (cktdefs.h) and SMPmatrix (smpdefs.h) structs; defaults 0/2/1 that MATCH klu_defaults set in cktntask.c (so behaviour is unchanged unless the user overrides) plus the task copy; task->ckt in cktdojob.c; ckt->CKTmatrix in niinit.c; and in klusmp.c SMPnewMatrix, right after `klu_defaults()`, `Common->ordering/scale/btf = Matrix->CKTkluOrdering/Scale/BTF`. KLU-only; changes only HOW the matrix factors, never the physical solution; the numerics are untouched. Verified (verify_klu_tuning.py, KLU-only) on a resistor grid: every ordering/scaling/BTF setting gives the physically identical solution (max relative spread 2.8e-14); AMD vs COLAMD and scale=max vs scale=none change the factorization arithmetic so the full-precision result differs in its last digits (rel diff ~1.2e-14 / 2.8e-14 -- a deterministic, nonzero proof the knobs reach KLU); invalid values warn; a plain `.option klu` equals amd/max/btf-on bit-for-bit; a wide-dynamic-range network solves correctly under none/sum/max

E-153 *maths/ni/niiter.c, maths/KLU/klusmp.c, include/ngspice/smpdefs.h, include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, examples/trustregion_examples/**

Levenberg-Marquardt trust-region Newton (`.option trustregion`, off by default), completing the damped/trust-region-Newton row alongside the E-111 line search. In the Newton loop (niiter.c): the E-111 KCL-residual merit is reused (gated on linesearch||trustregion); when a dimensionless damping lambda>0 is in effect, mu = lambda*||diag(J)|| is added to the effective diagonal gmin (trGmin, passed to SMPreorder/SMPluFac) AND mu*x_k to the RHS (the E-127 pseudo-transient coupling with x_prev=x_k), so the solve yields the exact damped step x_{k+1}=x_k-(J+mu I)^-1 F(x_k); a post-solve accept/reject block re-loads at the trial point, computes ||F(x_new)||, and either accepts (relax lambda*=0.25) or rejects (restore x_k, grow lambda, force another iteration),

reusing the E-111 state-save/limiting-reset machinery; a convergence guard forbids converging while lambda>0. The ||diag(J)|| scale (new SMPdiagNorm in klusmp.c, for both the KLU and Sparse matrices) makes lambda dimensionless (Marquardt / scale-invariant). Unlike the line search (which only shortens a fixed Newton direction) large mu re-aims the step toward steepest descent, regularizing an ill-conditioned Jacobian. Result-neutral: the fixed point is F=0 for any mu and lambda->0 on success, so it converges to the same operating point -- verified BIT-IDENTICAL to plain Newton (diode, BJT, resistor divider, plus transient) under both solvers. Option plumbing mirrors linesearch (OPT_TRUSTREGION, IF_FLAG, TSK/CKT fields + CKTtrLambda). Solver-independent. HONEST FINDING: instrumented step-rejection counting showed ZERO rejections on every circuit tried (diode strings, behavioral exponentials, cubics, negative-resistance oscillators) -- ngspice globalizes Newton at the DEVICE level (per-device junction limiting: limexp/pnjlim/fetlim, 30 families) which damps the controlling voltages before the residual is computed, so a residual-increasing overshoot never reaches the solver-level merit; the trust-region is thus a correct, safe solver-level regularization that stays inert on typical circuits

E-154 *frontend/com_envelope.c (new), frontend/com_envelope.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/envelope.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h*

Envelope Following (`envelope <node> <fc> <tstop> [nppp] [m] [maxm] [reltol] [settle]`), the last missing analysis in the RF/periodic-steady-state suite. For a carrier-driven circuit whose amplitude/phase envelope varies slowly over many carrier periods, it samples the state once per carrier period T=1/fc and integrates the slow drift, jumping M periods at a time. The exact per-period map is X_{n+1}=phi(X_n), phi = one-period DAE integration; the envelope obeys dX/dn ~ phi(X)-X. The naive forward-Euler jump X_{n+M}=X_n+M(phi(X_n)-X_n) is UNSTABLE on high-Q circuits (the one-period monodromy has unit-circle eigenvalues, so I+M(Phi-I) amplifies -> blow-up; this is what shelved an earlier attempt). The new engine (spicelib/analysis/envelope.c, EFanalysis) uses the IMPLICIT backward-Euler jump X_{n+M}=X_n+M(phi(X_{n+M})-X_{n+M}), solved by Newton [(1+M)I - M*Phi] dY = -G with Phi=dphi/dY the one-period monodromy (finite-differenced, one extra period-integration per state, dense NxN solve capped for modest circuits). A-stable, so it tracks a resonator's envelope without diverging; its fixed point phi(X)=X is the true steady state. Step size M is chosen by step-doubling local-truncation-error control. The one-period map reuses the transient primitives (NIcomCof + NIiter per fixed sub-step, the dctran state rotation) on a fixed grid of nppp points in TRAPEZOIDAL mode (backward-Euler damps a high-Q resonance); phi is SELF-STARTING (a frozen-history variant plateaued at a false low fixed point) with a sub-divided backward-Euler first sub-step to bound the restart damping. The com_envelope.c front-end command runs a short settling transient, calls EFanalysis, and emits an `envelope` plot (_amp = 2|V1|, _dc, _re, _im vs time; nutmeg vector API). Solver-independent. Verified against a full `.tran` under both linear solvers: EF amplitude tracks the transient across a Q~3160 tank ring-up to <3% and to the steady state (<0.5%), stays bounded over 3000 carrier periods (26 envelope samples), and tracks a Q~316 tank to ~1.6%. Completes the RF suite

E-155 *frontend/com_reduce.c (new), frontend/com_reduce.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/rcreduce.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h, examples/reduce_examples/**

RC network reduction (`reduce <fmax> [factor f] [file fname] [name subckt] [keep node ...]`), moving the post-layout "RC reduction / model-order reduction" gap to done. Post-layout extraction produces enormous linear parasitic R/C networks (thousands-millions of interior nodes); `reduce` collapses the R/C network to a small ELECTRICALLY EQUIVALENT one preserving the port behaviour over DC..fmax, written as an ordinary `.subckt` of R's and C's. Method (spicelib/analysis/rcreduce.c, CKTreduceRC): TICER (Time-Constant Equilibration Reduction) -- Schur-complement (Gaussian) elimination of interior nodes of Y(s)=G+sC, kept first order in s so the reduced network is REALIZABLE as R's and C's (no model-order-reduction black box, no passive-

synthesis step). Per elimination of node n, for each neighbour pair (a,b): G[a,b] -= G[a,n]G[n,b]/G[n,n]; C[a,b] -= (G[a,n]C[n,b]+C[a,n]G[n,b])/G[n,n] - G[a,n]G[n,b]C[n,n]/G[n,n]^2. The conductance update is the exact Schur complement so DC is preserved EXACTLY; the capacitance is matched to first order. A node is eliminated only when its self time-constant frequency f_n=G_n/(2*pi*C_n) > factor*fmax (quasi-static in the band of interest); `factor` trades reduction against in-band accuracy, monotonically. PORTS (kept nodes) are auto-detected as every node touched by a device that is NOT a resistor/capacitor (sources, transistors, OSDI Verilog-A devices), plus ground and user `keep` nodes, via the generic GENnode/terminal-count device interface -- so built-in and OSDI devices alike. The com_reduce.c front-end command enumerates the built-in resistor/capacitor instances (CKTtypelook), builds dense G/C over the RC nodes, runs TICER, and emits the reduced subckt (CKTnodName for node names). Solver-independent (reads devices + own dense solve). First cut is dense (node cap ~2500); sparse TICER is the follow-up. Verified under both linear solvers: a huge factor eliminates nothing and the emitted subckt reproduces the full AC response BIT-for-BIT; a moderate factor cuts nodes (e.g. 25->6, 4x) with <0.25 dB in-band error and DC exact; the accuracy/reduction tradeoff is monotone in factor; an OSDI device on a node auto-marks it a kept port

E-156 *spicelib/analysis/rcreduce.c, frontend/com_reduce.c, include/ngspice/cktdefs.h, examples/reduce_examples/**

Sparse RC reduction -- makes the E-155 `reduce` command scale to real post-layout size. The dense N*N TICER engine (capped ~2500 nodes) is replaced by a SPARSE one: the network is stored as per-node adjacency lists (RCadj: a growable edge list of {neighbour, g, c}), and interior nodes are eliminated in a MINIMUM-DEGREE order (a lazy binary heap keyed on current degree, like sparse LU) so fill-in stays tiny -- a degree-2 chain node merges two series elements with ZERO fill. A FILL GUARD (`maxdeg`, default 12) declines to eliminate a node once its degree grows past the threshold, so a dense mesh core (whose boundary Schur complement is inherently dense) is left intact instead of densifying the whole network (measured: unguarded, one mesh elimination created 3308 fill edges; guarded, 9). The TICER Schur-complement update, DC-exact conductance, frequency criterion (eliminate only when f_n=G_n/(2*pi*C_n) > factor*fmax), and automatic port detection are unchanged from E-155; in the edge representation the diagonal is implicit so eliminating a node just adds Schur edges among its neighbours. Node cap raised 2500 -> 5,000,000; a 65,017-node network reduces in ~4 s. New `maxdeg` command option (com_reduce.c) and CKTreduceRC parameter (cktdefs.h). Also fixes a terminal-ordering pitfall: the reduced .subckt's terminals are emitted in internal-index order (not the order keep nodes were typed), so instantiating with the wrong order silently swaps ports -- the command now prints the correct `x1 <ports...> <name>` instantiation line. Verified under both linear solvers: identity reduction bit-exact (0.00 dB), a moderate factor gives ~4x fewer nodes at <0.25 dB in-band with DC exact, monotone accuracy/reduction tradeoff, OSDI auto-port, and an 8001-node network (past the old dense cap) reduces successfully

E-157 *frontend/com_aging.c (new), frontend/com_aging.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/aging_examples/**

Device aging (`aging <t_target> [rate <opvar>] [param <ageparam>] [dynamic <tstop> [tstep]] [verbose]`), adding the reliability "stress -> degrade -> re-simulate (fresh/aged)" flow (HCI/NBTI/TDDB) and moving those reliability gap rows to done. The command finds every aging-capable device in the loaded circuit, computes how much it has degraded after `t_target` seconds of operation, writes that back into the device, and RE-STAMPS the circuit so any analysis run afterwards (op/dc/tran/ac) sees the aged devices; it runs the fresh stress simulation first, leaving it as the current plot (a "fresh" baseline). The design is MODEL-AGNOSTIC: a device opts in purely by exposing, in its Verilog-A/OSDI model, a degradation-RATE operating-point variable (default `agerate`, in dose-units/second) and a per-instance AGE parameter (default `age`, (`*type=\"instance\"*`)). The engine's only job is to integrate the rate into a dose and feed it back; the model owns the physics that maps `age` to a parameter shift (a sublinear power law, an Arrhenius factor, ...). Participants are detected by scanning each device TYPE's instance-parameter table

(DEVices[t]->DEVpublic.instanceParms) for the two keywords, so ordinary resistors/sources are silently skipped and probing never errors. Two modes: STATIC (default) reads the rate at the DC operating point and scales by the lifetime, age = agerate(op)*t_target; DYNAMIC (*`dynamic <tstop>`) runs a transient over one representative window, trapezoidally (time-weighted) integrates the rate, and extrapolates, age = (INT agerate dt / tstop)*t_target -- capturing duty cycle. Because `age` is per-instance, devices sharing a `.model` but at different bias age independently. Implementation (frontend/com_aging.c) reuses the com_optimize synchronous command-dispatch pattern to run op/tran, the nutmeg expression engine to read `@inst[agerate]`, and `alter @inst[age]=...` to write back; console chatter suppressed via ft_optimizing unless `verbose`. Solver-independent. Verified under both linear solvers with a demo square-law NMOS carrying an NBTI hook (agemos.va): enumeration picks exactly the ageable devices, the dose is exactly rate*t_target, the threshold shift matches the analytic ΔVth = dvth_ref*(age/age_ref)^0.25 law to 5 sig figs, aged drain current drops, degradation is monotone in lifetime, a near-threshold device loses a larger fraction of its current than a hard-driven one for the same shift, a 30%-duty pulsed gate ages at 0.30x the DC rate (dynamic), and a sub-threshold device accrues zero dose

E-158 *frontend/com_emir.c (new), frontend/com_emir.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/emir_examples/**

Power-grid EMIR (`emir [rail V] [thresh frac] [thick m] [jmax A/m2] [n exp] [tref s] [top k] [verbose]`), adding electromigration + IR-drop reliability sign-off and moving the last reliability gap row to done. After a DC solve of the power-distribution network the command reports two metrics. IR-DROP: for every node, the drop below the ideal rail (rail - V(node)); the rail defaults to the highest node voltage (the supply pad) unless given; reports the worst node and every node past a threshold (default 10% of the rail). ELECTROMIGRATION: for every wire-segment resistor, the current DENSITY J = |I|/(w*thickness), ranked; the worst-density segment, every segment past the current-density limit Jmax, and a Black's-equation relative lifetime MTTF/ref = (Jmax/J)^n (Black: MTTF ~ J^-n; a segment exactly at Jmax has MTTF = the reference lifetime tref, needing no process-specific prefactor). The physics the analysis exists to expose: EM is set by current DENSITY, not current -- a fat trunk carrying huge current can be safe while a thin wire at a fraction of that current voids first, which is why real grids taper wire width with carried current to hold J roughly constant. Implementation (frontend/com_emir.c) runs a fresh `op` (com_optimize synchronous-dispatch pattern), walks the current plot's node-voltage vectors (plot_cur->pl_dvecs filtered to SV_VOLTAGE) for IR-drop, and enumerates the resistor instances (the device type named "Resistor") reading each segment's current and width via the nutmeg expression engine (`@Rk[i]`, `@Rk[w]`) -- so it needs no device-struct access and works for any resistor; segments without a width are skipped and counted. Both tables are qsort-ranked (IR by drop, EM by density) and truncated to `top`. Solver-independent (reads a DC solution + per-resistor currents). Verified under both linear solvers on a 3-segment tapered ladder off a 1 V rail: worst IR drop 0.30 V (30%) at the far tap matching the exact ladder solve and linear in load current; J = I/(w*thick) exact; the worst-density segment is the narrow low-current wire, not the high-current wide trunk; the Black MTTF ratio between two segments equals (J2/J1)^n; with Jmax between the trunk and leaf density exactly the under-sized segments fail; rail auto-detect equals an explicit rail; and an OSDI (Verilog-A) current load at a tap is handled identically to a current source

E-159 *(no ngspice source change) examples/compactmodels_examples/**

Real production compact-model bring-up -- a validation/example enhancement that exercises the EXISTING toolchain on actual CMC (Compact Model Coalition) Verilog-A models, so no ngspice or openvaf-r source change. BSIM4 (the ~12,600-line industry-standard bulk MOSFET) is compiled through openvaf-r to OSDI (~6 s) and validated against ngspice's BUILT-IN BSIM4 -- the same model, so a rigorous self-check: the OSDI BSIM4.8 and native BSIM4.8.3 agree to ~2% on the Id-Vds output family and ~4.5% on the Id-Vgs transfer curve (the residual is the point-release version gap). EKV (EPFL compact MOSFET, which ngspice has NO built-in for) is brought up purely through OSDI, giving textbook saturation. A general finding surfaced: OSDI keeps ev-

ery internal node STATIC (no dynamic node collapsing), so BSIM4's internal drain/source nodes (di/si) -- which the default rdsmod=0 expects the simulator to collapse onto the external d/s -- are left floating and the device conducts ZERO current; enabling the external series-resistance nodes with rdsmod=1 (the model near-shorts them, 1e3 mho, when no S/D geometry is given) connects them and the device works. Models are compiled in place from the bundled OpenVAF integration-test sources (licenses stay with them). Verified under both linear solvers (6 checks)

E-160 *(no ngspice source change) examples/cmcsweep_examples/*, examples/_setup.py*

CMC compact-model coverage sweep -- the comprehensive follow-up to E-159, again a validation/example enhancement with no ngspice/openvaf-r source change (only _setup.py gains `cmcsweep` in its SPARSE_ONLY and REGRESSION_EXCLUDE sets so the slow 19-model compile runs single-solver and off the fast sweep). It drives EVERY real CMC (Compact Model Coalition) Verilog-A model bundled with OpenVAF through openvaf-r -> OSDI -> ngspice and reports a coverage matrix, the same idea as the E-84 LRM sweep but for production compact models. Result: 19 of 19 models COMPILE and 19 of 19 LOAD, spanning every device class -- MOSFET (BSIM3, BSIM4, BSIM6, BSIMBULK, EKV, HiSIM2, HiSIMSOTB, MVSG_CMC), FinFET (BSIMCMG, BSIMIMG), SOI (BSIMSOI, HiSIMHV), GaN HEMT (ASMHEMT), surface-potential (PSP102, PSP103), bipolar/SiGe-HBT (HICUML2, MEXTRAM), and diode (DIODE, DIODE_CMC). Deeper conduction+physics validation for a representative model per class: OSDI BSIM4 and BSIM3 match ngspice's BUILT-IN BSIM4/BSIM3 to ~2%/~7%; EKV gives monotonic saturating I-V; HICUML2 shows bipolar current gain beta ~ 100 on a Gummel plot; DIODE_CMC turns on exponentially (x16000 over 0.2-1.0 V). Two findings: (1) the E-159 internal-node/rdsmod=1 issue is BSIM4-SPECIFIC, not universal -- BSIM3 handles the zero-resistance case fine and conducts out of the box (its apparent zero at Vgs=1 V is just a high ~1.7 V default threshold); (2) HiSIMHV (6 terminals d,g,s,b,sub,temp) needs cosubnode=1 to enable the substrate node, and with the default cosubnode=0 its $fatal correctly guards the node-count mismatch (confirming $fatal works through OSDI). Models compiled in place from the bundled integration-test sources (licenses stay put). Verified (7 checks); the compile/load tiers are solver-independent

E-161 *(no ngspice source change) examples/dynmodels_examples/**

Dynamic (AC/RF) compact-model validation -- extends the E-159/160 DC validation into the dynamic domain, again a validation/example enhancement with no ngspice/openvaf-r source change. Where E-159/160 exercised DC I-V, this exercises the DYNAMIC behavior, which flows through a different code path: OSDI's REACTIVE (charge) Jacobian stamping and ngspice's `.ac` analysis, on the real production models (compiled in place from the bundled CMC sources). BSIM4 C-V: the gate capacitance Cgg(Vgs) is extracted from `.ac` as Im(I_gate)/omega and swept over gate bias; it rises from ~40 fF in subthreshold (overlap+fringe) to ~129 fF in inversion (approaching CoxWL) -- the textbook MOSFET C-V -- and the OSDI-compiled BSIM4 matches ngspice's BUILT-IN BSIM4 to under 1% at every bias (a tighter check than the ~2% DC I-V match, since Cgg is dominated by the version-independent oxide term). Cutoff frequency fT (the frequency where AC current gain |h21|=|I_out/I_in| falls to 1): OSDI BSIM4 fT ~ 3.5 GHz matches the built-in to ~1%; HICUML2 (SiGe HBT, which ngspice has no built-in for) has zero default transit time (t0=0) giving infinite fT, so a realistic dynamic parameter set (t0=10 ps, 1 fF junction caps) is supplied and the resulting fT lands right at the transit-time limit $1/(2\pi t0)$ ~ 15.9 GHz and rises with collector current -- textbook charge-control behavior. Verified under BOTH the Sparse and KLU solvers (`.ac` is supported by both), 4 checks

E-162 *frontend/inp.c, examples/hb_examples/verify_hb.py*

`.hb` dot-card for harmonic balance -- gives the E-134 `hb` command netlist dot-card parity with the PSS family (`.pss`/`.pac`/`.pnoise`/`.pxf`/`.psp`/`.sp` are dot-cards, while the HB family `hb`/`qpss`/`hbosc` were control-block commands only). A top-level `.hb <f0> <K> [points] [maxiter]` card now runs single-tone harmonic balance straight from the deck (no `.control` block needed). Rather than duplicating the HB machinery as a separate analysis JOB, `.hb` reuses

the deck->control mechanism Enhancement-146 introduced for `.sweep`: during deck loading (frontend/inp.c) a top-level `.hb ...` line is stripped of its leading `.` and appended to the post-parse control list, so it executes as `hb ...` (the com_hb command) once the circuit is built -- inheriting all of the E-134 engine (argument parsing, the E-121 conversion-matrix Jacobian, E-135 source-stepping continuation, solver independence). A boundary check (next char after `.hb` is whitespace or end-of-line) keeps it from swallowing any future `.hb*` card. Before this, `.hb` produced `unimplemented dot command '.hb'` -- the only `.hb` handler in ngspice was a disabled `#ifdef WITH_HB` upstream stub (dot_hb) that merely redirected to the PSS analysis, not the E-134 engine. Verified (examples/hb_examples/verify_hb.py, +2 checks): the `.hb` dot-card run in plain batch mode (no .control) converges and produces a harmonic-balance spectrum bit-for-bit identical to the `hb` command form on a junction-limited diode rectifier; it threads its optional [points]/[maxiter] arguments and coexists with a following .control block (deck order preserved); both Sparse and KLU give identical results. (Like `.sweep`, a bare command-style dot-card with no other analysis card prints a benign "no simulations run" batch notice even though HB ran.)

E-163 *frontend/inp.c, examples/qpss_examples/verify_qpss.py, examples/phasenoise_examples/verify_phasenoise.py*

`.qpss` / **`.hbosc`** / **`.phasenoise`** dot-cards -- completes the harmonic-balance family's netlist dot-card parity begun by E-162's `.hb`. Three more top-level command-style cards now run their analyses straight from the deck: `.qpss <expr> <f1> <f2> [periods] [maxorder]` (two-tone quasi-periodic steady state, E-133/136; add the `hb` keyword for the frequency-domain form), `.hbosc <oscnode> <K> [fguess] [tstab]` (autonomous harmonic balance for an oscillator, E-140; the deck needs a `.ic` to start the oscillation), and `.phasenoise <fstart> <fstop> [points]` (oscillator phase noise, E-140; run after `.hbosc`). Each reuses the same one-branch deck->control mechanism as `.hb` (E-162) / `.sweep` (E-146): in frontend/inp.c a top-level `.qpss`/`.hbosc`/`.phasenoise` line is stripped of its leading `.` and appended to the post-parse control list, executing as the corresponding command once the circuit is built -- inheriting the full E-133/136/140 engines and their solver independence. Because the bridge preserves deck order, `.hbosc` (which finds the oscillator PSS + frequency) and `.phasenoise` (which reads it) compose, so a complete oscillator phase-noise run needs no `.control` block. A per-card boundary check (the char after the name is whitespace or end-of-line) keeps the cards distinct -- in particular `.hbosc` is NOT matched by the `.hb` branch, whose own boundary check rejects the trailing `osc`. Verified: the `.qpss` dot-card produces a two-tone spectrum bit-for-bit identical to the `qpss` command (examples/qpss_examples/verify_qpss.py); `.hbosc` + `.phasenoise` in a deck reproduce the command form's oscillation frequency and phase-noise spectrum exactly with order preserved (examples/phasenoise_examples/verify_phasenoise.py); both under Sparse and KLU. No regression to `.hb`/`.sweep`.

E-164 *(no ngspice source change) examples/rfpa_examples/**

Large-signal RF characterization of a real transistor -- a validation/example enhancement (no ngspice/openvaf-r source change) that closes the production-model validation loop (DC E-159, coverage E-160, small-signal E-161, now large-signal). It drives a common-emitter amplifier built from the bundled HICUM/L2 SiGe HBT (compiled in place, with the E-161 dynamic params t0=10ps + 1fF caps) hard and extracts the large-signal RF figures of merit via TRANSIENT + Fourier/FFT: AM-AM gain compression (the power gain expands 17.8->20.7 dB then compresses -- the exponential-transconductance signature of a bipolar -- defining P1dB), harmonic distortion (the third harmonic follows the textbook 3:1 slope, HD3 ~ A^3), and two-tone third-order intermodulation (the IM3 products 2f1-f2 / 2f2-f1 at ~-30 dBc). IIP3, extracted from the single tone as A/sqrt(3*HD3) (the two-tone IM3 is exactly 3x the single-tone HD3 for a third-order nonlinearity), is constant at ~0.134 V across drive level, confirming the third-order model. A real finding: the frequency-domain harmonic-balance engines (hb/qpss, E-134/136, which just gained netlist dot-cards in E-162/163) do NOT converge on this stiff, many-internal-node production model in an amplifier configuration -- hb returns error 103 at any drive level (even 1 mV) and with extra iterations/

source-stepping, and the two-tone qpss is prohibitively slow (>2 min per point). Transient+FFT integrates reliably and is the robust route for large-signal RF on a full production compact model; HB/QPSS remain the tool of choice for lighter, well-conditioned circuits. Verified under both linear solvers (4 checks): transient small-signal gain matches .ac (7.744 vs 7.742), HD3 3:1 slope (62.9 vs 64), IIP3 constant (0.134 V, spread <2%), and gain compresses at high drive (7.74->5.97).

E-165 *(no ngspice source change) examples/modelnoise_examples/**

Production compact-model noise validation -- a validation/example enhancement (no ngspice/openvaf-r source change) exercising the last untested small-signal path, OSDI's `.noise` stamping of the models' own white_noise / flicker_noise sources, on production models compiled in place from the OpenVAF integration-test sources. BSIM4 (MOSFET): the output-noise spectral density Sv(f) of a common-source amplifier is compared to ngspice's BUILT-IN BSIM4 over the full band and matches to ~1.5% everywhere, covering both the low-frequency 1/f FLICKER region (amplitude density sqrt(Sv) ~ 1/sqrt(f)) and the flat high-frequency THERMAL floor -- a stringent check since the flicker coefficients, thermal-noise model, and their bias dependence all have to line up. HICUM/L2 (SiGe HBT): no ngspice built-in, so validated against physics -- its default noise is WHITE (shot noise on the junction currents + thermal noise on the parasitic resistances, no flicker by default), a flat spectrum in contrast to the MOSFET's 1/f rise; with a small source resistance so the intrinsic device noise dominates, the output-noise floor tracks the collector SHOT-noise line sqrt(2q*Ic*RC^2) across two decades of bias current (within 6% at high bias where shot dominates) and scales as sqrt(Ic). Verified under BOTH the Sparse and KLU solvers (`.noise` works under both since E-113 fixed the KLU adjoint solve), 5 checks: BSIM4 spectrum matches built-in <4%, BSIM4 1/f flicker slope (Sv(1Hz)/Sv(10Hz) ~ sqrt(10)), BSIM4 flat thermal floor, HICUM white noise, HICUM shot-noise tracking + sqrt(Ic) scaling. Completes the production-model validation loop (DC E-159, coverage E-160, small-signal E-161, large-signal E-164, noise E-165).

E-169 *frontend/syntaxhl.c (new), include/ngspice/syntaxhl.h (new), frontend/commands.c, frontend/Makefile.am, main.c, examples/syntaxhl_examples/**

Interactive command-line syntax highlighting -- the interactive prompt now colors the command line AS IT IS TYPED. The command word is shown GREEN the moment it is a recognized command, RED when it can no longer become one, and left the terminal default while it is still a valid PREFIX being typed (so `plo` is neutral, `plot` green, `plt` red); numbers are yellow, "quoted strings" magenta, and -option flags cyan. The command word is classified exactly as the interpreter resolves it -- the same case-insensitive walk of the live command table cp_coms that docommand() uses, plus the control keywords (if/while/foreach/begin/end/...) -- so the color always agrees with whether the word will actually run. Implementation: a pure engine cp_highlight_line() (frontend/syntaxhl.c) tokenizes a line and wraps each token in ANSI SGR color escapes; live coloring overrides GNU readline's rl_redisplay_function (installed by cp_synhl_init() from main.c right after rl_outstream is set). The replacement redisplay redraws the colorized line only when the prompt plus line fit on one terminal row and otherwise defers to readline's own rl_redisplay(), so a wrapped line is never corrupted; the cursor is repositioned with a plain CR + CUF (cursor-forward) so mid-line editing is unaffected. Because that manual cursor positioning bypasses readline's internal position tracking, readline's own accept-line stops emitting the closing newline, so the Enter key is bound to a wrapper (cp_synhl_accept) that emits the newline itself when coloring is active before running the normal accept-line -- otherwise a command's output would begin on the input line (when coloring is off the wrapper is a no-op and readline handles the newline as usual). Gated three ways: it is on by default only at an interactive TTY (isatty(rl_outstream)), is disabled by `set no_syntax_highlight` and by the NO_COLOR environment convention, and never fires in batch (`ngspice -b`) or any piped/redirected session -- so simulation output is byte-for-byte unchanged (confirmed by the full example regression). Because as-you-type coloring needs the terminal in raw mode it requires a readline-enabled build, which the shipped binaries are (CI builds with --with-readline=yes and the committed binaries link libreadline). A new `synhl <command line>` command prints the colorized form of a line non-

interactively -- useful for previewing and, since it needs no terminal, what makes the coloring engine regression-testable in batch. Verified (examples/syntaxhl_examples/verify_syntaxhl.py, 11 checks): the engine via `synhl` in batch (valid command green, unknown red, valid prefix neutral, numbers/strings/options colored, case-insensitive) and the live as-you-type behavior driven through a pseudo-terminal (green/red/neutral on the fly, NO_COLOR and a non-tty suppress all color); the live layer auto-skips on a build without readline. Solver-independent (front-end only).

E-170 *frontend/syntaxhl.c, include/ngspice/syntaxhl.h, examples/syntaxhl_examples/**

Semantic syntax highlighting -- extends E-169 from lexical to SEMANTIC coloring of the interactive line: it now checks whether the SIGNALS and EXPRESSIONS typed actually exist and parse. (1) A signal reference v(node)/i(source)/`@device[param]` is looked up read-only in the current plot (vec_fromplot, no evaluation, no vector creation); it keeps the default color if it exists and is drawn RED if it does not. (2) Inside an otherwise-valid expression only the invalid signal reddens -- `print v(a)+v(zzz)` colors just v(zzz), including signals nested in functions (sqrt(v(a))-v(bad)). (3) A genuinely malformed expression argument of an expression command (plot/print/gnuplot/asciiplot) reddens as a whole, tested with the real parser ft_getpnames_from_string(FALSE) with its tree freed. (4) Error output is drawn RED at an interactive terminal: cp_err is wrapped in a colorizing stream (funopen on BSD/macOS, fopencookie on Linux) that prepends the red SGR to each write; cp_curerr is pointed at the wrapper too so ngspice's per-command stream reset cp_ioreset() keeps it in place instead of restoring and closing it. A half-typed expression is NOT treated as an error and stays neutral -- a signal whose parenthesis has not closed (v(bP), an expression with unbalanced parens (v(a)+v(b) or one ending in an operator (v(a)+). KEY FIX: the expression parser's error handler PPerror writes straight to stderr, bypassing the cp_err muting, so an early version spewed `syntax error in line segment ...` onto the prompt while typing; expr_parses() now also mutes fd 2 (dup2 /dev/null) across the parse and restores it. Signal-validity is scoped to the unambiguous v()/i()/@ forms (bare words could be plot keywords like vs, functions like sqrt, or options -> false reds). Same three-way gating as E-169 (interactive TTY, set no_syntax_highlight, NO_COLOR); the parser + vector lookups run per keystroke but are read-only and do NOT corrupt command execution (a subsequent let z=v(a)+v(b) still computes correctly). Verified (verify_syntaxhl.py, 19 checks total): the E-169 lexical checks plus a semantic layer driven through a pseudo-terminal after simulating a small circuit -- valid signal not red, invalid signal red, invalid-in-expression reddens only the signal, malformed expression red as a whole, half-typed stays neutral with no parser-error spam, error output red, NO_COLOR suppresses it. Solver-independent (front-end only).

E-171 *maths/KLU/klusmp.c, spicelib/analysis/pzan.c, examples/klupz_examples/**

KLU/Sparse solver audit -- KLU pole-zero fixed (complex determinant + pivot tolerance) -- a deep audit of the KLU<->SMP bridge found that pole-zero analysis under 'option klu' SILENTLY PRODUCED GARBAGE for any circuit with complex poles or zeros: a series RLC reported four bogus real poles (-100, -10, -0.89, 0) instead of its conjugate pair -5000 +- j999987.5. Real-axis roots came out right (the mixed formula is accidentally correct for real pivots), which is why the existing single-real-pole RC regression never caught it. DEFECT 1 (spDeterminant_KLU): the complex determinant built each pivot as the mixed quantity (1/(Ux*Rs), Uz*Rs) and took its complex reciprocal -- KLU's Udiag stores the ACTUAL pivots (its solve divides by them) while Sparse's Diag stores RECIPROCAL pivots (its solve multiplies), and the adaptation kept Sparse's reciprocal dance for the real part only. The real branch was worse: its product loop NEVER RAN (the loop index was left at N by the preceding permutation scan, so det was always +/-1.0), it divided instead of multiplying, and it never wrote *piDeterminant (SMPcDProd consumed an uninitialized stack value). Both branches computed the permutation sign as #non-fixed-points/2, wrong for any cycle longer than 2 (a 3-cycle is EVEN but counted odd). Rewritten: det(A) = sign(P)*sign(Q)*prod(Udiag[k]*Rs[k]) with parity computed exactly by cycle decomposition (new PermutationParity helper); the KLU determinant now matches Sparse to ~14 digits at every Muller trial point. DEFECT 2 (SMPcReorder/SMPreorder): pole-zero calls SMPcReorder with PivRel = 0.0; Sparse's spOrderAnd-

*Factor SANITIZES a non-positive threshold to its stored default, but the KLU branch assigned it straight to Common->tol, making KLU's partial pivoting accept an EXACTLY-ZERO diagonal as pivot (|diag| >= 0*max always holds). At PZ's s=0 trial an inductor branch has a 0.0 diagonal (-sL), so the factorization returned KLU_SINGULAR and PZ recorded a spurious root at the origin -- and the poisoned deflated Muller search never expanded past |s| ~ 10, yielding the garbage above. Both reorder entry points now mirror Sparse's sanitization (in-range PivRel still takes effect; DC/AC/tran pass the 1e-3 default and are byte-identical). With both root causes fixed, the E-113-era guard in pzan.c ('pole-zero finite-zero computation is not supported with option KLU' -- added when the zero search was seen to go spuriously singular, i.e. defect 2) is REMOVED; the finite-zero search now works under KLU and a genuine E_SHORT reports the same 'input shorted' diagnostic as Sparse. The balanced/differential-output PZ guard remains (SMPcAddCol genuinely has no KLU branch). Verified (examples/klupz_examples/verify_klupz.py, 6 checks): full pole/zero root-set parity between the two solvers plus analytic anchors on a series RLC (conjugate pair), RC lowpass (old-good case, no regression), lead network (finite real zero), RC highpass (origin zero), RLC bandstop (PURELY IMAGINARY zeros +-j1e6, the hardest case), and RLC bandpass (the finite-zero search formerly guarded off) -- all six identical across solvers. Full 134-example regression clean. Residual scope (solver-independent, ~1990 Muller-on-determinant fragility): a twin-T's deflated conjugate zero pair can still stall under KLU (4 of 6 roots found) while on the bandpass it is SPARSE that hits its iteration limit (KLU converges cleanly).

E-172 *maths/KLU/klusmp.c, spicelib/analysis/cktpzset.c, spicelib/analysis/pzan.c, examples/_setup.py, examples/klupz_examples/**

KLU balanced pole-zero + full partial pivoting -- closes E-171's two scope items; no analysis card remains Sparse-only under KLU. (1) BALANCED / DIFFERENTIAL-OUTPUT pole-zero ('pz in 0 a b vol', non-ground output reference) was guarded off as unsupported: the algorithm's change of variables needs the per-trial column fold col(b) += col(a) (SMPcAddCol), which had no KLU branch -- and KLU's CSC sparsity pattern is FIXED at COO->CSC conversion time, so the fold's fill-in cannot be created on the fly the way Sparse's spcCreateElement does. Fixed in two halves: CKTpzSetup reserves the UNION pattern before the conversion (every row of the solution column is also registered in the balance column via SMPmakeElt; duplicate (row,col) COO entries fold into a single CSC slot by the conversion's group labeling, so the reservation is safe), and SMPcAddCol gains a KLU branch that merge-walks the two columns' sorted row lists adding the complex values in place (with a defensive error if an addend row is missing, impossible after the reservation). The two pzan.c balanced-output guards are removed. (2) FULL PARTIAL PIVOTING for PZ factorizations: validating a fully-differential configuration exposed spurious far-field roots at |s| ~ 1e19. .1e21 (including a positive-real pole). An exact-rational (fraction-arithmetic) determinant of the very matrix KLU factored proved the factorization itself was numerically rotten at extreme |s|: at s=4.52e19 the exact determinant was +4.36e11 while the pivot product gave -2.05e12 -- wrong sign AND magnitude. Root cause is architectural: KLU's ordering is fixed at klu_analyze time (pattern-only AMD/BTF) while Sparse re-runs value-aware Markowitz ordering every PZ trial; PZ sweeps |s| across ~20 decades, and with the relaxed default tol=0.001 the fixed ordering accepted catastrophically-cancelling pivots. KLU's only value-adaptive lever is the within-block partial pivoting, so the E-171 sanitization fallback is upgraded from 'keep current tolerance' to FULL partial pivoting (tol=1.0) whenever the caller passes an out-of-range PivRel -- only pole-zero does (0.0); explicit in-range tolerances (DC/AC/tran 1e-3) are honored unchanged. This single change eliminated the far-field junk AND cured the twin-T conjugate-zero-pair stall recorded in E-171's scope (what looked like vintage-Muller noise-floor fragility under KLU was largely this factorization inaccuracy; KLU now finds all 6 twin-T roots with notch-zero real parts ~1e-12, an order CLOSER to the exact 0 than Sparse's ~1e-11). Verified (verify_klupz.py, 6 -> 9 checks): the E-171 six plus the differential RC bridge (balanced output, formerly 'not supported'), a balanced output with a complex pole pair, and the twin-T notch -- all root sets identical across solvers. The dual-solver harness's balanced-pz KLU-skip detection is removed from examples/_setup.py (nothing remains Sparse-only). Full example regression 134/134.

E-173 *spicelib/analysis/cktpzeig.c (new), maths/dense/eig.c (new), maths/KLU/klusmp.c, spicelib/analysis/pzan.c, cktsopt.c, cktntask.c, cktdojob.c, include/ngspice/{smpdefs,optdefs,tskdefs,cktdefs}.h, two Makefile.am, examples/pzeig_examples/**

Eigenvalue-based pole-zero ('.options pzeig') -- a modern alternative root finder for .pz. The vintage driver hunts roots of det(G+sC) with a Muller iteration on determinant values and is famously fragile (iteration limits -- 'giving up after 231 trials' on a plain RLC bandpass -- noise-floor stalls, chaotic search paths; E-171/172 fixed the KLU-side defects but the ~1990 algorithm itself stays delicate under BOTH solvers). The new opt-in method computes every pole and zero at once as a dense eigenvalue problem, reusing the existing CKTpzSetup/CKTpzLoad machinery so it applies unchanged to the poles and zeros configurations, to balanced/differential outputs, and to both linear solvers. Steps: (1) the PZ-configured MNA matrix is AFFINE in s, A(s) = G + s*C, so two loads (s=0, s=1) extract the dense pencil via a new solver-agnostic SMPdenseExtractReal (KLU: walk Ap/Ai/Ax Complex; Sparse: walk FirstInCol through the Int->Ext translation maps); a third load at an arbitrary point VERIFIES affinity so a non-polynomial device falls back with a clear message instead of wrong roots. (2) C is structurally singular, so the pencil is solved by SHIFT-INVERT linearization: factor (G + sigma*C) ONCE at a non-root shift (tried from a small ladder of shifts, with the circuit's own sparse solver), form the dense M = (G+sigma*C)^{-1}C by n sparse solves; from (G+sC)v=0 <=> Mv = muv with mu = 1/(sigma - s), every finite root is s = sigma - 1/mu and the pencil's infinite eigenvalues land harmlessly at mu = 0 (dropped by a noise-floor threshold 64*n*eps*||M||). (3) eigenvalues of the real dense M from a new self-contained eigensolver maths/dense/eig.c -- the classical balance / Hessenberg / Francis double-shift QR chain (EISPACK balanc/elmhes/hqr lineage, written fresh, no LAPACK dependency), unit-tested standalone against companion matrices, complex conjugate pairs, a 50x50 tridiagonal with known spectrum, and a badly-scaled balance case. Roots are emitted as the same PZtrial list the Muller driver produces, so PZpost and all downstream output are unchanged. Option plumbing follows the standard task-option path (OPT_PZEIG -> TSKpzEig -> CKTpzEig flag -> dispatch in pzan.c). RESULTS: RLC bandpass -- same roots as Muller with NO iteration-limit warning; 10-section RC ladder -- all ten poles matching the analytic tridiagonal-eigenvalue formula s_k = -(2-2cos((2k-1)pi/21))/RC and Muller root-for-root; twin-T -- all 6 roots under both solvers; RLC bandstop -- purely imaginary zeros +-j1e6 EXACT; balanced/differential output works; purely resistive circuits yield no roots and no crash. DEFAULT REMAINS MULLER (pzeig is opt-in). Dense O(n^2) memory / O(n^3) QR, capped at 2000 unknowns with a clear error above. Accuracy is dense-QR class (absolute error ~ eps * dominant root magnitude): an exact origin-zero can print as ~1e-7 when poles sit at 1e6 rad/s; Muller refines locally and can resolve wider per-root dynamic range, while pzeig never misses or invents a root -- the two methods agree to >= 6 digits on every battery circuit. Verified (verify_pzeig.py, 13 checks: both solvers x both methods + analytic anchors + default-method check). Full example regression 136/136.

E-174 *src/frontend/commands.c, examples/helpcmd_examples/**

`help all` crash fix (help string used as printf format) -- running `help all` (or `help montecarlo`) in the interactive shell crashed the shipped ngspice with 'Error: tvprintf failed / ERROR: fatal error in ngspice, exit(-1)'. Root cause: ngspice's help printer uses each command's help TEXT as a printf FORMAT string -- out_printf(ccc[i]->co_help, cp_program) in com_help.c (and the parallel com_ahelp.c) -- passing only one argument (cp_program, the program name, intended for a single %s as in the legitimate 'Report a %s bug.'). Any other '%' in a help string is an invalid/incomplete conversion specifier, so tvprintf fails and ngspice calls a fatal exit(-1). The montecarlo command's help (added in E-151) read '... reports the yield with a Wilson 95% CI ...'; the '% ' (percent-space) is the invalid specifier. help all walks the command table alphabetically and died at montecarlo; help montecarlo crashed directly too. Present in BOTH command tables (spcp_coms for ngspice, nutcp_coms for nutmeg). Not X11/terminal/solver specific -- plain format-string handling, so it reproduced on every build; it was invisible in headless CI only because help all is an interactive command not exercised by the batch example decks. Fix: escape the literal percent as %% (printf

convention) in both tables, so the line renders 'Wilson 95% CI' and help all completes. New regression guard examples/helpcmd_examples/verify_helpcmd.py with two layers: RUNTIME -- drives an interactive ngspice on a pseudo-terminal and asserts help, help all, and help montecarlo each run to completion (no crash, no 'tvprintf failed') and that the montecarlo line renders a literal '95% CI'; and a STATIC class-guard that scans commands.c and fails if any command help string carries a format hazard (a '%' that is not '%s' or '%%', or more than one '%s' since only one argument is passed), catching a future unescaped '%' even when that command is never exercised at runtime. Full example regression 137/137.

E-175 *spicelib/analysis/dcpss.c, examples/rfconv_examples/**

RF-suite audit -- the dropped parametric term in every periodic small-signal analysis -- a full audit of the RF suite (.sp/Touchstone, PSS/PAC/pnoise/pxf, PSP, HB, QPSS + the two-tone family, hbosc/phasenoise) driven by analytic anchors and cross-analysis invariants. CLEAN: .sp + Touchstone (17 checks: series-R/shunt-C/3-port star S-parameters exact to 10 digits, the asymmetric-2-port S21/S12 column-order trap, reciprocity, RI/MA/DB + Y/Z (v1 normalization) + GHz round-trips through wrsnp/rdsnp, N-port 4-pairs-per-line layout); HB (analytic cubic: fundamental a1*A+(3/4)a3*A^3 and H3=(1/4)a3*A^3 exact to 7 digits, even harmonics at machine zero, Sparse/KLU identical); QPSS-HB/QPSS-tran (two-tone fundamentals, IM3 (3/4)a3*A^3 exact, commensurate aliasing guard fires correctly). THE BUG: every LPTV small-signal conversion matrix scaled its reactive coupling by the COLUMN (input-sideband) frequency, H_nm = G_{n-m} + j*omega_m*C_{n-m}. For a small signal riding a FROZEN periodic capacitance C(t)=dQ/dv|orbit the exact linearization is di = d/dt[C(t)dv] = Cdot*dv + C*dv_dot, and the product rule collapses to the ROW frequency: H_nm = G_{n-m} + j*omega_n*C_{n-m}. Using omega_m silently DROPS THE PARAMETRIC-PUMPING TERM Cdot*dv -- the very term that makes a pumped varactor convert. Every conversion sideband through a time-varying capacitance came out scaled by exactly omega_in/omega_out: on the audit circuit (1k -> OSDI varactor C(v)=c0(1+alpha*v) pumped 1V@1MHz, 1mV probe at 250kHz) the +-1/+-2 sidebands were 3x/5x/7x/9x too small, with measured pre-fix/truth ratios matching the omega ratios to FIVE DIGITS (0.3333/0.2000/0.14286/0.11111) -- diagnosis confirmed beyond doubt. AFFECTED: pac, psp, pnoise, pxf, qpac, qpnoise, qpxf, and the phasenoise adjoint, for any circuit with pumped/nonlinear capacitance (varactors, junction and MOS capacitances -- every real mixer and oscillator). UNAFFECTED: LTI circuits (no off-diagonal C harmonics; omega_n == omega_m on the diagonal) -- exactly what every prior regression check used; the E-171 accidental-correctness pattern again. THE SUBTLETY: the harmonic-balance residual uses the SAME builders and must KEEP the column frequency -- its reactive current is the exact chain rule d/dt Q(v(t)) = C(v(t))*vdot(t) whose n-th harmonic is sum_m C_{n-m}*(j*omega_m*V_m). That is why HB and QPSS-HB always matched transient ground truth while the small-signal analyses did not, and why a blanket fix would have broken HB. FIX: a smallsig mode flag on pac_build_matrix and qp_build_matrix plus the two inline copies (the PAC adjoint and the phasenoise adjoint): smallsig=1 -> row frequency (restores the parametric term); smallsig=0 (HB/HBOSC/QPSS-HB residual + Jacobian) -> chain-rule column frequency, byte-for-byte unchanged. Regression safety proven directly: the linear-circuit PAC control produces IDENTICAL output pre/post fix, and HB/QPSS-HB outputs are byte-identical. Ground truth = plain transient + one-beat Fourier projection (independent of all conversion-matrix machinery); the fixed qpac matches all five sidebands within 2% (integrator error class). Also noted: cktspnoise.c is a dead fossil (#ifdef RFSPICE_ never compiled). Verified (examples/rfconv_examples/verify_rfconv.py, 5 checks x both solvers: truth vs QPSS-HB unchanged, truth vs fixed qpac, pre-fix signature absent, HB + QPSS-HB analytic anchors). Full example regression 138/138.

E-176 *spicelib/analysis/dcpss.c, examples/_setup.py, examples/pssdriven_examples/**

Driven-mode PSS shooting -- no frequency hunt, no breakpoint flood -- diagnosis of the E-175 audit's flagged residual ('PSS shooting robustness: the varcap PSS ran 17+ minutes without converging'). Instrumentation showed 9,627,161 accepted timepoints by t=2us (~0.2 ps per step, zero

LTE rejections): ngspice's PSS (the Lannutti shooting) is OSCILLATOR-oriented -- it hunts the fundamental frequency through mid-period forced breakpoints whose spacing is proportional to steady_coeff. On driven circuits (everything the E-117..126 periodic small-signal stack runs on) this is doubly pathological: (1) at the standard decks' steady_coeff=5e-6 the grid spacing is ~0.5 ps -- millions of steps per shooting cycle, and the convergence-tolerance knob doubles as grid density so tighter tolerance quadratically explodes runtime (even the default 1e-3 forces thousands of points/period -- why a linear RC .pss took ~4 minutes); (2) the frequency estimate can never settle EXACTLY on the source frequency, the cycle endpoint phase-drifts against the source, and the period residual FLOORS (~1e-4) -- shooting never converges and burns the full sc_iter budget at flood speed. FIX: a DRIVEN mode, auto-detected when the circuit contains a time-varying independent source (SIN/PULSE/... V or I source, funcTGiven): the period is pinned to the exact source period (no estimation, no estimator grid; each cycle is one plain-transient period; varactor residual 2e-1 -> 8e-9 in 17 cycles / 0.3 s); the shooting-phase max step is clamped to T/psspoints so the orbit is integrated on the SAME discretization as the retained samples (validation caught this: without the clamp LTE grows steps toward T/2 and shooting converges -- to 7e-9! -- onto the fixed point of that coarse discretization, several percent off the true orbit: retained swing 0.1651 vs analytic 0.15718; the old flood had been masking the effect by brute force); the post-FFT 'relaunch at the strongest spectral line' is disabled when driven (a rectifier-like circuit with a dominant 2nd harmonic must not retain the wrong period); the AUTONOMOUS oscillator path is untouched (byte-identical behavior verified on oscillator decks; DC-only-supply circuits never trigger detection). IMPACT: rc_pss ~4 min -> 0.05 s with the retained fundamental EXACTLY 1000000 Hz (the old hunt returned 999999.8976); psp example 406 s -> 0.9 s; the rfpss battery (5 verifies, 61 checks) minutes-each -> <0.5 s each; rfanalyses 'several minutes even Sparse-only' -> 0.56 s; the KLU PSS pass 'prohibitively slow' -> 0.14 s. HARNESS: rfpss and rfanalyses removed from both REGRESSION_EXCLUDE and SPARSE_ONLY -- the whole RF periodic small-signal suite now runs in EVERY regression sweep under BOTH solvers; the direct .pac-on-pumped-varactor check became affordable, closing the E-175 loop on the direct PAC path (sidebands match transient ground truth within 1%). Verified (examples/pssdriven_examples/verify_pssdriven.py, 6 checks x both solvers: driven detection + exact retained frequency, retained swing == |H| within 0.1%, retained-period self-consistency, direct .pac vs transient truth, autonomous non-trigger). Full example regression 145/145.

E-177 *spicelib/analysis/dcpss.c, examples/pnoisefold_examples/**

pnoise noise-folding referee + the folded-flicker frequency fix -- closes the E-175 audit's last honesty-ledger gap: periodic noise analysis (pnoise/qpnoise/phasenoise) was 'correct by construction, not correct by measurement' -- the folding of device noise through LPTV conversion had never been checked against anything independent. THE REFEREE (three independent layers, on a 1-node LPTV-conductance circuit G(t) = g0 + g1*sin(w0 t) chosen so conversion transfers are O(1)): (1) a from-scratch Python conversion matrix, onoise(f) = sum_k |Z_k(f)|^2 * S(|f + k*f0|) -- same physics, zero shared code; (2) a TRNOISE transient Monte-Carlo (no shared code OR theory; 198-segment Welch PSD, with the pumped/unpumped RATIO cancelling the TRNOISE amplitude convention); (3) the LTI (pnoise == .noise) and white limits. VERDICT ON THE WHITE PATH: measured-correct -- pnoise matched the referee to 6 digits at every frequency and the MC arbiter confirmed the ~1.86x folding ratio within statistical error; the adjoint, sideband sum and thermal densities are right. THE BUG THE REFEREE CAUGHT: the stationary sideband loops set the noise-evaluation frequency ONCE, to the output frequency (data.freq = freq outside the k loop). But the sideband-k adjoint carries noise ORIGINATING at |f + k*f0*| -- *a frequency-dependent source PSD (flicker 1/f, noise_table E-109) must be evaluated at the source-side frequency. The proof was digit-exact in both directions: pre-fix pnoise reproduced the referee's deliberately-wrong evaluate-at-f model DIGIT-FOR-DIGIT (1.114323e-14 vs 1.114323e-14 at 1 kHz -- a 21% overestimate on this circuit, unbounded as f « f0); post-fix it reproduces the correct model digit-for-digit (9.175472e-15). Why three generations of checks missed it: white noise is frequency-flat and LTI circuits have no k != 0 transfer -- the THIRD occurrence of the accidental-correctness pattern (E-171 determinant,*

*E-175 parametric term). FIXED in all three stationary loops: pnoise (data.freq = |freq + k*f0| per sideband), qpnoise (|f_in + k1*f1 + k2*f2| per harmonic), and phasenoise -- where it matters most: folded far-sideband blocks near a carrier were evaluated at the tiny OFFSET frequency, inflating the flicker (1/f^3) region. CYCLO SCOPE: the cyclo mode's time-domain identity onoise = (1/P) sum_s S(t_s)|A_s|^2 treats the source PSD as frequency-flat across the folding span -- exact for modulated-white noise and for flicker on conversion-free circuits (the shipped E-125/126 use cases), not for flicker folded through k != 0 conversion (cannot be collapsed into that product with the aggregate device-noise API); documented at both cyclo sites, with the (now exact) stationary mode as the right tool when folded flicker matters. Verified (examples/pnoisefold_examples/verify_pnoisefold.py, 5 checks x both solvers, ~2 s thanks to E-176): white folding vs referee (<=0.1%), flicker folding vs referee (<=0.5%, sample-0 bias read from the retained orbit), pre-fix signature absent, pump/no-pump ratio == referee ratio, LTI limit == .noise. The rfpss/qpss/rccyclo suites pass unchanged (conversion-free or white -- provably unaffected). Full example regression 146/146.*

E-178 *spicelib/analysis/dcpss.c, spicelib/devices/dio/dionoise.c, examples/cyclofold_examples/*, examples/rfpss_examples/, *examples/qpss_examples/**

exact separable cyclostationary folding + the HB DC-source fix -- removes the cyclo mode's frequency-flat limitation E-177 documented. For the separable model S(t,f) = m(t)^2*g(f), onoise(f) = sum_g sum_q |B_q(g)|^2 * g_g(|f + q*f0|) with B_q(g) = (1/P) sum_s m_g(t_s)*dA_s(g)*e^{+jq theta_s} and A_s = sum_k Psi_k e^{-jk theta_s}. Per-generator amplitudes recovered with NO device-API change: the noise-summary machinery (prtSummary/outpVector, one density per generator; family totals excluded by the name-prefix rule) plus load POLARIZATION against a fixed reference load R = Psi_0 -- five DEVnoise sweeps per orbit sample (A+-R, A+-jR, R) give c_g,s = sqrt(S_g(t_s))*dA_s(g) up to a constant phase that cancels in |B_q|^2. Spectral shapes are MEASURED pointwise at the folded frequencies (no 1/f^EF assumption -- noise_table works) at several probe biases; flat generators keep the original time-domain identity (exact for ANY quadratic form incl. NevalSrc2-correlated pairs); stationary limit == the E-177 sum, flat limit == the old identity (Parseval). Ported 2-D to `qpnoise ... cyclo` (B_{q1,q2} at |f + q1 f1 + q2 f2|). THE PHYSICS: 'flicker sees ^2, white sees <m^2>' -- only the envelope's DC feeds the 1/f band; the old identity was 23% high (8/pi^2) on |sin|-modulated flicker and 34% high on the conversion circuit; the new sweep lands on a from-scratch referee to <=3e-4 (the old binary reproduced the deliberately-flat model to 6e-5 -- the error was the model, not the code). BONUS BUG (4th accidental-correctness occurrence): the cross-machinery check (same circuit through the QPSS-HB orbit) came out exactly 4x -- BOTH HB Newtons (`hb` E-134, `qpss ... hb` E-136) double-subtracted the DC sources (the settle-mode rhs folded into I_R already carries them; -lambda*Is subtracts them again): every DC bias voltage converged to exactly 2x, silently corrupting all bias-dependent noise and conversion (flicker ~I^AF off by 2^AF, shot 2x) while AC content and every AC-driven validation stayed exact. Fixed by adding the unscaled DC source block back (net DC drive exactly -lambda*Is_DC); `qpnoise cyclo` now agrees with `pnoise cyclo` DIGIT-FOR-DIGIT across the two orbit machineries. rccyclo's flicker law updated to the exact <|I|>^2 = (8/pi^2)<I^2> (pinned to 1e-4). 6 checks x both solvers; regression 147/147. FOLLOW-UP (same enhancement): the E-139 hard-pumped-diode deck exploded the cyclo path (~6.5e1 V^2/Hz) -- dionoise.c computes its three sidewall generators only when DIOresistSWGiven but the prtSummary block copies ALL DIONSRCS slots, so without a sidewall model the summary vector carried uninitialized stack garbage (latent in stock ngspice for decades; E-178 is the first consumer of per-generator densities). Fixed at the device (unwritten slots zeroed, dio/dionoise.c) and hardened in both cyclo consumers (finite non-negative PSD guards; garbage probe ratios fall back to the flat path). verify_qpnoise's 'cyclo > 2x stationary' diode expectation was an artifact of the garbage slots + doubled HB DC bias; replaced with a closed-form torus-average referee (resistive network => memoryless LPTV, instantaneous adjoints) matched to ~1%.

E-179 *spicelib/analysis/tfanal.c, spicelib/analysis/cktsens.c, frontend/com_measure2.c, examples/*

*stdaudit_examples/**

standard-analyses audit: referee battery + three fixes. The 'Standard analyses (analog)' gap-table rows are 1990s SPICE3 code whose VALUES had never been physics-checked. FIX 1 (tfanal.c, inherited from Berkeley SPICE3 1988): `.tf i(vm) vin` output impedance was ALWAYS 1e20 -- the output-impedance solve forces 1V on the sense branch and inverts the branch current as 1/MAX(1e-20, rhs), but that current is NEGATIVE for every passive network (the input-impedance path right above divides by -rhs); fixed with the same sign + fabs guard; feedback-amp referee exact (1000.990 == RL + node Thevenin). FIX 2 (cktsens.c): KLU AC sensitivity silently truncated to ONE frequency point -- the E-114 KLU complex-conversion block inside the frequency loop reused `i` (the OUTER loop variable) for its DEVmaxnum walk, so after the first point i=DEVmaxnum ended the sweep; the surviving point was correct, which is why E-62's single-point check passed. Fixed with a local index (dead second block hardened too); full sweep == analytic dV/dC = -jwR/(1+jwRC)^2 to 6 digits, both solvers. FIX 3 (com_measure2.c): `.meas DERIV{ATIVE}` was parsed but evaluation fell into an explicit 'currently not supported' stub (empty #if 0 measure_deriv() placeholder) -- implemented: 3-point Lagrange-quadratic derivative on the nonuniform time grid, AT= and WHEN forms sharing FIND's parse/locate flow, verified vs the analytic sine slope to 5 digits; DERIVATIVE/INTEGRAL long spellings accepted (only DERIV/INTEG matched); pre-existing -Wmissing-prototypes warning silenced (str_has_arith_char2 static). MEASURED-CORRECT: .disto HD2/HD3 == analytic diode Volterra kernels <=0.06% incl. a frequency-dependent load (harmonics at Z(2w)/Z(3w) -- no E-177-style frequency bug), SIM2 intermods (f1+f2, f1-f2, 2f1-f2) incl. cascades, exact DISTOF1/DISTOF2 scaling, junction-cap harmonics == Harmonic Balance to ~6 digits (two independent engines); .noise onoise_total^2 == band analytic (kT/C) 0.06% + flicker log-integral 6 digits; .sens DC == central finite differences at a nonlinear OP (5-6 digits, model params incl.); nonlinear-OP .pz works via the driving-point form (the E-62 'quirk' was a bias source shorting the injection node -- ngspice's refusal is correct); tran/op/meas battery exact. 8 checks x both solvers; regression 148/148.

E-180 *frontend/com_checkpoint.c, examples/checkpoint_examples/**

checkpoint/restart under KLU (the last Sparse-only feature falls). E-131 guarded `savestate/loadstate` off under KLU, recording the reason as 'KLU factorization objects absent on the restore path' -- a mis-diagnosis. REAL cause: `.option klu` lives in the task (TSKkluMODE) and reaches the circuit only in CKTdoJob at analysis dispatch, but loadstate calls CKTsetup DIRECTLY first, so the matrix was built Sparse (SMPkluMatrix NULL); the resume dispatch then set ckt->CKTkluMODE=1 without re-setup (by design -- it must preserve the restored state), and the first NIiter dereferenced the NULL SMPkluMatrix (KLUloadDiagGmin store; reproduced EXC_BAD_ACCESS +0x90). savestate was never broken (the checkpoint file holds only solver-agnostic vectors). FIX: com_loadstate copies the task's solver selection and the E-152 KLU knobs (ordering/scale/BTF/memgrow) into the circuit BEFORE CKTsetup, exactly CKTdoJob's block; both guards removed. BONUS: cross-solver restore -- all four save x load combinations (fresh processes) continue exactly on the uninterrupted trajectory (<=1e-9 at t=3.5ms of a 4ms diode+RC run). verify_checkpoint's 'KLU rejected' check replaced by the 4-combo matrix (22/22); solver-notes and gap-analysis tables updated. Nothing in the simulator is Sparse-only any more (E-172 closed the last analysis, E-180 the last feature). Regression 148/148.

E-181 *spicelib/analysis/dctran.c, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, include/ngspice/cktdefs.h, include/ngspice/tskdefs.h, include/ngspice/optdefs.h, examples/corenum_examples/**

core-numerics audit: the integrator certified exactly + **`.options ordfix`**. Nobody had ever verified that the Gear (BDF) corrector coefficients deliver their nominal order (orders 3-6 were dead code for ~30 years until E-128 selected them; E-128 verified order SELECTION, not coefficient CORRECTNESS). NEW OPTION `ordfix=K` (off by default): a fixed-order verification mode -- ev-

ery converged step is accepted (the step is ruled by tmax, not the LTE), the first step is shrunk 1000x (an order-1 starter proportional to the pinned step imposes an O(h^2) floor), the order ramps to K while the step is tiny, then the step grows gently (x1.15; large ratios destabilize high-order variable-step BDF). Each guard defeats a measurement trap that initially presented as a bug but is real numerics (incl. the LTE step preference h_k=(c_k*tol)^{1/(k+1)} INVERTING at loose tolerance -- the E-128 controller's refusal to raise there is correct). CERTIFIED: accepted trajectories satisfy the exact variable-step BDF-k formula (Lagrange differentiation on the actual nodes) to <=1.3e-13 at every order 1-6 (~90 uniform stencils each); measured global slopes confirm orders 1-3; trap-vs-Gear dissipation on a lossless LC matches theory (|R(iy)|=1 vs order-dependent damping). Also referee'd with NO defects: solver precision == conditioning theory under both solvers (1e18-conductance-spread asymmetric mesh == exact-rational MNA to 1e-15); all four DC convergence-aid paths (default/noopiter/gminsteps=0/srcsteps=0) agree on a 40-diode hard-DC chain (spread 2.2e-6, KLU==Sparse 7e-13); xmu=0.5 == trap bit-identical, xmu=0.45 damps; lvltim=1 works. Drive-by: DCtran_step_quit prototype added to cktdefs.h (pre-existing -Wmissing-prototypes). 8 checks x both solvers; regression 149/149.

E-182 *frontend/plotting/plotit.c, frontend/plotting/pyplot.c, examples/pyplot_examples/**

pyplot autoscale by default -- the matplotlib bridge (E-94/95/98/99) pinned every generated axis with set_xlim/set_ylim taken from ngspice's grid-rounded internal plot ranges. User request: rely on matplotlib's autoscaling + fig.tight_layout() instead. plotit() always fills xlims[]/ylims[] (user args OR data-derived), and the statics that distinguish the user case (xlim/ylim from getlims) are freed BEFORE the device dispatch -- plotit now captures user_xlim/user_ylim flags at the free point and passes NULL limits to ft_pyplot unless the user gave xlimit/ylimit explicitly (ft_pyplot already handled NULL; gnuplot/asciiplot back-ends untouched). verify_pyplot +2 checks (no pins + tight_layout on auto plots; exact pins for explicit limits). Regression 149/149.

E-183 *frontend/com_pyplot.c, frontend/plotting/pyplot.c, examples/pyplot_examples/*, features/

pyplot usability: distinct default names, deck-folder output, linewidth & backend. (1) REAL BUG: in window (interactive) mode pyplot launches the Python viewer in the BACKGROUND (`python3 pyplot.py &`); two plots that both omit the file name shared `pyplot.py`/`pyplot.data`, and the 2nd call overwrote them before the 1st viewer read them -- both windows showed the 2nd plot (its title and data). Invisible in PNG mode (synchronous). Fix (com_pyplot.c): a per-session counter names successive no-name plots `pyplot`, `pyplot_2`, ... (first unchanged). (2) The .py/.data/.png are written next to the CIRCUIT FILE via ngdirname(ci_filename) rather than ngspice's cwd; the script's loadtxt/savefig carry the deck-dir path so Python finds the data from any cwd; a bare in-folder deck name still lands in the cwd (unchanged). (3) `set pyplot_linewidth=<w>` -> linewidth in the plot()/step() calls. (4) `set pyplot_backend=<name>` -> matplotlib.use() ahead of import pyplot, overriding the auto backend (incl. the Agg forced by the png/svg/pdf terminals). All four opt-in; default output byte-unchanged. filename buffers in pyplot.c grown 128->1024 for full paths. verify_pyplot grows to 13 checks x both solvers; packaged as a standalone feature bundle under features/. Regression 149/149.

E-184 *frontend/outitf.c, examples/progressbar_examples/**

sweep progress bar reaches 100%. Fix to the E-129 progress bar: it sometimes froze below 100% at end of run (e.g. '[====] 82%' then 'No. of Data Rows'). CAUSE: the 'Reference value' status line is throttled -- redrawn only when >0.25 s has elapsed since the last update -- so the FINAL sweep point's print is almost always skipped (it lands within 0.25 s of the previous tick), freezing the bar at the last throttled value; reaching 100% was never reliable. FIX: outp_print_reference records the latest refval + a bar-shown flag every call; a new outp_finish_reference, called from fileEnd/plotEnd just before the 'No. of Data Rows' line, reprints the bar full at 100% in place using the sweep's TRUE endpoint (TRAN CKTfinalTime, AC ACstopFreq, NOISE NstopFreq, DC last source value). One-shot (reset at analysis start and after firing), silent for non-swept analyses (op/tf), same ft_optimizing/ft_norefprint/cp_background guards -- quiet/background/optimizer

runs and the rawfile unaffected. verify_progressbar's end-of-run check tightened from >=90% to ==100% (all four swept analyses); 22/22. Regression 149/149.

E-188 *spicelib/analysis/dcop.c, include/ngspice/cktdefs.h, frontend/com_sweep.c, examples/warmstart_examples/**

Monte Carlo warm-start (**`montecarlo ... -warm`**). Each MC sample (E-151) re-sources the deck and COLD-solves its DC bias point, running the full gmin/source-stepping homotopy from scratch even though consecutive samples move the operating point only slightly. Profiling: the solve, not the deck re-source, dominates for non-trivial circuits, and every `op` cold-starts -- two IDENTICAL ops back-to-back each burn the full homotopy (~52 Newton iters on a diode ladder; a warm .nodeset at the solution cuts it to ~5). CKTop tries a direct Newton from the caller's firstmode before any homotopy, but DCop always passes MODEINITJCT (cold junction voltages), discarding the previous solution. FIX: `montecarlo -warm` wraps the loop in CKTsetWarmStart(1/0); DCop keeps a buffer (outside the CKTcircuit that reset recreates; indexed by equation number, stable across resets of an identical-topology deck) of the last converged CKTrhsOld, preloads it and calls CKTop with MODEINITFLOAT (use existing node voltages) when a valid same-size guess exists, and snapshots each converged solution. A poor guess just fails the first NIiter and falls through to the normal cold homotopy, so the converged point is identical -- only a cheap failed attempt is wasted. First sample cold, rest warm; when -warm is absent dcop_warm_enable=0 and the path is byte-identical (one extra SMPmatSize read). CORRECTNESS: warm changes only Newton's starting point, so it converges to the same OP to within convergence tolerance -- warm==cold yield EXACTLY at reltol=1e-6, agreeing to within a couple boundary samples at default reltol (v(3)~3.8V, where reltol*|v|~=the narrow spec band; a tolerance effect, removed by tightening reltol). Iteration count ~52->~4 (~13x); wall-clock benefit scales with per-iteration cost (large/hard-converging designs win most; small circuits are dominated by deck re-source + command dispatch). Shared DC-op code, so Sparse+KLU; composes with -lhs. 5 checks (exact at tight reltol, close at default, iteration cut, +lhs, +klu). Regression 152/152.

E-189 *frontend/com_sweep.c, examples/sweepwave_examples/**

Sweep waveform overlay (**`sweep ... -overlay`**). A usability follow-up to the E-146 `sweep` command: `sweep` steps any knob, runs an inner analysis per point, and records each output's LAST value into a summary `sweep` transfer curve -- but overlaying the full per-point WAVEFORMS to compare shapes meant `setplot`-ing each retained run by hand. `-overlay` (alias `-ov`) collects every point's whole output waveform, resamples them onto a common independent-variable grid, and builds one `sweepwave` plot with a `<output>_<value>` vector per (output, knob value), left current so `plot <out>_<val> ...` works at once -- the HSPICE `.step` overlay in one step. Three static helpers in com_sweep.c: `sw_eval_vec` (evaluate the output expr and copy its FULL waveform + scale into caller buffers; complex/AC stored as magnitude), `sw_interp` (binary-search piecewise-linear resample, flat outside range -- each run lands on its own adaptive time/freq grid, so a shared grid is required before a common scale vector), `sw_wavename` (clean nutmeg name, non-alnum -> _). The sweep loop captures each waveform alongside the unchanged last-value recording; after the loop a common grid over [min,max] of all runs at the finest resolution is built, every waveform resampled onto it. GRACEFUL: a scalar inner analysis (`op`, no waveform) prints "-overlay ignored (analysis '...' has no waveform to overlay)" and yields the ordinary summary -- never an error. Without the flag the path is byte-identical to E-146. Front-end only, solver-independent. Verify: overlaid RC step responses match `1-exp(-t/RC)` (RC=1/2/4us) to worst <1e-4, distinct per-value vectors, graceful op, summary curve intact. 5 checks. Regression 153/153.

E-190 *frontend/com_sweep.c, examples/nestedsweep_examples/**

Nested multi-knob sweep (**`sweep ... -vs ...`**). Second usability follow-up to the E-146 `sweep` command (after E-189 `-overlay`). `sweep` stepped ONE knob; `-vs <knob> <spec>` (alias `-family`) adds OUTER knobs whose CARTESIAN PRODUCT forms a curve family: the inner (positional)

knob stays the x-axis of the summary `sweep` plot, and each combination of the outer knobs yields one curve per output, named `<output>_<outerknob>_<value>...` -- the parametric family of `.dc ... SWEEP ...` / HSPICE nested `.dc`. Up to SW_MAXKNOB=4 knobs, cartesian points capped at SW_MAXPTS. The three-form spec parser (lin/dec/oct, list, start/stop/step) was factored into `sw_parse_spec`, now shared by the inner knob and each `-vs` knob so all knobs accept the same forms. SET ORDERING (the subtle part): each knob is applied with its E-146 mechanism -- a `.param` needs alterparam+reset (re-source), an instance/model knob is in-place alter/altermod -- and a reset drops every in-place alter, so the loop iterates inner-fastest and re-sources ONCE only when a `.param` knob's value actually changed (outer-loop boundaries; an inner `.param` still resets every point, a pure in-place sweep never resets), staging all `.param`s via alterparam before the single reset, then RE-APPLYING every in-place knob after the reset (so an inner alter survives an outer `.param` re-source). By construction a single knob reduces EXACTLY to E-146: one outer combination -> curve named `<output>`, E-146 messages, identical data indexing (existing sweep/sweepwave examples pass untouched). `-overlay` composes: each cartesian point's full waveform becomes a `sweepwave` vector named with EVERY knob value (inner first), so a single knob is `<output>_<value>` exactly as E-189. New static helpers: `sw_parse_spec`, `sw_set_deck`/`sw_set_inplace` (split from the old `sw_set`), `sw_familyname` (summary curve name over outer knobs), `sw_pointname` (overlay name over all knobs; replaces `sw_wavename`). Verify: RC low-pass step, R inner x C outer, each family curve == `1-exp(-T/(R*C))` vs R to worst <5e-6, incl. a `.param`-OUTER knob (reset/alter ordering); cartesian run/curve counts; `-overlay` family names carry both knob values; single knob still names its curve `vo`. 5 checks. Regression 154/154.

E-191 *spicelib/analysis/acan.c, spicelib/analysis/span.c, examples/aclin2_examples/**

`.ac lin 2` / **`.sp lin 2`** off-by-one fix. Found while auditing the data-output path (`wrdata`/`wrsnp`/`save`/rawfile `write`): a ragged-length `wrdata` test wrote an AC vector one point short, which traced not to the writer but to the AC LINEAR frequency sweep. The AC (`acan.c`) and S-parameter (`span.c`) analyses computed the linear step behind `if (numberSteps - 1 > 1)` -- true only for N>=3 -- so exactly `lin 2` fell through to the single-point patch (`freqDelta = 0`). With a zero step the sweep loop (`freq += freqDelta; if (freqDelta == 0) goto endsweep;`) stops after ONE point, silently dropping the `fstop` point: a two-point linear AC/SP sweep produced one row. `dec`/`oct` (different step formula) and `lin 1/3/4/...` were all correct; only N=2. The single-point patch (Richard McRoberts) was meant for `lin 1`; `-1 > 1` over-reached by one. FIX: guard on `numberSteps > 1` in both files (the form `noisean.c` already used correctly), so N=1 -> one point at fstart (patch preserved), N=2 -> freqDelta = fstop-fstart -> two endpoints, N>=3 unchanged. The distortion analysis uses a different N+1-point convention (unaffected); noise was already correct. Verify: RC low-pass, both `lin 2` endpoints are GENUINE solved points matching `1/(1+jwRC)` to <1e-6 (not a duplicate); `lin 1` stays one point; `lin N` gives N linearly-spaced points (N=3,5,10); `.sp lin 2` gives two points with analytic series-R S-params (S11=1/3, S21=2/3). Both solvers. The wider output-path audit found the writers themselves CLEAN: Touchstone `wrs2p`/`wrsnp`/`rdsnp` round-trip across RI/MA/DB, S/Y/Z (Rbase normalization Y*R / Z/R), Hz-GHz incl. complex angles; 2-port special S11 S21 S12 S22 order + N-port row-major both correct for round-trip AND spec interop; `wrdata` complex `(re,im)`/singlescale/ragged padding; `save` filtering restricts the vector set; rawfile `write`/`load` bit-exact at 12 digits. 5 checks. Regression 155/155.

E-192 *frontend/com_checkpoint.c, frontend/com_checkpoint.h, frontend/runcoms.c, examples/autosave_examples/**

Auto-checkpoint on interrupt (**`set autosave=<file>`**). Follow-up to E-131 checkpoint/restart, prompted by "does savestate fire on Ctrl-C?" -- it did not. E-131 gave `savestate`/`loadstate` but a checkpoint had to be taken by hand, so an interrupted long transient lost its progress. E-192: `set autosave=<file>` makes an interrupted transient auto-write an E-131 checkpoint. SAFE HOOK POINT (the crux): writing a file from a signal handler is undefined behavior, so E-192 does NOT checkpoint in `ft_sigintr` -- that handler only sets `ft_intrpt` and returns (during a run `ft_`

setflag is TRUE, so no longjmp). At the next ACCEPTED timepoint, dctran's IFpauseTest (which is OUTstopnow) sees the flag and returns 1, so dctran returns E_PAUSE; if_run maps E_PAUSE to return code 1; dosim (runcoms.c) reports "interrupted" (err equals 1). E-192 hooks exactly that interrupted branch -- on the MAIN thread, at a consistent timestep boundary, with ordinary buffered I/O. CHANGE: com_savestate's body factored into ckt_write_checkpoint(ckt, fname) (the command is now a thin wrapper); the hook calls the SAME function, so an autosave file is byte-for-byte what savestate writes. In the interrupted branch: if cp_getvar("autosave", ...) yields a filename AND the run was a transient (CKTmode and MODETRAN set, CKTtime above 0), call ckt_write_checkpoint. OPT-IN (no var means byte-identical old behavior, nothing written), TRANSIENT-ONLY (the MODETRAN / CKTtime guard blocks stale state after, e.g., an interrupted AC). Interactive-mode only: the SIGINT handler is installed only under non-batch mode, so a batch-mode Ctrl-C still terminates (unchanged). A real Ctrl-C (SIGINT), a stop when ... breakpoint, and a Verilog-A stop task all funnel through the same interrupted branch (they differ only in how the flag is set, upstream of E-192). CORRECTNESS: autosave file byte-identical to a manual savestate at the same pause; loadstate resumes it exactly (E-131 round-trip). Verify (examples/autosave_examples): stop-paused transient writes a checkpoint; equals a manual savestate (byte-identical); loads/resumes; opt-in gate (no var means no file); and a REAL Ctrl-C (SIGINT) via a PTY (interactive) fires it (lenient: skip if the run finishes before the signal, the hook being covered deterministically). E-131 checkpoint example still passes (savestate refactor). 5 checks. Regression 156/156.

E-193 *spicelib/analysis/dcpss.c, examples/pnoiseunits_examples/*, examples/rfpss_examples/verify_rcpnoise.py, examples/rfpss_examples/verify_rccyclo.py, examples/cyclofold_examples/verify_cyclofold.py, examples/pnoisefold_examples/verify_pnoisefold.py, examples/rfpss_examples/rc_pac.cir*

.pnoise honors **sqrnoise** (noise-density units fix). Found auditing the RF/PSS suite. The ngspice manual (sec. .noise, pp. 326-327) defines onoise_spectrum/inoise_spectrum as a noise spectral DENSITY in V/sqrt(Hz) (or A/sqrt(Hz)) by default, switchable to the squared V^2/Hz form by the sqrnoise control variable. .noise obeys this (noisean.c reads cp_getvar("sqrnoise") and sqrt's onoise/inoise when unset). But .pnoise (periodic noise) ALWAYS emitted the squared V^2/Hz density and IGNORED sqrnoise -- so the same-named vectors carried DIFFERENT units across the two analyses (differing by a square), and the documented sqrnoise control silently did nothing for pnoise. Confirmed by the toggle: .noise flips 4.063e-9 (V/sqrt Hz) <-> 1.651e-17 (V^2/Hz) with set sqrnoise; .pnoise returned 1.651e-17 both ways. FIX: pnoise_sweep (dcpss.c) now reads sqrnoise via cp_getvar (added #include cpextern.h) and, when unset (the default), applies sqrt to the emitted onoise/inoise densities -> V/sqrt(Hz); set sqrnoise -> V^2/Hz. BOTH emit paths get the conditional sqrt: the E-178 cyclostationary folding branch and the standard adjoint-folding branch. Now .pnoise and .noise agree by default (match to 0.00). SCOPE: .pnoise only. The two-tone qpnoise command's single-point diagnostic already prints BOTH forms explicitly (V^2/Hz and V/sqrt(Hz)) so there is no ambiguity, and its swept + single-point paths share the V^2/Hz convention; an initial QPnoiseSweep change was REVERTED to avoid an internal swept-vs-single-point split (verify_qpss_sweep [3] compares them). EXAMPLES: rc_pnoise now checks the default V/sqrt(Hz) form (sqrt of the analytic) and cross-checks vs .noise without forcing sqrnoise (match to 0.00); rc_cyclo/cyclofold/pnoisefold are power-density relations (onoise*f* == ..., *referee sums* |*Z*|^2S) so they set sqrnoise to keep V^2/Hz, and two LTI-limit checks that compared pnoise(V^2) to .noise(V/sqrt Hz)^2 became direct equalities. WIDER AUDIT (all correct): PAC driving-point + the +-1 CONVERSION SIDEBANDS (via the .pac card's trailing maxsideband arg) match a clean transient DFT to ~0.1%; Harmonic Balance matches a transient DFT to 4-5 digits (an apparent 12x .disto HD3 and 2.4x PAC-sideband discrepancy both dissolved into fourier-command interpolation error at high harmonics, resolved by a proper DFT). Also documented the previously-undocumented .pac trailing maxsideband arg (dcpss.c comment + rc_pac.cir). Verify (examples/pnoiseunits): default pnoise == sqrt(4kTR/(1+(wRC)^2)) (V/sqrt Hz); set sqrnoise == the density (V^2/Hz);

default^2 == squared (exact sqrt relation); default pnoise == default .noise (0.00). 4 checks. Regression 157/157.

E-194 *frontend/com_optimize.c, examples/psoopt_examples/**

Particle swarm optimization (**`optimize -method pso`**). A new search method for the built-in optimizer. E-130/143 gave two LOCAL methods -- Nelder-Mead simplex (`-method nm`, scalar `-minimize`) and Levenberg-Marquardt (`-method lm`, gradient least-squares over `-targets`) -- both of which descend into whichever basin the start point sits in, so on a MULTIMODAL objective they return the local minimum nearest the start. E-194 adds a GLOBAL, population-based, derivative-free method: PSO. N particles fly through the normalized [0,1]^np box; each keeps its own best-seen point (pbest) and the swarm shares a global best (gbest); each iteration updates every velocity `v = chi*(v + phi*r1*(pbest-x) + phi*r2*(gbest-x))` and moves `x = clamp01(x+v)` with the standard Clerc-Kennedy constriction (chi=0.72984, phi=2.05 so the swarm converges without exploding), velocities clamped to half the box, positions to [0,1]. Particle 0 starts at the user init point, the rest uniform-random, so the whole box is explored. Reuses the existing `opt_eval` (returns the scalar cost for both objective kinds), so PSO works for scalar `-minimize` AND `-target` least-squares -- unlike LM, which needs targets. New options: `-swarmsize <N>` (aliases `-swarm`/`-npart`; default auto = 10+4*np capped at 60), `-seed <s>` *(PSO draws from a small self-contained splitmix64 PRNG independent of ngspice's global RNG, so a seed gives byte-identical results), shared* `-maxiter`/`-tol` *(tol is the gbest-stagnation tolerance held over several iterations). The nm/lm dispatch is unchanged. Verify (examples/psoopt): on the classic multimodal* `f(p)=sin(p)+sin(10*p/3)` *over [2.7,7.5] (global p*=5.1457 f*=-1.8996, several higher local minima) started at the p=2.7 corner, PSO finds the global (-1.8996, p=5.145) while Nelder-Mead from the same start is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is exactly reproducible; independent seeds all reach the global basin; PSO also solves a `-target` least-squares objective (recovers p to a 1e-12 residual) and a 2-D separable multimodal min (f*=-3.799). Existing optimize example (nm+lm, 20 checks) unchanged. 6 checks. Regression 158/158.

E-195 *frontend/com_optimize.c, examples/deopt_examples/**

Differential evolution (**`optimize -method de`**). A second GLOBAL method for the built-in optimizer, after PSO (E-194), from the same population-based derivative-free family. DE keeps a population of NP vectors in the normalized [0,1]^np box; where PSO pulls trials toward remembered bests, DE (classic DE/rand/1/bin) builds each trial from a scaled DIFFERENCE of random members and crosses it with the target: `v = a + F*(b-c)` (F=0.8; a,b,c distinct and != i), `u[j] = v[j]` if `rand<CR or j==jrand` else `x[i][j]` (CR=0.9 binomial crossover, one dimension always taken), then greedy selection (keep u in place of x[i] iff cost(u) <= cost(x[i])). The difference vector b-c self-scales to the population's own spread -- large while members are far apart (broad exploration), shrinking as they converge (fine refinement) -- which makes DE robust on rugged / discontinuous / poorly-scaled landscapes with no per-problem step tuning. Reuses the existing opt_eval (scalar cost for both objective kinds), so it works for scalar `-minimize` AND `-target` least-squares like PSO. Shares PSO's infrastructure: the same self-contained splitmix64 PRNG (so `-seed` is byte-reproducible), the `-swarmsize` population option (default auto 10+4*np capped at 60; clamped to >=5 because DE needs 4 distinct members to form a mutant), and shared `-maxiter`/`-tol` *(gbest-stagnation). The nm/lm/pso dispatch is unchanged; DE is only selected by* `-method de` *(aliases diffevol, differentialevolution). Verify (examples/deopt): on the multimodal* `f(p)=sin(p)+sin(10*p/3)` *over [2.7,7.5] (global p*=5.1457 f*=-1.8996) started at the trapping p=2.7 corner, DE finds the global (-1.8996, p=5.14574 -- matches p* to 5 digits) while Nelder-Mead from the same start is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is exactly reproducible; independent seeds all reach the global basin; DE also solves a `-target` least-squares objective (recovers p to a 4e-13 residual) and a 2-D separable multimodal min (f*=-3.799); and DE and PSO both reach the global while the local NM does not. Existing optimize (nm+lm, 20 checks) and psoopt (6 checks) examples unchanged. 7 checks. Regression 159/159.

E-196 *frontend/com_optimize.c, examples/saopt_examples/**

Simulated annealing (**`optimize -method sa`**). A third GLOBAL method for the built-in optimizer, completing the set: after PSO (E-194) and DE (E-195), both population-based, SA is the classic SINGLE-WALKER global optimizer. It holds one current point in the normalized [0,1]^np box; each step it proposes a random neighbour `xn = clamp01(x + step*U(-1,1))` and applies the Metropolis rule -- accept if `cost(xn) <= cost(x)` (downhill), else accept anyway with probability `exp(-(cost(xn)-cost(x))/T)` (uphill). While the temperature T is high uphill moves are readily accepted so the walker climbs OUT of local minima; T is then cooled geometrically (`T *= alpha`, alpha set so T drops ~4 decades over the -maxiter cooling levels, with L=8+4*np moves per level), so uphill moves become rare and it settles into a minimum. The best point ever visited is returned. ONE candidate per step (no population) -> the lightest-weight global method, the natural choice when each analysis is expensive. Everything auto-scales to the problem: T0 = mean |Dcost| of a dozen random probes (so an uphill move of that size is accepted about half the time when hot), and the step size = `0.30*sqrt(T/T0)+0.02` shrinks as T cools (broad exploration hot, fine refinement cold) -- nothing to tune. Reuses opt_eval (scalar cost both kinds) -> works for scalar `-minimize` AND `-target` least-squares. Shares the self-contained splitmix64 PRNG (`-seed` byte-reproducible); `-maxiter` is the number of cooling levels. The nm/lm/pso/de dispatch is unchanged; SA is only `-method sa` (aliases anneal, simulatedannealing). A single walker refines more loosely than a population, so SA reliably reaches the global BASIN rather than machine precision (a short nm/lm polish from its result sharpens it when that matters). Verify (examples/saopt): on the multimodal `f(p)=sin(p)+sin(10*p/3)` over [2.7,7.5] (global p=5.1457 f*=-1.8996) from the trapping p=2.7 corner, SA finds the global basin (f<-1.85, |p-p*|<0.05) while Nelder-Mead is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is reproducible; independent seeds all reach the global basin; SA solves `-target` least-squares and a 2-D multimodal min; and all THREE global methods (sa/pso/de) reach the global while the local nm does not. Existing optimize (20) + psoopt (6) + deopt (7) examples unchanged. 7 checks. Regression 160/160.

E-197 *frontend/com_optimize.c, examples/opt100_examples/**

A 100-parameter circuit optimization (raised **`optimize`** limits). The built-in optimizer sized its per-run arrays from two compile-time macros; E-197 raises them from OPT_MAXP 16 to 128 (parameters) and OPT_MAXT 64 to 128 (least-squares targets), and lifts the auto-population cap for the PSO/DE population methods from 60 to 256, so a genuinely large problem fits in a single call. Exceeding a cap is reported cleanly (`optimize: too many -param (max 128)`), not crashed. Testbed: 100 independent 1 mA sources, each into an unknown resistor R_i to ground (so v(n_i) = 1e-3 * R_i), fitted to 100 targets t_i = 1e-3 * (100 + 9800*i/99) ohm -- a linear voltage ramp 0.1 .. 9.9 V, a well-posed separable 100-D least-squares. Levenberg-Marquardt is the right tool for a well-posed high-D fit and solves it to machine precision (sum-sq residual 2.5e-29, rms 5e-16, 619 evaluations, ~2 s) with all 100 parameters and 100 targets honored. Two fixes make the derivative-free GLOBAL methods FUNCTION at that scale (they are intrinsically far slower than LM in high dimension, but were failing outright): (1) adaptive DE crossover -- classic DE/rand/1/bin uses CR=0.9, mutating almost every coordinate of a trial; in high dimension a trial that perturbs ~n coordinates at once is essentially always worse than its parent and rejected by greedy selection, so DE freezes at its starting guess. E-197 sets `CR = (n <= 16) ? 0.9 : 15.0/n`, capping the expected mutated-coordinate count at ~15 for large n (exactly the classic 0.9 for n <= 16, so low-D DE is unchanged), and DE descends again -- from ~1240 to ~882 over 35 generations at 100-D. (2) Dimension-scaled convergence patience -- high-D population runs plateau for several generations between improvements, so the gbest-stagnation counter that stops PSO/DE would cut a still-descending run short; the threshold now grows as `8 + n/4` (exactly 8 for n <= 3, so small-n tests are unchanged), giving 33 generations of patience at n=100. DE at 100-D remains a genuinely hard global search that does not fully converge in a short budget -- intrinsic to global optimization in high dimension, not a defect; the message is the division of labor between LM (well-posed high-D fits) and the global methods (multimodal, now usable as dimension grows). Verify (examples/

opt100): LM fits all 100 params to 100 targets at machine precision (both raised caps proven at once); DE descends substantially at 100-D (self-calibrated against its own starting cost, >= 15% reduction); a fixed `-seed` is byte-reproducible even at 100 dimensions; and a 129-parameter over-cap run is reported cleanly. Existing optimize (39) + psoopt (6) + deopt (7) + saopt (7) examples unchanged. 4 checks. Regression 161/161.

E-198 *frontend/com_stb.c, frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/stb_examples/**

`stb` stability / loop-gain analysis. A new front-end command that measures a feedback loop's small-signal loop gain T(f) and reports its phase and gain margin -- the everyday stability check for op-amps, regulators, PLLs -- which ngspice lacked (you had to hand-roll a `.control` script). It is the one-command equivalent of Spectre's `stb` (Middlebrook-Tian double injection). Usage `stb <Vprobe> <Iprobe> <ac-sweep>`: the user marks the loop break with a bias-transparent probe pair between the driving node A and the loaded node B -- a series `Vstb A B dc 0 ac 0` (+node = driver A, -node = load B) and a shunt `Istb 0 B dc 0 ac 0` (ground to load B). Breaking a loop and injecting a single test signal mis-reads the loop gain because of the LOADING at the break (a series voltage injection is exact only from a zero-impedance source into an infinite-impedance load); the double injection cancels that error. `stb` runs two AC sweeps by altering the probe AC magnitudes: voltage injection (Vstb ac=1) gives Tv = -v(A)/v(B); current injection (Istb ac=1) gives the current loop gain from the probe branch current a = i(Vstb) as Ti = -a/(a+1); combined T = (Tv*Ti - 1)/(Tv + Ti + 2). The combined T is INDEPENDENT of where the probe sits in the loop -- the property a single injection lacks -- verified directly (a loaded break gives the same margins as a clean break in the same loop). Numerically, the common clean-break case has a high load impedance so a -> -1 (all injected current returns through the shorted voltage probe) and Ti -> inf, which makes Ti = -a/(a+1) divide by zero; `stb` evaluates the algebraically identical singularity-free form T = -(Tv*a + a + 1)/(Tv*a + Tv + a + 2), exact at a = -1 where it reduces to T = Tv. The complex loop gain is stored as vector `loopgain` vs `frequency` in a new `stb` plot (`plot db(loopgain)` / `plot 180/PI*cph(loopgain)`); phase margin = 180 + angle(T) at the first abs(T)=1 crossing, gain margin = -abs(T)_dB at the first angle(T)=-180 deg crossing (both log-interpolated on an unwrapped phase). Implementation (`com_stb.c`, registered in `commands.c`, declared in `com_commands.h`): recovers the probe's two node names via `CKTinst2Node`, dispatches `alter/ac` through the command table (as com_optimize/com_sweep do), and reads the complex vectors v(A)/v(B)/Vstb#branch/frequency with `ft_evaluate`. Reuses the AC analysis, so both the Sparse and KLU solvers. Verify (examples/stb): PM/GM match the closed-form loop gain of an ideal-output op-amp loop; a loaded break (finite op-amp output impedance) gives the SAME margins as a clean high-impedance break (proving the current injection corrects the loading -- a single voltage injection is ~10% off and diverges past crossover); a 10x-gain design's reduced phase margin is tracked and matches analytics; the complex `loopgain` is stored; and an unknown probe is reported cleanly. 5 checks. Regression 162/162.

E-199 *examples/nport_examples/snp2va.py, examples/nport_examples/**

N-port Touchstone device (snp2va.py). Instantiate a measured or simulated S-parameter block (a filter, cable, connector, package, or amplifier stored in a Touchstone .sNp file) as a circuit element that works in AC AND transient. ngspice had no native n-port element: its .sp "ports" are tagged voltage sources used to MEASURE a circuit's S-parameters, and rdsnp/wrsnp (E-64/E-72) are reader/writer commands, not a device. Rather than a native convolution device (with its own history buffers and stability headaches), snp2va.py converts the file into a Verilog-A n-port realized with laplace_nd, so OpenVAF's OSDI laplace machinery (E-31 complex poles) supplies BOTH the AC response and the transient (recursive-state) response with no convolution code. Pipeline: parse Touchstone (any port count; S/Y/Z data; MA/DB/RI formats; Hz..GHz; arbitrary reference impedance) -> S(f) -> Y(f)=(1/z0)(I-S)(I+S)^-1 -> common-pole vector fit (Gustavsen) -> emit I(p_i) <+ sum_j [laplace_nd(V(p_j), num_ij, den) + e_ij*ddt(V(p_j))]. Two numerical details are essential. (1) The vector fit MUST be done in NORMALIZED frequency -- without it the se column

is ~1e9 while the 1/(s-p) columns are ~1e-7, so the least-squares is hopelessly ill-conditioned and the poles never relocate (the pole relocation is done as polynomial roots of the sigma numerator via Durand-Kerner, not a matrix eigensolve). (2) Y is IMPROPER whenever a network has shunt capacitance (Y ~ sC at high frequency), which laplace_nd -- a ratio of polynomials -- cannot represent (it gave a 63 percent AC error at the band edge); the fix is to give laplace_nd only the strictly-proper (pole) part and emit the s*e term as an explicit ddt (a capacitance), dropping the AC error to ~1e-6. HARDENING: automatic order selection CLIMBS the pole count to capture high-order and delay-dominated responses (a pure delay is uniformly poor at low orders, so the selection must not quit early on a small between-order improvement), but stops at the KNEE once the error has already come down near a floor -- past that knee extra poles just fit measurement noise into an over-fitted, ill-conditioned, unstable model, so stopping there fits clean data to machine precision and noisy data at its noise floor. It breaks when a fit turns unstable (polynomial-root finding degrades at high degree) and returns the best STABLE fit. Stability is enforced (right-half-plane poles reflected into the left half plane) so the emitted model is always BIBO-stable even for a noisy non-passive fit; passivity is checked and reported (enforcement by residue perturbation is a documented non-goal for now). Pure Python standard library, NO numpy: least-squares via Householder QR, complex matrix inverse via Gauss-Jordan, and polynomial roots via Durand-Kerner are all reimplemented (each validated against numpy to machine precision during development). Verify (examples/nport): each check generates a Touchstone file from a network whose response is known exactly, runs snp2va.py + OpenVAF, and confirms the device matches the ORIGINAL network in ngspice -- an R-L-C resonator to 5e-6 in AC (including the transmission peak) and 5e-3 in transient (one model, both analyses); a 5-pole LC ladder (order selection scales past 2 poles); a 3-port star network to 4e-9 on both coupled outputs; and a noisy measurement fit at its noise floor, reported non-passive, staying bounded in transient. Stress-tested to near machine precision on resonators, notches, high-Q blocks, and multi-pole ladders; a delay-dominated transmission line degrades gracefully (AC accurate over the fitted band, transient pulse edges ring -- inherent to rational fitting of a pure delay, for which a T/LTRA line is the right model). 6 checks. Regression 163/163.

E-200 *src/frontend/snp2va.c, src/frontend/snp2va.h, src/frontend/com_presnp.c, src/frontend/commands.c, src/frontend/inp.c, src/frontend/inpcom.c, src/frontend/Makefile.am, examples/presnp_examples/**

Built-in pre_snp command. Folds the E-199 Touchstone-to-Verilog-A converter into ngspice itself as a C command, so the workflow no longer needs an external Python script: inside a .control block, pre_snp <file.sNp> [module] parses the .sNp, common-pole vector-fits it, writes the .va, and invokes openvaf-r to compile the .osdi -- a one-line analog of pre_osdi. snp2va.c is a faithful C port of snp2va.py with identical numerics: it uses C99 double *Complex (typedef'd cplx, so it does not collide with ngspice's own complex.h macros) and reimplements the three vector-fitting primitives -- least-squares via Householder QR, complex matrix inverse via Gauss-Jordan, polynomial roots via Durand-Kerner -- plus the same parse -> S(f) -> Y(f)=(1/z0)(I-S)(I+S)^-1 -> vector fit -> emit I(p_i) <+ sum_j [laplace_nd(V(p_j),num,den) + e_ij*ddt(V(p_j))] pipeline, the same normalized-frequency fit, the improper es *(shunt-C) term split out as an explicit ddt, and the same automatic order selection returning the best STABLE fit (RHP poles reflected -> BIBO-stable). It was checked numerically identical to the Python reference (resonator RMS 4e-6, ladder 7e-7, 3-port 3e-8). The public entry point snp2va_convert(snpfile, vafile, module, msg, msglen) is called by the command wrapper com_pre_snp (com_presnp.c), which then shells out to openvaf-r. ORDERING: the command is registered under the name snp, so writing pre_snp in a deck triggers ngspice's existing pre* mechanism (inp.c strips the pre_ prefix and runs the command before the circuit is parsed). That alone is not enough -- pre_snp must run before the pre_osdi that loads its output, and control lines otherwise execute in deck order -- so the collected pre-commands are now executed in TWO PASSES: pass 0 runs every snp command (in deck order), pass 1 runs the rest. The .osdi a pre_snp generates is therefore guaranteed to exist by the time any pre_osdi loads it, EVEN IF the deck lists the pre_osdi line first; the ordering is a guarantee, not a convention the user must remember. snp/pre_snp are

also added to inpcom.c's case-preservation list (beside osdi/pre_osdi) so the file-path argument keeps its original case. The compiler is located by find_openvaf(): the openvaf ngspice variable (set openvaf=... in spinit or on the command line -- a set inside .control runs too late, after the pre-commands), then the OPENVAF environment variable, then $SPICE_LIB_DIR/openvaf-r, then PATH; if none resolve, pre_snp reports the failing openvaf-r invocation and lists all four options. Verify (examples/presnp): each check builds a Touchstone file from a network whose response is known exactly, lets pre_snp do the whole convert+compile+load inside ngspice, and confirms the device matches the ORIGINAL network -- pre_snp writes the .va and compiles the .osdi; the device matches an R-L-C resonator to 5e-6 in AC (including the transmission peak) and to 5e-3 in transient (one compiled block, both analyses); a 3-port star network compiles and matches on both coupled outputs to 4e-9; and with pre_osdi written BEFORE pre_snp the model still loads, proving the ordering guarantee. 5 checks. Regression 164/164. HARDENING (from an N-port stress sweep -- generate an N-port RLC network, extract S-params with .sp, round-trip through pre_snp, compare DC/AC/transient vs the original subcircuit): two defects the 2-pole checks never reached, both fixed in snp2va.c. (1) Order-selection double-free: the climb that keeps the best STABLE fit let its best and prev buffers alias and then freed the same allocation twice, so pre_snp aborted (SIGABRT) on any network whose fit needed >=3 pole orders (the 2-pole resonator/star escaped by breaking at tolerance first); fixed by not freeing a buffer the other pointer still holds. (2) Transient divergence from a non-passive realization: each Y_ij is fit independently, so the model could blow up in transient (v -> ~1e284) even though every pole is in the LHP and AC is exact -- a single degree-Np rational per element has coefficients spanning ~abs(p)^Np (~1e79 for 8 poles at 1e10 rad/s, an ill-conditioned companion form), and the improper e*s terms form a capacitance matrix with no passivity constraint (an indefinite "negative-capacitance" matrix -> unstable DAE); fixed by realizing each element as a PARALLEL bank of first/second-order laplace_nd sections (every coefficient <= O(abs(p)^2)) and projecting the e-matrix onto the symmetric PSD cone (symmetrize -> Jacobi eigendecompose -> clamp negative eigenvalues to 0). The presnp example gained a 4-port coupled-ladder check (fit climbs past two orders; transient must stay bounded and match) that fails on the pre-fix converter. Regression 164/164.

E-201 *src/frontend/snp2va.c, examples/presnp_examples/**

pre_snp scalability (fast vector fitting + shared-pole realization). The E-200 converter struggled from about 8 ports up (an 8-port took ~190s and larger port counts were infeasible). Two things scaled badly, both in snp2va.c: the pole identification stacked all N*N elements into one dense least-squares of size (2*NsNN) x (NN(Np+2)+Np) -- O(N^4) memory, O(N^6) compute, re-solved dozens of times over the order climb (~8TB of RAM at N=100); and the emitted model gave every one of the N*N elements its own laplace_nd bank -- O(N^2*Np) filter sections and OSDI state. Four fixes rebuild the scalability. (1) FAST VECTOR FITTING (Deschrijver/Gustavsen): the stacked pole-ID matrix has block-arrow structure -- each element's residue columns are independent and couple only through the shared common-pole (sigma) columns -- so instead of forming that matrix, each element block is Householder-reduced on its own (a small 2Ns x (Np+2) QR) and only its residual rows (orthogonal to that element's own columns) are stacked into one tall ((2Ns-Np-2)*Ne) x Np system solved for sigma; memory O(N^4)->O(N^2), compute O(N^6)->O(N^2*NsNp^2). (2) RECIPROCITY: a passive network has symmetric Y, so when detected only the upper triangle (N(N+1)/2 elements) is fit and residues are mirrored (~2x fewer solves). (3) ITERATION CAP: pole relocation stops once the poles settle (relative move < 1e-4) instead of a fixed 12 sweeps (typically 3-5). (4) SHARED-POLE REALIZATION: all N*N elements share the same poles, so the pole-filters are computed ONCE per input port -- a real pole gives one filter V(pj)/(s-p), a conjugate pair two real basis filters (s-sigma)/D and omega/D, with the residue entering each output as a real scalar weight -- and each output current is a cheap weighted sum; this is O(N*Np) laplace_nd and OSDI state instead of O(N^2*Np), the difference between a model that compiles/simulates at large N and one that does not. Three latent bugs the stress test also surfaced, all fixed: (a) an order-selection double-free -- at higher pole counts a relocated pole set could lose its adjacent-conjugate-pair structure and build_basis/ctil_to_cres (which walk i+=2 on a complex pole) wrote one slot past the residue

array; fixed by a canon_poles step that snaps near-real poles to real and rebuilds exact adjacent conjugate pairs. (b) a 256-entry stack array -- the Touchstone parser assembled each frequency's matrix in a fixed cplx pv[16*16], hard-capping the port count at 16 (N=32 smashed the stack); moved to the heap, auto-detect limit lifted to 512. (c) an OpenVAF crash -- a laplace_nd coefficient array assigned to a variable (as the shared realization does) crashes OpenVAF when a coefficient is an integer literal like '{1}' (which %.12g prints for 1.0); the emitter now forces a real literal '{1.0}'. Results: fit at N=8 220s -> 3.6s; N=64 fits in 47s (was ~860GB of RAM); N=100 fits in 2.75min at 9.6e-4 error; model laplace_nd count now N*Np (65 at N=8, 449 at N=32) not ~N^2*Np/2 (256, 7168). Correctness preserved: DC/AC/transient still overlay the original subcircuit (N=8 unchanged; N=16, previously infeasible, matches to 1.3%). The remaining bottleneck is OpenVAF's compile time for the (now far smaller) shared model (~26s at N=16, ~43s at N=20), so the practical end-to-end ceiling is ~N=20-24 while the fit itself scales to N=100. Verify (examples/presnp): an 8-port coupled ladder converts+compiles in a few seconds with the compact shared model (65 laplace_nd for 8 ports) and matches the original network in AC; the E-200 order/realization guards and resonator/3-port checks still pass. 9 checks. Regression 164/164.

E-202 *src/maths/dense/dense.c, examples/spscale_examples/**

.sp S-parameter matrix inverse is O(N^3), not O(N!). The pre_snp stress test (E-201) noted a separate wall: producing the input Touchstone file with ngspice's own .sp analysis was itself pathologically slow -- an 8-port took ~13s and a 10-port ~18 minutes, growing by roughly a factor of N per added port. That is in the RFSPICE analysis, not the converter. Profiling put ~99 percent of an 8-port .sp run inside CKTspCalcSMatrix (cktspdum.c), which builds the port S-matrix at every frequency by inverting N x N complex matrices -- several per frequency, for the S, Y and Z matrices. The complex inverse cinverse (maths/dense/dense.c) was computed by the adjugate/determinant method (Cramer's rule): cinverse = cadjoint(A) * (1/cdet(A)). cdet is a RECURSIVE COFACTOR EXPANSION -- O(N!) -- and cadjoint computes N^2 cofactors (each an (N-1)x(N-1) determinant), so cinverse was O(N*N!); with several inverses per frequency across a whole sweep the cost explodes by a factor of N per port (6! -> 7! -> 8! is exactly the measured x7, x8, x9 per added port). FIX: cinverse and cinversedest now invert by GAUSS-JORDAN ELIMINATION WITH PARTIAL PIVOTING on the augmented [A I], in O(N^3) -- a drop-in replacement with the same signature and the same result (the true inverse). Behavior preserved: the old cinverse never returned NULL (the adjugate always produced a matrix, garbage for a singular input), so the new one returns NULL only on a true allocation failure and zero-fills a singular matrix; the singular case does not arise for a well-posed .sp network, and the two cktspdum.c call sites that do not null-check are safe. cdet/cadjoint remain for any other callers. RESULTS: .sp at N=8 13s -> 0.3s, N=10 ~18min -> 0.02s (~50000x), N=16 0.03s, N=32 0.09s; the extracted S-parameters are unchanged, verified against the closed-form network to ~2e-7. The same inverse is used by the periodic S-parameter path (.psp/PSS in dcpss.c), which benefits too; full example regression (touchstone, rfanalyses, psp, ...) unchanged. Together with E-201 the whole extract-or-import -> convert -> simulate workflow now scales to many ports. Verify (examples/spscale): a 12-port RLC ladder -- a port count that took minutes before -- runs through .sp + wrsnp, completes in a fraction of a second, and every entry of the extracted 12x12 S-matrix matches the closed-form network across the sweep to ~2e-7. 2 checks. Regression 165/165.

E-203 *src/frontend/com_measure2.c, src/misc/string.c, examples/acmargin_examples/**

.meas ac gain/phase margin (+ batch vdb/vp auto-save fix). An audit against a 'richer TRIG/TARG, FIND...WHEN, integral/RMS/derivative, AC margins' wish-list found that ngspice's .measure already covers TRIG/TARG (with VAL, RISE, FALL, CROSS, LAST, TD, AT), FIND...WHEN, AVG/RMS/INTEG/DERIV, MIN/MAX/MIN_AT/MAX_AT/PP, ERR, and param='...', across tran/ac/dc/sp -- all present and working. The one genuine gap was AC margins. The textbook way to get them from the existing primitives, .meas ac gm FIND vdb(out) WHEN vp(out)=-180, CANNOT work: vp() returns the phase WRAPPED to (-180,180], so on any loop whose true phase sweeps past -180 the wrapped phase jumps from about -179 straight to about +179 and never equals -180,

and the crossover is never found. FIX: two first-class .meas ac functions computed on the UNWRAPPED phase -- phase_margin (PM = 180 + the unwrapped phase at the 0 dB gain crossover) and gain_margin (GM = -gain(dB) at the -180 deg phase crossover). The phase is unwrapped by undoing the +-360 deg atan2 jumps, the 0 dB and -180 deg crossings are found by linear interpolation, and the crossover frequency is interpolated in log space. A loop with no finite -180 crossover (a one- or two-pole loop, infinite gain margin) is reported as such rather than as a bogus number. Implemented as measure_margin() dispatched from get_measure2() in com_measure2.c. One subtlety: ngspice's phase (vp, and the measure 'p' vector type) honors a runtime cx_degrees flag that defaults to RADIANS, so radtodeg() is a no-op by default; the margin code computes phase in degrees explicitly. TWO BUGS the work surfaced, both fixed: (1) gettok_iv (misc/string.c), used only by the batch measure auto-save pass, copied the leading v/i then broke after the FIRST following character on n_paren==0 -- so vdb(out) became vd (and vm/vp/... were all truncated); the stray vd then failed to save ('can't parse vd') and left the node unsaved, so a batch .meas ac ... vdb(out) died with 'no data saved for AC'. Fixed to stop only once the parenthesised part has actually closed. (2) a companion strip_ac_vec reduces vdb(out) to v(out) so the base node is what gets .save'd. Verify (examples/acmargin): 5 checks against closed-form loop gains (buffered one-pole sections, so poles sit exactly where placed) -- a stable 2-pole loop's phase margin matches analytic to under 1 deg (84.89 vs 84.89) and its gain margin is correctly reported infinite; a 3-pole loop's phase margin (-42.75 deg) and gain margin (-15.92 dB) both match the closed form exactly; and a batch .meas ac ... vdb(out) dot-card auto-saves the node and returns the right value. 5 checks. Regression 166/166.

E-204 *src/include/ngspice/optdefs.h, src/include/ngspice/tskdefs.h, src/include/ngspice/cktdefs.h, src/spicelib/analysis/cktsopt.c, src/spicelib/analysis/cktntask.c, src/spicelib/analysis/cktdojob.c, src/spicelib/analysis/cktop.c, examples/convhelp_examples/**

auto-triggering DC convergence aids (.option convhelp). ngspice's CKTop operating-point cascade already auto-escalated through gmin stepping and source stepping, but the two strongest aids -- the globalized damped-Newton line search (E-111) and pseudo-transient continuation (E-127) -- only fired when the user hand-set .option linesearch or .option ptcont, and nothing reported which aid rescued a hard operating point. The ngspice-vs-Spectre gap analysis flagged exactly this: what remained was auto-triggering heuristics (reaching for these aids without the user asking) and folding them into the robustness presets. .option convhelp makes the whole cascade one automatic ladder -- Newton (with line search), then gmin step, then source step, then pseudo-transient, then optran -- and prints a one-line note naming the aid that produced the point. Under convhelp the op-point Newton, and its fallback homotopies (which call NIiter internally), run with line search enabled; E-111's line search reduces to a plain full Newton step whenever the full step already reduces the weighted residual norm, so points that converge easily take the same iterates and are unaffected. The E-127 pseudo-transient gate was widened so it also fires when convhelp is set, with no explicit .option ptcont needed. CKTop was refactored to a single done: exit that saves the caller's line-search flag on entry and restores it on every return (so the setting never leaks into the transient analysis that follows) and emits the aid note there. OFF BY DEFAULT: the default (convhelp unset) path is byte-for-byte the historical behavior with no new output. errpreset=conservative enables it through the E-110 TSKtolGiven given-flag mechanism (a new ERRP_CONVHELP bit), so an explicit .option convhelp=0 still overrides the preset regardless of .options line order. Plumbing mirrors E-127: OPT_CONVHELP enum and ERRP_CONVHELP bit (optdefs.h), TSKconvhelp bitfield (tskdefs.h), CKTconvhelp bitfield (cktdefs.h), the OPT_CONVHELP setter plus convhelp in the errpreset value table plus the convhelp OPTtbl keyword (cktsopt.c), copy and default-off (cktntask.c), CKTconvhelp = TSKconvhelp (cktdojob.c), and the auto-escalation plus reporting in CKTop (cktop.c). Verify (examples/convhelp): the option is accepted; on a battery of normal circuits (diode, BJT, two-diode, resistor network) convhelp is result-neutral (node voltages identical to a plain run) and silent (no aid note, since plain Newton already converges); on a deliberately stiff behavioral-exponential deck with no junction limiting -- root of 1e-14*(exp(V/0.026)-1) = (100-V)/100, i.e. V(1) = 0.837922 V -- where plain Newton overshoots to a spurious 70.5 V root

(and even the transient-op fallback lands there), convhelp reaches the correct 0.837922 V via auto-fired pseudo-transient continuation, reports the aid, and changes the outcome versus plain Newton; errpreset=conservative enables the same ladder; and an explicit convhelp=0 overrides the preset. Both linear solvers (KLU and Sparse 1.3). 35 checks. Regression 167/167.

E-205 *src/frontend/snp2va.c, examples/lowrank_examples/**

pre_snp low-rank residue factorization. The E-200/201 shared-pole realization emits an O(N^2)-term, O(N*Np)-filter Verilog-A model, so OpenVAF's compile time grows super-linearly with the port count and caps the practical N at ~20-24 (an N=32 block compiles in ~80s -- the vector fit itself scales to N=100, but the emitted model's compile does not). When an N-port block's ports couple through only a few shared modes -- the common case for multi-port filters, cavities, and packages with a shared plane -- each pole's N x N residue matrix has rank r much less than N. E-205 exploits that: per channel (the conductance d, the capacitance e, and each pole section) the emitter builds the N x N real weight matrix and, via a new in-C one-sided Jacobi SVD, picks the cheaper of a dense emit (a term per significant entry, one laplace_nd per input port) or a low-rank emit W = UV^T (rank kept above tol_rank = 1e-7, so a clean singular-value gap compresses fully while a slowly-decaying channel stays dense). The key is INPUT COMBINING: because laplace is linear, sum_j V[j][m]*laplace(V(p_j)) = laplace(sum_j V[j][m]*V(p_j)), so the low-rank form filters the r COMBINED inputs u_m = sum_j V[j][m]*V(p_j) ONCE (r filters, not N) and distributes them to the outputs via U -- collapsing filters from O(N*Np) to O(r*Np) (the piece that dominates the compile) and coupling terms from O(N^2) to O(N*r). A full-rank block has no such structure, so every channel keeps the dense form and is emitted bit-identically (the auto-detect leaves it alone); the E-201 PSD projection of the capacitance e is preserved, so the low-rank transient stays stable. The env var PRE_SNP_DENSE forces the old dense realization (escape hatch / A-B test). Measured on a 3-shared-mode block (openvaf-r compile of the emitted .va): N=16 16.9s to 1.5s (96 to 8 filters), N=24 42s to 5.7s (144 to 10), N=32 81s to 11.6s (192 to 8) -- super-linear to roughly linear, about 7-14x, with the response exact (AC ~4e-7, transient ~3e-5 versus a forced-dense build of the same fit). New code: a jacobi_svd and an emit_filter helper plus the per-channel chooser and input-combined emit, all in snp2va.c. Verify (examples/lowrank): a 12-port/3-mode block emits far fewer filters low-rank than dense (7 vs 73); its AC (3.9e-8) and transient (1.2e-4, bounded) equal the forced-dense build of the same fit; a full-rank 8-port ladder gets no compression with bit-identical AC. 5 checks. Regression 168/168.

E-206 *src/frontend/com_optimize.c, examples/dcenter_examples/**

design centering / yield optimization (optimize -center). The capstone that fuses two whole subsystems: the optimizer (Nelder-Mead / LM / PSO / DE / SA, E-130-197) and the Monte-Carlo yield suite (agauss/mccorr sampling + montecarlo specs, E-149-151). Design centering optimizes the NOMINAL design point to maximize parametric yield / Cpk under process variation. The outer optimizer searches the design knobs (-dparam/-param/-mparam, which feed the deck's agauss centres); the objective at each candidate is an INNER Monte Carlo of -samples N draws -- each reset re-samples the deck's process variation (agauss/.param, and any mccorr correlations) around the current centre -- scored against -spec [-max hi] [-min lo] limits (the same spec syntax as montecarlo) and reduced to the worst-case Cpk and the pass-fraction yield. The fusion is small and clean in opt_eval: the in-place-knob application was factored into opt_apply_inplace so the MC loop can re-apply the centre after each reset re-sources the deck, and a new opt_eval_center runs the inner MC and returns -min(Cpk); the method drivers (nm/pso/de/sa) are unchanged -- they call opt_eval, which now returns the centering cost. Cpk is the objective (yield is reported alongside) because raw yield is a granular step function that stalls the simplex, whereas Cpk = min(gap to each active limit)/(3 sigma), worst-cased across specs, is continuous and monotone in yield -- so maximizing it centres the distribution in its spec window. A fixed inner seed gives every candidate the SAME process draws (common random numbers), so the objective is deterministic and smooth; with -lhs the stratified sample-mean is about zero, so the optimum lands on the analytic centre. Reuses the E-149/151 sampling (mc_lhs_config, mc_sss_off, setseed); lm is rejected for -center (Cpk is not a least-squares residual) and the default method is Nelder-Mead; results

are published as dcenter_yield and dcenter_cpk vectors. com_optimize.c only. Verify (examples/dcenter): a synthetic problem output ~ N(xc, 0.5) with spec [4,6] (agauss(m,v,3) makes the actual sigma = v/3 = 0.5, so the analytic centre is the midpoint 5 and the analytic Cpk is 1/(3*0.5) = 0.667) is centred from an off-centre start xc=4.0 to xc = 5.00 with the expected Cpk 0.667 and a high yield; a montecarlo baseline at xc=4.0 sits near 50% while the centred design reaches ~96%; and a two-knob problem with a lower and an upper spec on two outputs centres both params independently (xa -> 5 on [4,6], xb -> 10 on [9,11]). 5 checks. Regression 169/169.

E-207 *src/frontend/com_eye.c (new), src/frontend/com_eye.h (new), src/frontend/commands.c, src/frontend/com_commands.h, src/frontend/Makefile.am, examples/eye_examples/**

eye diagram / jitter analysis (eye command). A new application domain: the core signal-integrity measurement for serial links / SerDes / high-speed I/O, which ngspice lacked. eye -ui [-tstart t0] [-threshold vth] [-window frac] is a self-contained front-end post-processing command (in the fft / linearize / meas family, registered in commands.c/com_commands.h/Makefile.am), solver-independent. It reads the transient waveform and its time scale via ft_evaluate; auto-detects the two logic rails from the 20th/80th percentiles of the samples (eye_level0, eye_level1, eye_amplitude) and the decision threshold (default the midpoint, or -threshold); finds every threshold crossing by linear interpolation; estimates the UI phase as the circular mean of the crossings modulo UI; computes each crossing's time-interval error (TIE, relative to the ideal UI grid) and reduces it to eye_jitter_rms (standard deviation of TIE) and eye_jitter_pp (peak-to-peak); measures eye_height (the vertical opening at the sampling instant, one half-UI from the transitions -- min of the samples above threshold minus max of the samples below, taken in a +/- window*UI slice) and eye_width (UI minus the peak-to-peak jitter, the horizontal opening at the threshold), plus eye_width_ber12 = UI - 14.069*eye_jitter_rms (the eye width at BER 1e-12 from the +/-7.035 sigma Gaussian random-jitter tail); and folds the waveform modulo 2*UI into the eye_wave vs eye_t vectors (built with dvec_alloc/vec_new, as stb and sweep build their result plots), whose scatter plot IS the eye diagram (plot eye_wave vs eye_t). No numerical-core changes. Publishes 11 permanent result vectors. Verify (examples/eye): an ideal 0/1 clock (perfectly periodic edges) recovers rails [0,1] and amplitude 1 with jitter about 1e-24 s (machine zero) and a full-UI, fully open eye; a clock whose edges carry a KNOWN injected Gaussian timing jitter (sigma = 20 ps) is recovered to within a few percent (measured about 18.6 ps rms, 99 ps pp) and eye_width equals UI minus the measured peak-to-peak jitter; and the folded eye vectors are published and `wrdata eye_wave` writes the folded eye without crashing (the eye_wave/eye_t vectors live in their own fresh `eye` plot -- as stb does -- so wrdata/plot pair equal-length vectors; dumping the length-m eye vectors into the transient plot, whose scale is the full much-longer time vector, segfaults). 7 checks. Regression 170/170.

E-208 *src/frontend/plotting/pyplot.c, src/frontend/plotting/pyplot.h, src/frontend/com_pyplot.c, examples/pyplot_examples/**

pyplot -eye: one-line matplotlib eye diagrams. A convenience bridge between the E-207 eye analysis and the E-94 matplotlib back-end: pyplot [name] -eye -ui [-tstart t0] [-threshold vth] [-window frac] runs the eye analysis AND renders the folded eye in one command. plot eye_wave vs eye_t draws a single connected line through the folded samples in file order (a scribble, not an eye); pyplot -eye instead draws the samples as a 2-D histogram (hist2d, log-scaled LogNorm, turbo colormap, empty cells masked with cmin=1) so overlapping traces accumulate into a persistence eye -- exactly how a sampling scope paints one -- annotated with the decision threshold (dashed), the eye-centre sampling instant at 1 UI (dotted), a vertical eye-height double-arrow, a horizontal eye-width double-arrow along the threshold, and a title carrying UI / eye height / eye width (% UI) / RMS jitter. Implementation: com_pyplot() detects the -eye marker (a single bare token before it is taken as the output base name; default eye), runs com_eye() on the trailing tokens -- which folds the waveform and leaves eye_wave/eye_t plus the scalar metrics in a fresh current eye plot -- then calls the new ft_pyplot_eye() in plotting/pyplot.c, which reads those back from the current plot, writes .data (the folded sample pairs) and a .py matplotlib script, and launches Python

with the SAME system(python ...) mechanism as ft_pyplot(), honouring every existing pyplot_* setting: pyplot_terminal=png/svg/pdf renders headless (Agg) and runs synchronously, otherwise an interactive window is launched in the background; pyplot_python picks the interpreter, pyplot_backend forces a matplotlib backend, pyplot_figsize sets the figure size, and pyplot_style applies a style sheet (a dark sheet flips the annotation colours to stay legible on a dark ground). No numerical-core changes; the eye math is entirely E-207's, and ft_pyplot() and the gnuplot/asciiplot back-ends are untouched. Verify (examples/pyplot): a self-contained data eye -- a pseudo-random NRZ bit stream (PWL) through a bandwidth-limiting RC channel (tau ~ 0.5 UI) -- is rendered with pyplot eyefig -eye v(rx) -ui 0.5n: the eye analysis runs and reports its metrics, eyefig.png is a valid PNG, the generated script is a hist2d persistence eye referencing the metrics, and the no-name form defaults the base to eye.png. 4 checks (17 total in the pyplot suite). Regression 170/170 (extends the pyplot suite, no new folder).

E-209 *src/spicelib/analysis/dcpss.c, src/frontend/com_hb.c, src/include/ngspice/cktdefs.h, examples/hb_examples/verify_hb.py*

hb publishes its spectrum as nutmeg vectors. The E-134 harmonic-balance command (hb) used to only PRINT a spectrum table: HBanalyze() freed the converged two-sided Fourier solution Vr/Vi as soon as the table was written, so the result could not be plot/print/wrdata-ed without scraping stdout (the amp2n2222 HB study had to parse the table in Python for every figure). Now, after a converged hb, a fresh current 'hb' plot holds a real scale hbfrequency (0, f0, 2*f0, ..., K*f0; K+1 points) and one COMPLEX vector per node (named by the node, e.g. out, vcc, q1#branch) carrying the single-sided amplitude, so mag(out)/vp(out) reproduce the printed

E-210 *src/frontend/inp.c, src/spicelib/analysis/dcpss.c, examples/rfpss_examples/rc_pss.cir, examples/rfpss_examples/verify_rcpss.py*

PSS polish: .pss dot-card runs in batch + complex spectrum vectors. Two usability fixes to the E-117 shooting-method periodic steady state (pss), in the spirit of E-209 (hb). (1) The pss COMMAND (inside .control) worked, but a top-level .pss card by itself silently did nothing in batch mode -- ngspice -b deck.cir with just `.pss ...` reported "no simulations run", unlike .tran/.ac/.hb which auto-run; a deck had to wrap it in .control ... run ... .endc. .pss is now dispatched to the pss command the same way E-146 (.sweep) and E-162 (.hb) handle their cards: during deck loading (frontend/inp.c) a top-level `.pss ...` line is stripped of its leading '.' and appended to the post-parse control list, so it executes as `pss ...` once the circuit is built. A boundary check (next char is whitespace or end-of-line) keeps it from matching a longer name, and it never matches .psp (periodic S-parameters, a distinct card). Decks that already wrap .pss in .control ... run keep working (one PSS run, unchanged output). (2) The pss analysis already published two plots -- pss1 (time-domain periodic waveform) and pss2 (frequency-domain spectrum) -- but pss2 stored MAGNITUDE ONLY (IF_REAL): the DFT computed each harmonic's phase (pssphases) and then discarded it, so vp(out) was unavailable and print out gave a bare magnitude, inconsistent with AC analysis (whose node vectors are complex) and with the hb command's E-209 vectors. pss2 now publishes COMPLEX node vectors (IF_COMPLEX): each harmonic is stored as mag * e^{j*phase} (the DFT returns the phase in degrees; the DC term is kept real), built into IFcomplex rows and emitted with OUTpData (as PAC/PXF/PSP already do) instead of the real-only CKTdump. mag(out) reproduces the old magnitude spectrum exactly and vp(out) now recovers the phase; the max-frequency-verification logic still reads the retained magnitude array (pssResults), so it is unchanged, and the time-domain pss1 plot is unchanged (a real waveform). BACKWARD COMPATIBILITY: because pss2 node vectors are now complex, print out / wrdata out emit re,im columns (exactly as AC analysis does) rather than a bare magnitude -- decks that parsed the magnitude directly should use mag(out); the bundled rc_pss.cir was migrated from `print b` to `print mag(b)`. This is the same magnitude-vs-complex convention the rest of ngspice's frequency-domain analyses use. The shooting-PSS numerical core, the retained periodic operating point, and every analysis built on it (PAC/Pnoise/PXF/PSP) are untouched. Verify (examples/rfpss): a top-level .pss card (no .control) auto-runs in batch and reaches convergence; the frequency-domain spec-

trum resolves as complex -- both vm(out) (magnitude) and vp(out) (phase) return, with a driven diode rectifier's harmonics showing the expected non-trivial phase; the existing PSS/PAC/PXF/ solver-parity checks pass unchanged. 2 checks (27 total in the rcpss suite). Regression 170/170 (extends the rfpss suite, no new folder).

E-211 *src/spicelib/analysis/dcop.c, src/frontend/inpcom.c, examples/codeanalysis_examples/**

Code-analysis bug fixes: XSPICE DC-op else + LTSPICE table free. A static-analysis pass over the ngspice tree (clang's analyzer run on every project-modified file, each finding then confirmed by reading the code, false positives -- e.g. symbolic-size array indexing in static math helpers -- discarded) surfaced two real defects. BUG 1 (dcop.c): DCop() chooses between the event-driven XSPICE solver (EVTop) and the analog solver (CKTop) with a BRACELESS else whose body was originally `converged = CKTop(...)`, so CKTop ran only when there were no event-driven instances. Enhancement-188 (DC warm-start) inserted its preload code between the else and CKTop, so the else then covered only its first statement (wsize = SMPmatSize(...)) and CKTop ran UNCONDITIONALLY: for a deck with event-driven code-model instances (A-devices) BOTH EVTop and CKTop ran, CKTop re-solving the analog part and overwriting the event-driven DC result, and wsize was read UNINITIALISED at the warm-start guard (line ~110) and snapshot (line ~129) on the EVTop path (undefined behaviour, masked today only because DC warm-start is off by default). FIX: hoist wsize = SMPmatSize(ckt->CKTmatrix) above the branch (always initialised; the warm-start snapshot reads it on both paths) and add braces so the else covers the warm-start preload + CKTop, restoring the original exactly-one-solve semantics. The #ifdef XSPICE braces vanish together when XSPICE is not compiled, so the non-XSPICE build is byte-unchanged. BUG 2 (inpcom.c): inp_compat() converts an LTSPICE `E ... table=(...)` controlled source using an UNINITIALISED char *ckt_array[100] stack array; the LTSPICE branch sets only ckt_array[1], and a single-pair table (table=(x0,y0), ipairs==1) then tfree(ckt_array[2]) -- a free of a garbage stack pointer (ckt_array[2] is written only on the other, non-LTSPICE, branch). FIX: zero-initialise the array (char *ckt_array[100] = { NULL };); ngspice's tfree(NULL) is a safe no-op (txfree guards `if (ptr) free(...)`), so a tfree of any slot not written on a given branch does nothing instead of freeing garbage. No functional change to correct analog decks -- the fixes remove undefined behaviour and a redundant analog solve on the XSPICE and LTSPICE-import paths respectively. Verify (examples/codeanalysis, 5 checks x both solvers): a mixed-signal DC op (an analog divider + an adc_bridge, an event-driven instance) solves the analog node to v(a)=0.5 without error; a single-pair LTSPICE `E ... table=(0.5, 3)` imports and yields the constant 3 V; a plain analog DC op is unchanged (a would-fail before/after test is impractical -- the redundant CKTop leaves a simple circuit's DC point unchanged, and freeing a garbage pointer is UB that may or may not crash). 5 checks. Regression 171/171 (new example folder).

E-212 *src/frontend/breakp.c, src/frontend/device.c, src/frontend/vectors.c, src/frontend/dotcards.c, src/frontend/inpcom.c, src/spicelib/analysis/cktpzset.c, examples/crashfix_examples/**

Crash hardening: seven user-triggerable crashes. A static-analysis pass (clang analyzer) plus argument fuzzing of every frontend command and of malformed/degenerate netlists surfaced seven reproducible, user-triggerable crashes (SIGSEGV/SIGABRT) in STOCK ngspice, all in user-input handling (command parsers, the .op output path, netlist recursion) and none in the numerical core (device models, OSDI loader, Sparse 1.3, KLU, and the analysis drivers were audited and found clean). Each is a missing edge-guard. (1) iplot (breakp.c, com_iplot): `iplot -w/-d` with the flag as the trailing token consumes its value word (wl=wl->wl_next), guards it with `if(wl)` (proving wl can be NULL), then the loop-bottom `wl=wl->wl_next` dereferences NULL. FIX: guard the advance. (2) altermod (device.c, com_altermod->com_alter_common): com_alter guards `if(!wl)` but com_altermod does not, so a bare `altermod` reaches com_alter_common with an empty list -> wlin=wl_head=NULL -> wl_nthelem(100,NULL)=NULL -> eq(wlin->wl_word, "]") derefs NULL. FIX: guard the empty wl_head with a clean error. (3) measure (vectors.c, vec_ get): a malformed measure statement (meas tran x FIND, ... WHEN, FIND v(1) with no WHEN/ AT, across tran/dc/ac/sp) leaves meas->m_vec NULL, which flows into com_measure_when ->

vec_get(NULL) -> copy(NULL)=NULL -> strchr(NULL,'.'). FIX: make vec_get NULL-safe (if(!vec_name) return NULL) -- the common sink; every caller already handles a NULL return, so each trigger reports "no such vector" instead of crashing. (4) empty .op (dotcards.c, ft_cktcoms): a circuit with no non-ground nodes -- e.g. a deck whose every component is commented out -- produces an op plot with no data vectors, so pl_dvecs is NULL; the former assert(pl_dvecs != NULL) aborted (SIGABRT; ngspice's autotools build defines no NDEBUG so asserts are live) and with -DNDEBUG the next line dereferenced NULL. FIX: guard `if(plot_cur && plot_cur->pl_dvecs)` and skip the empty op printout (the solver had handled the empty matrix correctly; this was purely output formatting). (5) KLU pole-zero (cktpzset.c): a bsearch miss for the drive pointer was reported with a printf and then dereferenced anyway (job->PZdrive_pptr = matched->CSC_Complex); latent (SMPmakeElt creates the element just above, so it is always in the bind table and the miss cannot occur today) but the guard was objectively broken. FIX: add the missing else/NULL path, matching the correct idiom in cktsetup.c. (6) .include recursion (inpcom.c, inp_read): inp_read tracked call_depth but never limited it, so a file that .includes itself -- directly, via an A->B->A cycle, or via a .lib section that .includes its own file -- recursed until the C stack overflowed (SIGSEGV); .lib files are de-duplicated by find_lib and read once, so the crash is always via .include. FIX: cap at INP_MAX_INCLUDE_DEPTH=50; a cycle now reports an error (a 49-level chain still reads fine). (7) .func recursion (inpcom.c, inp_expand_macro_in_str): the expander re-expands a substituted function body with no depth guard, so `.func f(x)={f(x)}` (or a mutual f<->g) expanded forever and overflowed the stack (each frame also holds a params[FCN_PARAMS] array ~8KB, so it fails near ~1000 deep). FIX: a macro_depth counter capped at INP_MAX_MACRO_DEPTH=100, reset on every error path. The netlist-recursion caps mirror the include/expression-depth hardening OpenVAF-r received in Enhancement-148. No functional change to correct decks -- the fixes turn crashes on malformed or degenerate input into graceful errors. Verify (examples/crashfix, 17 checks x both solvers): every repro now exits gracefully (no signal -- crashes are detected from the child's SIGSEGV/SIGABRT exit code, which the pre-fix binary returned) while every valid form still works (iplot -w 3 v(1); altermod nm vto=0.7; meas tran vmax MAX v(1) -> 1.0; a normal .op; KLU .pz; a legitimate nested .include; .func sq(x)={x*x} -> sq(4)=16). 17 checks. Regression 172/172 (new example folder).

E-215 *src/main.c, src/osdi/osdiload.c, src/include/ngspice/osdiitf.h, examples/plusargs_examples/**

command-line plusargs (+name[=value]) served to Verilog-A models. Lets `ngspice -b deck.cir +corner=ff` pick a corner or toggle a feature flag at run time without editing the deck -- the simulator half of E-215 (the compiler half is in the openvaf report). main.c treats a +-prefixed command-line argument as a plusarg rather than an input file (both the batch and interactive file loops) and calls the new ngspice_register_plusarg (osdiitf.h). get_simparams (osdiload.c) exposes each plusarg on the OsdiSimParas channel a compiled model already reads for $simparam$str (E-25), as namespaced simparams: numeric $test$plusargs$name=1 (present), $valset$plusargs$name=1 (given as name=value), $valnum$plusargs$name (value via strtod), and string $value$plusargs$name (value text). The base sim_params arrays are spliced onto the extended arrays only the FIRST time get_simparams sees any plusarg present, so a run with NO plusargs allocates nothing and is byte-for-byte unchanged; the extended arrays are built once (plusargs are constant for the run) and only the base numeric values + analysis_name are refreshed per call. No OSDI ABI change (the string/number channel already existed). Verify (examples/plusargs, 12 checks both solvers): a conductance set from +boost/+gain=25/+scale=2.5/+corner=ff|ss plus unmatched/unrelated/no-value edge cases. Regression 175/175 (new example folder).

E-216 *src/frontend/com_optimize.c, examples/pareto_examples/**

NSGA-II multi-objective / Pareto optimization. Adds a `-method nsga2` to the optimize command that trades TWO OR MORE competing objectives and returns a Pareto FRONT of non-dominated designs, rather than the single optimum the scalar methods return (E-130 Nelder-Mead, E-194 PSO, E-195 DE, E-196 SA) -- the multi-objective generalization of E-206 design cen-

tering. Objectives are given as repeated -minimize / -maximize ; the existing optimize infrastructure is reused wholesale (the -param/-dparam/-mparam knobs, the -analysis-driven expression objectives via opt_eval_expr, the normalized [0,1] search space and the opt_rand RNG). New: a vector-valued objective evaluator opt_eval_objs (maximized objectives negated to a common minimization convention) and nsga2() implementing standard NSGA-II (Deb 2002) -- fast non-dominated sorting (Pareto rank via domination counts), crowding distance (per-front normalized neighbour gaps, boundaries at infinity), binary-tournament selection by the crowded-comparison operator, real-coded SBX crossover (eta=15) + polynomial mutation (eta=20, rate 1/n), and elitist parent+offspring (2N->N) survivor selection so a non-dominated design is never lost. The command prints the final front (each design's objectives + parameters, sorted by the first objective) and publishes each objective column as a permanent vector pareto1/pareto2/... so the front plots directly (plot pareto2 vs pareto1). nsga2 needs >=2 objectives and a single -analysis stage; population defaults to 20+4*np (override -swarmsize), generations -maxiter, -seed for reproducibility. The scalar methods and their single-objective/least-squares/centering paths are unchanged (a lone -minimize still seeds the scalar objective for nm/pso/de/sa). Verify (examples/pareto): the Schaffer-1 benchmark -- minimize f1=px^2 and f2=(px-2)^2 over px in [-1,3], whose Pareto set is exactly px in [0,2] tracing f2=(sqrt(f1)-2)^2, with the knob range wider than the front so NSGA-II must discover [0,2]. The returned front is a reasonable size, every point is genuinely non-dominated, it spans the trade-off (px reaches ~0 and ~2), and the published vectors lie on the analytic curve to machine precision (8.9e-16). 6 checks. Regression 176/176 (new example folder).

E-217 *src/frontend/com_pyplot.c, src/frontend/plotting/pyplot.c, src/frontend/plotting/pyplot.h, src/frontend/plotting/plotit.c, examples/pyplothist_examples/**

pyplot -hist: histogram plots. `pyplot -hist <sig> ...` renders each listed signal's VALUE distribution as a matplotlib histogram (plt.hist) instead of a trace vs time/frequency -- for the amplitude spread of a transient, a Monte-Carlo measurement, or a noise sample. Unlike -eye (E-208, a distinct analysis) -hist is a render MODE, so it is small and reuses the whole pyplot pipeline: com_pyplot detects the -hist marker anywhere in the arg list, unlinks it, and dispatches the remaining [name] signal-list to plotit with a distinct device string pyplothist; plotit resolves the signal expressions exactly as for a normal plot (v(out), db(...), vector arithmetic, node names) and passes the resolved vectors to ft_pyplot with a hist flag; ft_pyplot writes the same .data table and emits ax.hist(value_column, bins, alpha, label) per signal instead of ax.plot(x,y). All existing pyplot facilities carry over unchanged (set pyplot_terminal=png/svg/pdf headless render, pyplot_subplots panels, style sheets, figsize, backend); two histogram-specific options are added -- pyplot_hist_bins (bin count, default matplotlib 'auto') and pyplot_hist_density (normalize to a density). Correctness details: the x-axis is the signal value and the y-axis the count (labels set accordingly); histogram panels do NOT share an x-axis (sharex=False -- different signals have unrelated value ranges, vs a line plot's shared time/freq axis); the FULL value length is histogrammed -- the data-table loop normally walks the shared scale vector, but a histogram of a raw `let` vector (e.g. a Monte-Carlo result) can have a scale whose length differs from the values', so for hist it walks the longest VALUE vector and pads shorter ones with NaN (the generated script filters via ~np.isnan), fixing a silent truncation to the scale length; overlaid histograms get alpha=0.6 transparency and a legend while a single one is drawn opaque. No change to the ordinary pyplot trace path or to pyplot -eye. Verify (examples/pyplothist): two signals with closed-form distributions -- ramp=i/(N-1) uniform on [0,1] (flat histogram), sine=sin(2*pi*i/100) arcsine on [-1,1] (U-shaped, a sinusoid dwells near its peaks so the edges tower) -- checked analytically from the generated .data (uniform bins within 15% of the mean; sine edge bins » middle), plus the -hist path taken (plt.hist not plt.plot, non-shared x), the full 20000-sample length not truncated, and a valid non-trivial PNG. 6 checks. Regression 177/177 (new example folder).

E-218 *src/frontend/com_pyplot.c, src/frontend/plotting/pyplot.c, src/frontend/plotting/pyplot.h, src/frontend/plotting/plotit.c, examples/pyplotcontour_examples/**

pyplot -contour: 2-D contour maps for 2-D parameter sweeps. `pyplot –contour <z> <x> <y>` renders a filled matplotlib contour map of a quantity z over the (x,y) plane -- the natural view of a 2-D parameter sweep (a device current over a (Vgs,Vds) grid, a gain over an (R,C) grid, a noise figure over a (freq,bias) grid). Like -hist (E-217) and unlike -eye (E-208, a distinct analysis), -contour is a render MODE dispatched over the ordinary signal list: com_pyplot detects the -contour marker anywhere in the arg list, unlinks it, and dispatches the remaining [name] z x y to plotit with a distinct device string pyplotcontour; plotit resolves the three expressions exactly as for a normal plot (v(out), db(...), vector arithmetic, node names) into a three-vector list (z first, then x, then y); the new ft_pyplot_contour writes an (x,y,z) data table and a matplotlib script that TRIANGULATES the (x,y) points (ax.tricontourf). Because the plane is triangulated rather than reshaped into a rectangular mesh, gridded OR scattered sweep data plots identically -- no grid-dimension metadata (nx,ny) is needed, and a sweep with a ragged tail still works; x/y/z come from any 2-D sweep leaving three equal-length vectors (nested .dc, the sweep command family E-146/E-190, a .step grid, or .control vectors). All existing pyplot facilities carry over (set pyplot_terminal=png/svg/pdf headless render, style sheets, figsize, backend); three contour-specific options are added -- pyplot_contour_levels (level count, default matplotlib-auto), pyplot_contour_lines (overlay labelled black contour lines on the filled map), pyplot_contour_cmap (colormap, default viridis). The colorbar is labelled with z's name and the axes with x's and y's. Robustness: -contour needs exactly three vectors, else it is rejected with a clear message (pyplot -contour needs exactly three vectors) rather than a confused plot; a degenerate 1-D (collinear) sweep cannot be triangulated, so the generated script catches the qhull/matplotlib failure and exits with a helpful line (a contour needs a genuine 2-D sweep -- the points must not be collinear) instead of a bare traceback. No change to the ordinary pyplot trace path, pyplot -hist, or pyplot -eye. Also removed a stale unused-variable warning (ft_pyplot's scale) left by E-217. Verify (examples/pyplotcontour): a closed-form surface z=x^2+y^2 over x,y in [-2,2] (a paraboloid whose contours are concentric circles, z=0 at the centre and z=8 at the corners) -- checked analytically from the generated .data: the -contour path is taken (tricontourf not plot/hist, a colorbar labelled z, axes x/y), the data table has three columns (x,y,z) for all 1681 rows, the column mapping is correct (z reconstructs x^2+y^2 to 9e-16), the sweep is genuinely 2-D (x and y each span the full range, z runs from ~0 at the centre to ~8 at a corner), and a valid PNG is rendered. A second demo (bridge_dc_demo.cir) exercises the feature on genuine simulation output: a diode-OR bridge whose output V(c) follows whichever of its two inputs is higher, swept with a nested .dc (v1 drives node a, v2 drives node b, so V(a)/V(b) are the two swept values at each point); pyplot -contour v(c) v(a) v(b) maps the output over the (V(a),V(b)) plane -- a max-like corner surface. It checks the nested .dc yields the flattened 51x51 = 2601-row 3-column grid, the axes are the two real swept sources (each spanning [-1,1]), and the output is ~0 when both inputs are low and rises when either input is high (the diode-OR signature, which confirms the columns map onto a real circuit result). 19 checks. Regression 178/178 (new example folder).

E-221 *src/frontend/inpcom.c, examples/busnodes_examples/**

array/bus node ranges in the netlist. ngspice has no bus/array node syntax: a node is an opaque string, and a range like a[0:1] is read as a single literal node name (so R1 a[0:1] r=2k gives R1 only one node token and misparses r=2k). E-221 adds a netlist preprocessing pass that expands a range token base[lo:hi] into the scalar node sequence base[lo] base[lo±1] ... base[hi], so buses can be written compactly: R1 a[0:1] r=2k -> R1 a[0] a[1] r=2k; X1 n[0:3] mysub -> X1 n[0] n[1] n[2] n[3] mysub; .subckt mysub p[3:0] -> .subckt mysub p[3] p[2] p[1] p[0]. The range supplies node tokens POSITIONALLY -- the 'two terminals' reading: R1 a[0:1] is a single resistor from a[0] to a[1], not an array of resistors; multi-terminal instances, subcircuit calls and .subckt port lists expand the same way. Semantics: base[lo:hi] -> base[lo] base[lo±1] ... base[hi], DESCENDING when lo>hi (Verilog convention d[3:0] -> d[3] d[2] d[1] d[0]); a[0:0] is a single element; the scalar names use the same bracket form so a bus a[0:1] and an explicit a[0] denote the SAME node (the two forms interoperate); only a WHOLE whitespace-delimited token that is exactly base[int:int] is expanded, so already-scalar names (a[0]), non-integer ranges (a[x:y]), malformed ones (a[1:2:3]),

device values, model names and XSPICE %vd[...] port groups are left untouched; .control ... .endc blocks are skipped and only element lines and .subckt lines are processed (.model/.param/other dot cards ignored); a range wider than 8192 elements is left literal (the device parser then reports it) so a typo like a[0:1000000000] cannot exhaust memory. Implementation: one pass inp_expand_buses in frontend/inpcom.c, run in inp_readall right after whitespace normalization and BEFORE subcircuit expansion and device parsing -- so every downstream consumer (numparam, .subckt expansion, the device parser, OSDI) sees the already-scalar node list and needs no changes; continuation lines are stitched earlier in inp_read, so each logical line is processed whole; the token test parses base (a plain node name) then [:] filling the rest of the token and emits the element list with DSTRING, a non-match copying the token verbatim. Verify (examples/busnodes): 6 checks, both solvers, all from node voltages/branch currents (solver-independent) -- R1 a[0:1] r=2k connects a[0]-a[1] so I(R1)=(1-3)/2k=-1mA; the scalar R2 a[0] addresses the same node as the bus element (I=+0.25mA, only correct if a[0] is one shared node); a subcircuit with a descending port bus p[1:0], called with X1 n[0:1], connects positionally (I(Rs)=-5mA); and non-integer (b[x:y]), malformed (c[1:2:3]) and scalar tokens are left literal while a .control let w=a[0:1] is not rewritten. One file; no device/solver/OSDI change; a deck with no base[lo:hi] token is untouched (a fast path skips any line without '['). Regression 181/181 (new example folder).

E-222 *src/frontend/inpcom.c, src/frontend/subckt.c, src/misc/string.c, examples/parserfuzz_examples/**

ngspice netlist-parser hardening (fuzzing): seven crashes/hangs -> clean errors. Fuzzing the netlist parser -- mutating real decks (byte flips, truncation, line duplication, delimiter/bracket/keyword/directive/number injection, and the new E-221 bus-range tokens) and running each under ngspice -b -- found seven ways to CRASH (SIGSEGV/SIGABRT, real memory-safety bugs since ngspice is C) or HANG on malformed input; a 12000-iteration campaign dropped the crash+hang rate from 26 to 1 (96%, all hangs eliminated). ROOT CAUSES, all fixed: (1) misc/string.c gettok_instance() returns an empty token WITHOUT advancing *s when it sits on '(' or ')', and frontend/inpcom.c get_number_terminals()'s OSDI ('n') case was the only multi-token case with no iteration cap -- a line starting with 'n' containing '(' (e.g. 'nan.func f(x)=...', nan lexing as a number) spun forever tmalloc-ing each pass; gettok_instance() now always advances (consumes the bracket as a one-char token) and the 'n' loop gained the same cap as its siblings. (2) inp_expand_macro_in_str(): a truncated '.model m' (no model type) ran nexttok off the end, leaving the scan pointer NULL, then strchr(NULL); NULL guards added on the .model skip and the scan loop. (3) frontend/subckt.c doit(): MAXNEST bounds subcircuit DEPTH, but a subckt that instantiates itself with branching factor >= 2 blows up to 2^MAXNEST instances (an effective hang) before the depth cap trips -- a total-instantiation cap now catches recursive subcircuits. (4) doit(): a bare 'X' invocation (nothing after the refdes) makes the find-last-token walk run off the FRONT of the line buffer -- an out-of-bounds read ending in strcmp(NULL); such a line is now skipped. (5) inp_modify_exp(): two copies into a fixed buf[512] with no bound -- an unterminated 'v(' (no closing ')') and a long identifier token (a run of '[', which is an accepted char) overran the STACK buffer (stack smashing); both loops are now bounded by the buffer size. (6) doit(): a malformed '.subckt' can register a NULL name; the invoked-name match eq(su_name,...)=strcmp() then deref NULL; guarded. (7) translate(): gettok_noparens() returns NULL at the end of a malformed controlled-source line, then strcmp(NULL,'POLY'); guarded. Method: the fuzzer classifies OK/clean-error/CRASH/HANG over self-contained seed decks exercising the parser surface (devices, .subckt, .param, .model, .control, expressions, continuations, buses); each crash was triaged to its faulting instruction with lldb (conditional strchr/strcmp-with-NULL breakpoints), sample for the hangs, and Guard Malloc (libgmalloc) to pin the stack overrun, then read at the source; fixes applied one at a time, verified against the recorded crash corpus, and a fresh 12000-iteration run confirmed convergence. No valid deck changes behaviour. Verify (examples/parserfuzz): 10 checks -- a minimal deck per root cause (nan.func, .model in .control, a self-recursive subckt, a bare X line, an unterminated v(, a 4000-'[' identifier, 4000 '{' in a subckt E-source) now yields a clean/bounded outcome (no signal/abort, no hang), and three valid decks (subckt hierarchy with an E-source and a parameter, a v(x)*v(x) B-source expression, a POLY source in a subckt) compute V to the analytic value. The

recorded 27-input crash corpus is all clean. RESIDUAL: one fuzz input in 12000 still crashes in XSPICE (xspice/mif/mifgetmod.c MIFgetMod) -- an 'a'-device whose model name matches a non-code-model (e.g. a diode .model) is processed as a code model, reading device->modelParms as the wrong type; a code-model type-confusion in the XSPICE subsystem, out of scope for this netlist-parser pass and flagged as a follow-up. Three files (frontend/inpcom.c, frontend/subckt.c, misc/string.c); no device/solver/OSDI change. Regression 181/181 (new example folder).

E-223 *src/xspice/mif/mifgetmod.c, examples/xspicemodel_examples/**

XSPICE a-device model-type validation (MIFgetMod): fixes the one residual crash left by the E-222 netlist-parser fuzzing pass. An `a` device is an XSPICE code-model instance; the last token on its card is a model name, which MIF_INP2A (xspice/mif/mif_inp2.c) hands to MIFgetMod (xspice/mif/mifgetmod.c). MIFgetMod looks the model up in the pass-1 table and, if it has not been instantiated yet, processes its `.model` card AS a code model -- it casts INPmodfast to MIFmodel, reads the device's XSPICE DEVpublic fields (param, num_param), and matches the `.model` parameters through the MIF parameter path -- with NOTHING checking that the resolved model is actually a code model. If an a-device's model name happens to match a NON-code-model `.model` (a diode, a BJT, a resistor model), MIFgetMod walked all of that machinery over a structure that is not a MIFmodel and over DEVpublic fields that a built-in SPICE device leaves unset, reading unrelated memory as the wrong type -> SIGSEGV. The crash bit hardest when the non-code-model's parameter keywords COLLIDE with the a-device's: a diode `.model` has is/n, so the parameter loop found a keyword match and dereferenced the type-confused setModelParm path. The fuzz-found trigger (the E-222 residual) was exactly this, reached through an E-221 bus-expanded a-device inside a subcircuit (`a[0:0] D1 a b dm` with `.model dm d(is=1e-14 n=1)`). FIX: a code model is exactly a device that carries a code-model evaluation function -- cmpp emits `.cm_func = <fn>` for every code model (xspice/cmpp/writ_ifs.c) and it is the pointer XSPICE calls to evaluate the model (mifload.c/evtload.c), while every built-in SPICE device sets `.cm_func = NULL`. MIFgetMod now validates `DEVices[INPmodType]->DEVpublic.cm_func != NULL` (and INPmodType < DEVmaxnum) BEFORE the MIFmodel cast and the parameter loop, returning a clean 'model X is not a code model; an a device requires an XSPICE code-model .model' error otherwise. The guard sits before the cast, so it catches the model whether or not an ordinary device of the same name already instantiated it. A NULL guard on the inner strcmp was rejected as whack-a-mole (the crash merely shifts to the next type-confused access); the model TYPE is the thing to validate. Legitimate code models are unaffected (cm_func != NULL). Scope: XSPICE only, one file (xspice/mif/mifgetmod.c); no netlist-parser, device, solver, or OSDI change. Verify (examples/xspicemodel): 8 checks, both solvers -- six pathological decks (a-device -> diode with and without colliding params, BJT, resistor model, the bus-expanded-in-a-subckt fuzz repro, and an undefined model) now yield a clean bounded error (no signal/abort, no hang), and a legitimate adc_bridge code-model a-device still binds and simulates (analog divider node v(a)=0.5). Regression 182/182 (new example folder).

E-224 *src/frontend/parse.c, src/frontend/evaluate.c, examples/arraynodeprint_examples/**

Reference array/bus node voltages in **`print`**/**`plot`**/**`let`**. Enhancement-221 added array/bus node ranges to the netlist (`R1 a[0:1]` -> `R1 a[0] a[1]`), naming the nodes with LITERAL brackets so the node voltage vector is stored as `a[0]`. But the frontend expression parser uses `[...]` as its vector-INDEX operator, so in `print`/`plot`/`let` the token `a[0]` was parsed as 'element 0 of a vector named a'. There is no vector `a`, so `print v(a[0])` and `print a[0]` silently failed (with a 'vector a is not available' warning) even though the node existed and solved correctly (it shows up in the `.op` node table and `display`). FIX: a literal-node fallback -- when the base name of an index expression is an UNRESOLVED name (a zero-length placeholder dvec) and the index is a constant and a node/vector named `base[index]` exists, use that node instead of indexing. It only fires when the base does not resolve, so ordinary vector indexing (`realvec[3]`, where the base IS a real vector with non-zero length) is unaffected. Three frontend spots, because `a[0]` reaches the machinery three ways: (1) frontend/parse.c checkvalid() -- the pre-evaluation validity check rejected the whole

term because base `a` is a zero-length placeholder; for an INDX node over an unresolved base it now accepts the term when the literal node `base[index]` exists. (2) frontend/evaluate.c op_ind( ) -- evaluates the bare `a[0]` form; it now resolves the literal node name BEFORE evaluating the bare base, so no spurious 'no such vector a' is emitted. (3) frontend/evaluate.c apply_func() -- the v() node-access special case needed arg->pn_value, which an INDX op node lacks (it errored 'bad v() syntax'); it now reconstructs the literal node name from the INDX argument and resolves it. RESULT: `print a[0]`, `print v(a[0])`, array nodes inside expressions (`v(a[0])-v(a[1])`), and `plot/let` all work; the recovered vector is a normal waveform (indexable, aggregatable) in transient. Branch current through the bus resistor stays the device parameter `@r1[i]` (ngspice has no current-between-two-nodes syntax; `i()` takes a source name). Scope: frontend only, two files (parse.c, evaluate.c); no device/solver/OSDI change. Verify (examples/arraynodeprint): 8 checks, both solvers -- `print a[0]/v(a[0])/v(a[1])`, the expression `v(a[0])-v(a[1])`, and `@r1[i]` read the correct DC values; ordinary indexing `unitvec(4)[2]` is unchanged; a genuinely-missing node `v(a[9])` stays a clean miss (no value, no crash). Regression 183/183 (new example folder).

E-225 *src/frontend/evaluate.c, src/frontend/com_measure2.c, src/frontend/fourier.c, src/maths/cmaths/cmath4.c, examples/cmdfuzz_examples/**

Command/expression-evaluator crash hardening (fuzzing). With the netlist parser hardened (E-212/E-222/E-223), the fuzzer was turned on a previously-unfuzzed surface: the interactive `.control` command interpreter and the vector-expression evaluator (`print/let/plot/meas/fft/fourier` and the operators `?:`, indexing `[ ]`, ranges `[[ ]]`, arithmetic). Mutating command lines/expressions over a running circuit and running each under `ngspice -b` found memory-safety crashes (SIGSEGV / SIGABRT, real since ngspice is C); 62 crashes in the first 2500 iterations, driven to 0 by FIVE fixes in three groups. GROUP A -- degenerate-input heap overflows in the math transforms (fft/deriv/fourier each crashed on a too-short (< 2-point) or synthetic vector like `fft(1)`, `deriv(vecmin(v(1)))` (a plot-derived scalar), `fourier 1k deriv(vecmin(v(1)))`; the malloc abort fired on a LATER allocation, so they looked non-deterministic): (1) maths/cmaths/cmath4.c cx_fft() -- the Green's FFT (rffts) derefs bit-reversal tables `BRLowArray[(M-1)/2]` that fftInit() only allocates for M>2, so an input of <=4 points read unallocated memory; and `time`/`xscale` were sized by the data `length` while the scale loops fill `pl_scale->v_length` entries and the scale vector uses `fpts` points, so a vector shorter than the plot scale overran both -- pad to N>=8, size buffers for the largest fill, reject length<2/scale-too-short. (2) maths/cmaths/cmath4.c cx_deriv() -- the polynomial-fit derivative reads degree+1 points per fit; a length-1 input ran the fit/edge loops over too few points and overran the heap -- for length<2 return a zero vector (derivative of a single point is undefined). (3) frontend/fourier.c -- a degenerate input gives a zero time span and overran the interpolation grid -- require >=2 data points. GROUP B -- frontend/evaluate.c ft_ternary(): the `?:` operator evaluated its condition and selected branch with no NULL check, so when either failed to evaluate (`1?0[3]:9`, where `0[3]` indexing-a-scalar returns NULL) it hit vec_copy(NULL) / cond->v_link2 and crashed -- the dominant crash; fixed with NULL guards (an audit confirmed op_ind, op_range, binary-op and apply_func already guard NULL). GROUP C -- frontend/com_measure2.c: measure error messages were formatted into a shared char errbuf[100] (passed by pointer to the measure_parse_* helpers); a failed measure writes its ENTIRE expression string as the vector name (sprintf(errbuf, "no such vector as '%s'", meas->m_vec)), so a measure expression longer than ~80 chars (trivially `meas tran m MAX (v(1)+v(1)+...)`) overran it -> SIGABRT (content irrelevant, only length, which is why it resisted reduction) -- fixed with a file-scope MEAS_ERRBUF_SIZE and a bounded snprintf at every write (28 sites). Note: `dowhile 1 ... end` (true constant condition, no break) is an intentional infinite loop, flagged as a HANG but correct. Scope: frontend/math only, four files (evaluate.c, com_measure2.c, fourier.c, cmath4.c); no device/solver/OSDI change. Verify (examples/cmdfuzz): 13 checks -- a minimal repro per root cause now yields a clean bounded outcome, and regressions confirm fft(v(1))/deriv(v(1)), a valid `?:`, and ordinary indexing still work. Regression 184/184 (new example folder).

E-226 *src/frontend/rawfile.c, examples/rawfuzz_examples/**

Rawfile-load crash hardening (fuzzing). Continuing the fuzzing campaign onto another untrusted-input surface: the nutmeg rawfile reader (`raw_read` in frontend/rawfile.c, reached by the `load` command). A `.raw` file is external data (often written by another tool), so its parser must survive malformed input. Mutating a valid `.raw` (produced by `write`) -- tweaking the header counts, corrupting the Flags:/Variables: lines, adding/removing value rows, truncating, flipping bytes -- and `load`-ing each found a NULL-dereference crash (SIGSEGV, memmove into 0x0). ROOT CAUSE: raw_read resets `flags = VF_PERMANENT` at the start of each plot and sets VF_REAL / VF_COMPLEX only when it parses a `Flags:` line; each vector is then allocated with dvec_alloc(NULL, SV_NOTYPE, flags, npoints, NULL), which allocates v_realdata OR v_compdata according to those bits. A rawfile with NO valid Flags: line -- the line missing entirely, or carrying only unknown flags (`Flags: xyz`) -- leaves flags with neither VF_REAL nor VF_COMPLEX, so dvec_alloc allocates NO data array at all, and the value-reading loop does fread(&v->v_realdata[i], ..) / writes v->v_compdata[i] into a NULL buffer -> memmove into 0x0 -> SIGSEGV. (A bad No. Points: / No. Variables: count -- huge, negative, non-numeric -- was already handled cleanly; only the missing TYPE was fatal.) FIX: when the header is complete and vector allocation begins (at the Variables: line), if flags carries neither VF_REAL nor VF_COMPLEX, default to real (the common case) with a warning instead of allocating a typeless vector; the VF_PERMANENT reset is per-plot so the guard runs for every plot in a multi-plot file, and a valid file is unaffected. Also fuzzed in the same pass: the interactive `.control` command interpreter and shell layer (variable substitution `\$var`, `alter/altermod/alterparam`, analysis reruns, structured control if/while/foreach/repeat/dowhile) -- clean over 7500 iterations (already hardened by E-212/E-222/E-225). Scope: frontend only, one file (rawfile.c); no device/solver/OSDI change. Verify (examples/rawfuzz): 4 checks -- a valid .raw is written, then loaded with its Flags: line removed and with an unknown `Flags: xyz`, each now a clean bounded outcome (was SIGSEGV); a regression confirms a valid .raw still round-trips write -> load with the right length. Regression 185/185 (new example folder).

E-227 *src/frontend/snp2va.c, examples/snpfuzz_examples/**

Touchstone **`.sNp`** parser crash hardening (fuzzing). Continuing the fuzzing campaign onto the Touchstone S-parameter parsers -- external network-data files (VNAs, EM solvers, other tools), so their parsers must survive malformed input. Two exist: `rdsnp` (the E-72 Touchstone-v1 reader) and `pre_snp` (the E-200/E-201 convert-to-Verilog-A path in frontend/snp2va.c that vector-fits the data and compiles it through openvaf-r). Mutating valid .s1p/.s2p/.s3p files -- corrupting the # option line, data values (NaN/Inf/overflow tokens, added/removed columns), duplicating/deleting rows, truncating, flipping bytes -- AND mismatching the port count encoded in the `.sNp` filename extension, then running each through both readers: `rdsnp` was clean (4,500 iters); `pre_snp` hit a heap-corruption crash (SIGSEGV) 103 times, ALL on a filename with an absurd port count (e.g. `.s2147483647p`). ROOT CAUSE: snp2va.c infers the port count from the extension (`N = atoi( dot+2)`) with no upper bound; a huge N slips past the parse itself because the record stride `rec = 1 + 2*N*N` and the per-record inner-loop bound `N*N` use the same wrapped int (N*N overflows to a small/zero value, so the token layout stays self-consistent and no error is raised), but N is stored in out->N and used DOWNSTREAM to size the vector-fit allocations (`malloc(nf*N*N*sizeof( cplx))`, `malloc(N*N*sizeof(cplx))`) -- the overflowing/mismatched allocation and the writes that follow corrupt the heap (the program aborts later on an unrelated malloc, hence flaky). The brute-force fallback (used when the filename gives no usable count) already caps inference at 512 ports, so 512 is the parser's own sanity bound -- the filename path just wasn't held to it. FIX: hold the filename-derived count to the same limit -- `if (N > 512) N = 0;` falls back to inferring N from the data. A valid file is unaffected (its extension is far below the cap) and a genuinely mis-extensioned file now recovers N from the data layout instead of trusting an absurd filename. Scope: ngspice frontend only, one file (frontend/snp2va.c); no device/solver/OSDI change. Verify (examples/snpfuzz): 2 checks -- a valid .s2p copied to a `.s2147483647p` name loads via pre_snp

without crashing (run repeatedly, was SIGSEGV), and a valid .s2p still converts through the full pipeline (pre_snp -> .va -> openvaf-r -> .osdi). Regression 186/186 (new example folder).

E-228 *src/osdi/osdiregistry.c, examples/osdifuzz_examples/**

OSDI **`.osdi`** loader descriptor-count hardening (fuzzing). A `.osdi` model is an external, compiled shared library that ngspice `dlopens` and reads descriptor metadata from (`load_object_file`, reached by the `pre_osdi` command). The loader strided/indexed the descriptor arrays by the counts the file declares -- `OSDI_NUM_DESCRIPTORS` (outer loop stride by `*OSDI_DESCRIPTOR_SIZE`) and each descriptor's `num_params` (inner loop indexing `param_opvar[param_id]` and dereferencing its `name[]` pointers), `num_opvars/num_terminals/num_noise_src/...` (offset arithmetic) -- with NO bounds. A `.osdi` that still dlopens but declares a count larger than its real array (a truncated file, a byte-corrupted one, or one built by a compiler whose OSDI descriptor ABI differs -- the drift the loader's own comments warn about) read past the array -> SIGSEGV during pre_osdi (same unbounded-count class as E-227, here on a binary rather than a text surface). FIX: two generous ceilings in osdiregistry.c (`#define OSDI_MAX_DESCRIPTORS 4096u`, `#define OSDI_MAX_DESC_COUNT (1u<<20)`) far above any real compact model (the biggest ship a few dozen modules and ~1k params); after reading OSDI_NUM_DESCRIPTORS, reject the file (clean diagnostic, INVALID_OBJECT) if it exceeds the first, before it sizes the registry or strides the array; inside the descriptor loop, reject if any of a descriptor's count fields (num_nodes/num_terminals/num_params/num_instance_params/num_opvars/num_noise_src/num_jacobian_entries/num_collapsible/num_states) exceeds the second, before those fields index/size anything -- a single load-time choke point protecting every downstream consumer (setup, eval), since a rejected file never registers its devices. HONEST SCOPE: this targets the egregious/out-of-range count (the signature of a truncated/byte-corrupted/ABI-drifted object, where corruption reliably lands -- an ABI mismatch reads a pointer's bits as a count -> huge value); it is NOT a claim that the loader is proof against every malformed .osdi -- a count that lies within the plausible range is an internally inconsistent binary indistinguishable from a valid one, and a .osdi is native code whose mere loading executes it, so a hostile object is out of scope by construction. On macOS byte-corrupting a signed .osdi also breaks its code signature so dlopen rejects it before the loader runs; the residual exposure is chiefly Linux (no signing gate) and toolchain/ABI drift. Scope: ngspice only, one file (osdi/osdiregistry.c); no device/solver/compiler change. Verify (examples/osdifuzz): 3 checks -- two malformed .osdi built from tiny C stubs (one declaring a huge OSDI_NUM_DESCRIPTORS, one with a valid count but a huge per-descriptor num_params built from the real OsdiDescriptor struct in osdi.h) now load-reject cleanly (both a SIGSEGV rc=139 on the pre-fix binary), and a positive control confirms a real openvaf-r-built .osdi still loads and simulates (v(a)=1). Regression 187/187 (new example folder).

E-229 *src/frontend/com_dl.c, src/frontend/commands.c, src/spicelib/devices/dev.c, src/spicelib/devices/dev.h, examples/osdireload_examples/**

`pre_osdi -f`: reload a recompiled OSDI model without restarting. A Verilog-A developer's inner loop is edit .va -> recompile .osdi -> re-run, but interactively ngspice loaded each .osdi once: `load_osdi` (dev.c) is guarded by a path dedup, a dlopen-handle dedup (osdiregistry.c), and a device-name dedup (a type already in the global DEVices[] is ignored because the model-card lookup scans front-to-back and the first/stale entry wins -- see E-223); underneath, dlopen caches by path (a re-open returns the cached handle, never re-reading the recompiled file). So a recompiled model never took effect without restarting. FIX -- an opt-in `-f`/`-force` flag: `com_osdi` (com_dl.c) parses `-f` anywhere in the argument list and passes a force flag to `load_osdi(path, force)`; on a forced reload of an already-loaded path it STAGES a byte-for-byte copy of the (recompiled) file under a fresh unique path and loads that (a new path is always re-read, sidestepping dlopen's cache; this avoids dlclose, which on macOS does not reliably unmap and where overwriting a still-mapped file faults the process), then `removes` the staged copy once it is mapped (the mapping keeps it alive on POSIX), and `osdi_add_device(..., replace)` swaps the registered device to the freshly loaded descriptor IN PLACE at its existing table index so no other device type's index

shifts. The previous SPICEdev and the library mapping it points into are intentionally left resident: a circuit still built against the old model keeps a valid device, and freeing descriptor-owned memory would risk a double free (documented caveat: don't re-run a pre-existing circuit built against the prior model version after reloading it). Behaviour is unchanged without `-f` (plain re-load still skipped, now with a hint pointing at `-f`); the `osdi` command help gains the flag. Scope: frontend + device registry, four files (frontend/com_dl.c, frontend/commands.c help text, spicelib/devices/dev.c, spicelib/devices/dev.h); no solver/analysis/OSDI-ABI/compiler change. Verify (examples/osdireload): compiles a resistor model at 1k then 2k, drives ngspice in pipe mode to load the 1k version (i(v1)=-1mA), recompiles the same file to 2k mid-session, confirms a plain re-load is skipped (still 1k), then `osdi -f` reloads it and the operating point changes to the 2k result (i(v1)=-0.5mA) -- one process, no restart. Regression 188/188 (new example folder).

E-231 *src/frontend/com_gnuplot.c, src/frontend/plotting/gnuplot.c, examples/csv_examples/**

CSV output for **`wrdata`** (**`set wr_csv`** + **`wrdata -csv`**). `wrdata` only ever wrote whitespace-separated columns (each vector prefixed by its own scale column); the knobs `wr_singlescale`/`wr_vecnames`/`wr_onespace`/`numdgt` tune spacing and precision but never the separator, so a genuine comma-separated export needed a post-processing pass. The only `-csv` in the tree was `show -csv` (device-parameter tables, not data). ADDED a first-class CSV mode. (1) `set wr_csv` -- a boolean read in `ft_writesimple` (plotting/gnuplot.c) alongside the existing wr_* vars; when set it takes a dedicated, self-contained branch that writes a single shared scale column once (implies wr_singlescale, so the existing equal-length sanity check applies), a header row of vector names (implies wr_vecnames), and comma-separated values with `"%.*e"` (no sign-padding, no trailing separator) honouring numdgt; complex .ac vectors become two columns (real, imag). The branch writes and returns before the space-formatted writer, so DEFAULT wrdata output is byte-for-byte unchanged. (2) `wrdata -csv <file> <vec...>` -- a per-call alias accepted in ANY argument position: `com_write_simple` (com_gnuplot.c) scans the wordlist for a `-csv` token, enables wr_csv for just that write, and restores the prior global state afterward (no leak). Because `plotit` COPIES its wordlist (never mutates/frees the caller's), the `-csv` node is spliced out for the call and relinked afterward, leaving the caller's list intact to free; the head case is detected as `csvnode == wl` (the incoming head's wl_prev is not necessarily null), and no-vector forms fall through the `if (wl)` guard writing nothing, node still relinked. Scope: ngspice frontend only, two files (frontend/com_gnuplot.c, frontend/plotting/gnuplot.c); no solver/analysis/device/OSDI/compiler change; default output preserved exactly. Verify (examples/csv_examples, 9 checks): default output unchanged (no commas); `set wr_csv` writes a header + comma-separated values with the right numbers; `wrdata -csv` in first/mid/last position is byte-identical to the option form; CSV numbers equal the default-format numbers bit-for-bit; the flag does not leak into the next plain wrdata; .ac emits real/imag columns under a frequency scale; .tran emits a time scale with uniform csv-parseable rows. Regression 190/190 (new example folder).

E-232 *src/maths/KLU/klusmp.c, examples/solverfix_examples/**

KLU solver-glue correctness hardening (source audit). A deep read of the active SMP dispatch layer -- `klusmp.c`, the ONLY sparse-matrix-package layer compiled when KLU is enabled (`sparse/Makefile.am` builds `spsmp.c` only `if !KLU_WANTED`; klusmp.c routes BOTH solvers via its `CKTkluMODE` branches) -- found four latent defects, all fixed, all behaviour-preserving for any circuit that actually solves (prior functional KLU bugs E-113/114/115/171/172 were already fixed and are confirmed still in place). (A) `SMPluFac` and `SMPsolve` dereferenced `KLUmatrixCommon->status` BEFORE their `== NULL` guard; the complex twin `SMPcLUfac` checks NULL first with an early return -- reordered both to match (SMPsolve, being void, now reads status only inside the non-NULL else). Common lives for the matrix lifetime so it is never actually NULL here; the fix aligns the paths and satisfies a static analyzer. (B, the substantive one) the complex solves `SMPcSolve`/`SMPcaSolve` copied the RHS with a plain identity map (`RHS[i+1]`), while the real `SMPsolve` routes RHS through the node-collapse permutation `KLUmatrixNodeCollapsingNewToOld`; if a structural-zero column ever collapsed a node (map != identity) the AC/noise/pz RHS

would be silently mis-ordered relative to the DC/tran solve. Dormant -- the map is the identity for any circuit that factors (an empty MNA column is structurally singular and does not solve), which is why KLU AC/noise/pz have always matched Sparse -- but the two paths were inconsistent; both complex solves now apply the identical gather/solve/scatter so a future change to collapse cannot quietly break complex analyses. (C) `SMPcZeroCol` indexed `KLUmatrixAp[Col-1]` with no ground guard (`Ap[-1]` if Col==0); added the `Col >= 1` guard its sibling `SMPfindElt` already has (pole-zero passes indices >= 1, so it never fired). (D) `SMPmultiply`'s real branch ended with a dead `iSolution = iRHS;` (reassigns a by-value local; a real matrix has no imaginary product) -- removed. Scope: ngspice only, one file (maths/KLU/klusmp.c); no analysis/device/OSDI/ compiler change and no behavioural change for any solvable circuit. Verify (examples/solverfix, 4 checks): on an ASYMMETRIC net (a VCVS makes the MNA matrix non-symmetric, so the noise ADJOINT solve genuinely differs from the forward solve) KLU still agrees with SPARSE 1.3 to the bit on AC, on the noise spectrum (exercising the most-changed SMPcaSolve), and on pole-zero roots (SMPcaSolve + SMPcZeroCol + SMPcAddCol), plus a clean DC-op + transient run under KLU. Regression 191/191 (new example folder).

E-233 *src/maths/KLU/klusmp.c*

KLU glue deeper audit: finish the null-check reorder + correct the collapse map. Follow-up to E-232 from a deeper read of the same file; two more latent defects, both behaviour-preserving (one path never triggers, the other is confirmed-dead code). FIRST two scarier candidates were investigated and CLEARED empirically, not just by inspection: (i) a suspected per-analysis memory leak -- `.pz`/`.sens` reuse ckt->CKTmatrix and re-call SMPconvertCOOtoCSC + SMPpreOrder without freeing the prior arrays/Symbolic, which LOOKS like a leak -- was REFUTED by instrumenting the lifecycle over op->pz->sens: every convert/preorder sees a FRESH matrix (Ap==NULL, Symbolic==NULL) and SMPnewMatrix/SMPdestroy are balanced (5 new / 4 destroy / 0 overwrite-without-destroy); ngspice tears down and rebuilds the matrix per analysis, so nothing leaks (no fix made -- no bug). (ii) node-collapse is DEAD code: instrumenting the collapse branch and sweeping the whole 191-example regression (every OSDI model, pz, sens, noise) plus pathological topologies fired it 0 times, because OSDI collapses nodes at allocation time (osdisetup.c: a collapsed node gets no matrix column) so the COO never has a gap and Gmin fills every diagonal. FIXES: (3) E-232 fixed the dereference-before-NULL-check in SMPluFac/SMPsolve but the SAME pattern survived in `SMPcReorder` and `SMPreorder` (they read KLUmatrixCommon->status before the ==NULL guard) -- reordered both to check NULL first with an early return, matching SMPcLUfac, and corrected two mislabeled diagnostics (the Common-NULL and Symbolic-NULL branches each printed a stray 'KLUnumeric object is NULL'). Common is never actually NULL, so behaviour is unchanged. (2) in SMPconvertCOOtoCSC the structural-zero-column compaction recorded `NewToOld[reduced] = MatrixCOO[i].col`, but with TWO OR MORE gaps the RHS is a partially-reduced index from an earlier pass, not the true original, so the map would resolve to the wrong node and the real SMPsolve gather/scatter would mis-address the RHS -- fixed by chaining through the existing map (`NewToOld[reduced] = NewToOld[MatrixCOO[i].col]`, identity on the first pass so single-gap behaviour is unchanged); defensive since the path is confirmed dead. Scope: ngspice only, one file (maths/KLU/klusmp.c); no analysis/device/OSDI/compiler change and no behavioural change for any circuit. Verify: behaviour-preserving, so no new example -- the existing examples/solverfix_examples (KLU vs SPARSE bit-agreement on AC/noise/pz over an asymmetric net) plus the full 191/191 regression remain unchanged; fix 2's dead path cannot be exercised by construction (correctness is by the chaining argument). Regression 191/191.

E-234 *src/frontend/com_loadpull.c (new), src/frontend/com_loadpull.h (new), src/frontend/commands.c, src/frontend/com_commands.h, src/frontend/Makefile.am, examples/loadpull_examples/**

`loadpull`: power-amplifier load-/source-pull analysis. Load-pull (sweep the load impedance a PA output sees over the Smith chart and contour Pout/gain/PAE/efficiency to find the optimum load) is a PA-design staple ngspice lacked, and open-source load-pull is scarce. NEW `loadpull` command rides the existing .tran engine + `alter` (as optimize/sweep do) + `pyplot -contour` (

E-218). `loadpull -load <R> <L> <C> -out <node> -drive <Vsrc> -f <f0> [-supply <Vsrc>]` sweeps Gamma over ‘

E-235 *src/frontend/com_stb.c, examples/stbfix_examples/**

`stb`: fix a latent use-after-free in the probe lookup. The defensive follow-up flagged in E-234. `com_stb` resolved its voltage probe with `INPretrieve(&name, symtab)`, which replaces the pointer with the interned symbol-table string -- the same memory the voltage source's own name field points at -- and does NOT free the old copy; the following `tfree(name)` therefore double-freed the source's live name (and the symbol-table entry). It never bit in practice because `stb` runs once with no re-setup, but it is real memory corruption and the identical defect that bit `loadpull` in E-234 (there the per-point re-setups re-created the drive's branch node from that freed, allocator-reused memory). Fixed by dropping INPretrieve (top-level device names need no subckt translation -- `findInstance` does its own name match) and lowercasing a private copy instead (ngspice stores instance names lowercased), so only our own copy is freed; as a bonus the probe lookup becomes CASE-INSENSITIVE -- `stb Vprobe Iprobe` used to fail ”no such probe source 'Vprobe'” and now resolves. The now-unused complex helper `stbsub` was removed to keep the build warning-free. Scope: ngspice frontend only, one file (com_stb.c); no solver/analysis/device/OSDI/compiler change; loop-gain results unchanged. Verify (examples/stbfix, 3 checks): a mixed-case probe name resolves and reports a loop gain (failed before); the loop gain is still correct (a gain-1e5 buffer loop -> ~100 dB DC loop gain, measured 99.91 dB); 60 repeated `stb` runs stay stable. Regression 193/193 (new example folder).

E-236 *src/frontend/com_measure2.c, src/frontend/com_measure2.h, src/frontend/measure.c, src/tclspice.c, examples/measovf_examples/**

`.meas`: fix a stack-buffer overflow on long measurement names. Found in a memory-safety deep dive; reproduces on the stock binary. `get_measure2()` formats each measurement result line with `sprintf(out_line, "%-20s= %.*e ...", mName, ...)` into the caller's fixed stack buffer `char out_line[1000]` (measure.c). `mName` is the measurement NAME token, taken verbatim from the `.meas <analysis> <name> <func> ...` card via `cp_unquote` -- an unbounded user string with no length check. A `.meas` name longer than ~1000 chars overruns out_line; on macOS the stack canary fires and the process aborts (SIGABRT, exit 134), elsewhere it is plain stack corruption. All ten result-formatting sites share the flaw. E-225 had already hardened the sibling `errbuf[100]` in this same file to snprintf (after a measure-syntax fuzz campaign), but the fuzzing never exercised a long measurement NAME, so out_line was missed. Fix: thread the destination buffer size through `get_measure2(... , size_t max_out_line, ...)` and convert all ten `sprintf(out_line, ...)` to `snprintf(out_line, max_out_line, ...)`. The three callers pass the real size -- `measure.c` batch `.meas` passes `sizeof out_line`, the autostop check and the `tclspice.c` interactive path pass `NULL/0` (all ten sites are already guarded by `if (out_line)`, so they never reach snprintf). An over-long name is safely truncated to 999 chars; the measurement value itself is computed and stored exactly as before. Scope: ngspice frontend only; no solver/analysis/device/compiler change; measurement results unchanged. Verify (examples/measovf, 2 checks): a `.meas` with a ~4000-char name no longer crashes (exit 0, was 134); normal measurements still correct (MAX 1.0, AVG 0.6, 0.1->0.9 rise time 0.8 ns). Regression 194/194 (new example folder).

E-237 *src/frontend/dotcards.c, src/frontend/vectors.c, examples/nameovf_examples/**

`.print`/`.plot`/`.four`: fix stack-buffer overflows on long vector/node names. Sibling of E-236, same deep dive. The SPICE2-compat rewrites for output tokens, and the vector-name helper they feed, each copied an unbounded user name into a fixed BSIZE_SP[512] stack buffer: `fixem()` (dotcards.c) rewrites `v(a,b)->v(a)-v(b)` (+ vm/vp/vi/vr/vdb variants) via `sprintf(buf, "v(%s)-v(%s)", a, b)` (19 writes); `gettoks()` (dotcards.c) rewrites `i(x)->x#branch` via `sprintf(buf[513], "%s#branch", x)`; `vec_basename()` (vectors.c) does `strcpy(buf, v->v_name)`, reached by `.print/fft/spec/linearize`. A `.print/.plot/.four` output token whose node/

branch name(s) exceed the buffer overran the stack -- macOS aborts (SIGABRT/SIGTRAP, shell exit 133), elsewhere plain corruption. Fixing fixem alone was NOT enough: with valid long node names the rewrite succeeds and the long `v(a)-v(b)` name then flows into vec_basename and crashes there, so all three needed hardening. Fix: size each scratch buffer to its input -- fixem allocates `strlen(string)+32` and every write is a bounded `snprintf(buf, bufsz, ...)`; gettoks builds the string with `tprintf`; vec_basename replaces the fixed buffer + strcpy with `copy()` of the source (allocates to fit), preserving branch semantics. No truncation, so a valid long differential still computes the correct value. The vectors.c recompile also surfaced 3 pre-existing `-Wconversion` warnings (v->v_type, enum simulation_types, passed to dvec_alloc's int param), silenced with explicit `(int)` casts (build stays warning-free). Scope: ngspice frontend only; no solver/analysis/device/compiler change; output values unchanged. Verify (examples/nameovf, 4 checks): long `v(a,b)` on nonexistent nodes no longer crashes; a VALID long differential v(A,B) with v(A)=2,v(B)=1 runs and prints v(A)-v(B)=1.0 exactly (no truncation); long `i(x)` via `.four` (gettoks) no longer crashes; short `v(1,2)` still rewrites to v(1)-v(2)=2.0. Regression 195/195 (new example folder).

E-238 *src/frontend/dotcards.c, examples/malftoken_examples/**

`gettoks`: fix a NULL-deref crash on a malformed differential token. Third find of the same deep dive (after E-236/E-237), from fuzzing degenerate output tokens; reproduces on the stock binary. `gettoks()` parses the output tokens of `.save`/`.print`/`.plot`/`.four` (via ft_savedotargs) and `.measure` vars (via com_measure2). For a differential `v(a,b)` it finds the comma `c` and close paren `r` (`r = strchr(t, ')')`, `c = strchr(t, ',')`) and splits the second operand at `r`: `if (c != r) { *r = '\0'; ...}`. A MALFORMED token like `v(1,` has a comma but NO ')', so `r` is NULL while `c` is not -- `c != r` is then true and `*r = '\0'` dereferences NULL -> SIGSEGV (EXC_BAD_ACCESS at 0x0; lldb: gettoks <- ft_savedotargs). A one-character typo (missing paren) in a `.print`/`.save`/`.plot`/`.four`/`.measure` token crashes the simulator. Fix: require `r` non-NULL before splitting -- `if (r && c != r)`. When there is a comma but no ')', the split is skipped and the token degrades to a harmless parse; every well-formed case is unchanged (`v(a,b)` both non-NULL c!=r -> split; `v(a)` c==r -> skip; `i(x)` sets c=r=NULL -> skip). Scope: ngspice frontend only, one guard in dotcards.c; no solver/analysis/device/compiler change; well-formed parsing unchanged. Verify (examples/malftoken, 3 checks): `.print tran v(1,` no longer crashes; the malformed shape across .save/.plot/.four/.meas + i()/vdb()/vm() no longer crashes; well-formed `v(1,2)` still rewrites to v(1)-v(2)=1.5. A broad malformed-token sweep (prefixes×bodies×cards, hundreds) is crash-free after the fix. Regression 196/196 (new example folder).

E-239 *src/spicelib/parser/inpptree.c, examples/funcarity_examples/**

Expression parser: fix a NULL-deref on a one-argument **`min`**/**`max`**/**`pow`**/**`pwr`**. Fifth find of the same deep dive (after E-235–E-238), from fuzzing the behavioral-source expression parser; reproduces on the stock binary. `PTeval()` (ifeval.c) evaluates the two-argument functions pow/pwr/min/max by reading their operands as `tree->left->left` and `tree->left->right` -- it assumes the argument is a PT_COMMA pair. The function-node builder `PT_mkfnode()` (inpptree.c) looks the name up but NEVER validates argument count, so a one-argument call like `min(1)` builds a node whose `left` is a scalar PT_CONSTANT (its `->left` is NULL); at load time PTeval does `tree->left->left` -> NULL deref -> SIGSEGV (PTeval <- IFeval <- ASRCload <- CKTload). Reachable from any B-source, E/G source, or `.param` expression -- a one-line typo crashes the simulator during setup. Crash is specific to these 4 functions and the scalar (non-comma) one-arg form: `min()` is caught by the existing `if (!arg)` guard, `min(1,2,3)` yields a PT_COMMA (no deref), and single-arg forms of other functions (`hypot(1)`, `atan2(1)`) fall through the single-arg path and error cleanly. Fix: reject the wrong arity at parse time in PT_mkfnode -- 'if ((POW

E-240 *src/xspice/icm/analog/s_xfer/cfunc.mod, bin/*/codemodels/analog.cm (rebuilt), examples/sxfer_examples/

XSPICE **`s_xfer`**: fix an out-of-bounds crash on a static-gain transfer function. Sixth find of the

same deep dive (after E-235–E-239), from fuzzing XSPICE a-device instantiation; reproduces on the stock binary. First find in the XSPICE code-model library rather than the core simulator. The `s_xfer` analog code model (a Laplace transfer function H(s)=num/den in controller-canonical form) allocates its `integrator` state array to `den_size` elements, then in the DC/transient branch unconditionally read `pout_pin = *(integrator[1])` (the d(out)/d(in) partial) -- assuming a first- or higher-order denominator. A 0-order denominator (`den_size == 1`, i.e. a STATIC-GAIN transfer function like `s_xfer(num_coeff=[k] den_coeff=[1])` = constant k) allocates `integrator` with a single element, so `integrator[1]` is one past the end -> read garbage -> SIGSEGV at load (cm_s_xfer <- MIFload <- CKTload). Depends only on den_size==1, NOT the coefficient values: `den_coeff=[1]` (valid unity denominator), `[5]`, `[0]` all crash; `[1 1]`/`[1 2 1]` fine -- so it bit ordinary static-gain input, not just degenerate decks. AC branch already safe (loops bounded by den_size/num_size). Fix: guard the read -- `if (den_size > 1) pout_pin = *(integrator[1]); else pout_pin = (num_size>0) ? *gain * *(num_coefficient[0]) : 0.0` (for den_size==1, out = gain*num[0]*in, so the partial is gain*num[0]); *the output value was already computed correctly. Zero denominator (H=inf) now gives clean non-convergence instead of a crash. Fix is in a .cm code model, so analog.cm was regenerated via cmpp and redeployed under bin//*codemodels/ ; the ngspice binary is unchanged. Scope: XSPICE analog code-model library only (cfunc.mod + rebuilt analog.cm); no core simulator/solver/analysis/device/compiler change; dynamic-filter results unchanged. Verify (examples/sxfer, 4 checks): static-gain s_xfer(num=[3] den=[1]) no crash & exact (H=3 -> 12 V); another static gain H=0.4 -> 1.6 V; zero denominator no crash; dynamic H=1/(s+1) still correct (

E-241 *src/frontend/com_fft.c, examples/fftnorm_examples/**

`fft`/`spec`: fix the amplitude normalization for non-power-of-2 records. First numerical-correctness (wrong-number, not crash) find of a correctness-audit campaign that checked ngspice analyses against analytic + numpy oracles. `com_fft.c` has two paths: FFTW3 (exact `length`-point transform, `scale = length/2`, correct) and a Green's radix-2 FFT (no FFTW3) that zero-pads the `length` input samples up to the next power of two N. The non-FFTW path normalized the single-sided amplitude by the PADDED size (`scale = N/2`) instead of the true sample count (`length/2`), so every FFT whose sample count is not a power of two reported amplitudes too small by `length/N` -- up to 2x. A `.tran` essentially never yields exactly 2^k points, so this bit the common case silently. The DC bin is the clean discriminator (always exactly bin 0, no window/scalloping ambiguity): a signal with DC offset D read `D*length/N`, e.g. a 1025-sample record (padded to 2048) reported a DC of 2.0 as 1.0009. `spec` carried the same error in its power normalization (`intres = N*N` instead of `length*length`; the FFTW path already used `length*length`). Fix: `scale = length/2` and `intres = length*length` in the non-FFTW path, matching FFTW. The frequency axis was already padding-aware (`freq[i] = i/span * length/N`), so only the magnitude scale needed the change; power-of-2 records (`length == N`) and FFTW-linked builds are unchanged. Validated against numpy as an independent oracle: after the fix ngspice's `fft` matches `numpy.fft.rfft` of the same zero-padded record (normalized by `length`) bin-by-bin to ~1e-9. Scope: ngspice frontend only, two normalization constants in com_fft.c; `fft`/`spec` amplitudes now change for non-power-of-2 records (they become correct). Verify (examples/fftnorm, numpy-free, 4 checks): DC offset 2.0 reads 2.0 on a non-power-of-2 record; same on a maximally-padded record (1025->2048, where the bug halved it to ~1.0); DC reading independent of sample count across four record lengths; `spec` PSD tone peak length-independent. Regression 199/199 (new example folder).

E-242 *src/spicelib/devices/nport/* (new device dir), src/frontend/snp2va.c, src/frontend/snp2va.h, src/frontend/com_presnp.c, src/spicelib/parser/inp2n.c, src/spicelib/devices/dev.c, configure.ac, src/spicelib/devices/Makefile.am, src/Makefile.am, examples/nport_native_examples/*, bin//ngspice (rebuilt)*

Native C n-port device + **`pre_snp -native`**: a pole/residue Y-parameter block stamped straight into the matrix, no OpenVAF compile. `pre_snp` (E-200/201/205) converts Touchstone -> Verilog-

A -> openvaf-r -> .osdi; the fit is cheap but the compile is the bottleneck and walls at ~24-32 ports (O(N*Np) laplace_nd sections, O(N^2) terms for full-rank coupling). This adds a built-in nport device (new src/spicelib/devices/nport/) that evaluates the same model Y_ij(s) = d_ij + s*e_ij + sum_k res_ijk/(s-p_k) directly. It stamps as a multi-terminal admittance (four-corner conductance per (i,j); no per-port branch unknowns -> N equations not 2N): DC stamps Y(0), AC the complex Y(jw) into the (real,imag) slots, and transient a trapezoidal companion -- d a conductance, s*e a capacitor, each res/(s-p) a first-order state dx/dt=p*x+u whose shared pole states x_jk are advanced once per load and parked in CKTstate0 (the stamp phase recovers the history B=x_{n+1}-a*u_j, so no per-instance scratch). Complex poles come as conjugate pairs so the summed current is real. It rides the generic N dispatcher (inp2n.c broadened from OSDI-only to also accept the nport model): N1 p1..pN ref mymodel / .model mymodel nport(file="m.nport"), ref an explicit reference terminal (tie to 0 for a ground-referenced Touchstone block; the extra terminal makes floating/differential blocks expressible). KLU supported via nportbindCSC.c (real/complex/complex->real bindings; a ground row/col keeps its Sparse trash pointer, like the RLC devices). snp2va.c was refactored so the shared parse+vector-fit lives in snp_fit(), feeding two emitters: pre_snp -osdi (default, unchanged VA->OSDI route) and new pre_snp -native, which writes the compact .nport for this device -- same fit, no compiler, scaling to hundreds of ports. Verify (examples/nport_native, 6 checks x 2 solvers, all vs closed-form oracles, no openvaf-r needed): RC one-port e-term (AC 1/Y + transient exponential); RLC one-port complex conjugate pole pair through resonance; Pi two-port off-diagonal d/e cross-coupling vs analytic MNA; pre_snp -native fits a known 2-port .s2p and reproduces its Y; a 20-port 6-pole full-rank model vs an analytic linear solve. Cross-checks (dev notes): native AC == OSDI backend to 1e-11 on 137mhz_bpf.s2p; a 100-port block solves an AC point in ~90 ms (4.85e-10 vs analytic); KLU == Sparse bit-for-bit. Scope: ngspice only, additive; default -osdi path unchanged so existing decks are unaffected; no solver/analysis/compiler change. Regression 200/200 (new device dir + example folder).

E-243 *src/frontend/snp2va.c, examples/presnp_examples/verify_presnp.py, bin/*/ngspice (rebuilt)*

pre_snp -osdi: emit an explicit **ref** terminal so the instance line is identical for both **-osdi** and **-native**. E-242's native nport device carries an explicit reference terminal (N1 p1..pN ref model), but the original pre_snp -osdi emitted a Verilog-A module with only the N signal ports referenced to implicit global ground (N1 p1..pN model) -- so the two backends were not drop-in interchangeable. snp2va.c now emits module m(p1..pN, ref) and takes every branch relative to ref: I(pi,ref) <+ ... V(pj,ref) ... (and laplace_nd(V(pj,ref), ...), ddt(V(pj, ref))). ref is a PLAIN electrical port -- NOT a ground-declared node: that keeps it connectable (tie to 0 for a ground-referenced block, or any node for floating/differential), and it appears in every I(pi,ref) stamp so it is well-posed (declaring ground ref would make it an internal 0V node -- no longer a terminal -- and risks the E-116 all-zero-row/structurally-singular-KLU failure a ground node in no Jacobian entry caused). The instance line is now identical for both backends. Compatibility: the -osdi module goes from N to N+1 terminals, so existing -osdi decks must add the ref node (tie to 0 to reproduce the old behavior); the presnp example decks were updated. Verify: the identical line N1 p1 p2 0 model through both backends matches to 1e-11 on 137mhz_bpf.s2p (ref=0 reproduces the prior result exactly); examples/presnp (9 checks: 2-/3-/4-/8-port pre_snp cases, AC+tran+ordering) passes with the ref terminal. Scope: ngspice only, the VA emitter in snp2va.c; -native / .nport device / vector fit / .osdi compile path all unchanged. Regression 200/200.

E-244 *src/spicelib/devices/nport/nportsetup.c, src/frontend/com_pyplot.c, examples/crashfix2_examples/*, bin//*ngspice (rebuilt)*

Crash-hardening round 2: two user-triggerable crashes in recently-added code (both reproduce on the shipped binary). Found by argument-fuzzing the newer devices/commands. (1) nport device (E-242): NPORTsetup never checked that the instance line connected all N+1 nodes the .model's port count claims. A .nport with more ports than the instance wires (N1 a 0 mod for a 2-port), or

nports beyond NPORT_MAXTERMS, made setup stamp an UNBOUND (-1) terminal -> the generic N dispatcher leaves unconnected terminals as -1, and passing a negative row/col trips the sparse builder assert Row>=0 && Col>=0 (spbuild.c:267 -> SIGABRT) / out-of-bounds. Fix (nportsetup.c): reject a .nport whose port count leaves no room for the reference (N+1 > NPORT_MAXTERMS) and verify every node[0..N] >= 0 before stamping -> clean E_BADPARM with a diagnostic naming the shortfall. (2) pyplot (E-217/E-218): the -hist/-contour render-mode markers were stripped by UNLINKING AND FREEING a node of the command's OWN argument wordlist -- but that list is owned/freed by the command loop. When the marker was the FIRST word, com_pyplot freed the list HEAD, which the command loop then freed again -> use-after-free/double-free (SIGSEGV) on pyplot -hist v(out) / pyplot -contour .... (pyplot v(out) -hist, marker not first, was fine -- so the E-217/E-218 examples, which never put it first, always passed.) Fix (com_pyplot.c): detect the marker by scanning (no mutation) and build a filtered COPY without it, freed by us at the end; the caller's list is untouched. Bonus: -hist as first arg now renders the histogram it silently degraded to a line plot before. Verify (examples/crashfix2, 9 checks x2 solvers): under-bound + over-max nport decks exit gracefully with the diagnostic; a correctly-bound 2-port still simulates; pyplot -hist/-contour first-arg and -hist-alone no longer crash; -hist-first now emits plt.hist(); trailing-marker form still works. Scope: ngspice only, device input validation + command-arg ownership; no numerical change, valid usage unaffected. Regression 201/201.

E-245 *src/frontend/com_measure2.c, src/frontend/spiceif.c, examples/crashfix3_examples/*, bin//*ngspice (rebuilt)*

Crash-hardening round 3: two NULL-deref crashes in CORE ngspice command parsers (stock code, both reproduce on the shipped binary). Found by argument-fuzzing meas and altermod. (1) meas: measure_parse_stdParams (com_measure2.c) splits each trailing param token on '=' with strtok; a LONE '=' token (a stray one, e.g. meas tran m find v(x) when v(x)=0.5 = y) makes strtok return NULL for the name, which was then passed to strcasecmp() -> NULL deref SIGSEGV. The pValue==NULL branch was guarded but pName was dereferenced first. Only the interactive meas command hits it (the .meas dot-card pre-tokenizes differently); key=val/=val/val= are all fine, only a standalone '=' produces the NULL. Fix: treat a NULL pName as a clean syntax error before strcasecmp. (2) altermod: altermod nm c where the 2nd token is a bare device-type letter (c/e/i/...) or a digit makes com_altermod treat it as another MODEL to alter with no param=value, so parmlookup (spiceif.c) is called with param==NULL; its model-parameter loop passed that straight to eq()/strcmp() (the instance-parameter loop just above already guarded !param) -> NULL deref SIGSEGV. Input-dependent: only tokens resolving to a real device type crash (c/e/i/0/1/2), others are dropped earlier. Fix: guard the model loop against a NULL param (and defensively a NULL keyword), matching the instance loop. Verify (examples/crashfix3, 12 checks x2 solvers): the stray-'=' meas forms and altermod nm <c

E-246 *src/xspice/icm/analog/pwl/cfunc.mod, src/xspice/icm/analog/pwlts/cfunc.mod, bin//*codemodels/analog.cm (rebuilt), examples/pwlfix_examples/

XSPICE **pwl**/**pwlts**: fix an out-of-bounds heap read on x_array/y_array length mismatch. Second find in the XSPICE code-model library (after E-240's s_xfer), from fuzzing code-model parameters; confirmed with AddressSanitizer. pwl and pwlts take two INDEPENDENT vector parameters x_array (breakpoints) and y_array (ordinates); the XSPICE framework allocates each to its OWN length. Both models size their working table from x_array only (size = PARAM_SIZE(x_array)+2) then copy BOTH arrays with that single index (y[i]=PARAM(y_array[i-1])), so a y_array SHORTER than x_array reads past the end of the y_array parameter -> heap-buffer-overflow READ (ASan: cm_pwl cfunc.c:321 / cm_pwlts cfunc.c:202), pulling adjacent/uninitialised heap into the interpolation table (undefined behaviour; can crash outright once the mismatch runs past the mapped heap). Reachable from any deck with such an a-device (.model m pwl(x_array=[0 .25 .5 .75 1 1.5 2 2.5] y_array=[0 1])). The sibling models already guard this -- oneshot checks cntl_size != pw_size, multi_input_pwl checks PARAM_SIZE(x)!=PARAM_SIZE(y) -- but pwl/pwlts, the oldest PWL models, predate the check. A sweep of

the other analog models with paired arrays found no further cases (sine/square/triangle copy into their own cntl_size-sized buffers, so a short array zero-pads rather than overrunning; summer/mult are validated by the framework against the connection size). FIX: add the same equal-length guard -- `if (PARAM_SIZE(x_array)!=PARAM_SIZE(y_array)){cm_message_send(size_error);return;}` -- before the alloc/copy in both models, aborting setup cleanly with `x_array and y_array must have the same length` instead of reading out of bounds. Fix is in a .cm code model, so analog.cm was regenerated via cmpp and redeployed under bin/*/codemodels/; the ngspice binary is unchanged. Scope: XSPICE analog code-model library only; no core simulator/solver/analysis/device change; valid equal-length pwl/pwlts usage unaffected. Verify (examples/pwlfix, 5 checks x2 solvers): a valid pwl interpolates exactly (in 0.5 -> 1.0 V); pwl with x longer than y, and the other way, both report the size error without crashing; a valid pwlts still runs; a mismatched pwlts reports the size error; self-skips if codemodels unavailable. The OOB was reproduced and shown fixed under an AddressSanitizer build of the code models. Regression 203/203 (new example folder).

E-247 *src/xspice/icm/table/table2D/cfunc.mod, src/xspice/icm/table/table3D/cfunc.mod, bin//*codemodels/table.cm (rebuilt), examples/tablefix_examples/

XSPICE **`table2d`**/**`table3d`**: fix out-of-bounds read + interpolation UB on degenerate or too-small tables. Third area of the XSPICE code-model deep dive (after E-240 s_xfer, E-246 pwl/pwlts), found by fuzzing table dimensions; confirmed with ASan/UBSan. Both models read a lookup table from a file whose first lines declare the axis point counts (ix, iy [, iz]); the loader (init_local_data) never validated them, so a small table broke two ways. (1) OUT-OF-RANGE RAMP (heap OOB read): for an input past the table edge the model ramps the derivative to zero using xcol[1]-xcol[0] (low side) or xcol[ix-1]-xcol[ix-2] (high side); a single-point axis (ix==1) has no xcol[1]/xcol[-1] -> heap-buffer-overflow READ (ASan: cm_table2D cfunc.c:298); table3d has the same for all three axes. (2) ENO INTERPOLATION (OOB/UB): the Madagascar ENO of order p reads a 2*(p-1)-point stencil per axis (order 2 needs 2 pts, 3 needs 4, 4 needs 6, 5 needs 8); the order param DEFAULTS to 3 (needs >=4 pts/axis) but the loader only clamped it UP to a minimum of 2, never DOWN to what the table supports -> the default order on any table with a <4-point axis (a 3x3 table, or a 2-plane z axis like the shipped test-3d-1.table with iz=2) ran off the stencil -> UB/OOB in eno2.c/eno3.c. FIX (both models, right after the order>=2 floor): compute mindim = min(ix,iy[,iz]); reject mindim<2 with a clean error (a 1-point axis can be neither ramped nor interpolated); and clamp interporder to mindim/2+1 (the inverse of the 2*(p-1) stencil rule) so a small table interpolates at a reduced but VALID order instead of reading out of bounds. Tables large enough for the requested order are byte-identical (every shipped example has mindim/2+1 >= order; the smallest are 7x7). Fix is in .cm code models, so table.cm was regenerated via cmpp and redeployed under bin/*/codemodels/; the ngspice binary is unchanged. No regression coverage impact: the regression table_model example uses OSDI Verilog-A table models, not these XSPICE code models. Scope: XSPICE analog code-model library only; no core simulator/solver/analysis change; tables large enough for the requested order unaffected. Verify (examples/tablefix, 6 checks x2 solvers): a valid 8x8 order-3 table2d interpolates exactly (out=x+10y at (2.5,3.0) -> 32.5); a single-x-point table2d and a degenerate table3d (iz=1) are rejected with a clean error; small 3x3 / 3x3x3 order-3 tables run (order clamped) without crashing. The OOB read and the ENO UB were reproduced and shown fixed across a full (dimension x order) grid under ASan/UBSan builds. Regression 204/204 (new example folder).

E-248 *src/spicelib/devices/cpl/cplsetup.c, bin//*ngspice (rebuilt), examples/cplfix_examples/

CPL coupled-line device: fix two out-of-bounds accesses (conductor count + R/L/C/G matrix sizes). First find in a CORE ngspice device (the XSPICE dive E-240/246/247 was the code-model library); found by fuzzing the p-device conductor count and matrix sizes, confirmed with ASan/UBSan; reproduces on the shipped binary. A CPL line has noL conductors (the instance `dimension`, set from the node count on the `p` card); each symmetric R/L/C/G matrix is its upper triangle, noL*(noL+1)/2 values. CPLsetup/ReadCpL (spicelib/devices/cpl/cplsetup.c) validated

NEITHER count. (1) UNDER-SPECIFIED MATRIX (heap OOB read): ReadCpL fills the matrices with `f = CPLmodPtr(here)->Rm[counter]` (and Lm/Cm/Gm) where counter runs 0..noL*(noL+1)/2-1, but Rm/Lm/Cm/Gm are allocated to the count the user actually gave (Rm_counter, ...); fewer values than the triangle needs -> read past the end (ASan: heap-buffer-overflow READ at cplsetup.c:474). (2) TOO MANY CONDUCTORS (fixed-array overflow): ReadCpL uses RLINE *lines[MAX_CP_TX_LINES] and CPLine.in_node[MAX_CP_TX_LINES] with MAX_CP_TX_ LINES==8 (include/ngspice/swec.h), indexed by noL; a `p` card with >8 conductors writes past them (UBSan: index 8 out of bounds for 'NODE *[8]' at cplsetup.c:430). Both reachable from a valid-syntax netlist; on the release binary the reads pull adjacent heap -> heap-layout-dependent garbage (here a downstream fatal-error exit, but could equally be wrong results or a crash). FIX: validate both up front in CPLsetup, right after `noL = here->dimension` and before any allocation -- reject noL<1

E-249 *src/spicelib/devices/urc/urcsetup.c, src/spicelib/devices/ltra/ltraset.c, bin//*ngspice (rebuilt), examples/tlinefix_examples/

URC + LTRA transmission-line input validation: reject a degenerate/unbounded lump count and negative line parameters. Two CORE transmission-line devices accepted out-of-range parameters that gave a silently wrong result or a resource-exhaustion hang instead of a clean error (same class as E-248 CPL). (1) URC (urcsetup.c): the uniform-RC line is expanded at setup into a ladder of `n` (the `n=` lumps) resistor+capacitor(/diode) sections -- one node group per lump -- and n is also an exponent of pow(k,n); the user's n was NEVER bounds-checked. n<=0 ran the loop zero times -> no lumps -> output node left silently unconnected (v(out)=0, a wrong answer, no diagnostic); a large n (typo n=100000000, or 2e9 near INT_MAX) exhausted memory / hung building the ladder (even ~20000 lumps takes tens of seconds; the source even carried a `may want to limit lumps to <= 100` comment). Fix: require 1 <= n <= URC_MAX_LUMPS (1000; auto-computed count is ~3..30), clean E_BADPARM otherwise, validated after the auto-lump default so it covers both given and computed counts. (2) LTRA (ltraset.c): the lossy line picks its RLGC special-case with `!= 0` tests, so a NEGATIVE L or C passed and fell through to sqrt(L/C)/sqrt(L*C) in LTRAtemp -> NaN characteristic impedance/delay and a degenerate all-zero run (a zero/too-few-param line was already the clean "at least two ... must be ... nonzero" error). Fix: reject any negative R/L/G/C up front (per-unit-length params must be physical). Scope: core ngspice, two device setups; binary rebuilt; no solver/analysis/numerical change; a validly-specified URC/LTRA line is unaffected. Verify (examples/tlinefix, 8 checks x2 solvers): a valid 5-lump URC divider and a valid LTRA lossless line still simulate; URC n=0/-3/100000000 each rejected cleanly (the huge case INSTANTLY, no hang); URC n=1000 (max) still runs; LTRA negative C rejected cleanly while C=0 keeps the pre-existing nonzero error. Regression 206/206 (new example folder).

E-250 *src/xspice/icm/digital/d_lut/cfunc.mod, src/xspice/icm/digital/d_genlut/cfunc.mod, bin//*codemodels/digital.cm (rebuilt), examples/dlutfix_examples/

XSPICE d_lut/d_genlut: fix an undefined-behaviour 1«n shift on the input-port count. Found with UBSan while sweeping the XSPICE code-model library (E-240/246/247 dive). Both digital lookup-table models size their table as 2^(#inputs) via a left shift: d_lut `size=PORT_SIZE( in); tablelen=1<<size` (cfunc.c:143-144), d_genlut `isize=PORT_SIZE(in); entrylen=( 1<<isize)` (cfunc.c:154/177). The input-port count was NEVER bounded: a left shift of a 32-bit int by >=32 is UNDEFINED BEHAVIOUR (UBSan: "shift exponent 32 is too large for 32-bit type 'int'" at cfunc.c:146/:177), and a count in the high twenties asks for a multi-gigabyte table alloc. Reachable from a valid-syntax netlist -- an a-device with ~30+ digital inputs fanned into the LUT (e.g. all tied to one pulled-up net). On arm64 the release UB is benign (shift amount taken mod 32 -> 1«32==1, no crash) so the table size is silently WRONG rather than loud -- which is exactly why it's worth fixing. Fix: cap the input-port count at D_LUT_MAX_INPUTS / D_GENLUT_MAX_INPUTS (24; a real LUT has only a handful of inputs, and the table_values string is 2^n chars) BEFORE the shift, cm_message_send clean error + return otherwise. Fix is in .cm code models, so digital.cm was regenerated via cmpp and redeployed under bin/*/

codemodels/; the ngspice binary is unchanged. Scope: XSPICE digital code-model library only; no core simulator/solver/analysis change; a normal LUT is unaffected. Verify (examples/dlutfix, 4 checks x2 solvers): valid 2-input d_lut and d_genlut still simulate; a 32-input d_lut and d_genlut are each rejected with a clean "out of range" error, no crash. The 1«32 UB was reproduced under a UBSan build and shown fixed. Regression 207/207 (new example folder).

E-251 *examples/hb_examples/verify_hb.py (verification only -- no ngspice source change)*

Harmonic-balance tight nonlinear correctness verification. A rigorous tightening of the HB (E-134/135) correctness cross-check. The existing checks compare the HB spectrum against ngspice's own transient+fourier steady state, but at K=8 harmonics and a 1.5-3% tolerance. This proves -- and enforces in the regression -- that HB converges to the EXACT periodic steady state, not merely within a few percent. AUDIT: HB and tran+fourier are independent code paths, so their agreement is a real oracle once the transient reference is accurate. Using a purely RESISTIVE rectifier (cjo=0/tt=0 -> steady state reached instantly, so the transient's only error is its O(dt^2) timestep, which is Richardson-extrapolated to dt->0) and sweeping K=8..48, the HB harmonics converge MONOTONICALLY to the dt->0 transient; at K=48 the match is ~1e-7 (h1/h2 exact, h3 -1.9e-7, h4 -3.1e-6, h5 +8.9e-7). The 13% apparent 5th-harmonic 'error' at K=8 is entirely HB truncation (aliasing of the discarded harmonics), NOT a modelling error -- it vanishes as K grows. TIGHTENED CHECK (verify_hb.py [10]): on the resistive rectifier, HB at K=24 matches a fine-dt (period/2000) transient fourier to < 0.5% for DC..4th (6x tighter than the loose checks, observed worst-case ~2e-5), and raising K from 8 to 24 strictly shrinks the 5th-harmonic mismatch (encoding 'converges as K grows' so a future regression that broke HB's harmonic content would be caught). Scope: verification only (verify_hb.py); no ngspice binary / HB-engine / analysis change; documents that HB is EXACT, not merely within tolerance. Regression 207/207.

E-252 *src/xspice/icm/analog/xfer/cfunc.mod, src/xspice/icm/analog/file_source/cfunc.mod, bin//* codemodels/analog.cm (rebuilt), examples/filefix_examples/

XSPICE xfer/file_source: fix two heap OUT-OF-BOUNDS WRITES in the file parsers. Found while sweeping the file-parser code models (d_state/d_source/xfer/file_source); confirmed with ASan. Unlike the earlier code-model finds (E-246/247/250, reads or bounded errors) these are OOB WRITES -- silent heap corruption. (1) xfer (read_file): reads a transfer-function file (Touchstone-style # option line + data), sscanf's up to 9 values/line and a state machine stores EVERY value (freq/real/imag triples), so a line with more than one record stores >3; but the alloc check reserved only 3 (`if(i+3>size)`, ALLOC=1024) -> a multi-record line writes past the buffer at the 1024-double boundary (ASan: heap-buffer-overflow WRITE, cm_xfer cfunc.c:232). Fix: reserve the sscanf max of 9 (`if(i+9>size)`; the loop appends at most count<=9/line since offset>=1 && span>=offset+2 are enforced). (2) file_source: stores one record/line = a timepoint + `size` channels = stepsize(=size+1) doubles, but reserved only `size` (`vecallocated - size`), one short; at the realloc boundary (count lands exactly at vecallocated-size once vecallocated grows out of stepsize alignment) the last channel writes one double past the end (ASan: heap-buffer-overflow WRITE, cm_filesource cfunc.c:326). Fix: reserve a full record (`-stepsize`). Both are heap OOB WRITES reachable from a valid-syntax netlist + crafted data file; release build corrupts adjacent heap silently rather than always crashing (UB either way). Siblings checked: d_source validates per-line token count vs declared width; d_state's fixed line buffer is fgets-bounded -- no analogous overrun. Fix is in .cm code models, so analog.cm was regenerated via cmpp and redeployed under bin/*/codemodels/; the ngspice binary is unchanged. Scope: XSPICE analog code-model library only; no core/solver/analysis change; a well-formed data file is unaffected. Verify (examples/filefix, 4 checks x2 solvers): valid transfer-function + file_source files simulate; an xfer multi-record file and a boundary-crossing file_source file run without overrunning. Both reproduced under ASan (cm_xfer/cm_filesource) and shown fixed. Regression 208/208 (new example folder).

E-253 *src/frontend/com_rfstab.c, src/frontend/com_rfstab.h, src/frontend/commands.c, src/frontend/*

com_commands.h, src/frontend/Makefile.am, bin//ngspice (rebuilt), examples/rfstab_examples/

`rfstab`: two-port stability & gain report -- the first RF design-aid on top of the S-parameter machinery. ngspice can compute S-parameters (.sp / n-port / Touchstone) but not turn them into the stability/gain figures of merit an RF designer works with. After a .sp analysis publishes S_1_1.. S_2_2 vs frequency, the new `rfstab` command post-processes them into the standard linear-two-port metrics per frequency: determinant D=S11S22-S12S21; Rollett K=(1-

E-254 *src/frontend/plotting/pyplot.c, src/frontend/plotting/pyplot.h, src/frontend/plotting/plotit.c, src/frontend/com_pyplot.c, bin/*/ngspice (rebuilt), examples/pyplotsmith_examples/

`pyplot -smith`: Smith-chart plotting mode -- the second RF design-aid (after rfstab, E-253). A Smith chart is the working surface for two-port RF design (reflection coefficients, matching trajectories, stability/gain circles), but ngspice's modern matplotlib output (`pyplot`) had no Smith mode -- only the legacy in-terminal plotter did. `pyplot [name] -smith <complex vectors>` draws each listed complex vector (S_1_1, S_2_2, a load reflection Gamma, a stability/gain-circle trace) as a curve in the reflection-coefficient plane (Re->x, Im->y) over the standard Smith grid: the unit circle

E-255 *src/spicelib/analysis/cktdisto.c, bin/*/ngspice (rebuilt), examples/distoexact_examples/

`.disto` proven machine-exact vs Harmonic Balance / QPSS-HB + behavioral-source (B) distortion warning. RF/distortion correctness frontier, in the E-251 HB-proof mold. (1) PROOF: .disto (the 1990 Volterra small-signal distortion analysis) reports the pure 2nd/3rd-order kernels scaled by the DISTOF1/DISTOF2 magnitudes; HB (`hb`) and two-tone QPSS-HB (`qpss`) are INDEPENDENT large-signal engines including all orders, whose k-th harmonic / mixing amplitudes converge onto the Volterra result with ~A^2 leakage. So HB(A)->.disto as A->0 -- the HB-proof structure applied to distortion; because HB and .disto share ngspice's identical device model, model-constant ambiguity (VT, DC-op) cancels, so the agreement measures the .disto engine itself. Measured (diode + series R, both solvers): single-tone HD2 vs HB 2f tightens rel err 4.0e-5->2.6e-7 as A:1e-3->1e-4 (HB->disto), HD3 ~4e-6; two-tone IM3 (2f1-f2) vs QPSS-HB (2,-1) ~1e-5; .disto output scales EXACTLY as A^2/A^3 (DISTOF1/2 applied exactly, kernels amplitude-independent). The residual floors are the reference engine's own readout/

E-256 *src/maths/ni/niiter.c, src/spicelib/analysis/cktop.c, src/include/ngspice/cktdefs.h, src/spicelib/parser/ptfuncs.c, bin/*/ngspice (rebuilt), examples/bsrcconv_examples/

Fix SILENT spurious DC operating point for singular-derivative behavioral sources. A behavioral (B) source whose small-signal conductance is INFINITE at the v=0 initial guess -- B I=sqrt(v(n)) (dI/dv=0.5/sqrt(v)->inf), I=0.1/v(n), I=ln(v(n)), v^0.5 -- pins the node at v~0 so Newton takes a vanishing step; the convergence test (iterate-to-iterate VOLTAGE change only) sees no change and declares "converged" at a point that grossly VIOLATES KCL (0.42A imbalance on a 0.6V/1ohm divider). The false convergence also PRE-EMPTED the gmin/source-stepping fallbacks (they never ran). Silent + data-dependent (i=1m*sqrt(v) converges fine; a .nodeset finds the true root). The B-source AC derivative itself is correct (matches analytic dI/dv ~1e-10 across the function library) -- the defect is purely the DC convergence ACCEPTING a KCL-violating point. FIX (3 coordinated parts): (1) niiter.c false-convergence guard -- after the voltage test declares convergence, also check the KCL residual F=G*x-b (the node-current imbalance, already computed for the E-111 line search); if the worst node merit >100x the current tolerance the point is spurious -> decline convergence so CKTop falls through to gmin/source stepping, which regularizes the singular node and finds the TRUE point. Separation is decisive: false convergence sits at merit~1/reltol~1000, every legitimate circuit <1 (measured 1e-7..0.35). (2) First-attempt isolation -- a new one-bit CKTdcFirstTry flag (cktdefs.h) set by CKTop (cktop.c) ONLY around the initial plain-Newton attempt, so the guard never fires inside a convergence-aid sub-solve (gmin/source stepping, pseudo-transient/optran, convhelp); an earlier unconditioned version broke convhelp/ptcont/corenum/tempphys by firing inside their homotopies. (3) ptfuncs.c pwr(0,negative) guard -> HUGE instead of raw

+inf (matches the existing /0 and sqrt(neg) guards) so a singular derivative stays finite rather than NaN-poisoning the Jacobian. Result-neutral for every circuit that already converged. Verify (examples/bsrcconv, both solvers): [1] I=sqrt(v) reaches analytic v0=0.178045 with KCL satisfied (i_B1==i_R1), not the spurious v~0; [2] I=0.1/v reaches upper-branch v0=0.887298; [3] result-neutral: v^2/v^3/exp/tanh unchanged. Full regression 212/212 incl. the convergence-aid suites convhelp/ptcont/corenum/tempphys. Scope: DC Newton loop + a one-bit CKT flag + a parser math guard; binary rebuilt; confined to the first operating-point attempt.

E-257 *src/maths/ni/niiter.c, bin//*ngspice (rebuilt), examples/bsrcconv_examples/ *(verify [4])*

Extend the E-256 false-convergence guard to the TRANSIENT operating point. E-256 fixed the silent spurious DC operating point for singular-derivative behavioral sources but gated the guard to MODEDCOP only; the transient operating point (MODETRANOP -- the bias a `.tran` starts from without uic) was left uncovered and still false-converged. So a `.tran` of a biased `B I=sqrt(v(n))` started from the spurious v~0 and showed a FAKE startup transient (t=0 v(n)=9.3e-17 ramping 0->0.178 over ~20ms) when the biased circuit should sit at 0.178045 from t=0. E-256 gated to MODEDCOP conservatively because the pseudo-transient/optran aids run in transient modes -- but the guard already fires only when CKTdcFirstTry is set (the initial plain-Newton attempt), and optran runs as a FALLBACK with CKTdcFirstTry=0, so it is safe to broaden the mode condition to MODEDCOP

E-258 *src/maths/ni/niiter.c, src/spicelib/analysis/dctrcurv.c, bin//*ngspice (rebuilt), examples/bsrcconv_examples/ *(verify [5])*

Extend the false-convergence guard to the **`.dc`** sweep cold-start point. E-256/257 fixed the silent spurious operating point for singular-derivative behavioral sources for .op (MODEDCOP) and the transient op (MODETRANOP), but a `.dc` sweep still false-converged on its FIRST point: `.dc V1..` of a sqrt(v) source gave v(n)=9.3e-17 at the first sweep value (true root 0.178045) while every later point (warm-started from the previous solution, away from the v=0 singularity) was correct. Why the guard missed it: it fires only when CKTdcFirstTry is set, which happens only inside CKTop -- but in the default (non-HS) path the .dc sweep (dctrcurv.c) solves each point with a DIRECT NIiter call, bypassing CKTop, and only falls back to CKTop if that solve FAILS; the false convergence "succeeds", so CKTop (and the guard) never ran. FIX (2 parts): (1) niiter.c -- the E-256/257 guard, which enumerated the op modes (MODEDCOP

E-259 *examples/integaccuracy_examples/* (verification only -- no ngspice source change)*

Transient integration accuracy PROOF (oracle-based, both solvers). A permanent regression guard for the transient integrator, in the E-251 (HB) / E-255 (.disto) mold: rather than checking one waveform, it proves the integrator MATH PROPERTIES vs closed-form analytics + Richardson self-convergence. The audit that produced it swept the whole transient path and found it CORRECT. [1] ORDER of accuracy on RC decay exp(-t/RC): global error ~dt^p with the theoretical p -- TRAP p=1.94, Gear2 p=1.93, Backward-Euler p=0.94 (order test freezes the LTE step controller trtol=1e11 so steps are uniform and the convergence is purely truncation order). [2] ENERGY signature on an LC cos(wt) over 30 periods: TRAP amplitude ratio 0.9999 (marginally stable/energy-conserving, AND the trapezoidal-ringing BE-switch is NOT firing spuriously), Backward-Euler 0.057 (dissipative) -- the fingerprint of a correct trapezoidal vs BDF. [3] BREAKPOINT: a PULSE edge into RC matches the piecewise-analytic charge (max err 6e-7, pre-edge pinned 0). [4] RLC damped sinusoid vs exp(-at)(cos wd t + (a/wd) sin wd t) (max err 4.3e-6, correct wd/envelope). [5] NONLINEAR charge: a diode+CJO rectifier converges under Richardson (TRAP dt->dt/2) at order p=2.00, so the nonlinear device charge integration is 2nd order too. Both solvers give identical results (integrator is solver-independent). Separately confirmed (needs .options dynorder + set ngdebug, not asserted): higher-order Gear engages -- maxord=6 reaches order 6 and rejects ~10x fewer steps than order 2 (14 vs 143 on the RLC), so the high-order BDF coefficients are functional not a no-op. Scope: verification only; no source/binary change. Regression 213/213 (new example folder).

E-260 *examples/integaccuracy_examples/* (verify [6]; verification only -- no source change)*

LTE step-controller accuracy PROOF on a stiff circuit. Extends E-259 (which verified the integration METHODS with a FIXED step, freezing the LTE controller to isolate truncation order) with the complementary property: the ADAPTIVE local-truncation-error step controller actually DELIVERS the accuracy requested via reltol -- on a stiff circuit, the hard case. Two independent decays v_f=exp(-t/1us), v_s=exp(-t/1ms) (stiffness ratio 1000) integrated with one adaptive step (the controller must take tiny steps to resolve the fast decay then coarsen for the slow tail). Swept over reltol the delivered error vs the closed form shrinks MONOTONICALLY with no plateau: reltol 1e-3->1e-9 gives err(fast) 9.0e-3->2.0e-6 while internal steps grow 1032->1884. The error scales ~reltol^0.6 -- the theoretical global-error rate for a 2nd-order method under LTE control (reltol is a LOCAL per-step tolerance; step~reltol^(1/3) => global err~reltol^(2/3)). Separately confirmed (not asserted): extreme stiffness ratio 1e6 (tau=1ns/1ms) still tracks reltol, and a re-excited fast mode (1MHz square wave) is handled by refining ~6x more steps at each edge. Verify (examples/integaccuracy [6], both solvers): on the stiff circuit err at reltol 1e-3/1e-5/1e-7 is strictly decreasing and improves >=10x (measured ~247x). Completes the transient-engine proof (E-259 methods + E-260 step controller). Scope: verification only; no source/binary change. Regression 213/213.

E-266 *src/spicelib/analysis/cktsetup.c, src/spicelib/analysis/cktpzset.c, src/include/ngspice/cktdefs.h, examples/solverannounce_examples/* (binaries rebuilt by CI)*

Announce the direct linear solver once, not on every analysis. `CKTsetup()` (and `CKTpzSetup()`) printed `Using SPARSE 1.3 as Direct Linear Solver` (or the KLU line) unconditionally at the top of setup, and `CKTsetup` runs once per analysis -- so the `sweep` command, which sets the knob and re-runs the `-analysis` command for every point (`alter/altermod` in place, or `alterparam+reset` which re-sources the deck), reprinted the line on EVERY point (a 5-point sweep, 5 times; a 100-point sweep, 100). Monte Carlo, `optimize`, and the pso/de/sa optimizers -- all of which re-run the analysis per point -- were equally noisy. FIX: an announce-on-change helper `CKTannounceSolver(int klu)` (defined in cktsetup.c, prototyped in cktdefs.h, called by both cktsetup.c and cktpzset.c) prints the line only when the active solver differs from the last announcement. The last-announced solver is tracked PROCESS-WIDE (a function-static), not per-circuit, because a `.param` sweep re-sources the deck and rebuilds the circuit each point -- a per-circuit flag would still repeat. Consequences: a multi-point sweep/Monte/optimize run announces the solver ONCE; a single analysis still announces once (unchanged); a genuine solver switch (`.option klu`/`.option sparse`) has a different mode and re-announces; a fresh ngspice process starts un-announced, so batch runs and the dual-solver test harness (which re-execs once per solver) still print it once each, and the HB (verify_hb) and benchmark suites that detect KLU by grepping for the line still see it. Result-neutral: the line is informational and part of no numerical comparison; full dual-solver regression unchanged. Verify (examples/solverannounce, 4 checks): a 5-point sweep announces SPARSE once (was 5x); a single op still announces once; a sparse->klu switch re-announces both; a KLU analysis still prints the KLU line. Scope: 3 source files (one small helper + two call-site swaps + one prototype); no analysis/result change.

E-267 *src/frontend/com_sweep.c, examples/sweepbus_examples/* (binaries rebuilt by CI)*

`sweep` records array/bus nodes under their natural names (**`ph[0]`**, not **`ph_0_`**). Sweeping a circuit whose outputs are array/bus nodes -- a node named `ph[0]` (E-221) -- stored the result vectors under MANGLED names: `ph[0]` became `ph_0_`, `ph[1]` `ph_1_`, so the sweep plot listed `ph_0_`, `ph_1_`, ... The `sweep` command (E-146/-189/-190) builds each result-vector name from the output plus, for a `-vs` family or overlay waveform, an appended `_<knob>_<value>` segment; that segment carries a float (`_rt_1.5k`, with `./-/e` illegal in a nutmeg vector name), so the name builders `sw_familyname`/`sw_pointname` mapped every non-alphanumeric char (except _) to _ -- but over the WHOLE string, INCLUDING the user's base output name, destroying a bus node's brackets. FIX (com_sweep.c): (1) a helper `sw_append_sanitized(base, suffix)` sanitizes ONLY the appended `_<knob>_<value>` segment, leaving the base name byte-for-byte intact, so `ph[0]` stays

ph[0] (and a -vs family curve is ph[0]_rt_2k); ordinary names are unchanged (a plain node has no special chars; an explicit -output name=expr still uses the clean given name). (2) A bare -output token that is a bus range ph[0:3] now expands to one output per index (ph[0] ph[1] ph[2] ph[3]), matching the netlist bus expansion (E-221), so sweep rt .. -output ph[0:3] records the four taps directly. Values are unaffected -- only the NAME the result is stored under changes (and range-token expansion); each output is still evaluated exactly as before. Verify (examples/sweepbus, 4 checks, a 4-tap bus divider): -output ph[0:3] records four vectors named ph[0]..ph[3] (range expanded, natural names); the mangled ph_0_.. names are gone; the recorded values are the correct divider ratios; a plain -output vo=v(out) is unaffected. Full dual-solver regression unchanged (the existing sweep suites use the -output name=expr form, whose clean names are untouched). Scope: one source file; no analysis/result change.

E-268 *src/frontend/spiceif.c, src/frontend/device.c, src/frontend/com_sweep.c, src/include/ngspice/fteext.h, examples/sweepwild_examples/* (binaries rebuilt by CI)*

Wildcard model-parameter knob **@*[param]**. Multiple .model cards can share a model-parameter NAME (two Verilog-A models both with a wavelength parameter). Nothing let you sweep that shared parameter across all such models in ONE knob: sweep @dev1[wavelength] (altermod) targets one model, and the .param idiom (.param wl, wavelength={wl} in each card, sweep wl) works but its .param knob commits with reset -- RE-SOURCING THE ENTIRE DECK at every point (slow) and needing the cards to reference the param. NEW: @*[param] sets param on every loaded model that has it, IN PLACE (altermod, no reset), so sweep @*[wavelength] ... co-varies all of them fast; it also works standalone (altermod @*[wavelength]=1.55u). IMPL: (1) spiceif.c if_setparam_wildcard(ckt, param, val) -- model params are a device-TYPE property, so one parmlookup per device type decides whether that type's models carry param; every model of a matching type (walking ckt->CKThead[typecode]->GENnextModel) is set with doset (the same path a concrete altermod @model[param]= uses); returns the count set and runs CKTtemp to push into live instances when called mid-run; models/types without the param are skipped. (2) device.c com_alter_common -- a wildcard device name @* routes to if_setparam_wildcard; a concrete @model[param] or device is unchanged; if no loaded model has the param, one clear warning. (3) com_sweep.c sw_kind -- @*[param] classified as an in-place altermod knob (like @model[param]), so the sweep applies it without reset; prototype in fteext.h. Additive -- @* was not previously a valid model name, so no existing alter/altermod/sweep behaviour changes. Verify (examples/sweepwild, 4 checks, both solvers): two wlmodel cards (R=wavelength*1k) + one unrelated model without wavelength; sweep @*[wavelength] co-varies BOTH wlmodel devices (equal currents tracking 1/(wavelength*1k)); the unrelated model is untouched; a concrete @dev1[wavelength] still targets only dev1; @*[<absent>] warns and changes nothing. Full dual-solver regression passes. Four source files.

E-269 *src/frontend/spiceif.c, src/frontend/device.c, src/frontend/com_sweep.c, src/include/ngspice/fteext.h, examples/sweepwild_examples/* (binaries rebuilt by CI)*

Instance wildcard **@#*[param]** (alias **@*[[param]]**). E-268's @*[param] sets a MODEL parameter on every model that has it; this adds the INSTANCE counterpart -- set an instance parameter on EVERY device instance that has it, in place (alter/altermod, no re-source), so sweep @#*[scale] ... co-varies an instance parameter across all instances. Two spellings: @#*[param] (canonical; # marks instance) and @*[[param]] (alias -- the outer @dev[param] parse leaves param as [param, whose leading [is stripped). IMPL: (1) spiceif.c if_setparam_wildcard_instance(ckt,param,val) -- instance params are a device-TYPE property, so one parmlookup(...,do_model=0,...) per type gates membership; every instance (walking each model's GENinstances->GENnextInstance) is set with doset(dev=inst,mod=NULL) -> setInstanceParm (the path a plain alter @dev[param]= uses). (2) device.c alter_set -- the device-name token picks the wildcard: @#* (token #*) or @*[[param]] (token * + param starting [) -> instance wildcard; @* -> model wildcard (E-268); else ordinary if_setparam; no-match -> one warning. (3) com_sweep.c sw_kind -- @#*/@*[[...]] classified as in-place

altermod-style knobs (no reset); prototype in `fteext.h`. `@*`=models, `@#*`/`@*[[...]]`=instances; concrete `@dev[param]` unchanged. Additive -- these were not previously valid names. Verify (examples/sweepwild, 7 checks, both solvers) on a `wlmodel` with a MODEL param `wavelength` and an INSTANCE param `scale` (R=wavelength*scale*1k): model wildcard co-varies both model cards AND reaches `.model` cards inside SUBCIRCUITS (they flatten to per-instantiation model copies); instance wildcard `@#*[scale]` and alias `@*[[scale]]` co-vary every instance; concrete `@dev1[wavelength]`/`@NX2[scale]` target just one; absent-param wildcards warn. Full dual-solver regression passes. Four source files.

E-270 *src/frontend/com_sweep.c, examples/sweepbounds_examples/* (binaries rebuilt by CI)*

`sweep` validates its numeric bounds (ASan/UBSan fuzz find). An ASan/UBSan instrumented build fuzzing the newest command parsers (sweep/alter/altermod/loadpull/rfstab/stb/pyplot) surfaced a bug in `sweep`'s spec parser `sw_parse_spec`. `sweep` reads each bound of its `<start> <stop> <step>` (and 'lin

E-271 *src/frontend/com_let.c, examples/letoob_examples/* (binaries rebuilt by CI)*

`let` no longer reads one byte before its buffer on an empty LHS (ASan/UBSan fuzz find). The same command-parser fuzz that produced E-270 flagged a one-byte out-of-bounds READ in `com_let`. `com_let` flattens everything after `let` into a heap string `p`, splits the RHS off at the first `=`, then NUL-terminates the destination vector name at the first `[`. It next trims trailing whitespace from the name: `for (q=p+strlen(p)-1; *q<=' ' && p<=q; q--) ; *++q='\0';`. When the LHS is a bare bracket -- `let [[ = ...`, where the leading `[` has just been turned into `'\0'`, leaving `p` an EMPTY string -- or is entirely whitespace, `strlen(p)==0` so `q` starts at `p-1`; the loop test evaluates `*q` BEFORE the `p<=q` guard can short-circuit, dereferencing `p[-1]` -- a read one byte before the allocation (ASan: `heap-buffer-overflow READ of size 1` at com_let.c, from `let [[ = @#*[r] v1 -o i(v1)`). The stray byte does not fault deterministically on the shipped build (almost always valid heap metadata), but it is a genuine OOB read on malformed input and could fault if `p` sat at the start of a page. FIX (src/frontend/com_let.c): test the bound FIRST -- `for (q=p+strlen(p)-1; p<=q && *q<=' '; q--)` -- so `&&` short-circuits and `q` is never read while it points below `p`. An empty or all-whitespace name then falls through unchanged to the existing `"bad variable name"` check, which rejects it cleanly. No valid `let` is affected (a one-line reordering of a loop condition). Verify (examples/letoob, 5 checks): the fuzz-found `let [[ = ..`, a single-bracket `let [ = 5`, and an all-whitespace LHS each error quickly with `bad variable name` and no crash/hang; a plain `let a = 5` and an indexed `let a[1] = 7` still assign correctly; the original input under an ASan build no longer reports the overflow. Full dual-solver regression passes. One source file.

E-272 *src/frontend/device.c, examples/alternull_examples/* (binaries rebuilt by CI)*

`alter`/**`sweep`** no longer deref a NULL parameter for an m-named device (ASan/UBSan fuzz find). The same command-parser fuzz crashed on `sweep mag(v(b)) 0 1k 5k 1k`. `com_alter_common` (src/frontend/device.c) supports altering a device's PRINCIPAL value with no named parameter -- `alter r1 = 2k` -- in which case the parsed `param` is NULL. Just before applying the value it runs a binned-MOSFET guard: 'if ((dev[0]=='m') && (eq(param,"w")

E-273 *src/maths/cmaths/cmath2.c, examples/mathcast_examples/* (binaries rebuilt by CI)*

cmaths integer operators guard their **`(int)`** casts against out-of-range doubles (ASan/UBSan fuzz find). Continuing the expression-layer fuzz flagged cmath2.c:765 on `1e30 % 5`; it is one of a cluster where a routine casts a `double` operand to `int` with a plain cast (UB when the value is non-finite or outside int range). (1) `cx_mod` (the `%` operator) computed `(int)floor(fabs(op))` per operand -- `1e30 % 5` is UB (UBSan) and on the SHIPPED build returned a garbage 2 rather than an error (a silently wrong result). (2) `cx_vector`/`cx_cvector`/`cx_unitvec` set the result length with `(int)fabs(arg)` -- `vector(1e30)`/`unitvec(1e30)` produced a saturated (INT_MAX-ish) length and then allocated and filled it: a multi-gigabyte `calloc` (macOS over-commit lets it succeed)

plus a billion-iteration fill loop, i.e. an apparent HANG on the shipped build (rc=142 under a watchdog), not merely a sanitizer report. FIX (cmath2.c): range-check before every `(int)` cast. `cx_mod` floors each operand into a `double`, then `rchecks` it lies in [0-or-1, INT_MAX] before the cast; a non-finite/too-large operand takes the existing `argument out of range for mod` path (rcheck -> EXITPOINT -> NULL). A shared helper `cx_veclen()` validates the length for the three vector builders (`fabs/cmag` are >= 0, so one `<= INT_MAX` check also rejects NaN/inf); an out-of-range length prints `vector length ... is out of range` and returns NULL, which `apply_func` already handles (it checks `if (!data) return NULL`). Valid expressions unchanged -- `17 % 5 == 2`, `vector(4) == [0,1,2,3]`, `unitvec(3) == [1,1,1]`, and any length within int range behaves exactly as before. Pre-existing (cmaths, not touched by the recent command work). Verify (examples/mathcast, 5 checks): `1e30 % 5`, `vector(1e30)`, `unitvec(1e30)` each error cleanly and fast (no UB, no hang, no garbage); valid `17 % 5` and `vector(4)` still evaluate correctly. Full dual-solver regression passes. One source file.

E-274 *src/frontend/evaluate.c, examples/idxcast_examples/* (binaries rebuilt by CI)*

An out-of-range vector index no longer invokes undefined behaviour (ASan/UBSan fuzz find). A fuzz of the expression layer flagged evaluate.c:779 on `v(a)[1e308]`. `op_ind` (vector indexing `v[expr]`) rounds the index with `(int)floor(value+0.5)`; casting a double that is non-finite or outside int range to int is UB. The code ALREADY clamps the resulting index to `[0,majsize-1]` with a warning -- the only defect was the cast, which runs before that clamp. FIX (evaluate.c): a helper `idx_floor()` clamps the value to int range (NaN->0) before the cast, applied to all three casts (the real index and the real/imag parts of a complex `[lo,hi]` range index); `v(a)[1e308]` then resolves to the last element with the pre-existing warning, exactly as a large in-range index already did. Verify (examples/idxcast, 5 checks): `vx[1e308]`, `vx[1e30]`, a NaN index each resolve cleanly (no UB); valid `vx[0]` and range `vx[0:2]` still return the right elements. One source file; no change to any valid index.

E-275 *src/maths/cmaths/cmath4.c, examples/ifftreal_examples/* (binaries rebuilt by CI)*

`ifft()` on a real vector no longer reads past its buffer (ASan/UBSan fuzz find). `cx_ifft` unconditionally did `ngcomplex_t *indata = (ngcomplex_t *) data;`. For a VF_REAL input, `data` is a plain `double[length]` (length*8 bytes), so reading `indata[i]` for i<length walks `length` COMPLEX elements (length*16 bytes) -- twice past the buffer (ASan heap-buffer-overflow READ: a 528-byte/33-element region read as 66 complex). cx_fft (Enhancement-225) already distinguishes real and complex input; cx_ifft did not. FIX (cmath4.c): for a VF_REAL input build a proper complex array (imag=0) sized `length`, use it, and free it before returning; a VF_COMPLEX input still points straight at `data`; a `length>=2` guard (matching cx_fft) rejects a degenerate input. Verify (examples/ifftreal, 3 checks): `ifft(vx)` on a real vector runs with no overflow; the fft->ifft round-trip keeps the length; complex `ifft` still works. One source file; no change to a valid complex ifft.

E-276 *src/maths/cmaths/cmath2.c, examples/rndcast_examples/* (binaries rebuilt by CI)*

`rnd()` guards its **`(int)`** cast against an out-of-range operand (ASan/UBSan fuzz find). `cx_rnd` (the `rnd()` random-integer function) turned each operand into a modulus with `j = (int)floor( dd[i]); rand() % j`. Casting a double outside int range to int is UB, so `rnd(1e30)` (or inf/NaN) tripped UBSan (cmath2.c:162). This is the same class as Enhancement-273, which hardened cx_mod and the cx_vector builders but not cx_rnd (at the time `rnd(1e30)` did not reproduce; the expression fuzz found the path). FIX (cmath2.c): a helper `cx_rnd_i()` clamps the value to int range (mapping NaN to 0) before the cast, applied to all three casts (the real operand and the real/imag parts of a complex operand); the result is unchanged for any in-range operand. Verify (examples/rndcast, 3 checks): `rnd(1e30)` and `rnd(inf)` resolve cleanly (no UB); a valid `rnd(5)` still yields an integer in [0,5). One source file; completes the E-273 (int)-cast hardening for cx_rnd.

E-277 *src/maths/cmaths/cmath4.c, examples/derivcx_examples/* (binaries rebuilt by CI)*

`deriv()` of a complex vector: fix a heap overflow AND a wrong result (ASan/UBSan fuzz find).

The fuzz flagged cmath4.c:314 on `deriv((0.5,time))` (complex input). `cx_deriv` fits a degree-d polynomial over a sliding window; its COMPLEX branch diverged from its (correct) REAL branch in three places, each an out-of-bounds access on the last block AND a numerical error: (1) it read the data window `c_indata[j+i+base]` while fitting against the scale window `scale + i - degree + base` -- offset by `degree`, so it fit mismatched (x,y) pairs and read `degree` points past the end (heap-buffer-overflow READ); the real branch reads `indata + i - degree + base`. (2) The real-part output loop used `j <= i + degree/2` where the imag-part loop and the whole real branch use `j <= i - degree/2`, overrunning scale[]/c_outdata[]. (3) The tail indexed `scale[j+base]/c_outdata[j+base]`; the real branch's tail uses `j` (its FIXME notes j+base crashed), so a grouped (base>0) complex derivative overran both arrays. FIX (cmath4.c, complex branch only): align to the real branch -- read `c_indata[j + i - degree + base]`, bound the real-part output loop with `i - degree/2`, index the tail with `j`. Every overflow is gone AND the complex derivative is now numerically correct. Verify (examples/derivcx, 3 checks): `deriv((0.5,time))` runs with no overflow; `deriv(t + 2t*i)` now returns (1, 2i) -- the correct derivative, mid-vector; a real `deriv(2t)` still returns 2; a focused deriv stress over complex/real/grouped inputs is clean under ASan. One source file; real derivatives unchanged, complex derivatives now memory-safe and correct.

E-278 *src/maths/cmaths/cmath4.c, examples/scaleguard_examples/* (binaries rebuilt by CI)*

Transform functions integ/deriv/ifft guard a length/scale mismatch (ASan/UBSan fuzz find). Continuing the expression-layer fuzz, `integ/deriv/ifft` overran the heap on a vector whose length differs from its plot scale (cmath4.c:500 on integ(unitvec(200)), :1065 on ifft(vector(5)), :407 on deriv(unitvec(200))). `cx_integ/cx_deriv/cx_ifft` index both the data (by `length`) and its plot scale; a SYNTHETIC vector (vector(n)/unitvec(n)/an expression result) carries the current plot's scale, whose length need not equal n. Two directions: (1) data LONGER than the scale (integ/deriv of unitvec(200) on a 66-pt plot) -- the loops read `pl_scale->v_realdata[i]` for i<length, past the end of the scale; (2) data much SHORTER than the scale (ifft(vector(5)) on a 66-pt plot) -- the Green's inverse-FFT sizes datax from `N=pow2(length)` but the output loop writes `tpts`(=scale length) points from datax[2*i], so N<tpts overran datax. Only cx_fft had been guarded (Enhancement-225); the siblings had not. FIX (cmath4.c): cx_integ/cx_deriv reject a data vector LONGER than its scale (the direction that reads OOB) with a clean error -- a SHORTER data vector, as produced by fft (freq-domain, where group-delay deriv(vp(3)) is legitimate), stays valid; cx_ifft grows its transform size N (and M) to cover the output length `tpts` before allocating datax, a no-op for a well-formed transform where pow2(length)>=tpts already (the fft->ifft roundtrip is unchanged). Verify (examples/scaleguard, 5 checks): integ(unitvec(200)), deriv(unitvec(200)), ifft(vector(5)) resolve cleanly (no overflow); the fft->ifft roundtrip still returns a full-length vector; valid integ(vx)/deriv(vx) still work; a transform stress over mixed lengths/types is clean under ASan. One source file; completes for integ/deriv/ifft the length/scale guarding E-225 gave fft.

E-279 *src/frontend/com_let.c, src/frontend/options.c, src/frontend/com_measure2.c, src/maths/cmaths/cmath4.c, examples/castguard_examples/* (binaries rebuilt by CI)*

The remaining unguarded **`(int)`** casts and the last unguarded scale-dependent transform (systematic audit, not fuzz). After the fuzz rounds a systematic AUDIT -- grep every `(int) floor/fabs(...)` of a runtime double, plus every cmaths routine taking a `struct plot *` and whether it guards its length against the scale -- found the tail of two already-fixed classes at sites the fuzzer never reached. (1) Unguarded `(int)floor(x+0.5)` of a user-supplied double (UB outside int range, the E-273/-274/-276 class): `com_let.c` (an index expression in an indexed `let`), `options.c` (`set numdgt` / `rawfileprec` / `measureprec` -- `set numdgt=1e30` tripped UBSan at options.c:359, `set rawfileprec=1e30` at :340), `com_measure2.c` (`meas ... rise/fall/cross=<n>` at :1580/:1584/:1588). (2) An unguarded scale-dependent transform (the E-278 class): `cx_mtimeavg` walks the scale data `dsc[j]` for j < length-1, so a vector longer than its plot scale -- `mtimeavg(unitvec(200))` on a shorter plot -- read past the scale (ASan heap-buffer-overflow @ cmath4.c:1188). FIX: a small clamping helper per file (`let_idx_int`, `opt_int`,

meas_int) converts to int only after clamping to int range (NaN -> 0), so an out-of-range option/count saturates instead of invoking UB; values already in range are unchanged. cx_mtimeavg gets the same length-vs-scale guard cx_integ/cx_deriv received in E-278. Verify (examples/castguard, 7 checks): set numdgt=1e30, set rawfileprec=1e30, set numdgt=-1e30, meas .. . rise=1e308, mtimeavg(unitvec(200)) all clean where they previously tripped UBSan/ASan; a valid set numdgt=6 and mtimeavg(vx) still work. Four source files, guard-only.

E-280 *src/frontend/com_let.c, examples/letidxoob_examples/* (binaries rebuilt by CI)*

An out-of-range **let v[i] = x** wrote past the end of the vector (heap-buffer-overflow WRITE). Hardening the index cast (E-279) exposed a far more serious defect underneath. get_index_values (com_let.c) parses either one index (v[i]) or a range (v[lo:hi]). It validates low > high and high >= n_elem_this_dim -- but those checks lived INSIDE THE RANGE BRANCH ONLY. The single-index path set high = low and returned completely unchecked, so let vx[100] = 1 on a 66-element vector walked straight into the byte-offset arithmetic and performed a heap-buffer-overflow WRITE (ASan: WRITE of size 8) -- memory corruption from an ordinary typo, no exotic input required; a very large index additionally overflowed the index * n_byte_elem product in int (com_let.c:771). The range form vx[0:999] was correctly rejected all along. FIX: move both checks out of the range branch so they validate a single index too; an out-of-range assignment now reports index/high range (N) exceeds the maximum value (M) and changes nothing. Reads are unaffected -- op_ind clamps an out-of-range READ index with a warning (E-274) -- and every valid assignment, including the last element and the range form, behaves exactly as before. Verify (examples/letidxoob, 6 checks): let w[10]=99 on a 10-element vector, let w[999]=99, let w[1e308]=99 each rejected cleanly where the first two corrupted the heap; the last valid index (let w[9]=42) and a mid-vector assignment still assign; the range form still rejected. One source file, moving two existing checks.

E-281 *src/maths/cmaths/cmath4.c, examples/derivgroup_examples/* (binaries rebuilt by CI)*

deriv() over-read on a partial block (grouping != length) (ASan fuzz find). cx_deriv walks its input in blocks: for (base = 0; base < length; base += grouping) for (i = degree; i < grouping; i += 1) ... fitting a window around i + base. grouping is the vector's v_dims[0]; for an ordinary vector that equals v_length, so there is a single block and the window always fits. But a vector whose declared dimension differs from its length -- as produced by a binary op on operands of UNEQUAL length, e.g. min(v(b), ac.v(b)) where a 66-point real and a 5-point complex vector yield length 66 with dims[0] = 5 -- leaves a PARTIAL LAST BLOCK: base climbs to length - 1 while the window still spans base + grouping - 1, reading past the end of the input (ASan heap-buffer-overflow READ @ cmath4.c:326, the same line E-277 had already realigned). FIX (cmath4.c): bound the inner loop with i + base < length in BOTH the real and complex branches, so the fit window can never reach past the input; a no-op whenever grouping == length (every ordinary vector), and a partial trailing block too short to hold a full window is simply skipped. Verify (examples/derivgroup, 4 checks): deriv(min(v(b), ac.v(b))) and deriv(max(v(b), ac.v(b))) clean where they previously over-read; an ordinary real deriv(2t) still returns 2; the complex derivative from E-277 is still correct (deriv(t + 2t*i) = 1 + 2i). One source file, a loop bound in each branch.

E-282 *src/frontend/plotting/agraf.c, examples/plotlabel_examples/* (binaries rebuilt by CI)*

asciiplot read past its axis-label buffer on a 3-digit exponent (audit lead). Following up the plotting (int)floor(...) casts noted during the E-279 audit turned up a different, real defect one layer over. ft_agraf allocates its two axis-label lines as maxy + margin + FUDGE + 1 bytes (FUDGE=7) and NUL-terminates them at the last byte, budgeting the exponent width with sprintf(buf, "%1.1e", 0.0); shift = strlen(buf) - 7;. Formatting 0.0 ALWAYS yields a 2-digit exponent, so shift is 0 and the whole layout silently assumes two digits -- but real data can need three: denormal or very large values produce labels like 1.00e-320 (9 chars vs 8). The last label's memcpy(&line2[i + margin - ((j<0)?2:1) - shift], buf, strlen(buf))

then runs one byte too far and overwrites line2's terminating '\0', so the following `out_printf( "%s\n%s\n", line2, line1)` walks off the end of the heap allocation -- ASan reports `heap-buffer-overflow READ of size 82` on an 81-byte region inside vsnprintf. On the shipped build it does not necessarily fault; whether it prints heap garbage or crashes depends on what follows the buffer. FIX (agraf.c): remember the allocation bound in `line_end` (`maxy` is REASSIGNED further down as `maxy = spacing*nsp`, so the bound cannot be recomputed at the label loop), clamp the label memcpy so it can never reach the terminator slot (handling a negative offset too), and re-assert `line2[line_end] = '\0'` after the loop. Verify (examples/plotlabel, 5 checks): asciiplot of denormal 1e-320, 1e300, 1e-300 data and of both extremes together are clean where they previously over-read; an ordinary plot still renders its legend, axis rule and points; rendering confirmed BYTE-IDENTICAL to the pre-fix binary. One source file; no change to any rendered plot.

E-283 *src/frontend/plotting/agraf.c, src/frontend/points.c, src/frontend/display.c, examples/plotcoord_examples/* (binaries rebuilt by CI)*

Plot coordinate math no longer casts a non-finite double to **`int`** (extreme-data fuzz). The same output/formatting fuzz that produced E-282 found three more sites where a plot coordinate goes through `mylog10()` -- which returns +/-inf for zero, denormal or overflowed values -- or through a division by a range that can be zero, and is then cast to `int` (undefined behaviour). All are reached by ORDINARY plot commands on pathological data. (1) `agraf.c`: the decade `mag = (int)floor(mylog10(...))` -- data that overflows (`-1e308 * 6` -> -inf, so `-ylims[0]` is +inf) or is zero/denormal makes the argument non-finite; the limits `lmt`/`hmt` = `(int)floor(ylims[0] / tenpowmag)` are equally exposed since `tenpowmag = pow(10, mag)` can itself be 0 or inf. (2) `points.c` `ft_findpoint()`: `(int)(((mylog10(pt) - tl) / (th - tl)) * (maxp - minp) + minp)` is 0/0 for a degenerate range (lims[0]==lims[1], or log endpoints sharing a decade). (3) `display.c`: the four screen-coordinate casts (log and linear, x and y). FIX: per-site clamping chosen so the result stays MEANINGFUL, not merely defined -- `agraf_decade()` bounds the exponent by `DBL_MAX_10_EXP` (the widest decade a double can represent) and maps NaN to 0, `agraf_int()` clamps the lmt/hmt ratios into int range; `ft_findpoint` computes the fraction in double, maps NaN (degenerate range) to 0 and clamps to [0,1] so the result always lands in [minp, maxp]; `disp_clamp_coord()` clamps each screen coordinate into the viewport. Verify (examples/plotcoord, 5 checks): asciiplot of overflowing (-1e308), denormal and all-zero data, and a min/max over denormal data, are clean where they previously invoked UB; an ordinary `asciiplot v(b)` still renders. Rendering additionally diffed against the pre-fix binary and confirmed BYTE-IDENTICAL for ordinary data. Three source files, guard-only.

E-284 *src/frontend/spiceif.c, src/frontend/device.c, src/frontend/com_sweep.c, src/include/ngspice/fteext.h, examples/wildparam_examples/* (binaries rebuilt by CI)*

Wildcard parameter diagnostics, and the **`sweep`** knob label (reported from real use). A Verilog-A parameter `L_um` could not be swept as `sweep @*[[L_um]]`, and ngspice's message named `'l_um'` -- which reads like the mixed-case name had been mangled by ngspice's case-insensitive netlist tokens. The case theory is a RED HERRING: the OSDI parameter lookup IS case-insensitive (`@*[L_UM]` resolves `L_um` correctly, verified). The real cause is a LEVEL mismatch that the diagnostics hid: `@*[[param]]` (and `@#*[param]`) is the INSTANCE wildcard, while a plain `parameter real L_um` in Verilog-A is a MODEL parameter -- an instance parameter needs `(* type = "instance" *)`. So the instance wildcard correctly matched nothing and reported `no loaded instance has parameter 'l_um'`: lower-cased, with no hint that the parameter exists at the MODEL level, and nothing pointing at the fix. Separately, `sweep` labelled every knob it applies in place as `(model param)` -- including the instance wildcards -- because `sw_kind()` returns SW_MODEL as a DISPATCH flag ("apply via altermod, no re-source") and the banner reused that flag as a description. FIX: (a) `spiceif.c` gains a probe `if_hasparam_wildcard(ckt, param, do_model)` reporting whether any loaded model (or instance) carries a settable parameter of that name -- it only probes, nothing is changed; (b) `device.c` uses it so a wildcard that matched nothing checks the OTHER

level and names the form that would work, in both directions (`... but a loaded model does -- use the model wildcard '@*[l_um]' (an instance parameter needs (* type = "instance" *) in the Verilog-A)`, and the mirror suggesting `@#*[...]`), while a parameter that genuinely does not exist still gets the plain message with no misleading suggestion; (c) `com_sweep.c` gains `sw_knobdesc()`, which classifies the knob from the NAME TOKEN, so the banner reads `(instance param, wildcard)` for `@#*[...]`/`@*[[...]]` and `(model param, wildcard)` for `@*[...]`. Verify (examples/wildparam, 8 checks) on a model with a mixed-case MODEL parameter `Wavelength` and a mixed-case INSTANCE parameter `L_um`: both cross-level hints appear and name the right form; a truly absent parameter gets no bogus hint; sweep labels instance and model wildcards correctly; the matching wildcard still sweeps to the right values; and ALL-CAPS `@*[[L_UM]]` still resolves the mixed-case `L_um`, confirming the lookup was never case-sensitive. Four files; diagnostics and one banner string, no change to which parameters any wildcard actually sets.

E-285 *src/frontend/plotting/plotit.c, src/frontend/plotting/agraf.c, src/frontend/plotting/gnuplot.c, src/frontend/com_measure2.c, examples/veclenmix_examples/* (binaries rebuilt by CI)*

Output paths indexed one vector by another's length (extreme-data fuzz). The remaining findings from the extreme-data output fuzz all proved to be ONE class in four places: a vector's OWN length need not equal its plot SCALE's length -- any synthetic vector carries the current plot's scale, so `let y = vector(8)` on a 66-point transient plot has 8 points and a 66-point scale -- and a COMPLEX vector holds its data in `v_compdata`, leaving `v_realdata` NULL. (1) `plotit.c`: the transient resampling passed `v->v_realdata` together with `v->v_scale->v_length` to `ft_interpolate()`, which indexes the DATA by that length, reading ~58 elements past a shorter vector; for a complex vector it passed NULL outright -- a HARD SEGV on the shipped build (`asciiplot sqrt(-1*vector(10))` returns rc=139). (2) `agraf.c`: the bracketing indices `lower`/`upper` are computed against the X scale and bounded by `xscale->v_length`, but were then used to index each plotted VECTOR. (3) `gnuplot.c` (wrdata): the "no more data" guard tested only `i >= scale->v_length` while the branch it protects indexes `v->v_realdata[i]`. (4) `com_measure2.c`: the plain tran/dc branch read `d->v_realdata[i] / dScale->v_realdata[i]` with no NULL check, though the ac/sp branches beside it already had one -- measuring a complex vector dereferenced NULL (present twice, in `measure_at()` and `measure_deriv_at()`). FIX: clamp each index to the vector it actually addresses -- `plotit.c` passes the length the data and scale SHARE and skips the transient resampling entirely when any vector is not real (`all_vecs_real()`), since that block replaces the shared scale and must convert all vectors or none (complex vectors keep plotting through the normal path); `agraf.c` clamps `lower`/`upper` into [0, v->v_length-1] per vector; `gnuplot.c`'s guard also requires `i < v->v_length`; `com_measure2.c` takes the real part when `v_realdata` is NULL, exactly as its ac/sp branches do. Verify (examples/veclenmix, 7 checks): a vector shorter than its scale, a vector longer than its scale, a complex vector (previously rc=139), wrdata of a short vector, and a measure over a complex vector are all clean and still render/measure; an ordinary `asciiplot v(b)` and `meas tran ... max v(b)` are unaffected. Ordinary plot output AND wrdata output diffed against the pre-fix binary and confirmed BYTE-IDENTICAL; all eight decks from the output fuzz are clean. Four source files, guard/clamp only.

(E-25's simparam exposure lives in the OSDI callback table populated at load time; its diff rides inside the `osdiload.c`/callbacks changes.)

Build-warning cleanup (post-E-127): a full rebuild under the repo's `-Wconversion` flags surfaced latent warnings. `configure.ac` had `-s` (a strip flag) in the compile `CFLAGS`, so clang warned "argument unused during compilation: '-s'" on every source file (~1526 warnings) — removed (binaries are stripped separately by the fold and CI). `-Wconversion` also flagged a real latent bug in `maths/ni/nipzmeth.c` (`b = 0.0` was an assignment where `b == 0.0` was intended) — fixed — and benign `double`→`bool` conversions in `maths/cmaths/cmath4.c` — made explicit. Full-build warnings dropped 1903 → 394; the remainder are `-Wconversion` warnings in third-party SuiteSparse (KLU) and CMC device models (HiSIM), which are left to their upstream sources.

Every entry above is guarded by a verify script under `examples/` and was regression-checked against the full example arsenal, the compiler's integration suite, and the VA_TEST industry-model corpus at the time it landed.

E-296 *src/frontend/plotting/pyplot.c*

pyplot appearance controls. Seven additive `set` variables: `pyplot_grid` (on/off/x/y, overriding the grid-follows-axis default), `pyplot_legend` (off, or a matplotlib location -- underscore form since `set` keeps only the first word), `pyplot_markers` (a cycling marker `o s ^ D v * P X` ON the line, distinct from `pointstyle=markers` which drops the line), `pyplot_axhline`/`pyplot_axvline` (SI-aware reference lines via `ft_numparse`), `pyplot_dpi`, `pyplot_transparent`. With none set the emitted script is byte-identical to before. Verify (examples/pyplotmore): each control appears in the script AND the script executes; the default path is unchanged. One file.

E-297 *src/frontend/plotting/pyplot.c, plotit.c, com_pyplot.c*

pyplot **`-fft`** magnitude spectrum. `ft_pyplot`'s `bool hist -> int mode` (LINE/HIST/FFT). `-fft <sig>` reuses the time-domain (t,y) table; the emitted Python RESAMPLES onto a uniform grid (`np.interp`) before `np.fft.rfft`, because transient data is adaptively sampled and a raw FFT over non-uniform samples is wrong. One-sided, scaled by `2/sum(window)` so a pure tone reads back its amplitude. Options `pyplot_fft_window` (hann/hamming/blackman/rect), `pyplot_fft_db`, `pyplot_fft_points`, `pyplot_fft_logf` (log-f in Python, dropping the DC bin -- NOT the command's `xlog`, which would validate the t=0 TIME scale and abort). Verify (examples/pyplotmore): a 2.0@1kHz + 0.5@3kHz tone reads 2.0 and 0.5 (amplitude oracle, rel<2%). Three files.

E-298 *src/frontend/plotting/pyplot.c, plotit.c, com_pyplot.c*

pyplot complex-aware AC views **`-bode`**/**`-nyquist`**/**`-polar`**. An ordinary `pyplot v(out)` on AC data silently keeps only the REAL part (0.5 not 0.7071 at -3 dB). New `ft_pyplot_ac()` emits a `<vec-index> <freq> <re> <im>` table -- the imaginary part is CARRIED -- grouped by the first column so multiple/ragged vectors stay separate; `-bode` = 20log10

E-299 *src/frontend/plotting/pyplot.c*

pyplot overlay robustness + hover cursor + export note. Cross-run overlay via ngspice's `plot name.vector` refs (`pyplot tran1.v(out) tran2.v(out)`) already worked, but the data table was sized by the FIRST vector's scale, so a longer second run was truncated; it now walks the LONGEST scale across the plotted vectors (each still uses its own scale for x, shorter ones pad with NaN which matplotlib skips; a single run is unchanged since all share one scale). `set pyplot_cursor` adds a hover crosshair via matplotlib's built-in `Cursor` widget (no extra package), interactive-only (not emitted for a hardcopy). Pan/zoom/save (matplotlib toolbar) and the `<name>.data` export were already provided. Verify (examples/pyplotmore): a finer second run keeps every sample; cursor present in a window script, absent from a hardcopy. One file.

E-300 *src/frontend/plotting/pyplot.c*

pyplot **`pyplot_mplcursors`** cursor backend. On top of E-299's `pyplot_cursor` (built-in `matplotlib.widgets.Cursor` crosshair), `set pyplot_mplcursors` selects the `mplcursors` package -- data cursors that snap to a trace and show its (x,y) on hover -- AND turns the cursor on by itself. Unset, the built-in crosshair remains the default. The emitted script imports mplcursors inside a `try` and FALLS BACK to the built-in Cursor on ImportError, so a deck written where mplcursors exists still runs where it does not. Window-only (not emitted for a hardcopy), like `pyplot_cursor`; mplcursors must be importable by the interpreter named by `pyplot_python`. Verify (examples/pyplotmore): the mplcursors branch + fallback are emitted on `pyplot_mplcursors` alone, `pyplot_cursor` alone still emits only the built-in Cursor, neither appears in a hardcopy; the mplcursors branch was executed under an interpreter that has the package (calls `mplcursors.cursor(hover=True)`) and the fallback under one that does not. One file.

E-301 *src/frontend/plotting/pyplot.c*

pyplot **pyplot_cursor** is the single cursor master switch. E-299 (built-in Cursor) and E-300 (mplcursors) had drifted: pyplot_mplcursors turned the cursor ON by itself, so there was no one "off" knob. The enable condition went from pyplot_cursor OR pyplot_mplcursors to pyplot_ cursor alone -- off by default, the ONLY thing that enables any cursor; pyplot_mplcursors now only SELECTS the backend when the cursor is on and does nothing on its own. Predictable, and matching every other optional pyplot behaviour. Verify (examples/pyplotmore): the full gating truth table -- default off, cursor-alone -> built-in, mplcursors-ALONE -> off, both -> mplcursors (with fallback), off in a hardcopy. One condition, one file.

E-302 *src/frontend/com_measure2.c*

.meas avg did not clip its window to [from,to]. Found by a closed-form oracle (a same-binary comparison is structurally blind to a uniformly present error). measure_minMaxAvg() trapezoid-summed only the samples INSIDE the window and divided by their span, without interpolating either boundary -- so the first and last partial intervals were dropped. rms/integ (measure_rms_ integral) already clip exactly, so ngspice's own two answers for one window disagreed: avg=1. 27114 vs integ/(to-from)=1.27324, closed form 1.2732395; and the echoed to= was the first sample OUTSIDE the window (2.50280e-04 for a requested 2.5e-4). O(dt/window) error: 1.6e-3 -> 4e-7 here. Fix interpolates both ends and reports sprev as the window end, all guarded to AT_AVG so min/max/min_at/max_at keep whole-sample semantics. The dc branch is deliberately EXCLUDED -- a dc sweep may descend and enters its window at the high end, where a clip extrapolates (an early version of the fix turned a descending-sweep avg of 0.270250 into -3.79e+11); every dc measurement is byte-identical. Known remaining: .meas dc avg still truncates (0.270250 vs 0.2708333), and on a descending sweep integ returns 0.0 with from= nan. Verify (examples/ measwindow): 28 checks vs closed-form integrals, windows landing BETWEEN samples; fails 14/28 on the pre-fix binary.

E-303 *src/frontend/com_measure2.c*

.meas dc avg window truncation, both sweep directions. Completes E-302, which fixed tran/ ac/sp but excluded dc because a sweep may DESCEND (dc v1 2 0 -0.001) and enters its window at the HIGH end, where an unguarded clip extrapolates outside the bracketing samples. .meas dc avg averaged over [first sweep point inside, last inside] instead of [from,to]: 0.270250 against a closed-form mean-of-x^2 of 0.2708333, echoing to=7.49000e-01 for a requested 0.75, while .meas dc integ over the same window was already correct. Fixed direction-agnostically in measure_minMaxAvg(): the clip works from the ACTUAL crossing between the previous raw sample and the current one -- open the window at the boundary crossed on the way in, close it on the way out -- so it fires at from->to ascending, to->from descending, and never on a sweep that does not cross. Guarded to AT_AVG, so min/max/min_at/max_at keep whole-sample semantics. The echoed window reports the upper end of the range actually covered, so a descending sweep no longer prints the nonsensical from= 0.25 to= 0.25. STILL OPEN (different function, measure_ rms_integral): on a descending sweep integ/rms return 0.0 with from= nan -- the loop meets the first sample already above to and interpolates with index i-1 == -1, an OUT-OF-BOUNDS read, then breaks with an empty array. Verify (examples/measwindow): 38 checks, the same window run in BOTH sweep directions vs closed form; 32/38 on the pre-303 binary.

E-304 *src/frontend/com_measure2.c*

.meas dc integ/rms heap-buffer-overflow on a descending sweep. Completes E-303. On dc v1 2 0 -0.001 these returned 0.0 with from= nan (rms printed nothing). measure_rms_integral() builds its clipped window assuming the scale ASCENDS -- it clips at from going in and to going out, and the widths x[i+1]-x[i] must be positive for the Simpson/trapezoid sums. A descending sweep starts at the top, so the FIRST sample is already above to: the end-clip fired at i==0 and interpolated against index i-1 == -1, reading v_realdata[-1]. AddressSanitizer: heap-buffer-

`overflow ... measure_interpolate com_measure2.c:195 / measure_rms_integral com_measure2.c:1415`. It then broke with a one-element array, so `while (i < xy_size-1)` ran ZERO times (sums 0) and `first` was never set (m_measured_at unwritten = the nan). Fix: walk the samples in order of INCREASING SCALE rather than index order -- the identity for ascending data, so tran/ac/ascending-dc are byte-identical -- and make the has-a-previous-sample guard the traversal position `n > 0` instead of `i > 0`, correct in both directions. integ 0.0 -> 0.135416 (closed form 0.135416667); rms -> 0.307458 (0.307459347). Also removed two dead reads in the `toVal < 0.0` tail that indexed d->v_realdata[] with the INTEGRATION loop counter (past the end of the data) into variables nothing reads again, and took the window end from xScale[v_length-1] -- the smallest scale value when the sweep descends, not the last traversed. Verify (examples/measwindow): 44 checks, both sweep directions for avg/integ/rms vs closed form; 40/44 on the pre-304 binary; and the OOB is PROVEN gone -- ASan reports it before the fix and is clean after.

E-305 *src/maths/misc/randnumb.c, src/include/ngspice/randnumb.h, src/frontend/com_sweep.c, src/frontend/commands.c*

`wcd` -- worst-case distance / MPFP high-sigma. The named remainder of the statistical suite, and the complement E-150's own write-up says it does not implement. `highsigma` uses direction-free scaled-sigma IS; `wcd` works in STANDARDISED NORMAL space and asks a geometric question -- which point of the failure region is most probable? With the margin as g(u)>0 for pass, that is the boundary point closest to the origin; its distance beta is the worst-case distance and, the density being spherical, P_fail = Phi(-beta) -- the sigma number a designer quotes. Hasofer-Lind/Rackwitz-Fiessler iteration with forward-difference gradients: cost BOUNDED at a few iterations of (1+ndim) sims, versus the 1e6-1e9 samples plain MC needs merely to SEE a 4.5-6 sigma event. `-is N` refines FORM (exact for a linear margin, approximate when it curves) with mean-shift IS centred on the MPFP, carrying the exact ratio phi(z)/phi(z-u*): 986 failures seen in 2000 shifted samples for a 3.4e-6 event. Implemented as two modes behind the EXISTING single Gaussian-draw funnel mc_sample_gauss(), so it composes with LHS/SSS rather than duplicating them: MC_MODE_WCD makes draws deterministic (dimension d returns a chosen u[d], so the deck is a plain function g(u)) and MC_MODE_SHIFT does the shifted sampling. Dimensionality is DISCOVERED (a deck's Gaussian-draw count is unknown until it has been evaluated once) via a running maximum cleared per evaluation -- a single margin evaluation triggers several deck-copy passes and a later pass that draws nothing would otherwise wipe the count. Verify (examples/wcd): 19 checks vs the ANALYTIC GAUSSIAN TAIL, never a previous build -- FORM is exact for a linear margin so beta is exact to machine precision at 3/4/5/6 sigma and P matches Phi(-beta) to ~1e-8; a 2-D case puts the MPFP on the symmetric point (4/sqrt2, 4/sqrt2), which is what shows the search finds the right DIRECTION and not a 1-D degenerate answer; a nominal-already-failing case gives signed beta=-3. The four existing statistical suites pass unchanged.

E-306 *src/maths/cmaths/cmath4.c*

The E-241 twin: **`fft()`** expression function normalised by the padded N. E-241 fixed an amplitude normalisation that divided by the ZERO-PADDED transform size instead of the input length, in the `fft` COMMAND (frontend/com_fft.c). The identical mistake survived in maths/cmaths/cmath4.c `cx_fft` -- the vector-expression function reached by `let F = fft(v)`, a separate implementation of the same computation. Found by continuing the oracle campaign that produced E-241, and located with E-241's own discriminator: bin 0 is unambiguous (no window/scalloping) and X[0] = D*length for a DC offset D, so dividing by N reads back D*length/N. On one signal (4001 samples padded to 4096, DC 2.0) the COMMAND read 2.000000 and the EXPRESSION read 1.953613 = 2.0*4001/4096. NOT a convention question: `cx_fft` holds TWO complete implementations (complex-input and real-input) and in BOTH the FFTW branch already used the input length (`(((double)length)/2.0, fpts`) while Green's radix-2 used the padded size (`(((double)N)/2, (double) N`) -- the correct version sitting a few lines from the wrong one, an internal contradiction like `avg` vs `integ` in E-302. HAVE_LIBFFTW3 is undefined in this build so Green's is live. Fixed both Green branches to the input length. INDEPENDENT confirmation: nothing here touches

ifft, yet `ifft(fft(x))` went 2.3e-02 -> 1.1e-16 (the pair only inverts when the forward normalisation is right), so cx_ifft was audited and deliberately left alone. Full audit of every Green-kernel caller: com_fft.c fft x2 and spec/PSD x2 correct (E-241); cx_ifft correct; trannoise/1-f-code.c cannot pad (n_pts grown to 2^n_exp by construction); fft/ifft are the only transform functions in the expression table so there is no spec twin. Verify (examples/fftexpr): 6 checks vs closed form (DC bin, round trip padded and unpadded, complex-input bin 0 =

E-311 *src/frontend/measure.c*

`param`/`expr` measurements now work in a **`.control`** block. Found oracle-checking the less-common `.meas` modes. The param/expr measure types are handled only by do_measure()'s second pass (via nupa_eval); the interactive/`.control` `meas` command (com_meas) skips do_measure and calls get_measure2() directly, which has no param/expr case -- so `.meas tran pp param='a1-a2'` (dot-card) worked but `meas tran pp param='a1-a2'` (.control) failed with 'no such function as ...', down to a bare `param=a1`. Fix: in com_meas the prior results (a1,a2) are already single-valued ngspice VECTORS, so a param/expr measurement is `let <name> = (<expr>)` via the ordinary vector evaluator -- detected BEFORE com_meas's single-valued-vector substitution loop (which would mangle the expression), reassembled from the tokens after `param=/expr=`, delimiter single quotes stripped, and re-lexed with cp_lexer so a spaced expression tokenises as the shell would. Works: `param=a1-a2` (unquoted), `param='a1-a2'` (quoted no-space), `param={sqrt(a1*a1 + a2*a2)}` (braces, spaces OK), `expr=`. Normal meas types (max/min/avg/when/...) route through get_measure2 unchanged (byte-identical vs shipped). KNOWN LIMITATION left in place: `param='sqrt(a1*a1 + a2*a2)'` (single quotes + internal spaces) is mangled by the .control shell's own quote pre-expansion UPSTREAM of meas -- use braces; the dot-card handles it (no shell tokenizer). Verify (examples/measparam): 8 checks vs closed-form triangle values; 7/8 fail on the pre-fix binary.

E-312 *src/xspice/cm/cm.c, src/xspice/icm/analog/s_xfer/cfunc.mod*

XSPICE integrating code models are second-order accurate. Found oracle-checking the `s_xfer` Laplace transfer function against a closed-form transient: its error fell only as O(h), not the O(h^2) every native ngspice storage element delivers. Two independent causes. (a) SHARED across all integrating code models (s_xfer, int, d_dt, ...): cm_static_integrate()'s TRAPEZOIDAL order-2 arm was a self-flagged stand-in -- `cur = -0.5*ag[0]*intgr[1]` with a compensating `geq *= 0.5`, carrying its own `/* WARNING - This code needs to be redone */` -- which works out algebraically to backward Euler (y(n)=y(n-1)+h*u(n)), a first-order method. Fixed to the true rule y(n)=y(n-1)+(h/2)(u(n)+u(n-1)); the "one previous value" the WARNING said was missing is u(n-1), stored in the SPARE state double cm_analog_alloc already reserves per integrator (doubles_needed = bytes/sizeof(double)+1), rotated through the CKTstates[] history for free -- the standard SPICE companion-model idiom of keeping a state and its rate in adjacent slots. geq is now ag[0] like the order-1 and Gear arms; a guarded legacy fallback preserves old numerics only where no spare exists (no real code model). (b) SPECIFIC to s_xfer: its controller-canonical feedback read the fed-back integrator states from the PREVIOUS timestep (cm_analog_get_ptr(i,1)), making the loop explicit/lagged one step, which caps H(s) at O(h) on its own even after (a). Now reads current-iteration states (offset 0) so the loop is solved implicitly within each Newton step (the DC/init pass already used offset 0). Both TRAP and Gear now converge O(h^2). Convergence verified vs an exact 1/(1+tau s) sine oracle (error ratio ~2 per h-halving before, ~4 after) and, for stability of the now-implicit feedback, vs SciPy signal.lsim across five transfer functions -- 1st-order, a zeta=0.05 near-oscillatory resonator, a 3rd-order Butterworth, a stiff pair with poles three decades apart, and one with a numerator zero -- all converge and match to <2.2e-4. Verify (examples/sxferorder): 6 checks under both solvers, including a self-contained convergence-ORDER test (ratios ~4, fitted p~2.1) that fails on the pre-fix binary with no reference build required. 246-example regression green under both solvers.

E-315 *src/spicelib/analysis/tfanal.c, src/spicelib/analysis/cktic.c, src/spicelib/analysis/distoan.c*

Three analysis crashes on adversarial-but-valid input -> clean errors (command/netlist fuzz, E-222..228 / E-270..285 family). [6] `.tf` on a SINGULAR circuit -- a dangling inductor (`l1 2 3 1`, nodes 2,3 floating) makes the operating point singular; tfanal.c IGNORED CKTop's return, so the matrix was never factored and the following SMPsolve() asserted `IS_VALID(Matrix) && IS_FACTORED(Matrix)` (spsolve.c:137, SIGABRT). Fix: propagate the CKTop error before solving. [7] a second `.pz` over a URC device -- CKTic zeroes the RHS vectors with a loop the compiler vectorises into memset(CKTrhs,...), but here ckt->CKTrhs is NULL, so it wrote through a NULL pointer (SIGSEGV in _platform_memset). Fix: CKTic returns cleanly when the RHS vectors/matrix are unallocated. [8] `.disto` with NO distortion sources (a plain resistor) -- distoan.c's output section left OUTpBeginPlot's result unchecked, so on failure acPlot stayed NULL and OUTattributes(acPlot,...) dereferenced it (SIGSEGV at OUTattributes+268, addr 0x28); the first plot checked its result but the five output-section plots did not. Fix: capture and check OUTpBeginPlot at each output plot, bail on NULL acPlot. Legitimate .tf/.pz/.disto unaffected -- guards fire only on the failure paths (valid .tf still gives 0.5 on a 1k:1k divider). Verify (examples/ngcrashanalysis): 4 checks.

E-316 *src/frontend/com_measure2.c*

`.meas AVG` ended one timestep short of **`to`**. Found continuing to oracle-check .meas against its own contract (E-302/303/304 family): AVG is the same quantity as INTEG/(to-from) over the identical window. measure_minMaxAvg()'s final window-clip (added by E-302, which made AVG integrate a trapezoid over [from,to]) guarded the WHOLE accumulation with `!AlmostEqualUlps(svalue, meas->m_to, 100)`, so when the first out-of-window sample fell within 100 ULPs of `to` the entire final trapezoid [sprev,to] -- a full timestep -- was skipped. AVG's window then ended one timestep short of `to`: for a 2.5e-7 timestep the AVG window echoed to=6.45990e-04 while INTEG (correct) echoed to=6.46240e-04, exactly 2.5e-7 apart, so avg != integ/(to-from) by ~1.6%. measure_rms_integral() (INTEG/RMS) does NOT have this bug -- it always adds the final point, interpolating to `to` only when the sample overshoots, else using the ~=to sample as-is. Fix: move the !AlmostEqualUlps guard so it gates only the INTERPOLATION, not the accumulation, matching INTEG -- the final trapezoid is now always added, and AVG covers the full [from,to] window (AVG==INTEG/(to-from), rel 1.6e-6). MIN/MAX keep whole-sample semantics (branch is AT_AVG-only). Verify (examples/measavgwin): AVG==INTEG/(to-from) + AVG window reaches `to`, both fail on the pre-fix binary; the E-302/303/304 measwindow suite (44 checks) and E-311 measparam still pass.

E-318 *src/spicelib/devices/vsrc/vsrcload.c*

SFFM/AM voltage sources dropped the DC offset before the delay. Found in the correctness campaign, oracle-checking transient waveform sources. vsrcload.c evaluated SFFM and AM with `if (time <= 0) value = 0;` (after subtracting TD), returning 0 instead of the quiescent value at time<=0 -- so an SFFM/AM source's OPERATING POINT (common TD=0) was 0 not VO, and over any pre-delay window it read 0, injecting a spurious startup transient (an SFFM(2,0.001,...) ~= 2V source into an RC started at 0V and charged to 0.785V, vs a DC-2 source flat at 2V). Three oracles show VO is correct: the SIN case in the SAME function holds VO+VA*sin(phase) at time<=0; ngspice's OWN current-source SFFM (isrcload.c) has NO such zeroing and evaluates the formula directly (the two SFFM implementations in one binary disagreed); SIN/PULSE/EXP/PWL all preserve their offset. Fix: hold the waveform's time=0 value -- SFFM value = VO + VA*sin(phasec + MDI*sin(phasem)); AM value = VO + (VMO + VMA*sin(phasem))*sin(phasec); (both reduce to VO for zero phases). No example/regression circuit used SFFM/AM, so nothing else changes; the current-source path was already correct and is untouched. Verify (examples/sffmoffset): SFFM->1.5, AM->2.0 over a delayed window (SIN control->1.5), all failing on the pre-fix binary.

E-319 *src/frontend/com_qpss.c*

Transient-form QPSS leaked the fundamental into every mixing bin. Found in the correctness campaign via the RF steady-state oracle (a QPSS steady state must equal the HB-form and a long

transient + DFT). The transient-form `qpss <expr> <f1> <f2> [periods] [maxorder]` (as opposed to `... hb K1 K2`) runs a two-tone transient then computes each 2-D harmonic k1*f1*+*k2*f2 by a TRAPEZOIDAL integral of v(t)*exp(-j2pi f t) over the "last period" [tt[i0], tt[end]] of the RAW transient grid. That window is not exactly the beat period T=1/gcd(f1,f2) (tt[i0] >= wstart but not ==, short by up to one tstep), is non-uniform, and has non-periodic endpoints, so the large DC/ fundamental leaks ~tstep/T into EVERY mixing bin. On a LINEAR two-tone RC (every product with

E-320 *src/frontend/com_sweep.c, src/frontend/numparam/{spicenum.c,xpressn.c,numparam.h,numpaif. h}*

Sweeping a **`.param`** no longer forces a full circuit reset at every point. A `.param` has no live binding into the built circuit -- numparam folds each `{expr}` on a device line to a fixed numeric literal at parse time -- so the only way to change one is `alterparam` (rewrite deck text) + `reset` (reparse + subckt-expand + CKTsetup + matrix REORDER), per point, even though topology never changes. E-320 adds a fast path to the `sweep` command: when every swept `.param` feeds ONLY addressable top-level device/model VALUES, `com_sweep` per point overrides the swept symbol(s) in the retained numparam dico (nupa_add_param), refreshes the derived-param closure (new nupa_recompute_params replays the dependent .param lines), re-evaluates each captured device value against the dico (new nupa_eval_expr -> the numparam formula() evaluator), and pushes it into the live instance with a resolved direct set (ft_sim->setInstanceParm = CKTparam -> DEVparam), refreshing each touched device type once via DEVtemperature -- NO reset, so the matrix ordering/factorization pattern is preserved. The template is the `temper` machinery (a temperature-dependent device value re-evaluated + alter'd in place every .dc step); the difference is that a parse tree cannot re-read an arbitrary .param cell (only time/temper/hertz bind to live ckt storage), so re-eval goes through numparam not IFeval. Two optimizations: each `@dev[param]` target is resolved to (instance, param-id) ONCE at arm time (no per-point lex or device-list walk), and identical value expressions are grouped so each unique expression is evaluated once per point. Conservative classifier: arms only when a scan of the ORIGINAL (pre-expansion) deck finds every swept-param occurrence to be a top-level device/model value slot (positional principal, or key={expr} named); DISARMS to the exact reset path on any subckt-body use (expression not retained past flattening), structural use (param in a node position, an instance/model/subckt name, a subckt call arg, .if/.elseif, .temp, an analysis card, .option, .ic, .nodeset, .global), or a derived .param (a non-swept param defined from a swept one). Because a disarm reproduces today's behaviour exactly, a sweep can only get faster, never change result. Measured on a 2500-stage ladder, 120-point sweep: param feeding ONE device in a large circuit 2.91s->0.26s (11.1x), param feeding ALL 5000 devices 3.42s->1.33s (2.6x); fast matches reset to ~7e-9 (numparam value-string formatting only), subckt fallback bit-identical. Scope: the `sweep` command (instance values fast, model values via textual altermod fallback); subckt-internal params and the optimize/Monte-Carlo .param reset sites stay on the reset path. Verify (examples/paramfastsweep): top-level param arms + matches closed-form R2/(rval+R2); subckt param falls back yet identical.

E-321 *src/frontend/com_sweep.c, src/frontend/numparam/{spicenum.c,numpaif.h}*

`.param` fast-sweep, subcircuit tier -- subckt-INTERNAL device values now take the fast path. E-320 was limited to top-level values: a swept param feeding a device inside a subckt disarmed to reset, because numparam freezes each `{expr}` to a literal during flattening. The expression is recoverable: a flattened instance card (`r.x1.r1 in out 1e3`) carries `card->linenum` back to its subckt-DEFINITION body line (`R1 a b {rval}`), a top-level template whose original text numparam retains in `dicoS->dynrefptr[linenum]` (two instances of a subckt share the linenum). So capture now walks the FLATTENED deck (ci_deck) and, per device card, recovers the value expression via the new `nupa_get_dynref(card->linenum)` while the card's own first token gives the full hierarchical instance name -- top-level and subckt-internal captured by one mechanism, no name reconstruction. (Empirically the key is card->linenum, NOT card->linenum_orig, which uses a different counter -- inpcom.c line_number vs line_number_inc -- and is off by one.) SAFE across

subckt scope, since a subckt can shadow/derive-from/pass-through a swept global: (1) a Pass-1 scan of the original deck INCLUDING subckt bodies disarms on a swept param in a subckt header default, a subckt-call (X) argument, a subckt-body .param that shadows (swept LHS inside a subckt) or derives from (swept RHS) the param, or any structural slot (node/name, .if, .temp, analysis card, .option, .ic, .nodeset); (2) an arm-time SELF-CHECK re-evaluates each captured expression against the GLOBAL dico at nominal params and requires it to reproduce the value numparam already baked into the flattened card (read straight off the card, no ask-API), so any shadow/pass-through that slips past (1) disarms the whole path; (3) a disarm falls back to the exact reset path. Validated by a 10-deck battery -- 5 that must arm (top-level, direct subckt use, multi-instance, nested subckt, scaled expr) and 5 that must disarm (local .param shadow, formal-param shadow, formal pass-through, subckt-local derived, top-level derived) -- where EVERY deck's fast output equals reset bit-for-bit and every arm/disarm decision is as expected; the subckt fast path matches closed-form R2/(rval+R2) to 9e-10 and reset to 1.9e-9. Speedup follows fan-out: a param feeding one device inside a subckt in a large fixed circuit 2.92s->0.24s (12.0x); a param feeding every device (400 subckt instances x 8 R) 2.50s->1.64s (1.5x). Scope: the `sweep` command (subckt-internal instance values fast, model values via altermod fallback); optimize/Monte-Carlo .param reset sites stay on the reset path. Verify (examples/paramfastsweep): top-level arm + subckt-internal arm + local-shadow fallback, all vs closed form (4 checks).

E-322 *src/frontend/com_optimize.c, src/frontend/com_sweep.{c,h}*

`.param` fast-sweep, optimizer tier -- the **`optimize`** command shares the fast-path engine. Every `-dparam` (symbolic `.param`) objective evaluation used to do `alterparam` + `reset` (re-parse + rebuild the whole circuit), and an optimization runs tens to hundreds of evaluations. E-322 exports the E-320/321 capture-classify-selfcheck-push engine (sw_fp_build / sw_fp_apply / sw_fp_free, de-static'd + declared in com_sweep.h) and wires it into the optimizer: opt_fp_arm() collects the OPT_DECKPARAM knob names and calls sw_fp_build ONCE before the search; opt_eval (Nelder-Mead / Levenberg-Marquardt / PSO / DE / SA) and opt_eval_objs (NSGA-II) push the candidate's deck-param values in place with sw_fp_apply instead of alterparam+reset; sw_fp_free at cleanup. The conservative classifier and arm-time self-check are unchanged, so a deck-param feeding a subckt shadow / structural slot / derived param disarms and the optimizer runs exactly as before. Two opt-outs: (1) `-center` design-centering (E-206) keeps reset -- its inner Monte-Carlo RE-SAMPLES process variation on each reset, a re-draw not a deterministic value push; (2) a device-count guard keeps the reset path on SMALL circuits -- the in-place apply has a fixed per-eval cost (numparam re-eval + dico ops) roughly independent of circuit size while a reset's cost grows with the deck, crossing at ~80 device instances, and because the in-place values differ from reset in the last few digits (numparam value-string formatting) an extremely tight -tol on a tiny circuit could otherwise send the two paths to different iteration counts; so opt_fp_arm counts flattened device instances and engages only at >=80. Correctness: across NM/PSO/DE the fast-path optimum is IDENTICAL to reset (122-device ladder fit: rtop=361.914 under NM and DE, 382.872 under PSO), and NSGA-II arms and returns the same Pareto front; all 41 existing optimize verify checks still pass. Speedup: a 4000-device fixed ladder, single -dparam knob, Nelder-Mead to convergence 0.94s->0.17s (5.6x); grows with circuit size, guarded to the sub-30ms reset on tiny circuits. Scope: `optimize` -dparam knobs, all methods except -center (-param/-mparam were already in-place). Verify (examples/optimize): a large-circuit -dparam optimization arms the fast path and still converges (2 new checks).

E-323 *src/frontend/com_optimize.c, src/osdi/osdiinit.c, src/include/ngspice/osdiitf.h*

`.param` fast-sweep, OSDI-aware optimizer guard. E-322 gated the optimizer's fast path behind a device-count guard (arm at >=80 device instances) because on a small circuit a reset re-parses faster than the fast path's fixed per-eval overhead. That threshold was calibrated on RESISTORS and is wrong for OSDI (compiled Verilog-A) devices: a resistor reset just re-parses an element line, but an OSDI reset RE-INITIALIZES each compiled model instance (re-runs its setup/temperature callbacks), ~30x costlier per device. Measured (fast path on vs off, same binary, OSDI-resistor

ladder): the fast path wins at 3 OSDI instances (2.1x), scaling to 3.2x at 161 -- where resistors needed ~80 to break even. So the E-322 count guard would keep a SMALL OSDI -dparam optimization (common in device model extraction/fitting) on the slower reset path, forgoing 1.5-3x (a 300-instance OSDI fit measured 3.0x fast vs reset, same optimum). Fix: the crossover is about reset COST not device COUNT, so opt_fp_arm now walks the built circuit's instances and WEIGHTS each by device kind -- primitive=1, OSDI=OPT_FP_OSDI_WEIGHT=30 (measured ~3 OSDI ~= the reset cost of ~80 primitives) -- and compares the weighted total to the same 80 primitive-equivalent threshold. A handful of OSDI devices arms the fast path; a small all-resistor circuit still keeps the cheap reset. OSDI types are identified by a stable marker: every OSDI SPICEdev is built by osdi_create_spicedev and shares the OSDIparam instance-parameter setter (no built-in device uses it), exposed via a one-line osdi_devtype_is_osdi(type) helper in osdiinit.c; the optimizer call is #ifdef OSDI-guarded so a build without OSDI is unaffected (every instance weighs 1). Correctness is unchanged from E-322 -- the guard only decides WHETHER to arm, not what the fast path computes -- so a small OSDI -dparam optimization now arms and converges to the SAME optimum as the reset path. Verify (examples/optimize): a small OSDI -dparam optimization (compiled optres.va) arms the fast path and converges (2 new checks); all 43 optimize checks pass.

E-338 *ngspice-46/src/frontend/inpcom.c*

A full 64-bit bus range HUNG ngspice and grew without bound -- 3.6 GB after 4 s, 7.6 GB after 9 s, from ONE netlist line, in the released binary. `R1 a[-9223372036854775808: 9223372036854775807] r=2k`. Found by fuzzing the array/bus node feature. ROOT CAUSE -- THE GUARD COULD NOT SEE THE REAL WIDTH: E-221 expands `base[lo:hi]` into scalar node names behind a deliberate BUS_MAX_WIDTH (8192) ceiling, computed as `long width = (hi >= lo) ? (hi - lo + 1) : (lo - hi + 1)`. strtol saturates at LONG_MIN/LONG_MAX, so the full span makes `hi - lo + 1` OVERFLOW a signed long -- undefined behaviour, which in practice wrapped to a small value, so the guard saw a tiny width and let it through. The loop then stepped from LONG_MIN toward LONG_MAX, about 1.8e19 iterations, appending to a string each time. The guard was not missing: it was computing its input with the very overflow it existed to prevent. ONLY THE FULL SPAN TRIGGERS IT -- the one-sided cases a[-9223372036854775808:0] and a[0:9223372036854775807] have width ~9.2e18, which fits and is correctly rejected. That is exactly why a corpus using only 32-BIT extremes (a[-2147483648: 2147483647], width 4.29e9) missed it: that width is representable, so the guard worked. FIX: compute the span in UNSIGNED arithmetic (exact for every pair of longs) and compare the SPAN rather than the width, so the `+ 1` cannot overflow either; strtol setting ERANGE now also rejects an endpoint too large to represent rather than letting it be silently clamped to LONG_MIN/LONG_MAX, values that then look like an ordinary acceptable range. A netlist is untrusted input, so this is a resource-exhaustion defect reachable from a single line with no diagnostic and no bound. FUZZ: 432 decks, 0 crashes/hangs/runaway -- 54 range tokens (32- and 64-bit numeric extremes, malformed brackets, missing parts, nesting, control characters, XSPICE %vd[. ..] groups, a 4000-character base name, 40-deep bracket nesting) crossed with 8 placements (device nodes, subckt call, .subckt port list, value position, model name, print v(), bare print, let). CORRECTNESS: 13/13 numeric checks -- ascending AND DESCENDING ranges map ports in the right order, verified through an ASYMMETRIC subcircuit (1k on one port, 10k on the other) so a reversed mapping changes the measured current instead of being invisible; `.subckt bus4 d[3:0]` expands descending and maps positionally across four distinct currents; `R1 a[0:1] 2k` is ONE resistor (series 2k+4k = 0.5 mA); a bus element and an explicit scalar are the SAME node; v(a[0]) and bare a[0] resolve correctly (E-224) while ordinary vector indexing still works; non-integer/malformed/already-scalar tokens are untouched; the guard boundary expands at 8192 and stays literal at 8193. Verify (examples/busoverflow): 5 checks.

E-339 *ngspice-46/src/frontend/parse.c*

`v()` with THREE OR MORE node names SIGSEGV'd ngspice -- found by fuzzing the pyplot com-

mand family, but the defect is in the SHARED EXPRESSION PARSER. `print v(in,out,0)`, `let q = v(in,out,0)` and `pyplot v(in,out,0)` all died; `plot` SURVIVED, which is why an obvious spot-check passes and this was easy to miss. ROOT CAUSE -- TWO PATHS WITH DIFFERENT OWNERSHIP: `v()` takes at most two node names (`v(a)` or the differential `v(a,b)`), and a third was never rejected. PP_mkfnode's comma branch recurses, `PP_mkbnode(MINUS, PP_mkfnode(func,left), PP_mkfnode(func,right))` then `free_pnode(arg)` -- that branch CONSUMES its argument, while the normal path at the end of the function BORROWS it (`p->pn_left = arg; pn_use++`). free_pnode recurses into children guarded by pn_use, so: with TWO names the children are plain nodes, each recursive call takes the BORROW path and bumps pn_use to 2, and the outer free merely decrements -- safe; with THREE OR MORE one child is ITSELF a comma node, so the recursive call takes the CONSUME path and frees it, and the outer free_pnode(arg) then walks into that already-freed child -- a DOUBLE FREE. Node existence is irrelevant: all-present v(in,out,0), all-missing v(a,b,c) and mixed v(in,out,zz) crash alike -- it is purely the arity. FIX: reject the invalid arity where it is detected (`Error: v() takes at most two node names.`) rather than relying on the inconsistent ownership. Repairing the mismatch instead would mean changing PP_mkfnode's contract for every caller in order to make a syntactically invalid expression work; the arity is not legal in the first place, so rejecting is both smaller and more honest. The ownership mismatch is recorded as the underlying hazard. FUZZ: 122 pyplot invocations, 0 crashes/hangs/unexpected writes after the fix -- the eight mode flags (-eye -hist -contour -smith -fft -bode -nyquist -polar) crossed against missing/extra/repeated/conflicting arguments, hostile signal lists (unclosed `v(`, empty `v()`, 8000-character names, control characters, 200 signals) and hostile output base names (3000 characters, spaces, quotes, shell metacharacters, unicode, leading –). NOT A DEFECT, flagged and dismissed: `pyplot ../name` and `pyplot /abs/path` write outside the working directory -- that is the documented `pyplot [file] plotargs` contract and matches wrdata/write, so the fuzz policy was wrong rather than the command. Verify (examples/vfuncarity): 5 checks.

E-341 *ngspice-46/src/frontend/com_sweep.c*

`sweep -analysis reset` and **`-analysis remcirc`** SIGSEGV'd -- found by fuzzing sweep/optimize for RECURSION and STATE. `sweep` runs its -analysis argument as a command at every point, dispatched by name through sw_run_cmd, and nothing stopped a user naming `reset` or `remcirc` -- which free and rebuild (or remove) the very circuit the sweep is iterating over. The loop carried on with its resolved knob bindings, the old CKTcircuit* and the old plot. WHY THE FIX REJECTS UP FRONT RATHER THAN RECOVERING: the obvious repair -- notice the circuit changed and break out -- was tried FIRST AND MADE THINGS WORSE. The sweep's post-loop plot finalisation touches the same freed state, so breaking out merely moved the crash, AND it turned a previously WORKING case (`–analysis 'optimize ...'`, which resets internally but re-establishes its own state) into a NEW crash. That attempt was reverted. Rejecting a destructive analysis before the loop starts cannot regress anything, and nothing legitimate is lost: a sweep re-sources the deck itself when a knob needs it, so a user `reset` in the per-point analysis has no purpose. `optimize` is deliberately NOT rejected -- it resets internally, recovers, and works today. FUZZ (E-270 already covered numeric bounds, so this EXTENDED THE STRATEGY rather than repeating seeds): ROUND 1, argument surface -- 684 invocations, CLEAN -- 20 knob kinds (instance param, `@model[param]`, .param, the E-268/269 wildcards @*[p]*/@#[p]/@*[[p]], malformed @-forms) x 29 sweep specifications (start/stop/step, lin

E-342 *ngspice-46/src/frontend/options.c, variable.c, variable.h*

Two ownership bugs in the synthetic user-variable list: a heap-use-after-free (SIGSEGV) and a double free (SIGABRT). cp_usrvars() synthesizes $plots, $curplot, $curplottitle, $curplotname and $curplotdate and returns a list its CALLERS free. That only works if the list owns every node. Two places assumed it did. [A] cp_enqvar() does NOT always allocate -- it clears its tbfreed out-parameter and returns a BORROWED pointer into plot_cur->pl_env when that live list already defines the requested name. cp_usrvars() declared tbfreed, passed it to all five calls and NEVER

READ IT, so it relinked the borrowed node (`tv->va_next = v`, orphaning the live list's tail) and the caller then freed a node somebody else still owned. Not hypothetical: a rawfile `Option:` line writes pl_env, so a loaded rawfile containing `Option: plots = 1` was enough -- SIGSEGV for all five names, ASan confirming allocate/free/use across three different functions (raw_read -> cp_setparse, cp_getvar -> free_struct_variable, cp_usrvars -> cp_enqvar). Fixed by honouring the flag and copying a borrowed variable; the five near-identical blocks became one loop so the rule cannot be applied to four of five sites. New var_copy() is written as the deliberate DUAL of free_struct_variable() -- deep, because that function recurses into va_vlist and a shallow copy would only move the bug. [B] found while fuzzing the fix for [A] and INDEPENDENT of it, no file involved: cp_remvar() freed the variable it looked up even when nothing had unlinked it. Only the US_OK arm unlinks; `plots` returns US_READONLY and the curplot* names return US_DONTRECORD, leaving the node inside uv1 to be freed a second time by the trailing free_struct_variable(uv1). `unset plots` aborted in malloc on a plain deck. Fixed by tracking whether the node is genuinely ours to free -- which also closes the same latent corruption for variables found in `variables`, pl_env and ci_vars. The ci_vars borrowed path is NOT reachable for these names today (plot_cur is always set, so all five return earlier) -- measured, not assumed -- but the fix keys on the ownership flag rather than the source list, so it covers that path anyway. Fuzz: 640 combinations (5 names x 8 Option: value shapes x 16 follow-up sequences), each run twice, plain and under ASan, all clean. No leak from the copy: 20000 iterations through cp_usrvars(), peak RSS 7.1 MB vs 7.2 MB control (LeakSanitizer unavailable on this host, so measured rather than asserted). Behaviour preserved and DIFFED against the old binary, not eyeballed. Found by tracing the E-341 performance finding through cp_enqvar() and asking what happens when tbfreed comes back 0. Verify (examples/usrvarown): 6 checks.

E-343 *ngspice-46/src/frontend/options.c, variable.c, include/ngspice/cpextern.h*

The O(N^2) sweep cost E-341 documented but did not fix: FIXED, 26.6x at 16000 points (39.55 s -> 1.49 s). cp_getvar(name,...) looks up ONE name, but its first statement built ALL FIVE synthetic user variables ($plots, $curplot, $curplottitle, $curplotname, $curplotdate), searched the result and freed it again. Synthesizing $plots walks the live plot list and copies a string per plot, so every call cost O(number of plots in the session) regardless of what was asked for; beginPlot() does two such lookups per analysis ('printinfo', 'interp') and a sweep creates a plot per point, so point k paid O(k). WHY THE FIRST ATTEMPT FAILED (the whole lesson): E-341 tried making the construction LAZY -- build the list only after the `variables` search misses -- and measured 1.0x. printinfo and interp are normally UNSET, so that search always misses and falls through to needing the list anyway; laziness only helps the lookups that were never the problem. The gate has to be on WHICH NAME IS REQUESTED, not on when the list is built. Neither hot lookup is one of the five, so the fix builds NOTHING at all. cp_usrvars() is kept for cp_vprint(), which genuinely prints all five. Safe because both gated callers already search pl_env and ci_vars separately, which is the only other thing cp_enqvar() could return; ownership still follows E-342. Measured: 7.3x / 13.0x / 18.9x / 24.1x / 26.6x at 1k / 2k / 4k / 8k / 16k points -- the speedup GROWS with N, which is what removing a quadratic looks like. The regression suite itself, same 274 examples, went 748 s -> 479 s, because every analysis in every example was paying two of these lookups. NOT LINEAR YET, and the remainder is characterised rather than hidden: profiling the NEW binary at 64000 points puts 89% of what is left in plot_alloc(), a different function, which scans the whole plot list with a case-insensitive compare to pick a unique name. Two fixes were considered and REJECTED HERE: an exact-membership index (correct, but plot_list is mutated in ~9 places and a missed insertion hands out a DUPLICATE plot name), and a cheaper superset index (contained, but CHANGES BEHAVIOUR -- today plot names are reused after `destroy all`, and a superset would skip past them). Both are changes to plot lifetime, not to this lookup path. Semantics diffed byte-for-byte against the old binary, including a deck with printinfo and interp explicitly SET across op/tran/ac. E-342's two vectors re-checked. ASan clean. Verify (examples/sweepscale): 5 checks.

E-344 *ngspice-46/src/frontend/com_sweep.c*

.model params now take the fast sweep's DIRECT SET -- the one tier E-320 left on the textual path. Model params now cost exactly what instance params cost (they were 2.1x-2.8x slower). The E-320..323 fast `.param` sweep resolves each swept device value to a slot ONCE and pushes it with ft_sim->setInstanceParm, skipping the per-point reset -- but sw_fp_resolve() bailed on 'if (!ckt

E-345 *ngspice-46/src/frontend/vectors.c, com_fft.c, linear.c, postcoms.c, spec.c, mw_coms.c*

The residual O(N) E-343 profiled but rejected fixing: FIXED. The sweep is now LINEAR -- 87x at 64000 points (54.97 s -> 0.632 s). plot_alloc() and plot_add() pick a plot name by counting a shared, monotone plot_num up until `<abbrev><plot_num>` is not the typename of any plot in plot_list. Plots are PREPENDED, so the colliding probe hits at the head in O(1); it is the SUCCESSFUL probe -- proving a name ABSENT -- that walks the whole list, with a tolower per character. One full walk per plot created, and a sweep creates a plot per point. Only the MEMBERSHIP TEST changed: a hash index of the typenames currently in plot_list answers it in O(1). The search still starts at the same shared plot_num and still counts up by one, which matters more than it looks -- plot_num is shared ACROSS abbreviations and only advances on a collision, so the sequence is `op1 tran1 op2 ac2 op3` not `op1 op2 op3`; and a number freed by `destroy all` is REUSED, which is exactly why E-343 rejected a 'remember every name ever issued' superset cache. Both copies of the loop (plot_alloc and plot_add carried the same one) now share plot_unique_typename(). E-343's OTHER objection -- an exact index is unsafe because plot_list is mutated in ~9 places and a missed insertion hands out a DUPLICATE plot name -- was answered by REMOVING the nine places rather than tracking them: plot_new() is now the SINGLE insertion point (six callers open-coded the same two lines: com_fft x2, linear x2, postcoms, spec), killplot( ) calls the new plot_forget(), com_removecirc() (which rewrites plot_list wholesale) calls plot_forget_all(), and the index builds LAZILY from plot_list so the static `const` plot needs no registration. PROVEN, not asserted: plot_name_taken() carries a PLOTNAME_SELFCHECK block that answers the same question the OLD way and abort()s on disagreement; the full suite was run with that build -- 276/276 OK, ZERO disagreements. The block stays in the source, unset, so it can be re-run against future changes. A REAL BUG was hit on the way and is worth recording: converting the open-coded prepends with a REGEX produced a DOUBLE INSERTION, because three sites ALREADY called plot_new() with the redundant two lines wrapped AROUND it, so the edit turned the trailing line into a second plot_new(). The second call ran `new->pl_next = plot_list` when plot_list was already `new` -- a SELF-LOOP -- and vec_gc spun on the circular list forever. Found by a stress case (fft) that hung, located by SAMPLING the hung process, not by guessing. A mechanical bulk edit needs each site read: the pattern being replaced was not the pattern actually there. Names verified IDENTICAL to the previous binary across 15 stress cases (mixed analyses, destroy all, destroy first/middle/last, setplot new, fft, linearize x2, spec, write/load x2, load-then-destroy, twelve-then-destroy, remcirc, sweep-then-destroy, nested destroy). Measured: per-point cost pinned at 10 us and the doubling ratio converged on exactly 2.0x -- the first time this sweep has been linear. With E-343, a 16000-point sweep went 39.55 s -> 0.163 s, about 240x. Verify (examples/plotname): 7 checks.

E-346 *ngspice-46/src/frontend/com_sweep.c*

A BUG FIRST, then the last tier: the fast **.param** path was silently FREEZING random draws that the reset path re-drew -- and fixing that IS the Monte Carlo tier. The fast sweep only re-evaluates brace expressions that MENTION the swept name. But numparam INLINES a .param's expression into the device line during preprocessing, so `.param rv = agauss(...) + R2 a b {rv}` arrives as `r2 a b {(agauss(...))}` -- carrying the random call but NOT the swept name, so it was skipped and never touched again. (Same discovery: nupa_recompute_params, added in E-320 to refresh the derived-param closure, replays ZERO lines on such a deck -- there is no .param line left.) Measured over 5 points: reset gave rv = 1097.7/1015.2/952.5/988.7/969.2, fast gave 1161.1 FIVE TIMES. E-320's guarantee that results never change, only speed, had not

held for any deck with a random .param since it shipped. Fix: capture brace expressions that draw from the RNG (agauss/gauss/unif/aunif/limit/mvnorm) even with no swept name present. Once that exists MONTE CARLO NEEDS NOTHING MORE -- a sample is a re-draw plus an in-place push, and `montecarlo` arms the same machinery with NO swept knob (sw_fp_build(NULL,0)). Bit-identical to reset required THREE things, each found by a test that failed first: (1) DECK ORDER -- random binds sort ahead of the expression-grouped ones by capture index, since only they consume the RNG, so the stream matches however the deterministic binds are grouped; (2) NO CACHING -- a random bind is never served from the by-expression-text cache, because two devices with identical text must draw INDEPENDENTLY (verified: two {agauss(1000,300,3)} resistors give 1113.1 and 1099.5, not equal); (3) the SAMPLE BOUNDARY -- a deck re-copy signals one MC sample via nupa_signal(NUPADECKCOPY) -> mc_sample_advance(), which rewinds the per-sample dimension counter and steps the Latin-Hypercube sampler; skipping the re-source skipped that signal, which is exactly why plain draws matched reset (240/400 both) while -lhs did NOT (239 vs 258) and -lhs -warm did not (252 vs 258). Raising the boundary in the apply path made all four agree. The E-321 arm-time self-check is SKIPPED for random binds -- re-evaluating one draws a NEW value so it could never reproduce the baked one, and the draw would perturb the very stream that must match; their safety rests on the structural disarms, which were extended so a random gets the same scrutiny a swept name gets (a random reaching .temp, a subckt call or any structural slot disarms). Verified against an INDEPENDENT ORACLE, not a golden number: the same experiment hand-written as a `reset` loop in the control language -- montecarlo 124/200, oracle 124. Battery of 12 cases run on BOTH paths with discriminating yields (60.3/57.7/27.7/84.0/89.7/90.8%): plain, -lhs, -warm, gauss, aunif, limit, two randoms, same-expression-twice, random model param, two specs, derived random (disarms), structural random (disarms) -- all identical. Speed 1.8x/2.3x/2.4x at 20/200/1000 devices -- smaller than the sweep tiers because an MC sample re-draws and re-pushes EVERY random value; the saving is the re-source, not the pushes. CHANGES EXISTING RESULTS: a sweep over a deck with a random .param now re-draws it per point instead of freezing it -- which is what makes the fast path agree with reset again. Regression 278/278. Verify (examples/mcfastpath): 7 checks.

E-348 *ngspice-46/src/spicelib/parser/inp2dot.c, src/spicelib/analysis/dcpss.c*

`.pss` segfaulted on a short argument list -- and, separately, on a COMPLETE one whose **`harmonics`** was 0. Found by a new sweep shape: take a KNOWN-GOOD invocation of each command and run every PROPER PREFIX of its argument list. Existing campaigns could not see this -- `cmdfuzz` mutates argument VALUES (every value it supplies is present) and the parser fuzzers feed malformed FILES; a command that reads its fifth argument after checking only that a first exists is invisible to both. Across 38 commands `pss` was the ONLY offender: 0 args hung, 1-4 args SIGSEGV; `qpss` (8 args) and `optimize` (12) truncate cleanly. The stripped binary gave only the shape -- `ldr d0,[x19]` with x19 == 0, then `fdiv`, then a store -- and the UBSan build named it: dcpss.c:4990, load of null pointer of type 'double', i.e. `Mag[0] = Phase[0] / (double)ndata`. THREE failures stacked. (1) dot_pss() reads seven required values with no end-of-line check, and INPgetValue() cannot report 'nothing left to read' -- it returns 0 -- so a short card reached the analysis with points/harmonics silently 0. (2) DCpss() never validated: CKTharms is the LENGTH of every array the DFT writes into, and TMALLOC(double, 0) is tmalloc(0), which returns NULL. (3) DFT() writes index 0 OUTSIDE any loop -- Mag[0], Phase[0], nMag[0], nPhase[0], Freq[0] -- so its numFreq bound never protected them. That third point is why a parser guard ALONE would have been insufficient: `pss 1meg 1u out 1024 0 50 5u` is well-formed at full arity and crashed by the same route, a second entrance that never passes the truncation check. Fixed at all three layers: a PSS_NEED_ARG macro naming the missing argument (the idiom dot_dc() already uses a few hundred lines earlier in the same file, which dot_pss simply never adopted); DCpss() rejecting fguess <= 0, points < 1 and harms < 1 before anything is sized from them; and DFT() guarding its own unconditional index-0 stores. WHAT WAS DELIBERATELY NOT TIGHTENED, because it was measured rather than assumed: every parameter was tested at 0 and negative, and `sc_iter` and `steady_coeff` are harmless at both (rc=0), so they are still accepted -- narrowing

the accepted input on no evidence could break a working deck. NO VALID ANSWER MOVED: 22 correct invocations (op/dc/ac/tran/tf/pz/sens/noise/disto/four/pss plain and uic/pac/pnoise/hb/sp) were run on the shipped and rebuilt binaries and every printed number compared EXACTLY -- all identical, the one apparent difference being `Total analysis time`, i.e. the clock. Truncation sweep 0 crashes from 5. Verify (examples/pssargs): 7 checks.

E-349 *ngspice-46/src/spicelib/parser/inp2dot.c*

One transposed letter in a node name killed the process -- **`op`** then **`tf v(ouy) v1`** and the session is gone, with every loaded circuit, plot and let-defined vector. Seven commands did it: tf, pz, noise, sens, pss, pxf, psp. Found by a STATEFUL CROSS-COMMAND fuzz -- random sequences of state-mutating commands followed by a canary circuit whose answer is known exactly (1 V across a 1k/3k divider, v(b) must be 0.75). The harness was first aimed at the pre-E-342 binary, where it found 38 aborts in 200 sequences, so a clean run means the detector works rather than that it is asleep. ROOT CAUSE: INPtermInsert() is CREATE-OR-FIND. That is right for a device card, where the device DEFINES its nodes (`R1 in out 1k` is what brings `out` into existence), and wrong for an analysis card, which only REFERS to one. A mistyped name was quietly created as a new unconnected node. The milder consequence is arguably the worse bug -- before setup nothing crashes and the invented node reads back as a perfectly plausible `v(bogus) = 0.000000e+00`, no error, no warning. The fatal one: from the .control section CKTsetup() has already snapshotted the node list into prev_CKTlastNode and sized the matrix, so the extra node makes the tail check ending CKTunsetup() fail -- and that check calls controlled_exit(EXIT_FAILURE). The check is CORRECT and was firing on a real inconsistency; the defect is upstream of it. This never bit in a plain netlist purely because of ORDERING: a .tf card in the deck is parsed BEFORE CKTsetup() runs, so the invented node is inside the snapshot and the tail still matches -- it is reachable only interactively, which is exactly where a typo is most likely and losing the session hurts most. Fix: analysis cards resolve through INPtermSearch(), the lookup-only half of the pair that already existed alongside INPtermInsert() and was simply never used here; an unknown name is reported (`no such node: ouy`) instead of created. DECK ORDER STILL WORKS, and this is the constraint that shaped the fix rather than an afterthought -- a .tf card may legally sit ahead of the devices that define its nodes, so creation remains permitted while the circuit is NOT yet set up, and is refused only once CKTisSetup is true, i.e. deck parsing is over. Applied at 14 call sites across dot_noise, dot_tf, dot_sens, dot_pss, dot_pac, dot_psp, dot_pnoise, dot_pxf and dot_hb; dot_pz reached the same insert INDIRECTLY through INPgetValue(IF_NODE) and its four node arguments are now read explicitly so they take the same path. Verified: 0 of 14 commands kill the process, from 7; the same reference as a deck card still parses; a .tf card placed BEFORE its devices still returns transfer_function 0.75, output_impedance 750, input_impedance 4000; 22 valid invocations compared digit-for-digit against the shipped binary, all identical; and the stateful fuzz that found it went 58/60 to 60/60. Verify (examples/nodetypo): 6 checks.

E-350 *ngspice-46/src/frontend/com_sweep.c*

A sweep never put its **`.param`** back, and after two sweeps **`reset`** could not either. `.param rload = 3k`, sweep it 1k..5k, sweep it again 2k..10k, then `reset` + `op` gave 0.909 -- rload still 10k while the deck said 3k. One sweep was fine; the second broke it. ROOT CAUSE: the two implementations disagreed about the state they LEAVE BEHIND, which is exactly what E-320 guarantees they never do ("a sweep can only get faster, never change its result") -- it held for the numbers, not for the aftermath. The reset path drives each point with `alterparam`, which edits the parameter PERMANENTLY, so a later `reset` re-sources a deck that now carries the last swept value; the fast path never touches the deck, so `reset` did restore. Same command, same netlist, different circuit afterwards, decided only by which path happened to arm. WHY THE SECOND SWEEP: arming self-checks each captured expression against the value numparam baked into the flattened card at nominal (a guard against a subckt-local shadow), but the first sweep left the dico holding its last swept value while the flattened card still carried the nominal one -- so the check could not pass for an honest reason and DISARMED. The second sweep of a parameter therefore fell silently back

to the reset path: no error, no banner, the `fast .param path armed` line simply stopped appearing -- and the reset path is the one that edits the deck permanently, which is why the damage needed exactly two sweeps to surface. Both symptoms, one root cause: the sweep never restored the parameter. FIX: capture each swept .param before the first point and restore it at cleanup -- deck text via sw_set_deck (what `reset` re-sources) AND the numparam dico via nupa_add_param + nupa_recompute_params (what the NEXT sweep reads and what arming self-checks against). BOTH HALVES ARE NEEDED and the first attempt shipped only the deck half: with the dico still dirty the second sweep read the first sweep's last value as its own nominal and dutifully restored THAT, giving 0.833 instead of 0.909 -- better, and still wrong. The capture is all-or-nothing (if any swept name cannot be read back nothing is restored) rather than putting some parameters back and not others. Restoring is the direction that makes the two paths agree; making the fast path permanent instead would have satisfied the letter of the E-320 invariant while leaving `reset` unable to undo a sweep and every repeat sweep disarmed, and it matches how ngspice already treats a swept source in `.dc`, which does not leave it parked at the last step. DELIBERATELY UNCHANGED: the live circuit still holds the last swept point, so a bare `op` afterwards answers with that value -- true of both paths before this change and unchanged by it; only the deck and dico return to nominal, so `reset` means something again. Verified: `reset` gives 0.75 after 1/2/3/ mixed-spec sweeps (was 0.909 from the second on); 3 of 3 sweeps arm the fast path (was 1 of 3); derived params and subckt-internal values restore exactly; and 12 sweep batteries compared digit-for-digit against the shipped binary -- plain/list/dec, repeated sweeps, derived, subckt-internal, model params, Monte Carlo draws twice over to catch RNG-stream drift, and a two-knob family, 5-11 numbers per case, none vacuous -- ALL IDENTICAL, so nothing a sweep computes moved. Regression 282/282. The example is a proven trigger, not decoration: on the pre-fix binary 4 of its 5 checks fail, and the one that passes is precisely the invariant that must hold on both. Verify (examples/sweeprestore): 5 checks.

E-351 *ngspice-46/src/osdi/osdisetup.c, ngspice-46/src/osdi/osdidefs.h*

`sens` killed the process on any OSDI model carrying an INTERNAL NODE -- which is every production compact model. BSIM, HICUM, PSP and EKV all allocate internal nodes for their terminal resistances, so .sens was effectively unusable with real Verilog-A models, and it failed by taking the session down: ”Internal Error: node allocation in DEVsetup() during sensitivity analysis, this will cause serious troubles !, please report this issue !” then exit(1). FOUND BY a campaign built on the observation that the previous OSDI sweep (83/83 clean) was LOPSIDED -- 40 dc and 22 ac checks against only 2 sens, 2 disto and 4 pz. Those thin three are exactly where an OSDI device is most likely to differ from a built-in, because they do not merely solve the circuit, they consume the DERIVATIVES the model returns. Three axes were run (OSDI vs equivalent built-in on both solvers; Sparse vs KLU on the same OSDI deck; repeat-invariance): 181/184, and all three failures were this one bug, on the single model in the set with an internal node. ROOT CAUSE: cktsens.c re-invokes every model's DEVsetup() to stamp the perturbation matrix and requires that doing so allocate NO nodes -- it snapshots CKTlastNode around the call and calls controlled_exit(EXIT_FAILURE) if it moved. The requirement is reasonable and every built-in device meets it by guarding allocation on ”not already allocated”: the inductor does `if (here->INDbrEq == 0) { CKTmkCur(...); here->INDbrEq = tmp->number; }` and INDunsetup() zeroes INDbrEq again. OSDI had NO such guard -- its setup called CKTmkVolt/CKTmkCur for each internal node unconditionally, so the second call, the one sens makes on a circuit that is still set up, allocated a fresh set and tripped the check. The check was CORRECT and firing on a real inconsistency; the defect is upstream of it. FIX: the same idiom the built-ins use. Each instance records in OsdiExtraInstData the node numbers its internal nodes were given, and a later setup reuses them instead of allocating. OSDIunsetup() deletes those nodes so it also clears the record -- the OSDI counterpart of INDunsetup() zeroing INDbrEq -- because otherwise a genuine re-setup after teardown would hand out numbers for nodes that no longer exist. Reuse is conditional on the node COUNT matching, since collapsing depends on parameters that `alter` can move; if it changes, the record is dropped and nodes are allocated afresh. NOT MERELY NON-FATAL -- CORRECT, checked two

independent ways because ”it stopped crashing” proves nothing: an internal-node model (r=3000 + rs=10) and a plain one (r=3010) are the SAME circuit (operating points agree to 14 digits), so their sensitivity to the shared built-in R1 must be identical -- and is, at -1.8718770344518e-04 from both decks, every printed digit, against a closed form -RL/(R1+RL)^2 = -1.87187890622571e-04 (the 1e-6 residue is the sensitivity method's own perturbation step, not the fix). The model's own parameter agrees too: n1_r 6.21886087563079e-05 vs closed form 6.21886679809205e-05. Verified: campaign 184/184 (from 181/184); sens completes on both solvers; sens twice in one session gives 41 bit-identical values; a reset -> sens -> reset cycle returns the operating point to 7.50623441396507e-01, guarding that the node record never outlives the nodes; all 7 other analyses on the same model unaffected. Regression 283/283. The example is a proven trigger: on the pre-fix binary 5 of its 7 checks fail, and the two that pass are precisely the ones that should -- the model WITHOUT an internal node, and the analyses other than sens. Verify (examples/osdisens): 7 checks.

E-352 *ngspice-46/src/osdi/osdidisto.c (new), ngspice-46/src/osdi/osdi.h, osdiinit.c, osdiregistry.c, osdiext.h, ngspice-46/src/include/ngspice/osdiitf.h, ngspice-46/src/spicelib/analysis/cktdisto.c*

SUPERSEDED BY E-359 (the capability stands and is still verified by the same oracles; the compiler-emitted-tensor IMPLEMENTATION below was replaced by numerical differentiation of the analytic jacobian in ngspice, removing the compile-time regression, the 30MB objects, the runtime penalty and the ABI bump). **`.disto`** now includes a Verilog-A model's nonlinearities; previously it printed a warning and left them out. Distortion is a VOLTERRA analysis: it needs each device's Taylor expansion of I(v) to THIRD order, not the operating-point linearisation the Jacobian provides. Built-in devices hand-code those coefficients (diodset.c computes id_x2/id_x3), which is why only FOUR of ~58 built-ins implement DEVdisto at all -- diode, BJT, BSIM1, MES -- and the OSDI ABI stopped at first derivatives, so cktdisto had nothing to ask an OSDI device for. KEY ENABLER, verified before anything was built: OpenVAF's autodiff is ALREADY arbitrary-order (mir_autodiff's DerivativeIntern chains through `previous_order`, with a comment anticipating 8th order), and nested ddx(ddx(ddx(...))) reproduces closed form to every printed digit. So the compiler work was not implementing higher-order AD but ASKING for it: build_taylor_tensors re-runs auto_diff over its own first-order results for the second, and again for the third. NO 3-VARIABLE CEILING: ngspice's framework caps at three controlling variables (Dderivs holds derivatives ”w.r.t 3 variables”; the DFx helpers take up to 27 scalar arguments with no fourth-variable form, DFn2F12 taking a struct because the list outgrew the calling convention). That cap belongs to those hard-coded helpers, not to the mathematics -- the contribution is a symmetric tensor contraction -- so osdidisto.c contracts directly over however many inputs a device has and never calls D1x/DFx. Proven with an ideal mixer I(out)=k*V(a, ref)*V(b,ref) whose ONLY nonlinearity is the mixed partial (both diagonal 2nd derivatives identically zero, so a dropped cross term gives exactly zero): matches closed form 0.5*k*A^2*R EXACTLY. TWO SILENT-ZERO BUGS found on the way, both indistinguishable from a linear model: (1) the tensors were OPTIMISED AWAY -- ensure_optbarriers protects residuals/noise/jacobian but the Taylor values feed nothing else in the MIR, so the optimiser deleted them and every coefficient arrived as zero (they also take the mfactor scaling, since m parallel devices inject m times the distortion current); (2) NOTHING STORED THEM -- eval's jacobian/residual stores are gated on CALC_ flags that .disto never sets.* Plus one wrong-by-a-constant bug only a reference implementation could catch: D1nF12 returns 0.5*S2vF12(...) and S2vF12 ALREADY sums both index orderings, so without that 0.5 the mixed-tone products came out exactly 2x -- and because IM3 consumes the f1-f2 kernel its error was a non-obvious 2.246x. KNOWN LIMITATION AT THE TIME, SINCE LIFTED BY E-353: a model whose contribution goes through $limit emitted NO tensors (the residual depends on the limited value, not the raw voltage read; build_jacobian handled that via intern.lim_state and the tensor pass did not). Limiting is standard in production diode/BJT/MOS models, so this was a real restriction; E-353 folds the limited values into the derivative chain and removes it. What remains unreachable is a nonlinearity in a GROUND-REFERENCED probe (the tensors are indexed by model input and a bare V(a) is not one, having no hi/lo pair). Registering DEVdisto

removed cktdisto.c's blanket warning, which would have turned such a model into a SILENT ZERO -- worse than before -- so OSDIdisto warns for itself. VERIFIED against four oracles, two involving no simulator at all: HD2/HD3 vs a closed-form polynomial (0.0 and 1.2e-16); the A^2/A^3 scaling laws (4.0000, 8.0000); f1+f2, f1-f2 and IM3 vs the built-in diode (1.87e-06, the bound the 0.24 ppm $vt difference justifies); IM3 vs transient+FFT in a DIFFERENT DOMAIN, converging 5.7% -> 1.4% -> 0.21% as the drive halves, exactly the A^2 signature of the 5th-order content a 3rd-order truncation excludes; and the multi-variable cross term (0.0). TWO HARNESS TRAPS recorded because both would have produced a confident wrong answer: the first IM3 transient was measuring SPECTRAL LEAKAGE from the fundamentals rather than IM3 (exposed by the A^3 law giving 2.93 instead of 8, not by the value looking wrong); and a first parameter choice made HD3 IDENTICALLY ZERO (the two contributions cancel exactly when c3 = 2c2^2/Ytot, and 2(1e-4)^2/2e-3 is precisely the 1e-5 chosen) -- a zero that would have passed a naive check for the wrong reason, the same trap as the old campaign's "HD2 ~ 0 on a LINEAR network". The suite now refuses to score a both-zero comparison. Regression 284/284. The example is a proven trigger: on the pre-fix binaries 5 of its 6 checks fail, and the one that passes is the one that should. Verify (examples/osdidisto): 6 checks.

E-353 *ngspice-46/src/osdi/osdidisto.c, examples/limitdisto_examples/*, examples/osdidisto_examples/

SUPERSEDED BY E-359 (`$limit` models still work -- the suite below passes unchanged -- but the chain-fold no longer exists; differencing the real jacobian makes the problem unable to arise). **`.disto`** now works for Verilog-A models that use $limit -- i.e. every production compact model. E-352 gave OSDI devices distortion analysis, but only for models reading their controlling voltages DIRECTLY; a model passing one through $limit contributed NOTHING, and limiting is how every production diode/BJT/MOS model converges, so the feature did not reach the models people actually run. ROOT CAUSE: $limit hands the model a LIMITED copy of a branch voltage and the residual is written in terms of that copy, so differentiating the residual by the RAW voltage read yields zero -- nothing in the expression depends on it. The model is not linear, but every derivative the tensor pass asked for was. This was never a problem for AC or transient: build_jacobian has always folded the limited values back in via intern.lim_state; E-352's build_taylor_tensors simply did not perform the same fold. FIX: each model input now maps to a CHAIN of autodiff unknowns rather than a single one -- new taylor_unknown_chain returns the limited values from intern.lim_state.raw (each with the sign it contributes) followed by the input's own unknown. The 2nd-order pass sums over every PAIR drawn from the two chains and the 3rd-order pass over every TRIPLE, each term carrying the XOR of its entries' signs; an entry is negated when the model limits a REVERSED branch, $limit(V(ref,a),...). VERIFICATION uses an exact expectation with no appeal to another simulator: $limit is a convergence aid, so at convergence the limited value equals the actual one and a model written with it is THE SAME DEVICE as the same model written without it -- every distortion product must agree. Both spellings are generated from ONE body string, which is what guarantees they differ only by $limit and cannot drift apart. Five shapes cover the chain combinatorics (where this can go wrong -- a model limiting a single branch never puts more than one entry on either side of a pair): two independently limited diagonals; a cross term with BOTH inputs limited (2x2 pairs); a cross term with ONE input limited (asymmetric chains, 2x1); a THIRD-order cross term whose chain has 3 entries, so the triple loop genuinely runs; and a reversed-branch limit, the only spelling producing a NEGATED chain entry. All 13 live comparisons agree, worst 7.1e-10 -- convergence noise, since the two spellings settle 8.3e-11 apart (limiting changes the Newton path, not the solution) and d2/dv2 of exp(v/vt) amplifies that by 1/vt^2. THREE WAYS THIS COULD HAVE PASSED WHILE WRONG, each caught and closed because each produces a confident green: (1) BOTH-ZERO comparisons -- 'no distortion' is precisely the pre-fix behaviour, so scoring 0 == 0 as agreement would certify the bug; the suite scores a both-zero product as a FAILURE except where the mathematics requires zero (a bilinear term k*v1*v2 has no third derivative, so its IM3 is correctly zero and is asserted as such). (2) COMPARING MAGNITUDES -- a dropped sign on a negated chain entry flips the result and leaves

E-359 *ngspice-46/src/osdi/osdidistonum.c (new), osdidistonum.h (new), osdidisto.c, osdi.h, osdiregistry.c, Makefile.am, src/include/ngspice/osdiitf.h, examples/osdidisto_examples/**

.disto for Verilog-A rebuilt on NUMERICAL differentiation of the analytic jacobian; the compiler-side tensor machinery is DELETED. E-352 obtained the 2nd/3rd-order Taylor coefficients by having the compiler emit a symbolic closed form. The capability was right but the cost was out of all proportion to what .disto does: on ASMHEMT 707k intermediate derivatives -> 1.4M MIR instructions -> a 30.2MB .osdi and a 126s compile against a 3.6s baseline (35x; hisimhv 20x), plus -- because the tensors were computed inside eval() -- a 3.3-3.9x RUNTIME penalty on every DC sweep, transient, AC and noise run, and an OSDI 0.8->0.9 ABI bump that made every already-compiled model unusable without a rebuild. ROOT MISTAKE: .disto needs the coefficients at ONE point, once per instance, once per analysis; E-352 computed a closed form valid at EVERY point in order to evaluate it once. WHAT REPLACES IT: the model already publishes its FIRST derivatives analytically -- that is the jacobian every analysis loads -- so the higher ones follow by differencing THAT at the operating point (d2 R_r/dv_p dv_q = d/dv_q[J_rp]; d3 = d2/dv_q dv_s[J_rp]). Differencing an already-analytic first derivative rather than the residual is what keeps it accurate. Two properties make it cheap: the tensors are evaluated at the DC OPERATING POINT so they are frequency-INDEPENDENT (built once at D_SETUP, reused by every mode and every frequency), and they are built in NODE coordinates where an instance has 5-20 nodes rather than the 52 'model inputs' ASMHEMT reports. Entries keep the exact (row, col...) shape the analytic path produced, so the whole verified Volterra contraction is reused UNCHANGED -- only the tensor source and the kernel differ. NODE COORDINATES ALSO LIFT A REAL LIMITATION: E-352 indexed by model input (a branch voltage with a hi/lo pair), so a nonlinearity in a GROUND-REFERENCED probe V(a) had no pair, was never recorded, and the device reported 'contributes no distortion' and returned zero; it now works and is pinned to a closed form (-4.16666665e-02 vs exact -4.16666667e-02). E-353's $limit chain-fold disappears for the same reason -- differencing the real jacobian sees what the simulator sees, so there is no chain to fold, and E-353's seven-shape suite (written specifically to break that logic) passes unchanged against an implementation containing none of it. ACCURACY, measured against EXACT CLOSED FORMS rather than against the old implementation: cubic 4.0e-09/5.4e-09, exponential diode 2e-10/~1e-10, internal-node 3.6e-12/1.2e-08, ground-referenced 4e-09, MEXTRAM vs the analytic implementation 5e-09/1e-07 -- all below the 1.9e-06 floor the built-in-diode oracle already sits at (a 0.24 ppm $vt difference). THE STEP SIZE IS THE WHOLE GAME: truncation is O(h^2) and roundoff O(eps/h) at 2nd order but O(eps/h^2) at 3rd, so one step cannot serve both (a single 1e-3*V step put the result 8.5e-5 out); and the right yardstick is not the operating voltage but the CURVATURE scale (v_t = 26mV for a diode, volts for a polynomial), so a voltage-relative step left a cubic biased at 0V 11% wrong. The 2nd-order pass measures

E-360 *ngspice-46/src/osdi/osdidisto.c, examples/osdidisto_examples/**

A circuit with TWO OR MORE different Verilog-A models reported distortion for only the LAST one; every other model contributed ZERO, silently. Introduced by E-359 and shipped with it. CAUSE: DEVdisto is dispatched once per DEVICE TYPE -- cktdisto.c walks DEVices[] and calls each type's handler in turn -- and every distinct .osdi registers as its own device type, so a circuit using two Verilog-A models calls OSDIdisto TWICE for every mode, D_SETUP included. E-359 cached the numerical tensors in ONE GLOBAL array and cleared it at D_SETUP, so model B's setup destroyed model A's tensors; in the mode passes A's instance pointers then failed the consistency check and every instance was skipped. That check is the only reason this produced NO numbers rather than WRONG ones -- without it model A would have contracted model B's tensors against A's kernels and produced confident garbage. FIX: the cache is keyed by descriptor (OsdiNumModelCache{descr, ent, n, cap}; numcache_for(descr,create) finds-or-adds, numcache_reset drops only that model's entries), so each model only ever clears its own; lookup is linear in the number of MODEL TYPES, a handful, not in instances. WHY THE TESTS MISSED IT: every distortion test -- the six E-352 checks, the seven E-353 $limit shapes, every model in the corpus

campaign -- used EXACTLY ONE .osdi. The bug needs two. Single-model coverage was thorough enough to hide a whole-feature failure. The new check [7] compares the cubic branch's value WITH a diode present against the same branch ALONE, so it fails on the silent zero rather than merely asserting non-zero; against the pre-fix binary it reports '0.00000000e+00 alongside a diode vs 6.24999998e-03 alone'. Also stress-tested: 50 instances (cache growth past its initial capacity of 32; first and last match the single-instance reference) and three model types with .disto run TWICE in one session (every branch correct on both runs, so the per-model reset is idempotent). Verify (examples/osdidisto): 7 checks. Regression 285/285.

E-361 *ngspice-46/src/osdi/osdidistonum.c, ngspice-46/src/spicelib/analysis/distoan.c, examples/osdidisto_examples/**

Two sanitizer findings in .disto that every ordinary build tolerates silently. (1) HEAP-BUFFER-OVERFLOW, mine from E-359: numdisto_jac_at copied the solution vector with `for (i = 0; i <= n; i++)` where n = CKTmaxEqNum, but CKTrhsOld holds CKTmaxEqNum entries -- ngspice's own code copies it as `CKTmaxEqNum * sizeof(double)` -- so the last valid index is n-1 and this read one element past the end on EVERY .disto run with a Verilog-A device. ASan: 'READ of size 8 ... 0 bytes after 32-byte region', a 4-double region faulting at index 4, matching exactly. It is a READ, so nothing was corrupted and the computed values are unchanged by the fix, but it is undefined behaviour on the main path; the same off-by-one was in the node-range guard (gp > n where the valid range is gp < n). (2) (int)NaN IN THE SWEEP POINT COUNT, pre-existing ngspice in distoan.c and reproducible with NO OSDI device in the circuit at all: a linear sweep with start == stop leaves DfreqDelta at zero so the tolerance term is 0/0, and a decade/octave sweep with ZERO steps does exp(log(10)/0) = inf and then evaluates 0 * inf -- converting NaN to int is undefined, so a nonsense request quietly ran with whatever point count the hardware produced instead of being refused. FIX: zero-step decade/octave sweeps are rejected the way an unknown step type already is, and the linear division is guarded (where DfreqDelta is non-zero the term is floor(DfreqDelta*reltol/DfreqDelta) = floor(reltol) = 0 for any sane reltol, so zero is the value consistent with every other case and normal sweeps are bit-identical). NOTE: this second one had been firing under the project's OWN test deck all along -- osdidisto check [4]'s mixer sweeps `lin 1 1e6 1e6`, precisely the start == stop trigger -- and nothing ever failed because an un-sanitized build tolerates it. Neither defect produced a wrong number: one is an out-of-bounds READ, the other an undefined conversion that happened to yield a usable value on this hardware. That is the class only a sanitizer finds, and the argument for running the suites under one after any change involving raw index arithmetic. Verify (examples/osdidisto): 8 checks, the new one asserting a zero-step sweep produces no output rather than inventing some (an ordinary build cannot observe the UB itself). Both suites 8/8 and 7/7 under ASan+UBSan with values unchanged. Regression 285/285.

E-362 *ngspice-46/src/spicelib/analysis/{distoan,dctran,cktsens,dctrcurv}.c, examples/sweepguard_examples/* (new)*

Fuzzing analysis-card sweep parameters against an ASan+UBSan build: SEVEN pre-existing defects, all reachable from ordinary input, three of the counts involved being used as ALLOCATION SIZES and one reaching the allocator NEGATIVE. E-361 found one (int)NaN conversion in .disto's point count; that looked like a family rather than a one-off, so this drives every sweep-taking analysis (.disto/.ac/.noise/.sens/.sp/.dc/.tran/.four/.fft) with degenerate and extreme parameters. FINDINGS: distoan.c DECADE and OCTAVE -- the count goes +-inf (a zero or denormal start frequency makes log() diverge) or exceeds INT_MAX, and the (int) cast is undefined, feeding DstorAlloc(..., NoOfPoints+2); distoan.c LINEAR -- `DnumSteps+1` is plain INT arithmetic and overflows at INT_MAX BEFORE any double-valued guard can run; distoan.c LINEAR -- the final count could be negative, reaching the allocator as ASan's 'requested allocation size 0xfffffffffffffff0'; dctran.c -- CKTtimeListSize overflows and is used both by TMALLOC and by osdiaccept.c to size a buffer; cktsens.c x2 -- the same cast pattern driving the sensitivity sweep; dctrcurv.c -- .dc has NO bound at all, advancing by a step and comparing against the stop value, so `dc V1 0 1 1e-`

30 (a typo for 1e-3) runs forever with no diagnostic, and a step below the ULP of the start never advances the value at all. .tran already declines the equivalent request and a zero step is already refused, so .dc was the outlier. ONE FIX MADE THINGS WORSE FIRST: clamping CKTtimeListSize to a representable value regressed `tran 1e6 1e30 1e6` from erroring to HANGING, because ngspice's existing rejection of absurd transients WAS this very overflow producing an allocation size that failed -- clamping made the allocation succeed and the run proceed forever. The accidental rejection had to become a deliberate one (E_PARMVAL); absurd transients now return rc=1 exactly as before and ordinary ones are bit-identical. TWO FINDINGS WERE NOT BUGS and were deliberately not 'fixed': an enormous but finite sweep is SLOW, not hung, and a timeout cannot tell them apart -- `sens ac dec 1000000` over 300 decades measured 0.05/0.20/1.78/17.4s at 1/10/100/1000 steps (linear, so simply large), and `ac oct 1000000 1e300 1e6` takes 12.8s identically on the shipped and fixed builds, exceeding the fuzzer's timeout only under ASan's ~5x slowdown. The scaling check that distinguishes them is documented in the harness. Final fuzz run 400 iterations: SANITIZER=0, CRASH=0, no real hangs. Verify (examples/sweepguard): 3 checks pinning eleven repros plus six ordinary sweeps that must be unaffected; a proven trigger -- against the pre-fix build it reports three HANGs. Its 'produces no output' check passes even pre-fix, because an ordinary build cannot observe an undefined conversion, only its consequences, which is why the sanitizer harness (fuzz_analysis.py) ships alongside it rather than being folded into the regression. Regression 285/285.

E-364 *src/osdi/osditrnoise.c (new), src/osdi/{osdidefs.h,osdiload.c,osdisetup.c,osdi.h,Makefile.am}*

TRANSIENT NOISE for OSDI devices: Verilog-A noise sources are now injected into **`.tran`**. `.noise` linearises about an operating point, so it cannot show jitter, noise-induced switching or oscillator phase noise; ngspice has had `trnoise` for independent sources for years, but OSDI devices were the only silent components in an otherwise noisy circuit. NO ABI OR COMPILER CHANGE was needed -- the compiler already emits a full parametric description that NOTHING read: `noise_source_type[]`, `load_noise_params()` (per-source power and exponent, i.e. S(f)=power/f^exp at the CURRENT bias), and each source's node pair and correlation name. `load_noise_params` and `noise_source_type` were dead ABI, partly because the `NOISE_TYPE_*` names lived only in the compiler-side header and had never been mirrored into ngspice's `osdi.h` (now fixed). ACTIVATION IS AUTOMATIC with no option to set: it turns on when the circuit already has a `trnoise` source and adopts that source's noise timestep, so all generators share one grid; it is deliberately NOT keyed on 'the device declares noise sources', which every real compact model does and would mean always-on. AMPLITUDE LAW, derived not fitted: `f_alpha` is Kasdin's fractional-noise method, H(z)=(1-z^-1)^(-a/2) so

E-365 *src/include/ngspice/cktdefs.h, src/spicelib/analysis/{cktpzset.c,cktsetup.c}, src/frontend/com_hb.c*

`pz` followed by **`hb`** returned a SILENTLY WRONG harmonic-balance result (DC term 0.2500 instead of the correct 0.2439, 2.5% out) and, underneath it, read freed memory. `CKTpzSetup` opens with `NIdestroy(ckt); NIinit(ckt);` -- freeing `ckt->CKTmatrix` and building a DIFFERENT one -- while leaving `CKTisSetup` asserted, so every device's cached matrix-element pointer (bound by `CKTsetup` into the old matrix) dangles. `com_hb` then guarded its setup with 'if (ckt->CKTmatrix == NULL

E-366 *src/frontend/com_qpss.c, src/include/ngspice/klu-binding.h*

Two more sites of the E-365 stale-binding class, found by continuing the SAME sequence-fuzzing campaign against the FIXED build. Widening the pool (RF/steady-state commands, sweep/wcd, and SOLVER SWITCHING `option klu`/`option sparse` between analyses -- added deliberately, since KLU and SPARSE keep separate per-instance pointer sets) took the yield from 500 iterations/1 signature to 700/3. (1) `com_qpss.c` carried the identical guard `com_hb.c` had -- 'if (ckt->CKTmatrix == NULL

E-367 *src/frontend/typesdef.c, src/frontend/com_sweep.c*

sweep produced plots nobody could name. (1) `plot_alloc()` names a plot from the `plotabs[]` table in typesdef.c and falls back to the literal ”unknown” when there is no entry -- and EVERY plot type this project added was missing: sweep, sweepwave, hb, envelope, eye, loadpull, rfstab, stb. `sweep` therefore created unknown4/unknown7/unknown10. All eight registered. ORDER IS LOAD-BEARING: ft_plotabbrev returns the FIRST entry whose pattern is a substring of the plot name, so ”sweepwave” must precede ”sweep” or a waveform plot would be abbreviated ”sweep” and collide with the point plot (the same rule already explains why a plot named ”spectrum” matches the earlier ”sp” entry, never ”spect”). The struct is {p_name, p_pattern} -- the SECOND field is matched and the FIRST is returned, which reads backwards from the table and is now documented in-source, because getting it wrong yields a table that compiles and matches nothing. (2) The summary printed `'sweep'` as a LITERAL, so the one hint the command gave for returning to an earlier sweep -- `setplot sweep` -- always failed with ”no such plot named sweep”; it now prints the real name (`sweep3`), in BOTH summary branches (single-knob and the -vs/-family curve family), and the stale header comment in com_sweep.c documenting the plot as named `sweep` is corrected. The numbers are not 1,2,3 because plot_unique_typename draws from a counter SHARED by every plot type and advances it only far enough to avoid a collision -- a 3-point sweep burns op1/op2/op3 and lands on sweep3. That is pre-existing behaviour and is NOT changed; what changes is that you no longer have to guess. FOUND ALONGSIDE, LEFT OPEN: `print alle` is dead in every build configuration -- findvec_alle() sits behind `#if defined(XSPICE) && defined(SIMULATOR)` with a `#ifndef SIMULATOR` stub returning NULL beside it, and -DSIMULATOR is set only on ngspice_CPPFLAGS/libngspice_la_CPPFLAGS (main.c, ngspice.c, sharedspice.c) while every src/frontend source is compiled ONCE into libfte.la with AM_CPPFLAGS alone, which both the executable and the shared library link. Confirmed by inspection: the string ”DigitalData” is absent from the binary. Adding -DSIMULATOR to src/frontend/Makefile.am was tried and REVERTED -- ngproc2mod/ngmultidec/ngmakeidx also link libfte.la but not the XSPICE libraries, so enabling the block breaks them at link time under --enable-oldapps. The guard is unsatisfiable BY DESIGN, and a real fix means moving the event-node lookup into a simulator-only object. Two latent items behind that dead code are recorded so they are not re-discovered: findvec_alle overwrites the pl_typename plot_alloc just assigned with a hardcoded copy(”dig1”) (leaking it, leaving the registered completion keyword pointing at a name no plot has, and giving every digital plot the same name), and get_all_type has a misplaced paren -- `tolower(word[0] != 'a')` instead of `tolower(word[0]) != 'a'` -- defeating its own case-insensitivity, currently harmless only because callers lowercase first (verified: `print ALLV` and `print allv` behave identically). Verified by examples/sweepname_examples: 6/6 fixed vs 1/5 on the pre-fix binary (got ['unknown3'], no 'into plot' message). Regression 291/291.

E-368 *src/frontend/typesdef.c*

The periodic small-signal analyses named their plots wrong; E-367's claim that the naming was fixed was too broad. E-367 found its eight missing plot types by grepping for STRING LITERALS passed to plot_alloc(), which is blind to the path every standard and RF analysis uses: outitf.c calls `plot_alloc(run->type)` with a RUNTIME string handed to beginPlot(analName) by each analysis (mostly ckt->CKTcurJob->JOBname). Checking properly found seven more, four of them COLLIDING with an unrelated analysis -- worse than ”unknown”, because the name looks right. ft_plotabbrev returns the FIRST entry whose pattern is a substring of the plot name, and each of these contains a more general pattern: `PXF Analysis` matched nothing at all -> unknown; `PAC Analysis` contains ”ac” -> collided with ordinary AC; `PSP Analysis` contains ”sp” -> collided with S-parameters; `PNoise Analysis`, `qpnoise` and `phasenoise` all contain ”noise” -> collided with ordinary noise AND with each other; `Frequency Domain Periodic Steady State Analysis` (QPSS) contains ”periodic” -> collided with PSS. Each now has its own pattern placed BEFORE the general one its name contains. TWO ORDERING TRAPS: ”qpnoise” contains ”pnoise”, so the qpnoise entry must precede the pnoise entry; and PSS/QPSS differ only by their leading words (Time Domain vs Frequency Domain Periodic), so the QPSS pattern must carry ”frequency domain periodic” rather than the ”periodic” they share. Verified by examples/

periodicnames_examples: 9/9 fixed vs 4/9 pre-fix (pac->ac2, pxf->unknown2, pnoise->noise2, PSS/QPSS both pss). The FOUR REGRESSION CONTROLS -- ordinary noise, ac, tran, sp -- pass on BOTH binaries, which is what shows the change is targeted: those names are relied on elsewhere in the repo (setplot noise1, print noise1.onoise_spectrum in noisecorr/noisejw/stdaudit/tempphys/physcheck) and none moved; no deck that runs pnoise selects a plot by name either. HARNESS TRAP RECORDED: the `setplot` listing has a blank line immediately after its header, so a non-greedy regex reaching to the first blank line captures NOTHING -- an early harness reported "no plot" for all 25 analyses and called the run clean. SCOPE STATED HONESTLY: envelope/eye/loadpull use literals and were registered by E-367, so they are correct by construction but no deck here reaches those paths; `sens` produced no plot under every invocation tried, consistent with the already-open sens breakage and not a naming problem. Regression 292/292.

E-369 *src/spicelib/devices/vsrc/vsrcbindCSC.c*

Closes the E-365/366 stale-binding class. E-366 fixed two sites and left the third OPEN on purpose, because it could not be closed with another guard: the binding was NOT NULL but STALE, and no NULL test distinguishes those. THE ASYMMETRY IS THE BUG: `VSRCbindCSC` assigns the pole-zero binding only INSIDE `if (here->VSRCibrIbrPtr)`, and that pointer is allocated ONLY by a pole-zero analysis -- so on any later analysis the gate is false, the assignment never happens, and the binding keeps its previous value, pointing into the BindStruct that SMPdestroy( ) freed when the pz matrix went away. Both consumers (VSRCbindCSCComplex, VSRCbindCSCComplexToReal) then dereference it, because THEIR guard is `VSRCbranch != 0` -- a property of the device -- rather than "was this binding re-established for THIS matrix". ASan on `option klu ; pz ; ac`: heap-use-after-free READ of size 8 in VSRCbindCSCComplex, freed by SMPdestroy, reallocated by SMPconvertCOOtoCSC. FIX: clear the binding BEFORE the gate, so a stale value can never survive a matrix rebuild -- one line, fixing both consumers at once since both NULL-test it; the duplicated `(x != 0) && (x != 0)` copy-paste slip is collapsed in the same pass. NO OTHER DEVICE NEEDS THIS: vsrc is the only one that hand-writes a pz binding block (grep "Pole-Zero Analysis" across every *bindCSC.c returns one file); every other binding goes through the klu-binding.h macros, which are self-consistent -- CREATE and CONVERT are gated on the SAME `(a>0 && b>0)` condition, so a skipped create implies a skipped convert. HONEST LIMIT, stated in the doc and the example: the PRE-FIX binary PASSES every behavioural check -- the freed BindStruct entry still held the right CSC_Complex pointer, so the UAF read plausible values and the numbers came out correct, verified deliberately on a 40-node ladder with a `tran` between the pz and the ac to churn the heap. This is a MEMORY-SAFETY fix, not a wrong-answer fix; the behavioural checks are a regression guard, and the check that DISCRIMINATES runs under a sanitizer (the example runs it when NGSPICE_ASAN is set and SKIPS LOUDLY otherwise, so nobody mistakes a pass for proof). UB that happens to give the right answer today is still UB -- the same read against recycled memory is how E-365 presented, 2.5% wrong and silent. examples/klubind_examples 8/8. Regression 294/294.

E-370 *src/spicelib/devices/urc/urcinit.c*

Every **`.pz`** re-expanded the URC subcircuit, creating nodes past the allocated RHS. FOUND BY RUNNING AN EXISTING FIXTURE UNDER A SANITIZER: a sweep of all 185 shipped decks with each solver forced produced exactly one real finding, in ngcrashanalysis_examples/pz_urc.cir -- the E-315 fixture, passing ever since E-315 stopped it SIGSEGVing. Nothing had run it under a sanitizer; the crash was gone, the corruption underneath was not. ASan: heap-buffer-overflow READ of size 8 in RESload (CKTload->NIiter->CKTop->PZan), buffer allocated by NIreinit. RESload indexes ckt->CKTrhsOld by node number, so the read past the end means a resistor's node number exceeded the allocated RHS. CAUSE: urc declared `.DEVsetup = URCsetup` AND `.DEVpzSetup = URCsetup`, but URCsetup is not a matrix-entry allocator like its neighbours -- it is a SUBCIRCUIT EXPANDER (CKTmkVolt per lump, CKTcrtElt per element) with NO idempotency guard, and CKTpzSetup calls DEVpzSetup for every device on EVERY pz job. Each `.pz` therefore expanded the URC again, creating fresh internal nodes AFTER NIinit had sized

the RHS. That is exactly why RESsetup/CAPsetup are safe in the same slot and URCsetup is not: they only allocate matrix entries, which is idempotent. URC was the only expander wired up this way. Even a SINGLE .pz triggered it (CKTsetup expands once, CKTpzSetup expanded again); the overflow took a second pass to cross the allocation boundary. Visible symptom on an ordinary build: `doAnalyses: device already exists, existing one being used` + aborted run. FIX: `.DEVpzSetup = NULL`. The URC stamps nothing itself -- DEVload, DEVacLoad and DEVpzLoad are all NULL -- and the RES/CAP instances it creates are ordinary elements under their own device types, for which CKTpzSetup already calls RESsetup/CAPsetup; that is what actually re-binds the matrix. NOT A SOLVER BUG despite being found in a KLU/Sparse hunt: it reproduces identically under both. BEHAVIOURAL CHANGE, not hidden: on the E-315 fixture the fixed build gets FURTHER -- instead of aborting on a spurious duplicate device it runs the analysis and reports `singular matrix: check node 3`, which is CORRECT (that fixture's node 3 is genuinely floating); the duplicate expansion had been masking a real topology diagnostic behind a spurious one. verify_ngcrashanalysis.py still 4/4. examples/urcpz_examples 10/10 fixed vs 5/10 pre-fix. Regression 294/294.

E-371 *src/frontend/vectors.c, src/frontend/outitf.c, src/frontend/spec.c, src/frontend/postcoms.c, src/frontend/com_fft.c, src/frontend/rawfile.c*

Per-type plot numbering, and a date on every plot. Reported from user observation: a 500-point sweep produced `sweep500`, not `sweep1`, and the plot's Date field was empty. (1) NUMBERING. `plot_unique_typename()` counted up a SINGLE counter shared by every plot type. A `sweep` runs one analysis per point and KEEPS its plot (deliberate -- `-overlay` and waveform inspection rely on it), so a 500-point sweep created op1..op500, each collision pushing the shared counter, and by the time the sweep's own plot was named the counter stood at 500. The number looked like the point count; it was really "how many plots exist". Each abbreviation now carries its own counter, so the first sweep is `sweep1` whatever else ran. A tran/ac/tran/ac/noise/sweep/sweep deck goes from `sweep12 op12..op3 noise3` to `sweep2 op10..op1 noise2`; the two agree except in the pathological case where one type's plots number an unrelated type. WHAT HAD TO BE PRESERVED: E-345 exists because this path was QUADRATIC over a sweep (89% of a 64000-point sweep in plot_alloc->cieq->tolower), fixed by starting the search from the shared monotone counter plus a hash membership index. A per-type counter restarting at 1 each time would have reintroduced exactly that quadratic, so the counter is REMEMBERED per type (nghash on the lowercased abbreviation) and the search resumes from it -- E-345's timing check still passes at 22->21 us/point, ratio 0.93 over a 4x longer sweep. `destroy` HAS TO WALK IT BACK DOWN: `destroy all` must restart numbering at tran1, which examples/lifecycle_examples (E-81) pins -- and that is what CAUGHT the first version of this change. `destroy` unlinks plots one at a time through plot_forget() rather than plot_forget_all(), so plot_forget() now lowers that type's counter to the freed number. That also makes a single `destroy op2` recycle the number (the shared counter went on to op4), which is the behaviour `destroy all` always had, so it makes the two consistent rather than adding a rule. (2) DATES. Only the analysis path set one: outitf.c did `pl->pl_date = copy(datestring())` after plot_alloc, so every plot created DIRECTLY by a command -- sweep, hb, envelope, eye, loadpull, stb, rfstab, qpac -- left pl_date NULL and `print` rendered it as `(null)`. The stamp now happens in plot_alloc(), the single point where every plot is created, covering all eight command paths and any future caller. The five sites that set their own date became redundant and were removed, except rawfile.c, where a loaded rawfile's OWN date must win -- that one frees the stamp before replacing it, so centralising does not leak. VERIFIED by updating and extending examples/plotname_examples -- the E-345 example whose expectations this deliberately alters, and the file that documents this behaviour: 9/9 fixed vs 4/9 on a true pre-fix build (all six files reverted). The `tran`-has-a-date row PASSES on both, as the control showing the analysis path always had one and only the command paths changed. Regression 294/294.

E-372 *src/frontend/variable.c*

`unset plots` reported a bogus "Internal Error". Found by a PLOT-LIFECYCLE sequence fuzzer

-- random analyses interleaved with plot-management commands (destroy, setplot, unset plots, write/load, fft, linearize, remcirc) under ASan. That area was chosen deliberately: it is where E-342 (borrowed-pointer UAF via `unset plots`) and E-345 came from, and E-371 had just added a new allocation and new pointer arithmetic to it. Fired in 18/120 cases and minimised to ONE command on the SHIPPED binary: `unset plots` -> "Error: plots is read-only." followed by "cp_remvar: Internal Error: var 112". TWO DEFECTS in that one line. (1) THE MESSAGE PRINTED AN ASCII CODE, NOT A NAME: the signature is `cp_remvar(char *varname)` and the format was `"...var %d", *varname`, which dereferences the string to its FIRST CHARACTER -- 112 is 'p' for plots, and `unset curplot` reported 99, which is 'c'. The diagnostic never named the variable it complained about (verified against the ordinals). (2) IT WAS NOT AN INTERNAL ERROR: the branch is guarded by `if (*p)`, and `*p` non-NULL only means the variable WAS FOUND in one of the lists walked just above -- the normal state for `plots` and `curplot` -- so it fired on 100% of ordinary unsets of them. An internal error valid input reaches every time carries no signal. FIX: drop both spurious prints. For US_READONLY the "%s is read-only" message is already the complete answer and nothing is unlinked or freed there (correct for a read-only variable), so removing the noise changes no behaviour; for US_DONTRECORD the case's own comment says "Do nothing..." and its siblings curplotname/curplottitle/curplotdate were always silent, so curplot now matches them. The other `Internal Error` in the same function ("US val %d", i) is CORRECT and left alone -- i really is an int and that default branch is unreachable from valid input. CHECKED AS A CLASS: a grep for a %d conversion fed a dereferenced char* across the frontend returns no other occurrence. VERIFIED by examples/unsetvar_examples (no sanitizer needed): 9/9 fixed vs 5/9 pre-fix. The FOUR CONTROLS -- curplotname, curplottitle, curplotdate and an ordinary set/unset round-trip -- pass on BOTH binaries, which shows the change is confined to the two affected cases and did not break `unset` itself. Re-running the fuzzer against the fixed build: 400 iterations, 0 findings, 382/400 cases holding live plots at the end (a real clean result, not a harness that stopped working); harness committed under examples/unsetvar_examples/fuzz/ and excluded from the regression sweep. PROCESS NOTES RECORDED: the investigation began by checking whether E-371 had leaked (it added a datestring() allocation per plot) -- it had not, killplot() frees pl_date and peak RSS stayed flat 7152->7168 kB across 202->6060 plots created and destroyed. That leak check was FIRST attempted with ASAN_OPTIONS=detect_leaks=1 and came back clean MEANINGLESSLY, because macOS/arm64 answers "detect_leaks is not supported on this platform" -- a sanitizer that silently does not run is indistinguishable from a clean result unless you check. Regression 295/295.

E-373 *src/frontend/rawfile.c*

A rawfile write/load round trip lost the x-axis column and renamed the sweep axis. FRESH AXIS: E-226 fuzzed rawfile LOADING with malformed input looking for crashes; this asks instead whether a rawfile written by ngspice and then loaded by ngspice still holds the same data. Round-trip identity is a perfect oracle (no reference implementation needed) and catches silent fidelity loss a crash fuzzer cannot see; it also lands on freshly-changed code, since E-371 had just touched rawfile.c. Two independent defects, NEITHER of which corrupted a value. (1) `print` LOST THE X-AXIS COLUMN for every loaded plot -- `before write: Index v-sweep v(mid)` became `after load: Index v(mid)`, so nothing indicated which x-value each row belonged to. print prepends the scale column only when `pl_ndims` is non-zero (postcoms.c); outitf.c initialises it to 0 and sets it to 1 when analysis data arrives, but `pl_ndims` appeared NOWHERE in rawfile.c, so every loaded plot carried 0 and failed the gate. PROVEN NOT INFERRED: before writing the fix, a one-line probe setting pl_ndims=1 on load was inserted, shown to restore the column exactly, and reverted. THE `op` CASE IS THE CONTROL THAT MATTERS: pl_ndims=1 could have made print INVENT a column for a plot with no real scale (an op plot's variable 0 is just data, and the reader assigns pl_scale to variable 0 unconditionally) -- it does not, because print's inner `pl_scale && !vec_eq(bv, pl_scale)` test still suppresses it, and op prints the same header before and after; checked explicitly, since that is why the fix is safe. (2) THE `.dc` SWEEP AXIS WAS RENAMED `v-sweep` -> `v(v-sweep)` in the written file. The `v(...)` form means "the voltage

AT node X" -- right for a node probe, wrong for a sweep AXIS. A .dc plot's scale is the synthetic, voltage-typed vector `v-sweep`, which has no `v(` prefix, so the writer wrapped it. It did NOT compound (verified over three successive write/load passes: the prefix test leaves `v(v-sweep)` alone), so the damage was a one-time rename. Fix: add '

E-374 *src/frontend/trannoise/wallace.c, src/maths/misc/randnumb.c, src/main.c, src/sharedspice.c*

setseed did not seed TRANSIENT NOISE. Found by a correctness campaign over the command set: two identical decks with `setseed 42` produced DIFFERENT noise waveforms, while `setseed 42` then `rnd(100)` returned exactly 50.0 both times -- so the command worked, just not for the generator transient noise draws from. CAUSE: `#define WaGauss` selects the Wallace normal generator, and `initw()` opened with `srand((unsigned int) getpid()); // srand(17); TausSeed();`. initw() runs at STARTUP and fills its two pools from that getpid()-derived stream, and GaussWa draws from those pools; a later setseed reset srand and the Tausworthe state but nothing refilled pools that already existed. FIX, in three parts (moving the seeding alone is not enough): (1) initw() no longer calls srand(), so it seeds from whatever state the caller established and can never clobber a user seed; (2) the two startup call sites do the srand(getpid()) themselves, so an UNSEEDED run stays random per process; (3) com_sseed() calls destroy_wallace(); initw(); so a new seed actually rebuilds the pools. TRAP RECORDED: the first version of the com_sseed change was guarded `#if defined(WaGauss) && defined(SIMULATOR)`, and SIMULATOR is NOT defined for library sources like randnumb.c -- exactly the situation E-367 documented after finding `print alle` dead for the same reason. The whole fix compiled out silently and setseed still did not reproduce. Guard is now on WaGauss alone. That is twice this macro has disabled working code; treat `#ifdef SIMULATOR` outside main.c/ngspice.c/sharedspice.c as a bug until proven otherwise. VERIFIED by examples/setseed_examples: 5/5 fixed vs 4/5 pre-fix. Three properties, the third mattering as much as the first (a fix that pinned one constant sequence would satisfy the first and be useless): same seed -> byte-identical waveform; different seed -> different waveform; NO setseed -> still varies run to run. Two controls pass on BOTH binaries: rnd() was never broken, and the seeded stream is still real noise (3.5 mV peak-to-peak over 160 points), not a frozen constant. CONSEQUENCE: this falsified a claim in docs/internals/ngspice_internals/ngspice_transient_noise_analysis.md that setseed fixes the stream -- untrue when written, true now; the note was corrected and its seed-averaged statistics stand, having been means over genuinely independent runs. Regression 297/297.

E-377 *src/osdi/osdi.h, examples/simparamdiag_examples/**

OSDI diagnostics were unreadable and carried no severity. Found by the correctness campaign over all 94 `\$`-prefixed system functions, when a wrong `\$simparam\$str("analysis")` reported `OSDI(debug) n1: unknown \$simparam_stranalysisOSDI(debug) n1: unknown \$simparam_str...`. FOUR defects in one line. (1) NO SEPARATOR: `concat("unknown \$simparam_str", name)` joins with nothing, so the function name and its argument run together and the reader cannot see where one ends -- the numeric `\$simparam` path had the same bug; now `unknown \$simparam\$str "analysis"`, and reported with the `\$simparam\$str` spelling the user actually writes rather than the internal `\$simparam_str`. (2) NO NEWLINE: `osdi_log` writes `fprintf(dst, "%s", msg)` and no message carried a `\n`, so consecutive reports concatenated into one line -- which also HID the repetition, the old output looking like two lines where the fixed one shows 373 (the same 373 reports were always there). (3) WRONG SEVERITY FOR EVERY MESSAGE IN THE OSDI LAYER -- the one with reach beyond `\$simparam`: ngspice's `osdi.h` had `#define LOG_LVL_MASK 8`, but the level occupies the low THREE bits (DEBUG 0 .. FATAL 5) so the mask must be 7; 8 selects bit 3, which no level ever sets, so `lvl & LOG_LVL_MASK` was 0 = LOG_LVL_DEBUG for EVERY level. `\$display`, `\$info`, `\$warning`, `\$error` and every fatal diagnostic alike were labelled "OSDI(debug)" and written to STDOUT -- severity invisible to the user and to any log scraping, and `\$error`/`\$warning` never reaching stderr. OpenVAF's own copy of the header (osdi/header/osdi_0_4.h) has always said 7, so the two sides of the ABI disagreed on how to decode the level. Measured: before, all four severities print "OSDI(debug)"

on stdout; after, `OSDI n1:/OSDI(info) n1:` on stdout and `OSDI(warn) n1:/OSDI(err) n1:` on stderr. (4) A LEAK: the message was malloc'd, passed to `osdi_log` and never freed -- `free` was not even declared in the runtime's `NO_STD` extern block -- so a failing operating point leaked 373 allocations, not one; both call sites now go through one helper that frees. THE REPETITION IS FIXED BY E-378: making the messages visible exposed that there were 373 of them, left open here as "a convergence-path change" -- an understatement. `CKTop` read the fatal's `E_PANIC` return as "did not converge" and walked the gmin/source-stepping ladder, re-evaluating the device each pass, then blamed `timestep too small`. E-378 aborts the operating point on a fatal, so the report now appears once. Regression 302/302. Verify (examples/simparamdiag): 12 checks, a proven trigger -- pre-fix 1/12, and that single pass is a negative check that succeeds for the wrong reason on the old binary.

E-378 *src/spicelib/analysis/cktop.c, examples/opfatal_examples/**

A Verilog-A $fatal now ABORTS the operating point instead of driving the convergence ladder. `CKTop` reads any non-zero return from `NIiter` as "did not converge" and answers it by working through gmin stepping, source stepping, pseudo-transient continuation and optran. A Verilog-A fatal arrives through that same channel as `E_PANIC`, so the operating-point solver could not tell "this model refuses to evaluate" from "this circuit is hard to solve" -- and responded to a fatal by trying harder. Every aid re-evaluates every device, so the model re-raised the same fatal on each pass: measured at 373 messages for one device, 746 for two, 1119 for three -- exactly 373*N, one per device evaluation, 373 being how many evaluations the whole ladder performs before giving up. THE DAMAGING PART IS NOT THE VOLUME: the run then ended with `Error: Transient op failed, timestep too small`, naming CONVERGENCE, so a model with a typo'd $simparam name told the user their circuit would not converge while the real cause sat several hundred lines above. THE SAME GUARD ALREADY EXISTED IN THE OTHER PATH: E-55 added it to the transient time-stepping loop in dctran.c, with a comment that the error was otherwise "swallowed by the retry logic"; the operating-point path never got it (guards before this change: dctran.c 2, cktop.c 0). A `.tran` is affected too, since a transient computes its operating point first -- which is why the transient case still emitted 44 messages despite E-55's guard: it never reached the loop where that guard lives. THE FIX tests for `E_PANIC` after the plain Newton solve AND after each aid (a model may only fatal at a bias some later aid reaches), then reports the real cause. The test is EXACT rather than heuristic: `E_PANIC` is 1 and the non-convergence code `E_ITERLIM` is `E_PRIVATE+3` = 103, distinct values, so it cannot mistake a stalled Newton solve for a fatal. Result: $fatal in an .op 380 messages/"timestep too small" -> 1/"aborting"; unknown $simparam 373 -> 1; $fatal through .tran 44 -> 1. Regression 303/303. Verify (examples/opfatal): 8 checks, a proven trigger -- pre-fix 2/8, and the two that pass pre-fix are the ACCEPT checks (an ordinary circuit still solves; a circuit forced onto the aids with `.options noopiter` still converges through gmin stepping), which are there because a guard that aborted too eagerly would break every circuit that legitimately needs an aid.

E-380 *src/spicelib/analysis/dctrcurv.c, examples/dcstate_examples/**

A **`.dc`** sweep inherited integration coefficients from a preceding analysis -- a 45% SILENT wrong answer. `dioload.c` gates its charge branch on 'MODEDCTRANCURVE

E-381 *src/frontend/com_stb.c, examples/stbrestore_examples/**

`stb` handed its probe sources back ZEROED, so a following **`.ac`** returned all zeros. `stb` measures loop gain by injecting through two EXISTING sources -- it drives one with `ac = 1` while holding the other at `ac = 0`, then swaps them -- and when finished "restored the probes to quiescent" by setting BOTH to zero. Zero is only quiescent if that is where the probe started: a source carrying a real AC drive had that value destroyed, and any following `.ac` came back with EVERY node exactly 0.00000000e+00, with no warning and no error (`@v1[acmag]` 1.0 -> 0.0; `vm(mid)` 0.3333333333 -> 0.0000000000). FIX: read each probe's `acmag` before the first injection and write those exact values back, instead of forcing zero. Only the magnitude is saved, because the injection

writes `ac = N` which sets `acmag` and leaves `acphase` untouched -- verified rather than assumed, and the example asserts it so the assumption cannot rot. SCOPE is narrower than it first looks: in the documented usage the probes are dedicated injection sources sitting at `ac 0`, where restoring zero happens to be right. It becomes wrong when an already-driven source is named as the probe, which is legal and is what the fuzzer did (`stb V1 V2` with V1 the deck's own `ac 1` source), and nothing indicates the drive has been destroyed. FOUND BY cross-analysis STATE fuzzing with a NUMERIC oracle -- `result(B after A) == result(B alone)` -- the same instrument as E-380; re-running the 196-pair campaign against the E-380 binary left four mismatches and this was the only genuine one. The other three were `-> hb` deviations of 1e-17..1e-23 on near-zero harmonics whose RELATIVE deviation reached 30%, which looks alarming and means nothing -- the classic no-absolute-floor trap, the same one that produced a false 2.12e-07 "mismatch" in the metamorphic campaign; with atol=1e-12 all three are within tolerance. WHAT THIS CLOSES OUT: the chase began from an asymmetry in `dcpss.c` -- five `spSetComplex` calls and zero `spSetReal` -- which turned out NOT to be a defect. Matrix mode is managed per factorization by the SMP layer (`SMPcLUfac`/`SMPcReorder` set complex, `SMPluFac`/`SMPreorder` set real), so there is nothing to restore; `dcpss.c` sets it manually because it assembles G + jC by hand (spSetComplex -> CKTload for the real halves -> CKTacLoad for the imaginary halves -> factor), which is the intended pattern and the reason it is the only file in the tree calling `spSetComplex` directly. Regression 305/305. Verify (examples/stbrestore): 7 checks, a proven trigger -- pre-fix 4/7. The four that pass on BOTH binaries are the accept half (a probe that started at ac=0 still ends at ac=0; acphase survives; stb still reports a loop gain; stb run twice reports the same one), and stb's own answer is BIT-IDENTICAL across the fix (

E-382 *src/frontend/com_loadpull.c, examples/lprestore_examples/**

`loadpull` left the user's tuner at its last swept point. It sweeps the R, L and C of the user's own matching network across the Smith chart, and when finished left them wherever the last grid point put them -- the old code said so itself, `/* restore something sane on the load (last set values are fine) */`, a comment with no code under it. The last set values are not a result, merely where the loop stopped: RL 50 -> 84.83 ohm, LL 1e-15 -> 1.34e-8 H. Any following analysis then ran against the wrong network -- on the test deck an `.ac` moved from 0.4789 to 0.6765, a 41% error, silently. THE SAME CLASS AS E-381 (`stb` handing its probe sources back ZEROED): both commands borrow parts of the user's circuit, drive them, then guess at what "putting them back" means -- one guessed zero, the other guessed "wherever we stopped". `sweep` already had it right (E-350 captures each swept parameter's nominal value and restores it at cleanup) and this follows that precedent. Two neighbours were checked and are NOT defects: `optimize` leaves the best point because that is its deliverable, and `aging` writes back the accumulated dose because that is the whole command. A TRAP WORTH RECORDING, and the reason this row is precise: a `@name[param]` string built in C must be LOWER-CASED and fails SILENTLY if it is not. Two wrong explanations preceded this one, so what follows is what was MEASURED. (1) THE PARSER REJECTS UPPERCASE -- A/B at a single call site, same context, same instant: `raw=<@RL[resistance]> len=0` versus `lower=<@rl[resistance]> len=1`; `ft_getpnames_from_string("@RL[...]", TRUE)` returns NULL. (2) IT FAILS INSIDE `PPparse`, BEFORE `checkvalid` -- instrumenting both branches showed "PPparse FAILED", and that path is `return NULL` with NO diagnostic, so `if_getparam` (which would have printed "no such device or model name") is never reached; `checkvalid` does warn on a zero-length vector and was confirmed not to fire. (3) TYPING THE SAME TEXT WORKS because the command never sees uppercase -- instrumenting `com_print`'s wordlist showed it already holding `@rl[resistance]` when the user typed `@RL[resistance]`, so the fold happens BEFORE command dispatch. That is why print/alter/show/sweep all accept uppercase (none passes it on) while a string a command builds internally from its own argument words bypasses the fold. (4) NOT ESTABLISHED: where that pre-dispatch fold lives -- not `com_print`, not `ft_getpnames_quotes`, not a command-table flag, and not a blanket fold of `.control` lines, since unquoted `echo UNQUOTED MixedCaseWORD` preserves case. Left stated rather than guessed at. The first attempt blamed placement instead (reading before loadpull's priming transient), a red

herring settled only by instrumenting the actual code path. Regression 306/306. Verify (examples/lprestore): 7 checks, a proven trigger -- pre-fix 3/7. The four failures are the defect (R and L left moved, the 41% .ac shift, and source-pull mode leaving its own tuner at 70.69 ohm); the three passing on BOTH binaries are the accept half -- loadpull still reports a peak Pout (3.9576 dBm, identical across the fix), running it twice still reports the same optimum, and CL was already unchanged because the last grid point happened not to move it. examples/loadpull_examples passes 4/4 unchanged.

E-383 *src/frontend/typesdef.c, examples/plotorder_examples/*, examples/sweepname_examples/verify_sweepname.py, examples/plotname_examples/verify_plotname.py*

Four plot-type entries in **`plotabs[]`** that could never be reached, so four analyses' plots were named after a DIFFERENT analysis. `ft_plotabbrev()` returns the FIRST entry whose pattern is a SUBSTRING of the plot name, and E-367/E-368 both wrote that ordering rule into the source -- then E-367 broke it with its own entry: `envelope` went in at the BOTTOM of the table, below `{ "op", "op" }`, and "envel-OP-e" contains "op", so the entry was dead and every envelope plot came out `op1`. `setplot envelope1` found nothing, and the plot took a number out of the operating-point sequence. Auditing every plot name in the tree that reaches `plot_alloc()` -- the literals, the names `qp_emit_plot()` passes, and the descriptive `analName` strings arriving via `plot_alloc(run->type)` -- found three more of the same shape: `qpac->pac1` (collides with `.pac`), `qpxf->pxf1` (collides with `.pxf`), and `spectrum->sp1` (collides with `.sp`; BOTH `spec` and `fft` set pl_name to that literal, so both were affected). A COLLIDING name is worse than an unnamed one because it looks right: `.pac` plus `qpac` in one session gave `pac1`/`pac2` with nothing to tell them apart. `qpnoise` was already correct, since E-368 had placed it ahead of `pnoise`/`noise`. The `spectrum` fix is a deliberate change to two stock ngspice commands' plot names (`spec` and `fft`) -- `{ "spect", "spect" }` had been sitting in the table unreachable for exactly that purpose, and E-367's comment had recorded the result as an unfixable quirk. WHERE it goes is the constraint: the noise analysis names its plot "Noise SPECTral Density Curves ...", so one row too high would have renamed every ordinary noise plot and broken `setplot noise1` across this repo; it sits below `{ "noise", "noise" }`. E-345's example asserts the `fft` name directly and its expectation was updated WITH this change rather than around it -- the full regression is what surfaced that, which is why it is run before the fix is called contained. `plot_alloc("digi")` in `findvec_alle()` looks like the same defect and is deliberately NOT given an entry: the next line overwrites `pl_typename` with a hardcoded `"dig1"`, so an entry would be dead on arrival, and the path proved unreachable from every invocation tried (`print alle`, `plot alle`, event nodes confirmed via `eprint`) -- adding an untestable entry is the defect being removed. Since two of the four defects were introduced BY the enhancements fixing this table, the ordering invariant is now asserted against the table itself rather than left to deck coverage. `loadpull` leaving 17 `tran` plots behind was checked and is NOT a defect -- `sweep` keeps 334 `op` plots for the same documented reason. Verified by 25 checks (25/25 fixed, 12/25 pre-fix; the 11 accept checks pin `op`/`tran`/`ac`/`noise`/`sp`/`.pac`/`.pxf`/`qpnoise` and E-367's `sweep`/`eye`/`hb` on BOTH binaries).

E-384 *src/spicelib/devices/vsrc/vsrcpar.c, src/spicelib/devices/vsrc/vsrcset.c, src/spicelib/devices/vsrc/vsrctemp.c, src/spicelib/analysis/span.c, src/osdi/osdiregistry.c, src/include/ngspice/devdefs.h, src/spicelib/devices/dio/dio.c, src/spicelib/devices/mes/mes.c, examples/sensstate_examples/**

A transient after **`sens`** came back with EVERY NODE EXACTLY ZERO -- including the node the source drives -- plus five more defects around **`sens`** and the RF ports. THE BIG ONE IS NOT IN `sens`: `VSRCparam`'s `pwr` and `freq` cases set `here->VSRCfunctionType = PORT` UNCONDITIONALLY, and EVERY voltage source carries those parameters, not just ports. `sens` perturbs every settable real parameter of every device, so it wrote `pwr`/`freq` on ordinary sources, flipped them to PORT (power 0), restored the VALUE afterwards and never the function type -- the deck's SIN was gone for the rest of the session. `alter @v1[pwr]=0` did the same damage in one line with no `sens` involved. Blast radius measured before the fix: tran and envelope ZEROED, hb/pss failing with ‘

E-385 *src/spicelib/devices/vccs/vccsset.c, src/spicelib/devices/cccs/cccsset.c, src/spicelib/devices/vsrc/vsrcpar.c, src/spicelib/devices/vsrc/vsrctemp.c, src/frontend/com_sweep.c, examples/staterestore_examples/**

Three more commands that left the user's circuit changed behind them -- found by a STATE-RESTORATION AUDIT that goes after the CLASS rather than instances (E-380 .dc coefficients, E-381 stb probes, E-382 loadpull tuner, E-384 sens/PORT were the first four). Oracle: for every (instance, parameter) pair, the value after a command must equal the value before. (A) `sens ... ac` KILLED VCCS AND CCCS SOURCES -- `@g1[gain]` 1e-3 -> 0 and a following `.ac` returned 0.0 where the answer is 1.0. Not a bug in sens: `VCCSparam` folds the multiplier into the coefficient when `gain` is written (`if (VCCSmGiven) VCCScoeff *= VCCSmValue`) and VCCSmValue was never defaulted to 1 -- `res` does exactly that in ressetup.c. `sens` perturbs every settable real parameter, so it wrote `m` (setting mGiven), read it back as 0, wrote that 0 back as the restore, and the next `gain` write multiplied by zero. Explains the scope exactly: VCCS/CCCS have a settable `m`, VCVS/CCVS do not, and only the first two were hit; and why one frequency point was harmless while three were fatal. (B) `sweep` NEVER PUT AN `alter`/`altermod` KNOB BACK -- `@r1[resistance]` 2000 -> 2199, following op 3.8% wrong. E-350 covered only the `.param` path. Fix follows E-350's all-or-nothing rule; needs its own reader because `sw_eval_expr` returns 0.0 on failure, indistinguishable from a knob that is legitimately zero. Later follow-up: that reader evaluated the knob name as a VECTOR EXPRESSION, which only the `@dev[param]` spelling satisfies -- so a BARE device knob (`sweep V1 0 5 1`, `sweep R1 1k 5k 1k`) could be SET but never read back, and the sweep left the device at its LAST swept value: V1 stuck at 5 V and every later analysis in the session using it, R1 at 5k, I1 at 1 mA, with only a `could not be read` warning that names neither the knob nor the cost. The APPLY side never had the trouble -- `alter <dev>=<val>` resolves a NULL parameter to the device's IF_PRINCIPAL one -- so the reader now resolves that same principal keyword from the device tables and reads `@<dev>[<keyword>]` through the proven expression path, capturing by construction the value the sweep is about to overwrite. The E-437 shape once more: the guard existed and one spelling never reached it. A first attempt using `if_getparam(..., NULL, ...)` was WRONG and made it worse -- it returned V1's branch current (-5.01e-04) and R1 as 1e-12, and the restore then wrote those back; the before/after differential caught it. sweep-restore 13/13 (up from 5), the six new checks failing on the unfixed binary and the two controls passing on both. (C) AND E-350's OWN PATH RESTORED ONLY HALF OF WHAT IT MOVED -- it put the numparam table back, but on the E-320 FAST path the point loop pushes values straight into the live circuit, so devices kept the last point's values: `.param rl=3k` + `sweep rl lin 3 1k 5k` left `@r2[resistance]`=5000 and the next op 19% wrong. Nominals are now pushed back the same way each point was, before sw_fp_free() drops the binds. E-384's FIX COVERED ONLY HALF ITS CLASS: `if (!VSRCfuncTGiven)` left the DC-ONLY source unprotected (no waveform -> guard passes), so `sens` still turned it into a PORT and a following tran read 0.0 where the answer was 1.0. The decision moved to VSRCtemp, inside the block that already knows whether the instance is really a port; VSRCparam's pwr/freq cases no longer touch the waveform selector at all. E-384's own example passes either way because its deck carries a SIN -- the audit is what caught it. HARNESS TRAPS, both load-bearing: the BEFORE snapshot must be BOUNDED by the AFTER marker or it swallows the AFTER block and (building a dict) later values overwrite earlier ones -- before == after, EVERY command clean; caught only by a POSITIVE CONTROL against a known-defective binary. And the control must be a UNION of benign analyses subtracted PER PAIR, not `op` vs `op` (same analysis reproduces the same operating point) and never by NAME (`p` is settable on some devices, an output on others). LIMITATION stated: 114/292 pairs are operating-point dependent and subtracted, so a corruption of one of those is masked. ALSO OBSERVED, NOT FIXED: `sens_cplx` reads uninitialised memory (denormal garbage varying run to run). The campaign harness ships as examples/staterestore_examples/audit/audit_state.py (31 commands x 292 pairs), labelling each pair `[INPUT]/[output]` from the device tables. Verified by 20 checks (20/20 fixed, 9/20 pre-fix; the 8 accept checks pin an explicit `m=2` still doubling the gain, sens's own analytic numbers, VCVS/CCVS, plain tran/ac, E-350's `.param` restore, and bit-identical S-

parameters, on BOTH binaries; check 14 is an audit canary -- a deliberate `alter` the embedded audit must detect).

E-386 *src/spicelib/devices/{res,cap,ind,dio,bjt,vccs,vcvs,cccs,ccvs}/*ask.c, src/frontend/spiceif.c, examples/senscplx_examples/

The sensitivity queries returned the PREVIOUS query's value. `print @r1[resistance]` -> 1000, then `print @r1[sens_cplx]` -> 1000; `print @r1[i]` -> 5e-4, then `print @c1[sens_cplx]` -> 4.99999616e-04. Every `*_QUEST_SENS_*` case had the shape `if (ckt->CKTsenInfo) { ... write... } return(OK);` -- and CKTsenInfo is set only by the SENS2 analysis, which is not compiled in, so on any ordinary run the handler wrote NOTHING and still returned OK. The caller then read whatever was already in its IFvalue. FIRST SEEN AS UNINITIALISED MEMORY (denormal garbage 2.12736e-314 varying between runs) -- same defect in disguise: the frontend's IFvalue is a static reused by every query (spiceif.c, doask, which sets only pv.iValue), and sens_cplx is IF_COMPLEX, so a preceding REAL query leaves the imaginary half stale -- a double read of bytes that were never a double is exactly what a denormal like 2.1e-314 is. Interleaving queries makes it plain: the answer is the last thing you asked for. FIXED IN THE HANDLERS, not the caller, because two other callers pass an uninitialised STACK IFvalue: `dctrcurv.c` (which saves the result as a parameter's NOMINAL to restore after a `.dc` sweep) and `cktsens.c`'s sens_getp (which feeds sgen the value it will write back). Both latent today -- they only ask for parameters whose handlers do write -- but a contract of "returns OK, may not have written" cannot be made safe downstream. All 60 cases initialise their output before the `if`: 10 device types (res, cap, ind, mutual, dio, bjt, vccs, vcvs, cccs, ccvs) x six queries (sens_dc/sens_real/sens_imag/sens_mag/sens_ph/sens_cplx). Zero is what the surrounding code already chose -- the MAG and PH cases explicitly set 0 when the response magnitude is zero. `doask` hardened as well (memset before use; pv.iValue set AFTER, since it is an INPUT). SCOPE MEASURED by querying every parameter of seven device types immediately after a sentinel: 44 returned the sentinel pre-fix, 2 after -- and those two are `@r1[r]`/`@r1[ac]`, genuine aliases of resistance that really are 1000. Verified by 8 checks (8/8 fixed, 6/8 pre-fix). One check passes on BOTH binaries deliberately: querying the sensitivity parameters FIRST, with no preceding query, already returned 0 because the static starts zeroed -- that is what made the defect easy to miss, since a probe asking only the suspect parameter sees nothing wrong. The 4 accept checks pin ordinary parameters (keyed by INSTANCE -- `gain` exists on both a VCCS and a VCVS and a name-keyed check silently kept only the last), operating-point readbacks asserted SELF-CONSISTENTLY as p = i^2 R rather than against a literal from a simpler deck, sens's own analytic numbers, and `show all : all`.

E-394 *src/frontend/inpcom.c, src/frontend/inp.c, src/osdi/osdiparam.c, src/osdi/osdisetup.c, src/osdi/osdiload.c, src/osdi/osdiinit.c, src/spicelib/devices/nport/{nport.c,nportparam.c,nportdefs.h}, examples/osdiplumb_examples/**

Six ngspice+OSDI plumbing defects; FIVE are the same shape -- it works for a BUILT-IN device and silently does not for an OSDI one. (1) THE SUBCIRCUIT MULTIPLIER NEVER REACHED AN OSDI DEVICE: `X1 a 0 sub m=3` scales a built-in R/diode inside the subckt exactly while an OSDI device contributed 1x -- in DC, AC, transient, thermal noise, flicker noise, charge AND S-parameters -- and `\$mfactor` read inside the model was always the device's OWN `m`, never multiplied by any enclosing X at any nesting depth. The cause is ONE CHARACTER: `inp_fix_subckt_multiplier` appends `m={m}` to each device line inside a multiplied subckt but skips lines whose first letter names a device with no multiplier, and `'n'` was in that skip list -- **`N`** is the OSDI dispatcher. PDKs wrap compact models in multiplied subckts, so this under-counted device area with NO diagnostic. `N` also hosts the native n-port, which has no multiplier and ERRORED on an unexpected `m=`, so nport now ACCEPTS `m` and reports a non-unity value rather than the multiplier being dropped in silence there too (applying a real multiplier would mean scaling a stateful convolution and its history terms -- no finding here concerns that device). (2) NESTED SUBCIRCUIT MULTIPLIERS DID NOT COMPOUND, for built-in devices as well -- only the OUTERMOST `m=` survived (2 around 3 gave 2x; 2x3x5 gave 2x, not 30x) because an existing

m= was multiplied ONLY in HSPICE compatibility mode and otherwise a second m={m} was appended and won. A subckt that DECLARES m as a parameter is a separate case and was always right (X's m binds it and must not also multiply) -- pinned in the accept half. (3) INSTANCE temp= WAS NOT CONVERTED CELSIUS->KELVIN: every built-in adds CONSTCtoK at parameter-set time (dioparam.c) and ngspice's own OSDI code acknowledges the convention where it hands tnom over as CKTnomTemp - CONSTCtoK, but the OSDI path stored the raw number and used it as the Kelvin device temperature -- temp=75 reached the model as $temperature=75 and $vt=6.5 mV instead of 30 mV, -2.5e+16 A on a VA diode where the correct answer is -4.85e-07 A; temp=0 made $vt EXACTLY ZERO so limexp(V/\$vt) divided by zero and the op failed outright; temp=-40 gave a NEGATIVE absolute temperature and a negative thermal voltage; and nothing warned. temp also STACKED with dtemp (75+10 meaning 85) instead of overriding -- it now overrides and prints restemp.c's own "Instance temperature specified, dtemp ignored", with the same sensitivity-analysis silence. .temp/.option temp/dtemp/temp sweeps were all correct throughout; only this path was raw. (4) .option scale NEVER REACHED AN OSDI MODEL: each built-in applies it in its own parameter setter (cp_getvar("scale")) and nothing scales an OSDI instance parameter -- nor can it, the OSDI ABI carries no units -- so the Verilog-A channel \$simparam("scale") is the answer, and real models ask for it (the EKV model in this project's own VA_TEST corpus has SIMPARSCAL \$simparam("scale",1.0)) while ngspice's ten-entry simparam table omitted it, leaving the model on 1.0 while a built-in MOSFET in the same netlist scaled l=2 -> 2e-6. shrink/imax/rthresh deliberately NOT added: ngspice has no such option, so the honest outcome is the model's own default. (5) .options savecurrents PRODUCED NOTHING FOR OSDI DEVICES -- @r1[i] appeared for a built-in resistor and the compact model beside it produced no current vector at all, @n1[i] did not exist, so the only way to see a terminal current was to EDIT THE MODEL. Every OSDI instance now answers to i_<terminal> and a two-terminal one also to the bare i that R/C/L use, taken from the device's own stamp into that node's KCL row (resistive residual + the integrated charge derivative in a transient) so it is EXACT, not a finite difference -- verified equal-and-opposite, KCL to 3e-16 on a 3-terminal device, and C*dV/dt in a transient. SCOPE BOUNDARY STATED: savecurrents is a TEXTUAL pre-pass and a model's terminal NAMES are not knowable there, so >2-terminal devices are read per terminal by name (.save @n1[i_d]) rather than auto-saved. (6) \$simparam\$str("analysis_name") CONTRADICTED analysis(): E-53 taught the ANALYSIS_* flags to consult the running job so an AC job's op phase reports ac, but the string channel stayed on the CKTmode-only derivation and OSDIfinalStep carried a THIRD derivation testing MODEAC without MODEINITSMSIG -- so inside ONE evaluation a plain op reported name=ac while analysis("ac") was false, and an AC job's op phase reported name=dc while analysis("ac") was true. The in-source comment claiming "the same convention as analysis()" had become FALSE. THE MISTAKE WORTH RECORDING: making the other two match OSDIload is the obvious repair and is WRONG -- examples/finalstep_examples pins that ac-qualified events stay SILENT in a plain op, and propagating OSDIload's derivation made final_ac fire there. OSDIload was the odd one out: it set CALC_REACT_JACOBIAN and ANALYSIS_AC together from 'MODEAC

E-395 *src/spicelib/parser/inpdpar.c, src/spicelib/parser/inpgmod.c*

Setting an OSDI parameter and its **aliasparam** to different values was accepted in silence and one spelling lost. An OSDI parameter and each of its alias names are registered by osdiinit.c as SEPARATE IFparm entries that all carry the SAME .id, so N1 a 0 md w=1 width=4 writes ONE slot twice; setting the same name twice does likewise. Both are now reported, on the instance line (INPdevParse) and on the model card (create_model), by tracking the ids written while parsing that one line or card. SCOPED to OSDI devices -- aliasparam is a Verilog-A construct, so no built-in device's parsing changes -- and deliberately placed in the DECK parser, because alter legitimately re-sets a parameter after the deck is read and must stay silent; an instance line overriding a model-card default is likewise a different thing and is not reported. THE MODEL-CARD MESSAGE DELIBERATELY DOES NOT NAME A WINNER, because the two channels on a card disagree: a MODEL parameter is written straight through so the LAST one on the card wins,

while an INSTANCE-parameter default is pushed onto `INPmodfast->defaults` with `wl_cons` and therefore replayed in REVERSE, so the FIRST one wins -- stating a rule would be wrong half the time; the instance-line message, which has no such split, does say the last value is used. The rest of E-395 is compiler-side and is recorded in the openvaf report. Verify (examples/langguard_ examples): 11 checks on this item alone -- 5 doubly-set forms reported, 6 legitimate forms silent AND giving the same number.

E-396 *src/osdi/osdiregistry.c, src/osdi/osdiinit.c*

`\$limit` with an unresolvable limiting function SEGFAULTED the simulator with ZERO output. `osdi_create_registry_entry` resolves the model's `OSDI_LIM_TABLE` against a fixed set -- `pnjlim` (2 extra args), `fetlim` (1), `limitlog` (1), `limvds` (0). On an unknown NAME or a mismatched ARITY it `printf`'d "ignoring..." and left `func_ptr` NULL; the compiled model then CALLED that pointer, so the next evaluation jumped to address zero. The warning never reached the user either: it went to STDOUT and the crash destroyed the buffer before it was flushed -- which is exactly why the symptom was a silent death rather than a warning followed by a crash. **`fetlim`** takes 1 extra argument here and 2 in the LRM (**`vto, vgst`**), so a model following the standard was one of the ways to crash. There is no safe continuation -- the `\$limit` call site is already compiled to an indirect call -- so an unresolvable entry is now a HARD LOAD FAILURE (`INVALID_OBJECT`) with a message naming the function, the arity given, the arity expected and the supported set, written to STDERR where a later abort cannot swallow it. A collision warning fired on almost the whole industry corpus. E-335's check compared KEYWORDS only, but a model that declares one of the loader's own names has two IFparm entries under that keyword BY DESIGN: `osdi_create_registry_entry` routes the built-in to the model's parameter (`dtemp` -> `dt = param_id`, `m` -> `has_m`, `temp` suppresses the loader entry), so both address the SAME id and nothing is unreachable. `dtemp` is a conventional CMC instance parameter, so 69 warnings fired across the 124-file corpus (PSP, MEXTRAM, VBIC, BSIM, HiSIM, L-UTSOI, EKV, MVSG, ASM-HEMT, JUNCAP200, r2/r3_cmc), none of them a real problem, and each worded "declared more than once differing only in case" when the spellings are IDENTICAL and there is no second declaration to find. `osdi_warn_case_collisions` now skips a pair whose `.ids` MATCH; the corpus emits 5, all genuine different-id clashes. A finding was WITHDRAWN here: this began as "a model parameter named `m` silently defeats E-394's subcircuit multiplier" because such a model gave 1x under `X1 ... m=3` -- but the probe declared **`m`** and never USED it. A real CMC model declares `m` and scales by it, which is what `has_m` exists for, so the appended `m={m}` lands there and the multiplier is DELIVERED through it (verified 1x/3x/5x). The warning added for it was removed. The compiler-side half of this release is recorded in the openvaf report. Verify (examples/ limguard_examples): the crash check asserts a POSITIVE return code and a message naming the supported set, because the defect's signature was `rc = -11` with an EMPTY output stream -- a naive "did it fail?" test scores that as a pass. E-394's multiplier is re-pinned end to end (1x/3x/5x) THROUGH a model-declared `m`. Load sweep over the corpus: 107 loaded, 0 failures, 0 crashes, warnings 69 -> 5.

E-397 *src/osdi/osdiinit.c, src/osdi/osdiregistry.c, src/osdi/osdiparam.c, src/osdi/osdisetup.c, examples/ instknobs_examples/**

`temp`, **`dtemp`** and **`dt`** could be WRITTEN and never READ on an OSDI device. `osdiinit.c` registered the loader's own three entries `IF_REAL + IF_SET` with no `IF_ASK`, while a model's own parameters get both -- nothing in the source defends the asymmetry. So `print @n1[temp]` answered "no such parameter" where every built-in reports one (`@r1[temp]` -> 27, `@r1[dtemp]` -> 0), `show n1` listed none of the three, and -- worst -- a **`sweep`** over any of them ended with a spurious error AFTER completing correctly: the swept data was right and printed, then ngspice's end-of-run vector bookkeeping failed to resolve `@n1[temp]`. A diagnostic that fires on SUCCESS is worse than none; it cost this project a wrong answer (see below). It also made the three look as though they existed only when the Verilog-A declared them -- declaring one turns it into a MODEL parameter, which is askable -- when in fact ngspice supplies all three to every OSDI device by default.

WHY A FLAG WAS NOT ENOUGH: the ids COLLIDED. `osdiregistry.c` gave the synthesized knobs `num_params + num_opvars` and +1, which is exactly the base `osdiinit.c` gives E-394's synthesized TERMINAL CURRENTS -- **`dt`**'s id WAS terminal 0's id and **`temp`**'s was terminal 1's, survivable only because the two groups were disjoint by DIRECTION (temperature knobs set-only, terminal currents ask-only). Adding `IF_ASK` on top would have made `@n1[temp]` return a terminal current: a wrong number where there had been an honest error. FIX: the synthesized ids move ABOVE the terminal range (`+ num_terminals`), making the three id spaces disjoint; `OSDIparam` dispatches on `param == entry->dt` explicitly so the write side follows unchanged; `OSDIask` serves the two new ids, guarded on `>= cur_base` so a model that DECLARES `dtemp`/`temperature` (whose routed id is BELOW that base) still falls through to the ordinary readable-parameter path. CONVENTION MEASURED, NOT ASSUMED: a built-in resistor reports `temp` as the BASE temperature in DEGREES CELSIUS, following `.temp`/`.option temp` when the instance does not set it and NEVER including `dtemp`; `dtemp` is the offset, 0 when unset. An OSDI device now answers identically in all seven cases tested. ONE BEHAVIOUR CHANGE, not merely reporting: `temp=` overrides `dtemp=` (E-394 established that and prints the same message restemp.c does) -- but `restemp.c` also FORCES `RESdtemp = 0`, which was invisible while `dtemp` could not be read. Leaving the written value would now report an offset that has no effect, so it is cleared at both `osdisetup.c` sites; the device temperature is unchanged (`temp=75 dtemp=10` is 348.15 K before and after). THE WRONG ANSWER THIS COST, RECORDED: asked whether `sweep` works with these knobs, this project answered no, on the strength of that trailing error -- the sweep had completed correctly and its five points were printed immediately above the message. The lesson is not "read more carefully" but that a diagnostic emitted after a successful operation will be read as a failure of it, by tools and people alike. Verify (examples/instknobs_examples): 127/127 fixed, 92/127 shipped -- 35 checks pin this. Every read-back case is checked twice, against the expected value AND against a built-in resistor in the same deck. The suite re-pins what the id move could have broken: E-394's terminal currents still resolve on a 3-terminal device and satisfy KCL to 1e-12 with `@n1[temp]` readable alongside; the 2-terminal bare `i` alias still resolves; a model-declared `dtemp` is still read from the model's own parameter; the physics is unmoved; and `sweep` over each of the four knobs yields the right point count with NO diagnostic. Corpus load sweep 107 loaded / 0 failures / 5 warnings -- identical to what E-396 left. Full regression 321/321.

E-399 *ngspice-46/src/spicelib/parser/inpgval.c, ngspice-46/src/spicelib/analysis/cktop.c, ngspice-46/src/osdi/osdiload.c*

Three ngspice-side defects from round 13: a value that depended on which door it came through, a diagnostic that stated the opposite of the truth, and three answerable $simparam names. (1) AN INTEGER PARAMETER ROUNDED DIFFERENTLY FROM THE NETLIST THAN IN-MODEL: `INPgetValue` converted with `floor(0.5 + x)`, rounding .5 toward +infinity, while Verilog-A's own conversion inside a model rounds half AWAY FROM ZERO per the LRM. `.model mm dut(n=-2.5)` therefore gave -2 where `ii = -2.5` gave -3, and EVERY negative half-boundary disagreed (-0.5 -> 0 vs -1, -1.5, -2.5, -3.5) while positives always agreed. Now `round()`, exactly the LRM rule. This is a GENERIC parser path shared by every device; the two forms differ only at exact negative half-integers, where no built-in uses a meaningful value and the old answer was the surprising one, and the full 322/322 regression confirms nothing depended on it. (2) A PLAIN CONVERGENCE FAILURE WAS ANNOUNCED AS A MODEL ABORT: CKTop's ordinary 'every convergence aid failed' exit FELL THROUGH into the `fatal:` label, so a model containing no $fatal at all was reported with 'a Verilog-A device raised $fatal ... This is not a convergence failure -- see the OSDI(fatal) message above', with no OSDI(fatal) line anywhere above to read. Both claims false, and the text denied the true cause -- a singular matrix that had just defeated gmin stepping, source stepping and optran, each of which printed its own correct diagnostic immediately above. E-378 added that block for a genuine device-raised abort and it landed in the fall-through path of the normal failure route; the path returns `converged` now, leaving E-378's own `goto fatal` jumps as the only way in. Verified both directions. (3) THREE MORE STANDARD $simparam NAMES -- `iteration` (STATnumIter), `abstime` (CKTtime), `simulatorSubversion` (0) -- the ones ngspice

can answer truthfully. `shrink`, `imax`, `imelt`, `rthresh` stay out, keeping E-394's stated reasoning. What made the absence matter: an unknown name is not a soft miss, `\$simparam("iteration")` with no default aborts the ENTIRE run.

E-401 *src/include/ngspice/osdiitf.h, src/osdi/osdiregistry.c, src/osdi/osdisetup.c*

`V(a,b) <+ 0` between two module TERMINALS connected nothing at all -- a silent OPEN CIRCUIT where the model wrote a short. The LRM's own page-155 `parares`, whose purpose is to degenerate to a wire, gave `i(v1) = 0.0` at r=1e-6 against -5.0e-4 for a real short, while r=1e-2 conducted correctly: crossing the model's own threshold flipped it from wire to open. This project's `diode_lim.va` test model read 1000 V of self-heating rise at its DEFAULT `rth=0`, where it ties its thermal terminal to ground. TWO CAUSES, NEITHER WRONG ALONE: `collapse_nodes` skips any collapse pair whose endpoints are simulator-allocated (*"Terminals can never be collapsed in ngspice because they are allocated by ngspice instead of OSDI"*), and the compiler's DAE build emits no equation for a trivial potential contribution -- so the collapse was dropped and nothing replaced it. THE COMPILER-ONLY FIX IS WRONG AND THE REGRESSION PROVES IT: emitting the 0 V source and stopping there passes every targeted case and `cargo test` 210/0, then breaks FIVE regression examples, because a 0 V source between two nodes the NETLIST has already shorted is SINGULAR -- node collapsing is idempotent, an equation is not (HICUM with its thermal terminal grounded, the normal wiring: *"Warning: singular matrix: check node n1#xf1"*). Only the simulator knows the netlist, so the fix is two-sided: the compiler emits the source AND lists the branch in two new ADDITIVE exported symbols (`OSDI_TERM_SHORT_COUNTS/_INFOS`, the absdelay/last_crossing pattern -- no descriptor layout change, no ABI version bump), and ngspice drops the branch current when the terminals do not resolve to two distinct connected circuit nodes. Recorded only for branches whose current the model never reads, which is what makes dropping it safe. Terminal-to-INTERNAL collapse is untouched and still exact (BSIM4's `V(s,si) <+ 0`), as is the op-dependent switch branch (the LRM page-114 `relay`). VERIFIED: regression 322/322 (the five failures gone), `cargo test --workspace` 210/0 with six snapshots regenerated, corpus 107 compiled by both, 0 rc differences, 20 byte differences -- the FIRST release in this run to change emitted bytes, all 20 thermal models (HICUM L0/L2, HiSIM-HV/SOI, BSIM-BULK/CMG/IMG, ASM-HEMT, MVSG, MOSVAR) that ground a thermal terminal, correlating exactly with the new metadata (changed models export `TERM_SHORT` symbols, the 87 unchanged ones export none). Structure changes, answers do not: HICUM L2 on a normally-wired circuit gives -1.86927e-02 on both binaries, self-heating off and on.

E-402 *src/spicelib/parser/inp2n.c*

An OSDI instance line with FEWER nodes than the model has terminals was accepted in silence, while every built-in rejects the same mistake (diode *"could not find a valid modelname"*, resistor *"is not a valid resistor instance line, ignored!"*, MOSFET *"Simulation interrupted"*). A 4-terminal device given 3 nodes still simulates and answers differently -- `v(g)` 1.333 fully connected against 2.000 mistyped, rc=0, empty diagnostic stream -- because the omitted pins DANGLE and every branch touching them goes dead. ROOT CAUSE: `inp2n.c` bounded the count on one side only (`numnodes > *dev->terms` errors; too few was never asked about), then bound the missing terminals to `-1`. REJECTING IT WOULD BE WRONG: that `-1` is the same sentinel `osdisetup.c` reads as "deliberately unconnected", i.e. the LRM optional-terminal feature 32 corpus models query via `\$port_connected`; and a dangling node is no substitute, since it reports `\$port_connected == 1` where an omission reports 0. The lower bound therefore WARNS, naming each absent terminal from `dev->termNames` and spelling out that they are NOT grounded -- the natural assumption, and wrong by 67% on the measured node. Validated against BSIMSOI compiled with `PORT_CONNECTED` and instantiated with 4 of 7 terminals: the three it names (`p`, `b`, `t`) are exactly the optional ones its own logic is written for, and that case keeps working. Semantics unchanged; regression 322/322 with the warning firing 0 times across the suite, so every future firing is signal.

E-403 *src/spicelib/devices/res/resnoise.c, src/spicelib/devices/bjt/bjtnoise.c, src/spicelib/devices/dio/*

dionoise.c, src/spicelib/devices/vbic/vbicnoise.c, src/spicelib/devices/hicum2/hicum2noise.c, openvaf/hir_ty/src/inference.rs

Writing **`temp=`** equal to the ambient inflated an instance's thermal noise power by 9%. `rd x 0 1k` and `rd x 0 1k temp=27` are the same circuit at the same temperature, yet read 4.069337e-09 and 4.248250e-09. Five device noise routines computed the delta handed to `NevalSrcInstanceTemp` (which evaluates `4k(CKTtemp + param2)G`, so `param2` is a DIFFERENCE) as `inst->Xtemp - ckt->CKTtemp + (model->Xtnom - CONSTCtoK)` -- the third term adding the NOMINAL temperature in CELSIUS, 27 by default, to a difference. Instance temperatures are stored in Kelvin so the first two terms were the whole answer; the third is dropped in res, bjt, dio, vbic and hicum2 alike. HOW IT SURFACED: a bug hunt found `sens` silently changing every later `noise` in the session (2.878894e-09 alone against 3.005592e-09 after `sens`, sticky, no analysis clearing it). The route to the cause matters because the symptom pointed away from it -- the shift was EXACTLY constant at 1.08996 in power across resistor values and topologies (so a global scale, not a perturbation); `onoise` and `inoise` scaled equally (so the sources changed, not the transfer function); 1.089955 = 327.15/300.15 = +27 K, and repeating at 100 C and -50 C gave +27.00 K EVERY time (a constant, not a scaling); and an OSDI device's own `white_noise` was unaffected at ratio 1.00000 while the built-in resistor beside it shifted, which located the fault in the built-in device rather than in `ckt->CKTtemp`. `sens` restores parameter VALUES but leaves `...tempGiven` set, flipping the noise routine into that branch; correcting the arithmetic fixes both, since an instance at ambient then contributes zero either way. ALSO (compiler side): a type mismatch on `ac_stim` reported *"expected string literal, string literal or string literal"* -- `hir_ty` collected the requirement at the failing argument from every surviving CANDIDATE SIGNATURE and joined the list verbatim, and three of `AC_STIM`'s four signatures take a string there; the list is now deduplicated, leaving `\$limit`'s genuinely distinct *"string literal or function"* untouched. DELIBERATELY NOT SHIPPED: a save/restore for `CKTnumStates`, which `cktsens.c` overwrites with one device's state offset and never restores -- it was written, changed no measured result, and was reverted, because a fix must be no wider than its evidence. Regression 322/322, `cargo test --workspace` 210/0, corpus 107 compiled by both, 0 rc differences, 0 byte differences.

E-408 *src/frontend/inpcom.c, src/frontend/parse.c, src/frontend/vectors.c, src/frontend/device.c, src/spicelib/analysis/dctrcurv.c*

Three ways a bracketed name missed what it pointed at. (1) A LEADING ZERO split a node in two: `n[1]`, `n[01]` and `n[001]` were three distinct nodes -- `print all` listed all three -- while the vector lookup canonicalised the index first, so every probe returned the `n[1]` value (1.0 against the 2 V and 3 V actually driven). With one node it is worse: a netlist that built `m[02]` was unreachable by EVERY spelling including its own, since the lookup only ever asked for `m[2]`. E-221's own header settles it (the scalar names a bus produces use the same bracket form, so `a[0:1]` and `a[0]` denote one node), so the netlist now canonicalises the index, negatives included, as a byte-identical no-op when already canonical; a contradictory deck that used to answer 1.0 now correctly reports a singular matrix. (2) `@dev[param]` COULD NOT NAME ANY BRACKETED PARAMETER: `show` lists the bus terminal currents `i_a[0]..i_a[3]` (E-394) and array parameter elements `ap[0]..ap[2]` with correct values and the instance line can set them, but four separate splitters each cut the name at the INNER `]` -- *no such parameter i_a[0.* -- so read failed, `alter` failed SILENTLY (the current stayed put) and a `dc` sweep was a FATAL ERROR, with no workaround: the name the simulator prints was not one it accepts back. All four had to change because each serves a different command (`PPlex` lexes the token at all, `vec_get` serves print/let, `com_alter` serves alter, `DCTfindInstParam` serves E-62's sweep). E-269's wildcard alias `@*[[param]]` works BECAUSE of the old first-`]` split, so a name beginning with `[` deliberately keeps it -- all four wildcard spellings measured unchanged and `wildparam` still passes. (3) A `[lo:hi]` BUS RANGE expanded on instance lines, R/C node lists, subcircuit calls and port lists but NOT on the cards that name nodes rather than connect them: `.save`/`.print`/`.plot` produced nothing and `.ic`/`.nodeset` warned and ignored. Those were skipped by design -- `inp_expand_buses` admits lines carrying BARE

node fields and an output card writes `v(a[0:3])` -- so a wrapper-aware form emits one copy of the whole token per index, carrying the wrapper and any `=value`; the spaced `.ic v(a[0:1]) = 0.25` absorbs its value into the token first. The four expanded `.print` columns check against the resistor dividers (0.5, 1/3, 0.2, 1/9), not merely against each other. DELIBERATELY UNCHANGED: a token naming two ranges has no unambiguous expansion and stays literal; and one unparseable probe voiding its whole `.print` card is general behaviour for any unresolvable probe, no longer reached by the bus case, and measured to stop at exactly one card. Compiler untouched. New example busname_examples 30/30; regression 325/325.

E-409 *src/frontend/com_sweep.c, src/frontend/spiceif.c, src/include/ngspice/fteext.h*

A wildcard **`sweep`** knob was never put back, and said so in a way that looked like a broken parameter name. `sweep @*[wavelength_nm] ...` ran correctly and then printed *Error: no such device or model name / PPerror: syntax error in line segment* @[wavelength_nm] near wavelength_nm]* -- the stray `]` because the expression lexer's `specials` set contains `*`, so `@*[p]` cannot lex as ONE token and the parser reports the leftover text. THE MESSAGE WAS THE HARMLESS PART: `sw_read_knob()` is how E-385 captures the nominal it restores at the end of a sweep, it parses the knob name as an EXPRESSION, and that call can never succeed for a wildcard -- so the read reported failure and E-385's deliberate ALL-OR-NOTHING rule skipped restoring EVERYTHING. Measured: `@*[wavelength]` left two model cards that started at 2.0 and 9.0 both sitting at 3.0; `@#*[scale]` and the E-269 alias `@*[[scale]]` the same; and with `-vs` it took a CONCRETE co-knob down with it (`@dev1[wavelength]` left at 3.0 although it restores perfectly alone), so any following analysis silently ran against the last swept point. ONE NUMBER COULD NOT FIX IT: a wildcard sets every matching model or instance to one value but the values it overwrites all differ, so undoing it needs one reading PER TARGET. New `if_saveparam_wildcard()`/`if_restoreparam_wildcard()` walk exactly the same targets in exactly the same order as the existing `if_setparam_wildcard{,_instance}` (same device-type loop, same model and instance chains, same `parmlookup(.., inout=set)` predicate), reading through the parameter's askable twin and the same `doask`/`doset` pair a plain alter/altermod uses; saving stays all-or-nothing (a set-only or vector-valued parameter refuses the whole capture) and the replay is pinned to the circuit the readings came from, so a `.param` co-knob that re-sources the deck cannot have them replayed onto a different one. `sw_read_knob()` now recognises the three wildcard spellings and returns without parsing, which removes the diagnostic. WHY IT SURVIVED THE AUDIT THAT EXISTS FOR THIS CLASS: E-385's own oracle `staterestore_examples/audit/audit_state.py` sweeps `@r1[resistance]`, a CONCRETE knob, and reports `sweep` clean on the DEFECTIVE binary too -- no wildcard form was ever exercised; the audit is unchanged and still reports the same four known offenders (hb, pz, sens, sens_ac) on both binaries. New example wildrestore_examples 24/24, and 14/24 on the pre-409 binary, so it detects the defect rather than describing it. Compiler untouched; regression 326/326.

E-410 *src/frontend/spiceif.c, src/frontend/gens.c, src/frontend/com_sweep.c, src/spicelib/analysis/dctrcurv.c, src/include/ngspice/fteext.h*

A device inside a subcircuit could not be named the way the node beside it is. `@r.x1.r1[resistance]` worked; `@x1.r1[resistance]` -- the obvious spelling, and the one that matches the node `x1.m` -- named nothing. THE TYPE LETTER IS NOT AN ACCESSOR CONVENTION: `r.x1.r1` is the device's real name, because ngspice flattens a subcircuit by rewriting its cards back into the deck and re-parsing them as ordinary element lines, and the parser takes the device type from the FIRST CHARACTER of the card (`inppas2.c`: `c = *(current->line); switch (c)`) -- the card `x1.r1 a m 1k` would be re-read as another SUBCIRCUIT CALL. Two details prove that is the reason rather than a plausible story: `translate_inst_name()` EXEMPTS `x` devices (`if (tolower_c(*name) != 'x')`) because a subcircuit instance already starts with the right letter, so `x2` in `x1` is plain `x1.x2`; and NODES, which have no type, keep plain hierarchical paths (`x1.m`, `x1.x2.q`) -- which is exactly the asymmetry. THE RECONSTRUCTION NEEDS NO SEARCH: the letter prepended is literally the leaf name's own first character (`bxx_`

putc(buffer, *name)) and a device name must already begin with its type letter, so x1.r1 can only mean r.x1.r1; two device types cannot share a leaf name. New if_find_instance_hier() is consulted ONLY after the exact lookups fail, so every name that resolves today resolves to exactly what it does today. Following E-408's lesson that @dev[param] is resolved in several independent places, every consumer is routed through it: finddev() (print/let/alter/altermod), finddev_special(), DCTfindInstParam() for E-62's .dc @inst[param] sweep, and sw_fp_bind() for the sweep knob -- where the old behaviour was NOT EVEN AN ERROR: the knob silently failed to bind and the three swept points came out IDENTICAL. show needed its own answer, since its grammar takes the query's first character as the device-TYPE selector (type = *word++) and uses :/# rather than . as the subcircuit delimiter, so x1.r1 parsed as *type x*; dgen_hier_match() is consulted ALONGSIDE that grammar as a whole-word alternative and can only add a match (a first attempt widened the strcmp inside the decomposition instead and was measured as DEAD CODE, then reverted). BACKWARD COMPATIBILITY DIFFED, NOT ASSUMED: across 17 show forms 14 are byte-identical and the only 3 that changed are the new spellings; legacy :r1/#dmod/showmod/-v unchanged; an x leaf gets no phantom prefix; bogus names and bogus parameters still reported; hierarchical NODES untouched. E-49's hiername_examples is a different subject (hierarchical names INSIDE Verilog-A) and is unaffected. New example hierdev_examples 32/32, and 15/32 on the pre-410 binary. Compiler untouched; regression 327/327.

E-411 *src/frontend/inpcom.c*

A descending netlist bus range binds nodes to a device's terminals in REVERSE order, and was silent. nd1 d[3:0] 0 mm against a model declaring inout [3:0] a compiles clean, simulates clean and is wired backwards -- and it is the spelling that LOOKS consistent, since both sides say [3:0]. TWO CONVENTIONS DISAGREE: the COMPILER declares a bus port's terminals in ASCENDING bit order whatever direction is written (hir_def item_tree lowering sorts the endpoints and loops upward -- *declare from lsb to msb (ascending), matching natural bit order; direction of the original [msb:lsb] only affects range checks*), so [3:0] and [0:3] declarations give the SAME positional terminal list; while the NETLIST does honour direction, E-221 expanding d[3:0] to d[3] d[2] d[1] d[0]. The instance line's Nth node binds the model's Nth terminal, so a descending list reverses the mapping. MEASURED with one conductance per bit (a[k] -> (k+1) mS, so a node's current names the bit it landed on): wired d[0:3] the nodes read 1,2,3,4 mS, wired d[3:0] they read 4,3,2,1 -- and the [3:0] and [0:3] model rows are IDENTICAL, confirming the declaration contributes nothing. inp_expand_bus_token() now reports msb-first expansion and inp_expand_buses() warns on element instance lines only. SCOPE CAME FROM SCANNING THE SUITE, not taste: a .subckt descending PORT list is a deliberate interface choice E-221 documents and busnodes_examples tests; .save/.print/.ic bind nothing; and a range too wide to expand never binds (E-338's guard), so warning there would be false. Verilog-A part-selects (v[3:2], .i(v[1:0])) are compiler-side and untouched. THE OPT-OUT WAS CORRECTED AFTER TESTING: set nobusdirwarn does NOT work inside .control, because bus ranges expand while the netlist is read; it works from .spiceinit, verified both ways, and the message says which. New example busdir_examples 20/20 (13/20 on the pre-411 binary), pinning the binding itself as well as the diagnostic. Regression 327/327 with the warning firing 0 times across the suite, so every future firing is signal. Compiler untouched; no binding changed, only a diagnostic added.

E-412 *src/osdi/osdiload.c*

An OSDI operating-point variable read after **.ac**/**.noise** returned the SMALL-SIGNAL solution, and CHANGED WITH FREQUENCY. An opvar defined as nothing but vbias = V(a,c) read 0.4939733 after op -- the true bias, from solving the circuit's KCL cubic by hand -- but 0.4823410 / 0.4819028 / 0.0473587 after .ac at 1 k / 10 k / 1 meg, a multi-point sweep leaving the LAST point's. An operating point cannot depend on frequency, which is what separates this from a tolerance argument: identical at reltol 1e-3, 1e-6 and 1e-10. CAUSE: E-53 fires @(final_step) by issuing one dedicated evaluation per instance when an analysis completes, at CKTrhsOld -- which

after a sweep holds the small-signal solution, not a bias. The model is genuinely RE-EVALUATED rather than mis-read, confirmed with opvars chosen to distinguish the two (`3V+1` and `V^2` matched the ac solution to nine digits while a constant opvar stayed 42). Built-ins are unaffected, having no such post-analysis evaluation -- the recurring *works for a built-in, silently not for OSDI* vein. NOT WRONG, established BEFORE fixing anything: the analyses themselves. A bias-dependent `white_noise(kf*V^2)` source integrated to `onoise_spectrum = 1.342573180366e-01`, matching the dc-bias prediction EXACTLY and not the ac-bias one (7.21e-02) -- so this is a READBACK defect, not a numerical one, and it is not sticky (any following op/dc/tran restores the correct value). THE EVALUATION CANNOT SIMPLY BE SKIPPED: it is the only thing that fires `@(final_step)`, and `@(final_step(\"ac\"))` plus the noise variant are supported and tested by finalstep_examples. `OSDIfinalStep` therefore snapshots the instance data around the call and restores it, for MODEAC and MODEACNOISE only -- event bodies still run and their `\$strobe` output stands, while what the evaluation wrote is discarded, which is correct because those results are by design never loaded into the matrix or RHS. DC, DC-sweep and transient are deliberately NOT snapshotted, their final solution being a real operating point (measured unchanged). WHY IT SURVIVED: with a purely resistive device the dc and ac solutions coincide, so the natural small test cannot see it; the example is reactive on purpose. New example opvarac_examples 19/19 (12/19 on the pre-412 binary); finalstep and simctrl pass unchanged; regression 329/329. Compiler untouched.

E-413 *src/frontend/breakp2.c, src/osdi/osdiparam.c, src/include/ngspice/osdiitf.h*

`.options savecurrents` registered an EMPTY vector for a multi-terminal OSDI device. It produced `@nd1[i] : current, real, 0 long` -- present in the listing, empty in use, with no diagnostic -- while a built-in BJT in the same deck produced four filled waveforms (`@q1[ic]/[ie]/[ib]/[is]`). CAUSE: `inp_savecurrents()` is a TEXTUAL pre-pass over the deck, running long before any `.osdi` is loaded, so it cannot know a compact model's terminal names and emits the same bare `@dev[i]` that R/C/L use -- E-394's own comment says exactly that. E-394 defines the bare `i` alias only for TWO-terminal devices, where it is unambiguous; for anything wider the save named a parameter that does not exist. THE VALUES WERE NEVER WRONG, only uncapturable: read as scalars after `op` the terminal currents were exact (1.000e-3 / 4.000e-3 / 1.200e-2 / -1.700e-2, KCL sum 0) and an explicit `.save @nd1[i_d]` always worked -- what was missing was the WAVEFORM, which is what savecurrents exists to provide. Since the names cannot be known during a textual pre-pass, the expansion moves to where they can: `ft_getSaves()` runs at analysis start with the circuit set up, so a bare `@dev[i]` belonging to an OSDI instance that does not define it is expanded there into one entry per terminal, through a new `OSDIterminalNames()`. The two-terminal case is deliberately untouched (`@dev[i]` is real there, byte-identical to the pre-413 binary) and built-ins never enter the path. Also explains why `meas tran ... MAX @nd1[i_a]` had quietly returned a scalar snapshot instead of a maximum over time. New example savecur_examples 15/15 (7/15 on the pre-413 binary); regression 330/330. Compiler untouched.

E-415 *src/osdi/osdinoise.c, src/spicelib/analysis/cktsgen.c, examples/evtnoise_examples/ (new)*

A per-device noise total that was the wrong quantity, and one knob counted twice. (1) `nVar(i,j)` indexed `noise_vals` with a stride of `num_noise_src`, but each of the NSTATVARS state variables needs one slot per source PLUS one for the whole-device total accumulated at index `num_noise_src` -- which is exactly what `osdiregistry.c` allocates (`NSTATVARS * (num_noise_src + 1)`). With `n` sources `nVar(OUTNOIZ, n)` = `noise_vals[1*n+n]` and `nVar(INNOIZ, 0)` = `noise_vals[2*n+0]` are THE SAME ADDRESS, so `onoise_total_<dev>` came out bit-identical to `inoise_total_<dev>_<first source>`: an input-referred, single-source number reported as the device's output-referred total (2.24x out in the measured case), while every built-in device's total matched its own components exactly. The spectra and the grand totals were never affected -- `onoise_spectrum` matched two real 1k resistors to 9e-08 relative and the grand total already equalled the quadrature of the per-SOURCE entries -- so the regression pins the spectrum bit-identically against the pre-415 binary rather than against a derived value. (2) `m` is registered as

an alias of `\$mfactor` with the SAME parameter id, so anything walking the parameter table saw the multiplier twice: `sens` listed both `<inst>_m` and `<inst>__mfactor` with identical non-zero sensitivities and summing the table double-counted it. IF_REDUNDANT -- the marking built-in devices give their alias keywords -- was tried and REJECTED by the regression: that flag also hides the keyword from listings, and `show n1` is required to list `m` (E-397 went 126/127). Marking the other spelling would invert the flag's meaning, since `spiceif.c` walks BACKWARDS from a redundant entry to find its principal. The de-duplication therefore happens where the repeat mattered: the sensitivity generator skips a parameter whose id an earlier keyword in the same table already claimed. `sens` lists it once, `show` still lists `m`, and `m` remains settable from an instance line, a `.model` card and `alter` (all three verified identical).

E-439 *src/maths/KLU/klusmp.c, examples/klusingular_examples/ (new)*

A SUCCESSFUL **`klu_refactor`** IS NOT NECESSARILY A USABLE ONE. KLU FAILED an operating point that SPARSE solves -- the midpoint of two series capacitors, a node with no DC path: SPARSE gave v(nb)=0.5 in 289 iterations, KLU produced NaN and burned 33,911 before giving up with 'timestep too small', naming neither the node nor the cause. Instrumentation settled it in one run after four wrong hypotheses. TWO facts, both necessary: (1) `LoadGmin_CSC` skips diagonal pointers that are NULL ('Not all the elements on the diagonal are present') and a node with no DC path has NO diagonal entry -- so the one row needing regularisation is the one row that cannot receive Gmin; (2) `klu_refactor` REUSES the pivot ordering from the last full `klu_factor` and only refills values -- no pivoting, no singularity test -- so once gmin stepping ramped to 1e-12 it filled the LU with zero/NaN pivots and returned `ret=1, status=KLU_OK`. NaN then neither converges nor trips a singularity check, so the Newton loop AND every rung of CKTop's ladder (dynamic gmin, static gmin, source stepping, pseudo-transient, optran) ran to its full iteration budget: 33,835 further non-finite solves with refactor reporting OK 33,191 times, and a count INVARIANT at 33,911 for any circuit size -- the signature of a fixed budget, not real work. SPARSE solves it because its refactor path DOES detect the zero pivot and forces a reorder. Fix: `klu_rcond` walks diag(U) and sets rcond=0/status=KLU_SINGULAR on the first zero or NaN pivot; calling it after a successful refactor and returning E_SINGULAR routes the caller into the reorder-and-factor-again path it already had -- the same recovery SPARSE performs, no new plumbing. Cost measured: +8-11% on the factor step, nothing measurable on total analysis time (0.105->0.105s, 0.418->0.416s, 1.489->1.454s at 500/2000/6000 nodes), iteration counts on healthy circuits identical (2032/2032, 2035/2035, 2037/2037). NOT fixed, deliberately: Gmin still cannot reach a row with no diagonal entry -- the CSC pattern is fixed at symbolic-analysis time; allocating a diagonal for every node would address the root at the cost of a denser pattern for every circuit, and has its own performance case to make. FOUR WRONG HYPOTHESES recorded in the write-up so they are not re-walked, the first of which (short-circuit CKTop on E_SINGULAR) would have BROKEN working decks, since gmin stepping legitimately rescues a floating node. Verified klusingular 16/16 (split closed, no NaN, rescue survives with each aid disabled, plus healthy-circuit and transient controls), op campaign 105/105 with 0 splits (was 3), regression 350/350 both solvers.

E-440 *src/spicelib/analysis/cktsens.c, src/spicelib/analysis/dcpss.c, src/spicelib/analysis/cktsopt.c, src/spicelib/parser/ptfuncs.c, src/spicelib/devices/asrc/asrcload.c, src/spicelib/devices/asrc/asrcdefs.h, src/frontend/options.c, src/frontend/parse.c, src/frontend/com_measure2.c, src/frontend/spiceif.c, examples/_setup.py, examples/sensrestore_examples/ (new), examples/argguard_examples/ (new)*

`sens` LEFT THE CIRCUIT ALTERED, AND FIVE INPUTS WENT UNCHECKED. Perturbing a parameter marks it GIVEN and nothing un-marks it, so a BJT fell into `bjttemp.c`'s `BJTBEsatCur-Given && BJTBCsatCurGiven` branch with `ibe`/`ibc` both 0, lost its junction saturation currents, and every later analysis in the session solved a dead transistor -- 12.8% wrong on one stage, 101% on a differential pair, silent, sticky, cured only by `reset`. Not a tolerance artifact (identical at reltol 1e-3 and 1e-12), not a stale guess (`.nodeset` did not move it; `sens` run FIRST corrupted too), and 1 of 20 commands -- `alter`, `altermod`, `sweep`, `montecarlo`, `optimize`, `tf`, `pz`, `disto`, `noise`, `hb`, `pss`, `sp` all restored perfectly. Bisecting the loop showed the damage came from `sens_setp`

setting a parameter to its own current value, and needed ibe AND ibc together -- which is the &&. Fix: snapshot every model struct before the loop, restore after (and on error paths), then CKT-temp(); generic via DEVmodSize, because the hazard is any given-flag branch, not the BJT. ALSO: pss validated none of its numeric arguments where tran/ac/dc/hb/sp/noise all do -- a negative stabtime sets CKTfinalTime behind the current time and pss 1k -1m 0 1024 5 50 1u ran past 100 s with no output (E-348 had guarded fguess/points/harmonics, not these three); .meas took an inverted window and, because m_to == 0.0 doubles as 'no upper limit', FROM=1m TO=0 measured [1m, end] and returned a confident number (the parser now records whether each bound was written; dc still normalises a downward window but says so); pow(0,-1) and 0**-1 returned a raw infinity where every other singular case is clamped to HUGE and reported (x/0, sqrt(-x), log(0), and pwr(0,-1) since E-256) -- an overflow like 1e300*1e300 is now reported but deliberately NOT clamped, since there is no defensible finite substitute for arbitrary user arithmetic; the temperature guard was one-sided, .temp 1e6 silently giving a diode divider -2.7e-15 V; set curplotname=x before the first analysis called FREE() on a string LITERAL (plot_cur is the static constantplot until an analysis runs) and killed ngspice with a heap abort, while unset ignored its isset flag and performed the assignment too; and PP_mkfnode() copied an unbounded function name into char buf[BSIZE_SP] with strcpy+sprintf, so fourier 1k <600-char-name>(v(a)) smashed the stack. Plus E-438's own warn_physics reported from dosim(), once per analysis RUN, repeating one unchanged warning 20 times for montecarlo 20 and 6 for a 6-point sweep; now once per circuit, keyed on the VALUE so a sweep through a bad range still reports each distinct one. And examples/_setup.py pointed SPICE_LIB_DIR at the committed bin/ bundle while preferring the LOCAL ngspice, so code-model edits were never exercised -- and with no SPICE_LIB_DIR at all ngspice loads a THIRD PARTY's models from its compiled-in prefix, which is how four 'crashes' (d_source, s_xfer, table2D, table3D) were attributed to this tree when they belong to an unrelated Feb-2025 /usr/local install and do not reproduce here at all. Verified sensrestore 16/16 (including that sens is still NUMERICALLY right: both signs of a divider's analytic 2.5e-4 V/ohm) and argguard 44/44, both solvers; regression 352/352.

E-441 *src/frontend/inpcom.c, src/frontend/parse.c, src/frontend/vectors.c, src/frontend/device.c, src/include/ngspice/fteext.h, examples/arrayinst_examples/ (new), examples/xspicemodel_examples/*

ARRAY INSTANCES. A range on the instance NAME makes an array of devices: R[0:3] a b r=1k is four resistors in parallel, N[0:3] a[0:3] b model gives each element its own bit, N[0:3] a[0:3] a[1:4] model is a chain. The name selects the reading — E-221 already made a range in a NODE field mean one device with a WIDE PORT (X1 bus[0:3] sub), decks depend on it, and the two cannot both apply to a line; a scalar name keeps E-221's in-place expansion, a ranged name switches to one card per element with node ranges indexed IN STEP. Nothing that parses today changes meaning (a name carrying [lo:hi] was not a usable device name). New pass runs BEFORE inp_expand_buses and both read ranges through one shared parser, so they cannot drift on what a range is (an XSPICE port group [d1 d2] has no base and is invisible to both). REFUSED: a width mismatch is named and the deck is rejected — an earlier revision printed the error and let the line through, so ngspice built ONE resistor named r[0:3] and completed the run; and an A-device array, since XSPICE already uses brackets for port groups. E-221 also no longer expands the FIRST token of an element line (a name, never a node list). ADDRESSING: the lexer ended the token at the first] and the splits took everything before the first [as the device name, so @r[2][resistance] sought a device r with parameter 2; ft_accessor_param_start() fixes it with a narrow rule — an integer-only group immediately followed by another [belongs to the name — leaving @nd1[i_a[0]] (E-408) and @*[[p]] (E-269) untouched. The split lives in FIVE places: verifying against subcircuit hierarchy and the whole .control surface found two more beyond the lexer/vectors.c/device.c set — dctrcurv.c, where .dc on an array element failed FATALLY ("not in the circuit", card and command alike), and outitf.c's parseSpecial, where save @r[1][i] matched nothing and a save list that matches nothing loses the WHOLE plot, so the run ended with "no data saved ... analysis not run" instead of dropping one vector. ALSO: inp_rem_levels() reads p->line->level, so the scope tree must be freed BEFORE the deck; the

vdmos rejection site had them reversed and was a latent use-after-free — copying it crashed the new path with EXC_BAD_ACCESS at the `level` member. Verified arrayinst 36/36 both solvers (every structural check paired with a hand-computed electrical one; the Verilog-A array is bit-identical to the resistor array; a subckt array instantiated twice gives 8 uniquely-named devices, arrays NEST as `r.x1.x[0].r[0]`, and the `.dc/save` paths are checked analytically), regression 353/353.

E-442 *src/frontend/inp.c, src/frontend/commands.c, examples/listtree_examples/ (new)*

`listing tree` — THE HIERARCHY, DRAWN. `listing e` gives one line per device with the structure only implicit in the dotted names; this draws the subcircuit tree, each `X` labelled with what it instantiates and placed where it is USED, plus a summary of instances/devices/depth. `listing t` too. Walks `ci_origdeck` (the expanded deck no longer knows which subcircuit an instance came from), which also means array instances (E-441) already show as the instances they are. The subcircuit name is NOT 'the last token before the parameters' — numparam rewrites `X1 in 0 sub PARAMS: rv=2k` into `x1 in 0 sub 2k`, so counting tokens picks `2k`; resolve against the collected definitions instead. A **`.control`** block's commands begin with a letter like element lines, so unskipped they were drawn AND counted as devices and broke last-child detection (final entry kept a tee where it needed a corner). No 'never instantiated' report — ngspice comments those definitions out first, so it could never fire. Verified listtree 23/23 both solvers (shape checks are specific: last-child corner, blank-not-bar subtree indent, 8-level alignment, hand-computed summary counts), regression 353/353.

E-443 *src/frontend/inpcom.c, examples/idxlist_examples/ (new)*

BRACKETED INDEX LISTS. `a[1,3,5]`, `R[1,3,5]` and the mixed `a[0:1,7]` join the ranges of E-221/E-441 — only the SOURCE of the indices is new, so every established reading carries over (node list = consecutive terminals; instance list = one card per element; node list on an array instance = taken IN STEP). Indices in written order, not sorted or deduplicated, since the order binds nodes to terminals — so a list has no direction to mistake and E-411's reversed-binding warning still fires for `a[1:0]` but not `a[3,1]`. A lone **`a[2]`** is NOT a list — it is a scalar bus bit and stays a node name (E-221's contract); that one condition is what makes the change additive, since the only new spellings are ones that previously meant nothing. Parse split into `inp_bus_indices()` (contents) + `inp_bus_index_parse()` (base), shared by FOUR consumers including `inp_bus_rewrite_wrapped` so `.save v(a[1,3,5])` expands — its old range-specific parse REMOVED rather than left beside the shared one. New diagnostic: a malformed list is NOT harmless — `R2 a[1,] 0 1k` re-tokenises the line and ngspice built a resistor with NO VALUE, warning only 'too small, set to 1e-12'; a group that looks like an index list and is not one now says so, on a test tight enough that `mem[addr]` and `a[2]` stay silent. Verified idxlist 26/26 both solvers (structural checks paired with analytic electrical ones; the CONTROLS are the point — scalar bit, non-numeric name, E-221 range, E-441 range instance, E-411 warning scope), regression 354/354.

E-451 *src/frontend/inp.c, src/frontend/spiceif.c, examples/optname_examples/ (new)*

AN OPTION MATCHED BY SUBSTRING, AND THREE CALLED UNKNOWN. Both from asking what E-450 had just answered for `savecurrents`: which other options are decided by searching the line for a word? (1) `eval_opt()` found `seed=` and `cshunt=` with a bare `strstr`, so any option whose NAME MERELY ENDED in the watched text was taken as that option -- `.options myseed=7`, `noseed=7` and `xseed=7` all set the RNG seed identically to `seed=7`, and for `cshunt` a node reading 1.0000000000e+00 reads 6.9209970972e-07, six orders of magnitude, under `noc-shunt=/mycshunt=/xcshunt=`. Each ALSO printed ”unknown option” -- told not recognised, changes the answer anyway; for Monte Carlo a mistyped option name silently changes the random sequence. Whole-token matched now on E-450's boundary rule, `seedinfo` included so `noseed-info` no longer switches it on. NOT a defect though it reads like one: the loop has no `.options` test, but the function receives the option deck already sorted (its own comment says so) -- a `.param seed=7`, a model parameter named `seed` and a comment containing `seed=` were each measured

and leave the sequence at baseline. (2) Three options took effect while being reported unknown: `scale=2` doubles `@m1[w]`, `rseries=100` moves a transient, and **`autostop`** truncates one from 567 rows to 2 -- the case E-447 fixed for savecurrents/seed/numdgt, with names it did not cover; `scale` and `rseries` are read from the deck's own option cards so they never reach the parameter table the check consults. **`scalm`** is flagged too and deliberately NOT registered, having no demonstrable effect -- adding a name on the strength of it appearing in the source would be the same mistake in the other direction. Verified optname 24/24 both solvers and 14/24 on the previous binary, the ten failures there being exactly the defect-specific checks while every control passes on both; regression 364/364.

E-450 *src/frontend/inp.c, examples/savecuroff_examples/ (new)*

`savecurrents` COULD BE REQUESTED BUT NEVER DECLINED. Whether `.options savecurrents` was in force was decided by `strstr(options->line, "savecurrents")` -- a bare substring search -- so EVERY option line merely CONTAINING the word switched it on, whatever the line actually said: `savecurrents=0` ON, `savecurrents=false` ON, **`nosavecurrents`** ON and even `mysavecurrentsxyz` ON. The `no` spelling is the one that matters, because `noacct`, `noinit`, `nomod` and `nopage` are all real ngspice options, so a `no` prefix is exactly what a user reaches for -- and it did the OPPOSITE. Once on there was no way back. Silent throughout, because a deck that quietly saves every terminal current still simulates correctly; it just carries vectors nobody asked for -- on a large deck the difference between a small rawfile and a very large one, and on an OSDI device one vector per TERMINAL (`i_<term>`, E-394), which is how it surfaced. Reachable ONLY from an option CARD (deck, `.include`, `.lib`) and never from `spinit`/`.spiceinit`/`set/.control`, since `inp_savecurrents()` is a netlist pre-pass -- the same "too late" ordering E-411 recorded and E-449 had to work around -- which narrows the search but makes the wrong answer stickier. Now read as TOKENS: exactly `savecurrents` or a declared `savecurrents_<variant>`, a `no` prefix or a `0`/`false`/`no`/`off` value turns it off, later card wins. WHICH card is retained is deliberately unchanged, because the caller re-searches that same line for `_bsim3`/`_bsim4`/`_mos1` to choose the MOS current set -- those four sets stay BYTE-IDENTICAL (4/6/3/11 params). Before/after over 15 option spellings: 8/15 -> 15/15, every changed case moving the intended way and no control moving. Verified savecuroff 20/20 both solvers; regression 363/363.

E-449 *src/frontend/subckt.c, src/frontend/subckt.h, src/frontend/inp.c, examples/subbus_examples/ (new)*

AUTOBUS ACROSS A SUBCIRCUIT BOUNDARY. E-444 expands a bus port in `INP2N`, where the model is known; inside a `.subckt` that is too late. A definition declaring `a[0:4]` has the formals `a[0]`..`a[4]`, so a device line writing the bare `a` matched NONE of them, became the local node `x1.a`, and `INP2N` expanded THAT into five FLOATING **`x1.a[0]`**..**`x1.a[4]`** with the device wired to those rather than the ports -- all five bits reading one identical voltage, the device contributing nothing. Nothing was reported, because every terminal DID receive a node, so E-402's underconnected warning had nothing to say: turning the option ON removed the diagnostic the same deck gets with it OFF -- the shape E-445 recorded when autobus indexed ground. Fixed at the SUBSTITUTION: a token that is not a formal but for which formals `token[i]` exist expands to those formals' actuals, needing only the `.subckt` line. The option is not reachable from there the obvious way -- `cp_getvar` is false during flattening (`inp_subcktexpand` runs before `inp_dodeck` publishes options) AND the `.option` cards have already been split out of the deck, so a deck scan finds nothing; `inp.c` reads them from the option lists, as it already does for `scale` just above. Emitted by ascending BIT INDEX, because a compiled model orders bus terminals that way whatever direction the Verilog-A declared (`inout [4:0] a` still yields `a[0]`..`a[4]`, measured) and the line is positional -- so a DESCENDING `.subckt` declaration binds in reverse, E-411's rule one level up. OSDI lines only, so a bare `a` on an R line stays a local node. The other form was already correct: `.subckt mysub a b` with SCALAR formals passes the bus base to the caller's scope and expands there -- verified for one and two instances, unchanged. Out of scope: one bit where the bus is expected (`N1 a[0] b`) still floats silently and is BYTE-IDENTICAL before and

after, because after flattening nothing distinguishes it from a legitimate top-level line; `Xi a b` on a subcircuit instance fails loudly with "too few nodes". Verified subbus 16/16 both solvers, every check a differential against the written-out form on a ladder where all five bits differ; regression 361/361.

E-448 *src/frontend/evaluate.c, src/frontend/evaluate.h, src/frontend/parse.c, src/frontend/vectors.c, examples/constname_examples/ (new)*

A NODE NAMED AFTER A CONSTANT. ngspice's permanent `const` plot holds twelve vectors (`c`, `e`, `i`, `pi`, `kelvin`, `boltz`, `echarge`, `planck`, `TRUE`, `FALSE`, `yes`, `no`) and `vec_get()` falls back to it — which is how a bare `pi` resolves, and stays. (1) **`v(X)`** answered with the speed of light: a node context resolved the name before considering the context, so `v(c)` on a renamed or mistyped node gave 2.9979245800e+08 where `v(nosuch)` correctly failed — and the other eleven are more dangerous than the speed of light because they look like results, `v(no)` a flat 0.0 and `v(i)`/`v(yes)`/`v(TRUE)` a flat 1.0. It defeated an existing guard: E-431 refuses an `-output` that never resolves, and the constant walked through it BECAUSE the name did resolve, drawing a flat curve where `v(nosuch)` was refused. Only `v` has a null handler in the function table, which is why `v(c,0)`, `vm/vp/vr/vi/vdb(c)`, `i(c)` and `mag(v(c))` were already refusing. (2) A bus bit **`c[0]`** was UNREACHABLE — exactly what `.option autobus` (E-444) builds for a bus called `c`: `display` listed it and `print all` printed 5.0000000000e-01 while `print c[0]` said "indexing a scalar (c)" and `print v(c[0])` said "bad v() syntax"; E-444's own five-bit ladder with the bus renamed `a->c` printed NOTHING. E-224 preferred the literal name only when the base was UNRESOLVED, and a vector called `c` always exists. (3) The silent one needs no constant: the same gate returned the wrong vector's element whenever the base was LONGER, so a circuit with both a node `q` and a bus bit `q[0]` answered `print q[0]` with node q's value at t=0 as a SCALAR instead of the bit's waveform — and errored in `op` instead, so the failure mode flipped with the analysis type. The gate is now the literal vector's OWN existence, shared as `e448_literal_index()` rather than re-derived in the four places that had it (E-408 recorded what drift costs); the three sites that render the index as text round through E-274's `idx_floor()` rather than a bare `(int)` cast -- UB for `1e308`/`inf`/`NaN`, and the validating and the resolving site must round identically -- with 16 index expressions bit-identical before and after). A BARE name stays ambiguous and still resolves to the constant, now warning only when the name also exists elsewhere. Deliberate change of meaning: with both `x` and a literal `x[0]` present, `x[0]` is the literal one — the only working expression whose meaning changes, replacing the silent-wrong-answer reading; ordinary indexing and E-433's quoting are untouched. One change was made and WITHDRAWN: a `checkvalid()` hook to report `v(c)` once per command rested on a probe artifact — the two paths use different wordings and the filter matched one; counted properly both emit 23 lines on a 21-point sweep, which is pre-existing `sweep` behaviour. Baseline captured on the PRE-FIX binary so the controls are measured: six defects reproduced, thirteen controls already held. Verified constname 57/57 both solvers, the decisive check being E-444's ladder run with the bus named `a` and named `c` reading bit-identical; regression 360/360.

E-447 *src/spicelib/analysis/cktsopt.c, src/spicelib/devices/res/restemp.c, src/spicelib/devices/dio/diosetup.c, src/spicelib/devices/vsrc/vsrcask.c, src/spicelib/devices/vsrc/vsrcpar.c, src/spicelib/devices/isrc/isrcask.c, src/spicelib/devices/isrc/isrcpar.c, src/spicelib/devices/isrc/isrc.c, src/spicelib/devices/isrc/isrcdefs.h, src/spicelib/parser/inpdpar.c, src/frontend/spiceif.c, src/frontend/commands.c, examples/guardspell_examples/ (new)*

THE GUARD THAT COVERED ONE SPELLING OF A BAD VALUE — eight of them. In every case a working guard already sat a few lines away covering a DIFFERENT spelling. (1) A negative **`gmin`** wrecked the operating point in silence — a diode 0.63 V drop became -0.001 V, rc=0, the error growing SMOOTHLY with magnitude so there is no threshold at which it announces itself — while reltol/abstol/vntol/chgtol/trtol/maxord/temp were all already guarded by E-426; `gmin+gshunt` were the holes, zero stays legal. (2) **`scale=0`** SHORTED a resistor while a resistance written as `0` warned AND was clamped; the scale route was silent and unclamped (exactly

0.0). (3) **trrandom** TYPE outside 1..4 = a silently DEAD source (0/5/9/-1/100 gave rms 0.0). (4) Diode **level=99/-1** accepted silently while level=2 was Fatal — the check tested one value, not a range. (5) **cshunt** from **.control** silently ignored: it works by adding node capacitors at setup from eval_opt()'s cshunt_value, so the control path stores a value nothing reads — the only card-only option of nineteen tested, and the card form is BIT-IDENTICAL to writing the capacitor by hand; the notice is gated on that variable being absent so correct decks stay quiet. (6) **show** claimed a source had ALL EIGHT transient waveforms — one shared coefficient array answered every query, so a sin-only source reported 0/2/3000 eight times; a dc-only source correctly showed –, so it separated "none" from "some" but not WHICH. Returning EMPTY rather than an error is what makes **show** render **–** instead of a column of **<<NAN, error>>** — pick the return the generic printer already renders well. (7) **savecurrents**, **seed**, **numdgt** flagged unknown while working (without savecurrents @r1[i] is a scalar, with it a waveform) — E-446 downgraded this class for shell variables, which these three are not. (8) snload's help named one file while the command required two, and pwl r= on a current source failed as "unknown parameter"; now refused by name — a precise diagnostic, NOT feature parity, since porting repeat/delay means replacing ISRC's pwl evaluator. THREE reported defects were NOT: **m=0** is E-426's documented "disable this instance" idiom and **@r[resistance]** is deliberately NOMINAL with 1/@r1[conductance] as the documented effective route — both settled by E-426, both caught by its suite failing in regression, both left unchanged and now pinned as controls; and option method=bogus DOES print "unsupported integration method" (the pre-E-447 binary too) — my probe's filter missed it. Verified guardspell 54/54 both solvers; regression 359/359.

E-446 *src/spicelib/devices/vsrc/vsrcload.c, src/spicelib/devices/vsrc/vsrcpar.c, src/spicelib/devices/isrc/isrcload.c, src/spicelib/devices/isrc/isrcpar.c, src/spicelib/devices/cap/capask.c, src/spicelib/parser/inp2dot.c, src/spicelib/parser/ptfuncs.c, examples/argdiscard_examples/ (new)*

WHAT THE NETLIST WROTE, QUIETLY DISCARDED — six of them. ZERO USED AS THE "NOT SUPPLIED" SENTINEL accounts for three: the source loaders tested coeffs[n] != 0.0, so a legitimately written 0 was indistinguishable from an omitted argument. (1) An explicit **TD1=0** on an EXP source became **ckt->CKTstep** — the waveform started one step late, every printed row carrying the exact analytic value of the PREVIOUS timepoint to 10 decimals, and because the substitute IS the timestep the answer depended on the timestep: err -3.7e-03 at 10 us, -4.0e-02 (~4 %) at 100 us, with 1e-30 instead of 0 making it exact. Both source types; SIN/PULSE/PWL/SFFM exact throughout, which pinned it to EXP. (2) PULSE **PW=0**/**PER=0** meant different things to a V and an I source — PW=0 gave no pulse on one and an endless one on the other, PER=0 disagreed the other way; a zero WIDTH is now the degenerate pulse asked for and a zero PERIOD means "do not repeat", on both, while a zero rise/fall still falls back to the timestep. (3) A PWL list with an ODD token count had a value invented — pwl(0 0 1m 1 2m) on a V source was BYTE-IDENTICAL to pwl(... 2m 0), ramping the stimulus to zero at a time never assigned one, while the I source ate the stray token AS a value; two guesses at one input, so it is refused. Then (4) a third **.dc** sweep source was neither run nor refused — 9 rows not 27 with V3 pinned, proven by making V3 dominant (R3=1 ohm vs 1k) and watching nothing move — plus surplus **.ac** arguments dropped in silence while **.tran**/**.dc** refuse them. (5) **pow(fabs(x),y)** DROPPED THE SIGN of a negative base: (-2)**3 = +8, (-2)**1 = +2, so odd exponents wrong and even ones coincidentally right — while pwr(-2,3), a Verilog-A model's own pow(-2,3), and both the LTspice and HSPICE branches all give -8 (E-399's rule: an expression must not mean different things depending on whether the netlist or the model computed it). An integer exponent now carries its sign; a non-integer one keeps the historical magnitude rather than a NaN that would poison the Jacobian (E-256/E-440). (6) **@c[capacitance]** folded in **m=** while @r[resistance] reported the written value; the capacitor now reports what it was given and the simulation does not move, since capload.c applies m to the stamps itself. One reported defect was NOT one: an AM source with a 5th argument emits a constant because ngspice's AM is AM(VO VMO VMA FM FC TD PHASEM PHASEC) — the 5th argument is the CARRIER FREQUENCY, so am(1 0 100 1k 0) sets FC=0; against the right order AM matches analytic to 1e-12, unchanged. Verified argdiscard

37/37 both solvers, every fix paired with a control that must not move (omitted arguments, positive bases, complete PWL lists, one- and two-source `.dc`); regression 358/358.

E-445 *src/frontend/fourier.c, src/frontend/inpcom.c, src/frontend/subckt.c, src/frontend/com_sweep.c, src/frontend/spiceif.c, src/spicelib/analysis/cktsopt.c, src/spicelib/parser/inp2n.c, src/spicelib/parser/inp2r.c, src/spicelib/parser/inp2c.c, src/spicelib/parser/inp2l.c, examples/guardgaps_examples/ (new)*

NINE SILENT FAILURES MADE LOUD. (1) A bare **`.four`** card SEGFAULTED — `dotcards.c` warns "no nodes given" but does not DROP the card, so `fourier()` read the fundamental off a NULL wordlist; the only one of 25 zero-arg dot-cards that died — **`.print`**/**`.plot`** print the identical message from the same function and were always safe. (2) `.four 1e400` overflows to `+INF` and produced a full report, THD 201.971 %, harmonic frequencies printed `inf` — while `0`, `-1000`, `abc`, literal `inf`/`nan` and even `1e-400` (underflows to 0) were already refused; overflow was the one hole. (3) **`R1 in nb 1,5k`** built a 5 kOhm resistor: `,` is a token separator, so an unlabeled trailing number OVERWROTE the value already read (that overwrite is how `R1 a b rmod 2k` works, but there nothing had been read) — decisive because every other separator obeys the documented ignore-trailing-text rule (`1;5`, `1:5`, `1_000` → 1). R, C and L. (4) An array instance past 8192 collapsed to ONE device named `r[0:8192]` — 9x wrong at the boundary, 21x at `R[0:19999]`, silent; the same range in a NODE field already got E-443's warning, only the instance-NAME slot was quiet because E-441 made `inp_expand_buses` skip the first token. (5) **`.option autobus`** indexed ground into five FLOATING `0[0]`..`0[4]` nodes (and `a[0]` into `a[0][0]`), device contributing nothing: correct -1.9375e-3 vs 5.4e-20 — and the same line with the option OFF is reported by E-402, so switching the feature on removed an existing diagnostic. (6) A failed **`sweep`** point republished the PREVIOUS solution — three priors gave three different "results" (0.5/0.8/0.2), a flat plausible shelf that `wrdata` wrote unmarked; now `NaN`, which keeps the scale alignment E-438 preserved the points for. **`montecarlo`** was the guarded sibling all along. (7) A legal 20-deep hierarchy was refused as "infinite subckt recursion" — a DEPTH cap (`MAXNEST` 21), not name length; now 256, with runaway recursion still bounded by the million-instantiation cap. (8) `itl1`/`itl2`/`itl4` below `NIiter`'s floor of 100 were stored, echoed by `option`, and never applied (13 iterations under `itl1=3` with both fallbacks off); E-426 judged that floor deliberate and upstream and that judgement STANDS — it is now announced instead of silent, and only for an explicit setting since the shipped `itl2=50`/`itl4=10` are themselves below it. (9) `nfreqs`/`nperiods`/`polydegree`/`fourgridsize` were reported "unknown option" while demonstrably taking effect (`fourgridsize=10` moves the grid 200→10); registered — scope deliberately those four, since 176 of the 179 `cp_getvar` names are flagged and most are `set` variables. One reported defect was NOT one: an OSDI integer param taking `n=0.5` as 1 is E-399's LRM round-half-away-from-zero conversion, and 1 is in range — unchanged. Verified guardgaps 59/59 both solvers, every fix paired with the sibling that must not move; regression 357/357.

E-444 *src/spicelib/parser/inp2n.c, src/frontend/spiceif.c, examples/autobus_examples/ (new)*

`.option autobus` — CONNECT A VERILOG-A BUS PORT BY ONE NAME. `inout [0:4] a` compiles to five OSDI terminals `a[0]`..`a[4]`; with the opt-in option `N1 a b busdev` expands to the full six-token form, and `a[2]` elsewhere binds to the same node. The information was already in hand: `INP2N` resolves the model BEFORE binding nodes and `dev->termNames[]` holds those names — E-402 was already reading that table just to REPORT unconnected terminals. Terminals group into ports (consecutive `base[i]` = one bus port); a line with one token per PORT expands. Indices copied from the model's own terminal name, so a `[4:1]` port gives `a[1]`..`a[4]`, not an assumed `0..n-1`. OPT-IN because a short line already means `\$port_connected` (BSIMSOI, BSIMCMG/IMG/BULK, BSIM6, PSP-HV) — and even with the option on, an all-scalar model has port count == terminal count so it can never be mistaken for the shorthand. Option name registered with E-438's checker. Expansion is at DEVICE-PARSE time (it needs the model), so unlike E-441 it does not appear in `listing e`. Verified autobus 12/12 both solvers — every check a differential: the short form BIT-IDENTICAL to the explicit one on a ladder where all five bits differ, a

`[4:1]` bus matching explicit, two buses on one device, `\$port_connected` identical on/off, and the explicit form unaffected by the option — regression 356/356.

E-438 *src/frontend/com_sweep.c, src/frontend/com_optimize.c, src/frontend/com_measure2.c, src/frontend/spiceif.c, src/frontend/inp.c, src/frontend/inpcom.c, src/frontend/runcoms.c, src/frontend/com_hb.c, src/spicelib/devices/ind/muttemp.c, src/include/ngspice/fteext.h, src/include/ngspice/cktdefs.h, examples/failacct_examples/ (new), examples/warnphysics_examples/ (new)*

A FAILED SIMULATION MUST NOT BECOME DATA. Every command that drives an analysis in a loop read its metric back out of whatever plot was left behind -- and ngspice leaves the PREVIOUS point's plot in place when a run fails, so the read-back succeeded and returned stale or zero data. **`montecarlo`** COUNTED FAILED SAMPLES AS PASSING: 6 'DC solution failed' lines immediately above 'yield: 100.000% (20/20 pass)', monotonic in the failure count (0/3/4/6/7 all reported 100%), and with a built-in diode at sigma=10 14 of 20 samples failed and yield still read 100%. Reproduced by a second mechanism (illegal tstep from a sampled param): 37 errors, 100% (12/12). NOT OSDI-specific. The machinery was otherwise correct -- with converging samples yield reads 85/70/65%, an impossible spec gives 0%, LHS and plain bracket the analytic 49.8% -- only the FAILURE path was wrong, which is why the statistical/yield audit passed it. The signal already existed: runcoms.c publishes `sim_status` per analysis. Samples that never solved now leave the yield population and are reported; `sweep` names how many points did not converge (their outputs are 0.0 and persist into `wrdata`); `optimize` scores a failed evaluation as worst-case instead of scoring the previous point's plot, and reports the count rather than claiming it CONVERGED. NEW **`.option warn_physics`** -- opt-in, because every value it flags is one a simulator has reason to accept (negative resistance is a small-signal equivalent): `abs(k) > 1` makes the inductance matrix indefinite and the pair GENERATES energy (measured abs(v(sec)) = 1.178 > abs(v(pri)) = 0.9986 on a 1:1 transformer), `ron=-1` makes a switch a -1 ohm resistor (divider reads -1/999 V), `l=-1u` pushes a node ABOVE the supply (1.0306 V from 1 V). THE CONTROLS SHAPED IT: flagging ZERO warned six times on a healthy deck (`l`/`w` sit at 0 on every device not using them) so only strictly-negative is flagged; and `is` is dropped entirely because it is the saturation current on a diode model but the SOURCE CURRENT on a MOSFET instance. The coupling check lives in MUTtemp -- the askable `k` returns the mutual inductance M, not the coefficient, so only the device layer has both. Also: a bogus CO-KNOB no longer cancels the whole sweep restore (E-385's all-or-nothing premise fails when the failing knob is a different knob from the one that moved -- the circuit is NOT untouched; the next `op` read 0.3333 instead of 0.5) and capture is now order-independent; `.meas ... WHEN` respects its own MEASUREMENT_FAILURE instead of printing a fabricated 0 formatted like a real measurement (`meas ac ... WHEN mag(v(nb))=0.707` recorded 0 Hz as the -3 dB point while its sibling FIND failed loudly); an unknown `.options` NAME is reported, on the card path only (if_option is called by cp_usrset for EVERY shell variable, so warning there makes ngspice complain about its own `rndseed`); an unmatched `.if` now sets the exit status like its `.subckt`/`.ends` sibling; and a successful `hb` exits 0 (main.c reads `sim_status`, which hb never set, so every good HB run in batch mode exited 1 with 'incomplete or empty netlist'). DEFERRED with reasons: `pow(v,0<e<1)` aborting (the solver is right to object to an unbounded derivative), numparam's 1-ULP decimal scanner, XSPICE pwl array-length mismatch, PWL odd token count, and subckt parameter typos (positional dollar-placeholder substitution in numparam). Verified failacct 9/9, warnphysics 18/18 (every positive paired with the option OFF, plus five clean decks), regression 347/347 both solvers.

E-437 *src/frontend/com_sweep.c, src/frontend/spiceif.c, src/frontend/inp.c, src/include/ngspice/fteext.h, examples/modelwild_examples/, examples/sweepguard_examples/*

A SWEPT MODEL WILDCARD IS PUT BACK, AND A **`.temp`** WITH NO VALUE. Two silent wrong answers found by a hunt run against the binary shipped the same day, both cases where the guard that should have caught them already existed and simply was not reached. (1) E-436 gave `@*:rmod[param]` a SET path but never the capture-and-replay path E-409 built for `@*[param]`, so `sweep @*:rmod[res] 1k 3k 1k` ran correctly and then left all three matched models at 3000

-- the next analysis silently used the wrong circuit. That is exactly the defect E-409 exists to fix (*a wildcard knob was never put back*), reappearing for a newer spelling. ONE OMISSION, TWO SYMPTOMS: `sw_wildcard_knob()` is consulted in exactly two places -- to NOT read a wildcard as a scalar (`*` is in the lexer's `specials`, so PPparse emits a stray-`]` pair; E-409 added that guard and its comment quotes the very lines `@*:rmod[res]` printed) and to capture per target. Matching neither, the new form fell through to the scalar reader (spurious `no such device or model name + PPerror`) with nothing captured (no restore); teaching that one function fixes both. The new `if_{save,restore}param_wildcard_model_named()` reuse E-409's `wild_ask_scalar/wild_set_scalar` and its all-or-nothing rule, differing only by the `eq(model_leaf(nm), leaf)` filter -- they MUST walk exactly the targets `if_setparam_wildcard_model_named()` walks, in the same order, or index i names a different model on the way out and a decoy is restored to a value that was never its own (pinned by moving an unrelated `omod` to 7000 first and requiring it to still read 7000). (2) A `.temp` card with NO VALUE ran the circuit at 0 C, rc=0, nothing on stderr but the routine `TEMP = 0.000000` line: `strtod("")` returns 0.0 AND leaves the end pointer at the start, so the trailing-garbage test that correctly rejects `.temp abc` saw a clean parse. A divider with `tc1=0.01` answered 0.5780 instead of 0.5000 -- 15.6% wrong, silently, no Verilog-A involved. Its own sibling `.options temp=` already refused the identical mistake; probing all 14 value-taking dot cards showed `.temp` is a SINGLETON (`.tnom/.include/.lib` abort, `.four/.print/.plot` warn, `.ic/.nodeset/.save/.param/.global` are silent but INERT) -- the only card both undiagnosed and able to change the answer, so it is fixed alone. DELIBERATELY NOT CHANGED: `.temp 1e9` (E-426 drew its line at the physically IMPOSSIBLE, not the implausible); `@(initial_step)` firing once per `sens` re-solve (each re-solve IS an initial step); `tran uic` omitting t=0 (long-standing, general ngspice, and emitting it would change every uic transient's row count). One finding WITHDRAWN: a malformed `optimize` IS diagnosed (`needs hi > lo`) -- stdout/stderr ordering hid it. Verified modelwild 21/21 (up from 16, with the swept curve pinned so the fix cannot hide an answer change) and sweepguard 43/43 (up from 35, including a positive control that `.temp 125` still SETS 125 C); both suites run against the UNFIXED binary as negative controls, where exactly the defect-specific checks fail. Full regression 347/347 both solvers, including `wildrestore`.

E-436 *src/frontend/spiceif.c, src/frontend/device.c, src/frontend/com_sweep.c, src/include/ngspice/fteext.h, examples/modelwild_examples/ (new)*

ONE MODEL NAME, EVERY INSTANCE PATH. Subcircuit expansion renames a `.model rmod` inside x1 to `x1:rmod`, so a deck declaring `rmod` at top level AND inside a subcircuit ends up with several distinct models -- and only two ways to reach them, one too narrow and one too wide: `@rmod[param]` hits the TOP-LEVEL CARD ONLY (every subcircuit copy silently keeps its old value -- easy to write while meaning *the model*), and `@*[param]` hits every model that has `param`, sweeping up unrelated ones. Neither expresses the ordinary want: one definition, several instantiations, plus a top-level copy. New **`@*:rmod[param]`**: `*` stands for the INSTANCE PATH and matches any path including none, so the top-level card is covered alongside every `<path>:rmod`. Matching is on the LEAF NAME (after the last ':', or the whole name), which is depth-independent -- `rmod`, `x1:rmod`, `x1.x2:rmod` all match -- with no pattern syntax. `@*.rmod[param]` accepted too (E-433/435 taught the dotted spelling elsewhere). Deliberately NOT a glob: `@x*:r*` stays an error, since E-269's wildcards are a fixed token set and a mistyped pattern would match nothing silently. **`@rmod`** was deliberately NOT broadened: that would change what existing decks DO with no diagnostic, and every hierarchical fix here (E-410/428/433/435) was safe precisely because it could only turn an ERROR into a hit, never one working behaviour into a different one; targeting a single card also has to stay reachable for mismatch work. What was missing was information, so it now reports how many copies it left untouched and names `@*:rmod[...]`. Two distinct failure messages replace a shared not-found: an unknown model name explains the flattening, a known model with an unknown parameter says so. A GAP THAT TURNED OUT NOT TO EXIST: the instance wildcards `@#*[param]`/`@*[[param]]` looked like they missed subcircuit instances -- they do not. The probe measured a divider in which the wildcard changed BOTH resistors equally so

the ratio stayed 0.5; reading the parameter back shows `r.x1.rx` and `r.x2.rx` going 1000 -> 3000 like the top-level one, and `if_setparam_wildcard_instance()` walks every instance in the circuit, where flattened ones are ordinary members. Fourth non-discriminating probe of the session. Verify (examples/modelwild_examples, 11/11, new): the deck makes every level of reach distinguishable (v(a)/v(b) subcircuit copies, v(e) top-level card, v(c) an unrelated `omod` that must not move); both spellings checked, both existing forms checked UNCHANGED, `sweep` checked on a copy and the top-level card in one sweep, both diagnostics pinned. Full regression 347/347.

E-435 *src/frontend/com_sweep.c, examples/hierdev_examples/ (extended)*

The FOURTH consumer of a hierarchical model name, and the one E-433 missed. A `.model` inside a subcircuit is renamed `<instance-path>:<model>`, and E-433 taught the name funnels to accept the dotted `@x1.rmod[res]` users actually write -- reaching `altermod`, `optimize -mparam` and `@`-readback because all three resolve through `finddev/finddev_special`. **`sweep`** does not: **`sw_kind()`** calls **`ft_sim->findModel()`** itself, right where it classifies the knob. The failure was worse than a refusal: an unrecognised `@name[param]` falls through to the INSTANCE branch, `alter` reports a no-such-parameter error for a MODEL parameter, and the sweep runs on with a knob that never moved -- a full set of points, rc=0, and a plottable FLAT curve behind a diagnostic that scrolled past mid-run. Same shape as the zero column E-431 removed, except it is the KNOB not the output, and a constant curve is easier to mistake for a result than a zero one. Fix is one condition mirroring the two funnels; application already worked, since `sw_set_inplace` issues `altermod` which E-433 taught the dotted form. SCOPE CHECKED, NOT ASSUMED: `sw_kind()` has exactly ONE caller -- montecarlo/highsigma/wcd perturb Gaussian `.param` values rather than named knobs, and `optimize` classifies its own via `-param`/`-mparam`/`-dparam` whose model path already worked. Verify (examples/hierdev_examples, 38 -> 43 checks): all four spellings (`@x1.rmod[res]`, `@x1:rmod[res]`, and both nested forms) measured against the analytic divider `v(out)=1k/(res+1k)` -> 0.5/0.3333/0.25, plus a top-level `@rmod[res]` the new fallback must not disturb. POSITIVE-CONTROLLED: reverted and rebuilt, exactly the two dotted checks fail with the flat 0.5/0.5/0.5 that is the whole point; the colon forms and the top-level model pass either way. Full regression 346/346. KNOWN NOT FIXED: a knob that fails to apply for ANY reason still yields a flat curve at rc=0 (`sweep @x1.nosuch[res]`); catching that means verifying after the first application that the knob took (E-385's `sw_read_knob` can read one back) and refusing the sweep otherwise -- a behaviour change with its own regression surface, so recorded rather than bundled.

E-434 *src/frontend/com_sweep.c, src/osdi/osdiload.c, src/spicelib/analysis/tfanal.c, examples/silentloss_examples/ (new)*

Three silent wrong answers, a silent truncation, and a swallowed request, from the round-35 hunt. (1) FOUND, NOT FIXED -- and the attempt is the finding. A `save` list that resolves to NOTHING aborts the analysis (`no data saved ...; analysis not run`) and every result is lost; `save` is alone in this (.probe/.print/print/wrdata diagnose and carry on) and disagrees even with itself, since a bad `@dev[param]` merely warns (E-418). The obvious fix -- fall back to recording everything, as when no save list is given -- was written and withdrawn because it creates a CRASH: the regression's `nameovf` suite went SIGTRAP. The abort is LOAD-BEARING, stopping execution before a latent long-name stack overflow on the `.four`/gettoks path that E-237 hardened but did not fully reach (`.four 1k i(<600 chars>)`: rc=1 -> rc=133). E-399's lesson running backwards -- there, REJECTING a wrong answer created a crash; here ACCEPTING one does. Removing the abort must follow a dotcards.c fix, not ride along with this one. (2) **`\$simparam("temp")`** was not supplied though **`tnom`** was, despite `ckt->CKTtemp` being right there: with a default it returned the caller's value (silently the wrong temperature), without one it killed the run with OSDI(fatal). Returned in CELSIUS like `tnom`. `temperature/timestep/maxstep/freq` deliberately stay OUT -- no simulator answers those names and `\$temperature` already works, so supplying them would invent an interface rather than complete one. (3) **`\$simparam("abstime")`** returned the DC SWEEP VALUE (`dc V1 0 5 1` -> 5.0; `-3 -1` -> -1.0; a different swept source -> that source's

value) because .dc reuses CKTtime as its abscissa -- an aging model read a VOLTAGE as a time. Guarded on MODETRAN, exactly as OSDIfinalStep in the same file already did for E-412; the pollution never reached the solver (ddt exactly 0 in DC, verified to 10 digits). (4) montecarlo/highsigma/wcd silently truncated **-analysis** at a 512-byte **strncat** buffer and ran a DIFFERENT command past 509 chars; they now refuse. DISCLOSURE: for **wcd** that exposure was created by E-433 three commits earlier, which gave it the same multi-token collector as its siblings; the other two always had it. (5) **.tf** was the one analysis that reports a result while swallowing a model's **\$finish** -- dcop/dctrcurv/acan/noisean/dctran each have E-426's notice, tfanal had none; it now reports and, as in dcop, does NOT act (a single computed point, nothing left to truncate). WITHDRAWN WHILE FIXING: the x1:rmod model-name collision is real (1000 -> 7777 ohm, solution moves) but ngspice already warns model "x1:rmod" is already defined; the hunt missed it by checking only values on that probe, and a second warning would duplicate a diagnostic -- the very thing E-430 removed from .probe -- so NOTHING was changed and the existing warning is pinned instead. Also re-confirmed: **\$finish** at an operating point is reported but not obeyed, E-426's deliberate decision (the hunt's 'ignored' came from a head -4 eaten by three \$strobe lines). KNOWN NOT FIXED: a refused alter still reads back the REFUSED value; opvars named temp/dt/m/dtemp are shadowed by E-397's instance knobs (dtemp a fourth case not previously recorded). Verify (examples/silentloss_examples, 23/23, new): every fix checked in BOTH directions -- the failure is gone AND the neighbouring good case is unchanged. Full regression 346/346 (the new suite included).

E-433 *src/frontend/{com_sweep,com_optimize,spiceif}.c, src/include/ngspice/fteext.h, examples/{sweepguard,hierdev}_examples/ (extended)*

Two spellings that worked everywhere except in one place. (1) QUOTING: ngspice's lexer treats the two quote characters differently and says so in its own comments (parser/lexical.c) -- '...' forms one word WITHOUT the quotes, "..." forms one word INCLUDING them, leaving each command to strip the survivors with cp_unquote(), which ~17 frontend files do (com_echo.c among them, hence echo "a b" looking fine). com_sweep.c and com_optimize.c called it ZERO times, so -analysis "tran 1n 20n" reached the command lookup with its quotes attached (unknown command '"tran 1n 20n"') while the single-quoted spelling worked; for -output the residue was worse than an error -- "v(d)" recorded a vector literally NAMED with quotes and "gain=v(d)" split at the wrong = so nothing resolved. Tokens are unquoted INDIVIDUALLY, since stripping one outer pair from a joined "tran" "1n" "20n" would eat the wrong two characters. The same flag was also collected THREE ways by the five commands that document it identically: sweep/optimize used collect_until_flag(), montecarlo/highsigma stopped at ANY leading – (so a legitimately negative analysis argument, disto lin 3 1e5 1e6 -0.5, ended the list and came back as an unexpected token), and wcd **strncpy**d ONE token so it alone REQUIRED quoting (-analysis tran 1n 20n kept tran and rejected 1n). All five now use is_flag() -- – followed by a LETTER -- the same distinction E-432 drew for the -output terminator, and for the same reason: -v(d) and -0.5 are values, not flags. (2) HIERARCHICAL MODEL: modtranslate() renames a subcircuit-local .model as tprintf("%s:%s", scname, model_name), so a model in x1 is x1:rmod and one in x1/x2 is x1.x2:rmod -- levels joined with ., the model attached with :. Nothing else in the hierarchy is spelled that way (devices @x1.rx[p] per E-410, nodes v(x1.mid)), so the natural @x1.rmod[res] was refused and the colon had to be found by reading the expansion code. if_find_model_hier() turns the LAST . into : -- exactly the instance-path/model boundary at any depth -- wired into BOTH funnels (finddev, finddev_special) AFTER the exact lookups and after E-410's instance fallback, so the rule can only turn a former error into a hit; a name already containing : is skipped. One fallback reaches altermod, optimize -mparam and @-readback together, since all three resolve through those funnels. Verify (sweepguard 23 -> 35, hierdev 32 -> 38): five commands x three quote styles + the negative argument; the dotted model resolves to the same model as the colon one, altermod drives it (v(out) 0.5 -> 0.25), optimize -mparam finds res=3000. Both sets POSITIVE-CONTROLLED -- reverted and rebuilt, 9 of 12 quoting checks and 3 of 6 model checks fail; the rest are deliberate no-regression

controls, incl. a hierarchical DEVICE that must keep resolving as a device. Full regression 345/345. FOUND BY two questions, not a hunt. Three method notes, all one mistake: a pass/fail filter keyed on error WORDING scored two real failures as ok; a probe using a bad analysis NAME could not discriminate (`sw_run_cmd` reports only the FIRST word, so a collected `nosuch lin 3 1e5` and a truncated `nosuch` print identically); and **`optimize`** prints 'converged' with a nonzero residual when the knob never moved the metric (0.0625 = the square of the untouched starting error).

E-432 *src/frontend/com_sweep.c, examples/sweepguard_examples/ (extended)*

`sweep -output` read only its FIRST token, though its usage line has always advertised `-output <expr> ...`; every token after the first fell through to the `unrecognized token` branch and the sweep ran on with a silently shorter output list. The obvious termination rule is wrong here: stopping at the next token beginning with - would break `-v(d)`, a legitimate NEGATED output expression that the old single-token form accepted because it took the next token unconditionally. So the list ends at the flags `sweep` actually knows (`-vs`/`-family`, `-analysis`/`-a`, `-overlay`/`-ov`, `-output`/`-o`) and at nothing else, which leaves a MISTYPED flag to be read as an expression -- the right outcome, since Enhancement-431 then names it in full (`sweep -output -ouptut never resolved`). One case improves rather than merely surviving: a flag directly after `-output` is now a missing expression rather than an output NAMED `-vs`, so the outer knob that follows is honoured instead of consumed. Each element takes the same path as before, so `name=expr` and E-267 bus expansion `base[lo:hi]` work from any position, not only the first. SECOND HALF -- an Enhancement-431 regression, and the worse of the two. E-431 gives each output a tally of the points at which it failed to resolve and zeroes it with `for (k = 0; k < nout; k++)`. With no `-output` at all the outputs are auto-collected from the first analysis's plot INSIDE the point loop, below that line, so `nout` is still 0 there and every auto-collected output inherited stack garbage -- which E-431 read as a resolve failure. On a 25-node deck, a plain `sweep @r1[resistance] 1k 2k 1k -analysis op` DELETED six of the twenty-five node curves with `never resolved` and falsely warned about others. That is the default invocation of the command. Fixed by zeroing the whole array; the heap arrays sized per output (`data`, `ovy`) were checked and are fine, being allocated AFTER the auto-collect. Verify (examples/sweepguard_examples, 9 -> 23 checks): the variadic list agrees value-for-value with the one-flag-each form it replaces, each terminator ends it, a negated expression survives, an empty `-output` is diagnosed without swallowing the flag after it, a bus range expands from a non-first position, and both auto-collect checks were POSITIVE-CONTROLLED -- with the fix reverted they fail, naming the six nodes that disappear. Full regression 345/345.

E-431 *src/frontend/com_sweep.c, examples/sweepguard_examples/ (extended)*

A **`sweep -output`** naming a vector that never resolves was recorded as a full column of zeros -- a clean, plottable, entirely fictional flat line, behind nothing louder than a `checkvalid` warning. The cause was already written down in this file: `sw_eval_expr()` returns 0.0 on failure, and Enhancement-385's comment says exactly why that is a problem -- "indistinguishable from a knob that is legitimately zero" -- having solved it for the knob-restore path by adding an `*ok` out-param to `sw_read_knob`. The `-output` path has the same defect and a worse symptom, because the zero is not used once and discarded but recorded, named and drawn. Same fix shape: `sw_eval_expr_ok()` reports whether the expression RESOLVED, independently of its value; the five other callers (`optimize` and the metric evaluators) keep the old signature through a thin wrapper and are untouched. Reported PER OUTPUT, from a per-output tally of failing points: failed EVERYWHERE -> named in an error and the curve is not emitted; failed at SOME points -> a warning with the count, curve still emitted because the points that resolved are real; resolved to ZERO -> recorded, unremarked, which is the distinction the change exists for and is pinned with `v(d)-v(d)`. A bad output does not cost the good ones. Verify (examples/sweepguard_examples, +9 checks): the refusal, the surviving siblings, and both directions of the distinction. Full regression 345/345. NOTED NOT FIXED: `-output` consumes only its FIRST token, so `-output v(a) v(`

b) silently records v(a) alone (one –output per expression is the working syntax) -- pre-existing, identical before and after, and the usage line's [–output <expr> ...] implies otherwise, so which spelling is right wants its own decision.

E-430 *src/frontend/{inpc_probe,inpcom,spiceif,inp}.c, src/spicelib/analysis/optran.c, examples/probeshort_examples/ (extended)*

A diagnostic-only change: what **.probe** says when it refuses a token. .probe @r1[i] printed "Warning: Strange parameter in line *probe @r1[i], ingnored" -- TWICE -- naming neither the accepted forms nor the right tool, echoing the card in its internal *probe form, and misspelling "ignored". The refusal itself is CORRECT and is unchanged: this was raised as a capability gap and is not one. Per the manual 11.7.1 .probe MEASURES a current by placing a voltage source in series with the device's node -- hence its <inst>#branch results -- while @device[param] is read out of the device and needs no extra node, which 11.7.3 describes as the .options savecurrents route "because the data are retrieved [internally]", generating .save @r1[i] lines. Three distinct documented mechanisms; teaching .probe the fourth spelling would conflate a circuit-MODIFYING command with a vector-SELECTION one, and the vipVIP gate is principled because .probe needs something it can put a source IN SERIES WITH. The message now names the accepted forms (v(...), i(...), p(...), alli), points an @ token at .save/.options savecurrents (and only an @ token -- a plain .probe nonsense gets no irrelevant advice), fires ONCE (the type test warned and advanced past the token, then the !nextnode test warned again on the wreckage; a rejected flag suppresses the second), and echoes the card as the user wrote it. ingnored -> ignored also in five further places that have nothing to do with .probe: inpcom.c (x2, .hdl and .biaschk), spiceif.c (snsave), inp.c (a comment) and optran.c. Verify (examples/probeshort_examples, 36 -> 48 checks): the message content, the single firing, the card echo, the absent typo, and in the other direction that .probe i(r1), .probe alli and .probe v(b) stay silent and still produce r1#branch. Full regression 345/345.

E-429 *src/include/ngspice/{cktdefs,inpdefs}.h, src/spicelib/parser/{inpsymt,inp2dot}.c, src/spicelib/analysis/{tfanal,noisean}.c, examples/inputguard_examples/ (extended)*

A **.tf** CARD answered **transfer_function = 0.000000e+00** for a node that does not exist, with no diagnostic -- a plain typo or a device-internal node alike, while the .control command form of the same thing refuses it properly. E-426's bounds guard is not wrong, it simply cannot fire here: a node named by a COMMAND is created after CKTsetup() sized the matrix so its equation number lands past the end (the out-of-bounds read E-426 documented), while a node named by a CARD is created BEFORE setup and is counted when the matrix is sized -- the phantom sits inside it and the analysis merely reads an all-zero row. E-426's own write-up recorded this ("created early ... it lands inside the array and the read is merely wrong") and did not act on it. Refusing at parse time is not available: E-349 deliberately lets an analysis card name a node the deck has not reached yet, and that case is pinned in examples/. So the question is not "does this node exist" -- the card made it -- but "did anything OTHER than the card ever refer to it". One bit on CKTnode, devRef, records it: every REAL creation path sets it (INPtermInsert for a device card, whether it finds or creates -- finding is what makes a forward-referenced node real; INPmkTerm for the simulator's own internal nodes; INPgndInsert for ground), and exactly one path clears it -- inp_analysis_node, the only inventor. CKTnodePhantom() is the test; ground is never phantom. The default is REAL, not phantom, deliberately: a creation path added later and not taught about the flag behaves exactly as today rather than being silently rejected. Also solver-agnostic, unlike the obvious alternative of asking the matrix whether the node has any entries -- SMPfindElt asserts Sparse and would need a KLU special case, and the regression runs both. noisean.c has the same shape and gets the same guard; sens needs none, its card path fails earlier. Verify (examples/inputguard_examples, 88 -> 93 checks): the refusals plus both directions of the boundary -- a real node still answers 0.75 and E-349's card-before-its-devices case still answers 0.75. Full regression 345/345.

E-428 *src/frontend/vectors.c, src/frontend/outitf.c, src/include/ngspice/fteext.h, examples/hiernode_*

examples/ (new)

The last place the device-type letter leaked out was a NODE name. Enhancement-410 made `@x1.r1[resistance]` work; it left a device's own INTERNAL nodes behind. `v(n.x1.n1#mid)` resolved and `v(x1.n1#mid)` did not -- and a node is the one place a user never expects a device letter, since the plain node beside it is `x1.m`. The letter is INHERITED, not chosen: subcircuit expansion re-parses the emitted card and dispatches on its FIRST CHARACTER, so the flattened refdes must keep a type letter (`n1` inside `x1` becomes `n.x1.n1`), and an internal node is named `<instance>#<node>`, so it comes along for the ride. The reconstruction needs no search and cannot be ambiguous: the letter is literally the leaf instance name's own first character and ngspice requires a device name to begin with its type letter, so `x1.n1` can only ever mean `n.x1.n1` -- one-to-one at any depth (`x1.x2.n1#mid`), with `x` instances exempt exactly as `translate_inst_name` exempts them. STRICTLY a fallback, consulted only after the exact lookups fail, so every name that resolves today is unchanged. TWO independent resolution paths, and fixing one was not enough (the E-408 lesson, which nearly bit again): `findvec` (vectors.c) serves print/let/meas/wrdata/write and the .print/.plot cards, while `.save` matches on its own comparator `name_eq` (outitf.c). With only the first fixed, `save v(x1.n1#mid)` silently matched nothing -- and that failure is destructive, not local: the run ends with "no data saved for Transient analysis; analysis not run", losing the WHOLE plot rather than one vector. Both paths now share one helper, `cp_hier_devname()`. Verify (examples/hiernode_examples): 17 checks on a 1k/3k divider, so the internal node sits at 0.75 -- a value nothing else in the circuit takes, which makes a wrong resolution visible rather than plausible; both spellings in every consumer, at one and two nesting levels, case-insensitively, plus the things that must not move (the device-letter form, an ordinary node, an ordinary HIERARCHICAL node, and a genuinely bad name). Full regression 345/345.

E-427 *src/spicelib/analysis/dctrcurv.c, src/include/ngspice/trcvdefs.h, examples/sweepparam_examples/ (new)*

A swept parameter value the device REFUSED was applied anyway, and the sweep set one value past its own stop. (1) `.dc @inst[param]` was the ONE parameter-supply path that applied a value a Verilog-A `from` range forbids, published the results and exited rc=0: `dc @n1[r] -2000 -1000 500` on `r ... from (0:inf)` gave three rows at R = -2000, -1500, -1000 with "Parameter r is out of bounds!" printed four times. The instance line, `alter` + a run, and the `sweep` command all refuse the same value. The range is not checked where you would look: `OSDIparam` has no range check at all (which is why `alter @n1[r]=-5` stores -5 and `print` shows it), it is enforced when the device is set up again inside `DEVtemperature` -> `OSDItemp` -> `setup_instance`; `DCTsetInstParam` was `void` and discarded BOTH returns, so a fix keyed on `DEVparam` would have changed nothing. Keyed on the device refusing, never on the value: a negative resistance is legitimate for a built-in resistor (resparam.c has an explicit branch, per E-426) and that sweep is unchanged. The abort exits through the RESTORE path, not where it stands -- leaving the instance holding the rejected value is the E-381/E-382/E-385 class. (2) The sweep handed the device one value PAST **`stop`** -- it advances then tests the stop criterion -- so a sweep that legitimately ENDS AT a range edge stepped outside it: `k ... from [0:1]` with `dc @n1[k] 0 1 0.25` printed a spurious "out of bounds" while producing five correct rows, and that predates this enhancement. The first version of the fix turned that valid sweep into a hard error (rc=1 where it was 0) and an adversarial review of the FIX caught it; the past-stop value is no longer applied, which removes the spurious message too. The TEMP_CODE arm has always declined its own overshoot for the same reason. (3) An INTEGER instance parameter could not be swept and the refusal lied: "Voltage source, current source, or resistor named "`@n1[n]`" is not in the circuit" -- every clause false, from an `IF_REAL` test folded into the keyword match so a wrong-type hit was indistinguishable from a miss, sharing a catch-all with ten distinct causes. Integer sweeps now work; a FRACTIONAL one is refused, because the accumulator must stay real (a rounded one with a 0.25 step never advances -- the E-362/E-426 non-advancing-loop class in this same function) and rounding only at the device boundary would publish duplicate points under an abscissa that disagrees with the value applied. (4) NOT FIXED,

documented instead: `.ic`/`.nodeset` on a device-INTERNAL node. `INPpas3` resolves those names before `CKTsetup()`, which is when every device -- built-in and OSDI alike -- creates its internal nodes, and inppas3.c's own header comment says so; a built-in diode's `d1#internal` is rejected identically. Verify (examples/sweepparam_examples): 32 checks; full regression 344/344.

E-426 *src/spicelib/parser/{inp2dot,inpeval,inpdpar,inpgmod,inpmkmod}.c, src/frontend/{spiceif,com_measure2}.c, src/include/ngspice/inpdefs.h, src/spicelib/analysis/{cktsopt,dctrcurv,noisean,span,acan,dcop,tfanal}.c, src/spicelib/devices/res/res.c, src/osdi/osdisetup.c, examples/inputguard_examples/ (new)*

Inputs that were taken at face value, and the confident wrong answers that followed. (1) A mistyped analysis OUTPUT node was invented. E-349 already refuses one, but gated the check on `CKTisSetup` -- false for the SESSION'S FIRST analysis, since nothing has run so `CKTsetup()` has not either. `tf v(nosuch) v1` returned `transfer_function = 0.0`, **`tf v(a,nosuch) v1`** returned exactly 1.0, `sens v(nosuch)` printed every sensitivity as `-0.0` and `noise v(nosuch)` reported `onoise_total = 0.0`, all silently; the same typo after any `op` was diagnosed correctly, which is what hid it. A card `if_run()` synthesises from a `.control` command is by construction not deck parsing, so that path now says so instead of inferring it. The phantom node is also an out-of-bounds heap access -- `CKTrhs`/`CKTrhsOld` are sized from `SMPmatSize()` but indexed by `node->number` from `CKTmaxEqNum`; ASAN reports heap-buffer-overflow reads at `tfanal.c:121` and `noisean.c:465` and `tfanal.c:153` WRITES through the same index, the shipped binary printing what it read as `transfer_function = 3.999110e+252`. A DECK card may legitimately precede its devices, so a bounds check at the analysis entry is the last line of defence. It immediately surfaced a live bug in this repo: `rfpss_examples` ran `.pss` on node `1`, which that circuit does not have, and the PSS self-check had been printing `osc-node swing [0, 0]` ever since. (2) Sweep arguments: **`.tran`** validates, almost nothing else did. `.ac` SUBSTITUTED defaults and ran a different sweep -- `ac dec 10 100k 1k` became 31 points over 1e5..1e8, `ac dec 10 -1k 100k` became 51 points from 1 Hz -- under one generic warning; `noise` with an inverted range published `onoise_total = 0.0` manufactured by a loop that never executed; `sp` had no validation at all; `.dc` with a step pointing away from stop produced a plot with no vector. The boundary is the whole difficulty: re-measurement WITHDREW most of the `.dc` finding -- `dc v1 1 1 1`, `0 1 1.5` and `0 1 2` correctly give ONE point, and 13 decks here depend on that while 2 more sweep descending -- so the predicate is exactly `(stop-start)*step < 0`, false for equal endpoints and false for a genuine descending sweep. Equal endpoints are pinned in all four (19 `.ac`, 4 `.noise`, 9 `.sp`, 13 `.dc` cards). (3) **`meas`** consumed a one-point vector as a waveform. `@dev[param]` is a documented scalar snapshot unless `.saved` and `save all` deliberately does not cover it, but every meas loop runs `i < d->v_length` against the full-length scale, so MAX/MIN/PP/INTEG reported that one sample as the extremum -- the `at= 0.00000e+00` was the tell. Six entry points; XSPICE event vectors carry their own `d->v_scale` and are exempt. The same function also had a heap write overflow: `measure_rms_integral` sized three buffers from the scale and filled them from the data vector. (4) A temperature below absolute zero was accepted on all six supply paths and became a NEGATIVE thermal voltage -- a model read `\$vt = -0.0195 V` in silence. Three paths share the `CKTsetOpt` funnel; `.dc temp` writes `CKTtemp` directly and instance `dtemp` is composed per device. -25 C is ordinary and one deck uses it: the line is absolute zero, not freezing. `dtemp=-300` is WITHDRAWN -- 0.15 K is physical. (5) Non-positive tolerances and iteration limits. A tolerance <= 0 made the convergence test unsatisfiable and the run blamed the CIRCUIT. `itl2=0` is the one with teeth: `CKTdcTrcvMaxIter` is an UNFLOORED `/4` and `3*/4` continuation threshold in four `cktop.c` heuristics, so zero collapses the gmin ramp -- 736,920 Newton iterations against 55. `itl1`/`itl4` are floored at 100 by `NIiter` (left alone, upstream and deliberate) so those two are reporting hygiene. Keyed on the `OPT_` enum, never the name: **`itl6`** is a SYNONYM for srcsteps where 0 is documented and four decks rely on it. (6) A negative multiplier inverts a device -- `m=-1` stamps -2000 ohm on a 2k resistor and makes OSDI noise NaN via `sqrt(\$mfactor)`. Identified by parameter id, so both `m` and `_mfactor` are covered; **`m=0`** is deliberately untouched, being the ordinary disable idiom. Duplicate same-

name `.model` cards (first wins) and `m=` written on a `.model` card (discarded, while `_mfactor=` works) are now announced rather than silent. (7) Numbers. `1e400` became inf and a resistor an open circuit; `0e400` became NaN and the OP failed five levels away. The exponent accumulated into a plain `int`, so `1e2147483648` was signed overflow yielding `pow(10,INT_MIN)`; it now saturates. An exponent marker with no digits was SWALLOWED, letting the next letter be read as a scale factor -- `10Emitter` = 0.01, `1em` = 1e-3, both against `src/ngspice.txt:499`. Documented forms are pinned unchanged (`1k2`, `2meg5`, `1e5x`, `1kk`, `0x10`, `5kohms`), and **`1..2`** is deliberately NOT touched: `version=4.8.2` is the universal BSIM4 spelling and reads as 4.8 by exactly that rule. (8) A SIGSEGV. A failing OSDI `setup_model` had its error OVERWRITTEN by the next model's success, so `OSDIsetup` returned OK while that model's instances were never set up and ngspice dereferenced a NULL jacobian: two `.model` cards gave rc=139 and ZERO bytes, while putting the bad one first gave a clean diagnostic -- deck order decided crash versus message. First error now latched. (9) The effective resistance became readable via new READ-ONLY `conductance`/`acconduct` entries whose handlers already existed in `resask.c` as dead code (`1/@r1[conductance]` = 1200.0 where `@r1[resistance]` correctly still reads the nominal 1000). `@r1[resistance]` is deliberately NOT changed: the capacitor scales its readback and that CORRUPTS the round trip (1n -> 1.2n -> 1.44n permanently under a sweep at 227 C) -- a separate defect, flagged not fixed. (10) **`\$finish`** reached three analyses out of eleven. The reported headline is WITHDRAWN -- ngspice does print `Note: \$finish requested ...`; the hunt harness filtered every line containing `"Note:"`. The real gap: under `.ac` the request was dropped in silence and the whole sweep ran. `.ac` now aborts per the E-56 precedent, `.op` reports and keeps its result. Verify (examples/inputguard_examples): 88 checks, every boundary pinned from both sides; full regression 343/343.

E-419 *src/maths/ni/nicomcof.c, src/maths/ni/niinteg.c, src/spicelib/analysis/dctran.c, src/spicelib/analysis/cktterr.c, src/spicelib/analysis/cktsopt.c, src/spicelib/analysis/cktdojob.c, src/spicelib/analysis/cktntask.c, src/include/ngspice/{cktdefs,tskdefs,optdefs}.h, examples/integmethod_examples/ (new)*

Three new integration methods, all OPT-IN, plus the measurements that say when to use them. ngspice shipped trapezoidal (A-stable but NOT L-stable, so it RINGS at a sharp transition) and Gear 1-6; the only cures for the ringing were `xmu<0.5`, which costs an order, or `method=gear`, which costs accuracy everywhere. (1) **`method=trbdf2`** -- a one-step COMPOSITE: trapezoidal over [t,t+g*h] then BDF2 across t, t+*gh, t+h. With g = 2-sqrt(2) (the root of g^2-4g+2) both sub-steps share the leading coefficient 2/(g*h) == *(2-g)/((1-g)*h) == 3.4142/h, so the Jacobian scaling does not change between stages. Order 2 and L-STABLE. Neither sub-step needed a new formula -- stage 1 is the existing trapezoidal order-2 rule at delta=g*h, stage 2 the Gear order-2 shape with unequal-step BDF2 coefficients -- and the two earlier charges stage 2 needs are supplied by ROTATING the state slots once mid-step (the rotation dctran already does on acceptance), the interior slot being spliced back out so the accepted history never contains the stage value. Self-starting, so it never needs the order-1 starter E-181 showed imposes an O(h^2) floor. (2) **`method=sdirk`** -- Alexander's 3-stage order-3 L-stable SDIRK, restricted to STIFFLY ACCURATE tableaux (a[s][j]==b[j], c[s]==1) so the last stage IS the answer; without that a step ends in a weighted COMBINATION of stage values and every value a device holds has to come out of a solve, not an assignment. Coefficients derived from gamma in code, since a mistyped Butcher digit does not crash -- it quietly costs an order. Fully implicit RK is out of scope: coupled stages need one *s*n x *s*n system. (3) **`method=adams`** -- Adams-Moulton at order `maxord`, weights derived PER STEP from the actual spacing (the textbook (5,8,-1)/12 are fixed-step and ngspice never takes fixed steps); normalising to tau=(t-t_n+1)/h keeps the Vandermonde entries O(1) where raw seconds would underflow. NOT stiffly stable above order 2. Error constants are derived, not borrowed: for an RK method R(z)=1+sum z^(k+1) b^T A^k e, so C = b^T A^p e - 1/(p+1)! -- a formula that reproduces the two constants ALREADY in cktterr.c exactly (1/12 trapezoidal, 1/2 backward Euler), which is what makes it trustworthy on new tableaux. TR-BDF2 0.04044 (trapezoidal's 1/12 overstated it 2.06x) and SDIRK 0.025897 (Gear-3's 3/22 overstated it 5.27x); the derived TR-BDF2 constant predicts a 2.06x per-step advantage and the fixed-step test

independently MEASURES 2.02x. What measuring showed: per STEP the new methods win (SDIRK observed order 2.76 vs 1.80 for the second-order trio, 1.26e-07 vs trap 1.66e-05 at h=2.5e-8), but per NEWTON ITERATION -- the honest cost unit, since TR-BDF2 solves twice per step and SDIRK three times -- trapezoidal or Gear wins three of four circuits, and Adams-3 wins the oscillatory non-stiff case by ~100x. Recalibrating the constants barely moved that, which is the finding: the (cost,error) curve belongs to the method, so a wrong constant costs predictability of `trtol`*, not efficiency. So TR-BDF2's case is L-stability alone -- a 1ns edge arriving at a step pinned to 20*tau leaves trapezoidal ringing 25 times and still 2.7e-02 off, while TR-BDF2 settles in 2 steps; trapezoidal's measured ratio is -0.8182, exactly the theoretical (1-h/2tau)/(1+h/2tau) = -9/11. None becomes the default. Two earlier claims corrected in the write-up: trap/gear do NOT hit an accuracy floor (that was a trtol sweep stopping at 0.5), and verify_syntaxhl.py was wrongly named as the orphan source. Explicit methods (forward Euler, Adams-Bashforth, explicit RK) are NOT included and cannot be: with ag[0]=0 a reactive element becomes a fixed current source and a series RC returns v_{n+1}=v_ n -- at Newton convergence the element's current IS state0[ccap] whatever ag[0] is, so charge, being the dependent quantity q(v), cannot be prescribed by a conductance+current stamp. OSDI is verified explicitly, since it loads its Jacobian with CKTag[0] ALONE while TR-BDF2 stage 2 and Adams order>=3 need ag[1]/ag[2] -- it routes its reactive residual through NIintegrate, and all five methods agree to 0.0e+00 between SPARSE and KLU on an OSDI device. Verify (examples/ integmethod_examples): 13 checks, both solvers; full regression 336/336.

E-418 *src/osdi/osdisetup.c, src/osdi/osdipzld.c, src/osdi/osdidefs.h, src/frontend/outitf.c, src/frontend/ com_measure2.c, examples/saveguard_examples/ (new)*

Four defects that share a shape: something was accepted, counted, or concluded, and never verified -- the fourth found by fixing the first, and older than it. (1) `absdelay()` and `last_crossing( )` read 0.0 whenever their result was only OBSERVED rather than contributed -- the E-415 open item, now CLOSED. They are the only operators whose output row the compiler deliberately leaves empty (ngspice fills it from its history buffer via elements `OSDIsetup` allocates later), so the node appeared in no descriptor Jacobian entry and E-116's decoupled-node scan tied it to ground; `eval()` then read `CKTrhsOld[0]`. A contribution -- even at weight 1e-30 -- is what put the node into a Jacobian entry, which is why only it rescued the value, and why `transition()` (same lowering) worked: its output feeds the slew residual. Confirmed rather than inferred: before the fix `n1#implicit_equation_1` does not exist as a circuit node. Five lines, idempotent for contributed models, no OpenVAF change and no ABI change -- E-415 had looked in the compiler and reverted when the descriptor came out byte-identical, correctly. The rule is now written at the site: a node coupled only by ngspice-side stamping must be marked there. (2) **`.save`** never validated a device name. `addSpecialDesc` only interns the string and `getSpecial`'s caller discards `INPaName`'s `E_NODEV`/`E_BADPARM`, so `@nosuchdev[i]`, `@r1[nosuchparam]` and `@*[i_p]` each produced a registered vector that stayed 0 long, silently, while `print`/`meas`/`wrdata` all report the same name loudly. The check now calls `INPaName` -- the very routine the per-point read uses -- and a HIERARCHICAL spelling is rewritten rather than rejected, so E-410's `@x1.n1[i_p]` yields a full waveform under the user's own spelling. Unresolvable names are warned about and still added (dropping them would change the plot; a bracket-less `@name` is a statistic on another path). Wildcards are diagnosed, not expanded: every wildcard mechanism in the tree is setter-side and none yields instance NAMES. Also corrects a round-25 note -- a bad NODE name is silently dropped too, so `.save` validated nothing. (3) **`meas ... when`** invented a time: `-inf`, 1.15292e+05 seconds in a 3 us run, negative times, silently. The initialisation counted a crossing in the FIRST interval and the evaluation -- which deliberately starts one sample in -- applied that leftover count to a later, crossing-free interval, dividing by a difference that was exactly zero or ONE ULP (hence every bad value being a power of two times a power of ten). The first interval spans the operating point to the first timepoint, so it straddles thresholds for reasons unrelated to the waveform: an earlier attempt to evaluate it properly broke `defaulttransition` (expected 5.0e-07, got 1.0002e-10), which is what identified starting one sample in as deliberate rather than as the bug. So the count is no longer taken, every interpolation is gated on the interval actually BRACKETING the target (

mirroring `measure_at`, which has always refused to interpolate outside a bracketing pair), and a `.dc` restart resets to sample 0 so the previous sweep's last point is never an endpoint. Clamping to `[tstart,tstop]` was rejected -- it would turn 1.15292e+05 into `tstop`, a plausible wrong answer in a delay measure. `cross=1` improves outright: garbage becomes the genuine later crossing at 1.00025e-06. (4) **`pz`** filled the simulator-stamped row nowhere. Those rows are the simulator's to fill, so every load path has to fill them -- `osdiload.c` does for dc/tran and `osdiacld.c` for ac, but `osdipzld.c` did for neither: the row was identically zero, the matrix singular at EVERY trial `s`, every trial looked like a root, and pz blamed the netlist ("the input signal is shorted on the way to the output", naming neither cause nor device). This predates (1) for a model that CONTRIBUTES the value -- its row was already live -- and (1) would have widened it to every observed-only model. The AC stamp cannot be reused: it is exact and bounded there (‘

E-417 *src/osdi/osdisetup.c, src/osdi/osdiparam.c, src/osdi/osdiinit.c, src/osdi/osdiregistry.c, src/osdi/osdidefs.h, src/include/ngspice/osdiitf.h, src/spicelib/analysis/cktsens.c, src/frontend/breakp2.c, examples/collapsestate_examples/ (new)*

A node collapse is decided once in **`setup_instance`**, and **`OSDItemp`** re-decides it on every temperature update without being able to rebuild anything derived from it. Three consumers reported over the mismatch. (1) `sens` printed ROUNDOFF as a derivative: with the collapse taken, four Jacobian entries map onto ONE matrix element, so the perturbed device's `+g/-g` (g = 1/delta = 1e6) cancel on that diagonal and what survives is `eps*E/(Y*delta^2)` -- `2.5289e-02` where the true derivative is `2.2676e-04` (112x; the hunt's original case was 14000x and sign-flipped), with a magnitude tracking `1/rd` and a SIGN that moves with unrelated parameters. It cannot be computed here -- `DEVsetup` runs only at the base value and re-running it would allocate nodes and trip cktsens's own `CKTlastNode` guard -- so it is reported as 0 with a warning naming instance and parameter, leaving every other row of the table untouched. (2) A `.dc temp` sweep ran the whole range on ONE topology, in both directions: sweeping INTO a collapse leaves the terminal floating (exactly 0 A), sweeping OUT of one keeps the collapsed topology so the series element never appears (1 mA where the truth is 500 uA -- a plausible 2x error, the more dangerous of the two). `.temp` and `set temp` were always right because each re-does the setup; the sweep now warns once per instance, naming instance, model type and temperature. Rebuilding mid-sweep is not a fix but a re-architecture (nodes would be allocated and deleted, `SMPmakeElt` called on an ordered and factored matrix -- a hard no under KLU -- and every other device's cached element pointers invalidated), and a hard ERROR would mean propagating five bare `CKTtemp(ckt);` returns in `dctrcurv.c`, changing error handling for every device model in the tree. (3) `.options savecurrents` expanded `@dev[i_<term>]` at three or more terminals but bailed out at exactly two, where `@dev[i_p]` stayed a length-1 SCALAR and `meas/plot` silently used one point -- the shape E-413 existed to remove, surviving at a different arity. The expansion is now unconditional and the bare `@dev[i]` E-394 defines is KEPT beside it, with synthesized names de-duplicated against explicit `.save` entries. The detection has one trap that is the whole fix: OpenVAF's collapse callback only ever STORES TRUE, so `OSDItemp` must CLEAR `collapsed[]` before re-deciding or a collapse that stopped happening is invisible -- precisely the direction the defect takes -- and must restore the snapshot before the error check, because the matrix implements the snapshot. E-416's own "deliberately not changed" note is corrected in place: its ~2e-3 terminal-vs-source gap was defect (3) measuring a scalar against a waveform (the plain R

E-416 *src/osdi/osdiparam.c, src/osdi/osdisetup.c, src/osdi/osdiregistry.c, src/osdi/osdiinit.c, src/osdi/osdidefs.h, examples/collapsecur_examples/ (new)*

A terminal current read exactly 0.0 whenever that terminal was COLLAPSED onto an internal node. `if (rd == 0) V(d,di) <+ 0; else I(d,di) <+ V(d,di)/rd;` is the standard compact-model idiom for an optional series resistance and `rd = 0` is the shipped default (BSIM `rdsmod=0`, HICUM `re=0`), so the broken path is the COMMON one. The model writes its current into `di`'s residual and terminal `d`'s own slot stays zero, while `OSDIask` read only `descr->nodes[t].resist_residual_off` -- 0.0 for a terminal carrying the device's full 2.001 mA,

through `@dev[i_<term>]`, `show` and `.options savecurrents` alike. With `rd` AND `rs` both zero EVERY terminal read 0.0 and the reported currents summed to zero, so a Kirchhoff check passed on `0 = 0`. The solution was never wrong: `osdiload.c` stamps a collapse group into one matrix row, so the collapsed and uncollapsed forms give identical node voltages and source currents -- which is why nothing a user plots ever moved. `OSDIask` now sums the terminal's whole collapse group (a group holds at most one terminal, since `collapse_nodes` refuses to merge two simulator-allocated terminals), resistive residual plus the integrated reactive residual in a transient, advancing the state cursor past non-members exactly as `osdiload.c` does -- the reactive half is load-bearing, not a refinement: 1 nF at 1 V/us makes the terminal current ~1 mA of pure displacement current against a few uA of conduction. The grouping cannot be recovered after setup, because `write_node_mapping` replaces the local mapping with GLOBAL node numbers and those cannot express a group -- two terminals on one net share a global number without being collapsed, and ground additionally collects grounded terminals, nodes collapsed to ground, E-116's decoupled internal nodes and E-401's dropped term-short flow nodes (grouping by global index makes a grounded terminal read 1.1 mA where 1.0 mA is correct). It is therefore snapshotted the moment `collapse_nodes` returns, into one `uint32_t` per descriptor node trailing `OsdiExtraInstData`, holding `t + 1` so the `calloc`ed zero still means "setup has not run". An uncollapsed terminal owns only itself, so every uncollapsed number is bit-identical to before; an unconnected terminal (E-402) is unowned and reports 0.0. A differential over the 40 real compact models in `integration_tests/` shows how common the broken path was: 23 bit-identical, 14 differing in terminal-current lines only (all previously all-zero: HICUM/L2, PSP103/_nqs/103t, ASMHEMT, BSIM3, BSIMBULK, BSIMSOI, HiSIM2, HiSIM-HV n4/n5, DIODE, DIODE_CMC, BSIMCMG), and nothing else moved anywhere -- the models that were already right (BSIM4 `rdsmod=1`, MEXTRAM 505, VBIC 1.3, BSIM6, PSP102, JUNCAP200, EKV, MVSG) are bit-identical, as an uncollapsed terminal owning only itself requires. One cosmetic consequence: the sum accumulates into `0.0`, so a residual of exactly negative zero now prints `0` rather than `-0` in `print/show` (numerically identical; nothing in the suite depends on it). Verify (examples/collapsecur_examples): 31 checks over four `rd/rs` combinations, a three-node collapse chain, a ground-collapsed internal node beside a grounded terminal, a capacitive transient, `show`, savecurrents and E-397's instance knobs; 13/31 on the pre-416 binary. The grown instance block was checked rather than assumed: size-vs-offset recomputed against all 589 descriptors in the repo, and the suites re-run under guard malloc (faults on any access past the allocation) -- clean. Both solvers; full regression 333/333.

E-454 *src/frontend/inp.c, src/frontend/subckt.c, src/spicelib/parser/inp2n.c, examples/autobusopt_examples/ (new)*

ONE OPTION, TWO READERS, TWO ANSWERS. `.option autobus` is decided in two places and they disagreed, so the same card switched the feature ON inside a subcircuit and off at the top level, or the reverse, depending only on the spelling. (1) The SUBCIRCUIT reader (E-449) must read the option cards directly, because the variable is not published until after expansion. Its `strstr` plus token-boundary test correctly ignored `noautobus`, `myautobus` and `autobusx` -- but = is neither alphanumeric nor _, so it passed as a clean terminator and the value was never read: `.option autobus=0`, `=false` and `=no` each switched the feature ON, silently. E-450 had already fixed this exact shape for `savecurrents`, which gets all four spellings right; autobus was its unguarded sibling. (2) The TOP-LEVEL reader (E-444, in INP2N) asks `cp_getvar(.., CP_BOOL, ..)`, but the spelling decides the published TYPE -- bare is a BOOL, `autobus=1` a NUMBER, `autobus=true` a STRING -- so only the bare form was ever seen and `.option autobus=1`, an ordinary way to write a boolean option and NOT reported as unknown, left a top-level bus port unbound. Both readers now answer by the same words: `e454_value_is_off` is the single list of off-words and `savecurrents` was moved onto it so the two cannot drift again. Precedence is deliberately matched to the OPTIONS MACHINERY, not to savecurrents -- within one card the later token wins, but across cards ngspice keeps the FIRST, and since the top-level reader can only follow what is published, making the card reader last-card-wins (as savecurrents is) would

put the two paths back into disagreement; both orders are pinned. A deck card now also beats a `set autobus` from .spiceinit, which used to be OR'd so an init file could turn the feature on and no deck could turn it off. (3) A bus could be bound by name at the level that DECLARED it but not passed down: `Xi a b inner` failed with "too few nodes" because the expansion was allowed on an OSDI device line only. It now applies to a subcircuit call too, so a bus travels by name through any depth. The subtlety is the node budget -- `numnodes` for an OSDI line counts the TOKENS written, so a token costs one however many nodes it expands to, while for an X line it is the CALLEE'S FORMAL COUNT, so a token standing for five nodes must spend five; `e449_expand_bus_port` therefore returns the number of actuals emitted instead of a flag. A short X call at the top level still fails cleanly (no enclosing formal list to take bits from), and with the option off X lines are untouched. The audit also confirmed much of the feature CORRECT and left it alone: ascending and descending `.subckt` bus declarations both match the explicit spelling, a bus port that is not first, a one-bit bus, array instances (E-441), the >1024-bit guard, and the sparse/over-wide formal lists which warn and error. Verified autobusopt 27/27 both solvers vs 15/27 on the previous binary, the twelve failures being exactly the defect checks while every control passes on both; autobus, subbus, busportsub, busdir, busoverflow, busname, busnodes, sweepbus, savecur, savecuroff and optname all still pass; regression 368/368.

E-478 *src/frontend/com_sweep.c, src/frontend/spec.c, src/frontend/evaluate.c, src/frontend/fourier.c, src/frontend/outitf.c, examples/guardmatch_examples/ (new)*

FIVE DEFECTS OF ONE SHAPE from bug-hunt round 46: a check was performed and then something ELSE was used -- a different parser, a different end of the range, a different lookup, or a different loop's state. (1) A COUNT WAS VALIDATED WITH A FLOAT PARSER AND CONSUMED WITH **`atoi()`**, which stops at the `e`. `sw_isfinitenum` accepts `2e2` as 200 and `1e6` as 1000000; `atoi` returned 2 and 1, and the float the validator had already computed was thrown away. So `sweep lin 2e2` ran TWO points, `montecarlo 2e2` drew two samples and printed a yield computed from them, `highsigma 2e2` the same, and `sweep lin 1e6` ran ONE point while the identical number written `1000000` was correctly refused as too many. Five sites read a count this way (sweep, montecarlo, highsigma, wcd `-maxiter`, wcd `-is`); they now share `sw_count_arg()`, which reads the value the validator produced. The same block also SILENTLY REWROTE a count below 1 -- for `lin` a one-point run, for `dec`/`oct` a silent change of the SPACING rather than the count, and on a `-vs` knob the collapse of a whole sweep dimension -- while every sibling names it (`ac` *number of points is invalid*, `dc`, `tran`, montecarlo, highsigma, wcd, `.for`). E-270's *too many points* wording is kept verbatim because `sweepbounds_examples` pins it. (2) **`spec`** CHECKED ITS STEP AGAINST THE SPAN BUT NEVER AGAINST ZERO (`stepf > stopf - startf`, upper end only): a negative step makes `fpts = (stopf-startf)/stepf + 1` NEGATIVE, that count reaches the allocator (*can't allocate -784 bytes*), and the returned NULL is taken into the fill loop -- a DETERMINISTIC SIGSEGV (EXC_BAD_ACCESS at 0x0 in **`com_spec`**), present in the shipped binary; threshold at about -1e4. The guard now also requires finite and positive. (3) INDEXING A **`@dev[param]`** WAVEFORM RETURNED THE DEVICE'S LIVE VALUE, NOT THE ELEMENT: `e448_literal_index()` builds the name `<base>[<k>]` and asks `vec_get()` for it -- a clean miss for an ordinary base, which is what makes E-448's probe safe -- but **`vec_get`** answers a name beginning with **`@`** from the DEVICE rather than the plot, so `@c1[i][50]` was read as *parameter i of device c1*, the index discarded, and every element of a saved waveform read back as the value after the run (`@c1[i][50]` and `@c1[i][218]` were EQUAL). `length()`, `maximum()` and `wrdata` were right all along -- they take the vector whole and never build that name, which is why this survived; E-441's `@r[2][resistance]` is unaffected because the lexer keeps it in ONE token. (4) **`fourier`** GUARDED A FUNDAMENTAL BELOW THE TIME SPAN AND NOT ONE ABOVE THE SAMPLE RATE, printing a full report whose every normalised magnitude was `nan`, summarised as *THD: nan %*, in silence. It now WARNS rather than refuses, because E-445's suite pins that a large but FINITE fundamental still runs -- the run stands and the nan is explained. (5) A LOOP COMMAND RUN AS ANOTHER'S **`-analysis`** OVERWROTE THE SHARED PROGRESS STATE (E-477): the inner `begin()` took the outer's label, total and index and the inner `end()` cleared `outp_`

`loop_active` for both, so the outer bar vanished and the line was left reading *montecarlo: sample 6/6 100%* while the outer sweep still had points to go. A nested begin is now counted and ignored so the OUTER loop keeps the line -- the same reasoning that made this feature not reuse the per-analysis bar. Display only; the numbers were byte-identical throughout. NOT CHANGED, pinned by checks: `.four 1e30` still runs, and **`psd 0`** still clamps to 1 -- because it ANNOUNCES it (*Number of averaged data points: 1*); round 46 reported that as a sixth, silent clamp and was WRONG, because the probe filtered for error/warning keywords while this is a plain informational line. Suite `guardmatch_examples` 21/21 (4/21 against the shipped pre-fix binary, the four being exactly the pinned decisions).

E-477 *src/frontend/outitf.c, src/frontend/com_sweep.c, src/include/ngspice/fteext.h, examples/loopbar_examples/ (new)*

A PROGRESS LINE FOR THE COMMANDS THAT RUN N ANALYSES IN A LOOP -- `sweep`, `montecarlo`, `highsigma`, `wcd`. All four set `ft_optimizing` to silence per-point chatter (Enhancement-130) and `outp_print_reference()` returns early on that flag, so all four printed a banner and then nothing at all: a forty-point sweep of a slow transient looked hung for minutes. LETTING THE INNER ANALYSIS BAR THROUGH IS THE OBVIOUS FIX AND THE WRONG ONE -- it runs 0 to 100% for EVERY point, so it resets N times and never answers the question being asked (how far is the SWEEP), and it redraws the same terminal line with a carriage return, so an outer bar and the inner bar would overwrite each other. But the inner number is not worthless either: with three points and a one-minute transient each, an outer-only bar sits at `1/3` and reads as frozen. So ONE LINE CARRIES BOTH -- `sweep: point 7/40 [=====  ] 17% (tran 63%)` -- and the inner fraction also advances the OUTER bar within the point, so it moves smoothly instead of stepping once per analysis and the percentage is continuous across a point boundary (32% at the end of point 1 of 3, 34% at the start of point 2). TWO DRIVERS ARE REQUIRED AND NEITHER COVERS BOTH REGIMES: while a point runs the loop command is blocked inside the analysis, so only `outitf.c` -- on the analysis's own data path -- can refresh; but the DEFAULT analysis is **`op`**, which produces no swept data points and never reaches that path, so the command also draws at each point boundary. The suite pins one driver each: a check that the percentage advances WITHIN point 1, and a plain `op` sweep where only the boundary driver can fire. Both draws use the same 0.25 s throttle as the analysis bar, since a sweep may run to `SW_MAXPTS` -- a 4000-point `op` sweep finishing in 69 ms draws twice, which is correct. **`wcd`** IS INDETERMINATE: it iterates a Hasofer-Lind search to convergence, usually well before `maxiter`, so it gets a counter rather than a bar that would sit low and jump to done; its line is released right after the loop, because the **`ft_optimizing`** restore it first used runs at the END of that function and put the last frame UNDERNEATH the worst-case distance it was meant to precede. THE FIRST POINT'S FRAME IS NOT DRAWN: a frame ends with a carriage return, leaving the cursor at column 0 of a line filled to a constant width, so anything printed before the next redraw overwrites only its LEADING columns and the rest survives as a tail -- and the first analysis prints the solver announcement, which is shorter than a frame (`Using SPARSE 1. 3 as Direct Linear Solver  ]  0%`). Every LATER point is safe because its frame follows the preceding analysis's output; only the first has nothing in front of it, so it waits for the first intra-point refresh or the second point's boundary. The check that pins this had to be written twice: the residue does NOT exist in the byte stream (the text after the last carriage return is just the banner, so the line must be COMPOSITED the way a terminal does), and **`subprocess(text=True)`** applies universal-newline translation that turns every carriage return into a newline before the check can see it -- both earlier versions passed against the build that had the bug. Auto-enabled only when stdout is a terminal (the line is redrawn in place; a redirected run would collect one enormous line of frames -- the existing analysis bar does NOT make that distinction, which is why capturing a plain `tran` to a file yields a screenful of frames); `loopbar`/`noloopbar` force it either way and all eight spellings are tested, because Enhancements 450, 451, 454, 466 and 467 each shipped an option here where a spelling meaning OFF turned the feature ON. Two checks allocate a real pty, since auto-on-a-terminal is the default path and is

otherwise indistinguishable from forced-off. NOT CHANGED: a standalone `tran/ac/dc/noise` keeps its own bar (`progressbar_examples` still 22/22), `optimize` stays silent, and a sweep's numbers are byte-identical with the bar on and off. `outp_bar_shown` is deliberately left alone by the new branch so `outp_finish_reference()` cannot stamp a stray 100% "Reference value" line over the loop line at the end of every point. Suite `loopbar_examples` 18/18 (5/18 against the shipped pre-fix binary).

E-476 *src/osdi/osdinoise.c, src/osdi/osdiparam.c, src/osdi/osdiload.c, src/osdi/osdidefs.h, src/frontend/spiceif.c, examples/reportguard_examples/ (new)*

FOUR DEFECTS OF ONE SHAPE from bug-hunt round 45: the simulator's account of itself did not match what it does. None raised an error. (1) AN OSDI OPERATING-POINT VARIABLE ANSWERED WITH A NUMBER WHEN NOTHING HAD COMPUTED ONE -- the opvar storage lives in the `calloc`'d instance block, so a read after *op simulation(s) aborted*, with no analysis run at all, or after a `$fatal` killed an evaluation part-way, returned a clean 0.0 -- and 0.0 is a perfectly ordinary current, voltage or conductance, so nothing distinguished it from a result. **`i(v1)`** in the SAME **`print`** correctly reported *vector ... is not available*, and the equivalent built-in read produced nothing -- OSDI was the only channel manufacturing a plausible answer. ngspice already had the rule and applies it elsewhere: **`param_forall()`** in **`device.c`** will not even ask for an ask-only parameter unless **`ckt->CKTrhsOld`** exists, which is why **`show`** never displayed a fabricated opvar -- the direct `@dev[opvar]` read was the one path without that guard. A per-instance bit is now set in `eval()`, the SINGLE FUNNEL every evaluation passes through, gated on `$fatal` because an evaluation that raised one was abandoned part-way; `OSDIask` returns `E_NOTFOUND` (the parameter exists, it has no value) and `doask()` lets the handler's own message stand instead of adding a vaguer one. Parameters are deliberately NOT gated -- they are inputs. (2) EVERY OSDI DEVICE'S INTEGRATED NOISE TOTAL WAS ADVERTISED AND UNREACHABLE -- `onoise_total_<dev>` and `inoise_total_<dev>` were built with a `" "` suffix instead of `""`, so `tprintf` stored the name with a TRAILING SPACE: `display` pads the name column so the blank is invisible and the vector looks present and *1 long*, while every read matches literally and misses. Its own N_DENS sibling eleven lines above already passed **`""`**, and every built-in passes a names array whose element 0 is **`""`** -- which is why **`onoise_total_r9`** read back and only OSDI's did not. The per-source children and the grand total were unaffected, so a noise run looked complete and only the per-device attribution -- the number a contributor ranking is built from -- was missing. These were the ONLY two occurrences of `" "` in the whole `src/osdi/` tree. (3) AN INSTANCE-SCOPE WRITE TO A ROUTED MODEL PARAMETER WAS DISCARDED IN SILENCE -- E-397 routes ngspice's own `dt/dtemp/temp` knobs onto the model's parameter when the model declares one (deliberate: PSP, MEXTRAM, VBIC, HiSIM and BSIM all declare `dtemp`), and that also places the name in the instance table, so `alter @n1[dtemp]=20` found it, stored the value in `inst->dt` where nothing reads it once the model owns the name, and said nothing while `@n1[dtemp]` went on reporting the old value. Every other model-scope parameter reaching that setter already returned **`E_BADPARM`** -- only the two routed names were quiet. `param < num_params` separates a routed id from the ids the loader synthesizes above the parameter/opvar/terminal ranges; it is now refused with E-467's wording, issued from `OSDIparam` because `doset()` discards the setter's return code at the `alter` call site. An INSTANCE-scope `dtemp` -- the corpus spelling -- never reaches that branch and is unaffected, `altermod` still works, and the READ still routes to the model's parameter by design. NOT CHANGED, pinned by checks: the routed read, case-sensitive `$simparam` matching, and an opvar staying readable after a LATER analysis fails once one has succeeded. LEFT OPEN: `@n1[mul]` reads 0.0 before any analysis rather than its declared default, because OSDI parameter defaults are applied during setup -- same family, predates this work, needs its own evidence. Suite `reportguard_examples` 31/31 both solvers (17/31 against the shipped pre-fix binaries); full regression 390/390.

E-475 *src/spicelib/devices/vsrc/vsrcload.c, src/spicelib/devices/isrc/isrcload.c, src/spicelib/analysis/transetp.c, src/spicelib/analysis/cktsopt.c, src/frontend/inpcom.c, src/frontend/measure.c, exam-*

ples/explicitvalue_examples/ (new)

SEVEN DEFECTS OF ONE SHAPE from bug-hunt round 44: a value the deck STATED was discarded and something else quietly put in its place, or a refusal named a fault other than the one it found. None raised an error. (1) **sin** WITH AN EXPLICIT **freq=0** TOOK ITS FREQUENCY FROM TSTOP -- `functionOrder > 2` already asks the only question that matters (was a frequency SUPPLIED?), but an `&& coeffs[2] != 0.0` made an explicitly-written zero fall through to `1/CKTfinalTime`, so `sin(0 1 0)` became ONE CYCLE PER SIMULATION and lengthening the run changed the stimulus (first 0.9 crossing 1.78/3.57/7.13/17.8 us for TSTOP 10/20/40/100 us, exactly linear; an explicit 50 kHz stays at 3.56 us). **TD** and **THETA** on the next two lines test only **functionOrder** -- FREQ was the odd one out, `PULSE` directly above DOCUMENTS its own zero-substitutions in a comment and `SINE` had none, and unlike a zero rise time a zero frequency is meaningful: it is DC. Omitting the frequency still defaults to 1/TSTOP. The identical line in `isrcload.c` is fixed with it. (2) AN UNKNOWN PARAMETER ON A SUBCIRCUIT CALL WAS SILENTLY DROPPED -- `X1 a 0 div rr=5k` against `.subckt div p n r=1k` built a 1k divider, the default, exactly as if nothing had been passed; one transposed character and the circuit is not the one written. The two sibling constructs both speak: an unknown param on a **.model** WARNS and on a device instance is a hard ERROR -- only the subckt call was quiet. Now a warning (not an error: the name is unusable in the body either way, so nothing that works stops working); `m` exempt. (3) A FAILED **.measure** LEFT THE PREVIOUS ANSWER UNDER ITS NAME -- and in a loop that is the last iteration's number standing in for a measurement that never happened, undetectable because **sim_status** is not set by **meas**. The name is now dropped, so a read after a failure says *not available* -- which is already what happens when the FIRST measurement fails; only that case was honest before. Same family as E-438. (4) **tran** TMAX HAD NO VALIDATION AT ALL: a negative one reached the integrator and came back as *singular matrix: check node b*, blaming a trivial RC that solves fine with any other TMAX, while TSTEP/TSTOP/TSTART all say *is invalid, must be...*. (5) SIX OPTIONS ACCEPTED NONSENSE IN SILENCE -- `pivtol`/`pivrel` were the only tolerance-family members with no check (reltol/abstol/vntol/trtol/chgtol refuse <=0, gmin/gshunt refuse negative) and `minbreak`/`srcsteps`/`gminsteps`/`ramptime` had none while sibling counts itl1/itl2/itl4 refuse a negative; all six now use the same `E426_BAD_OPT` path, `pivrel` additionally refusing >1 because Sparse silently clamps anything above 1 back to its own default. Zero stays legal wherever zero is its meaning. (6)+(7) TWO E-474 REFUSALS NAMED THE WRONG FAULT (defects in an enhancement shipped hours earlier, contradicting its own one-fault-one-message rule): every unevaluable **{{ }}** reported *outside any .for loop* -- 13 ways of being wrong, all INSIDE a loop -- now split by asking whether anything remains that a later binding could fill in (`{{1/0}}`, `{{1+}}`, `{{}}` named where they stand; a surviving NAME left for an inner loop and, if still there at the end, reported as *never resolved*; only a deck with no `.for` at all keeps the original message); and a nested **.for** reusing the enclosing index produced duplicate lines and died with *device already exists*, naming the symptom -- now refused during the walk that already looks for the matching `.endfor`. THREE LOOKALIKES FROM THE SAME ROUND ARE DELIBERATELY NOT CHANGED, each re-confirmed by READING THE CODE rather than the behaviour: negative R/C/L stay unflagged (E-438: *negative passives are the very idiom this project's own examples use*), `pow(-4,0.5)`=2 and `1/0`=1e32 stay finite (E-256/446: *a NaN here poisons the Newton Jacobian*), and `pulse tr=0` still takes the timestep (documented in a comment above the code) -- checks [20]-[22] pin them so a later round does not *fix* them. LEFT OPEN: no `pivtol` value changes a result on either solver, including 1e30 which should reject every pivot, though the plumbing into `spOrderAndFactor`'s (Rel,Abs) order is correct and AbsThreshold IS consulted in the pivot search -- validating an input is a separate question from whether it then does anything. Suite `examples/explicitvalue_examples/` 41/41, both solvers; full regression 389/389.

E-474 *src/frontend/inpcom.c, examples/forloop_examples/ (new)*

NEW **.for** / **.endfor** NETLIST CONSTRUCT. A deck often needs a run of near-identical instance lines differing only by an index -- a ladder, a stack of periodic sections, a bank of taps -- and writ-

ten out by hand they are long and wrong in ways that are hard to see: one node name out of step in the middle of forty lines still parses, still simulates and still gives an answer. `.for i in range(1,4)` + `XP{{i}} P{{i}} P{{i+1}} hl_periodic n1={nL}` + `.endfor` now expands to exactly the four instance lines. Forms: `range(first,last)`, `range(first,last,step)` and an explicit list `[7,2,9]`. **`range`** INCLUDES BOTH BOUNDS -- **`range(1,4)`** is FOUR iterations, not Python's three -- deliberate, because an index range in a netlist reads as *1 through 4* the way a bus range does, and it has its own check because it is the one thing a Python-literate reader assumes wrongly. `range(3,1)` counts down; a step contradicting the bounds is refused rather than silently yielding nothing. `{{i}}` is the index and `{{expression}}` is integer arithmetic over it (`+ - * / %`, parens, unary minus), substituted as TEXT so it builds node names, instance names, `.model` names and parameter values alike. Loops NEST, and an inner bound may be an expression over an outer index (`range(1,{{i}})`) -- which falls out of the design: one binding is applied at a time, so `{{i+j}}` becomes `{{2+j}}` and waits for the inner loop. WHY **`{{ }}`** AND NOT **`{ }`**: numparam owns single braces and the body carries BOTH kinds; doubling the brace needs no ordering rule from the user, and the pass removes every `{{ }}` it introduces so numparam never sees one. WHY IT RUNS FIRST: at the top of `inp_readall`, before the syntax check, the scope tree, numparam, bus and subcircuit expansion -- it emits ordinary netlist lines and nothing else, so every later stage sees what the user would have typed and none of them needs to know the construct exists (`.include`/`.lib` are already pulled in, so it works in an included file and inside a `.subckt`). The cost of that placement is that the BOUNDS CANNOT BE **`.param`** VALUES, and that is refused by name rather than misread. `.control` is skipped entirely -- it has its own `foreach`. ONE FORWARD PASS SUFFICES: copies are appended AFTER the closing `.endfor`, never into the body still being read, which is the use-after-modify E-441 records for the array expander; the walk then continues into the copies so a nested `.for` is reached later in the same pass. 14 refusals, each producing exactly ONE message -- a header that fails to parse also retires its own `.endfor`, or the walk reports a second error about a line that is perfectly correct. THE SUITE'S ORACLE IS THE HAND-WRITTEN DECK: every check runs the `.for` version and the lines it stands for and requires them identical -- expansion line for line, solved result character for character -- because *expands to something plausible* is the failure that matters and an analytic comparison would not catch a ladder wired one node out of step (a 2000-section ladder is built both ways). Suite `examples/forloop_examples/` 31/31, both solvers; full regression 388/388.

E-473 *src/frontend/com_sweep.c, examples/mcarming_examples/ (new), examples/reuseloops_examples/verify_reuseloops.py*

THE MONTE CARLO FAST PATH ARMED ON A DRAW IT COULD NOT PUSH -- A SILENTLY WRONG *AND UNSTABLE* YIELD. E-346's fast path re-draws the random values and pushes them into the live circuit instead of re-sourcing per sample, which is sound only if EVERY use is pushed. Only a BRACED expression is ever captured, and NUMPARAM DECIDES THE BRACES: a quoted `rd='rv'` reaches the capture pass as `rd={(agauss(...))}` and is captured, but a BARE `v=rv` in a B-source reaches it as `v= ( agauss(...) )` with NO BRACES -- `sw_fp_scan_valueline()` walked past it, found no swept token and reported the line ELIGIBLE. A B-source's value is substituted TEXTUALLY AT PARSE TIME, so nothing short of a re-source moves it and every sample after the first saw the FIRST draw. A 40-sample run whose spec depended only on that value reported 100% or 0%, DIFFERING BETWEEN RUNS OF THE SAME DECK AND SEED (the frozen value depended on surviving state), where the correct sampled answer is ~45% -- `rv ~ N(100, 60)` against a `0 <= v <= 100` spec is P = 0.452. Same family as E-438, where a Monte Carlo counted failed samples as PASSES. REACH: the deck must ARM at all, so it needs a capturable random value ALONGSIDE the uncapturable one (`.model rd='rv'` plus `B1 bs 0 v=rv`); a bare `.param` name is not legal as an ordinary device value (ngspice rejects `R1 a b rv`), so the hole is confined to expression-valued elements. FIX: a random draw OUTSIDE ANY BRACES is ineligible -- the identifier walk already rejected a SWEPT token found outside a brace, it now rejects a random call there too, so the deck disarms and takes the reset path. Decks that were always fine are untouched (a quoted `.model` value still arms, a braced device value still arms) and **`v=rv`** /

v='rv' / **v={rv}** -- the same circuit written three ways -- now AGREE. THE **-warm** GATE, AND WHY E-188's SUITE CAUGHT IT: not tearing the circuit down also leaves the previous sample's SOLUTION in place, which WARM-STARTS the next one -- and E-188 made that OPT-IN deliberately (it keeps its guess in a buffer OUTSIDE the CKTcircuit precisely because a reset destroys the solution, and `DCop` asks MODEINITJCT rather than MODEINITFLOAT when it is off). An unconditional reuse cut E-188's COLD path from 20606 to 1416 iterations per sample -- the homotopy that option exists to avoid, avoided WITHOUT BEING ASKED. Every yield still matched exactly (215/215, 214/214, LHS 258/258, KLU 215/215), so nothing was wrong -- but a starting point decides which operating point a sample finds on a circuit that has more than one, and MC samples are meant to be independent. Silently turning an opt-in on is the fault E-450/451/454/466 each shipped once already, so the reuse is offered exactly where the user already asked for state to carry between samples. WHAT IT UNBLOCKED: E-472 gave `optimize` the E-471 setup reuse but withheld it from `montecarlo` precisely because the per-sample teardown was all that limited this defect's damage. With arming honest, arming MEANS every varying value is pushed -- the guarantee `sweep` already relies on -- so `montecarlo` keeps the circuit standing too (1.29x on a 1200-instance ladder, yield unchanged) -- BUT ONLY UNDER **-warm**, with every E-471 guard unchanged: *setup reused at 8 of 20 samples, 11 rebuilt after a node collapse moved*, yield 50% with the reuse on and off alike on a deck whose spec passes only a COLLAPSED sample. Suite `examples/mcarming_examples/` 20/20, both solvers; E-472's two `montecarlo` checks flipped from *never reuses* to *does*; full regression 387/387.

E-472 *src/frontend/com_optimize.c, src/frontend/com_sweep.c, src/frontend/com_sweep.h, examples/reuseloops_examples/ (new)*

E-471 GAVE THE SETUP REUSE TO **sweep** AND NOTHING ELSE. `optimize` never re-sources the deck for `-param`/`-mparam` knobs -- nor for `-dparam` once E-322's fast path arms -- yet still tore down and rebuilt a circuit it had not touched on EVERY evaluation, hundreds of times in a fit. It now asks for the same reuse on the same terms: only when a circuit is standing that this evaluation did not re-source, never after an analysis that failed, never under `-center` (its inner Monte Carlo resets per sample). Every guard is E-471's, unchanged. THE WORK WAS PROVING IT HOLDS WHEN A SEARCH STEP MOVES THE COLLAPSE, NOT A SWEEP POINT -- E-417's `cs_gate` collapses at exactly `rd == 0`, which a sweep can step onto but an optimizer never lands on, so a new `cs_thresh` collapses below a THRESHOLD and a search range straddling it moves the topology on its own: *setup reused at 15 of 17 analyses, 1 rebuilt after a node collapse moved*, with the identical residual, evaluation count and parameter as the reuse-off run. THE TRAJECTORY QUESTION IS REAL BUT BOUNDED: on a well-posed fit `lm` and `nm` both return the identical parameters to 13 digits in the identical evaluation count (the FD Jacobian step is 1e-3 normalised against a 5e-12 perturbation, so the derivative error is ~5e-9), but a DEGENERATE fit -- unreachable target, pinned at a bound -- took 8 evaluations with the reuse and 17 without, same residual to ten digits, because LM's accept/reject test was comparing costs that agree to 1e-12 and its damping diverged on noise. **montecarlo** WAS BUILT, MEASURED AT 1.29x, VERIFIED COLLAPSE-SAFE (11 rebuilds in 20 samples, the collapse-sensitive yield tracking draw for draw at 50%) AND THEN TAKEN BACK OUT: its fast path can ARM WHILE A RANDOM `.param` STILL HAS A USE IT CANNOT PUSH (a B-source value is substituted textually at parse time), and the per-sample rebuild is only what accidentally limits the damage. THAT DEFECT IS ALREADY LIVE WITH NO REUSE INVOLVED -- on the SHIPPED binary such a deck reports a 100%/0% yield that DIFFERS BETWEEN RUNS OF THE SAME DECK AND SEED, where re-sourcing every sample gives the correct 45%. Seeding the fast path's closure from the random `.param` names -- what makes the equivalent sweep disarm -- did NOT disarm it, so the fix lies further in, in what pass 2 accepts as a capture; it needs its own change and its own evidence. `montecarlo` is left exactly as it was and the suite asserts it, so it cannot be switched on by accident. `highsigma`/`wcd` re-source every sample by design and never have a setup to keep. Suite `examples/reuseloops_examples/` 17/17, both solvers; full regression 386/386.

E-471 *src/spicelib/analysis/cktdojob.c, src/frontend/com_sweep.c, src/osdi/osdiparam.c, src/osdi/osdisetup.c, src/include/ngspice/cktdefs.h, src/include/ngspice/osdiitf.h, examples/reusesetup_examples/ (new)*

A REPEATED ANALYSIS TORE THE WHOLE CIRCUIT DOWN AND BUILT IT AGAIN FOR EVERY POINT even though only a parameter VALUE had changed. `.dc` has never done that -- including the parameter sweeps of E-427 it sets up ONCE and walks its points inside the analysis. After E-470 removed the quadratic teardown, what remained of that per-point rebuild was still most of the run: a 1001-point sweep over a 2448-unknown stack, 7.24 s -> 0.57 s (12.8x) under SPARSE with BYTE-IDENTICAL results (all 5005) and 6.35 s -> 1.38 s (4.6x) under KLU, where reusing the matrix ordering can put a solve on a slightly different Newton path -- exactly as it can under `.dc`, which has always reused both -- and moved ONE number of 5005 by 4.7e-09 relative, five orders of magnitude inside `reltol`. THE TWO SOLVERS SWAPPED PLACES: KLU was the faster of them on this deck before the change (6.35 s vs 7.24 s), but reuse lets SPARSE skip the `spOrderAndFac-tor` E-470's profile left dominating at 51% and KLU gains less from the same trick, so SPARSE is now 2.4x the faster. With E-470 the deck went 29.78 s -> 0.57 s. WHY IT COULD NOT SIMPLY BE DONE: NODE COLLAPSE. A device may decide, at setup and from its parameters, to merge two of its nodes, and the matrix is built for that topology; reuse the setup and the topology FREEZES at whatever the first point decided. A FIRST VERSION DID EXACTLY THAT -- sweeping `cs_gate`'s `rd` across the value that collapses its internal node returned `0.5 0.5 0.5 0.5 0.5` instead of `0.5 0.3333 0.25 0.2 0.1667`, no error and no warning, a plausible flat line. It was measured at 3.5x, confirmed numerically sound on the deck that motivated the work (which simply never moved the collapse) and REVERTED. THE FIX IS SAFE BY CONSTRUCTION, NOT BY ARGUMENT: on a reused point `CKTdoJob` skips the unsetup/setup pair but STILL RUNS `CKTtemp`, which for an OSDI device is what re-decides the collapse and compares it against the snapshot the matrix was built from (E-417) -- so collapse is ALWAYS re-decided, never assumed -- and if any instance reports a change it throws the reused matrix away and does a genuine unsetup/setup/temp. THIS IS THE THING E-417's OWN WARNING SAID WAS IMPOSSIBLE (*the matrix was built for the collapse decided at setup and cannot be rebuilt here*); the warning is suppressed while the request is live, with its once-per-instance flag left unburned. A BUILT-IN device decides its collapse in `DEVsetup` and nowhere else, so there is nothing to re-check and no honest way to detect a change: reuse is offered ONLY to circuits built entirely from types whose topology is known fixed (the linear elements, the four controlled sources, `ASRC`, the independent sources) plus OSDI, and anything else keeps the old behaviour exactly. A FAILED POINT HANDS NOTHING ON -- THE REGRESSION FOUND THIS, NOT THE DESIGN: a point whose analysis fails leaves a matrix factored to NaN and states half-written, and reusing it carried the wreckage into every later point -- E-445's `guardgaps` suite requires the FORBIDDEN points of a range-violating sweep to be NaN and the LEGAL ones to keep real values, and with the first cut all five were NaN (KLU failed the whole sweep). Reuse is now declined after any failed point; `sw_run_failed()` from E-438 already knew, so the fix was to carry its answer one iteration forward. Sweeps driving `tran`, `ac`, `noise` and `dc` are each checked identical on and off. THE PIVOT-ORDER REUSE IS NOT E-439's HAZARD: `NIiter` already refactors with the existing ordering between Newton iterations, and BOTH solvers force a full reorder on a singular refactor. Default on; `reusesetup=0`, `noreusesetup` and the `set` form turn it off -- and because E-450/451/454/466 each shipped an option here whose off-spellings silently meant ON, the value is read as a NUMBER and a STRING before `CP_BOOL` and every spelling is a check. **`set ngdebug`** now reports the decision (*setup reused at 3 of 5 points, 1 rebuilt after a node collapse moved*), which is what lets the suite assert the MECHANISM rather than infer it from a clock the circuits are far too small to move. FOLLOW-UP FIX: the option WORKED off a `.options` card but was also reported as an *unknown option ... ignored* -- `if_is_option()` did not list it. That is exactly the failure E-445's note on that list describes (a warning that fires on a setting the run then honours teaches the user to ignore the check), and the deck timing is what exposed it: 0.55 s with the option vs 6.81 s, i.e. demonstrably not ignored. `reusesetup`/`noreusesetup` registered, and the suite now asserts no `.options` spelling is called unknown. Also hardened the REGRESSION

RUNNER: several suites need numpy/matplotlib, and an interpreter without them (most easily by putting `/usr/bin` ahead of the real python3 on PATH) did not say so -- they exited `rc=1` in 0.0s and the sweep ended with ~22 unrelated-looking FAILUREs (every `pyplot` suite plus a few that import numpy directly), which has cost a full 13-minute run more than once. `run_regression.py` now preflights those imports and exits 2 with one clear line naming the interpreter. Suite `examples/reusesetup_examples/` 37/37, both solvers; full regression 385/385 at release, 387/387 after E-472/E-473.

E-470 *src/spicelib/analysis/cktdltn.c, src/osdi/osdisetup.c, src/include/ngspice/cktdefs.h, examples/teardown_examples/ (new)*

TEARING AN OSDI CIRCUIT DOWN WAS QUADRATIC IN ITS NODE COUNT. `CKTdltNNum()` finds the node it is asked to delete by scanning the circuit's node list from the HEAD, and `OSDIunsetup()` calls it once per internal node -- O(k*N), and EVERY repeated analysis pays it: a `sweep`, an `optimize`, a `montecarlo`, anything that runs an analysis more than once. A profile of a 1001-point parameter sweep over a 2448-unknown circuit found 77% of the ENTIRE RUN inside it -- not in the solve, not in setup, but in the teardown BETWEEN points (`com_sweep -> sw_run_cmd -> dosim -> if_run -> CKTdoJob -> CKTunsetup -> OSDIunsetup -> CKTdltNNum`). THE SCOPE FOR THIS WORK WAS WRONG, AND INSTRUMENTING IS WHAT SHOWED IT: it was planned (and shipped as a scope note beside E-469) as *move `sweep`'s loop into the analysis kernel the way `.dc` does*, on the theory that the ~20 ms/point gap between a sweep point (~30 ms) and an `op` analysis (9.4 ms) was per-point SETUP. That note put instrumentation first and said in as many words that the whole case rested on it -- the profile overturned the hypothesis, and the kernel-loop rewrite would have been a large, risky change aimed at the wrong 20 ms. The fix: `OSDIunsetup()` knows every node number it wants gone before it deletes any of them, so it marks them and ONE walk of the list removes them all -- O(N) for the whole unsetup, in a new `CKTdltNodeSet()`; `CKTdltNNum()` is unchanged for every other caller. A SECOND PROFILE WAS NEEDED: the first version sized its mark array from `ckt->CKTmaxEqNum`, which SHRINKS as nodes are deleted, and `OSDIunsetup()` runs once per model TYPE -- so the second type's numbers sat above the now-smaller bound, failed the range test and fell back to the per-node path, giving 23% with `CKTdltNNum` still at 64%. The bound is now the highest node number actually present, taken by walking the list once. Measured per sweep point: 5 periods 1.7 -> 1.2 ms (1.4x), 10 periods 4.0 -> 2.3 ms (1.8x), 25 periods / 2448 unknowns 32.9 -> 7.6 ms (4.3x) -- the speedup GROWING with circuit size is the signature of removing a quadratic -- and the full 1001-point deck 29.78 s -> 7.12 s with byte-identical results. 7.6 ms/point is close to the 9.4 ms a single `op` reports, the honest ceiling here. Teardown fell from 80% to 9% of the run, what remains of it is `IFdelUid` rather than list scanning, and `spOrderAndFactor` now dominates at 51%. Reusing the matrix ORDERING between points is the next opportunity and is deliberately NOT taken: that is what `klu_refactor` does, and E-439 recorded it reusing the old pivot order with NO pivoting and NO singularity test, producing a NaN solve on a circuit SPARSE handled -- that hazard has to be answered in its own change. Verified teardown 11/11 both solvers, nine of the eleven checks about the numbers and the node bookkeeping rather than the clock (a teardown that frees the wrong node, frees one twice or leaves one behind corrupts the NEXT analysis instead of failing loudly), the two timing checks asserting SCALING rather than milliseconds; regression 384/384.

E-469 *src/frontend/dotcards.c, src/frontend/inp.c, src/frontend/spiceif.c, src/include/ngspice/fteext.h, examples/saveused_examples/ (new)*

`.option saveused` -- SAVE ONLY THE VECTORS THE CONTROL BLOCK ACTUALLY READS. An analysis stores every node at every point unless a `save` says otherwise. On a small circuit that costs nothing; on a large one it IS the run -- a 2448-unknown dielectric stack over a 201-point parameter sweep costs 104.73 s (521 ms/point) with everything stored and 7.22 s (35.9 ms/point) with a hand-written `save` of the four vectors it writes, a factor of 14.5 from one line the author has to remember and then keep in step with the `wrdata` beside it; nothing warns when the two drift apart. With `.option saveused` and no `save` line at all the same deck runs in 7.08 s (35.2 ms/

point) and its written results are byte-identical to the hand-saved run. WHAT IS COLLECTED IS DELIBERATELY WIDER THAN THE LETTER OF THE REQUEST: every `v(...)`, `i(...)` and `@dev[param]` reference ANYWHERE in the block, not merely the arguments of the output command, plus the plain node names given to one. The reason is correctness -- `let r = v(out) - v(mid)` followed by `wrdata f r` names only `r`, and `r` is not a node, so a narrow scan would leave the two vectors that build it unsaved and a deck that worked before would fail. Under-saving turns a performance option into a wrong answer, so the scan errs the other way; over-saving costs a little memory. It STANDS ASIDE ENTIRELY -- the run is exactly what it would have been -- for an explicit `save`/`.save` (the author has already said what they want, and quietly adding to it would make their line mean something other than what it says), for `all` as an argument, and for a block with no output command; the explicit-`save` case is asserted IDENTICAL with the option on and off. Runs in `ft_saveused()`, called immediately after `ft_dotsaves()` and immediately before the controls execute -- that ordering is the design: the explicit `.save` is already installed so the stand-aside test can see it, and no analysis has run so the list is complete before it is needed. The option cards are read in `inp.c` beside `autobus`, because the option variable is not published until **`inp_dodeck()`** runs -- reading it any other way is what E-454, E-462 and E-464 each had to repair here. NOT CALLED **`autosave`**, THE NAME ASKED FOR, BECAUSE E-192 ALREADY OWNS IT for `set autosave=<file>`, a checkpoint written when a transient is interrupted -- and a checkpoint FILENAME is a string that is not an off-word, so reading `autosave` as a boolean here would have switched vector filtering on for every deck using E-192's checkpointing, silently discarding nodes it never asked to lose. The first implementation did exactly that and its suite file overwrote E-192's; the tell was that the regression suite count did not rise when a suite was added. E-192's suite is restored and passing, and the two options are pinned independent. All ten spellings pinned on and off, per the four repairs (E-450/451/454/466) this class has already needed. TWO CORRECTIONS SINCE IT SHIPPED. (1) A WILDCARD ACCESSOR IS A KNOB, NOT A VECTOR: the scan takes `@dev[param]` from every line, and the commonest one in a control block is the sweep knob itself -- `sweep @*[wavelength_nm] ...` -- which `save` cannot expand, so a real deck that dropped its `save` line (exactly what the option is for) got "a wildcard device name is not expanded here, so this vector will stay empty" once per sweep point. Results were right; the noise was this feature's own invention. Wildcards are no longer collected, a NAMED accessor still is. (2) THE 14.5x NO LONGER HOLDS FOR SWEEPS, BECAUSE E-470 REMOVED ITS CAUSE: that factor was never the cost of STORING vectors, it was the quadratic node teardown between sweep points. On the same deck at 1001 points today: hand-written `save` 6.67 s / 88.4 MB, `saveused` 6.71 s / 83.6 MB, no save at all 6.72 s / 86.6 MB -- identical, and identical again at 5001 points. The surviving benefit is TRANSIENTS, where every timepoint accumulates into one plot instead of being torn down: a 601-node chain over 20001 timepoints peaks at 56.6 MB with no save, 9.2 MB with a hand-written one and 9.3 MB with **`saveused`** -- six times less, scaling with nodes x timepoints. Verified saveused 26/26 both solvers, comparing vector NAMES rather than counts so a check cannot pass on the right number of the wrong vectors; regression 384/384.

E-468 *src/frontend/com_fft.c, src/frontend/com_measure2.c, src/frontend/numparam/xpressn.c, src/spicelib/analysis/cktsens.c, src/spicelib/parser/inpdpar.c, src/spicelib/parser/inpgmod.c, src/spicelib/parser/inppas3.c, src/xspice/icm/analog/limit/cfunc.mod, examples/mathguard_examples/ (new)*

SEVEN NUMBERS THAT WERE WRONG -- TWO OF THEM IN CODE WITH NO TEST COVERAGE AT ALL. **`psd`** reported a total power set by the WINDOW, not the signal: a constant 1 V has total power exactly 1 V^2, and psd said 1.499908 under the default hanning window, 1.362744 hamming, 1.726652 blackman, 1.215261 gaussian -- and 1.000000 under **`specwindow=none`**, which is what identified the cause. `fft_windows` scales every window for unit COHERENT gain, which is right for the amplitude spectra `fft` and `spec` produce; a PSD sums SQUARED bins and needs the POWER gain, sum(w^2)/length. Zero padding inflated it again by N/length -- E-241 corrected this normalisation from N^2 to length^2, and the padding factor it removed belongs on ONE of the two lengths, not neither -- so the same signal reported 1.5 V^2 at one stop time and 3.0 V^2 at

another, a total power that depended on where the transient happened to end. Parseval over the padded sequence gives sum

E-467 *src/frontend/variable.c, src/frontend/subckt.c, src/frontend/inpcom.c, src/frontend/spiceif.c, src/frontend/com_measure2.c, src/frontend/numparam/xpressn.c, src/frontend/numparam/numpaif.h, src/spicelib/analysis/cktsopt.c, src/spicelib/parser/inpdpar.c, src/spicelib/parser/inp2n.c, examples/silentaccept_examples/ (new)*

ELEVEN SILENT ACCEPTANCES -- THE DECK SAID ONE THING AND NGSPICE QUIETLY DID ANOTHER. The root of a class four enhancements had patched one site at a time: an option's SPELLING decides its published TYPE (bare a BOOL, `=1` a NUMBER, `=true` a STRING) and `cp_getvar`'s coercion table had no CP_BOOL case at all, so all ~110 `cp_getvar(..., CP_BOOL, ...)` readers saw only the bare word. `set sqrnoise=1` reported V/sqrt(Hz) where the bare word gives V^2/Hz; `set interp=1` left 165 transient rows where the bare word interpolates to 101; `set autostop=1` ran to the end. Answered once in `cp_getvar`, so `e454_autobus_var` and `autoadapt_mode` stop being the only readers that get it right; a number is on when non-zero, so `=0` still means off. The fix found its own hazard: a cascade that leads with CP_BOOL to mean "was the bare flag given?" worked only while `cp_getvar` refused to coerce, so `autoadapt_mode` returned "on, quiet" before it could see `=debug` -- it now reads the STRING first, and two existing suites caught this, which is what they are for. One word, one unreachable feature: `case OPT_DEFAS` assigned `TSKdefaultMosAD`, so the source-area default could NEVER be set and asking for it silently overwrote the DRAIN area (`defas=7e-10` gave `@m1[ad]`=7e-10, `@m1[as]`=0). An instance temperature below absolute zero was accepted although `.option temp=` has warned since E-426 and `osdisetup.c` makes the same check for OSDI: `R1 ... temp=-300` answered -0.998 V from a +1 V source, `temp=-1e6` the same and `dtemp=-400` by the other road -- the guard sits in `INPdevParse`, the one place every built-in applies its `name=value` pairs, and -25 C stays an ordinary temperature. The KiCad bit spelling never reached the SUBCIRCUIT formals: E-462 made a bit `a_0_` rather than `a[0]`, INP2N learned it and `subckt.c` did not, so formals written the way KiCad actually emits them matched nothing and the device floated at 1.0 against 0.5238095 -- with no not-connected warning, because every terminal WAS connected, to nodes nothing drives; two readers of one rule for the third time in this feature (E-454, E-464, here), and the bracket spelling is still always accepted. **`.func sqrt(x)`** silently replaced the built-in for the whole deck (2236.07 became 100000, `ln` 621.46 became 150000, `abs` +1000 became -998); the user's definition still wins, checked against `fmathS` exported rather than copied. **`.adapt`** validated the adapter MODEL but not its NODE list, so one typo switched the feature off and the deck answered the unadapted 0.7560976; reported per member, so `.adapt b, nosuchnode` still adapts `b`, and naming a model that is also a device model is reported too while our own injected `n_adapt<N>_` lines stay silent for idempotency. **`meas`** lost every WINDOW function over a **`.dc`** of a device parameter -- max/min/avg/rms/integ all said "out of interval" at any range and with an exact from/to, while find-when and when worked on the SAME plot -- because the dc scale was a fixed list of four names and a parameter sweep is called `param-sweep`, the plot's `[default scale]` all along; the list is now a fast path with the plot's own scale behind it, written once where it had been duplicated. And **`alter @dm[is]`** blamed the parameter ("no such parameter is") when the cause was that `dm` is a model. Four candidates withdrawn rather than shipped: the instance wildcard does NOT drop `resistance` writes -- the probe measured a voltage-divider ratio, invariant when `@#*` moves every resistor together, and the current shows it working (-5e-4 to -2.5e-4); out-of-range vector indexing does warn; and a NEGATIVE-GEOMETRY GUARD WAS WRITTEN, TESTED AND THEN WITHDRAWN -- `.option defw=-1e-5` and an instance `w=-1e-5` each flip `i(vd)` from -1.8e-04 to +1.8e-04, but E-438's `.option warn_physics` ALREADY reports both routes and deliberately KEEPS the value, and the new guard's ignore-the-value contract silenced it; E-438's own suite caught that, which is what it is for. Three of the four were rejected by re-verifying the measurement, the fourth by the regression suite -- both cheaper than shipping the fix. Verified silentaccept 42/42 both solvers, roughly half the checks pinning what must NOT change, including E-438's contract; regression 381/381.

E-466 *src/spicelib/parser/inp2n.c, examples/adaptquiet_examples/ (new), examples/autoadapt_examples/*

`.option autoadapt` IS QUIET BY DEFAULT, AND ITS OFF-WORDS FINALLY WORK. E-463 reported per NODE -- a line for every adapter injected and another for every node that did not qualify -- which on a deck with many shared bus nodes buried the run's own output. The reporting is now opt-in: `.option autoadapt=debug`. ERRORS ARE NEVER SILENCED: a missing or wrong-shaped adapter model, a name collision, the same node on both ports of one device, `autoadapt` without `autobus` -- each means the option cannot do what the deck asked for, and a deck that asked for an adapter and did not get one must not run on in silence. Only the two INFORMATIONAL classes moved behind `=debug`: the per-split note and the per-node did-not-qualify notices. Found while making the value mean something: E-463 never looked at the value, only at its PRESENCE, so every spelling that means off turned the feature ON -- `autoadapt=0`, `=false`, `=no` and `=off` each injected an adapter and moved `v(a[0])` from 0.7560976 to 0.7590361, a deck silently gaining an adapter its author had just switched off; only `noautoadapt` worked. This is the FOURTH appearance of that defect in a sibling option: E-450 found `savecurrents=0` and `nosavecurrents` both turning it on, E-451 found `nocshunt=` moving a node by six orders of magnitude while printing "unknown option", E-454 found `autobus=0` meaning ON at the top level and off inside a subcircuit. The word list is the one they share, `e454_value_is_off` -- a new boolean-ish option in this code base should be assumed to have this bug until its off-words are tested. An unrecognised value (`autoadapt=bogus`) is reported once and then proceeds quietly, the same shape E-462 gave an unknown `autobus` style. Verified adaptquiet 22/22 both solvers, every check testing the VALUE as well as the text -- because "quiet" has to mean the messages stopped and not that the adapters did; E-463's own suite now asks for `=debug` where it inspects what the feature did, 26/26 otherwise unchanged; regression 380/380.

E-465 *src/frontend/com_sweep.c, examples/sweepsubfast_examples/ (new)*

`sweep` STOPPED TEARING THE CIRCUIT DOWN FOR SUBCIRCUIT AND DERIVED PARAMS. `sweep` has two ways to move a `.param` knob: E-320's FAST path writes each point straight into the live circuit, while the fallback stages `alterparam` and issues a **`reset`** -- re-sourcing the WHOLE DECK once per point. Measured 1.58s vs 0.50s on a 3000-element deck at 201 points and 0.54s vs 0.07s on 800 subcircuit instances, the gap widening with both deck size and point count because the rebuild is O(deck) per point. Almost anything involving a subcircuit or a derived parameter fell off the fast path, and silently -- there is a banner when it arms and nothing at all when it does not, so one unused `.param b={rv*2}` anywhere in a deck turned a 0.50s sweep into 1.58s with no indication why. Now on the fast path: derived params and chains, a param passed on the X line, two instances carrying DIFFERENT expressions, nested X calls, a swept param in a `.subckt` header default, a derived `.param` inside a subcircuit, and a local shadow beside a real dependence. The obstacle was that a subckt param is bound PER INSTANCE: `r.x1.r1` and `r.x2.r1` share ONE template -- keyed by source line -- but need different values, so a single global `nupa_eval_expr("r")` cannot serve both. Nothing is evaluated per instance at run time: each captured expression is rewritten ONCE at build time into global names by walking the instance's scope chain outward exactly as numparam would -- the subcircuit's own `.params` first (they may reference its formals), then the formals bound to that call's actuals or to the header default when the call omits them, then the enclosing frame -- so `{r}` becomes `{(rv)}` and the point loop is untouched. A local `.param` shadowing the swept name resolves to its own constant, which is what makes the shadow correct rather than merely refused. numparam has already rewritten the call by the time the original deck is readable -- `X1 a 0 sub r={rv}` arrives as the positional `x1 a 0 sub {rv}` -- so the `.subckt` header supplies the parameter ORDER; E-442 met the same rewrite from the other side. Derived TOP-LEVEL params needed one more thing: matching used the swept names only, so a device carrying `{rd}` was never captured; it now uses the transitive CLOSURE of derived names, the values having been refreshed by `nupa_recompute_params` since E-320. SECOND HALF -- the reset path never

put the knob back. E-385 closed that hole for the fast path and left it open here: `alterparam` rewrites the DECK TEXT, and nothing re-sourced it, so the devices kept the LAST POINT's values and every later analysis was quietly wrong -- after `sweep rv 900 1100` a fresh `op` read `v(a) = 0.99099099` with `@rtop = 1100` where nominal `rv=1000` gives `0.9900990099`. Fixed with a `reset` after the nominal `alterparam`, which the next comment in that same function already assumed this path would do. Still falling back deliberately: structural dot-cards (`.if`/`.temp`/`.tran`), which change the deck's shape, and an ISOLATED local shadow, pinned by E-321. Two name collisions were hit while building this, both from the matching set growing beyond the swept names: a DEVICE called `Rd` collided with `.param rd` so a line matched on its NAME and the scan disarmed on a line carrying no value (matching now skips the leading instance/model token), and a deck TITLE beginning with `X` was read as a subcircuit call whose callee could not be found, disarming the whole sweep -- the near-identical deck beside it passed only because its title began with "t". Verified sweepsubfast 28/28 both solvers, every case against an ANALYTIC value rather than the other code path; regression 379/379.

E-464 *src/frontend/subckt.c, src/frontend/subckt.h, src/frontend/inp.c, src/spicelib/parser/inp2n.c, src/include/ngspice/inpdefs.h, examples/busmix_examples/ (new)*

A BUS FORMAL AND A LOCAL BUS ON THE SAME OSDI INSTANCE LINE GAVE A WRONG ANSWER. Inside `.subckt s a[0] a[1] a[2] a[3]`, the line `N1 a b mymodel1` had E-449 expand `a` into the caller's four actuals -- the formals exist -- while `b`, having none, stayed ONE token. The line then carried five node tokens where autobus needs two (one per port) or eight (one per terminal); being neither, INP2N expanded nothing and the tokens bound positionally: `a[0..3]` correctly, `x1.b` onto `b[0]`, and the top three bits dangling. Measured 1.0 against 0.5238095 for the same circuit flattened by hand. It warned -- `3 of the 8 terminals ... are not connected` -- but the warning named the missing terminals rather than the cause, and the deck still ran. The fix: once ANY port on the line has been expanded from formals the line cannot still be in shorthand, so every remaining bus port is expanded at the same point, to `x1.b[k]` -- exactly the node INP2N would have produced, and the same one a `b[0]` written elsewhere in the subcircuit translates to, so references still unify. The widths are reachable at flattening time even though the model TABLE is not: `pre_osdi` has already registered the module and the deck's own `.model` cards map a model name onto it, so `.model mymodel1 chan` plus `INPtypelook("chan")` gives the IFdevice, and the port grouping is `INPbusPorts` -- INP2N's own function, exported rather than copied, so the two cannot drift. A SECOND DEFECT SURFACED WHILE FIXING THE FIRST: the initial version spelled the local bus's bits with brackets even under `.option autobus=kicad`, so `x1.b[k]` never met the `b_0_` written beside it and the bits floated -- the very failure being fixed, in a new place. The cause is the one E-454 already repaired in this same option: the **`autobus`** variable is not published at flattening time, which is the entire reason `inp_set_autobus()` exists, so asking `INPbusKicadStyle()` there silently returned "brackets" for every deck. The style now travels with the flag, same precedence -- a deck card wins, otherwise a `set autobus=kicad` from `.spiceinit`. Two readers of one option disagreeing, for the THIRD time in this feature's history. Only mixed lines change: all-formal, all-local, bus-BASE-formal (`.subckt s2 a`, which always worked) and scalar-beside-bus forms are each pinned unchanged. Note for E-463: a mixed line now leaves flattening fully explicit, and `.option autoadapt` only considers shorthand lines, so a local bus shared across mixed lines is not auto-adapted -- a narrowing of reach, not of correctness, recorded rather than fixed. Verified busmix 13/13 both solvers, every check a differential against the flat circuit on a ladder where all four bits differ; subbus 16/16, autobus 12/12, autobuskicad 27/27, autoadapt 26/26 and busportsub 9/9 all still pass on the changed path; regression 378/378.

E-463 *src/frontend/spiceif.c, src/frontend/inpcom.c, src/spicelib/parser/inp2n.c, src/include/ngspice/inpdefs.h, examples/autoadapt_examples/ (new)*

`.option autoadapt adapter=<model>` -- INJECT A TWO-BUS-PORT ADAPTER BETWEEN TWO OSDI DEVICES THAT SHARE A BUS NODE. `N1 a b m1` / `N2 b c m2` becomes `N1 a b_f m1` / `N2 b_r c m2` plus `n_adapt1_ b_f b_r <adapter>`, instead of the deck renaming the shared

node on both instances and keeping the halves consistent by hand. It runs between INPpas1 and INPpas2 -- the only seam where all three requirements hold at once: pas1 has built the model table, so a line's PORT STRUCTURE (and therefore "is this token a bus node, and how wide?") is knowable, which a textual pass in inpcom.c cannot answer; subcircuits are already flattened, so "inside a subcircuit" needs no second implementation, the bill E-449 paid to give autobus one; and INP2N has NOT run, so the rewrite is at TOKEN level and `.option autobus` then expands all three lines -- the bus handling is a consequence of the seam rather than extra work. The forward side comes from the PORT INDEX, not the deck order: a SPICE deck is order-independent, and making a reordering change the circuit would be a worse bug than the one being fixed, so the suite reverses the two instance lines and requires the same answer. Every non-qualifying case is REFUSED AND REPORTED rather than guessed at -- a node also touched by a resistor (three occurrences), three OSDI ports, both ports of one device, mismatched widths, a missing or wrong-shaped adapter, `autoadapt` without `autobus`, an existing `b_f/b_r`. Three defects the prototype surfaced. The adapter's `.model` card was DELETED by `inp_rem_unused_models` before the pass could use it -- nothing references the adapter until the instance is injected -- and the symptom blamed the injected line (`Unable to find definition of model`) rather than the card silently removed thousands of lines earlier; the cull now protects the model named by `adapter=`. The pass was NOT IDEMPOTENT: a deck already carrying adapters had them adapted in turn, `b_f` becoming `b_f_f/b_f_r` with a second adapter between and the answer moving from 0.7590 to 0.7647. And a reference to a bus BIT is a use of the bus -- the occurrence count looked only for the bare token, so `Rb0 b[0] 0 1k` was invisible and the resistor was left on an orphan node, silently; both `b[0]` and E-462's `b_0_` now count. Optional `.adapt n1, n2` restricts the node set by WHOLE TOKEN (a substring test would make `.adapt bb` select `b`), accepting a flattened node's trailing component so one card covers every instance of a subcircuit. Known limitation, tracked separately: an instance line mixing an expanded bus FORMAL with a local bus node is not adapted -- correct for an interface port, and it falls out of the exactly-twice rule -- but that configuration is ALREADY broken without this feature (control: adapter hand-written, autoadapt off, `3 of the 8 terminals not connected`, 1.0 where the flat equivalent gives 0.5238095), because E-449 expands the formal into four actuals while the local bus base stays one token. Verified autoadapt 26/26 both solvers, every value check a differential against the hand-written deck on a ladder where every bit differs; regression 377/377.

E-462 *src/spicelib/parser/inp2n.c, examples/autobuskicad_examples/ (new)*

`.option autobus=kicad` -- THE BIT SPELLING A SCHEMATIC CAN ACTUALLY WRITE. E-444 names an expanded bus port's terminals `a[0]`..`a[4]`, copying the bracket text from the model's own terminal names -- right everywhere except the one place a bus port is most useful: a schematic. KiCad's SPICE exporter rewrites every `[` and `]` in a net name to `_`, so a sheet labelling a wire `AA[0]` puts `/AA_0_` in the netlist; measured, with multi-digit indices intact (`ZA[10]` -> `/ZA_10_`), while KiCad's INTERNAL net name keeps the brackets -- the `kicadxml` export shows them -- so the rewrite belongs to the SPICE exporter alone and nothing on the schematic side can prevent it. `/AA_0_` never unifies with autobus's `/AA[0]`, so under KiCad a bus port's bits could not be labelled (the label made a DIFFERENT node and the bit floated), wired to ordinary parts (a resistor drawn to `AA[0]` landed on `/AA_0_`), or plotted -- the simulator's signal list is built from schematic nets, so it offered `/AA`, which after expansion has no device on it at all, and answered `vector V(/AA) not or not yet available`. The only workable sheet was one where every net was a whole bus. The option changes the generated SPELLING and nothing else: indices still come from the model's terminal names, so a port declared `[4:1]` still expands 1..4 and never invents an `a_0_`. Only one reader synthesises a name -- `inp2n.c` concatenates token and index; the subcircuit path (`e449_expand_bus_port`, E-449) maps a bus base onto the FORMALS the `.subckt` line already declares, so it has no spelling to choose, and the suite pins both modes giving bit-identical answers there rather than leaving it to reasoning -- two readers disagreeing about one option is exactly what E-454 had to repair here. An unknown style is REPORTED: `autobus=kicad2` would otherwise fall back to the default spelling and keep solving, with the bus bound to fresh `a[k]` nodes and every

a_k_ dangling at the source voltage -- a plausible-looking number as the only symptom, the silent-degenerate shape E-447/451/455 each had to go back and fix. Verified autobuskicad 27/27 both solvers, every check a differential (each spelling must produce ONLY its own node names, on a ladder where all five bits differ); measured end to end through KiCad's own `kicad-cli` netlist and through `libngspice`, ERC clean; regression 376/376.

E-461 *src/frontend/inpcom.c, src/spicelib/parser/inpgstr.c, examples/strparam_examples/ (new)*

A QUOTED STRING PARAMETER VALUE WAS CORRUPTED BEFORE THE MODEL SAW IT. `.model mm dut(ty="PMOS")` reached the Verilog-A model as **`pmos`**, and `ty="with space"` as **`with`** -- the remainder then reported as "unrecognized parameter (space)". So a string parameter could never match a mixed-case value, and a `from '{...}` set written in upper case rejected every one of its own members (the compiler half of that report is the companion row in openvaf_changes_full-report.md). Two causes. The netlist reader lower-cases whole lines, with case retention wired only to Cider `.model` cards, `ic.file`, and a FIXED LIST of XSPICE code models -- an OSDI model card matches none of them -- and the existing helper only preserves a line carrying EXACTLY ONE pair of quotes, which a model card with two string parameters already exceeds. Separately `inp_casefix` turns quotes into SPACES unless the line is `.param`, `.subckt` or an X line, which is what cut the value at its first space. A quoted value on a model card or a device instance line is DATA -- a selector compared with ==, a file name, a `from` set member -- not an identifier SPICE may fold, so both paths now keep it verbatim, on the card, on its + continuations and on instance lines, gated on the line actually containing `=\"` so nothing else changes. A trap inside the fix: adding `.model` to `keepquotes` made the case WORSE before it made it better -- the quoted text came back lower-cased even though the quotes now survived, because with `keepquotes` set the code never stepped PAST the opening quote, so the skip-to-closing-quote loop exited immediately and the tail of the same loop lower-cased the value one character at a time, keeping the quotes while losing the case they exist to protect. One `else string++;`. Verified strparam 33/33 both solvers (value checked verbatim on model cards, continuation lines and instance lines); regression 375/375.

E-480 *src/spicelib/parser/inpgmod.c, src/frontend/com_measure2.c, src/maths/cmaths/cmath2.c, src/frontend/control.c, src/frontend/inp.c, src/frontend/inpcom.c, src/frontend/spiceif.c, src/include/ngspice/cpextern.h, src/spicelib/analysis/dctrcurv.c, src/spicelib/devices/tra/trasetup.c, src/xspice/icm/analog/limit/cfunc.mod, src/xspice/icm/analog/pwl/cfunc.mod, examples/gatecheck_examples/ (new)*

A CHECK THAT COULD NOT FIRE WHERE IT MATTERED -- most of round 49 is not a MISSING check but a written one made unreachable by something upstream. THE DUPLICATE-PARAMETER TEST ON A **`.model`** CARD FAILED TWICE OVER: it was gated on the tracking list not being FULL, so once every distinct parameter had been seen once a repeat was never even LOOKED UP and a device with a single model parameter could never report one (`.model mm dut(r=1k r=4k)` took 4k in silence); and it counted the model TYPE token as a parameter, because a built-in card leaves that token in the line for the parse while an OSDI card has it removed -- harmless where the type is not also a parameter name (`d` is not a diode parameter, which is why `d(is=..)` was always clean) and A FALSE ACCUSATION ON THE MOST ORDINARY CARD THERE IS for `r`, `c` and `l`: *.model rmod r(r=1k)* was told *parameter 'r' is set more than once ... remove one.* **`.measure`**'s EDGE COUNT COLLIDED WITH A SENTINEL -- the field's *not given* and *LAST* markers are -1 and -2, so `fall=-1` was read as *no fall given* and answered with A RISING EDGE (0.5 ms where `fall=1` correctly gives 1.5 ms), `cross=-2` was read as LAST, and `cross=-3` simply failed; the guard tests the TOKEN THE USER WROTE, since by the time the value is a number `LAST` has already become -2. `%` DISAGREED WITH ITSELF: `.param` and a B-source call fmod, while the control-language operator took floor(fabs()) of BOTH operands and did an integer `%` -- `(0.5) % 3` was 0, the value gone entirely, and `(-5) % 3` was 2 where C and `.param` give -2; the manual lists `%` as *modulo* and gives a SEPARATE `\` for *integer divide*, and E-273's RANGE CHECK IS KEPT so `1e30 % 5` still errors as mathcast_examples pins. AN UNTERMINATED CONTROL BLOCK WAS CHECKED PER LINE, NEVER AT THE END OF THE SECTION, so an

`if` with no `end` swallowed every command after it and exited 0 in silence while a stray `end` has always been reported; the state is now compared BEFORE AND AFTER feeding the section, because `reset` inside a running `dowhile` re-enters that path with a block legitimately open -- a first version tore down the live structures there and hung lhs_examples. Guards that were genuinely absent: a transmission line's `nl/f` (`f=0` returned A TABLE OF nan PRINTED AS AN ORDINARY AC RESULT, every row, rc 0), a switch's `vh/ih` (the only members of that model missing from the physics table), the code-model PWL's breakpoint ORDER (it checked lengths, and the SOURCE `pwl` warns about exactly this), a duplicate subcircuit call parameter (an UNKNOWN one already warned), and a `.dc` step larger than its span. And the limiter's reversed-limits message carried CLIMIT's `TIME != 0` guard, which is never true in `op` or `dc` -- silent in those analyses and 214 messages in a transient; both its faults are model-card properties and now fire once at INIT. NOT CHANGED, each reported and withdrawn on reading the code: `vector(-4) == vector(4)` (cx_vector documents the MAGNITUDE), a negative `pulse` TR/PW (vsrcload documents negative = use the default), `.dc` with start == stop (E-426: 13 decks rely on it) and `ac lin 1` (span.c: nine cards use it). AND ONE FIX BUILT, MEASURED AND REVERTED: the constant plot IS writable -- `let pi = 3` redefines pi for the session while `destroy const` is refused -- but shadowing the write into the current plot leaves READS still resolving to the constant first, and **`run`** is itself a built-in constant that lhs_examples writes and loops on, so its `dowhile` never advanced; making that safe means changing name resolution across the interpreter, far wider than the evidence, and check [13] pins the behaviour as it stands. Suite `gatecheck_examples` 47/47 (24/47 against the shipped pre-fix binary); full regression 394/394.

E-481 *src/spicelib/parser/inp2n.c, src/frontend/spiceif.c, examples/silentports_examples/ (new)*

`.option silentports` -- AN OPT-OUT FOR A NETLIST NOBODY TYPED. E-402 made an omitted OSDI terminal audible and that stays the DEFAULT, for good reason: an omitted terminal looks exactly like a typo, and it DANGLES rather than grounding, so the natural assumption is the wrong one. What it did not account for is a netlist written by a TOOL -- a schematic front end emits the short form for every instance of a model that declares an optional thermal port, and the author cannot add the pin from the schematic, so a five-device sheet collects TWENTY-FIVE LINES about a choice nobody made. KiCad's SPICE exporter is the case that prompted it. The option suppresses that warning and nothing else: the terminal is still absent, `$port_connected()` still reports 0, the branch still carries no current, and the circuit solved is the same one. Opt-IN, so every existing deck is untouched, and a front end that cannot edit netlists can ship `set silentports` in `.spiceinit` instead. IT SILENCES A WARNING; IT DOES NOT MAKE AN ILL-POSED CIRCUIT WELL POSED -- measured on misc/bsimbulk_thermal_repro, the reproducer E-402 was decided on: with `t` written as `0` the deck is clean either way, and with `t` OMITTED the option removes the five warning lines and leaves all six **`singular matrix: check node n1#t`**, because BSIM-BULK pins its thermal node with a POTENTIAL tie-off so an absent terminal carries no current. That is E-402's decided territory and the answer is unchanged -- write `0` for the pin -- and checks [7]-[9] pin the distinction so a later reader does not take `silentports` for a fix. NO openvaf-r CHANGE: the warning is raised entirely inside ngspice, in INP2N, from `numnodes < *dev->terms` against the terminal count already carried in the `.osdi` descriptor; compiling a model with an unused optional port the compiler prints nothing at all. REGISTERED IN BOTH PLACES -- `if_is_option()`'s name list AND the type dispatch beside it: the first build had the option working while `.options silentports` still printed *unknown option ... ignored*, which is E-451's defect and is how the second registration point was found. Every OFF spelling is tested, because `cp_getvar(.., CP_BOOL, ..)` reports a merely PRESENT variable as true and `=0`, `=false`, `=no`, `=off` would each have turned the feature ON -- the defect E-450, 451, 454, 466 and 467 each shipped exactly once. The suite compiles BOTH model shapes from source (thermal network contributed unconditionally, which stays well posed with the port absent; and one gated on `$port_connected`, whose node floats) so it exercises the real compile-and-simulate path rather than a canned `.osdi`. Suite `silentports_examples` 24/24 (11/24 against the shipped pre-fix binary); full regression 395/395.

E-482 *src/spicelib/parser/inp2n.c, src/frontend/spiceif.c, examples/groundports_examples/ (new)*

.option silentports=ground -- GROUND THE TERMINALS A NETLIST LEAVES OUT. E-481 turned the absent-terminal warning off and that job is unchanged here -- the bare card still means exactly what it shipped and `silentports_examples` still scores 24/24 -- but what E-481 could not do was make the deck RUN. An omitted OSDI terminal is NOT GROUNDED: `INP2N` binds every terminal the line did not reach to `-1` and `osdisetup.c` builds a private node `<inst>#<term>` for each, UPSTREAM behaviour that E-402 only started saying out loud. Ten of the twelve corpus models with an optional pin tie it off with a POTENTIAL contribution (`Temp(t) <+ 0.0`), which puts nothing into a node ngspice allocated itself, so the operating point dies on `singular matrix: check node n1#t`. E-402's answer was write **`0`** for the pin, and a schematic front end that hides the pin cannot write anything -- on the E-402 reproducer, silencing removed five warning lines and left all six singular-matrix reports, quiet and still broken, which is worse because the warning had been the only clue. THE OPTION WRITES THE **`0`**: every omitted terminal is bound to ground in the parser, so `$port_connected()` reports 1, the skipped branches are built with the node held at 0, and no private node is created. `four.cir` now returns `i(vd) = -6.58515e-07`, EXACTLY the hand-grounded `gnd.cir` value, and ngspice's own three broken BSIM-BULK decks go from 36, 12 and 12 singular reports with every sweep aborting to zero of both. THREE STATES, AND THE BARE CARD KEEPS E-481's MEANING: bare -- equally `=dangle` or `=quiet` -- silences and leaves the terminal DANGLING, held to the warned default VALUE FOR VALUE and on the gated shape just as singular; `=ground` alone binds the pin, asked for BY NAME because it changes the circuit; `1/true/yes/on` mean the bare card, `0/false/no/off` turn it off. THE READ ORDER IS LOAD-BEARING: `silentports_mode()` asks `cp_getvar` for the CP_STRING FIRST, CP_REAL next, CP_BOOL LAST where it now means only the bare card -- because E-467 gave `cp_getvar` a CP_BOOL COERCION answering TRUE for any value that is not an off-word, right for a two-state option and fatal for a three-state one since it swallows every value word. Measured with the BOOL query still first: `=quiet` GROUNDED the terminal, and so did `=bananna`. An unrecognised word is named once per distinct spelling and falls back to the DEFAULT, not to either ON state, following ngspice's own `.options method=banana` handling. WHY THE PARSER: `osdisetup.c` reads the `-1` sentinel to count connected terminals and passes it to `setup_instance`, which is what the model reads back through `$port_connected` -- bind in `INP2N` and the count, `$port_connected`, the absent private node and the untouched collapse machinery all follow; `INPpas2` inserts ground before any device card so `"0"` never creates a node, checked with the OSDI card FIRST in the deck and from inside a SUBCIRCUIT (global ground -- where a front end actually puts the instance). EVERY omitted terminal is grounded, not just the first: a two-optional-terminal model reads the current back to prove the LAST one was too. Grounding is NOT stock ngspice behaviour and never was -- the pre-E-402 `inp2n.c` binds `-1` and `osdisetup.c`'s private-node loop arrived with the vanilla import untouched by this fork. NO openvaf-r CHANGE. Suite `groundports_examples` 59/59 (41/59 against the E-481 binary; eighteen checks discriminate, every one about the terminal's STATE rather than the message), `silentports_examples` 24/24 unchanged; full regression 396/396.

E-483 *src/spicelib/analysis/dcpss.c, src/frontend/com_qpss.c, examples/hbconv_examples/ (new)*

`set qpss_tol` / **`set qpss_maxiter`**, AND HARMONIC-BALANCE STALL DETECTION. `QPSShb` was called with its convergence bound and iteration cap COMPILED IN (`0, 0, 60, 1e-10`), and `tol` is an ABSOLUTE bound on the residual norm, so what is reachable depends on the circuit -- the diode two-tone deck settles at `2.3e-15` while an FET amplifier carrying tens of milliamps floors near `1e-8` and can never satisfy `1e-10`. THE WAY IT FAILED WAS THE DEFECT: the Newton iteration reached a residual of `9.482e-09` at iteration 4, NINE ORDERS down and quadratic, then sat FLAT TO SEVEN DIGITS for the remaining 55; having failed the level the E-135 ladder halved its step and walked back to `lambda=0` -- 1022 Newton iterations -- and returned a bare `E_ITERLIM`. A good answer found in five iterations was discarded after minutes of work, six to seven of them at `hb 4 4`, which is indistinguishable from a hang and was

reported as one. Both numbers are now readable from a deck via `qpss_knob()`, asking BOTH published types (E-454: the spelling decides the type) and parsing with `strtod` not `atoi` so `qpss_maxiter=2e2` is 200 and not 2 (E-478's trap); a present-but-unusable value is REPORTED and the default kept. The stall test: `QP_STALL_RUN` (4) consecutive iterates improving the residual by less than `QP_STALL_RATIO` (0.1%) is a stall, accepted as the answer only if it sits **`QP_STALL_ACCEPT`** (1e6) below the residual the level opened with -- a stall that never came down is still a failure, reaching that verdict in a few iterations instead of `maxiter`. The design is not give-up-early; it is stop pretending a converged answer has not converged, and the closing message names which test carried it. ACCEPTANCE DOES NOT CHANGE THE ANSWER: against the same circuit under a loosened bound the fundamentals are BIT-IDENTICAL and the third-order products agree to 1.1e-05, about 0.0001 dB on OIP3; the counterweight is that `hb 3 3` still fails at a residual of `5.5e-05` at `lambda=0` and is refused, a check asserting the PROPERTY rather than the digits. On the ATF-34143 two-tone IP3 deck that prompted it, `hb 2 2` goes from `E_ITERLIM` after 14 s to converging in 10 iterations under a second, giving OIP3 = +34 dBm with both sidebands agreeing to 0.1 dB. DELIBERATELY NOT FIXED: `hb 3 3` and above still fail and the bound does not rescue them (`1e-8`, `1e-6`, `1e-4` all die at the same residual before the continuation takes its first step) -- a separate fault; what changes is the COST OF FINDING OUT, 48 s instead of 6-7 minutes. NO openvaf-r CHANGE. Suite `hbconv_examples` 23/23 (6/23 pre-fix; seventeen checks discriminate) while `qpssleak`, `distoexact` both solvers, `plotorder` 25/25 and `pzklu` 4/4 are unchanged; full regression 397/397.

E-484 *src/spicelib/analysis/dcpss.c, src/frontend/com_qpss.c, examples/hbtrust_examples/ (new)*

A CONVERGED FLAG IS NOT A CORRECT ANSWER. E-483 made the harmonic-balance bound reachable and taught the Newton loop to spot a stalled residual, and left a hole: the STALL path reported what it settled for, the ORDINARY `tol` path reported nothing, so a deck that loosened the bound far enough had its solution accepted with a clean *converged in 3 iterations*. On a two-tone FET amplifier `set qpss_tol=1e-1` at K=4 accepts a residual that came down by only ~100x and the third-order products land 6 dB out -- 27.818 dBm against the trustworthy 33.976 dBm of the K=2 run -- silently. WORSE THAN THE FAILURE E-483 FIXED: a refusal is honest, a wrong number is not, and E-483 had handed users a knob whose obvious misuse was silent corruption. The reduction behind an accepted solution is now reported on the ordinary path too, naming the residual, the reduction, the bound that permitted the early stop and both remedies; the run is not aborted and the spectrum still prints, since the user may know exactly what they are doing. THE THRESHOLD IS CALIBRATED, NOT CHOSEN: `QP_LOWRED_WARN` is deliberately a different constant from `QP_STALL_ACCEPT` -- one decides whether to accept, the other whether to believe -- and its value comes from measurement, 101x putting OIP3 6.16 dB out and warned against 55345x agreeing to 0.033 dB and silent, with two suite checks holding the bar from both sides because a warning that fires on a good answer is one people learn to ignore (E-445's note). IT DOES NOT FIX **`hb 4 4`**, whose Newton step degrades as harmonics are added (5.3e8x at K=2, 7.8e4x at K=3, 1.5e2x at K=4) with drive amplitude, `beta`, `is`, `alpha`, `cgs`/`cgd`, the passive network, the topology and the DFT oversampling each ruled out by measurement, `pss_csolve` pivoting and `tmalloc` being `calloc` -- leaving `qp_build_matrix`'s harmonic-domain Jacobian as the open question. What it guarantees is narrower and worth having: when the solver cannot give a correct answer it does not quietly give a wrong one. Also folds a `const char *` qualifier fix in `com_qpss.c` that E-483 introduced, two discards-qualifiers warnings in a tree kept at zero, surfaced by a `build-shared` rebuild whose output had been filtered to errors only. NO openvaf-r CHANGE. Suite `hbtrust_examples` 16/16 (10/16 against the E-483 binary; six checks discriminate) with `hbconv` 23/23, `qpssleak`, `distoexact` both solvers, `plotorder` 25/25 and `pzklu` 4/4 unchanged; full regression 398/398.

E-485 *src/xspice/cm/cmutil.c, src/xspice/icm/analog/{limit,int,d_dt,pwl,hyst,slew}/cfunc.mod, src/spicelib/parser/inp2dot.c, src/frontend/inpcom.c, src/frontend/inp.c, src/frontend/com_measure2.c, examples/guardsweep_examples/ (new)*

EIGHT GUARDS THAT DETECTED A FAULT AND THEN USED THE BAD VALUE. An hour-long hunt found the frontend and OSDI paths clean almost everywhere and the defects concentrated in the XSPICE code models, all of one shape. THE HEADLINE: `cm_climit_fcn` had its bail-out COMMENTED OUT -- it detected the crossed linear range, printed, and used the bad value regardless. It could NOT be uncommented: those lines assign the LOCALS and `return`, while the out-parameters are written at the end of the function, so restoring them verbatim would have left `*out_final` uninitialised -- very likely why they were disabled rather than repaired. Fixing the INPUT instead (clamping the smoothing half-width to half the limit span, leaving hard limiting at the declared bounds) takes `ilimit` from 24.48 to 0.437 against rails of +/-1, its messages from 26 per op to one, and its text from naming CLIMIT in a deck with none. SAME SHAPE FOUR MORE TIMES: `limit`, `int`, `d_dt` never computed `linear_range`, so an oversized `limit_range` reached 24.5057, 95.04 and 24.25 against limits of +/-1 and 249999.75 at 1e6 -- unbounded and silent, with those limits MANDATORY parameters; E-468's own comment says it added its checks *as the CLIMIT sibling already does* and ported two, not CLIMIT's own. `pwl`'s monotonicity guard ended in `break`, leaving only the CHECKING loop, so the table was built from rejected data and `x=[0 2 1]` answered 5.5 for an input of 0.5; it cannot `return` in place (`STATIC_VAR`-owned arrays), so the test moved beside the `size_error` check. `hyst` and `slew` had NO checks: an inverted pair and an oversized `hyst` left the block dead at 0.0, a negative `rise_slope` drove the output to -2.0. THREE IN THE FRONTEND: `dot_sens` validated nothing while `dot_ac` in the same file names the same faults -- a reversed range swept a FABRICATED decade 1e6 to 1e7 ascending; `dot_disto` reported a DEVICE-parameter fault it does not have; `.include/source` of a DIRECTORY succeeded in SILENCE (`fopen()` on a directory succeeds on macOS/BSD/glibc) so the deck solved a different circuit, v(out) 1.0 not 0.5, while `.lib` was already guarded; `meas` clamped a negative FROM and then reported the window the user asked for. THE REPAIR IS THE CODEBASE'S OWN PATTERN -- `sine/square/triangle` already detect, report, substitute a safe value -- which is what makes these defects and not design. THREE WITHDRAWN, one at fix time: `sweep`'s negative step is deliberate (`com_sweep.c:2119`), XSPICE `Limits:` ARE enforced, and `.nodeset` on an OSDI internal node really is ignored (the counter-evidence was a diagnostic ECHOING the card). Suite `guardsweep_examples` 37/37 (24/37 pre-fix; thirteen checks discriminate), every fix paired with the control that must not move; full regression 399/399.

E-486 *src/spicelib/analysis/cktsens.c, src/xspice/enh/enhtrans.c, src/xspice/mif/mifmpara.c, src/xspice/icm/table/{table2D,table3D}/cfunc.mod, src/xspice/icm/digital/d_state/cfunc.mod, src/xspice/icm/xtradev/{capacitor,inductor,core}/cfunc.mod, src/xspice/icm/tlines/{mlin,cpline,cpmlin}/cfunc.mod, src/xspice/icm/analog/{file_source,xfer}/cfunc.mod, five ifspec.ifs Limits, examples/guardpair_examples/ (new)*

TWELVE GUARDS THAT ONE SIBLING HAD AND THE OTHER DID NOT. An hour-long hunt found OSDI itself clean -- state restoration across all nine analyses including E-483/484 `hb`, array instantiation, instance-parameter ranges, round-34 still fixed -- and one shape repeated: the check EXISTS in the tree and is absent next door. THE HEADLINE: `table2D`'s data-row loop is driven by the FILE and wrote `table_data[lLineCount-1]` unbounded, while the x row, the y row, each row's WIDTH and even a premature EOF were all checked. 3 declared y values and 5 supplied rows indexed past the allocation and SEGFAULTED (`EXC_BAD_ACCESS` at 0, rc 139, no diagnostic) from a twelve-line file; too FEW rows left `calloc`'s zeros in place and a probe there returned 0.0, a plausible no-current. E-247 had already fixed the OOB READ and the too-small case *in this same file* and left the WRITE; the sibling `table3D` refuses the truncated file, so the two disagreed and only one crashed. THE SECOND: `count_steps()` returns a POINT COUNT, and E-362's guard signalled failure with `return(E_PARMVAL)` -- `E_PARMVAL` is 11, the caller never tested it, so an impossible sweep ran eleven points with `*stepsize` never written. Two silent repairs sat below it, rewriting a stated 0 Hz start to `1e-3` and a stated stop to a decade above the start -- in the LOCAL copy only, so the count used the repaired bounds while the sweep ran from `job->start_freq`: `.sens ac dec 5 0 1meg` printed a table of all 0 Hz rows, `.sens ac dec 5 1k 1k` swept a full decade past the stop (6 rows where `.ac` gives 1). The new rules are `.ac`'s own. TEN

MORE: d_state ran ROW 0 for a state the file never defines; file_source lacked the monotonicity check its sibling pwl has had since E-480; xfer's file path validated nothing while its table path ten lines above refuses the same data; hyst's unbounded input_domain drove the output to 250000 against declared limits of 0 and 1; core indexed B with H's size; mlin/cpline/cpmlin accepted impossible geometry (a NEGATIVE length returning a plausible 0.5011 against 0.4992); poly() reported each fault with the OTHER's message; and four models were the only ones of twenty-plus with unbounded delays. DELIBERATELY UNCHANGED, with the built-ins as oracle: NEGATIVE capacitance and inductance are legitimate -- built-in C and L behave identically -- so only ZERO is fixed; mlin's rho=0 looked like a perfect-conductor idealisation until it measured as NaN; xfer duplicates stay legal because the table path allows them. Both headlines trace by git -S to 4f29ffad, the vanilla UPSTREAM import. Verified guardpair 65/65 (24/65 pre-fix, 41 discriminating). No openvaf-r change.

E-487 *src/spicelib/analysis/dcpss.c, src/include/ngspice/cktdefs.h, src/frontend/com_hbosc.c, src/frontend/com_hb.c, src/frontend/com_hb.h, src/frontend/com_loadpull.c, src/frontend/typesdef.c, examples/rfplots_examples/ (new)*

EVERY RF ANALYSIS LEAVES ITS RESULTS IN A NUTMEG PLOT. An RF result that is only PRINTED cannot be plotted, printed again, wrdata'd, diffed or read by a script. Six of the eight RF entry points published; **hbosc** and **phasenoise** printed a table and stored NOTHING, leaving the session with hbosc's own STARTUP TRANSIENT as the current plot. THE CAUSE: E-209 added struct hbspectrum *out to HBanalyze so com_hb could publish; HBOSCanalyze computes the SAME (2K+1)*N spectrum in the SAME layout and never got the parameter -- one signature extended, its sibling not, the shape E-486 spent its length on. Both now hand their results back, and rather than let com_hbosc grow a second copy of hb_publish() the two share ONE parameterised publisher. hbosc also stores **oscfreq**, the frequency an autonomous circuit SOLVES for. THE SUBTLE PART: ft_plotabbrev() matches by SUBSTRING, so hbosc hit the hb pattern and came out named hb1 -- indistinguishable from a driven run -- while phasenoise hit noise. Both get entries AHEAD of the shadowing pattern, per E-383's rule; plotorder_examples asserts that rule for every name in the tree but its NAMES table is HAND-MAINTAINED and was passing over these two vacuously. TYPES: loadpull's pout_dbm/gain_db are dB and now carry SV_DB, while gamma_re/gamma_im, pae, eff and stb's loopgain stay untyped -- dimensionless ratios with no enum member, where inventing one would be worse. NOT CHANGED: loadpull leaves 61 intermediate transient plots at -n 9 and 333 at -n 21; discarding plots a user may inspect is a behaviour change, so it is recorded. Verified rfplots 17/17 (7/17 pre-fix, 10 discriminating) with checks [1]-[6] and [8] passing BEFORE and AFTER as the control that the six working analyses did not move; hbosc 13/13 (6/13 pre-fix). No openvaf-r change.

E-488 *src/frontend/com_sweep.c, examples/sweeptemp_examples/ (new)*

sweep temp SWEPT NOTHING. sweep resolves a knob as a .model param, an instance/device param, or a deck .param; the GLOBAL circuit temperature is none of those, so a bare temp fell through to instance/device, alter temp= found no such device, and the sweep ran to completion over a knob that never moved -- a full set of points, rc=0, and a plottable FLAT curve reading as *temperature has no effect on this circuit*. E-431 removed that shape for an unresolved -output and E-435's own comment here describes it for subckt-local model names; temp was a third instance, and the one users meet, because every OTHER route worked (.option temp=, set temp=, alter @dev[temp]=, sweep @#*[temp]). THE OBVIOUS FIX DOES NOT WORK: writing ckt->CKTtemp (what .dc temp does) left the curve flat, because CKTdoJob opens with ckt->CKTtemp = task->TSKtemp -- .dc survives only because its whole sweep runs INSIDE one CKTdoJob, while sweep runs a fresh analysis per point, so the write was discarded before the next point solved. The TASK must move, so it issues option temp=, which also carries the value through the guarded OPT_TEMP funnel and inherits E-426's absolute-zero refusal and E-440's sanity check. DELIBERATELY NOT IDENTICAL TO **dc temp**: over a temperature-driven node collapse dc temp returns 0.0 where a static op returns 0.5 -- it holds one setup for the whole sweep and never rebuilds (

round-24, still open, NOT fixed here) -- while `sweep` rebuilds via E-471 and is right; the suite pins all three against the static `op` and asserts `dc temp` DISAGREES, so "fixing" sweep into agreement FAILS. STRICTLY ADDITIVE: a deck `.param temp` is tested first and still wins. Also matched: an unphysical range is refused UP FRONT (left to the funnel, bad points are ignored one at a time and the axis still carries them, so a row LABELLED -600 C held the 27 C answer), and the axis carries `SV_TEMP`. Verified `sweeptemp` 20/20 (10/20 pre-fix, 10 discriminating). No openvaf-r change.

E-490 *src/spicelib/parser/inp2n.c, src/frontend/subckt.c, src/include/ngspice/inpdefs.h, examples/busmixed_examples/ (new)*

MIXING SHORTHAND AND WRITTEN-OUT BITS ON A BUS PORT SILENTLY MISWIRED THE DEVICE. E-444's `.option autobus` fires on a token count equal to the PORT count; positional binding covers a count equal to the TERMINAL count. A line that leaves one port in shorthand while writing another port's bits out is NEITHER, so `N1 a b[0:2]` against `inout [0:4] a; inout [0:2] b` fell through both and bound positionally against the flat terminal list -- `a` onto `a[0]`, `b[0]` onto `a[1]`, `b[1]` onto `a[2]`. Measured on a model whose eight bits each carry a different conductance, every node sat one or more terminals off and every current was wrong. The only thing said was E-402's warning about the terminals left over at the TAIL -- the symptom, not the cause -- so a user who supplies the two nodes it asks for still has a circuit wired entirely wrong, now with no warning at all. E-445's diagnostic (*already carries an index, so it cannot be expanded as the bus port*) exists for exactly this mistake but sits INSIDE the port-count check: the guard and the failure were one `if` apart. NOTHING HERE IS AMBIGUOUS, so the fix does not guess -- it walks the ports left to right and lets each token declare its form: a bare name on a bus port is shorthand for that port's bits, and a token already carrying an index (or ground, which E-445 established can never be indexed) means that port was written out, so take one token per bit. `N1 a 0 0 0` reads correctly under the same rule. The rewrite is accepted only when the walk consumes EXACTLY the tokens the line has, and the walk is bounded by that count because `gettok_instance` cannot tell where the node tokens end -- without the bound a port claiming more bits than the line has left would swallow the MODEL NAME and report against it. WHERE NO READING EXISTS IT REFUSES: if the counts do not reconcile, the bits written do not match the width the model declares and nothing can repair that, so it is refused where the port and both counts are still in hand to say so -- naming the port, its width, the tokens written and the two counts that would have worked. That replaces a wrong answer, not a working deck; warning and binding anyway is the *detect, announce, then use the bad value regardless* shape E-485 had to undo eight times in one round. ONE RULE, ONE READER: whether a token already carries a bit index is spelling-dependent (`a[0]` always, `a_0_` only under `autobus=kicad`), and `subckt.c` already answered it for `.subckt` formals -- a second copy in `inp2n.c` would have been free to disagree, the two-readers-of-one-rule shape E-454 had to repair in this same option -- so the rule moved to `INPbusTokenIndexed` beside `INPbusBitSuffix` and `subckt.c` now delegates. TWO SMALLER FIXES IN THE SAME OPTION: `.option autobus=1` reported a style that does not exist, the on-word list holding `true/yes/on` but not `1`, the mirror of the `0` in the off-word list -- the comment on the early return says *bare flag, or =1: a NUMBER, not a string*, true of `set autobus=1` but not of a deck **`.option`** card, which publishes a STRING; and E-445's own message named a TERMINAL where a PORT was meant, reporting a `[0:2]` port as *the bus port 'b[0]'*, an index the user never wrote. UNCHANGED BY DESIGN: with autobus off nothing moves, all-shorthand and all-explicit lines take the paths they always did, a short line with no shorthand token keeps E-402's warning and the `$port_connected` idiom it protects, and E-445's two refusals still fire on a full-width line. Eighteen further shapes measured and needing no fix: unequal port widths, descending `[4:0]` and non-zero-based `[1:5]` ranges, a 16-bit port, three buses of sizes 2/3/4, bus/scalar/bus interleaving, one token feeding two ports (which superposes exactly), uppercase tokens, the option placed AFTER the instance, two instances in parallel, `m=` multipliers, all four subcircuit shapes including reversed formals, `autobus=kicad` on unequal widths, and too-many-nodes as a hard error; a `[0:0]` port connects correctly in both spellings, its node merely named `a` rather than `a[0]`, which is inherent. Verified `busmixed` 46/46 (20/46 pre-fix, 26 discriminating). No openvaf-

r change.

E-491 *src/misc/printnum.c, src/misc/printnum.h, src/include/ngspice/cpstd.h, src/xspice/evt/evtprint.c, src/spicelib/parser/ptfuncs.c, src/spicelib/parser/inpptree.c, src/spicelib/analysis/cktsens.c, src/frontend/measure.c, src/frontend/device.c, src/frontend/inpcom.c, src/frontend/{diff,dotcards, postcoms}.c (18 call sites), src/xspice/icm/analog/s_xfer/cfunc.mod, examples/numguard_examples/ (new)*

AN UNBOUNDED NUMBER USED AS A LENGTH, AND FOUR WRONG ONES. Ten defects of one shape from a bug hunt: a number the deck supplies, used without being measured against what it was about to control. THE CRASHES: `set numdgt` is the user's print precision and nothing bounded it -- `printnum()` `sprintf`'d `%.*e` into caller buffers of `BSIZE_SP`(512) and `evtprint` into a `char[100]`, so **`numdgt=510`** with **`print`** ABORTED and **`numdgt=94`** with **`eprint`** TRAPPED, both from a plain batch deck with no interactivity and both exactly where `n+9` bytes says they must. `printnum()`'s OWN COMMENT recorded the hazard -- *it can cause buffer overruns, the size of buf is unknown* -- without bounding it, while the DSTRING sibling `printnum_ds()` directly below could not overflow, which is precisely why `fourier`, `wrdata`, `write`, `display` and `diff` were unaffected and only the `print` family crashed. It now takes the buffer size and CLAMPS rather than truncates -- a truncated number is not the number computed, and past ~17 digits the rest is zero padding -- saying so once; `evtprint` uses the same clamp so the two printing paths cannot disagree. Both trace by `git -S` to `4f29ffad`, the vanilla UPSTREAM import. THE WRONG NUMBERS: `PTdivide` added `PTfudge_factor` to EVERY divisor rather than a zero one, and that factor is `gmin*1e-20` -- not even a fixed perturbation but one scaling with an unrelated convergence option -- so **`1/boltz`** came out 42% low under **`.option gmin=1e-3`** and 88% low at `gmin=1e-2`, while `1/0` returned 1e26/1e32/1e50 by gmin alone; the nudge now applies only to an EXACT zero with a fixed epsilon, so a non-zero divisor is used as written and `1/0` is the same number in every deck. And B-source trig reduced with `x-(int)(x/2pi)*2pi`, undefined above `2^31*2pi`, so **`sin(1e20)`** returned +0.9993 where the answer is -0.6453 -- while numparam and a Verilog-A model each already matched libm exactly, making the B-source the SOLE OUTLIER, the divergence E-399 forbids and `ptfuncs.c`'s own comment quotes. THE SILENT REFUSALS: a `sens` filter matching no parameter returned OK with no plot and no message, leaving the PREVIOUS analysis current for a following `print` (and the trailing words on a `sens` line ARE filters, so `sens v(b) r8` legitimately works -- which made a typo indistinguishable from a filter doing its job); the interactive `meas` enforced its analysis keyword for INTERVAL measurements (E-468) and ignored it for POINT ones, so `meas tran ... FIND AT=` read a DC sweep as a transient; `s_xfer` blamed the SOLVER for a model error, an all-zero denominator going NaN and being reported as *Dynamic gmin stepping failed*, and `num>den` announced once per evaluation (1238x over 300 steps) with rc=0 and the device contributing nothing; `printf("oops ")` reached users on stdout from two `default:` arms in `inpptree.c`, one of them the very printer the *internal check of parse tree failed* diagnostic uses to SHOW the offending expression; `show` printed `?????????` where `print/alter/altermod/wrdata/sweep` all name the parameter; and a duplicate user `.func` was silently last-wins while shadowing a builtin warned (E-467). ONE COMMENT CORRECTED RATHER THAN OBEYED: E-440's claim that the singular routes *clamp to HUGE* so *NIiter can reach for gmin* is not what the code does -- `ifeval` treats a result equal to HUGE as an ERROR FLAG and aborts, the opposite of continuing, and the routes were never uniform; recorded rather than unified, since making `log(0)` abort or `pow(0,-1)` not abort would each move an answer a working deck gets today. THE FIX'S OWN TRAP: the new `meas` gate first refused BEFORE the standard failure path, so it skipped both the `failed!` line a script tests on and E-475's `vec_remove` -- **`mathguard`** caught it, and the gate now refuses the way every other refused measurement does. UNCHANGED: ordinary precisions, `1/0` still finite so a solve continues, interval measurements, an unfiltered or matching `sens`, a valid `s_xfer`, and `.func` last-wins. Verified `numguard` 68/68 (18/68 pre-fix, 50 discriminating), `mathguard` 57/57, regression 405/405. No openvaf-r change.

E-492 *src/spicelib/parser/inp2dot.c, src/spicelib/parser/inp2{e,g,s}.c, src/spicelib/parser/inppas3.c, src/*

include/ngspice/inpdefs.h, src/include/ngspice/cktdefs.h, src/spicelib/analysis/cktop.c, src/osdi/osdiload.c, src/maths/KLU/klusmp.c, examples/ctrlnode_examples/ (new)

A NODE NAMED ONLY IN A CONTROL POSITION WAS SILENTLY CREATED. `E`, `G` and `S` take a controlling node PAIR, bound exactly the way the output pair is -- `INPtermInsert`, which CREATES the node -- so a typo simply invented a node and the run continued against it. For `E`/`G` the invented node has no path to ground, the matrix goes singular, and the user is told *singular matrix: check node nosuch* -- a node they never wrote, reported as a fault in their circuit. **`S`** IS WORSE: a switch only READS its control voltage to decide open or closed and stamps nothing for it, so the matrix stays non-singular, the solve succeeds, and the answer is silently wrong -- `S1 a b ctl 0 sw` gives 0.999001 while `S1 a b nosuch 0 sw` gives 9.99999e-07, a factor of a million from one mistyped character, rc=0, with no diagnostic at all. A phantom reference is dangerous exactly where it is READ but not STAMPED. EVERY OTHER ROUTE ALREADY ANSWERED THIS: `.ic`/`.nodeset` report *IC on non-existent node*; `F`, `H`, `W` and a B-source `i()` report *unknown controlling source*; all thirteen output constructs name a missing vector; and `warn_physics` even validates the switch's own `ron`/`roff`/`vh` -- only the controlling-node pair skipped it, under any option. THE MECHANISM IS E-429's, UNCHANGED: `devRef` records whether a node was ever named by a device or made by the simulator, and a control reference is the same kind -- it names without making real -- so it no longer sets `devRef` and `CKTnodePhantom()` answers as it stands. Marking is left to `INPtermInsert` for every other position, so a control node that IS connected somewhere, including one that is the source's own output (`E1 out 0 out 0 2`), stays marked and is never reported; the check runs in PASS 3 for the same reason `.ic`'s does, because only once every device card has been read is *did anything connect to this?* answerable. It REFUSES rather than warns -- `E`/`G` already fail on their own, while a switch would carry on and answer from a node that is not in the circuit, the detect-announce-then-use-it-anyway shape E-485 had to undo eight times in one round. TWO DIAGNOSTICS THAT NAMED THE WRONG THING: CKTop blamed Verilog-A for faults that were not its own -- E-378 added the message and E-399 narrowed its entry to `E_PANIC` checks, arguing the test exact because `E_PANIC`(1) and `E_ITERLIM`(103) differ, which is true, but the invariant the MESSAGE relies on is **`E_PANIC`** = a Verilog-A fatal, and **`E_PANIC`** has ~10 producers (`cktsetup` x2, `osdiparam` x2, `osdiload`, `osdisetup`, `dcpss` x2, `cktpzstr`, `ifeval`, CIDER's `twosolve`), so `.option klu` with a current-source-only matrix and no Verilog-A device anywhere claimed one had raised it and pointed at an `OSDI(fatal)` line that does not exist, through op, dc, ac and tran alike since each calls CKTop; `CKTvaFatalRaised` is now set only where one is actually detected and cleared on entry so a previous analysis cannot be inherited. And KLU printed NINE LINES FOR ONE CONDITION -- its own PreOrder comment says an empty matrix is legitimate (*XSPICE pure digital circuits produce empty KLU matrix*) and returns success for one, yet all three sites reported it per call and Solve added two NULL-object errors that are its consequences, re-entered per Newton iteration; it now says once, in words, what the state is. UNCHANGED: a control node connected anywhere else -- the source's own output, one defined later, ground, or one through a subcircuit port; `F`/`H`/`W` and B-source `i()`; a real Verilog-A fatal, still named and still pointing at its `OSDI(fatal)` line; and either solver on an ordinary circuit or a mixed-signal KLU run. Verified `ctrlnode` 37/37 (19/37 pre-fix, 18 discriminating), regression 406/406. No openvaf-r change.

E-493 *src/frontend/device.c, src/frontend/outitf.c, src/spicelib/parser/inp2r.c, examples/namelookup_examples/ (new)*

THREE NAMES THE SIMULATOR WOULD NOT LOOK UP. One shape, three times: a name the user wrote was not looked up where it was written, and what came back described something else. **`showmod <MODEL>`** COULD NOT FIND THE MODEL: the device generator reads a bare word as an INSTANCE name and only `#name` as a MODEL, so with `.model dm d` used by `D1`, both `showmod d1` and `showmod #dm` printed it while **`showmod dm`** answered *No matching instances or models* -- of a model that plainly exists. The command's own help calls its argument *models* and its write sibling takes the model name directly (`altermod dm is=1e-12` works), so the one command dedicated

to models was the one that could not be handed one; OSDI models were affected identically, the defect being in the name grammar rather than in any device. RETRY, NOT REINTERPRET: the bare-name-is-an-instance reading runs first and unchanged, so `showmod d1` -- and any name that is both a device and a model -- behaves exactly as before, and only when NOTHING matched is each bare word retried with the # the grammar wants. A SAVED NAME THAT MATCHED NOTHING WAS DROPPED IN SILENCE: Pass 1 marks the `.save`/`.probe` items it places and left the rest unmarked, so `.save v(n) v(nosuch)` recorded `v(n)`, discarded the typo and said nothing -- the analysis succeeded and the vector the user asked for was merely absent. `.probe v(nosuch)` shares that path, which is why a mistyped NODE there was silent while a mistyped SOURCE in the same card is reported by the measure-source pass, and why E-418 already spoke for the `@dev[param]` spelling -- the plain node spelling was the one route left quiet. It WARNS rather than refuses (an absent vector is not a wrong answer, and a deck saving a node it does not always build is a real idiom) and honours `savesused[]` first, so the `all`/`allv` keywords and items belonging to another analysis are never reported, falling back to matching the name itself under `save all` -- which `.probe` turns on and where Pass 1 never runs. A RESISTOR MODEL NAMED **`r`** WAS UNREACHABLE: `INP2R` excluded the token `r` outright, which `R1 a b r=1k` needs since there `r` is the keyword that writes the resistance, but that also locked out a model actually CALLED `r` -- `.model r r rsh=1k` with `R1 a 0 r l=1u w=1u` bound neither model nor value, so the device fell through to the default and came out 1 mOhm with *resistance too low or not given*: the symptom, when the cause is that the model named on the line was never looked up. Every other name works, including every other resistor keyword (`rsh`, `l`, `w`, `tc1`, `tc2`, `temp`, `dtemp`, `m`, `scale`, `ac`, `dc`, `noisy`, `short`, `narrow`); only this one letter was unreachable. The two spellings differ by what FOLLOWS the token -- the keyword form is `r=`, a model name is not -- so asking that keeps `r=1k` meaning what it always did, and a deck with no such model is unaffected because `INPlookMod` fails and the existing else branch restores the line exactly as for any other unrecognised token. UNCHANGED: `showmod` by device name, by `#model` and bare, and `show`, including a name that is both; `altermod <model>`; `.save all`/`allv` and `.probe alli`, and items belonging to another analysis; `@dev[param]` saves; and `r=1k`, `r = 1k`, `R=2k`, a plain value, `r=1k tc1=0`, a value with `m=`, and **`r=4k`** winning when a model named **`r`** also exists. Verified `namelookup` 39/39 (29/39 pre-fix, 10 discriminating), regression 407/407. No openvaf-r change.

E-494 *src/spicelib/parser/ptfuncs.c, src/frontend/inpcom.c, examples/bexprvalue_examples/ (new)*

TWO WAYS A B SOURCE EXPRESSION DISAGREED WITH EVERY OTHER VALUE PATH. Round 54 compared the B source path against the paths that read the *same number on the same deck*. **`x/0`** LOST ITS SIGN: E-491 replaced a gmin-derived fudge factor in `PTdivide` with a fixed epsilon and wrote the nudge as `arg2 = (arg1 >= 0.0) ? PTDIV_EPS : -PTDIV_EPS;`, meaning to keep the sign of the numerator -- but a negative numerator was then divided by a NEGATIVE epsilon, so the quotient came out POSITIVE either way. `v(p)/0` with v(p) = -3 returned +3e+32 instead of -3e+32, and `-1/0`, `(0-2)/0`, `-1.0/0.0`, `-1/(1-1)` and `E0 b0 0 vol='-1/0'` were all positive too. Every case E-491 measured had a positive numerator, which is why its own suite did not see it. A divisor of exactly zero carries no sign to recover, so the side zero is approached from is now chosen ONCE rather than per operand: the epsilon is unconditionally positive, and the quotient keeps the numerator's sign -- the limit of `x/eps` as `eps` tends to zero from above, and what the comment E-491 left in place already described. NUMERIC LITERALS WERE ROUNDED TO 11 SIGNIFICANT DIGITS: every B line is rewritten before parsing by `inp_modify_exp()`, which re-emitted each literal with `"%18.10e"`, so `v=1.2345678901234567` reached the parser as `1.2345678901e+00` -- a relative error of 1.9e-11 -- while the same literal on an `R`, `C` or `V` card, in a `.param`, or as an OSDI **`.model`** parameter kept all seventeen. The B source was the only path in the simulator that could not carry a double, and it reached the arithmetic rather than just the printout: `tan(1.5707963267948966)` returned -1.96e11 against libm's 1.633123935319537e+16, so the tangent was right and its argument was not. MORE DIGITS WOULD NOT HAVE FIXED IT: `INPevaluate()` reads the text back and accumulates the mantissa as `mantis = 10 * mantis + digit`, losing a bit past 2^53, so a fixed `"%.17e"` turns the sixteen digits of `0.7853981633974483`

into eighteen and brings it back 1 ulp out when the user's own text would have survived -- the new `inp_num_text()` emits the SHORTEST text that round-trips, trying 15 then 16 then 17 digits. DELIBERATELY NOT WIDENED: `INPevaluate`'s own scaling still costs 1-2 ulp, but a `V` source shows it identically on the same literals, so the suite pins parity and a 2 ulp bound rather than bit-exactness, against an 11-digit rounding that had put those values 1.0e5-1.6e5 ulp out; and `mkb()` in `inpptree.c` folds a constant `/` with a raw division and no zero guard, bypassing `PTdivide`, but probed from `B`, `E` and `G` in constant, parenthesised, nested and mixed forms it never produced an infinity, so it appears unreachable from user input and was left alone rather than fixed past the evidence. UNCHANGED: E-491's magnitude and gmin independence; ordinary divisors; the SPICE suffixes, leading-dot and exponent forms; `pwl` B sources, which `inp_bsource_compat()` excludes from the rewrite; current B sources, node references mixed with literals, and the value surviving a transient; and `E`/`G` expressions, which never passed through `inp_modify_exp()` at all since only `b` lines do. Verified `bexprvalue` 63/63 (37/63 pre-fix, 26 discriminating), regression 408/408. No openvaf-r change.

E-495 *src/spicelib/parser/inpgmod.c, src/spicelib/analysis/dctrcurv.c, src/frontend/numparam/xpressn.c, src/frontend/rawfile.c, examples/binstale_examples/ (new)*

SEVEN WAYS A DECISION MADE ONCE WAS NEVER REVISITED. Round 55 probed MOSFET model binning, untouched by any earlier round, and the shape it exposed led into the DC sweep. THE TOLERANCE WAS ABSOLUTE: `is_equal` tested `fabs(a-b) < 1e-9` on values in metres -- a fixed one-nanometre slop -- so a device up to 1 nm outside *every* declared bin was silently placed in one (`l=31n` bound to a bin reaching 30n; `31.1n` refused, so the edge really was the constant). The magnitude does not scale, so as a *fraction* it grows without bound: 0.03% of a 3 um width but 5% of a 20 nm channel. Now relative. ADJACENT BINS OVERLAPPED: `in_range`'s own comment states `min <= value < max` while the code also accepted `is_equal(value,max)`, closing the interval, so a device on a shared boundary matched both neighbours and **`.model`** declaration order decided -- reversing two cards moved `l=1.999u`, unambiguously inside the LOWER bin, into the upper one and changed i(V1) by 2.95x. Selection now runs the strict rule first and the closed one only where it matched nothing, which keeps the top bin's `lmax` binding and means nothing that works today moves. AN OSDI MODEL COULD NOT BE BINNED: the set was eleven hardcoded built-ins, so a Verilog-A model written as a PDK writes one died with *Unable to find definition of model nv* for a model defined twice; OSDI is now asked through E-323's predicate and the four bin limits consumed like `level`. A DEGENERATE **`agauss`**/**`gauss`** WAS SILENTLY FLAT: a zero or negative variation or sigma returns the nominal for every draw, so a montecarlo over a parameter that never moves reported 100% yield where the real spec gave 12% -- one character apart, silent under `warn_physics`. Now audible; behaviour unchanged; `unif`/`aunif`/`limit` deliberately untouched. **`.dc`** NEVER REVISITED WHAT SETUP DECIDED: E-471's own comment says `.dc` *sets the circuit up once and walks its points inside the analysis* and that a frozen topology makes *the sweep quietly draw a flat line* -- it gave `sweep` the machinery to rebuild and `.dc` never got it, so a swept `l`/`w` leaving its bin kept the parse-time model (2.9x out) and a swept parameter changing an OSDI node collapse kept the old matrix (a flat line: -1.000e-03 at rs = 0, 500, 1000 where the answers are -1e-3, -6.67e-4, -5e-4). `alter` and `sweep` both get these right, so `.dc` now REFUSES the point it cannot compute and names `sweep` (E-485's rule). Detection is free when nothing moves: the collapse is read from a flag `OSDItemp` already sets and `CKTdoJob` already consumes (E-417) -- `DCTsetInstParam` calls `DEVtemperature` and never asked -- and the bin from the four limits on the bound model. The refusal deliberately does NOT borrow E-427's *the device refused ... also refused by* `alter`, false on both counts here. THE ASCII RAWFILE WAS ONE DIGIT SHORT: `DEFPREC 15` emits sixteen significant digits where a double needs seventeen, so `write+load` changed the last ulp while the BINARY format was exact (`%.15e` fails to round-trip 51390 of 200000 random doubles, `%.16e` none). UNCHANGED: bin selection that works today in either card order incl. the top `lmax`; `alter`; `sweep`; `unif`/`aunif`/`limit`; a genuine device refusal; source and `m` sweeps; an unbinned model's sweep; the binary rawfile and an explicit `rawfileprec`. Verified `binstale` 64/64 (40/64 pre-fix, 24 discriminating), regression 409/409. No openvaf-r change.

E-496 *src/frontend/dotcards.c, src/frontend/breakp2.c, src/frontend/outitf.c, src/include/ngspice/ftedebug.h, src/include/ngspice/ftedefs.h, src/include/ngspice/fteext.h, examples/savekw_examples/ (new)*

A PLOT KEYWORD IS NOT A SIGNAL NAME. Reported from use: with `.option saveused`, `pyplot v(a[0]) xlabel 'something'` printed *save 'xlabel': nothing of that name is in this analysis*. E-469's bare-word scan took EVERY argument of an output command that was not a number, a redirection or an expression, with no knowledge of those commands' own keywords -- all 22 plot keywords, `hardcopy` likewise, and `meas` worst (`meas tran m1 FIND v(b) AT 50u` offered `tran`, `m1`, `find`, `at`). THE ANSWER WAS NEVER AFFECTED: E-469 over-collects deliberately (its own comment says under-saving would cost the answer) and a save matching nothing produces nothing -- the vector asked for returns bit-identical either way. The names were gathered in SILENCE until E-493 added the unmatched-save warning, which exposed the flaw rather than causing it. FIXED IN TWO PARTS, THE OBVIOUS ONE SECOND. (1) An INFERRED save is never reported: E-493's warning exists to catch a name the AUTHOR wrote and got wrong, so registration records which is which (`db_auto` -> `save_info.autosaved`) and the warning skips what `saveused` guessed; the flag is set only around `ft_saveused`'s own `com_save`, leaving `.save`, `save`, `.probe` and savecurrents untouched. This covers every keyword of every command, present and future. (2) The plot family now knows its own grammar and `meas` contributes no bare words (its vectors arrive as `v(...)`/`i(...)`, already taken by the reference scan) -- but alone that would be the E-487 trap, a hand-maintained list that rots invisibly (it mirrors `CT_PLOTKEYWORDS`, unusable here because it is a tab-completion table built only in an interactive session), which is why (1) answers the report. UNCHANGED: everything about WHAT IS SAVED, pinned against the same run without the option (plain plot, plot with keywords, `plot ... vs ...`, `meas` then plot, `let` then plot); `all` still means everything; `wrdata`'s file name is still not a vector; an explicit `.save` still makes saveused stand aside; a node genuinely named `title` or `vs` keeps its value; `.save`/`.probe` of a name matching nothing still warns. Verified `savekw` 46/46 (15/46 pre-fix, 31 discriminating), regression 410/410. No openvaf-r change.

E-497 *src/spicelib/analysis/dsetparm.c, src/spicelib/analysis/nsetparm.c, src/maths/misc/randnumb.c, src/frontend/inpcom.c, src/xspice/icm/analog/{s_xfer,sine,square,triangle,oneshot}/cfunc.mod, examples/argcheck_examples/ (new)*

CONSTRAINTS THE DOCUMENTATION STATES AND THE CODE DID NOT CHECK. Round 56 mined the manual for stated numeric constraints; three of five findings came from one grep. **`disto`**'s f2overf1 was unvalidated -- the manual says *between (and not equal to) 0.0 and 1.0*, yet 0, 1, 1.5, 2 and -0.5 were accepted in silence and move the answer (2F1-F2 read 1.695 / 1.630 / 1.477 / 1.580 vs 1.711 for a legal 0.5); at ratio 1 the plot labelled *IM: f1-f2* holds a product at DC, at a negative ratio the second tone is at a negative frequency. Both neighbouring cases in the SAME switch already return `E_PARMVAL`. Refused, not clamped -- the ratio IS the experiment. **`setseed`** truncated a fractional seed: `%d` stops at the first unusable character so `2.5` became 2, while `0`/`-3`/`abc` are each named and sibling `repeat` names a fractional count. **`s_xfer`** indexed **`int_ic`** by the WRONG array's size -- `PARAM(int_ic[den_size - 2 - i])` with nothing consulting `PARAM_SIZE(int_ic)`: too short and the unreached ICs read zero, `v(out)` at 10 us 7.00005 -> 4.99995e-05. The oscillator family went SILENTLY DEAD -- `sine`/`square`/`triangle`/`oneshot` announced an array mismatch and returned without setting an output, every evaluation: rc=0, source at zero, 2025 message blocks per 2 ms (~1e6 for 1 s). Exactly the shape E-491 fixed in `s_xfer` and named there; `d_osc`/`d_pwm` check inside `INIT` and are untouched. A duplicate **`.param`** was the only one of its family to pass in silence -- `.func` (E-491), `.model` and `.subckt` all warn -- and it takes the LAST where they keep the FIRST, so two includes that each set `vdd` resolve by include ORDER and a deck's own `.param` is displaced by a later library. Which value wins is unchanged; only made audible, scoped to top-level cards so subcircuit parameters are not mistaken for duplicates. Also: `D_STOP`/`N_STOP` reset the start field on a refused stop frequency (no observable consequence). UNCHANGED: legal ratios and single-tone `disto`; `setseed` with an integer or blanks; `s_xfer` cor-

rect/absent int_ic and E-491's num>den guard; matching oscillator arrays; a single .param, two names on one card, subcircuit parameters, and the .func/.model/.subckt messages. Verified argcheck 52/52 (33/52 pre-fix, 19 discriminating), regression 411/411. No openvaf-r change.

E-498 *src/spicelib/devices/vsrc/vsrctemp.c, src/osdi/osdisetup.c, examples/reusestate_examples/ (new)*

PER-RUN STATE LEFT STALE BY THE SETUP-REUSE FAST PATH. E-471 keeps a circuit standing between points -- CKTdoJob skips CKTunsetup/CKTsetup and re-runs only CKTtemp. That reasoning was about topology and for topology it is right; what was never asked is which state DEVsetup initialises that is not topology at all. A VOLTAGE SOURCE STOPPED SCHEDULING BREAKPOINTS ENTIRELY -- VSRCbreak_time is the source's breakpoint cursor, armed only when CKTtime >= VSRCbreak_time and seeded to -1.0 in vsrcset.c; reused, the instance carried the PREVIOUS run's break time, at or past that run's TSTOP, so the test was false at time zero and stayed false, and a PULSE or PWL source armed no breakpoints at all (0 of 5 PWL corners landed on, against 5 of 5; 81 native timepoints against 103). Against a standalone run of the identical circuit maximum(v(n)) -- grid-INDEPENDENT, so a different ANSWER not a different sampling -- was +16%, +44%, -10% across five points and 106% at other spacings, in silence; a 0.3 ppm change of the swept value sufficed, since the schedule is either armed or it is not. Only the FIRST point of a sweep kept a correct schedule, so the same resistance gave two different answers by direction of travel (R=1000: 0.40217511 up, 0.33963131 down). **optimize** RETURNED A WRONG FIT AND CALLED IT CONVERGED -- fitting R for maximum(v(n))=0.20 gave 2501.777336 with reuse off (peak 0.2000000003) against 2156.690117 with it on (peak 0.2264, 13% out), in 106 evaluations rather than 67. Contradicts E-471's own comment that reuse *changes nothing a user can see except the time ... a few ulp*, whose stated cause is refuted too: KLU and Sparse give bit-identical deviations. OSDI's **last_crossing** CACHE -- per-run state whose contract osdiaccept.c states outright, *starts at 0.0 before any crossing has been observed*; only OSDIsetup seeded it, so a reused point held the previous point's crossing and a 100 kHz sine over 40 us read 3e-05 at time zero of points 2 to 5 instead of 0. Sibling absdelay was already right, re-seeding on is_init_tran. Both re-armed in **DEVtemperature**, which CKTtemp runs once per job on BOTH paths and which does NOT run on a resume. Suites stayed green because reusesetup_examples and reuseloops_examples contain no PULSE or PWL source and the two shipped sweep+tran+PULSE examples use -overlay, whose resampling damps the error below their tolerances -- both moved when this was fixed. UNCHANGED: the fast path itself (still reused at 4 of 5 points, 0 rebuilt), sources registering no breakpoints (SIN, EXP, dc-only), a PULSE current source (ISRC recomputes from CKTtime and keeps no cursor), and op/ac/tf/noise. REPORTED, NOT FIXED: under KLU, sweep -analysis ac with reuse returns 0.0 for reused points and crashes in about one run in ten, nondeterministically -- a different defect in a different layer, reproducing on the shipped binary. Verified reusestate 34/34 both solvers (17/34 pre-fix, 17 discriminating), regression 412/412. No openvaf-r change.

E-499 *src/maths/KLU/klusmp.c, src/include/ngspice/klu.h, src/frontend/com_optimize.c, src/frontend/com_sweep.c, examples/loopguard_examples/ (new)*

LOOP-COMMAND ARGUMENTS, VERDICTS, AND A KLU REFACTOR KIND. Round 58 reprobed the fast path after E-498; the reuse arithmetic was clean, everything around it was not. A COMPLEX REFACTOR HANDED A REAL FACTORISATION -- klu_z_refactor refills in place, walking the L/U index arrays klu_z_factor built; given a REAL Numeric it walks half-sized arrays with complex strides and a later klu_free_numeric frees what it scribbled (malloc names object 0x3ff0000000000000, the bit pattern of the double 1.0 -- a matrix VALUE freed as a pointer). Reachable because every AC/SP/NOISE run follows a real operating point and E-471's reuse keeps the matrix standing: under .option klu, sweep -analysis ac/noise gave 0.0 at every reused point, optimize -analysis ac fitted 10x wrong saying *converged*, sp SIGSEGV'd 9 of 10 runs. SPARSE and every real analysis were always fine; both directions now check the kind. FOUR ARGUMENTS PARSED BY A WEAKER PARSER THAN THE ONE IN THE SAME FILE -- optnum( ) served bounds/-target/-spec while -maxiter/-samples/-swarmsize used bare atoi and -

`tol` bare `atof`, so **`1k`** meant 1000 in a bound and 1 in **`-maxiter`** on one line; `-samples 2e2` ran 2 MC samples yet design centering printed a yield and a Wilson interval. `sweep lin 2e2` and `montecarlo 2e2` were already right, and `sweep` grew a strict count parser in E-478. **`-seed`** had the matching hole: E-497 guarded the COMMAND, the OPTION truncated `3.7` silently and `0`/`-3` left the run unseeded and NOT reproducible; all four sites now apply the rule. CONVERGED WHEN NOTHING WAS OPTIMISED -- a parameter the objective cannot depend on, or one that does not resolve, hands the STARTING value back after 3 evaluations; `-target x 0.4 0` printed *residual = 0* for a fit that never happened; a bound-pinned answer was not distinguished from a minimum. A KNOB THAT NAMES NOTHING fell through `sw_kind`'s `SW_ALTER` fallback to a full set of points, rc=0 and a plottable FLAT curve named after the typo (the shape E-435/E-488/E-431 each removed elsewhere); `list 1k inf 2k 3k` ran ONE point calling the valid `2k`/`3k` unrecognized. **`-overlay`** DISCARDED THE SOLVER'S TIMEPOINTS -- a UNIFORM grid of the same count reported a peak 39% low (0.244 vs 0.402) from the same 121 points; now the union of the runs' own points, bounded, and it says which grid. Its `%g` names collided (three vectors all `vn_1000`), now shortest-round-trip per E-494. TWO WARNINGS CONTRADICTED EACH OTHER: *recorded as NaN* (true) and *those entries are zero* (false). UNCHANGED: a real fit is not annotated; every legitimate knob spelling, a good `list` and a two-knob sweep still parse; a transient sweep and the reuse tally are untouched; a legal `-seed` still pins the run. Verified `loopguard` 49/49 both solvers (18/49 pre-fix sparse, 13/49 klu -- 31 and 36 discriminating), regression 413/413. No openvaf-r change.

E-500 *src/frontend/com_dl.c, src/frontend/inp.c, src/frontend/com_presnp.c, src/frontend/commands.c, src/osdi/osdiregistry.c, src/include/ngspice/osdiitf.h, examples/vacompile_examples/ (new)*

`pre_osdi -va` COMPILES VERILOG-A AND LOADS IT IN ONE STEP. A deck shipping its own models needed a separate build first. `pre_osdi -va rmod.va gmod.va` now invokes openvaf-r and loads what it produces. Objects go in an **`osdi/`** directory BESIDE THE NETLIST -- a model directory stays clean, and an input/output collision becomes structurally impossible (E-452: `openvaf-r m.va -o m.va` destroyed the source, exit 0). The `.va` resolves against the netlist too, as `load_osdi` already does -- the first version got this wrong and worked from one directory but not another. Compiler lookup is **`pre_snp`**'s, exported not reimplemented: `openvaf` variable, `$OPENVAF`, `$SPICE_LIB_DIR/openvaf-r`, `PATH` -- a bare `PATH` search would miss the compiler in the shipped bundle. RECOMPILING IS THE DEFAULT, `.option osdicache` opts out: a `.va` timestamp says whether the SOURCE changed, nothing about the COMPILER, so while openvaf-r is under development a cached object may come from a compiler that no longer exists (E-453's shape). Read from the options CARDS, not `cp_getvar` (E-464: nothing is published at pre_ time), matched as a whole word not by `strstr` (E-451), every off-spelling off. **`-f`** FORCES THE REBUILD -- E-229 added it for the edit/recompile/re-source loop and the compile is now part of that loop, so forcing only the load reloaded the object being replaced; suite edits a `.va` and checks 0.5 -> 0.3333 only with `-f`. Staleness is strictly newer, never `>=`: `st_mtime` is 1 s coarse on POSIX, 2 s on FAT, so a tie rebuilds -- a tie costs a compile, the other way costs a wrong answer. Portable: `stat`/`st_mtime` as `inpcom.c` has used for years, `mkdir` guarded on `_WIN32` -> `_mkdir`. Mixes with plain **`pre_osdi`** in every arrangement, since `-va` only changes how a `.va` ARGUMENT becomes a path. Refusals name the cause; a read-only directory falls back beside the netlist. UNCHANGED: plain `pre_osdi file.osdi`; `-va` with no `.va` argument is a no-op. Verified `vacompile` 33/33 both solvers (pre-fix the suite cannot complete: `-va` was taken as a filename), regression 414/414. No openvaf-r change.

E-501 *src/frontend/com_aging.c, src/frontend/com_aging.h, src/frontend/com_sweep.c, src/frontend/com_optimize.c, src/frontend/runcoms2.c, examples/dcenter_examples/dcenter_demo.cir, examples/agestate_examples/ (new)*

THE AGED DEVICE CAME BACK FRESH. `aging` (E-157) writes an accumulated dose into each device's `age` with `alter` and leaves the circuit standing. That dose lives ONLY in the running circuit -- it was never in the deck -- so every command that re-sources between evaluations threw

it away on the first one and ran the rest of the loop on a FRESH circuit, silently: `wcd`, `highsigma`, `optimize -center`. A 95-year dose moves the worst-case distance 8.66 sigma -> 1.81 sigma; `wcd` printed 8.66 for both. `montecarlo` escaped by accident -- E-346's fast path arms whenever a random value binds and does *no per-sample reset*. Doses are now recorded as written and replayed after an INTERNAL reset; a `reset` the USER types drops the record, because that is what `reset` means. **`aging`** ACCEPTED TARGETS IT COULD NOT AGE TO: `aging nan` wrote NaN doses and reported them aged (every comparison with NaN is false, so `t <= 0` let it through), `aging abc` aged to 0 s, `dynamic verbose` ate `verbose` as the stop time then blamed `agerate` -- E-497's shape. A NaN SPEC BOUND IS NEVER VIOLATED, so it is identical to omitting it: `montecarlo -max nan` reported 100% yield with a Wilson interval. `-max/-min/-tol/-step/-scale` now parsed across all four statistical commands -- with `ft_numparse`, NOT `strtod`: the first guard refused `dynamic 20u`, E-157's own documented spelling, and the existing `aging` suite caught it. A NEGATIVE bound stays legal, which is why these are not E-499's `-tol` reader. A YIELD FROM SAMPLES THAT NEVER VARIED is one point measured N times; E-495 warns per `agauss` call, this is the whole-run version. **`optimize`** PUBLISHED THE SCORE BUT NOT THE ANSWER -- `dcenter_yield/dcenter_cpk` but never the centred knob, so the shipped dcenter demo printed *checkvalid: vector xc is not available* on every run since it shipped; now `optimize_<name>`. **`aging 1e8 param w`** wrote 4.095e8 into a MOSFET WIDTH (409,500 km), reported 1 device aged, rc=0 -- warns and proceeds when the device also has an `age`. Verified `agestate` 30/30 both solvers, 19 fail pre-fix, regression 415/415. No openvaf-r change.

E-502 *src/frontend/parser/numparse.c, src/include/ngspice/fteext.h, src/frontend/com_envelope.c, src/frontend/com_emir.c, src/frontend/com_eye.c, src/frontend/com_qpss.c, src/frontend/com_reduce.c, src/frontend/com_hbosc.c, src/frontend/com_compose.c, src/frontend/com_rfstab.c, examples/nanguard_examples/ (new)*

THE GUARD THAT REFUSES THE WRONG VALUE AND ADMITS NaN. Six report-producing commands wrote `if (x <= 0.0) refuse;`; every comparison with NaN is false, so that refuses 0, refuses a negative number, and lets NaN straight through. **`envelope <node> nan <tstop>`** SIGSEGVd 5/5 -- `T = 1/fc` NaN, internal `tran nan nan 0 nan` REFUSED, refusal never checked, `EFanalysis` derefs a matrix never built (`SMPmatSize`, 0x10). Both halves fixed: the argument, and the unchecked internal run that any refused transient could trigger (E-438's shape). A LIMIT NEVER EXCEEDED: `emir jmax nan` = *0 segments over Jmax*, all `ok`, on a grid with 2 genuine violations; `reduce nan` = *26 nodes -> 26 nodes (1.0x)* and a `reduced.sp` that reduced nothing; `qpss v(o) nan 1.1e6` = a 25-row all-nan spectrum; `eye -ui nan` = eye height 0, a fully closed link -- E-501's NaN spec bound one command over. TWO RETURNED A DIFFERENT ANSWER, NOT A BLANK: `eye -tstart nan` never skipped (the skip test is a comparison too) so the startup transient was folded in for 660x the RMS jitter; `qpss maxorder nan` silently became `order <= 1`, dropping every intermodulation product. Both clamped through an undefined double->int conversion, as did `compose lin=1e12` -- INT_MAX, 17 GB really allocated. **`emir`** COULD NOT TELL A GIVEN WIDTH FROM THE DEFAULT: `@r[w]` answers 1e-5 for an undimensioned wire, so the likely oversight was analysed as a 10 um conductor and reported `ok`, while `w=0`/`w=-0.5u` were skipped and mislabelled *no width given*; now `REWidthGiven`, two distinct messages. The idiom was already in the tree (`hbosc K >= 1`, `compose`'s per-parameter NaN refusal, `stb` delegating to `ac`); guards now share `ft_argpos`/`ft_argfinite`/`ft_argcount` in `numparse.c`, parsing with `ft_numparse` so `thick 0.5u` and `1meg` keep working. Verified `nanguard` 34/34 both solvers, 23 fail pre-fix, regression 416/416. No openvaf-r change.

E-503 *src/spicelib/analysis/cktdojob.c, src/include/ngspice/cktdefs.h, src/frontend/com_sweep.c, src/frontend/com_sweep.h, examples/reusedev_examples/ (new)*

ONE TRANSISTOR NO LONGER DISABLES SETUP REUSE FOR THE WHOLE CIRCUIT. E-471 offered reuse only to circuits built entirely from types whose topology is fixed UNCONDITIONALLY (linear elements, sources) plus OSDI, so a single BJT in an otherwise linear deck cost 3.46x at 300 sections, 10.09x at 1200. A per-TYPE gate for a per-PARAMETER hazard: a built-

in semiconductor decides its collapse in `DEVsetup` from a knowable set and nothing else -- BJT `rc/rb/re/`**`rco`**, diode `rs/rsw/vp/tt`, JFET `rd/rs`, Mos1/2/3/6/9 `rd/rs/rsh/nrd/nrs` -- each read off the device's own setup by auditing EVERY `CKTmkVolt` and its guard, not from the parameter docs, which do not say which parameters build nodes (`rco` gates a FOURTH BJT node via its `Given` flag; missed on the first pass). The sweep now DECLARES what it varies: 2.28x / 3.25x / 6.69x recovered. THE ABSENCE OF A DECLARATION KEEPS IT SAFE -- `sw_request_reuse( )` CLEARS it, so montecarlo and any `.param` knob inherit nothing and keep the strict gate; whole-token match, so `rsh` is not `rs`. The declaration is a file-static, NOT a **`CKTcircuit`** field: in the struct it grew `CKTcircuit` 256 bytes and made `argguard`'s warn_physics count random -- PADDING ALONE reproduces that, so a stale-layout read exists elsewhere and is recorded as an OPEN finding rather than made reachable here. Verified `reusedev` 16/16 both solvers, 8 fail pre-fix, every reuse check BIT-IDENTICAL against `reusesetup=0` after asserting the curve moves. Regression 417/417. No openvaf-r change.

E-504 *src/osdi/osditrunc.c, src/osdi/osdidefs.h, examples/rtdomain_examples/ (new)*

A MODEL MAY NOT DEMAND AN UNBOUNDED STEP COUNT. Two ways to hang the transient >150 s with no output, no error, no *timestep too small*. `$bound_step(1e-18)` is a perfectly LEGAL positive request taken literally -- that check compares against `CKTdelmin` (~5e-20 for a 12 ns analysis), far BELOW what was asked, so nothing fired: the step was not too small for the solver, it was too small to FINISH. No clamp makes 1.2e10 steps work, so the rule is the only one that bounds the run -- a model may force at most 10^6 steps across the analysis window, then it is clamped and the instance NAMED ONCE. `$discontinuity(0)` outside any conditional pinned the step to the last accepted delta so it could never grow (E-55 already made the RETRY edge-triggered for this reason; the CAP needed the same floor); `$discontinuity(-1)` means *no discontinuity*, is fine, stays silent. The compiler-side halves are in the openvaf-r report. The noise sign is deliberately NOT fixed here: by `osdinoise.c` the contribution factor is folded in as 'fac*

E-505 *src/osdi/osdiinit.c, examples/dropguard_examples/ (new)*

AN OPVAR NAME LOST TO THE SIMULATOR. ngspice writes its own instance parameters -- `m`, `temp`, `dtemp`, `dt` -- into the lookup table BEFORE anything the model declared, so a model operating-point variable with one of those names is computed on every evaluation and can never be read back: `@n1[temp]` returned the ambient temperature, `@n1[m]` the multiplier. Legal Verilog-A, and nothing said a word. Named at load time, not refused -- the rest of the model works. OPVARS ONLY: a model may legitimately declare its own `m`/`dtemp` PARAMETER (E-394's `has_m`; a CMC-style model scales by it and the loader routes the deck value in), and the first, too-broad version warned for those too until `limguard` caught it. The three compiler-side halves are in the openvaf-r report. Verified `dropguard` 22/22 both solvers, 11 fail pre-fix, regression 419/419.

E-507 *src/spicelib/parser/{inpgval.c,inpgmod.c,inpptree.c}, src/include/ngspice/inpdefs.h, src/frontend/outitf.c, examples/scanfmt_examples/ (new)*

A VALUE NOTHING CHECKED HAD BEEN READ. numparam substitutes the TEXT of a `{...}` expression, and `{1/0}` substitutes `inf`, which `INPevaluate` refuses. `INPgetValue`'s `IF_REALVEC` path honours that refusal; its `IF_REAL` and `IF_INTEGER` paths took the same `error` and THREW IT AWAY, so the value landed as ZERO. `.model nm nmos level=1 vto=0.5 kp={1/0}` therefore built a transistor conducting 1e-12 against 1.25e-4 -- eight orders of magnitude, exit code 0, no diagnostic. Writing `inf`/`nan`/`1e400` DIRECTLY on the same card is refused, and so is `{1/0}` on an instance line, as a built-in device's value, in an analysis argument (TSTEP/TSTOP/AC points) and through `altermod`: the model card was one path out of six, for OSDI and built-in devices alike, and the zeroing happened BEFORE the range check so a `from` clause could not catch it either. `INPgetValue` cannot simply return NULL for these types the way its vector path does -- none of its call sites test for NULL, which would trade a wrong number for a null dereference -- so the failure is RECORDED and the model-card path asks for it, that being the one caller with no other way to find out; it reports and keeps the parameter's DEFAULT, as the surrounding `.model` con-

ventions already do for an unknown or duplicated parameter. AN ORDINARY LEAF WAS NOT IN THE RELEASE SWITCH: `PT_PLACEHOLDER` is node type 0 and a normal leaf (*for i(something)*), so freeing one fell to `default` and printed *Internal error: unhandled parse-tree node type 0* -- on an ERROR path, where a half-built tree is released, ahead of the real and correct diagnostic. AND **`save`** CALLED AN OPERATING-POINT VARIABLE A MISSING PARAMETER: `INPaName` both FINDS a name and ASKS the device for its value, and any failure of the second half arrived as the first half's message, so a name the device does have -- and that `print` and `meas` both resolve -- was reported as *device has no parameter*, while the netlist `.save` form reported the same case correctly. Only `E_BADPARM` produces that message now. The compiler-side halves are in the openvaf-r report. Verified `scanfmt` 31/31 both solvers (19 fail pre-fix), regression 421/421.

E-509 *src/spicelib/parser/inpgval.c, src/spicelib/parser/inpgmod.c, src/include/ngspice/inpdefs.h*

AN INTEGER MODEL PARAMETER THAT DOES NOT FIT WAS SATURATED BEFORE ITS RANGE CHECK. `(int) round(tmp)` is undefined behaviour once the rounded value leaves the integer range; on this target it saturates, so a card saying `n=1e300` silently applied 2147483647 -- 291 orders of magnitude from what was written. The saturation happening BEFORE the parameter's own range check is what made it dangerous rather than merely surprising: a parameter declared `from [0:2147483647]` -- the idiomatic *any non-negative integer* -- accepted 1e300 because the clamped value landed exactly on its upper bound and passed. `sp=200` against `[0:100]` was refused correctly the whole time; only the values that OVERFLOWED slipped through, and those are the absurd ones. That is E-507's shape, a value mangled before the check that would have caught it. Flagged now the same way E-507 flags a `{...}` that evaluates non-finite: the model-card path reports it and keeps the parameter's DEFAULT rather than applying a fabricated number, via a new `INPlastRangeError`. Exactly INT_MAX, ordinary values and in-range values that ROUND are all still accepted. Verified in `domainrt` (28/28 both solvers), regression 423/423.

E-510 *src/spicelib/parser/inpgmod.c, src/frontend/com_dl.c*

EVERY RESISTOR **`.model`** CARD REPORTED A PARAMETER THAT IS NOT ON IT. `.model mm r`, `.model mm r rsh=1` and `.model mm res tc1=0.01` all produced *Error on .model mm : parameter (r) is not a number this parser accepts* -- and for the `res` card the message names the model TYPE token itself. The first token of a .model card IS the model type, and ngspice's resistor model has a parameter named `r`, so the type token matched a real parameter and the value check fired on whatever followed. The simulation was right and the message was not, on every resistor model card in every deck, on the shipped binary. **`first_tok`** already existed for exactly this collision -- it gates the duplicate-parameter warning a few lines above -- and the value checks added by E-507 and E-509 simply never consulted it; they are guarded by it now, and a genuinely unparsable value on a model card is still refused (E-507's contract is asserted by check [21]). Also: `pre_osdi -va` printed `system()`'s WAIT STATUS as the compiler's exit code, so 101 came out as 25856 (101 « 8) and 2 as 512 -- the comment at that site quotes *exit 512* as if it were an exit code, which is the same encoding gone unnoticed. Decoded with `WEXITSTATUS` now, and a signal death is named as such rather than printed as a status word. Verified in `lrmintrin` (21/21 both solvers), regression 424/424.

E-511 *src/frontend/spiceif.c, examples/optknown_examples/ (new)*

AN OPTION THE RUN HONOURS, REPORTED AS UNKNOWN. User-reported: `.option osdicache` printed *Warning: unknown option 'osdicache' on a .options card; ignored.* and then cached the compile anyway -- the run said it was ignoring a setting it had already acted on. Two enhancements colliding. E-500 reads `osdicache` straight off the OPTION CARDS in `inp.c`, deliberately NOT through `cp_getvar`, because `pre_osdi -va` runs before any option has been published and cp_getvar would answer for the previous deck (the trap E-464 recorded for `autobus`). E-438 later made an unrecognised name on a `.options` card a warning, and rightly so -- a misspelling used to be silently inert, so `.options reltoll=1e-12` left reltol at its default while the user believed the tolerance had been tightened. `osdicache` never reaches the simulator option table at all, so

E-438's check called it unknown. This is precisely the failure the allow-list in **spiceif.c** exists to prevent, and its own comment states it: *a warning that fires on a setting the run then honours is worse than no warning: it teaches the user to ignore the check E-438 added.* autobus, saveused, klu, reusesetup and noinit are all listed there; osdicache was simply never added. **seedinfo** had the IDENTICAL defect, found by checking whether osdicache was alone: read from the deck the same way and acted on by setseedinfo(), which makes randnumb.c print the seed it chose. The check must still bite, or the fix would undo E-438 -- a misspelt osdicaches and E-438's own reltoll=1e-12 are both still reported (checks [4] and [5]). NOT A DEFECT, worth knowing: a .va and its .osdi written within the SAME SECOND skip the cache and the model rebuilds -- st_mtime has one-second granularity so the staleness test is STRICTLY newer; a tie costs one needless recompile, the other way loads the object built from the PREVIOUS text. Also observed and left alone: seedinfo and seed= must share one option card. Verified optknown 13/13 both solvers (6 fail pre-fix, measured 7/13 before), regression 425/425.

E-513 *src/spicelib/devices/vsrc/vsrcdefs.h, src/spicelib/devices/vsrc/vsrctemp.c, src/spicelib/devices/isrc/isrcdefs.h, src/spicelib/devices/isrc/isrctemp.c, examples/srcnote_examples/ (new)*

A NOTE ABOUT THE DECK, REPEATED ONCE PER ANALYSIS. User-reported from a 100-sample Monte Carlo: *Note: v1: dc value used for op instead of transient time=0 value.* printed 100 times. The note describes a STATIC property of the deck -- the source carries both a DC value and a transient function whose t=0 value differs, so the operating point uses one and the transient starts from the other -- but it is emitted from VSRCtemp, which CKTdoJob runs once per ANALYSIS. Any loop command repeats it per point, and three plain tran commands in one .control block printed it three times for the same reason. The worse half is the collision: E-477's loop bar redraws with \r, so every copy landed on that line and mangled both -- ...time=0 value.ran 47%) is the note's tail and the bar's ran 47%) sharing a row. Two features that are individually correct, producing corrupt output together. FIX: latch the note per instance (VSRCdcNoteDone, and the same for ISRC), following the bitfield idiom already in those structs. Device instances are rebuilt when the deck is re-sourced, so *once per instance* is exactly *once per deck load, and again after a reset*, with no extra bookkeeping and no global state. Three trans -> 1 note, 100 MC samples -> 1, across a reset -> 2 (said again, correctly), and a source whose dc EQUALS its t=0 value or has no dc at all stays silent, as before. DELIBERATE TRADE: an alter that later creates or removes the mismatch is not re-announced -- an informational note about a deck property, where *once per load* is the normal treatment, but a trade rather than a free win. NOT A DEFECT: a deck can avoid it entirely -- V1 in 0 dc 1 PULSE(0 1 ...) asks for 1 V at the operating point and 0 V at t=0, and dropping the dc 1 removes the disagreement. Verified srcnote 12/12 both solvers (3 fail pre-fix, measured 9/12), with checks [7]/[8] asserting no progress frame carries the note's text and the bar still reaches 100%; regression 427/427.

E-514 *src/osdi/osdiload.c, src/osdi/osdisetup.c, src/osdi/osdidefs.h, examples/lrmops_examples/ (new)*

Analog-operator state seeded with ZERO instead of the converged operating point. LRM 4.5.7 states Output(t) = Input(max(t-td,0)), so before one delay has elapsed absdelay must return the input at t=0; absdelay_stamp_tran stored a literal **0.0**, so a delay around any non-zero bias read 0 for the whole first td and then stepped to the bias. Seeded from CKTrhsOld at MODEINITTRAN, where the operating point has converged. LRM 4.5.10 requires last_crossing to return a negative value until the expression has crossed -- the only way a model can distinguish *not yet* from *crossed at t=0* -- and it returned **0.0**; the sentinel is now OSDI_LAST_CROSSING_NONE, and the crossing history is seeded from the operating point too, because for an expression already above zero the zero-filled history made the first accepted point look like a rising edge and overwrote the sentinel before any model could read it. Three existing suites were updated to match: reusestate expected the literal 0.0 sentinel, and limguard/defaulttransition are covered in the openvaf report.

E-516 *src/osdi/{osdicallbacks,osdiload,osdiaccept,osdiregistry}.c, src/osdi/osdidefs.h, src/include/*

ngspice/osdiitf.h, src/spicelib/analysis/dctrcurv.c

THE SIMULATOR HALF OF LRM 9.4.6/9.5.9 DEFERRAL. Every Verilog-A display funnels through `osdi_log`, so the accepted-iteration buffer lives THERE: DISPLAY/MONITOR-class messages without the compiler's immediate tag are held, OSDIload drops the superseded iteration's output when a new Newton iteration starts, and OSDIaccept / OSDIfinalStep / a new per-point flush in the .dc sweep print what the accepted point produced. Detecting 'new iteration' is the subtle half: STATnumIter is bulk-updated only when NIiter RETURNS, so it cannot separate the iterations of one solve -- but NIiter swaps CKTrhs/CKTrhsOld once per iteration, so the composite (circuit, RHS pointer, time, mode, iteration total) differs between adjacent iterations. $monitor change detection compares the k-th monitor text of the flush with the k-th of the previous flush. The file-side hooks (`osdi_io_iter_begin/osdi_io_flush`) are resolved per loaded object with dlsym and are OPTIONAL both ways: an old .osdi lacks them and writes through; an old ngspice never calls them and the runtime stays unmanaged. Display output produced BEFORE the first Newton iteration (instance setup evaluating hoisted constant statements) prints through immediately, preserving the documented init-time behavior.

E-517 *src/spicelib/parser/{inpgmod,inpdpar,inpgval}.c, src/include/ngspice/inpdefs.h*

THE NETLIST HALF OF LRM 3.4.7 AND 3.4.1. Setting a parameter by its original name AND an aliasparam (or two aliases) on one .model card or instance line is an ERROR now -- the LRM says 'it shall be an error ... regardless of how the override is done', and E-395's warning let one value silently win (last-wins for model params, first-wins for instance defaults, deliberately unspecified). Setting the SAME spelling twice stays the E-395 warning. And the silent lossy round is audible: a non-integral value applied to an integer parameter is still rounded per the LRM rule (E-399's round-half-away-from-zero), but the E-507/E-509 flag pattern in `INPgetValue` now records that rounding CHANGED the value, and both the .model-card and instance-line appliers print a warning naming the parameter -- the footgun being an UNTYPED Verilog-A parameter whose type froze as integer from its default, where `untyped=2.5` simulated as 3 without a word. Integral values stay silent, verified.

E-520 *src/osdi/{osdiregistry,osdicallbacks}.c, src/osdi/osdidefs.h*

$LIMIT FALLBACK PER LRM 9.17.3, AND THE vdslim ALIAS. 'If the string refers to an unknown or unsupported function, the simulator is responsible for determining the appropriate limiting algorithm, just as if no string had been supplied.' E-396 made an unresolved $limit name a hard LOAD FAILURE -- justified then, because the alternative was a NULL function pointer the compiled model still calls (SIGSEGV, no output). The mandated fallback exists now: the slot binds `osdi_limit_unknown`, a pass-through returning the access-function value -- one adapter serves every arity, because extra C arguments are simply never read -- and a load-time WARNING names the function, the arity, and the supported set. The model loads and runs; only its convergence aid is absent, exactly the no-string behavior, and the SIGSEGV stays fixed because every slot always gets a real function. Annex E Table E.2's preferred name `vdslim` -- which this tree spells `limvds`, so a model written against the LRM's own table was REFUSED -- now binds to the same DEVlimvds implementation, in both the compiler's L020 lint set and this loader.

E-522 *src/osdi/osdiload.c*

THE SIMULATOR HALF OF LRM TABLE 5-1: STEP EVENTS FIRED FOR THE WRONG ANALYSES. @(initial_step('dc')) fired at the operating point of an .ac and a .noise analysis -- the phase runs with MODEDCOP set, so it carried ANALYSIS_DC in addition to the owning analysis's flag from E-53's job consultation -- and @(final_step('ac')) fired at the end of a .noise run, whose CKTmode carries MODEAC

E-523 *src/osdi/osdiload.c, src/osdi/osdidefs.h, src/osdi/osdiregistry.c*

THE SIMULATOR HALF OF LRM 4.5.7: ABSDELAY'S FROZEN td. The compiler's absdelay

info structs grew a flags word (absdelay_info_size 12 -> 16, gated on the descriptor's declared size so older .osdi files load unchanged) with OSDI_ABSDELAY_TD_FROZEN marking the two-argument form; the instance data lazily allocates a latched-td array, captures the live td value at the first transient evaluation (MODEINITTRAN, negative values clamped to 0), and every later evaluation of a frozen entry reads the latch while maxdelay entries keep reading the live value -- so `absdelay(V(in), td_expr)` holds the delay 4.5.7 fixes at initialization (the audit's deck reads the input 4.99ms back where tracking gave 3.99), and the three-argument form keeps its bounded tracking semantics. Verified under `lrmcoreops` 18/18 both solvers.

E-527 *src/osdi/osdiload.c*

$SIMPARAM$STR SERVES analysis_type AND cwd (LRM Table 9-28). The string channel served only analysis_name and the non-LRM 'simulator'; requesting any other Table 9-28 name was a compile warning and then FATAL at run time. analysis_type answers with the same derivation as analysis_name (ngspice gives analyses no user-chosen names distinct from their type -- the honest answer) and cwd answers the working directory, refreshed per query so a .control `cd` is followed. module/instance/path stay OUT, documented: the simparam channel is filled per circuit with no instance identity in reach, and inventing one would be worse than the honest warn-then-fatal. The compiler's served-name list moved in lockstep so L025 stays truthful. Verified under `lrmkernel` 43/43 both solvers.

E-528 *src/osdi/{osdiload,osdiacld,osdinoise}.c*

ANALYSIS PHASES AND NOISE, AUDITED AGAINST LRM 4.6 (simulator half). Table 4-22's TRAN OP column holds: analysis('ic')/analysis('static') are 1 at the t=0 operating point that precedes a transient and 0 at the timepoints -- the bits rode MODEINITTRAN, ngspice's FIRST ACCEPTED TIMESTEP, so the LRM's own if(analysis('ic')) initial-condition idiom fired mid-transient; they map to MODETRANOP now. analysis('nodeset') has a writer: the ANALYSIS_NODESET flag existed in the shared header, the compiler tested it, and nothing ever set it -- it is raised exactly while ngspice enforces .nodeset values (MODEINITJCT/MODEINITFIX with CKThadNodeset), latch-verified 1 with a card and 0 without. ac_stim matches the RUNNING small-signal analysis (4.6.3): .noise reuses the AC loader, and the activation compared the literal 'ac' -- an ac_stim('ac') source was injected into the noise gain solve, poisoning the input-referred noise 1000x (measured 5.75e-12 against a 5.76e-9 control), while ac_stim('noise') could never activate even though this simulator names the analysis 'noise'; the two decks swap roles exactly now. And the noise bridge correlates by CALL SITE: the compiler exports each source's call id after a \x1f separator in the name; grouping compares the full string (one call's multi-branch entries keep summing coherently), the per-label summary vectors strip the suffix, and a reporting fold combines a label's powers onto one entry per 4.6.4.1 -- two separate same-labelled calls no longer cancel as signed amplitudes where the LRM requires uncorrelated power addition. Verified `lrmnoise` 17/17 both solvers.

E-529 *src/osdi/{osdiregistry,osdiinit,osdiload,osdiparam}.c, src/osdi/osdi.h, README_OSDI.md*

THE OSDI LAYER, AUDITED -- the conformance audit's last area. Original-OpenVAF v0.3 objects are REJECTED with a recompile message: the 'must be version 0.3' path accepted them and read them through the EXTENDED in-repo layout -- the OsdiNode stride grew 48 -> 56 bytes (E-45), so every node record past the first was misread (a spec-conformant fake object listed its second terminal's current as i_V, the units field); osdiacld.c/osditrnoise.c read num_ac_stim_src and friends past the 0.3 descriptor's end; and 0.3's five-argument load_noise was called with four -- subtly wrong DC metadata and a transient SIGSEGV with zero diagnostics. osdi.h drops its stale 'matches the OSDI specification' claim and the v0.3 CURR constants for an explicit divergence note. The phantom IFparm row is gone: the synthesized m-alias slot was counted whenever the model lacked its own m but only WRITTEN when a $mfactor parameter exists, leaving the table's last row zeroed -- keyword NULL, a '(null)' devhelp row one strcmp from a crash for foreign objects; the count now mirrors the writer's condition. The OpenMP eval branch gets the serial

branch's initial-step gating (a task-local OsdiSimInfo, one task per instance): an --enable-openmp build never fired @(initial_step) in any OSDI device. A NEGATIVE multiplicity is warned and ignored on every route -- alter included, which bypassed E-447's parser warning and APPLIED the value: a 1k resistor model sourcing +4mA, and .noise printing 'onoise_spectrum = nan' through the compiled sqrt(m) factor; m=0 stays E-426's silent disable-this-instance idiom (the inputguard/instknobs suites caught the first over-broad cut of this guard immediately). README_OSDI gained a version-support and deliberate-bounds section, the $bound_step (tstop-tstart)/1e6 per-model floor (E-504) now disclosed there and in the compliance doc. Verified `lrmosdi` 10/10 both solvers, including a fake v0.3 object the suite compiles itself.

E-530 *src/osdi/{osdisetup,osdiregistry,osdiparam}.c, src/osdi/{osdi.h,osdidefs.h}, src/include/ngspice/osdiitf.h, src/frontend/spiceif.c, README_OSDI.md*

`.OPTION OSDIMC` -- THE SIMULATOR RUNS THE WHOLE MONTE-CARLO LOOP (simulator half). With the option set (alias `automc`), every run-class command starts a fresh trial: each parameter the Verilog-A declared with `(* std= *)` statistics is written nominal+draw through the ORDINARY parameter setter -- the `alter` path -- so there is no `reset`, no netlist re-expansion, and no `gauss()`/`agauss()` expressions in the deck; the existing setup/temperature machinery derives everything downstream, node-collapse rebuilds (E-471) included. The FIRST run after sourcing is the nominal BASELINE -- defaults of parameters the deck never set resolve during setup (E-476), so nominals are only knowable after one pass; draws begin with run two, uniformly. A MODEL parameter draws once per card per trial (process: instances sharing the card move in lockstep, distinct cards independently); a `(* type="instance" *)` parameter draws per instance (mismatch). Draws are PURE functions of (mcseed, trial, owner name, param id) -- splitmix64 into Box-Muller, no stored RNG state, the compiler's (seed, salt) $random philosophy -- so a fresh process reproduces every value bit-for-bit, `.option mcseed=` swaps whole ensembles, and `resume` never redraws. `alter`/`altermod` RECENTER a parameter's nominal (a hook in OSDIparam/OSDImParam -- draws are nominal+delta, never a random walk); dropping the option restores every drawn parameter exactly; a re-`source` invalidates the table; `.option osdimc_verbose` prints every draw; all four option names joined if_is_option's list so no honoured card is called unknown (the E-471/473 lesson). The suite MEASURES the machinery: over 300 trials gauss sigma=25 reads 25.1, uniform fills exactly [nominal-std, nominal+std], std_rel 5% of 2.0 reads 0.0998, mismatch sigma 9.65/10 with per-instance independence beside per-card lockstep; an attribute-free model is provably untouched and the draw path adds no measurable overhead beside a PSP103. DELIBERATE BOUND, documented in README_OSDI and handbook 3.6: a draw violating the parameter's Verilog-A range fails that run with the device's own located error, exactly as the same `alter` would (the descriptor exports no ranges) -- size sigmas accordingly; correlated parameters are future work. Verified `osdimc` 24/24 both solvers; full sweep 444/444 ALL OK.

E-531 *src/frontend/{spiceif,device,dotcards,inp,rawfile}.c, src/include/ngspice/{fteext,osdiitf}.h, src/osdi/{osdisetup,osdiparam,osditrnoise}.c, examples/huntfix_examples/ (new), examples/osdimc_examples/ (24 -> 29 checks)*

THE BUG-HUNT ROUND (simulator half): the E-530 machinery hardened and the alter/batch/rawfile surface made honest. MACHINE WRITES NO LONGER RECENTER MONTE-CARLO NOMINALS: the recenter hook lived in the raw parameter setter, so a `.dc` sweep's save/restore -- and the `sweep` command's, and sensitivity's perturbations -- permanently shifted a statistical parameter's nominal to whatever value was in flight (a random walk seeded by one sweep). Recentering moved into a `doset_user` wrapper on the frontend's user `alter`/`altermod` sites (wildcards included); the pinned check shows the post-sweep draw landing on the exact nominal-0 delta. `reset`/re-source RESTARTS the MC cleanly (inp_dodeck drops the nominal table; `reset` reuses the CKTcircuit pointer, so the old identity check missed the reallocation -- entries leaked and a heap-address reuse could have matched a stale nominal to a fresh device). A FAILED TRIAL SAYS SO IN-BAND: the range error names the model and value ("Parameter rd of 'mm' is out of bounds (value -0.489)!") and a once-per-trial notice states the previous run's vectors remain cur-

rent -- a 1000-trial script can no longer silently duplicate its last good sample. Draws are isfinite-checked (an overflowing sigma wrote -inf into parameter storage; only a declared range caught it). THE SURFACE: `altermod` reaches STRING parameters (`@mm[mode] = "quad"` died as `no such vector "quad"`; a new if_setparam_string routes it with altermod's CKTtemp propagation); whole-array altermod gets per-element guidance instead of "has no parameter cf" two lines above a showmod that prints it; `alter` REFUSES non-representable values (1e400 stored +inf: a built-in resistor silently became an OPEN -- i=0, rc=0, a perfectly plausible wrong answer -- and an OSDI device died in convergence noise pointing nowhere near the cause); repeated `run` of an .op+.ac+.tran deck keeps ALL its jobs (the batch epilogue's .op save-all was analysis-restricted, so the automatic run's other analyses failed with "no data saved ... analysis not run", plots silently lost); rawfile roundtrips keep qualified cross-plot names (`dc2.i(v1)` came back as the unaddressable `i(dc2.i(v1))`; same for v(...)); `alter` of a model parameter through a device name points at `altermod` instead of denying the parameter exists; the transient-noise noise_table warning prints the user's label instead of the raw 0x1F-suffixed correlation name (od-verified: the control byte reached the terminal). Three hunt findings were RETRACTED as artifacts of the hunt harness itself and are recorded as such; the handbook's stale ".disto not supported" row was corrected -- E-352's OSDIdisto matches an analytic ground truth (pointwise periodic solve + FFT) to <1%. Verified `huntfix` 12/12 and `osdimc` 29/29 both solvers; full sweep 445/445 ALL OK.

E-532 *src/osdi/{osdisetup,osdiload,osdiacld,osdipzld}.c, src/include -- via src/osdi/osdidefs.h, examples/huntfix2_examples/ (new)*

THE OPENVAF-R HUNT ROUND (simulator half): a chained terminal short was silently an OPEN circuit. `V(a,m) <+ 0.0; V(m,b) <+ 0.0;` with m internal is physically the SAME circuit as `V(a,b) <+ 0.0` -- and the direct spelling shorts correctly (E-401 emits a real 0 V source for a terminal-terminal 0 V contribution). The chained spelling conducted NOTHING: 0 A and the full source voltage across the device, zero diagnostics from either tool. The first collapse merged m onto one terminal; the second then faced a terminal-terminal merge, which node collapsing cannot perform (ngspice allocates terminal nodes), and the old loop silently `continued` -- the V = 0 EQUATION ITSELF vanished from the system. Chains of any depth failed identically; a chain landing on an internal node was fine, which is exactly why the hole survived (BSIM4-class gate ladders chain collapses like this, saved only by their conditions rarely all firing). THE FIX STAMPS WHAT THE HINT EXPRESSED: when collapse_nodes refuses a terminal-terminal or terminal-ground merge it records the pair, and OSDIsetup builds a SYNTHETIC IDEAL 0 V SOURCE between the two global nodes -- a branch equation V(n1)-V(n2)=0 plus the +-1 current entries, the vsrc stamp at dc 0 -- following the absdelay extra-entry pattern end to end: SMPmakeElt at setup, KLU COO->CSC rebind and real/complex pointer switching on every DC<->AC transition, stamps in the DC/tran, AC and PZ loads (OpenMP branch included), branch-equation REUSE across `sens`'s double-setup (which forbids node allocation), deletion in OSDIunsetup, netlist-tied endpoints and duplicate pairs dropped as redundant. REWRITING THE LOOP AS PROPER GROUP MERGES FIXED TWO LATENT CORRUPTIONS, both reachable through hint chains: a group already collapsed to GROUND, merged again, had its members re-mapped onto the other group -- quietly UN-grounding them -- while the node count decremented for a row that never disappeared; and a redundant hint between nodes ALREADY IN ONE GROUP (a collapse triangle) renumbered every node above the representative and corrupted the count the same way. Verified `huntfix2` 21/21 both solvers -- direct/one-internal/two-internal spellings identical in op, AC and transient, across `reset`, through `sens`, plus the ground-chain, un-ground and triangle shapes against closed-form currents; full sweep 446/446 ALL OK.

E-533 *src/frontend/com_sweep.c, examples/sweepdc_examples/ (new), examples/{loopbar,reusedev}_examples/ (re-pointed)*

`sweep` HANDS ELIGIBLE OP-POINT SWEEPS TO ONE DC ANALYSIS. With the default `analysis op`, a single dc-sweepable knob and evenly spaced points, the point loop -- npt COLD operating points, a full `op` job with the whole gmin/source-stepping fallback chain and one

retained plot per point -- becomes ONE dc analysis: a warm NIiter continuation from point to point (E-258), the sweep's outputs served from the dc plot. Measured on the motivating deck, a 1000-device OSDI Monte-Carlo ladder swept over 9900 resistance points: 21.2 s -> 2.16 s, bit-identical to a direct `.dc`, within Newton tolerance of the per-point loop (~2e-4 relative on a clamped diode divider); `-perpoint` forces the old loop. SAFE BECAUSE THE TWO ENGINES WERE ALREADY COMPLEMENTS, each pointing at the other: `.dc` REFUSES a point that moves an OSDI node collapse or leaves a model bin (E-495 -- its own message recommends `sweep`), aborts when a device rejects a value (E-427), and stops on non-convergence -- every such outcome reaches the handover as a failed run and FALLS BACK to the per-point loop unchanged, which re-decides topology per point and is the correct instrument for exactly those sweeps (pinned: an instance-parameter sweep across a collapse boundary falls back and lands on closed-form series-resistance values to 3e-13). Both engines set the swept value through the machine-write path and restore it afterwards, so neither recenters osdimc nominals (E-531) and frozen agauss() draws stay frozen. ELIGIBLE: bare source/resistor names, non-wildcard `@inst[param]` (E-62's `.dc` spelling), spacing judged from the VALUES (positional, `lin`, and a uniform `list` qualify; `dec`/`oct`/uneven lists cannot be represented as start + k*step); `@r2[resistance]` and `@v1[dc]` are rewritten to the bare spelling so `.dc`'s classic resistor/source arms step the value directly instead of routing every point through CKTsetInstParam + a full CKTtemp pass over every device. STAYING ON THE LOOP BY DESIGN: model-parameter and `.param` knobs (no dc arm), `-vs` families, `-overlay`, live `@dev[param]` outputs (only the loop can read the circuit at each point -- prescreened BEFORE any dc is run, so none is wasted), and `sweep temp` whenever the deck contains an OSDI device: `dc temp` holds one setup for the whole sweep and cannot follow a temperature-moved collapse -- the known-open finding the sweeptemp suite pins, whose own check names the per-point path as the correct instrument; built-in-only decks keep the temp speedup. FLOAT CARE, twice: `.dc` accumulates `value += step` against an ABSOLUTE overshoot test, so a span/(N-1) step can lose its endpoint to accumulated rounding -- the issued stop is padded by 0.49*step (the endpoint always lands, the next point always exceeds the pad and is dropped) and the dc plot's own scale values are ADOPTED into the sweep scale rather than assumed, with a count mismatch falling back. STATED HONESTLY: dc points run MODEDCTRANCURVE -- a Verilog-A `analysis("dc")` is true where the loop's op had `analysis("static")` -- and a warm continuation TRACKS a solution branch where independent cold ops re-decide it at every point; both are `.dc`'s own readings, restored by `-perpoint`. The announce line names the engine, so a log always says which path computed the numbers. Two suites were re-pointed at the machinery they actually pin: `loopbar`'s op-sweep vehicles carry `-perpoint` (the point-loop bar needs a point loop; a new check pins that the eligible default hands over and draws none) and `reusedev`'s eligible knobs likewise (the E-471/E-503 reuse decision lives in the loop). Verified `sweepdc` 15/15 both solvers; full sweep 447/447 ALL OK. No openvaf-r change.

E-534 *src/spicelib/analysis/{dctrcurv,dctsetp}.c, src/spicelib/parser/inp2dot.c, src/include/ngspice/trcvdefs.h, src/frontend/com_sweep.c, examples/dcxsweep_examples/ (new), examples/sweepdc_examples/ (15 -> 17 checks)*

`.dc` LEARNS THE REST OF THE PARAMETER SURFACE: model knobs, wildcards, subcircuits, and lin/dec/oct scales. The sweep variables the `sweep`/`altermod` family established now have a NATIVE dc arm, card and command alike: MODEL parameters (`@mod[p]`, with the dotted subcircuit `@x1.rmod[p]` resolved through the same E-433 hierarchy funnel the rest of the tooling uses) and the wildcard families `@*[p]` (every model with p), `@#*[p]` / `@*[[p]]` (every instance with p), `@*:leaf[p]` (every model named leaf, wherever expansion put it). Every knob kind, old and new, takes ‘lin

E-535 *src/osdi/{osdisetup,osdiparam,osdicallbacks}.c, src/osdi/osdidefs.h, src/frontend/{com_sweep, com_optimize,com_loadpull,evaluate}.c, src/include/ngspice/{osdiitf,trcvdefs}.h, src/spicelib/analysis/dctrcurv.c, examples/mcpolicy_examples/ (new)*

THE OSDIMC TRIAL POLICY FOR LOOP COMMANDS — AND THE HUNT ROUND THAT

MADE IT HONEST. E-530's per-run draw was right for a lone `op` and wrong at both ends of the loop-command spectrum: a swept "curve" was N DIFFERENT samples stitched together, `optimize` fit a STOCHASTIC objective (measured "converged" 2.4% off a closed-form optimum, reported with full confidence), and `montecarlo` — the one command that WANTS a fresh draw per sample — got none, its per-sample internal reset restarting the counter so every sample recomputed the nominal. THREE LEVERS: HOLD (`sweep`'s per-point path, `optimize`, `wcd`, `loadpull` bracket their inner analyses — the whole loop consumes exactly ONE trial; draws are pure functions of (mcseed, trial, owner, id), so the held sample survives even an internal re-source), PRESERVE (a command's own internal reset keeps the sequence running where a USER reset still restarts at the baseline), SCALE (`highsigma -scale` inflates attribute-declared sigmas along with the netlist ones). A ONE-HOUR ADVERSARIAL HUNT over the first cut produced seventeen findings; the five correctness fixes ship with the policy: (1) DRAWS NOW PRECEDE THE INIT-RESIDENT CODE — the first cut applied a preserved trial at the END of OSDItemp, after every setup_instance had cached its intermediates from the NOMINALS, so a parameter feeding only a hoisted assignment (geff=1/rr) was drawn in storage and nominal in the physics: every montecarlo sample computed the exact nominal while osdimc_verbose logged distinct draws (5/5 threshold violations where the drawn samples give 3 — the log said the samples differed, the answers said one circuit). The apply moved to the end of OSDIsetup: defaults are resolved there and CKTtemp/OSDItemp comes after, so the hoisted code evaluates the drawn values; a draw that moves node collapse is caught by the E-417/E-495 machinery like any between-setups change. (2) MACHINE WRITES PIN THEIR PARAMETER — the late-apply also ran on every OSDItemp (per `.dc` point, per `sens` perturbation) and the held re-apply per sweep point, overwriting machine writes with nominal+delta: `dc @n1[dr] 0 1000 500` returned ONE POINT THREE TIMES (flat 0.49596 where the curve runs 0.50→0.33), `sweep @n1[dr]` likewise, `sens` reported ~1e-14 where -2.5e-4 is closed-form. E-531 said machine writes never RECENTER; the missing complement: they must WIN — every successful OSDIparam/OSDImParam store pins its osdimc entry, the applier skips pinned entries, pins clear at bracket edges and at every un-held run (so `alter` recentering still lands next run, exactly as osdimc check 19 pins). (3) THE BASELINE IS EXPLICIT — trial-1-never-draws was an empty-table accident that an optimize flow or an `unset/set osdimc` toggle defeated (both then DREW on the run every contract calls the baseline); a trial<2 guard replaces it. (4) SETUP DISPLAY PRINTS PER SETUP — display_managed latched at the first Newton iteration forever, so a re-setup's hoisted $strobe was deferred-then-dropped: a `.dc temp` sweep printed the FIRST point's strobe, dropped points 2..N, then flushed the RESTORE's strobe where it read like a sweep point; OSDIsetup and OSDItemp now re-enter setup-phase display and every point prints at its own temperature. (5) $MONITOR HISTORY RESETS PER ANALYSIS — a second identical `op` printed nothing (its first accepted point matched the PREVIOUS run's last flush); reset per analysis, deliberately NOT per temperature point. ALSO: a nested `.dc` REFUSES two levels that move the same knob in any spelling (`v1` vs `@v1[dc]`, `@r2[resistance]` vs `r2`, wildcard-list overlaps — the first point was computed with the outer start and labeled with the inner value, and the restore parked the knob on the wrong "nominal"), with a full reverse-order restore on refusal; dec/oct point counts keep the PARSED N apart from the derived total (a re-run of a still-loaded card no longer refines its own grid 5→11→23); `evaluate.c` warns once when two genuine multi-element vectors of different lengths meet (the silent flat-extension of a truncated import now says so). KNOWN-OPEN, recorded with repro decks: interrupt longjmps leak the hold/sigma-scale/preserve flags (an interrupted highsigma measured continuing its sequence through a user re-source); optimize's internal runner lacks the preserve call (`-center` sees no osdimc variation); `highsigma -scale` carries no likelihood-ratio term for the inflated OSDI draws (P(fail) 0.42 reported where 0.354 is true — use netlist `.param` statistics with `-scale` until the weight channel exists); the hold is a flag, not a depth counter (a loop nested as another's `-analysis` releases the outer bracket); loop commands never poll ft_intrpt; a failed later-level `.dc` resolution leaves earlier levels applied. Verified `mcpolicy` 12/12 (new; closed-form expectations from the run's own readbacks, both solvers) and `osdimc` 29/29, `optimize` 43/43, `sweepdc` 17/17, `dcxsweep` 20/20, `montecarlo/highsigma/wcd` green; full sweep ALL OK. No openvaf-r change.

E-536 *src/frontend/{signal_handler,com_sweep,com_optimize,com_loadpull,outitf}.c, src/include/ngspice/{fteext,osdiitf}.h, src/osdi/osdisetup.c, src/spicelib/analysis/dctrcurv.c, examples/mcpolicy_examples/ (12 -> 18 checks)*

THE OSDIMC LEDGER CLOSED: interrupt safety, nested holds, weighted high-sigma, and the `.dc` restore. E-535 shipped its loop-command policy with an honest known-open ledger of nine hunt findings; this closes all nine, each repro now a pinned check. THE THROUGH-LINE IS STATE THAT OUTLIVED ITS COMMAND. AN INTERRUPT NO LONGER POISONS THE SESSION: `ft_sigintr` LONGJMPs to the prompt, and every loop command's cleanup sat AFTER that point of no return -- an interrupted `highsigma -scale 5` left the INFLATION ARMED (every later osdimc draw in the session used 5x sigmas, silently), a dangling hold froze all later runs on one Monte-Carlo sample, a dangling preserve carried the interrupted run's trial sequence through a USER re-source (measured: a fresh `source` continuing at trial 91828), `mc_wcd_off`/`mc_sss_off` never ran so the netlist gaussians stayed pinned to a chosen u-vector, `ft_optimizing` left TRUE muted every later banner, `aging_internal_reset` left elevated made later USER resets keep age records they must drop, and a half-drawn progress bar kept the terminal line; `ft_sigintr_cleanup()` -- which exists for exactly this -- now resets all of it. THE HOLD IS A DEPTH, NOT A FLAG: a loop command nested as another's -analysis (`optimize -analysis sweep .. .`, a swept-curve objective) had the inner sweep's release un-hold the OUTER optimizer, so every evaluation took a fresh trial -- measured trials 2..15 across 14 evaluations, the stochastic-objective bias E-535 exists to eliminate, reintroduced through nesting; only the outermost edges now reset/release, and every caller releases exactly what it took (an error path that never took the bracket cannot pop someone else's). OPTIMIZE JOINS THE PRESERVE PROTOCOL: `opt_run_cmd` -- the optimizer's own copy of the runner E-535 taught -- never got the call, so a `-dparam` fit's per-evaluation re-source zeroed the counter (the objective silently changed mid-search) and `-center` saw ZERO osdimc variation (every sample of every candidate computed the nominal, so its yield ignored the model's declared statistics entirely); `-center` now runs WITHOUT the hold its samples must not have and instead REWINDS THE TRIAL COUNTER to a pre-search checkpoint, replaying the same trial window per candidate -- variation sampled, determinism kept, exact because draws are pure functions of (seed, trial, owner, id). HIGHSIGMA NOW WEIGHTS THE DRAWS IT INFLATES: `-scale` inflated the attribute-declared sigmas but `mc_sample_weight()` carries likelihood ratios only for netlist `.param` draws, so a metric depending on an OSDI statistical parameter had every sample at weight 1 and P(fail) was just the lambda-inflated failure fraction (measured 0.42 reported where the true value is 0.354, with a confident +-0.03 bar under it); a new `OSDImcSampleLogLR()` recomputes each gauss dimension's standard-normal deviate and sums log(lambda) - n^2(lambda^2-1)/2, multiplied into the weight beside the netlist term -- recomputed rather than accumulated, so the E-471 double-setup is harmless by construction, and uniforms are now explicitly NOT inflated (matching the netlist SSS policy: an inflated uniform can land where the true density is zero, which no finite weight represents). Validated against quadrature: 0.29670536 closed-form, the seeded suite run reporting 0.341 against a raw inflated fraction of 0.454. CTRL-C STOPS A LOOP COMMAND: none of them polled `ft_intrpt`, and with `ft_setflag` an interrupt aborts only the inner analysis it lands in while the next dosim() clears the flag -- so a Ctrl-C mid-`sweep` still announced "100000 points into plot 'sweep1'", every remaining point an insta-aborted analysis stitched in as data; each loop now polls at its iteration boundary (sweep marks un-run points `nan` per E-445 and says how far it got, montecarlo/highsigma report over completed samples, wcd refuses a half-converged MPFP, loadpull stops the grid, the optimizer methods break out and `opt_eval` short-circuits so a population method owes no further analyses), with the documented limitation that an interrupt arriving INSIDE a long inner analysis is consumed by that analysis's own OUTstopnow() -- ngspice's contract, shared by every multi-analysis command, left alone. THE `.dc` RESTORE COMPLETED: resolution APPLIES each nest level's start as it walks the levels, and a failure at a LATER level returned without putting the earlier ones back -- `dc v1 0.5 1.5 0.5 @n1[nonesuch] 1 2 0.5` left v1 parked at 0.5 V, a temp outer level left every temper-baked expression at the sweep's start temperature; the reverse-order restore

E-535 wrote for its clash refusal is now a shared helper every resolution-failure path calls. Plus: the clash guard catches a WILDCARD instance level covering a source/resistor level's principal parameter (`dc v1 0.5 1.5 0.5 @#*[dc] 0 2 1` moved v1's dc from both levels and left it corrupted), and `DCTresolveXParam`'s integer-refusal paths drop their collected target list instead of leaving dangling owner pointers on the job. Verified `mcpolicy` 18/18 (12 -> 18, both solvers, every new check a former ledger entry) and `osdimc` 29/29, `optimize` 43/43, `sweepdc` 17/17, `dcxsweep` 20/20, `sweeptemp` 20/20, montecarlo/highsigma/wcd/nestedsweep green; full sweep ALL OK. No openvaf-r change.

E-537 *src/frontend/{com_sweep,com_optimize,com_aging,com_emir,com_loadpull,evaluate,spiceif}.c, src/osdi/osdisetup.c, src/include/ngspice/osdiitf.h, examples/mcpolicy_examples/ (18 -> 28 checks)*

THE SECOND HUNT ROUND: WHAT THE STATISTICAL COMMANDS WERE NOT SAYING. A second one-hour adversarial hunt over the shipped E-535/536 work produced sixteen findings (fourteen with repro decks); this resolves all sixteen, two of them regressions E-536 itself introduced. THE HEADLINE IS A MIS-ESTIMATE E-536 SHIPPED: its `highsigma -scale` importance weight is a PRODUCT over every inflated statistical parameter in the circuit -- including ones the metric cannot depend on -- so its variance grows exponentially with their number. Measured with bystander devices on a DISCONNECTED subcircuit (identical `failures observed` every run, proving the physics untouched) against a true P(fail) of 0.2967: 0 bystanders 0.282 correct, 2 -> 0.072, 5 -> 0.018, 10 -> 0.0015, 20 -> 2.5e-11, ten orders of magnitude low and printed with the same confidence as a good answer. E-536 had traded a BOUNDED bias (0.42 against a true 0.354) for an UNBOUNDED variance -- better in one dimension, far worse in the decks people have, where per-instance mismatch supplies dimensions automatically. Unbiased-but-useless is what importance sampling DOES in high dimension, so the command now measures its own reliability: it computes the EFFECTIVE SAMPLE SIZE (sum w)^2/sum w^2 and, when that falls under a tenth of the run, states that the weights collapsed, gives the number (measured 1.2 out of 400), explains the cause and says P(fail) is not trustworthy -- a guard that also covers the pre-existing netlist SSS path, which degenerates identically (20 extra gaussians -> 2.4e-04; that weakness predates E-536, what E-536 changed is how easily an ordinary deck reaches it). Two more honesty repairs in the same summary: a weighted mean is NOT automatically a probability (it printed 1.0445 where all 200 samples failed and the true answer is exactly 1) so it is clamped into [0,1] and says when; and the equivalent-sigma guard substituted 0.000 at the boundary, which READS AS P = 0.5 -- the opposite of certain failure -- and now reports n/a. FIVE COMMANDS WERE READING ANOTHER RUN'S NUMBERS: ngspice keeps the previous plot when an analysis fails, and the E-438/E-445 guard was applied in sweep and optimize but MISSING in highsigma, wcd, aging, emir and loadpull. highsigma counted failed samples using the previous sample's metric -- on one deck montecarlo reported "10 of 60 samples failed to simulate and are EXCLUDED" while highsigma said nothing and folded all 60 in, and the bias has a DIRECTION: with -scale the failures cluster in the TAIL, exactly the region being measured, so the rare-failure probability came out systematically LOW; it now excludes, averages over what solved, and states that direction. wcd differenced its gradient against a stale value, making that component exactly zero -- with one dimension the whole gradient went to zero and the command announced "the metric does not respond to any statistical parameter", blaming the model for a point the SEARCH had chosen out of range (proven false on the same deck: at -max 0.6 it finds beta = 0.3704, and only when the search steps into the failing region does it accuse the model); it now names what happened, separately at the nominal point, at an iteration and at a gradient probe. aging computed a dose from a foreign bias point and then WROTE AND PERSISTED it; emir reprinted a complete reliability sign-off ("0 segments over Jmax") on an operating point that no longer existed; loadpull entered a failed grid point as a duplicate of its neighbour, where the reported optimum could sit on it. A MISTYPED MODEL PARAMETER NO LONGER KILLS THE SESSION: `altermod` with a value an OSDI Verilog-A range refuses called controlled_exit(1) -- harmless with no analysis yet and after `op` (the block is guarded by CKTtime > 0) and FATAL after any dc or tran, the state a real session is almost always in, while the same refusal on an INSTANCE parameter and on every built-

in model was only a warning; it now refuses and says the update did not take effect, as E-531 established for `alter` and as the sibling string-parameter path already did (E-534's comment shows the hazard was known -- it declined to reuse these setters for .dc points precisely because they "controlled_exit() on a CKTtemp error"). WHAT THE SAMPLING COMMANDS WERE CLAIMING: `montecarlo N` drew N-1 samples on a freshly sourced deck (sample 1 was osdimc's nominal baseline, a deterministic point folded into the yield and its Wilson interval, with the effective count depending on session history) and now steps past it; `-seed` did not vary the draws at all (`-seed 1` and `-seed 999` were byte-identical, so every "independent" replication returned the same points and an estimate looked perfectly stable when nothing had been re-sampled) and is now mixed into the draw key when given; `-lhs` silently did nothing for model-declared variability (it stratifies the netlist draw functions, which osdimc's pure-hash draws never enter -- byte-identical with and without the flag on E-530's headline deck) and now says so. highsigma also gained E-501's degenerate-metric check: a metric that never varies is a mistyped node far more often than a rare failure, and the old "increase -scale or N" sent the user to spend a bigger run chasing a quantity that does not exist. THE TWO E-536 REGRESSIONS: its sweep interrupt poll filled the results array at point 0, which is not allocated until the first point has run, so an interrupt arriving before that wrote NaN through a NULL pointer (now skips to the restore); and an interrupted `optimize` printed "converged" -- E-499 had already qualified that word for three other situations, and interruption is the fourth, now "INTERRUPTED -- best point so far". ALSO: aging's dose recentred an osdimc nominal because it writes through the `alter` USER channel while being a machine writer, so a statistically-declared aging parameter had the draw's randomness written permanently into its nominal and random-walked across repeated runs (0 -> 0.495253 -> 0.49919 -> 0.499013, non-monotonic, proving noise not accumulation) -- aging's writes and its E-501 replays are now marked machine writes, restoring E-531's rule; result variables were never invalidated, so after a wcd reporting "cannot locate an MPFP" the previous run's $wcd_beta = 0.37037 was still readable by the scripts they exist for, and each command now clears its namespace on entry; and the E-535 vector-length warning repeated per operation despite promising "once" (50 identical lines from a 50-iteration loop) and now latches. The handbook records the one finding that is a contract rather than a defect: under `.option osdimc` the two sweep engines are separate run-class commands, so a default run and a `-perpoint` run compare DIFFERENT Monte-Carlo samples (~2% apart), each correct for its own. Verified `mcpolicy` 28/28 (18 -> 28, both solvers, every new check a hunt finding with its own repro); full sweep ALL OK. No openvaf-r change.

E-538 *src/osdi/osdisetup.c, src/include/ngspice/osdiitf.h, src/frontend/{com_sweep,commands}.c, examples/mcpolicy_examples/ (28 -> 33 checks)*

`highsigma -scale` BECOMES SCOPEABLE, AND SO BECOMES USABLE. E-537 gave the command an effective-sample-size guard that says when its importance weights have collapsed -- half an answer, because nothing could name a subset of the variability to inflate, so the diagnosis came with no way out. WHY IT COULD NOT BE RESCUED BY ARITHMETIC: scaled-sigma importance sampling corrects an inflated density with a likelihood ratio per inflated dimension, and the weight is their PRODUCT, so its variance grows exponentially with their number -- while `-scale` inflated EVERY gauss statistical parameter in the circuit, including ones the metric cannot depend on. Under `.option osdimc` that count is not a modelling choice: a `(* type="instance" *)` mismatch parameter contributes one dimension PER INSTANCE, so an ordinary deck arrives in the degenerate regime without anyone asking for it (E-537 measured a true P(fail) of 0.297 reported as 2.5e-11 once twenty statistically-declared bystander devices joined the product). The estimator's algebra is not wrong -- it stays unbiased in expectation -- so there is nothing to correct; the only fix is to stop putting irrelevant dimensions into the product, which needs a way to say which ones matter. THE SPELLING: `-inflate <param>`, repeatable, taking either a bare parameter name (`rr`, meaning that parameter wherever it occurs) or the project's ordinary accessor `@owner[param]` with * allowed as the owner (`@mm[rr]`, `@*[rr]`), matched case-insensitively like every other name in this surface. With no `-inflate` the behaviour is byte-for-byte what it was, so no existing deck changes; the announce line says when a run IS scoped ("scale (sigma inflation) = 3 on the -inflate

parameters only"), so a log never leaves it ambiguous which mode produced the numbers. THE WEIGHT FOLLOWS THE SCOPE EXACTLY: a parameter left out of scope is drawn at its TRUE sigma and therefore carries no likelihood ratio at all -- including one anyway would bias the estimate and omitting an inflated one would too -- so the draw applier and the weight walker consult the SAME predicate on the same (owner, parameter) pair and cannot disagree about which value was drawn or how it is weighted. That equivalence is pinned: the same circuit, once with twenty bystander devices and `-inflate rr` and once without the bystanders at all, produces IDENTICAL numbers (0.29665 both ways) -- the bystanders now cost exactly nothing, which is the whole point. SAYING WHEN THE SCOPING DID NOT WORK: a spec matching NO statistical parameter inflates nothing, so the run samples the nominal spread while looking like a scoped result, and now reports that nothing was inflated; a MALFORMED spec (`@bad[`) is refused before the run rather than silently ignored, so a typo cannot quietly widen the scope back to everything; and E-537's collapse note now distinguishes the two cases it must advise differently -- an unscoped run is told to name the parameters that matter, a run already scoped and still degenerate is told to narrow further. MEASURED on the deck E-537 used to demonstrate the collapse (metric depends on one parameter, twenty statistically-declared bystanders on a DISCONNECTED subcircuit, true P(fail) = 0.29670536 by quadrature): unscoped at 2000 samples 3.35e-05 with the weights flagged as collapsed; `-inflate rr` at the same 2000 samples 0.2967; at 8000 samples 0.28944 +/- 0.00715; and the no-bystanders deck 0.29665, identical to the scoped run. Deliberately unchanged: netlist `.param` gaussians are inflated wholesale as always, since `-inflate` names OSDI statistical parameters -- a deck mixing both with many netlist gaussians can still degenerate, which the E-537 guard reports, and that side has its own established idiom; the osdimc side is where dimensions appear without being asked for, which is why it is the side that needed naming. Verified `mcpolicy` 33/33 (28 -> 33, both solvers); full sweep ALL OK. No openvaf-r change.

E-539 *OpenVAF: openvaf/osdi/stdlib.c, openvaf/hir_lower/src/{fmt,callbacks,expr}.rs, openvaf/osdi/src/{lib,compilation_unit}.rs; ngspice: src/osdi/{osdi.h,osdidefs.h,osdicallbacks,osdiregistry,osdisetup}.c, src/include/ngspice/{osdiitf,cktdefs}.h, src/maths/ni/niconv.c, src/spicelib/analysis/{cktdojob,cktntask}.c, examples/lrmio_examples/ (new, 17 checks)*

THE SECOND LRM AUDIT WORKED: FIVE DIVERGENCES FIXED, ONE FINDING WITHDRAWN. The 2026-09-02 re-audit raised six findings; five were real and are fixed here, and the sixth is withdrawn because it was never a defect. THE WITHDRAWAL FIRST, because the audit was wrong in a way worth recording: it reported that `$random`'s seed never advances and listed it among defects producing wrong numbers "with NO DIAGNOSTIC ON ANY CHANNEL". Both halves are false. The behaviour is DELIBERATE -- `hir_lower/src/expr.rs` calls it a "DELIBERATE DEVIATION from LRM 9.13.1's inout seed" because an advancing seed returns a different value on every Newton iteration and DESTROYS CONVERGENCE, which Enhancement-10 and `examples/rng_examples/README.md` also record -- and it is already reported by default as lint L019, naming the cause, the reason and the remedy. The audit tested the simulator and never read the compiler's own output. Three further findings (`%m`, multichannel descriptors, `$fscanf`) were likewise already on record in E-417, E-516 and E-11, so the audit's framing as "findings the first pass did not reach" was inaccurate for four of six; the audit page is corrected rather than quietly amended. The DIVERGENCES were real. **`%m`** NAMES THE INSTANCE (LRM 9.4.4): one module instantiated four ways printed the same string every time because `%m` expanded at compile time to the MODULE name, defeating the clause's entire purpose. E-417 had declined to fix it for a real reason -- a handle-to-name callback would have a model built by a new openvaf-r call a NULL slot on an older ngspice, the E-396 hazard -- but that argues against a NAIVE fix, not against the fix: the null check now lives INSIDE the model (`osdi_inst_name` in stdlib.c), so an old simulator gets exactly today's module-name behaviour and the crash cannot occur (verified both directions against the shipped binaries). Measured: `na`, `nb`, `n.x1.nsub`, `n.x2.nsub`, hierarchical where the hierarchy is real, against `fmt` four times before. MULTICHANNEL DESCRIPTORS ARE ONE-HOT BITS (LRM 9.5.1): both descriptor kinds came from one namespace of consecutive small integers, so `$fopen("a")`=1 and `$fopen(`

"b")=2 made an OR-ed $fdisplay compute 3 -- the descriptor of an unrelated THIRD file, which silently received the text meant for both -- and the first user file was handed descriptor 1, the value the LRM reserves for stdout. E-516 recorded this as "documented-missing"; a write landing in the wrong file is worse than missing, which is why it is implemented rather than re-deferred. **$fscanf** LEAVES WHAT IT DID NOT CONSUME: lowered as "read a line, then scan it like $sscanf", it took the WHOLE line however little the format matched, so the scan — the — numbers—take—the—trailing—label idiom lost everything after the last conversion and a following `f the NEXT line. A distinct FgetsScan callback now records where the line began and each field scanner leaves the descriptor at the first unconsumed byte -- distinct precisely because $fgets$ `proper IS specified to consume the whole line, so a plain `fgets+$sscanf$ `must l *`fclose+$fopen$ `REWINDS** : the same—named edup returned the live handle with its read position then—open readline 2; the reopen now rewinds and cancels the deferred close, but only when the stream mode stream for reading is a mode change needing a real freopen (the first version missed that and bro then—reopen—to—read round trip in `string io_e xamples`).**ANATURE′SABSTOLREACHEST * : openvaf—rhas always written each declared abstol into the .osdi nature tables and NOTHINGIN wide abstol/vntol what ever its nature declared. No ABI change was needed because the data was a lread the loader now reads $OSDI_N ATURES$/$OSDI_D ISCIPLINES$/$OSDI_A TTRIBUTES$, resolves e 06 on amy voltage potential and 1e−12 on amy current flow, and it works for models compiled BEFOR *ANEXPLICIT`.option`STILLWINS, ANDTHATISNOTADETAIL * * :` disciplines.vams` declares abstol on the STANDARD natures too (`Voltage` = 1u, `Current` = 1p), so this path runs for every OSDI node in every deck, not just exotic ones. Those v —but they stop being harmless the moment a user LOOSEN Sa tolerance, and a first version of this cha 1e−4`set to get a stubborn circuit to converge, from a declaration the user never wrote. The nature′s val ngspice already records which tolerance options the user set (`TSKtolGiven`, E − 110), but the analysis task inherited every tolerance VALUE from the default task and then CLEARE given vntol reached the solver with the right value and no memory of who chose it; the flag snow travel w 110′s preset rule is unaffected——a preset and the explicit options land on the same task, so an explicit. *COMPATIBILITY** : the descriptor encoding changes, visible only to a model that PRINTSade returns 0 on failure and every success is non-zero, and nofd > 0 idiom exists in the tree. Both ngspice-side additions are optional-symbol lookups, so an .osdi from an older compiler is handled exactly as before. **TWO EXISTING CHECKS ASSERTED THE BEHAVIOUR BEING CORRECTED** and were updated with the reason rather than worked around: display's "%m prints the module path" becomes the instance path, and lrmfuncs' FTELL 19 becomes 18, since $fscanf leaves the trailing newline exactly as C's own fscanf does. Verified lrmio‘ 17/17 (new suite, both solvers), scoring 6/17 against the previously shipped binaries so every check discriminates; full sweep 450/450 ALL OK. openvaf-r changed (first compiler change since E-534).

E-542 *ngspice: src/spicelib/devices/dev.c, src/spicelib/parser/{inp2n,inpdomod,inpgmod,inpmkmod}.c, src/frontend/inpcom.c, src/include/ngspice/{inpdefs,osdiitf}.h, examples/multimod_examples/ (16 -> 24 checks); OpenVAF: openvaf/hir_def/src/nameres/{collect,diagnostics}.rs, openvaf/test_data/ui/reserved_module_name.{va,log} (+6 regenerated logs)*

A VERILOG-A MODULE NAMED LIKE AN NGSPICE BUILT-IN IS USABLE, NEVER SILENTLY REPLACED, AND CASE-TWINS ARE CAUGHT. The 2026-09-04 hunt's two registry findings. A module whose name is one of ngspice's own -- a built-in device's name (`diode`) or a `.model` type keyword (`res`, `d`, `sw`, `nmos` ...) -- could not be used from a netlist: on an `n` line the deck aborted with "incorrect model type" and no cause; on the built-in's own device letter ngspice's built-in simulated in the module's place, taking the card's parameters where the names coincided (the junction diode took `is`), and finished with a plausible, wrong answer. Worse, `pre_osdi -f` on such a module replaced the built-in for the session: after a forced reload of a module named `diode`, every plain `.model ... d` card ran the Verilog-A model. The keyword family (`c cpl csw d l ltra nhfet njf nmf nmos npn nsoi phfet pjf pmf pmos pnp psoi r res sw txl urc vdmos ...`) was not even refused -- registered and then never

selectable, because INPdomodel matches those keywords before it consults the device table. THE FIX DECIDES THE AMBIGUITY WHERE IT IS NOT AMBIGUOUS: an n-line instance can mean nothing but an OSDI device, and ngspice materialises a model lazily, at the first instance line that uses it. So the loader registers a colliding module under its own slot as SHADOWED (a new `INPbuiltinModelTypeKeyword` reads the same keyword table as INPdomodel), saying what the name costs; INPdomodel keeps the card's type token; the N-line parser re-binds a card whose type resolved to a built-in to the shadowed module BEFORE INPgetMod materialises it (`Note(osdi): ... this n-line instance can only mean the Verilog-A module "res" ... binding the card to it`) -- `.model mm res r=2000` on an n line now reads -5.00000e-04; a card materialised as a built-in says so at that moment (`create_model`), the one place it is certain a built-in device letter is using it; one card cannot serve both letters (n-then-d fails on the d line, d-then-n fails on the n line with the explanation); the deck-reading pre-pass no longer pre-judges n lines (it runs before any `pre_osdi` has loaded a library, so it cannot know, and INP2N reports every genuine mismatch with the cause); and `-f` swaps the shadowed module's own slot, never the built-in's. F2: two modules in one .osdi whose names differ only in case (`Foo`/`foo`, legal in Verilog-A) collapsed into one with no diagnostic -- the second unreachable, every card asking for it getting the first (1 mA where the module implements 7 mA). The hunt blamed a case-sensitive comparison; the cause was that the duplicate scan stopped at DEVNUM, advanced only after the whole file, so a module never saw the ones registered earlier in the same load. The scan now covers them and reports both spellings. THE COMPILER extends L018 (`reserved_module_name`) to the keyword family with its own wording, both matching the simulator (`.model <name> res` resolves to the built-in; ngspice re-binds only for an n-line instance), with the `_va` convention in the help. Verified `multimod` 24/24 (was 16; the `vcvs` E-29 fixture and `res` now RUN from an n line, the d-line card warns at materialisation, `-f` keeps the built-in, both mixed orderings fail loudly), cargo 209 fast / 235 slow, full sweep 453/453 ALL OK.

E-541 *ngspice: src/osdi/{osdicallbacks,osdiload,osdiparam}.c, src/osdi/{osdi.h,osdidefs.h}, src/include/ngspice/cktdefs.h, src/spicelib/analysis/cktop.c, examples/lrmvoice_examples/ (new, 28 checks), examples/simparamdiag_examples/ (assertion updated); OpenVAF: openvaf/hir_lower/src/{ctx,lib,fmt,callbacks,expr,state}.rs, openvaf/hir_ty/src/validation{,/body}.rs, openvaf/hir_def/src/builtin.rs, openvaf/hir/src/elaborate.rs, openvaf/osdi/src/compilation_unit.rs, openvaf/osdi/src/metadata/osdi_0_4.rs, openvaf/osdi/header/osdi_0_4.h, openvaf/osdi/stdlib.c, openvaf/openvaf/tests/load/osdi_0_4.rs, examples/lrmvoice_examples/ (new, 28 checks)*

THE THIRD LRM AUDIT WORKED: NINE FINDINGS, ALL ABOUT A MODEL'S VOICE. The 2026-09-02 round-3 audit went at what the first two swept past -- the TIMING and ADDRESSING of everything a model says to the outside world -- and five of the nine reduce to one sentence: a guard exists, is correct, and does not cover every route into the thing it guards. (1) AN **`analog initial`** BLOCK SAID NOTHING AT ALL. LRM 5.2.1 forbids access functions, analog operators, contributions and event controls inside it, so initialising variables and REPORTING are the only two things it can do -- and the reporting half produced zero output in every analysis, with the file it opened created and left at 0 bytes. `$debug` and `$info` printed once per analysis throughout, which is what proves the block RAN and only its output was thrown away. Cause: `hir_lower/src/lib.rs` gates the block on `IsInitialStep` inside eval (E-456, and right for what it was for), which puts its statements on the instance's FIRST Newton iteration -- exactly the one the 9.4.6/9.5.9 deferral treats as superseded. The compiler already had the concept: `stmt.rs` sets `in_event_ctx` for event-controlled statements and the comment above it states the hazard verbatim for the neighbouring construct. A new `in_analog_initial` joins it. THIRD APPEARANCE OF ONE HAZARD -- E-516 handled `@(initial_step)`, E-535 handled re-entered setup, this route was still open. (2) THE SEVERITY TASKS RAN ON EVERY NEWTON ITERATION. LRM 9.7.3 states the rule in its OWN words rather than by reference to 9.4.6 -- "non-fatal system severity tasks called during a rejected iteration shall have no effect" -- which is very likely why it was missed. One diode `.op`, `$strobe` and `$warning` on adjacent lines: 1 line versus 21, walking the unconverged sequence 0 -> 1.000000 -> 0.974135 -> ... -> 0.629443. That is verbatim the August

audit's headline surviving on the one family its remediation did not sweep. Fixed in `osdicallbacks.c` by adding INFO/WARN/ERR to the deferral gate; DEBUG and FATAL deliberately stay out, each for its own clause, and the suite pins `$debug`'s exemption as a check because a fix that deferred everything would also stop the 21 lines. Prior art is PARTIAL and is quoted rather than paraphrased: E-516 carries the parenthesis "(`$fdebug`/severity tasks keep their immediate paths)" with no clause cited, and states the exemption 34 lines earlier as "only `$fdebug` exempt" -- so the behaviour was chosen, by grouping the severity tasks with the one task 9.4.6 exempts and 9.7.3 does not cover. (3) **`$error`** IN AN **`analog initial`** BLOCK LET THE RUN PROCEED. 9.7.3: "the message is issued and the initialization continues. However, the simulation shall not proceed past initialization." It printed and handed the deck `v(1) = 1`. The mechanism was already wired up and correct -- the same model with `$fatal` aborts exactly as it should -- and `$fatal` lowers to a display PLUS `SetRetFlag(RetFlag::Abort)` while `$error` emitted the display and nothing else. New `RetFlag::InitErr` / `EVAL_RET_FLAG_INITERR` carries the weaker requirement, and the distinction is real: `$fatal` abandons the evaluation, `$error` must let initialization FINISH and only then stop the analysis. `CKTop` reports it under its own flag so the message names the task that actually ran. (4) NONE OF THE FOUR REPORTED WHEN IT WAS CALLED. Same clause: "shall also report the simulation run time", or the swept value during a `.dc`, or "that the call was made during initialization". Messages carried the instance name and nothing else. Now a trailing parenthetical, so the `OSDI(warn) <inst>:` head every reader matches on is unchanged -- and RESOLVED AT OUTPUT TIME, not at format time, because during a `.dc` `CKTtime` only takes the point's swept value AFTER the solve that produced the message; the first implementation reported the previous point's value and `(at sweep value 0.5)` on a message raised at 0.75 is how it was caught. (5) A NEGATIVE **`$mfactor`** FROM VERILOG-A SIGN-INVERTED THE DEVICE. LRM 9.18 Table 9-29 says `$mfactor > 0`; `#(.$mfactor(-3))` made a plain resistor model SOURCE 3 mA with no diagnostic anywhere, while the identical `m=-3` on the netlist line has been refused all along by a message that spells out the consequence. A second partial guard existed too -- with a `white_noise` contribution the constant-folded `sqrt(-3)` trips the domain check -- so a model without noise got no luck at all. Literal instance overrides are now judged against Table 9-29 where they are collected; only a literal, which is the same boundary the constant-only `sqrt` check draws. `$hflip`/`$vflip` get the same on both routes. `m=0` on a netlist line is UNTOUCHED (E-426's disable-this-instance idiom, a SPICE convention this does not relitigate). (6) **`$angle`** WAS NEVER REDUCED MODULO 360. Table 9-29's resolution rule is a sum "modulo 360 degrees" into 0 <= angle < 360; two levels of 200 degrees read back as 400. Harmless to a model that uses the angle trigonometrically, wrong for one that COMPARES it. Normalised wherever a value is materialised, so every route lands in range, with `x - 360*floor(x/360)` rather than a `%` -- the value is a real and this form is correct for negative angles too. (7) THE 31ST MULTICHANNEL DESCRIPTOR WAS THE RESERVED BIT. 9.5.1 reserves bit 31 and says so precisely so an implementation can tell the two descriptor kinds apart -- and it is the bit E-539 took for file descriptors, with `0x8000_0000` being STDIN, so the 31st `$fopen(name)` aliased a read-only stream and every write to it was discarded. The allocator now stops at bit 30 and the 31st open returns the clause's own failure value, 0. (8) A REOPEN IN A DIFFERENT MODE WAS IGNORED. Table 9-24 gives "w" as "truncate to zero length or create for writing"; read a file, `$fclose`, reopen "w" and the same-name dedup handed back the READ-mode stream, so nothing was truncated and every write vanished without an error. E-539's own comment names the general rule and implemented it for write -> read; read -> write was the other half, and it failed silently where the case E-539 fixed at least produced visibly wrong data. The descriptor's queued deferred writes are performed BEFORE the reopen -- the model has closed the file, so a reopen for reading must see them. ONE VERSION OF THAT FIX WAS WRONG AND THE SUITE ESTATE CAUGHT IT: it also applied 9.5.1.1's append-on-rewrite rule there, which looks like consistency and is not -- that clause is about content written from following ANALYSES, while ngspice runs instance initialization twice per analysis (setup and temperature), so a write-read-write model appended its second run's output onto its first's and broke E-516's byte-for-byte re-run guarantee. No check output moved; `stringio_examples`' committed round-trip artefacts did, one line becoming two, which is the whole

reason those files are in the repository. (9) **$write** COULD NOT COMPOSE A LINE. 9.4.1 gives it "the same capabilities as $strobe, but with no newline", and assembling one line from several calls is the only thing that distinguishes it -- which the per-call OSDI <inst>: head defeated. The head is now written only at the start of a line, tracked per stream; every other task ends its text with a newline and is unchanged, which the suite pins with a $strobe immediately after three $writes. PLUS LRM 9.20's OTHER TWO ERROR RULES (a port as the aliased net, and a target that is another call's aliased net), the second judged only where it does not need hierarchy resolution because a false error on a legal alias would be worse than the missing one. COMPATIBILITY: two ADDITIVE ABI bits on E-55's precedent (EVAL_RET_FLAG_INITERR, LOG_FLAG_INIT), so an older simulator ignores an unknown return bit and an older .osdi never sets the log flag; nothing crashes and nothing changes meaning. Two behaviours change for existing decks, both deliberately: a per-iteration $warning becomes one per accepted point (the fix), and the three non-fatal severity messages gain a trailing (at ...) -- with the head unchanged, so matches on OSDI(warn) <inst>: <text> still hold. One existing check was updated with its reason: simparam–diag's "newline-terminated" assertion tested the property by requiring the line to end at a closing quote, and now admits that one parenthetical. Verified lrmvoice 28/28 (new suite, both solvers), scoring 11/28 against the audited binaries where the eleven passes are the compile checks plus four deliberate controls; full sweep ALL OK.

E-543 *src/frontend/{com_sweep,spiceif}.c, src/frontend/numparam/xpressn.c, src/maths/misc/randnumb.c, src/maths/KLU/klusmp.c, src/osdi/{osdiload,osdiregistry,osdisetup,osdicallbacks}.c, src/osdi/osdidefs.h, src/spicelib/analysis/cktacct.c, src/include/ngspice/{osdiitf,randnumb,klu}.h, examples/osdilimit_examples/ (new, 12 checks), examples/{montecarlo,highsigma,wcd,mcpolicy,yield,display,klu_tuning}_examples/ (+34 checks), examples/benchmark_examples/large_bench.py (new)*

THE MONTE CARLO COMMANDS ANSWER HONESTLY, AND A COMPILED MOSFET CONVERGES LIKE A BUILT-IN ONE. Two 2026-09-04 hunts folded, nine findings. The Monte Carlo commands: a quoted or unresolvable –spec/–metric was scored as 0 and reported as a confident 0% or 100% yield (the evaluator's validity flag was discarded) -- now unquoted like –analysis and refused with the spec, sample and plot named; every banner states its seed and an un-seeded run says a rerun repeats the netlist draws (montecarlo_seed/highsigma_seed/wcd_seed for scripts); wcd searches over model-declared statistics through a new osdimc WALK mode (OSDImcWalk: every Gaussian statistical parameter takes nominal + sigma*z[k]; uniforms held) -- the osdimc-only deck reports beta = 4.0000 where it was refused, and a 1-ohm netlist dimension beside those statistics 3.9601 where it read 106.7; mvnorm(3) against a 2x2 matrix is a .param error and mccorr reports an index the deck already used beyond it; –max below –min is refused. Large circuits (to 40 000 BSIM4 MOSFETs, 67 000 OSDI instances, both solvers agreeing to 1e-15): an OSDI MOSFET or BJT whose model calls no $limit now gets the built-ins' Newton step limiting -- recognized by terminal names (d,g,s[,b] / c,b,e[,s]) with polarity from type and threshold from vth0/vto, the cold-start guess and DEVfetlim/DEVlimvds/DEVpnjlim applied in the type-normalized frame across the model's live internal DI/SI/GP/BI nodes; models with a thermal terminal, another live internal node (MEXTRAM) or their own $limit are left alone. A 100-stage BSIM4 or PSP103 chain converges in 8 iterations (was 333/387 with gmin stepping; built-in 9), a 40x40 grid in 8 and 0.56 s (was 167, 5.2 s), every operating point identical to 1e-16; .option noosdilim / set osdilim_verbose. KLU's refactor reused a pivot order the matrix had outgrown without any test (137 iterations with gmin stepping where Sparse took 8): a collapse of its rcond estimate by more than 1e-6 relative to the last full factorization now reorders, as Sparse's small-pivot test does. rusage no longer reports a negative KLU fill-in (the diagonal was counted twice, the off-block entries never) nor a zero Sparse-mode total in a KLU build; a model line repeated by thousands of instances is shown five times and then counted.

E-544 *src/frontend/{com_aging.c,com_aging.h,control.c,device.c,spiceif.c,com_sweep.c,com_optimize.c,runcoms2.c}, src/include/ngspice/fteext.h, examples/mcpolicy_examples/ (34 -> 41 checks)*

THE USER'S **alter**/**altermod** WRITES SURVIVE THE LOOP COMMANDS' INTERNAL RESETS. Finding F3 of the 2026-09-04 ngspice + OSDI hunt. The sampling commands redraw a deck's random `.param` values with an internal reset -- a full re-source -- on `highsigma` and `wcd` at every evaluation and on `montecarlo` whenever E-346's fast path cannot arm (every deck whose only variability is model-declared), and the re-source discarded every `alter`/`altermod` the user had typed, statistical or not: `wcd` reported beta = 4.00 for a nominal the user had just moved 4 sigma, `highsigma` P(fail) = 2e-5 where the recentred nominal made it 0.35, and the `op` after `montecarlo` read the un-altered circuit. The user's writes are now JOURNALED at the command dispatcher (only commands the user typed; the optimizer's, sweep's, temper's and aging's own writes stay out), one entry per target with the value as evaluated, and REPLAYED after every internal reset beside E-501's aging replay, under `ft_optimizing`, with a circuit-title check; a user `reset` forgets them; `optimize` journals its final optimum. Seven checks in `mcpolicy_examples` (34 -> 41), twenty related suites green on both solvers. The hunt also records F14 -- `set temp` dropped by every reset -- as pre-existing and unfixed.

E-547 *src/frontend/plotting/pyplot.c, examples/pyplot_examples/ (17 -> 24 checks), docs/internals/ngspice_internals/{ngspice_pyplot,ngspice_commands}.md*

pyplot QUOTES ITS LAUNCH AND JUDGES ITS EXIT STATUS. Defects 1 and 2 of a review of the `pyplot` command (six defects, five improvements; E-548 to E-551 work through the rest). The command line that ran the generated matplotlib script was built unquoted through `system()`, so with the deck-folder output of E-183 a folder named `My Circuits` handed Python the file `My` and an apostrophe left the shell waiting for a closing quote -- the .py and .data were written, no image was, and ngspice went on with exit status 0. And only a -1 from `system()` was ever looked at, so a missing interpreter or a missing matplotlib printed Python's own complaint and nothing else: a batch deck finished green with no figure. One launch path (`pyplot_run`) now serves the five renderers: the script path is shell-quoted on both platforms (`pyplot_python` is still spliced verbatim so it can carry options; a value naming an executable path with a space is one word), a hardcopy's non-zero exit is named together with the image that was not written, every pyplot publishes **`pyplot_status`** as `shell` publishes `shellstatus` (so `if $pyplot_status ne 0 .. quit 1`), and the script's own 'wrote' line quotes the file name as a Python literal. `pyplot_examples` 17 -> 24, both solvers; the seven other pyplot suites, eye and rfplots pass.

E-548 *src/frontend/plotting/pyplot.c, examples/pyplot_examples/ (24 -> 29 checks), docs/internals/ngspice_internals/ngspice_pyplot.md*

pyplot WRITES EXACT DATA, HONOURS ylimit UNDER ylog, GIVES MIXED UNITS TWO SCALES, AND ITS SCRIPT RUNS FROM ANYWHERE. Defects 3 to 6 of the pyplot review. The data table was written with `%e`, six significant digits, in the file the reference calls the export: a time axis offset to 1 s with 1 ns steps collapsed to ONE distinct x value in 59 rows, a microvolt ripple on 1 V to eight levels -- every table now carries 17 significant digits. `ylimit lo hi` under `ylog` set the log scale and silently no limits; it is applied now (a non-positive bound never reaches the backend: plotit refuses it first). A voltage and a current on one plot shared one axis with no label at all, the milliamps flat along the bottom of the volt scale; within a panel the first trace's type now owns the left axis and any other type is drawn on a `twinx()` axis on the right, each labelled with its type, the legends combined, every trace given an explicit colour (a twin axis restarts the colour cycle). And the generated script looked for its data relative to the directory ngspice ran in, so the reference's own advice -- edit it and run it again -- failed from anywhere else; it now resolves its data table and its image against its own location. `pyplot_examples` 24 -> 29, both solvers; the twelve suites that exercise pyplot pass; full sweep 455 of 455.

E-549 *src/frontend/{com_pyplot.c,commands.c}, src/frontend/plotting/{plotit.c,pyplot.c,pyplot.h}, examples/pyplot_examples/ (29 -> 35 checks), examples/{pyplotcontour,pyplotsmith,pyplotmore, pyplothist}_examples/ (read either format), .gitignore, docs/internals/ngspice_internals/{ngspice_pyplot,ngspice_commands}.md*

**pyplot WRITES ITS DATA TABLE AS .npy, ‘set pyplot_export=bin

E-550 *src/frontend/plotting/pyplot.c, src/frontend/commands.c, examples/pyplot_examples/ (35 -> 40 checks), docs/internals/ngspice_internals/ngspice_pyplot.md*

pyplot DRAWS A LONG TRACE AS ITS PER-PIXEL MIN/MAX ENVELOPE, RE-DECIMATED ON ZOOM. Improvement 2 of the pyplot review. A pixel column can only show its extremes, so a trace with more samples than twice the axis width in pixels is drawn as its min/max envelope per column: the same picture, drawn in milliseconds -- matplotlib took 4.8 s of a 5.2 s run to draw four million points onto a 640-pixel canvas; the million-point, four-trace hardcopy now takes 2.34 s end to end against 6.81 s without. The envelope bins by x (adaptive time steps are handled), keeps each column's actual extreme samples in time order, skips NaN padding, and leaves point plots, step plots and a `vs` plot whose x runs backwards alone. An interactive window keeps the full data beside each line and re-decimates on every zoom, pan and resize, so zooming into a slice redraws it in full detail; a hardcopy decimates once at its dpi; the script says what it did. ‘set pyplot_decimate=auto

E-551 *src/frontend/plotting/pyplot.c, src/frontend/commands.c, examples/pyplot_examples/ (40 -> 47 checks), docs/internals/ngspice_internals/ngspice_pyplot.md*

pyplot TICKS IN ENGINEERING UNITS, AXES LABELLED BY TYPE, THE DECK'S * OFF THE TITLE. Improvement 3 of the pyplot review. The default figure showed `0.0005, 0.0010` on an axis labelled `s`, `V` on the other, and `* rc lowpass` for a title. Each axis is now formatted by matplotlib's EngFormatter in the unit of the vector type it carries -- `500 us`, `1 ms`, `−500 mV`, `10 kHz`, `1 kOhm` -- with prefixes only on SI units (s, Hz, V, A, W, F, C, Ohm, S); dB, rad, Celsius and the noise densities stay plain; the formatter is installed after the axis scale, since `set_xscale('log')` replaces whatever was there. The default labels read `time [s]`, `voltage [V]`, `frequency [Hz]`, `decibel [dB]`, a mixed plot `voltage [V]` left and `current [A]` right; a label or title the user gives is kept verbatim; the default title drops the deck's leading `* .` `set pyplot_eng=off` keeps plain numbers. The Bode frequency axis, the eye's time axis, a contour's axes and colour bar and a histogram's value axis get the same ticks. `pyplot_examples` 40 -> 47, both solvers, reading the rendered tick texts; the twelve suites that exercise pyplot pass; full sweep 455 of 455.

E-552 *src/frontend/{com_sweep.c,commands.c,typesdef.c}, examples/mcrecord_examples/ (new, 13 checks), docs/internals/ngspice_internals/{ngspice_statistics,ngspice_commands}.md, docs/handbook/03-ngspice-workflows.md*

`montecarlo -expr` RECORDS A VALUE PER SAMPLE INTO A **`montecarlo<n>`** PLOT, AND A YIELD IS JUDGED ONLY WHERE A SPEC HAS A LIMIT. The packaged Monte Carlo command answered one question, the yield: a `−spec` was mandatory, a limit on it was mandatory, and nothing per sample survived the run (the analyses run under the loop commands' plot recycling), so 'run the sweep N times and keep a value from each' had to be hand-written around `reset`. `−expr [name=]<expression>` is evaluated after every sample and recorded, unjudged, into a plot of its own -- `montecarlo1`, `montecarlo2`, ... one per invocation, named in `$montecarlo_plot` -- with `sample` (1..N) as its scale. A scalar per sample is an N-long vector; a waveform per sample (a dc/ac/tran output of L points) is an N x L two-dimensional vector with the analysis scale copied beside it, which `plot` draws as a family and `vo[k]` indexes as sample k. A complex value is its magnitude, a failed sample is nan, a waveform whose point count differs between samples is refused with the reason (the scalars beside it still recorded), an expression that never varies is noted, and one that resolves to nothing is refused on sample 1. A `−spec` without `−max`/`−min` is refused with a pointer to `−expr`; with no `−spec` there is no yield; nothing to judge and nothing to record is refused naming both; `−spec` and `−expr` combine; a limited `−spec` alone is unchanged and creates no plot. `montecarlo` is registered as a plot type (the plots came out as `unknown1` without it). `mcrecord_examples` (new, 13 checks, both solvers); the 22 suites that use montecarlo pass unchanged; full sweep 456 of 456.

E-553 *src/frontend/{control.c,inpcom.c,quote.c}, src/frontend/parser/{lexical.c,glob.c}, src/include/ngspice/cpextern.h, examples/rawfstring_examples/ (new, 14 checks), docs/handbook/{03-ngspice-workflows,04-limitations-and-gotchas}.md, docs/internals/ngspice_internals/{ngspice_commands, ngspice_pyplot}.md*

RAW STRINGS **r"..."** KEEP THEIR CASE THROUGH THE DECK READER; F-STRINGS **f"...{expr:.3f}..."** EVALUATE IN A CONTROL SCRIPT. A deck is folded to lower case as it is read, and the fold reached the text a control script hands to a command: `set t="ABC"` stored `abc`, `pyplot ... title "RC Low-Pass"` printed `rc low-pass`; the fold was command-specific and uneven (`echo` lines and the title tokens of `plot` and `gnuplot` were exempt, `pyplot`'s were not), and the same reading pass dropped the spaces around an = inside a quoted string. After Python: `r"..."` / `r'...'` is a RAW string, copied through the deck reader as written, case and spaces kept; `f"..."` is an F-STRING, every `{expression}` evaluated with the control-language evaluator when the command runs and replaced by its text, with an optional format after a colon (`.3f`, `.4g`, `e`, `d`; a scalar prints with `%g`, a vector as its elements, a complex value as `re,im`; `\{` and `\}` are literal braces -- `{{ }}` stays with the netlist's `.for`); `rf"..."` is both. The prefix counts only at a token start (a resistor `r1`, a variable `f`, `set r=...` are untouched). An expression that resolves to nothing, an unbalanced brace or a bad format is an error naming the string, and the command is not run. `$variables` are substituted before the braces are evaluated; interactive input, never folded, simply accepts the prefixes; a plain string folds as before. `pyplot` keeps the case of its `title`/`xlabel`/`ylabel` tokens the way `plot` and `gnuplot` always did. `prefixed_span_end` in inpcom.c (the control-line fold and the whitespace pass skip a prefixed literal), `cp_string_prefix_len` in the lexer (a prefixed `'...'` becomes the double-quoted form, backslashes kept), `cp_unquote` strips the prefix with the quotes, `cp_doglob` leaves a prefixed word alone, `cp_fstringsubst` in control.c evaluates after globbing, also in `foreach` lists. `rawfstring_examples` (new, 14 checks, both solvers); full sweep 457 of 457.

E-554 *src/osdi/{osdi.h,osdiregistry.c,osdisetup.c}, src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c, examples/osdidist_examples/ (new, 18 checks), docs/handbook/03-ngspice-workflows.md, README_OSDI.md, docs/internals/ngspice_internals/ngspice_statistics.md*

THE LOGNORMAL AND TRUNCATED DRAWS, THEIR IMPORTANCE WEIGHT, AND A wcd THAT NAMES A CLAMPED WALK. The registry merges the INFOS records and the optional TRUNCS array into in-memory OsdiStatParam records. A truncated draw is deterministic rejection: the first attempt is the plain key's deviate, so declaring a truncation changes only the draws that would have exceeded it (282 of 300 identical, 18 redrawn inside the window); a rejected attempt is redrawn from a sub-key of the same trial, 64 times, then clamped. A lognormal is nominal * exp(s z). Both inflate under `highsigma -scale` with the matching weight: with a truncation both densities are confined to the same value window and their normalisers erf(t/sqrt2) and erf(t/(sc sqrt2)) join the likelihood ratio -- 0.0346 measured against 0.0336 exact for a 2-sigma window, 0.024 without them; the untruncated control 0.0568 against 0.0548. Both take a `wcd` walk coordinate, clamped at the truncation, and `wcd` now says *the walk is held at the trunc truncation of N model-declared parameters ... the spec boundary lies beyond the declared window* instead of *zero gradient*. Measured over 300 draws: lognormal std_rel=0.2 gives ln(v/nom) sd 0.203, every draw positive; std_rel=1.0 on (0:inf) loses no trial where the gauss control loses 47; gauss trunc=1 stays within +-1 sigma with sd 0.54 sigma and no clamp. `osdimc_verbose` names the shape. The 13 suites that share the draw machinery pass; full sweep 458 of 458.

E-555 *src/osdi/{osdi.h,osdiregistry.c,osdisetup.c}, src/include/ngspice/{osdiitf,trcvdefs}.h, src/spicelib/analysis/dctrcurv.c, src/frontend/com_sweep.c, examples/paramgiven_examples/ (new, 16 checks), examples/elabguard_examples/ (adjusted), docs/handbook/03-ngspice-workflows.md, README_OSDI.md, docs/internals/ngspice_internals/ngspice_statistics.md*

A GATED PARAMETER THE DECK NEVER GAVE IS NOT DRAWN, AND EVERY SWEEP RESTORE PUTS THE GIVEN FLAG BACK. The simulator half of F1. `osdimc` draws a parameter the

model tests with `$param_given` only when the deck gave it, and says so once -- *`mos_va:toxp` is not given by the deck and the model tests `$param_given(toxp)`: a draw would switch the model to its "given" branch instead of varying it -- not drawn. Give it on the card, or altermod it, to vary it* -- and such a parameter is no dimension of the `wcd` walk and no factor of the `highsigma` weight. The registry loads the object's given-flag entry point; `OSDIparamGiven` / `OSDIparamGivenByName` let the `.dc` restores (instance and model targets, `dctrcurv.c`), the `sweep` command's restore and `unset osdimc` clear the flag of a parameter the deck never gave -- where before a `dc @mos_va[toxp]` with the option off left BSIM4 at -76.30 uA for good, and the built-in BSIM4 under the same sweep read -108.96 uA before and after. BSIM4 with only `toxe` on the card now reads -112.39 uA on every trial, after the sweep, and in a recorded ensemble; with `toxp` on the card it is drawn as before. F2's runtime effect: `altermod mm lmin=1.5` with `l` defaulted, `dc @mm[lmin] 0.5 2.0 0.5` and the draws of `lmin` past `l` are refused with the model's own range error; `elabguard`'s `freeze(aa=5)` check had set exactly that shape and now sets `aa=3`, with a new check pinning the refusal. The statistics, sweep, range and CMC runtime suites pass; full sweep 459 of 459.

E-556 *src/frontend/parser/{lexical,glob}.c, src/frontend/{control,cpitf,com_pyplot}.c, src/include/ngspice/cpextern.h, examples/rawfstring_examples/ (14 -> 18 checks), examples/pyplot_examples/ (47 -> 49 checks), docs/handbook/03-ngspice-workflows.md, docs/internals/ngspice_internals/ngspice_pyplot.md*

AN F-STRING AFTER **`name=`** IS EVALUATED, ITS RESULT IS PLAIN TEXT, AND **`pyplot`** UNQUOTES ITS NAME. F3, F4 and F5 of the 2026-09-05 hunt, on E-553's string forms. The lexer accepted a string prefix at a word's start or after `=`, `(`, `,`; the glob skip and the f-string pass looked at the word's start only, and the deck reader collapses `let z = f"{7}"` into `let z=f"{7}"` and `if f"{1+1}" = 2` into `if f"{1+1}"=2` -- so in a deck an f-string after `name=` had its braces globbed away and was never evaluated, and no `set` variable could take one at all. `cp_string_prefix_at` finds a prefixed string wherever the lexer does, with a tail after the closing quote kept and more than one per word; `cp_doglob` skips a word carrying one, `cp_fstringsubst` rebuilds head + text + tail, and `cp_istrue` -- the `if`/`while` condition path, which never saw the pass -- runs it. The substituted text was a quoted word that `let`, `alter`, `setplot`, file names and numeric options refused; it is plain text now, quoted only when it has whitespace so a command that re-reads its arguments (`set u=rf"{x:.3f} V"`) still sees one word (`foreach x f"{3}"` yields `x=3`, not `x="3"`). And `pyplot` used a quoted name verbatim, quotes and all, so `pyplot -export "my dir/sig"` -- the one spelling that can carry a space -- wrote `"my dir/sig".npy` nowhere; the name is unquoted, the eye path included. `rawfstring_examples` 14 -> 18, `pyplot_examples` 47 -> 49; full sweep 459 of 459.

E-557 *src/frontend/{com_pyplot,com_sweep}.c, src/frontend/plotting/pyplot.c, examples/pyplot_examples/ (49 -> 53 checks), examples/mcrecord_examples/ (13 -> 14 checks), examples/paramgiven_examples/ (the pre-E-555 compiler from git), docs/internals/ngspice_internals/{ngspice_pyplot,ngspice_statistics}.md*

`pyplot_status` ON EVERY **`pyplot`**, A COLLIDING **`-expr`** NAME REFUSED, A BIN COUNT BELOW 2 SAID. F6, F7 and F8 of the 2026-09-05 hunt. E-547 promised that every `pyplot` publishes `pyplot_status`, and five paths did not -- a refusal before the backend ran (a usage error, plotit's own *X values must be > 0 for log scale*), a table or script that could not be opened (a read-only file, a missing directory), and a successful `-export` -- so the deck's `if $pyplot_status ne 0` died with *no such variable* on exactly the failures it was written to catch; `com_pyplot` assumes a failure at entry (status 1) and every path that succeeds says so, the launch path with Python's exit status, the export path with 0. `montecarlo` refused two `-expr` of one name and let one take the record plot's own -- `sample`, its scale, or a result such as `montecarlo_n` -- which then shadowed the record; those names are refused with the reason. ‘set pyplot_decimate=1

E-558 *src/osdi/{osdiregistry.c,osdisetup.c}, src/include/ngspice/osdiitf.h, src/frontend/{inp.c,outitf.c, postcoms.c,com_gnuplot.c}, examples/paramgiven_examples/ (16 -> 18 checks), examples/*

rawfstring_examples/ (18 -> 19 checks), docs/handbook/03-ngspice-workflows.md, README_OSDI.md

THE RANGE ERROR SAYS THE DECLARED RANGE AND WHAT MOVED IT, QUOTED FILE NAMES ARE UNQUOTED, **`.save`** TAKES A MODEL CARD'S PARAMETER. F9, F10 and F11 of the 2026-09-05 hunt. The registry slices the compiler's new range texts per descriptor and the out-of-bounds message appends the text and the current value of every parameter it names, read where the failing parameter lives: `Parameter l of 'mm' is out of bounds (value 1.2; range from [lmin:inf), lmin = 1.5)!`, `(value 2; range from (0:w], w = 1)` for a per-instance range, `(value −5; range from (0:inf))` for a static one. `wrdata "o3.txt"`, `write "w2.raw"` and `source "sub1.cir"` used the quoted name verbatim, quotes and all -- the spelling for a path with a space, and what an f-string with whitespace yields -- and `source` reported the errno the last call had left (*Inappropriate ioctl for device* for a file that does not exist); each unquotes its name, and `source` probes the file once more and says *No such file or directory*. `.save @mm[s]` of a model card's parameter was refused as *no such device* while `print @mm[s]` read it; a name that resolves to a model card is accepted at save time, checked against the card's parameters (an unknown one warns), and read per point through the resolver `print` uses. `paramgiven_examples` 16 -> 18, `rawfstring_examples` 18 -> 19; full sweep 459 of 459.

E-559 *src/osdi/osdisetup.c, src/include/ngspice/osdiitf.h, src/frontend/{com_sweep,control}.c, src/frontend/parser/lexical.c, examples/mcpolicy_examples/ (41 -> 46 checks), examples/rawfstring_examples/ (19 -> 23 checks), docs/handbook/03-ngspice-workflows.md, README_OSDI.md*

A LOOP COMMAND'S **`−seed`** PINS THE MODEL-DECLARED DRAWS, AND THE F-STRING EDGES. F13 and F12 of the 2026-09-05 hunt. E-537 mixed a loop command's `−seed` into the osdimc draw key but left the session-wide trial counter in it, so `montecarlo 3 −seed 1` run twice reproduced the netlist `agauss` values and not the model-declared draws (1020.17 / 815.62 / 719.15, then 1213.59 / 1198.65 / 1133.03), `highsigma 1000 −scale 2 −seed 3` gave 0.0594 and 0.0481 for one seed, and the published `montecarlo_seed` could not regenerate an ensemble -- what the draws depended on was how many run-class commands had happened before. Inside `montecarlo`, `highsigma` and `wcd` the key now carries `(−seed, sample number counted from the command's start)`; the seed defaults to 1 exactly as the netlist half's always has, so an unseeded run repeats itself whole and equals `−seed 1`, and the NOTE says so. The trial counter still advances, so the plain run after the command is a fresh trial as before; `osdimc_verbose` shows `[sample 2 of −seed 1]`. A never-run deck on `montecarlo`'s fast path (no per-sample reset) spent sample 0 on the nominal baseline because the first run's new circuit pointer restarted the count; the baseline skip blesses that pointer. F12: `(long)` of a value beyond a long's range is undefined and `{1e20:d}` printed 9223372036854775807; `:d`/`:i` print such a whole number exactly, `:x`/`:X`/`:u` refuse a negative or too-large value with the value named, `{{1+1}}` is told braces do not nest (the message named a vector `{1)`, `{ }` is an empty `{}`, a colon tail with a format's shape but no conversion letter (`{x:.3}`, `{x:.3q}`) gets a hint, a ternary's colon does not, and an unterminated `r'abc` passes through as typed instead of re-quoted as `r"abc`. `mcpolicy_examples` 41 -> 46 (one one-sided limit widened to a two-sided band), `rawfstring_examples` 19 -> 23; full sweep 459 of 459.

E-560 *src/frontend/plotting/pyplot.c, src/frontend/{spiceif,device,inp}.c, src/spicelib/analysis/dctrcurv.c, src/spicelib/devices/dev.c, src/osdi/osdisetup.c, src/include/ngspice/{ftedefs,osdiitf}.h, examples/pyplot_examples/ (53 -> 54 checks), examples/instdep_examples/ (20 -> 23 checks), examples/osdireload_examples/ (5 -> 6 checks), docs/handbook/03-ngspice-workflows.md, docs/internals/ngspice_internals/{ngspice_pyplot,ngspice_commands}.md*

`pyplot` LABELS AN UNTYPED AXIS BY NAME, A PROMOTED PARAMETER IS NAMED INSTANCE-LEVEL THROUGH ITS MODEL, AND A CIRCUIT BUILT AGAINST A RELOADED OBJECT IS REFUSED UNTIL REBUILT. F14, F15 and F16 of the 2026-09-05 hunt. `pyplot mcv rr` on a `montecarlo` plot (scale `sample`, value `rr`, both untyped) wrote a script with neither `set_`

`xlabel` nor `set_ylabel` -- E-551 labels by vector TYPE and plotit hands a NULL abbreviation for notype; the scale's name and a single signal's name are the labels now, `-hist` of an untyped value labels its x-axis by name, a user's label still wins. `altermod mm l=10` on a parameter E-546 resolves per instance (`l = 2.0*w`, lint L028) was refused as *model 'mm' has no parameter l* after a stray MOS width probe (any name starting with `m` and a `w`/`l`), while the card and `alter n1 l=10` worked; `altermod`, `print @mm[l]` and `dc @mm[l]` now say the parameter is INSTANCE-level and point at `alter @<instance>[l]=...` and at `l=...` on the card, where it is the instances' default, and the bin probe runs for instance writes only. The hunt's *unlabelled run after `pre_osdi -f`* was the first run's nominal baseline, miscounted -- but the reload itself was unguarded: `osdi -f` swaps the device type in place, so a circuit loaded earlier would run the NEW object's code on the OLD layout's data (E-229's documented caveat); the reload now names every such circuit and its next run is refused until `reset` or a re-`source` rebuilds it, while the deck whose control block carries the hoisted `pre_osdi -f` is built against the new object as before. `pyplot_examples` 53 -> 54, `instdep_examples` 20 -> 23, `osdireload_examples` 5 -> 6; full sweep 459 of 459.

E-565 *src/osdi/{osdiinit,osdiregistry}.c, src/include/ngspice/osdiitf.h, src/spicelib/parser/inpgmod.c, examples/paramsetoverload_examples/ (new, 14 checks)*

A **`.model`** CARD NAMING AN OVERLOADED PARAMSET IS BOUND TO THE MEMBER ITS PARAMETERS SELECT (LRM 6.4.2). The simulator's half of E-565. The registry reads the object's family names and literal defaults beside E-558's range texts; when a card is materialised (`create_model`, before the instance line reads the model's type) and its type heads a family, the clause's rules run over the card: every parameter it gives is one of the member's own (a parameter the paramset fixed is not), the given values and the un-given defaults lie within the ranges parsed from the range texts (SI suffixes, `inf`; a bound that is not a literal is unbounded), the fewest un-overridden parameters win; the card is bound to the winner and told so -- *paramset 'nch' resolved to its member 'nch__2' (LRM 6.4.2)* -- a card naming a member (`nch__4`) is taken as written, and none applying, or several, is the card's error with every member's reason. `.model dm nch mm=1` / `l=1u w=10u` / `l=3u w=5u ad=1.2p as=1.3p` / `l=0.5u ad=1p` select the mismatch, default, long-channel and short-channel members of the LRM's example. Thirteen paramset and model-creation suites; full sweep 463 of 463.

E-566 *src/maths/KLU/klusmp.c, src/include/ngspice/{klu,smpdefs,spmatrix}.h, src/maths/sparse/{spbuild,sputils}.c, src/maths/ni/{nireinit,niaciter}.c, src/spicelib/analysis/{cktsetup,cktic,cktpzset,acan,cktdojob}.c, examples/solvercore_examples/ (new, 17 checks per solver)*

A NODE NOTHING CONDUCTS TO NAMES ITSELF INSTEAD OF CORRUPTING THE SOLVE; THE MATRIX SPANS EVERY NODE; KLU'S COMPLEX REFACTOR IS GUARDED. Seven of the eight findings of the 2026-09-06 KLU/Sparse solver-core hunt. F1: under KLU a node with no matrix entry (a current source's only load, a controlled-current-source output, a forgotten monitor load) made SMPconvertCOOtoCSC 'collapse' its column and mis-address every other node's RHS -- a 1 V source read 0 V and sourced 1 A, silently, in op/dc/tran/ac/sens, and two such nodes overflowed the RHS vectors. F8: the same node numbered last was outside the matrix under BOTH solvers (the matrix starts at size 0 and grows to the highest node with an entry); Sparse's RHS vectors were one short, ISRCload wrote past them, and both printed the injected current as the voltage. F2: .ic/.nodeset on a node without a diagonal (stacked supplies, inductor nodes) aborted every analysis under KLU as 'out of memory'. F3: .option rshunt was bound to the real array only, so ac/noise/sp/disto ran without it under KLU. F4: an AC interrupted by a breakpoint left the devices bound complex; the next sweep point's operating point was NaN. F7: every AC point reused the first frequency's pivot order unchecked -- 26 dB off at 1 THz on a wide-range ladder, 613 dB with pivrel=1, while Sparse was exact. F5: an infinite pivot hung the determinant loops. Now CKTsetup gives every node without an entry a zero diagonal (with one warning naming it) and every nodeset/ic node its diagonal, then tells the solver the true unknown count (SMPsizeHint, SMPmarkOccupied, spEnsureNode); KLU's conversion keeps empty columns and the solves use the identity map; NIreinit never sizes the vectors below the numbering; CKTacLoad adds the shunt

through SMPfindElt; CKTdoJob rebinds to real after any analysis returns; SMPcLUfac checks klu_z_rcond after every complex refactor and NIacIter re-pivots silently; isfinite guards the loops. Both solvers now agree on every deck of the hunt, the wide-range ladder is within 0.05 dB of a 70-digit reference, and guard-malloc is clean. F6, an XSPICE batch-exit double free, stays open.

E-567 *src/frontend/inp.c, examples/initfile_examples/ (new, 5 checks per solver)*

AN INITIALISATION FILE NO LONGER KILLS THE RUN AT START-UP. Finding F6 of the 2026-09-06 solver-core hunt, which was not a solver defect. Since Enhancement-558 made com_source unquote every word by freeing it and installing the unquoted copy, inp_source()'s BORROWED word -- the caller's own pointer -- was freed inside the call and again by the caller: every .spiceinit or spice.rc found in the deck's directory, in the current directory or in $HOME ended the run before the first line of output ('pointer being freed was not allocated', exit 134), whatever the file or the deck contained. The hunt met it as an XSPICE deck that 'aborted on exit'; main.o rebuilt with line information put the bad free at read_initialisation_file() at start-up. inp_source() now hands com_source() a heap copy and frees whatever comes back; E-558's unquoting is kept. Pinned by initfile_examples (deck directory, current directory, $HOME, and the F6 deck with its code models loaded from the file) on both solvers; 466 of 466.

E-568 *src/maths/ni/niiter.c, src/maths/KLU/klusmp.c, src/include/ngspice/smpdefs.h, src/spicelib/analysis/cktop.c, src/xspice/icm/analog/pwl/cfunc.mod, examples/oprobust_examples/ (new, 32 checks per solver), examples/linesearch_examples/*

THE OPERATING POINT STOPS FAILING ON FOUR ORDINARY CIRCUITS. A 29-deck OP torture battery under KLU and Sparse found no disagreement between the solvers but four decks that cost hundreds to tens of thousands of iterations, or failed, for reasons outside the circuit. R1: Enhancement-256's false-convergence guard measured a row's residual against

E-569 *src/maths/KLU/klusmp.c, src/include/ngspice/smpdefs.h, src/spicelib/analysis/cktsetup.c, examples/floatnode_examples/ (new, 16 checks per solver)*

A NODE A DEVICE ONLY READS IS HELD AND NAMED. Enhancement-566 gives every node that owns no matrix entry a zero diagonal so gmin can hold it, and names it -- but it judged ownership by the node's COLUMN. That is right for a node nothing conducts to (a current source's only load, a CCCS output: an equation with no unknown) and wrong for a node a device only reads: a B-source v=2*v(x) or an XSPICE input port stamps its derivative into its own row, column x, so x had a column entry and an EMPTY row, no gmin could rescue it, and the operating point failed on both solvers after 374 iterations -- every rung, E-568's damped Newton included -- with Sparse blaming x and KLU the reader's branch; a compiled Verilog-A module that only probes a port (V(out) <+ gain*V(in)) failed the same way. SMPmarkOccupied now reports rows and columns separately and CKTsetup creates the diagonal when either is empty: the node reads 0 V on both solvers, named, reached through optran like the other floating shapes; E-566's empty-column cases are unchanged, three ordinary shapes are pinned not to warn, and OSDI needs nothing of its own (the module converges named; a BSIM4 with an open gate still goes through optran, both solvers agreeing). Spectre's topology check does the same ('Gmin installed to provide path'); .option rshunt gets there too, globally and silently. Pinned by floatnode_examples on both solvers; 468 of 468.

E-570 *src/maths/KLU/klusmp.c, src/include/ngspice/smpdefs.h, examples/singularname_examples/ (new, 12 checks per solver)*

'SINGULAR MATRIX: CHECK NODE' NAMES THE NODE WHOSE EQUATION IS VACUOUS, THE SAME ONE UNDER KLU AND SPARSE. The index a factorization stops at is the column where its own elimination order first ran out of pivots -- for a rank-deficient block, any of the block's columns -- so the same defect carried two names: a BSIM4 whose gate hung on a Verilog-A capacitor was 'check node g' under Sparse and 'check node d' under KLU, and a CMOS inverter chain with its input open got 'in' six times from Sparse and 'in' five times plus 'o2' once from KLU. SMPgetError now scans the loaded matrix (new SMPzeroLine, one pass, only after a failure) for

a row that is all zero -- a node nothing conducts to in this analysis -- and names it whatever the pivot order, then an all-zero column, and only then the pivot; complex values are read for AC. Both solvers now name g, in, x alike; decks with no zero line (parallel sources, an inductor loop) keep their old names. Pinned by singularname_examples on both solvers; 469 of 469.

E-571 *src/spicelib/analysis/acan.c, src/maths/KLU/klusmp.c, src/include/ngspice/smpdefs.h, examples/acgminhold_examples/ (new, 10 checks per solver)*

A NODE THE OPERATING POINT HOLDS ONLY BY GMIN IS HELD IN THE SMALL-SIGNAL MATRIX TOO. A node nothing conducts to (E-566, E-569) or an open MOSFET gate with no capacitance gets its operating point with CKTdiagGmin left at gmin on every diagonal -- optran solves with it, and the node reads I/gmin. The AC load added nothing to any diagonal, so the same node's AC row (or, for a current-source output, its column) was all zero and ac, noise and sp ended in 'matrix is singular' under both solvers, right after a successful operating point. CKTacLoad now scans the loaded matrix once (new SMPzeroLines, the complex-aware pass behind E-570's report) and adds to the diagonal of every node whose AC row or column is all zero the conductance the DC hold used -- gshunt when set, else gmin -- and nothing else: any node with an admittance at all is untouched, so no existing AC result moves. The AC-driven current-source node reads 1/gmin (1/gshunt when set), the CCCS output

E-572 *src/spicelib/parser/inp2n.c, inppas3.c, src/frontend/dotcards.c, inp.c, spiceif.c, src/osdi/osdisetup.c, src/include/ngspice/inpdefs.h, examples/autoopts_examples/ (new, 27 checks per solver)*

A DIG INTO AUTOBUS, AUTOADAPT, SAVEUSED AND AUTOMC. Some fifty probe decks over the four options, both solvers agreeing throughout. Found and fixed: a ONE-BIT bus port has as many terminals as ports, so `N1 a b bus1` in shorthand was taken as spelled out and bound `a` as a plain node, a different node from a[0], in silence (0 V; now 0.5, the explicit line's); a bus base the deck also uses as a plain node (Rx a 0 1k beside N1 a b busdev) sat on a node nothing else touched, unnamed -- pass 3 now names it with the bit spelling to write; saveused read only the control block, so the deck's own .meas / .print / .plot / .four cards found their vectors unsaved ('no such vector', an empty table) -- they are scanned now; a control block still naming a bit of a node autoadapt split failed as 'vector b is not available' -- the split is named, the line quoted, b_f / b_r offered; mcseed=1.5 and mcseed=abc silently ran the seed-1 ensemble -- said now, once; the documented OFF spellings nosaveused / noautobus / noautoadapt / noautomc / noosdimc drew 'unknown option' -- registered. Pinned as documented: a [3:0] bus expands to the compiler's a[0]. .a[3]; under automc a whole sweep is one trial, op then tran two, montecarlo one per sample, the off-words leave everything nominal. autoopts_examples on both solvers; 471 of 471.

E-573 *src/frontend/com_dl.c, inp.c, spiceif.c, src/spicelib/devices/res/restemp.c, src/spicelib/devices/bjt/bjtsetup.c, bjttemp.c, bjtdefs.h, examples/reusecache_examples/ (new, 24 checks per solver)*

A DIG INTO OSDICACHE AND REUSESETUP. A probe battery over the two options, both solvers agreeing throughout. Found and fixed: the `pre_osdi -va` cache watched the source alone, so an edit to a file it `includes` kept loading the object of the previous text -- the staleness test now walks the includes the way the compiler resolves them; an object older than `openvaf-r` was loaded anyway, the very hazard E-500 gave for making the cache opt-in -- it is rebuilt now and the run says so; two sources with the same stem in different directories compiled onto one object, the second overwriting the first while loaded and then reported ”already loaded”, never registered -- a source with a directory names its object after it (a/m.va -> osdi/a_m.osdi); `noosdicache` was an unknown option. Under the setup reuse two quantities computed at setup only went stale on every path that changes a parameter without one: the resistor's flicker-noise area (a noise sweep over `@r1[l]` rose where standalone runs fall) is recomputed with the resistance; the BJT's SPICE2 c2/c4 form (an `ise` above 1e-4 is a multiplier of `is`) was folded into the parameter in place, so a swept `is` kept the first point's leakage with the setup reused or rebuilt alike -- the given value is kept and resolved at temperature time. Pinned alongside: every off spelling, `-f`, a bare source's object name, `.ic` with `uic` at every point, the reuse tally. Full sweep 473 of 473.

E-574 *src/frontend/com_dl.c, examples/vacompile_examples/verify_vacompile.py, examples/reusecache_examples/ (25 checks per solver)*

THE CACHE'S COMPILER CHECK FINDS A BARE NAME ON PATH. The second sweep failed vacompile under both solvers in a shell that names the compiler by path and passed it under a harness that reaches it by bare name: E-573's rule that an object older than the compiler is stale could act only when the compiler's path could be stat'ed, so it was silently inert for a bare `openvaf-r` on PATH and live for `set openvaf=` or $OPENVAF -- and three vacompile checks that backdate their objects to 2023 tripped it wherever it was live. `pre_osdi -va` now locates a bare compiler name on PATH the way system() will, so the rule holds however the compiler was named and the rebuild message prints the path it found; vacompile stamps its up-to-date objects now, newer than the source and the compiler; reusecache pins the PATH case.

E-575 *src/spicelib/analysis/cktsetup.c, cktload.c, acan.c, src/osdi/osdisetup.c, src/include/ngspice/{cktdefs,osdiitf}.h, src/frontend/spiceif.c, examples/dcpath_examples/ (new, 37 checks per solver), floatnode/singularname/solvercore re-pinned*

GMIN INSTALLED WHERE A NODE HAS NO DC PATH TO GROUND. The proposal of 2026-09-07 implemented: Spectre's topology check for ngspice. After the device setups the nodes are joined by every DC-conducting device path -- built-in types from a per-type table of DC-connected terminal groups (a controlled voltage source's output pair, never its controlling pair; a controlled current source, a capacitor, a current source join nothing; the MOSFET gate is isolated), OSDI models from the non-zero resistive entries of their Jacobian pattern taken with their transpose (an entry alone is a dependency, not a path; the constant-resistive flag also covers a constant zero, which a ddt()-only port carries; a flow unknown coupled to one voltage node is a branch to an implicit ground -- a port driven by a single-ended contribution, nearly every node of a large deck named before that rule), XSPICE code models from their port kinds -- and walked from ground. Every voltage node the walk does not reach gets a diagonal and `gmin` on it after the device loads in DC, transient and AC alike, and is named: ”no DC path from node 'g' to ground; gmin (1e-12 S) installed to provide one”. The E-569 shapes and E-570's capacitor-coupled gate go from ~277 iterations through optran, or six singular reports, to three iterations on the same values, and the held node leaks with C/gmin in a transient. ‘.option dcpath=gmin

E-576 *examples/run_regression.py*

THE SWEEP RUNS SUITES IN PARALLEL. `run_regression.py --jobs N` (or NG_JOBS=N) runs N suites at once on a thread pool, each in its own subprocess and directory as before; the per-suite lines come in completion order and the summary is unchanged. Every verify script keeps to its own directory -- a search of all 475 found none writing outside it -- and the three that assert a speed ratio measured on the machine (benchmark, nested_cond, reusesetup) run alone after the batch as the SERIAL set. The full sweep: 475 of 475 in 316 s under --jobs 8 against 1391 s sequentially; the remainder is the longest suites, not the count. The default stays sequential.

E-577 *src/frontend/com_track.c (new), com_track.h, src/maths/cmaths/cmath4.c, src/frontend/{parse, evaluate,commands,control,typesdef}.c, examples/track_examples/ (new, 36 checks per solver)*

THE TRACK COMMAND: EVERY PLACE A CONDITION HOLDS, AS A PLOT. The proposal of 2026-09-06 implemented, taking one or more expressions under one spec. The spec -- a locator (localmax, localmin, globalmax, globalmin), a crossing lhs==rhs interpolated linearly or in log x, a region lhs<rhs with interpolated entry and exit, or any boolean -- becomes the x positions where it holds; every expression is read there and both go into a plot trackN whose scale is named and typed after the source scale. Extrema are parabola-refined unless -raw, a plateau reports at its midpoint, and -prominence applies the alternating hysteresis rule so ripple never counts. The four locators are also `let` functions. Zero hits makes no plot and prints `track failed!`. Found and pinned on the way: the lexer splits < and > into words, ft_numparse advances its pointer, an ac plot keeps frequency complex-typed, a nested dc sweep is one vector whose scale turns back,

the plot-name table matched ”ac” inside ”track”, a quoted expression keeps its quotes. Full sweep 476 of 476.

E-581 *src/frontend/com_track.c, docs/handbook/03-ngspice-workflows.md, examples/track_examples/ (4 checks added, 40 per solver)*

$track_plot AND $track_hits. Asked whether `track` (E-577) combines with a 1000-run Monte Carlo: the locators (`localmax`, `globalmax`, ...) are ordinary expressions, so `montecarlo N -analysis ... -expr npk=length(localmax(v(out)))` records a peak count or position per sample with no loop (a vector-valued locator is a family only when every sample has the same hit count; E-552 refuses it otherwise and keeps the scalars); `track` itself is a command and goes in a `repeat` loop -- which worked, except that the script had to name the result `track$&n`, and `track` numbers its plots by SUCCESSFUL calls only, so one trial with no hits made every later trial read the wrong plot. After every `track`, `$track_plot` now names the plot it made and `$track_hits` holds the count, both cleared on entry (empty, 0) so a miss or a refusal never leaves the previous call's answer -- the `$montecarlo_plot` convention. The handbook gains the loop idiom, the `montecarlo` combination, and the two control-language traps that bit on the way (a one-hit track's length-1 vectors read as scalars; a bare > in `let` is redirection). VERIFIED: track 40/40 per solver (six seeded trials with misses), sweep 479/479.

E-582 *src/frontend/{com_sweep.c,com_track.c,com_track.h,commands.c}, docs/internals/ngspice_internals/{ngspice_statistics,ngspice_commands}.md, docs/handbook/03-ngspice-workflows.md, examples/mctrack_examples/ (new, 17 checks per solver)*

montecarlo -track: **`track`** RUNS AFTER EVERY SAMPLE'S ANALYSIS AND ITS HITS ARE RECORDED PER SAMPLE. Asked for `montecarlo 1000 -analysis "track v(a) -spec localmin"`: measured, it runs no analysis and tracks the stale plot a thousand times, since `-analysis` runs exactly one command; and E-552's `-expr` records a locator's positions only when every sample has the same hit count, while a VARYING count is the point of tracking under variation. New flag `-track "<track arguments>"` (one quoted word, `track`'s options begin with -; repeatable up to eight) runs `track <arguments>` after every sample's analysis, quietly, and records the result in the `montecarlo<n>` plot beside the `-expr` vectors: `track_hits` per sample (0 a miss, nan a sample that never solved) and every vector of the track plot -- the scale, `value`, `index`, a region's `x_out` and `width`, each with its type -- as `track_<vector>`, an Lmax x N family, HIT-MAJOR: row k (`track_time[1]`) is hit k of every sample on the `sample` scale, nan where a sample had fewer, so `plot track_value[0] vs r` puts the first peak against a recorded parameter; plain N-long vectors when no sample has more than one hit. Several `-track` record as `track1_*`, `track2_*`; combines with `-spec` (the yield too), `-expr`, `-lhs`, `-warm`. The per-sample track plots are destroyed as recorded. Two hooks in `track`: `track_quiet` drops the per-call summary and makes a miss silent (still setting $track_plot/$track_hits), `track_error` marks every failure other than a miss so montecarlo stops on a bad argument at sample 1 with the message once, recording nothing. `-track` without its quoted argument is refused showing the form; ”nothing to do” names it. Help text, statistics guide §6.2, commands table, handbook §3.3/§3.6. VERIFIED: mctrack 17/17 per solver (record equals `length(localmax(v(out)))` per sample, `-raw` first hit equals `globalmax`, [Lmax,N] dims with exact per-sample columns, `track_time[1]` the second hit, a region with `-which first` as plain vectors with `width == x_out - time`, two -track, misses, errors, refusals, with a yield), sweep 480/480.

E-583 *src/frontend/com_sweep.c, examples/mctrack_examples/ (a dc-sweep section, 20 checks per solver)*

montecarlo -track ON A DC SWEEP: **`track_v_sweep`**, AND A SPURIOUS ”Phi is not positive” FROM THE FAST PATH. The first dc-sweep example (a CMOS inverter's switching point and gain-region width under threshold variation) showed two things the transient examples could not. (1) A dc sweep's scale is `v-sweep`, so E-582 recorded `track1_v-sweep` -- which `print` reads as the subtraction `track1_v - sweep`. The record now maps every character that is not a letter, digit or underscore to _ (`track1_v_sweep`), the rule `sweep` uses for its own scale names. (2) Every run

printed `Fatal error: pm: Phi is not positive.` once, with phi 0.6 throughout and the results right: the fast path (E-320, models since E-344) refreshes each touched device type with a `DEVtemperature` pass after pushing a sample's values, and on the FIRST sample of a fresh circuit that ran before CKTsetup had applied the model defaults (MOS1 gives phi its 0.6 in MOS1setup), so MOS1temp saw phi == 0 and printed the error -- harmless only because CKTdoJob ran CKTsetup and CKTtemp for the analysis anyway. The pass now runs only when CKTisSetup is set. Every MOS level-1/2/3 model without an explicit phi under montecarlo, sweep or optimize on the fast path was affected. VERIFIED: mctrack 20/20 per solver (the dc scale spelled and typed, the switching point equal to (vtn + VDD + vtp)/2 within 20 mV and moving with the draws, the region's `width == x_out - entry`, no fatal message with the fast path still armed), sweep 480/480.

E-584 *src/frontend/{com_sweep.c,vectors.c,com_track.c,commands.c}, include/ngspice/fteext.h, examples/mctrack_examples/ (rewritten, 21 checks per solver), examples/track_examples/*

montecarlo -track RECORDS INTO A PLOT OF ITS OWN, track. E-582 put a -track's record into montecarlo as prefixed vectors (`track_hits`, `track_time`, `track1_*`). The point made against it: a `track` result already has a shape -- a scale, `value`, `index`, `x_out`, `width` -- and the Monte Carlo record is that shape stacked over the samples, so it belongs in a plot called track1 with those vectors bundled in. Now every -track is a plot `track<k>` named "Monte Carlo track: ": `sample` as its scale, `hits` per sample (0 a miss, nan a failed sample), and every vector of the per-sample track plots under its own name (`time`/`frequency`/`v_sweep`, `value`, `index`, `x_out`, `width`), Lmax x N hit-major as before; `track1.time[1]` is the second hit of every sample, `plot track1.value[0] vs r` the first peak against a recorded parameter. ngspice numbers plots per type from the last number handed out, so after a thousand destroyed per-sample plots the record would have been track1000: new `plot_typenum_forget()` in vectors.c drops that memory and the records take the first free names. montecarlo stays current with the counts and the -expr vectors; $track_plot names the last record and $track_hits counts the samples with a hit. `track`'s own plot now spells a dc scale `v_sweep` too (`print track1.v-sweep` was a subtraction). VERIFIED: mctrack 21/21 per solver (the plot's exact vector set and name, montecarlo1 free of track vectors, numbering track1/track2 with a later standalone track3, the dc sweep across plots), track 40/40, sweep 480/480.

E-585 *src/osdi/osdisetup.c, examples/abstolperf_examples/, examples/lrmdisc_examples/*

THE OSDI NATURE-ABSTOL REPORT UNDER ngdebug IS ONE LINE PER MODEL AND TOLERANCE. A 1000-device OSDI Monte Carlo deck with `set ngdebug` printed 2000 lines before its first sample -- E-539's `OSDI: node N convergence abstol = 1e-06 (declared by its nature)`, one per node the model touches, which read as a flood of warnings and said nothing a count would not. `flush_node_abstol` now tallies the nodes it changed by tolerance and prints one line per model type and distinct tolerance: `OSDI: mcdev2: convergence abstol = 1e-06 on 2000 nodes (declared by its natures)`; a model with several natures gets one line per tolerance (eight tallied, a closing count beyond). The setup is untouched and nothing prints without ngdebug. abstolperf and lrmdisc read the count instead of counting lines. VERIFIED: abstolperf 4/4, lrmdisc 34/34, lrmio 17/17 per solver, sweep 480/480.

E-586 *src/frontend/{device.c,spiceif.c,commands.c}, examples/altermulti_examples/ (new, 14 checks per solver)*

alter AND altermod APPLY EVERY name=value PAIR. `alter n1 ga=5m gb=6m` set `ga` and dropped `gb`, `altermod mm ma=3 mb=4` set `ma` only, and built-ins did the same (`alter r1 r=2k temp=50` left temp at 27) -- the worker split the first word carrying '=' and read the rest as value; its single-pair error sat on the legacy '='-less path. The front end now counts the assignments and hands the one-pair worker one `<target> name = value` list per pair (pre-split, since wl_splice frees the node it splits), each with its own E-544 journal bracket; an `@dev[param]=value` pair carries its own target; one pair, `dev = value`, vector values and the legacy form are untouched. Also from the hunt: `altermod mm mode=quad` (a bare word for a STRING parameter) said 'no such vector quad' -- a bare identifier is tried on the string setter first; and a $fatal during the operating

point no longer trails E_PANIC's 'doAnalyses: impossible error' after its own message. VERIFIED: altermulti 14/14 per solver, sweep 483/483.

E-589 *src/frontend/device.c (the classic show/showmod table), examples/showwidth_examples/ (new, 19 checks per solver)*

SHOW AND SHOWMOD PRINT EVERY NAME AND VALUE IN FULL. The table's name column was a fixed 11 and its device columns a fixed 21, and both constants were the PRECISION of the format as well as its width (`%*.*s`), so `averyveryverylonginstanceparam` listed as `averyveryve`, a hierarchical child's `a__a__a__r` as `a__a__a_` (a 30-level chain's parameters all identical), and a 35-character instance name, a model name or a string value longer than 21 lost its tail -- in the one listing meant to show them, while `print @lm[name]` printed the value under the full name. The widths are now decided per table: the name column is the longest name the table will print (the printer's own filter, plus explicit names after ':'), the device columns the longest instance or model name in the batch, the devices-per-row count follows from the widths so the table still fits `width` when it can, and every `%*.*s` of that printer is a `%*s` -- names, header rows, string cells and the '-' placeholder of a missing vector value, which now sits under the numbers. Short names keep the classic layout byte for byte; a device type without a model table is guarded. Hunt finding F7 of 2026-09-08. VERIFIED: showwidth 19/19 per solver, sweep 484/484.

E-590 *src/frontend/numparam/xpressn.c (the `{...}` substitution error), examples/hunt3diag_examples/ (shared with the compiler half)*

A BARE WORD ON A MODEL CARD SAYS HOW TO QUOTE IT. `.model pm prng s=b` failed with "Undefined parameter [b]" and "Cannot compute substitute", nothing pointing at the one-character fix: the card's bare word is wrapped as the expression `{b}` before numparam sees it, so numparam is where a `.param` name and a string value meet. When the fragment that failed is a bare identifier the message now adds "'b' is not a .param name; if it is meant as a string value, quote it: "b""; a broken expression keeps the plain message, and `s="b"` sets the string as before. Hunt finding F8(i) of 2026-09-08; the compiler half is in the openvaf change log. VERIFIED: hunt3diag 42/42 per solver, sweep 485/485.

E-591 *src/frontend/dotcards.c (the saveused reference scan), examples/saveforms_examples/ (new, 16 checks per solver)*

SAVEUSED KNOWS THE ACCESSOR SPELLINGS OF v(). The scan behind `.option saveused` collected `v()`, `i()` and `@dev[param]` only; ngspice's own aliases `vm() vp() vr() vi() vdb() vg()` (cpitf.c, one or two nodes) name the same vectors and were invisible. `print v(in) vdb(out)` after an `ac` saved `in` alone and `out` was "not available"; `.meas ac gain find vdb(out)` failed "no such vector"; and a block using ONLY `vm(out)` worked by accident, because the scan found nothing and the option stood down, saving every node on exactly the decks it was set for. A suffixed form is now registered as the plain accessor of each node it names (`vdb(a,b)` as `v(a)` and `v(b)`), which is what `save` understands; the plain forms keep their verbatim registration and `i()` takes no suffixes, as it has none. F2 of the 2026-09-09 dig. VERIFIED: saveforms 16/16 per solver, saveused and autoopts unchanged, sweep 486/486.

E-592 *src/spicelib/parser/{inp2n.c,inppas3.c}, src/include/ngspice/inpdefs.h, examples/adaptmsg_examples/ (new, 10 checks per solver)*

AUTOADAPT SAYS WHICH ADAPTER RULE FAILED, AND A .ic OR .nodeset ON A SPLIT BIT IS EXPLAINED. One compound test printed one message for three faults, and for a scalar or `[0:0]` adapter it read "must have exactly two bus ports of equal width (found 2 port(s), widths 1/1)" -- two ports, equal widths, refused, the rule (autoadapt splits BUS nodes only; a shared scalar node is never adapted) stated nowhere. Three tests, three messages: the port count, a port of width 1 with the rule and the `inout [0:N-1] p, n;` to write, or the two differing widths. And `.ic v(x[2])=0.5` on a bus autoadapt had split into `x_f`/`x_r` got the generic "IC on non-existent node" while the `.save v(x[2])` beside it got E-572's explanation: `.ic`/`.nodeset` resolve their

nodes in pass 3, before the split check runs on the surviving dot cards. The split list is intact there; both branches now name the split and offer `x_f[2]` / `x_r[2]`. F4 and F5 of the 2026-09-09 dig. VERIFIED: adaptmsg 10/10 per solver, autoadapt 26/26, sweep 487/487.

E-593 *src/spicelib/parser/inp2n.c (the candidate refusals in INPadapt), examples/adaptlisted_examples/ (new, 8 checks per solver)*

A .adapt-LISTED NODE THAT AUTOADAPT REFUSES IS REPORTED, AND THE REFUSAL NAMES THE EXTRA USE. E-466 made the option quiet by default and put the per-node "did not qualify" notices behind `=debug` -- right for a deck with many shared buses -- but that silence also covered a node the deck asked for BY NAME: `.adapt x` with a capacitor on `x[2]` printed nothing and ran unadapted (0.94118 where the split gives 0.94111/0.93935). E-467 reports a member that selects nothing; `x` selected a candidate the occurrence rule then refused, and E-466's own principle (a deck that asked for an adapter and did not get one must not run on in silence) was not applied. In `=debug` the refusal was a count, "occurs 3 times, not exactly twice", with no word about which line. A listed candidate's refusal is now a Warning under the quiet default (unlisted nodes stay quiet); the third OSDI instance is recorded and named ("n1, n2 and n3"), and `adapt_extra_line` quotes the first line outside the two ports that names the bus or a bit of it -- `c1 x[2] 0 1p` -- with the two ports and what to do. F3 of the 2026-09-09 dig. VERIFIED: adaptlisted 8/8 per solver, autoadapt 26/26, adaptquiet unchanged, sweep 488/488.

E-594 *src/frontend/dotcards.c (ft_saveused), src/frontend/outitf.c (name_eq), src/frontend/breakp2.c (save_already_present), examples/savenoise_examples/ (new, 11 checks per solver)*

.OPTION SAVEUSED NO LONGER REFUSES A NOISE OR SP ANALYSIS, AND **`save S_2_1`** MATCHES THE VECTOR IT NAMES. The option (E-469) infers a save list from the control block and the output cards; its one promise is that the deck still works. A noise run's plots hold onoise_spectrum, inoise_spectrum and the totals -- no node -- so the inferred list (`out`, from the `noise v(out) v1 ...` line itself) matched nothing of theirs and beginPlot refused the analysis: "no data saved for Noise analysis; analysis not run", on every deck with a noise run; a `.noise` card run through `run` failed the same way. An sp run was pruned to nothing of its own (S_i_j, Y_i_j, Z_i_j, Rbase), and `print S_2_1` failed even though the scan had registered the name: the deck reader lowercases a save name, the analysis publishes `S_2_1` in mixed case, and name_eq compared with strcmp -- stock `save S_2_1` fails the same way while `print S_2_1` finds the vector. Fix: whenever the option acts it also registers a `save all` restricted to NOISE and one restricted to SP, which every other analysis ignores, so a transient beside them stays pruned; name_eq compares without case (node names are lowercase throughout); and the E-417 dedup in ft_getSaves, which dropped a second `save all` differing only in its analysis restriction, now respects the restriction. Stock semantics untouched: an explicit `save out` before noise still refuses the run. F1 of the 2026-09-09 dig. VERIFIED: savenoise 11/11 per solver, sweep 489/489.

E-595 *src/spicelib/analysis/cktsetup.c (a reactive walk, the option words, the stamp), cktload.c, acan.c, src/osdi/osdisetup.c (OSDIdcpathEdges), osdiitf.h, cktdefs.h, spiceif.c, src/maths/KLU/klusmp.c (SMPzeroLines), examples/dcpath_examples/ (50 checks per solver)*

THE DCPATH HOLD IS RELEASED OUTSIDE DC -- dcpath=dc IS THE DEFAULT, dcpath=all KEEPS SPECTRE'S RULE. E-575 installs gmin on a node the DC walk cannot reach and kept it for the whole run, the one place the feature changed a result: a node reached only through a capacitor leaked with C/gmin in a transient (2 pF against 1 pS, tau = 2 s) and its AC bent below gmin/C. The hold is needed only where the matrix is evaluated at DC; in tran the companion conductance carries such a node, in ac jwC does. A second walk now counts the reactive edges (a capacitor, a MOSFET gate, an OSDI REACT entry); a node it reaches is held at DC only and released in tran and ac, one it also misses -- a current source into a lone node, an isolated transformer secondary -- is held in every mode, since optran runs as a transient. A released node whose diagonal is still exactly zero in tran (a zero-valued capacitor, a ddt() with a zero coefficient) keeps the hold. Words: `dcpath=dc` (default), `dcpath=all`, `.option dcpathall` beside a value. Found on the

way: under Sparse, E-571's all-zero-row hold read only the real part of the AC row (the matrix is flagged complex only at factorisation), so every capacitor-only node was held with gmin in every AC analysis -- vm() at 0.1 Hz 0.391 under Sparse, 0.500 under KLU; SMPzeroLines now reads the imaginary part regardless. Item 1 of the 2026-09-10 dcpath plan. VERIFIED: dcpath 50/50 per solver, sweep 489/489.

E-597 *src/spicelib/parser/inpdpar.c (e597_value_refusal in both loops of INPdevParse), src/spicelib/parser/inp2n.c (the n-line model refusal), src/frontend/inpcom.c (the model-usage scan), examples/barevalue_examples/ (new, 21 checks per solver)*

A PARAMETER NAME WITHOUT A VALUE IS REFUSED, NOT APPLIED AS ZERO. `n1 a b am w=2 m` -- the `=3` lost from `m` -- read the bare `m`, asked for its value at the end of the line, got 0 from the evaluator's error path and applied it: m = 0, the device gone from every analysis, nothing said. `temp` alone was 0 C, a second bare `w` was w = 0 refused deep in setup, a .model card's instance default (`.model im istr w`) put 0 into every instance, an integer that does not fit was applied as 0 on an instance line (the card refused it since E-509), and a built-in `r1 a 0 1k tc1` did the same after a "can't find model 'tc1'" that named the wrong cause. A bare word straight after the model (`n1 a 0 im k`) was taken as the model, reported as "Unable to find definition of model k", and the card it hid commented out as unused. Now the line is refused the way an unknown name is, and the message names the parameter and the shape: no value, not a number, does not fit an integer, on the .model card; the deck reader keeps the hidden card so the n-line parser can say "'k' is not a model and not a name=value parameter; the model is 'im'" or "'3' is a value without a parameter name". `tag=""` stays legal, `off` stays a flag, `m=0` stays the silent disable idiom. F2 of the 2026-09-10 integration hunt. VERIFIED: barevalue 21/21 per solver, sweep 491/491.

E-599 *src/spicelib/parser/inpdpar.c (the card-default side table), inpdefs.h, src/frontend/spiceif.c (altermod_instance_default, the read-side message), device.c (showmod rows), inp.c (the pre-pass Note), commands.c (pre_osdi live), src/spicelib/devices/dev.c + src/osdi/osdiregistry.c (the reload copy's source path), examples/carddefault_examples/ (new, 15 checks per solver), instdep re-pinned*

A CARD'S INSTANCE DEFAULTS ARE LISTED BY showmod AND MOVED BY altermod; A HOISTED pre_osdi -f SAYS WHERE IT RUNS. F4: `.model am cd width=3` was honoured but invisible to `showmod` and refused by `altermod am width=5` with a message recommending the very card it could not change -- after the card's defaults are replayed onto the instances nothing told one that took the default from one that wrote its own. The parser keeps the distinction now (INPcardDefaultNote, per instance and parameter id, cleared per deck), so `altermod` moves the default onto the followers, leaves the instances that set their own (on the line or by `alter`), records it on the card (an alias and the base name are one parameter) and says what it did; `showmod` lists the card's instance defaults; the read-side refusal names the card's default. F5: every `pre_` line is hoisted into the pre-pass, so a `pre_osdi -f` written after a `shell` recompile ran before it and said "reloaded"; a `-f` behind other commands in its block now gets a Note naming `osdi -f` as the form that acts there, `pre_osdi` is a live command at the prompt (was "no such command"), and the forced reload's staged copy resolves a relative name like the first load did (was "could not stage a reload copy" from another directory). VERIFIED: carddefault 15/15 per solver, instdep 23/23, sweep 493/493.

E-600 *src/frontend/outitf.c (the save probe), src/osdi/osdiparam.c (OSDIask quiet under it), fteext.h, src/spicelib/parser/inpdomod.c (unknown-type cards remembered), inpkmods.c, inpdefs.h, inpgmod.c (INPgetMod and INPgetModBin messages), inp2n.c, inp2m.c, examples/savemiss_examples/ (new, 11 checks per solver)*

A SAVED OPVAR WARNS ONCE AND AN INFERRED SAVE NOT AT ALL; A MISSING MODEL SAYS WHY. D1: a `save` of an OSDI operating-point variable before any analysis printed two warnings for one item -- the device's own "no operating point has been computed", raised by the ask beginPlot makes to check the name, on top of E-418's "has no value yet ... recorded per point once an analysis runs" -- and printed them for a save `.option saveused` inferred and the author never

wrote. The device is quiet under the probe (ft_save_probe), a plain read still gets its line, and an inferred save (E-496's autosaved mark) gets no warning. D2: a `.model` card whose type nothing defines got the bare "Unable to find definition of model ma": pass 1 drops the card with its own warning, which the error loop loses whenever the card sits after the line that uses it -- the usual layout, and where a plain `osdi` in the control block (run after the netlist) leads. Pass 1 now remembers such cards and INPgetMod names the card, its line, the type, `pre_osdi`, and why `osdi` is too late; an instance outside every bin of a binned model (E-495) gets the bins with their ranges and its own l and w, for `n` and `m` lines alike (was "could not find a valid modelname" for a MOSFET). A bin hit and a model nowhere in the deck read as before. VERIFIED: savemiss 11/11 per solver, sweep 494/494.

E-601 *src/frontend/spiceif.c (say_instance_level, the instance-side write branch), src/osdi/osdisetup.c (the range refusal's value), examples/opvarmsg_examples/ (new, 10 checks per solver), hunt2diag repinned*

AN OPERATING-POINT QUANTITY IS NAMED AS ONE; AN INTEGER OR STRING OUT OF RANGE SHOWS ITS VALUE. D3: reading an operating-point variable through the model name answered with E-560's instance-PARAMETER message ("declared (* type="instance" *), or resolved per instance because its default reads an instance parameter, lint L028") for something that is not a parameter at all, and a write got "has no parameter" or "no such parameter" for a name that exists. A read-only entry of the instance table -- an opvar, a terminal current, a built-in's `i` -- is named as an operating-point quantity now, on the model and on the instance, read and write; an instance parameter keeps E-560's text and an unknown name "no such parameter". D4: the OSDI range refusal printed the value for a scalar real and omitted it for an integer (where `k=0.4`, rounded to 0 against [1:3], is the whole question) and a string; both show it now. The duplicate-plus-alias line of D4 was closed by E-597 and is pinned. VERIFIED: opvarmsg 10/10 per solver, sweep 495/495.

E-602 *src/frontend/breakp2.c (settrace's dedup), src/frontend/runcoms.c (dosim: every analysis's measures), src/frontend/measure.c (the header), examples/multisave_examples/ (new, 9 checks per solver)*

TWO DOT CARDS NAMING ONE VECTOR FOR DIFFERENT ANALYSES KEEP BOTH ANALYSES; A BATCH RUN EVALUATES EVERY ANALYSIS'S .meas CARDS. `.print dc v(a)` beside `.print tran v(a)` in a batch deck: "no data saved for Transient analysis; analysis not run" -- each card registers a save restricted to its analysis and settrace's dedup kept one save per node NAME whatever the restriction, so the transient had no save at all; a `.meas dc` beside a `.tran` lost the transient the same way, a `.meas tran` beside a `.dc` the dc. The insert-time sibling of the ft_getSaves dedup E-594 fixed: a repeat is a repeat only for the same analysis, or under an unrestricted save. Behind it, dosim evaluated `.meas` for the LAST analysis only, so the other analyses' cards were skipped in silence -- every tran/dc/ac/sp plot the run produced is evaluated on its own plot now, in run order -- and the "Measurements for ..." header was chosen by the first card's type before the card was matched, naming the wrong analysis or standing over nothing; it names the analysis being evaluated. N6 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: multisave 9/9 per solver, sweep 496/496.

E-603 *src/frontend/outitf.c (beginPlot: the plot sequence of a multi-plot analysis, the deferred unmatched-name warning), src/include/ngspice/cktdefs.h (CKTplotsToFollow), src/spicelib/analysis/noisean.c (the count set before each plot; the integrated plot's result checked), src/frontend/dotcards.c (dotcard_on_plots), src/frontend/parse.c (ft_pnode_item_valid), examples/noisecards_examples/ (new, 21 checks per solver); examples/savenoise_examples/ [7b] re-pinned*

A NOISE ANALYSIS IS ONE ANALYSIS TO ITS SAVE LIST, AND A .print noise CARD PRINTS WHAT EACH PLOT HOLDS. A noise analysis publishes two plots in sequence -- the spectral densities (noise1) and the integrated totals (noise2) -- and the save list a `.print noise ...` card registers was applied to each plot on its own. `.print noise onoise_spectrum` named nothing

of the totals plot, so it was refused (”no data saved for Noise analysis; analysis not run”) after two false ”can't parse” warnings, and the analysis went on into a run descriptor with no plot behind it (its OUTpBeginPlot result was unchecked); `.print noise onoise_total` named nothing of the densities plot, opened first, so the WHOLE analysis was refused; E-493's unmatched-name warning spoke at each plot, reporting a name the other plot held; and ft_cktcoms ran the card on every noise plot with the whole item list, so a card that reached both plots printed ”vector ... is not available” from the other, and a card naming an item of each printed nothing. The analysis now tells the front end how many plots follow (CKTplotsToFollow, consumed by beginPlot): within the sequence a plot the saves do not reach is kept whole (given a save that applies to the analysis; a deck asking nothing of its `.noise` is refused as before), the unmatched-name warning is deferred to the last plot and printed once for the names no plot held, the ”can't parse” line is kept for `@`-spellings only, and a `.print` on a type with several plots gives each plot the items it can serve, reporting an item no plot serves once. N7 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: noisecards 21/21 per solver, savenoise 11/11 re-pinned, sweep 497/497.

E-604 *src/frontend/numparam/spicenum.c (nupa_eval: a card's failure attached to the card; nupa_done: the deck refused, not exit(1)), numparam/xpressn.c (message: the sink, the line labels, the empty-expression text), numparam/numparam.h, numpaif.h, src/include/ngspice/inpdefs.h (card.nupa_error), src/frontend/inp.c (folded in after the parse; the whole message printed; a .model card's numparam failure an error), src/frontend/subckt.c, examples/nupafail_examples/ (new, 13 checks per solver)*

A NUMPARAM FAILURE REFUSES THE LINE, OR THE DECK -- NOT THE PROCESS. `w=nan` or a trailing `w=` on an instance line reached numparam as the brace expression {nan} / {} and every numparam error ended the process at nupa_done() (”fatal error in ngspice, exit(1)”), the interactive session included, where `w=1e400` on the same line is refused at the line level. A failure on a device, model or dot card is now attached to the card (nupa_error, folded into the card's error after inp_dodeck's reset) and the deck reader refuses the line -- the author's text, numparam's reason and the parser's message, the error printed in full where only its first line was; a `{nosuch}` in a node position is refused too (the parser alone took it as a node); a .model card's failure is an error, not a ”Model issue” warning that ran the model on its default. A failure on a .param, .func or subcircuit-call line refuses the deck as an unknown subcircuit does (”the circuit is not loaded”), batch exit 1, the terminal's ”Run Spice anyway?” kept. The message's line labels were swapped (internal printed as netlist) and are right; `w=` with nothing after it says ”the value is missing -- nothing follows the '='”. N5 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: nupafail 13/13 per solver, sweep 502/502.

E-605 *src/frontend/inpc_probe.c (probe_head: the head insertions placed past a leading .control block; deck no longer moved onto the inserted card), examples/probeblock_examples/ (new, 5 checks per solver)*

A .probe CARD IN A DECK THAT OPENS WITH A .control BLOCK NO LONGER PRINTS ”.save: no such command available in ngspice”. inp_probe adds a `.save all` card when the deck has no save of its own and inserted it right after the deck's first card -- inside the .control block every OSDI deck opens with (for `pre_osdi`), where it ran as a command and was lost; the probes' generated `.save` line and a differential probe's source went to the same place. All three head insertions now go after that block's .endc, and `deck` stays on the first card. N3 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: probeblock 5/5 per solver, sweep 502/502.

E-606 *src/frontend/gens.c (dgen_hier_model_match, consulted in dgen_next beside E-410's instance match), examples/showmodhier_examples/ (new, 8 checks per solver)*

showmod FINDS A SUBCIRCUIT'S MODEL BY ITS HIERARCHICAL NAME. Expansion renames a model card inside x1 to `x1:rm` (nested `x1.x2:rm`); `altermod x1.rm` and `altermod x1:rm` reach it and `showmod x1.r1` shows it through the instance, but `showmod x1.rm` and `showmod x1:rm` answered ”No matching instances or models” -- the generator's grammar compares

`#x1.rm` whole and reads the ':' of `#x1:rm` as its subcircuit delimiter. A whole-word match beside that grammar (as E-410 did for instances) takes the query, with or without `#`, equal to the model's name with '.' standing for ':'; model queries only, and a bare name keeps its top-level meaning. N1 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: showmodhier 8/8 per solver, sweep 502/502.

E-607 *src/frontend/outitf.c (fileInit_pass2, the batch -r writer), src/frontend/rawfile.c (raw_write; raw_read puts an old file's i(...) / v(...) form of a device-parameter vector back), examples/rawparam_examples/ (new, 7 checks per solver)*

A DEVICE-PARAMETER VECTOR KEEPS ITS NAME IN A RAW FILE. The writers spell a node voltage v(x) and a branch current i(x), and a loaded file's v(out) and i(v1) resolve again; but the current-typed `@r1[i]` was written `i(@r1[i])`, a name nothing reads back (after `load`, `@r1[i]` asks the absent device and `i(@r1[i])` is the parser's i() of a non-source) -- every OSDI terminal current `.option savecurrents` records, in `write` and batch -r alike. A `@` name is a device parameter, never a node or a branch: both writers keep it verbatim, and the reader restores an old file's wrapped form. N4 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: rawparam 7/7 per solver, sweep 502/502.

E-608 *src/spicelib/analysis/cktmkvol.c (CKTmkSignal: adoption of a parse-time node, revival of a retired one; CKTmkVolt/CKTmkCur on it), cktmkcur.c, cktdltn.c (a device-local node retired, not freed; an adopted one left in place -- CKTdltNNum and E-470's CKTdltNodeSet), cktsetnp.c (the pending .ic/.nodeset entries), cktsetup.c, cktdest.c, src/include/ngspice/cktdefs.h, src/spicelib/parser/inppas3.c, examples/internalnode_examples/ (new, 11 checks per solver); examples/sweepparam_examples/ [5] re-pinned*

A DEVICE'S INTERNAL NODE CAN BE NAMED BY .ic, .nodeset, .tf AND .pz, AND A JOB BOUND TO ONE STAYS BOUND. A device builds its internal nodes (`n1#mid`, `d1#internal`) at setup, after the deck is parsed. A .tf/.pz card naming one had the deck reader make a node of that name ahead of the device (E-429's inp_analysis_node), and CKTmkVolt, told the name existed, linked a SECOND node under it: the deck's node floated ("held only by gmin", singular-matrix warnings, gmin and source stepping), the .nodeset/.ic went to it, `.print op v(n1#mid)` printed its 0 beside the real 0.75, and E-429 refused the .tf output as a node no device connects to; a .ic/.nodeset alone was refused "on non-existent node". Interactively, `tf v(n1#mid) v1` after an `op` bound to the live node, which the run's own unsetup FREED and setup rebuilt -- the job read freed memory (a transfer function of -5e-4 for a 0.75 divider, the same number as its output impedance). The device now ADOPTS a parse-time node of its internal node's name (devRef, adopted; unsetup leaves it, the next setup adopts it again); a device-local node is RETIRED under its name at unsetup and revived as the same struct at the next setup, so a job's pointer stays valid; and a .ic/.nodeset on `<instance>#<node>` is kept by name and placed by CKTsetup once the node exists, a suffix the device does not build reported there once and no floating node made. N2 of the 2026-09-10 integration hunt, a stock ngspice defect. VERIFIED: internalnode 11/11 per solver, sweepparam 32/32 re-pinned, sweep 502/502.

E-609 *src/frontend/com_sweep.c (com_montecarlo: the exprs in two passes around the tracks, an expr defined as a vector of the sample's plot; each sample's track plot kept until the specs and exprs have read it; mc_track_refs, mc_expr_sample; the parse-time check; the report), examples/mcyield_examples/ (new, 14 checks per solver); handbook 3.3*

montecarlo: THE YIELD OF A TRACKED QUANTITY, IN ONE COMMAND -- AN -expr FEEDS A -track OR A -spec, AND A -spec OR -expr READS THE TRACK. `-analysis`, `-track`, `-expr` and `-spec` were accepted together, but a -spec was evaluated on the analysis plot before the tracks ran, so nothing in it could reach a track's result (`-spec value`, `-spec track1.value`: "did not resolve"); the locators as functions covered the plain cases only, not a track's -prominence, -edge, -which or a region's x_out/width, and the yield of a tracked quantity was a hand-written repeat loop. Now the tracks run first and each sample's plot is kept until the specs and exprs have read

it: track<k>.<vector> in a -spec or -expr names the k-th -track of the command for the sample being judged (track1.value, track2.x_out, track1.value[0] for the first of several hits, track1.hits the hit count -- 0 on a miss, so -spec track1.hits -max 0 is the yield of "nothing there"), rewritten to the sample plot's own name before evaluation; a metric may mix the two plots. A spec on a track that had no hit is a violation, counted apart in the report; an -expr on one is nan for that sample, with a NOTE; a track beyond the flags given is refused at parse time. The other way round, an -expr that does not read a track is evaluated before the tracks and defined as a vector of the sample's plot, so it can feed a -track or a -spec: -expr q=v(out)*2 -track "q -spec globalmax -output pk" -spec track1.pk -min 4 chains; a name the analysis plot holds is left to the plot, said once. Reported by the user. VERIFIED: mcyield 14/14 per solver, sweep 503/503.

E-610 *src/frontend/mcsave.c, mcsave.h (new: the recorder, csv/txt, a stored-zip .xlsx writer), src/frontend/numparam/xpressn.c (a .param or subcircuit value drawn; a device slot whose brace draws or reads a drawn symbol), numparam/spicenum.c, src/frontend/com_sweep.c (the fast path's slots), src/osdi/osdisetup.c and src/include/ngspice/osdiitf.h (OSDImcEnabled/HasStats/Snapshot), src/frontend/spiceif.c (the row per run; the option names), runcoms2.c, main.c, Makefile.am, examples/savemc_examples/ (new, 17 checks per solver); handbook 3.6*

.option savemc -- EVERY PARAMETER WITH STATISTICS, ONE ROW PER RUN, IN A FILE BESIDE THE NETLIST. For every run-class command (op, tran, run, ...; a failed run marked) a row with the value in force of every parameter with statistics: a device slot whose value draws (r1 in out {agauss(1k,50,1)}, or a random .param inlined there -- each use its own draw, as ngspice has it), named r1 or m1:w; a subcircuit call's own drawn value, x1.p; under .option osdimc every OSDI parameter with declared statistics, read off the devices as @sm[r] / @n1[dr] (the baseline trial the nominal, a pinned or pending draw as the device ran). The file is mcparams__. in the netlist's directory, unique within a second; csv by default, savemc=txt tab-separated, savemc=excel a genuine .xlsx (a stored zip, readable by Excel/Numbers/LibreOffice/openpyxl, written in full every 25 rows and at the end), savemc=<name>.<ext> a named file; a reset continues the file (every montecarlo sample is a row; on the fast path a subcircuit's own value is not re-derived and is empty there), a different deck starts another, nosavemc turns it off. .option automc_save (alias osdimc_save) records the OSDI parameters only. Nothing to record, or OSDI statistics with osdimc off, is said once. Requested by the user. VERIFIED: savemc 17/17 per solver, sweep 504/504.

E-611 *src/frontend/mcsave.c, mcsave.h (MCSAVEappend: a value onto the last row, the csv/txt last line rewritten in place; com_writemc; MCSAVEactive), src/frontend/com_sweep.c (com_montecarlo: -writemc, its items evaluated per sample after the tracks, specs and exprs), src/frontend/commands.c, src/frontend/evaluate.c (op_ind: element 0 of a one-point vector), src/frontend/parse.c (ft_getpnames_from_string_quotes), src/frontend/numparam/xpressn.c (mcs_slot_name: a .model card's slot), examples/writemc_examples/ (new, 15 checks per solver; mc_example6 beside it); handbook 3.6*

writemc AND montecarlo -writemc: VALUES COMPUTED AFTER A RUN, ONTO THAT RUN'S ROW OF THE savemc FILE. E-610's row was written the moment the run ended, and what the . control block computes from the run (a peak, a meas result, a track's hit count) came after it. The row of the last run stays addressable: writemc [name=]<expression> ... puts each scalar value onto it after a run in a repeat/reset loop (a vector is refused with a message, an undefined name too; before any run there is no row and it says so; with savemc off it says so once and does nothing), and montecarlo ... -writemc [name=]<expr> ... does it per sample after the tracks, specs and exprs, so an -expr name or a track. may be listed (a non-scalar reported once, that cell empty; a track beyond the flags refused at parse; savemc off noted once). A column is added on first use, a row that never gets it is empty there, a name that is a draw's lands as <name>* beside it; the csv/txt last line is rewritten in place (seek, truncate, rewrite, flush) so the file stays complete row by row, a new column rewriting the header once. Found on the way: op_ind refused any index on a one-point vector ("indexing a scalar"), so track1.time[0] failed on every sample with exactly

one hit and a `-spec track1.value[0]` became an unresolved metric there; element 0 of a one-point vector is now itself, any other index still refused naming the one element -- which exposed sweepguard's `disto lin 1 1e6 1e6` (a legitimate one-point run that had passed the suite's leak test only because d[0] failed); re-classified. Then driven through the shared library from a schematic front end (nets spelled /name, its own run a plain .op), which gave way in three places, all fixed: v(/mid)/v(/in) did not parse in writemc, -writemc, -expr, -spec or sweep -output (print and plot auto-quote a v() argument holding an arithmetic character; the expression evaluators called the plain parser -- a string form of the same quoting now serves them); a .model card whose parameter draws was a `<model>:<key>` column on montecarlo's fast path but not on a plain run (the re-source path's slot naming refused every dot card; a .model line's key={...} slot is now named the same on both); and montecarlo with only -writemc was refused as nothing to do (a -writemc with a savemc file to land in is a result; with savemc off it still is nothing to do, said without 'the run proceeds'). Through libngspice (ngSpice_Circ, then bg_run) the file holds the 20 montecarlo rows with gain/vmid and the host's own .op row with them appended after the run. Requested by the user. VERIFIED: writemc 15/15 per solver, sweepguard 43/43 re-classified, shared-library run 21 rows, sweep 505/505.

E-612 *src/frontend/inpcom.c (the reader's case folding: an .option card, or the `option` command of a .control block or .spiceinit, that names a file; inp_casefix(), which inp_getopts() runs over every .option card; three helpers naming the file-valued options savemc=, automc_save=, osdimc_save=), examples/savemc_examples/ (17 -> 24 checks per solver); handbook 3.6, the statistics guide 8*

THE NAME OF A savemc FILE KEEPS ITS CASE AND ITS BYTES. `.option savemc=<name>` names a file, and the deck reader treated the name as any other word: the case was folded (`savemc=MyRun/Draws.csv` wrote `myrun/draws.csv` -- the same file on the default macOS volume, a directory that does not exist on a case-sensitive one, and there every row was lost in silence; under KiCad the absolute path in the note read `/private/tmp/…/-users-meisam-…` while the `pre_osdi` line beside it kept its case, because that command is on the reader's exemption list and `.option` was not), inp_casefix() -- run by inp_getopts() over every .option card -- replaced each non-ASCII byte with _ (`Résumé_MC.csv` became `r__sum___mc.csv`), and a quoted name lost its quotes and so its spaces. The value of a file-name option is now left alone by both passes: the option's name and every other word on the card are folded as before (`OSDIMC MCSEED=5` beside it is still honoured), the value keeps its bytes, a quoted value keeps its quotes so the lexer keeps it whole and cp_unquote strips them (`savemc="dir with space/My Draws.csv"` is a path with spaces), `listing` shows the card as written; a format keyword and an extension are still matched case-insensitively (`savemc=CSV`, `automc_save=Osdi.TXT`). F6 of the 2026-09-12 hunt. Verified `savemc` 24/24 both solvers; full sweep 505 of 505.

E-613 *src/frontend/mcsave.c (MCSAVErun's first row: mcs_mkdirs() makes the directories of a given name, mcs_can_open() tries it, a failure is reported with the reason and the rows go to the dated default beside the netlist, or the recorder says so and records nothing; text_rewrite() and xlsx_write( ) report a failed open once instead of returning in silence), mcsave.h (MCSAVEappend -3: no file for this circuit; writemc says so once), examples/savemc_examples/ (24 -> 30 checks per solver); handbook 3.6, the statistics guide 8*

A savemc FILE THAT CANNOT BE OPENED IS SAID, AND ITS DIRECTORY IS MADE. `.option savemc=nodir/p8a.csv` printed `recording ... to ./nodir/p8a.csv` and then nothing: the only open was inside the row writers, and each answered a failed fopen() with a bare return, so every row and every `writemc` value was lost without a word -- the same for a name that is a directory, a read-only tree, a directory removed during the run. The name is now tried when it is chosen, at the first row: the directories of a given name are made, each level in turn (as `pre_osdi` makes `osdi/`); a name that still cannot be opened is reported with the reason (`cannot create directory ro/sub: Permission denied`, `Is a directory`) and the rows go to the dated default beside the netlist, the note naming the file used; when that fails too the recorder says so once and records nothing for the circuit, and a `writemc` there says once that its value is not recorded; a later

open that fails is said once and the rows are kept in memory, the file written complete when the directory is back. A name that could always be opened behaves exactly as before. F7 of the 2026-09-12 hunt. Verified `savemc` 30/30 both solvers; full sweep 505 of 505.

E-614 *src/osdi/osdisetup.c (OsdiMcNominal.stale; OSDImcNoteUserWrite marks the never-given statistical parameters of the written device stale and clears their given flag, and marks a user-written statistical parameter given; osdimc_capture refreshes a stale entry from the re-resolved default; OSDImcNewRun defers the trial's draws to OSDIsetup when an entry is stale; the option-off restore skips a stale entry), src/frontend/spiceif.c (doset_user reports integer writes to the hook), examples/osdimc_examples/ (29 -> 36 checks per solver, smcdep.va new); handbook 3.6, the statistics guide 7*

A NEVER-GIVEN STATISTICAL PARAMETER FOLLOWS THE PARAMETER ITS DEFAULT READS. With `.option osdimc`, a statistical parameter the deck never gave is resolved by the model's setup from its default expression, and that expression may read another parameter -- a child's binding in a Verilog-A hierarchy (`leaf #(.r(rl)) c1`, flattened to `c1__r`), a plain `parameter real rb = rl`, an instance default from an instance parameter, a default from an integer. The nominal table captured the value at the FIRST setup and never again, so after `altermod tm rl=500` the child still drew around 1000 -- the device ran at 1011 where 500 was set, with no message (a `montecarlo` loop was right: its internal re-source drops the table). A user write now marks the never-given statistical parameters of its device stale and clears their given flag; the next run's setup re-resolves the default, the capture re-reads it and the trial's draws are applied after that -- the draw is the same pure function of (seed, trial, owner, id), so `c1__r` moves from `1000 + d` to `500 + d`. The written parameter itself is given now and is not re-resolved, and the option-off restore keeps a value the user gave instead of the default (hunt F3: `alter @n1[dr]=50`, `unset osdimc` wrote 0 back). Integer writes reach the hook. Machine writes (sweeps, `sens`, optimizers) still neither recentre nor re-resolve. F2 and F3 of the 2026-09-12 hunt. Verified `osdimc` 36/36 both solvers; full sweep 505 of 505.

E-615 *src/frontend/mcsave.c (a session registry of the names written and the deck that wrote each: mcs_used_by, mcs_note_used, mcs_unused_variant; MCSAVErun's first row takes `<stem>_2.<ext>` when another deck wrote the given name in this session), examples/savemc_examples/ (30 -> 33 checks per solver); the statistics guide 8*

A SECOND DECK NAMING THE SAME savemc FILE GETS THE NEXT FREE NAME. `source a.cir` then `source b.cir`, both with `.option savemc=shared.csv`: the second deck's first row opened the file for writing and the first deck's rows were gone, the note reading exactly as before -- E-610's "a different deck starts its own file" was an overwrite when the name is fixed. The recorder now remembers every name it has written in the session and the deck's title; a different deck naming one of them takes `shared_2.csv`, `_3`, ... (the first this session has not written) and says `shared.csv holds the rows of '* deck a' from earlier in this session and is kept; this deck's rows go to shared_2.csv`. The same deck re-sourced still continues its file; a name from an earlier ngspice run is still replaced, as any output file is. F17 of the 2026-09-12 hunt. Verified `savemc` 33/33 both solvers; full sweep 505 of 505.

E-616 *src/osdi/osdisetup.c (OSDImcNoteRedraw: the in-place re-draw of a parameter's netlist expression becomes its osdimc nominal and releases the pin), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c (sw_fp_apply reports a random bind's direct write to the hook), examples/osdimc_examples/ (36 -> 39 checks per solver); the statistics guide 8*

A LOOP COMMAND'S NETLIST RE-DRAW IS THE NOMINAL THE MODEL'S STATISTICS SIT ON. One parameter on both channels -- `.model rm rstat r={agauss(1000,300,3)}` with `( * std=25 *)` on `r` under `.option osdimc` -- composed on montecarlo's re-source path (each sample's reset drops the table and the re-capture takes the fresh draw as the nominal) but not on its fast path, the one armed whenever every random expression sits in a braced slot: the in-place machine write pinned the entry, montecarlo runs without a hold so the pin was cleared at the run, and the trial's delta was applied over the FIRST sample's draw on every sample -- the record said

909.89 for the netlist draw while the device ran at 1033.69 + delta, the row contradicting itself by up to 173 ohm and the two paths of one command disagreeing. Under `sweep` the pin held and the model's statistics were silently off that parameter for the whole sweep. sw_fp_apply now reports a random bind's write to OSDImcNoteRedraw: the value is the sample's nominal and the pin goes; the delta is the same pure function as before, a swept knob stays pinned, a user write still recentres through its own hook. `@rm[r]` = `rm:r` + delta on every row, the same delta per sample on both paths. The guide now says that a `repeat ... reset ... op` loop varies the netlist channel only (a user reset restarts the trial sequence, E-535) and `montecarlo` varies both. F18 of the 2026-09-12 hunt. Verified `osdimc` 39/39 both solvers; full sweep 505 of 505.

E-617 *src/frontend/mcsave.c (mcs_col.model; styles.xml with a bold font and a second cell style; xlsx_str_cell_style for the header row), mcsave.h (MCSAVEparam takes the kind), src/frontend/numparam/xpressn.c (mcs_slot_name reports a .model card's slot), src/frontend/com_sweep.c (a bind's tier), src/osdi/osdisetup.c + osdiitf.h (OSDImcSnapshot's callback gets is_model), examples/savemc_examples/ (33 -> 34 checks per solver); handbook 3.6, the statistics guide 8*

THE savemc WORKBOOK'S HEADER SETS A MODEL PARAMETER'S NAME IN BOLD. A `savemc` file mixes the draws a model card carries, shared by every instance (`rm:r`, `@rm[r]`), with the draws an instance carries on its own (`r1`, `n1:dr`, `x1.c`, `@n1[dr]`), and the header did not say which was which. Every column now records whether its parameter is a model card's -- the numparam slot namer says so for a `.model` line, a fast-path bind carries its tier, the OSDI snapshot's callback says whether the owner is a card or an instance -- and the xlsx writer declares a bold font and a second cell style and writes a model column's name with it; instance, `.param` and `writemc` columns and the three fixed ones stay regular. Read back with openpyxl: `rm:r` and `@rm[r]` bold, nothing else. csv and txt are unchanged. Verified `savemc` 34/34 both solvers; full sweep 505 of 505.

E-618 *src/osdi/osdisetup.c (OSDImcNewRun's option-off restore skips an entry pinned under an open hold -- the bracketed command's own push -- and still restores over a completed command's leftover pin), src/frontend/mcsave.c (MCSAVErun reads the OSDI columns on every row once the file has them, option on or off; cols_have_osdi), examples/osdimc_examples/ (39 -> 42 checks per solver); the statistics guide 7*

THE RESTORE AFTER unset osdimc KEEPS A LOOP COMMAND'S PUSH. The first run after `unset osdimc` puts every drawn parameter back to its nominal; `sweep rr 500 1500 500` right after the `unset` had already pushed its first point's 500 through the machine setter, and the restore wrote 1000 over it -- readback 1000, 1000, 1500, and right only if an `op` happened to sit between. Under an open hold a pinned entry (E-535) is the bracketed command's own write of this point and the command restores what it pushed when it ends, so the restore now leaves it alone; outside a hold a pin is a completed command's leftover (a `.dc` sweep restores the drawn value it found) and the restore proceeds. The `savemc` rows after `unset osdimc` used to repeat the last trial's draws for the OSDI columns because the recorder stopped reading them with the option off; once the file has those columns they are read on every row, so the rows show the restored nominal and the sweep's points. F19 of the 2026-09-12 hunt. Verified `osdimc` 42/42 both solvers; full sweep 505 of 505.

E-619 *src/frontend/mcsave.c (styles.xml built from four fonts -- every cell's and the header's three styles; mcs_fonts, mcs_font_parse, xlsx_font_xml; the options savemc_font, savemc_fontsize, savemc_model, savemc_instance, savemc_writemc read at each write), src/frontend/inpcom.c (savemc_font keeps its case), src/frontend/spiceif.c (the five names known), examples/savemc_examples/ (34 -> 36 checks per solver; writemc_examples' xlsx parser widened); handbook 3.6, the statistics guide 8*

writemc COLUMNS IN BLUE, AND THE WORKBOOK'S FONTS AS OPTIONS. The `savemc` workbook's header row now has three styles: a model card's parameter bold (E-617), an instance's regular, and a column that `writemc` or `montecarlo –writemc` put on the row -- neither kind -- blue. Five options set the fonts, read at every write of the workbook: `savemc_font=<name>`

(Calibri; the name keeps its case, quoted when it has spaces), `savemc_fontsize=<pt>` (11), and `savemc_model` (bold), `savemc_instance` (regular), `savemc_writemc` (blue) as style lists joined with + -- `bold`, `italic`, `underline`, `regular`, a colour name or a hex RRGGBB (`savemc_model=bold+navy`, `savemc_writemc=red+underline`; a comma cannot join them, the .option card's lexer splits there). A token that is none of these is said once and ignored. csv and txt are unchanged. Verified `savemc` 36/36 both solvers; full sweep 505 of 505.

E-620 *src/osdi/osdisetup.c (osdimc_zero_sigma: the draw applier says once per parameter when a relative sigma meets a nominal of 0, and skips the draw), examples/osdimc_examples/ (42 -> 44 checks per solver, smczero.va new); the statistics guide 7*

STATISTICS THAT CAN NEVER VARY ARE SAID. `(* std_rel=0.1 *)` on a mismatch parameter's natural default of 0: the sigma is relative to the nominal, so it is 0, the draw is exactly 0 on every trial (`osdimc_verbose` printed `= 0 (nominal 0)` forever) and 300 montecarlo samples gave min = max = 0, without a word. The draw applier now says once per parameter -- `n2:dr declares std_rel=0.1 on a nominal of 0: the sigma is 0 and the parameter never varies; give it a nonzero value, or declare an absolute std` -- whatever the source of the 0 (the default, the deck's own `dr=0`, an `altermod` to 0), and skips the draw; a recentre to a nonzero value draws again. The compiler's half (the same enhancement) warns at the declaration. F16 of the 2026-09-12 hunt. Verified `osdimc` 44/44 both solvers; full sweep 505 of 505.

E-621 *src/frontend/inpcom.c (inp_fix_agauss_in_param: mvnorm on the list of random functions whose .param is inlined per use), src/frontend/inp.c (eval_agauss: mvnorm(i) in a B-line, one argument, through mc_corr_component), examples/mcfastpath_examples/ (7 -> 10 checks); the statistics guide 5*

A .param THAT CALLS mvnorm() IS INLINED LIKE ONE THAT CALLS agauss(). ngspice turns a .param calling agauss/gauss/unif/aunif/limit into a .func and inlines it into every line that reads it -- each use its own draw -- and that is what lets montecarlo's fast path see the draw in a device's brace and re-draw it per sample (E-346). mvnorm, the correlated draw of E-151, was not on the list: `.param pm = 1000 + 50*mvnorm(1)` stayed one value for the whole run as soon as any other random binding armed the path -- sigma 3e-12 over 200 samples on the guide's matched divider, a yield of 0 or 100 % with full confidence -- while the reset idiom, highsigma, wcd and mcsample were right. On the list now: the brace carries the call, the fast path counts and re-draws it, every use in a sample reads the same correlated component (the vector is drawn once per sample and cleared at the boundary the fast path raises), a B-source reading such a .param is evaluated through mc_corr_component. Measured: sigma 53/52 and rho 0.907 for the 0.9 asked. F1 of the 2026-09-12 hunt. Verified `mcfastpath` 10/10; full sweep 505 of 505.

E-622 *src/maths/misc/randnumb.c + include/ngspice/randnumb.h (the -inflate scope table, mc_scope_*; the netlist SSS draw honours it; mc_dim_push/pop name the dimension; the .param -> slot aliases), src/frontend/numparam/xpressn.c (the .param evaluation and the device brace push their savemc names), src/frontend/inpcom.c (inp_stat_alias_register records the alias when the inliner rewrites a reader; the table cleared on a new deck), src/osdi/osdisetup.c (OSDImcScaleScopeAdd/Clear/Hits and osdimc_scale_for as views on the shared table), examples/highsigma_examples/ (8 -> 12 checks); handbook 3.6, the statistics guide 4.1*

highsigma -inflate SCOPES THE NETLIST DIMENSIONS TOO. E-538's scope table lived with the OSDI draws; the netlist scaled-sigma sampler never looked at it and inflated every Gaussian .param at full -scale whatever the specs said -- `-inflate rr`, `-inflate r1`, `-inflate @r1[resistance]` all "matched nothing" while rr was inflated regardless (the banner and the NOTE both false), and an OSDI metric beside ten netlist bystander .params collapsed its weights to an ESS of 2.6 of 3000 with `-inflate @mm[r] -inflate @n1[dr]` unable to rescue it. One table now, consulted by both kinds of draw: under a scoped run the evaluator names each dimension as savemc names it (a .param, a device slot r1 / r1:key / rm:r / x1.p) and a dimension out of scope draws at its nominal spread with weight 1; a .param's name still works because the inliner records

`rr -> r1` when it rewrites a reader (the table survives the resets highsigma makes). Measured: rr and r1 reproduce the unscoped 132/2000 exactly, nosuch gives 0 with the note, and the bystander deck goes from ESS 2.6 (P 6e-7 +/- 46 %, flagged) to ESS 631 (P 3.94e-4 +/- 14 %, against Phi(-3.35) = 4.15e-4). F4 of the 2026-09-12 hunt. Verified `highsigma` 12/12; full sweep 505 of 505.

E-623 *src/osdi/osdisetup.c (osdimc_lhs_stratum, osdimc_lhs_u: under -lhs a draw's uniform is (stratum + jitter)/N with the stratum this sample's rank among the N samples' hashes under the dimension's key; osdimc_z takes the inverse CDF, a truncation confines the uniform; OSDImcLhs, cleared by OSDImcSeedOffset(0) and the interrupt reset), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c (montecarlo -lhs turns it on; the E-537 note gone), examples/lhs_examples/ (12 -> 16 checks), examples/mcpolicy_examples/ ([26] checks the strata); handbook 3.6, the statistics guide 3*

montecarlo -lhs STRATIFIES THE MODEL-DECLARED DRAWS TOO. The netlist's Latin-Hypercube sampler lands each dimension's N samples one per stratum; the osdimc draws were pure hashes of (seed, sample, owner, id) with no stratum to land in, so beside a stratified agauss the (* std *) draws clumped (0.71..0.78 six times of twenty) under a banner that said "20 Latin-Hypercube samples", and E-537 had the command say so. Under -lhs a model-declared draw's uniform is now (stratum + jitter)/N: the stratum is this sample's rank among the N samples' hashes under the dimension's key (seed, owner, id) -- a random permutation of 0..N-1 as a pure function, nothing stored, no RNG state -- the jitter the draw's own hash; the Gaussian coordinate is the inverse CDF of it, a truncation confines the uniform instead of rejecting, a uniform shape takes it directly. Each dimension has its own permutation (independent strata), the same seed reproduces, a run without -lhs draws byte-identically to before, and highsigma/wcd never see the flag. F5 of the 2026-09-12 hunt. Verified `lhs` 16/16 both solvers, `mcpolicy` 46/46; full sweep 505 of 505.

E-624 *src/frontend/mcsave.c (every row remembers the plot its run made -- MCSAVErunBegin notes the plot current before the run, a run that failed before making one records none; MCSAVEplotIsRow: may a value read off the current plot go onto the last row; com_writemc asks it first), src/frontend/spiceif.c (if_run calls MCSAVErunBegin before doAnalyses), examples/writemc_examples/ (15 -> 18 checks); the statistics guide 8.1*

writemc LANDS ONLY ON THE ROW OF THE RUN WHOSE PLOT IT READS. A trial whose op was refused at setup (a draw outside the model's range) made no plot, its row was written as failed, and the previous successful run's plot stayed current -- so `writemc ia=i(v1)` copied the previous trial's value onto the failed row (hunt F8 of 2026-09-12); after a run stopped at a breakpoint (no row) every writemc landed on the row before it (half of F9). Every row now remembers the plot its run made (none for a run that failed before making one; a transient that died part-way keeps its partial plot), and writemc writes only when the current plot is the row's own or an older one chosen with setplot -- refused, once per command, when the row's run made no plot ("row 2 (op) failed before it made a plot, so the current plot op1 is another run's; nothing is put on that row") or when the current plot was made after the row by a run that has no row. The cell stays empty, as montecarlo's -writemc already left it on a failed sample.

E-625 *src/frontend/mcsave.c (MCSAVErun takes MCS_PAUSED and writes the trial's row at the pause, status paused, with its draws and its plot; MCSAVEresumed sets that row ok or failed when the resume that continued the same plot ends; text_rewrite_last factored out of MCSAVEappend), src/frontend/spiceif.c (if_run emits the paused row at E_PAUSE and reports the resume branch's outcome), examples/writemc_examples/ (18 -> 21 checks); the statistics guide 8*

A RUN STOPPED AT A BREAKPOINT IS A ROW -- paused, THEN THE resume's OUTCOME. A transient drew its trial at setup, stopped at `stop when time > 2u` and was resumed to the end, and the savemc file never had a row for it: the recorder is called when a run-class command returns, a pause returns E_PAUSE before that, and resume is not a run-class command (hunt F9 of 2026-09-12). The row is now written at the pause, marked paused, with the draws in force and the run's plot, so a writemc there lands on it; the resume that completes the run turns the same

row ok, one that fails turns it failed, one that pauses again leaves it paused -- the resume starts no row of its own, and touches the most recent paused row only when the plot current at its end is that row's. One row per trial wherever it stopped on the way; a run left paused stays paused in the file.

E-626 *src/frontend/mcsave.c (a per-row list of the parameters the dc swept -- the OSDI snapshot's per-run write count and span, the dc job's resolved names: a source or resistor, the literal `@owner[param]` as owner:param, E-534's targets while present; those cells empty on the row, one note with the levels), src/osdi/osdisetup.c + osdiparam.c + osdidefs.h (OSDImcNoteParamWrite takes the stored value; run_writes per entry, zeroed at OSDImcNewRun, spanned but for the restore), include/ngspice/osdiitf.h (the snapshot callback takes an OSDImcSnapshotItem), examples/savemc_examples/ (36 -> 37 checks); the statistics guide 8*

A dc THAT SWEEPS A RECORDED PARAMETER LEAVES ITS CELL EMPTY, SAID ONCE. `dc @rm[r] 900 1100 100` drew its trial, swept r through the machine setter (each level pinning the entry past the draw) and put back what it found; the row, made after the run, read the devices and said `@rm[r]` = 966.08 -- a draw the device never ran at (hunt F10 of 2026-09-12); `dc r1 ...` over a resistor whose value draws recorded the slot's draw the same way, and `sweep @rm[r] ... -analysis op` became one dc run with the same row. A parameter the dc swept now has no value on that row -- the cell is empty and the row says once which parameters, with their levels -- while the other columns keep what the device ran with. Which parameters comes from two sides: the OSDI entries count the machine writes they took during the run (a sweep command's push before a point's run is not one), the netlist slots come from the dc job's own names. Only a dc row (or a run whose job is a .dc card) is read this way; a per-point sweep stays one row per point with the pushed level.

E-627 *src/spicelib/parser/inp2n.c (the bus bases E-572 remembers carry every bit in both spellings -- autobus_note_base takes the port's terminal names; INPreportBusBases lists the other-spelling nodes the deck wires, with the cure for each direction), examples/autobuskicad_examples/ (27 -> 31 checks)*

THE OTHER SPELLING OF THE BITS A BUS PORT EXPANDS TO IS NAMED. `V1 in[0] 0 1` beside `N1 in b kb` under `.option autobus=kicad` put the source on `in[0]` and the device on `in_0_` -- two nodes per bit, nothing in common -- and the bus floated with nothing said (hunt F15 of 2026-09-12); a KiCad export (`/in_0_` nets) run without the option is the same silence the other way round. E-572 named a bus BASE reused as a plain node, never the other spelling of the bits themselves. Each expanded base now remembers its bits in both spellings, and pass 3 -- every node known -- lists the other-spelling nodes it finds: "the deck also wires in[0], in[1], ... -- the bracket spelling of the same bits, a different node from each, so the bits float. Write them as in_0_ .. in_4_, or set `.option autobus` without `=kicad`", and under the default spelling the mirror text offering `autobus=kicad` (what a KiCad export needs). Only the exact other-spelling names of the expanded bits are looked for; nothing binds differently.

E-628 *src/frontend/inpc_probe.c (the alli pass leaves OSDI lines and subcircuit calls to inp_probe_osdi, run after the pre_ commands and the autobus option: a shorthand line written out against its model's ports, one source per terminal under the model's terminal name; an X line keeps its base token and gets one source per bit the formal stands for inside, found through nested calls; the splice factored into probe_splice), src/frontend/subckt.c + subckt.h (E-464's model -> ports lookup exported as inp_osdi_port_widths, inp_model_of_line, inp_get_autobus), src/frontend/inp.c + inp.h (inp_autobus_of_deck; the second pass called before subcircuit expansion), examples/probebus_examples/ (new, 6 checks)*

`.probe alli` ON A BUS DEVICE -- KiCad's DEFAULT PROBE NO LONGER FLOATS A BUS PORT. The pass counted the node tokens of `N2 /mid /out vares` (two 4-bit bus ports in autobus shorthand), took the device for a two-terminal one and spliced its measuring source into the second token; autobus then expanded the invented base into four bits the source never touched, the bits floated, and every output read 0 V with ten warnings that never named the probe (hunt F11

of 2026-09-12); a subcircuit call whose formal is a bus base inside failed the same way through the X line. It could not be fixed where it stood: the pass runs during the deck read, before pre_osdi loads the objects. OSDI lines and subcircuit calls are now probed in a second pass once the modules are registered and `.option autobus` is resolved: a shorthand line is written out against its model's ports, in the deck's spelling, and every terminal current is a vector by the model's terminal (`n2:n_3_#branch` = -4 mA); an X line keeps its base token -- the subcircuit expands it -- and gets one source per bit the formal stands for inside, through nested calls (`x1:out_3_#branch`). A two-terminal scalar device or a two-formal plain subcircuit keeps `<inst>#branch`.

E-629 *src/frontend/com_dl.c (com_osdi -- pre_osdi/osdi, -f and -va included -- and com_codemodel unquote each file name with cp_unquote before opening it), examples/osdireload_examples/ (6 -> 10 checks)*

`pre_osdi "a path with spaces"` LOADS -- THE LOADERS UNQUOTE THEIR FILE NAMES. A path with spaces has to be quoted, and the double-quoted spelling -- the one a schematic tool writes -- is the one the command lexer keeps as a single word with its quotes on; com_osdi handed it to the loader as it came, so the file `"dir with space/va res.osdi"`, quotes included, did not exist (`Error opening osdi lib ""dir with space/va res.osdi""`) and only single quotes loaded; every KiCad project under a directory with a space met it at its pre_osdi line (hunt F12 of 2026-09-12). codemodel read its argument the same way. Both unquote now, before the -f reload, the -va compile or the plain load. The backslash spelling is the lexer's word splitting, before any command sees the words: quotes, of either kind, are the way.

E-630 *src/frontend/parse.c (PPlex remembers the token it returned last, reset per parse; a / followed by a name character where an operand is expected -- the start, after (, [or an operator -- begins a vector name, read on through further / to the next special), examples/track_examples/ (40 -> 45 checks)*

A **`/name`** IS A VECTOR NAME WHEREVER AN OPERAND IS EXPECTED. A schematic tool spells every net /name; E-611's quoting pass made `v(/out)` parse in print and the commands that evaluate their own text, but `vdb(/out)` and its siblings are `defined` functions whose argument is parsed before the substitution, `ph(/out)` takes a vector expression, `track` used the plain parser and a bare `/out` is an expression on its own -- in every one the lexer met / and returned the division operator with nothing to its left (hunt F13 of 2026-09-12); the workarounds were `vdb("/out")` and `db(v(/out))`, and track had none. The lexer now reads `/name` as a name where an operand is expected (nothing stands to its left, so a division it cannot be), a further / inside the name included (`/sheet1/out`); after an operand a / is the division it always was. track, montecarlo -track, vdb/vm/vp/vr/vi/ph(/out), a bare /out, `-/out`, `(/out)/2`, `let y = /out + v(/in)` all follow.

E-631 *src/frontend/inpcom.c (inp_expand_macro_in_str leaves a call whose argument count does not match the .func's to the built-in of that name; inp_get_func_from_line takes whether the card is ngspice's own -- the PSpice compatibility set -- and does not blame "your definition" for it; the user warning says what the definition covers), examples/silentaccept_examples/ (42 -> 44 checks)*

A .func SHADOWS A BUILT-IN ONLY FOR CALLS WITH ITS ARGUMENT COUNT -- limit() WORKS UNDER KiCad's COMPATIBILITY MODE. `set ngbehavior=kiltpsa` (what KiCad sets) turns on the PSpice compatibility pass, which inserts `.func limit(x, a, b)` and five more at the top of the deck; a .func replaces the built-in of its name by textual expansion, and a call with the wrong count was fatal -- so the random `limit(nom, avar)`, ngspice's own two-argument built-in, died with "parameter mismatch ... fatal error in ngspice, exit(1)", after three warnings that said "your definition" of .func cards the user never wrote (hunt F14 of 2026-09-12). A call whose count differs from the .func's is now left to the built-in of that name: limit(1000, 100) is the coin and limit(1500, 100, 1200) PSpice's clamp in one deck; a mismatched call of a name that is no built-in is still the error. The compatibility set's own cards (linesource internal) draw no warning; a user's .func over a built-in still does, saying "every call with its argument count".

E-632 *src/frontend/spiceif.c + spiceif.h (if_refuse_stale: E-560's message with the asking command's*

name; if_run returns 3, a run that did not start, where the refusal returned 1, an interrupt), src/frontend/com_sweep.c (montecarlo, highsigma, sweep and wcd refuse up front through it), examples/osdireload_examples/ (10 -> 13 checks)

THE LOOP COMMANDS STOP ON A CIRCUIT BUILT AGAINST A RELOADED OBJECT. After `osdi -f` a circuit built against the previous object is refused by if_run (E-560) -- a plain op, dc, tran were -- but montecarlo and highsigma ran every sample (the fast path pushing into the stale circuit and running through refusals nothing noticed), sweep reported its points of stale values and wcd turned the refusals into a "zero gradient" (hunt F20 of 2026-09-12): the refusal returned 1, which dosim takes for an interrupt, and sim_status stayed clean. The refusal is now a run that did not start (3, sim_status set), so every loop that asks after its inner run sees a failure, and the four loop commands refuse up front with the same message and their own name; after `reset` they run on the rebuilt circuit.

E-633 *src/osdi/osdi.h (OsdiStatParam.derived), osdiregistry.c (reads the optional OSDI_STAT_PARAM_DERIVED table), osdisetup.c (osdimc_apply_type leaves a derived never-given parameter to the setup's tail; osdimc_apply_derived clears its given flag, runs the model's setup once more so the default re-resolves from the trial's other draws, and puts the draw on that value; the option-off restore leaves it stale to re-derive), examples/osdimc_examples/ (44 -> 49 checks, smcderiv.va new)*

A DRAWN PARAMETER WHOSE DEFAULT DEPENDS ON ANOTHER DRAWN PARAMETER FOLLOWS IT. `(* std *) r3 = 2*r` drew around a nominal captured once (2000 + d, whatever r drew) while a plain `r2 = 2*r` beside it followed the drawn r at every setup: a written parameter counts as given, so the setup never recomputed its default (hunt F21 of 2026-09-12). The compiler now says which statistical parameters have a derived default; those the deck never gave are drawn in the setup's tail after the model's setup is run once more with the trial's other draws in place, so their default re-resolves (2 * r_drawn) and the draw goes on top; OSDItemp then re-runs the setups with every value final, so a hoisted geff = 1/(r+r3) sees it. A given parameter keeps its given nominal; `unset osdimc` re-derives instead of restoring the last trial's 2r. Plain and reset paths agree.

E-634 *src/osdi/osdisetup.c (the seed's 32-bit range; a recentre outside the declared range warned), src/frontend/inp.c (controlswait under -b noted; an option card given twice warned), src/frontend/mcsave.c (the fixed columns refused; the no-row message; the txt cell empty; wcd's rows labelled), src/frontend/com_sweep.c (-writemc fixed columns; highsigma's nothing-inflated note; the setseed set aside), src/spicelib/parser/inp2n.c (autoadapt's warnings; a bus wider than its port), inpdpar.c (a model parameter on the instance line), src/frontend/outitf.c + parse.c (the #flow vector typed current, its name read), src/maths/misc/randnumb.c (mc_wcd_active), examples/hunt12diag_examples/ (new, 20 checks); the statistics guide 7.3/7.4, the handbook 3.2*

THE DIAGNOSTIC FINDINGS OF THE 2026-09-12 HUNT, D1-D24. Twenty-two places where the simulator or the compiler did something defensible and said nothing, or said the wrong thing; nothing binds, draws or solves differently. mcseed is a 32-bit pattern, 0 .. 4294967295, and a value outside it is refused and said (it used to be called "not an integer" and folded onto 2147483647, a negative one taken in silence); `set controlswait` under `ngspice -b` says there is no host run to wait for; `writemc trial=/analysis=/status=` are refused as the row's fixed columns; highsigma says when nothing could be inflated; an .option given twice is warned and automc_save beside savemc likewise; writemc after a row-less run names the reason; autoadapt without adapter= is a warning that the option is ignored; a model parameter on the instance line is named as such; `altermod rm r=0` onto (0:inf) is warned at the write; the OSDI flow vector is typed current and its name parses; the txt cell is empty; a bus wider than its port is named; a setseed set aside by a loop's default seed is said; wcd's own runs are labelled `op (wcd probe)`. D15-D18 and D20 are documented (resume of a completed run is a new trial; rows per sweep path; a host loading twice; a derived random .param recorded twice; `set mcseed` after the card). D24 has held since E-622.

E-643 *src/spicelib/parser/inpeval.c (INPdecimal; INPevaluate, INPevaluate2 and the RKM parsers*

through it), src/frontend/parser/numparse.c (ft_numparse), src/frontend/numparam/xpressn.c (suffixed literals), src/osdi/osdiinit.c (value sets in a range text; only the paramset's own parameters judged and counted; the tie names what selects), src/osdi/osdiregistry.c + src/include/ngspice/osdiitf. h (OSDI_PARAMSET_OWN), examples/cardbound_examples/ (new), examples/hbconv_examples/ + inputguard_examples/ (expectations)

A NETLIST NUMBER IS THE DOUBLE ITS TEXT NAMES (IHP HUNT B2), AND A PARAMSET IS SELECTED BY ITS OWN PARAMETERS (B1). Every ngspice number parser computed mantissa x 10^exponent and rounded twice: `1.2` was 1.2000000000000002, `0.96u` 9.600000000000001e-07, `10e-6` 9.999999999999999e-06. The compiler reads the same spelling correctly, so a card value AT a declared bound was refused by the model's own range check -- `vmax=1.2` against `from [0: 1.2]`: 'value 1.2 is out of bounds' -- and the paramset selection, re-reading a bound's text, judged a member's own default outside its own range. The digits and the whole power of ten now reach strtod as one number, in the netlist parser and its four copies, in ft_numparse (`alter`, `let`, `print`) and in numparam's suffixed literals; `0e400` is zero, not refused. The selection also reads the `{a, b}` value sets the compiler writes for every single-value constraint (`exclude {0}` used to exclude everything -- r3_cmc's `type` disqualified each IHP resistor paramset), judges and counts only the paramset's own parameters as LRM 6.4.2's last sentence says (the compiler exports which, OSDI_ PARAMSET_OWN; a member binding more module parameters no longer wins a tie by having fewer 'un-overridden' ones), and a tie names the ranges that would break it: 'rsil takes mm_ok from [0:0]; rsil__2 takes mm_ok from [1:1]'. VERIFIED: cardbound 10/10 per solver (7 fail on the E-642 binaries), the IHP resistor paramsets select by mm_ok with their real ranges, hbconv's loosened bound moved to 1e-7 (the deck's residual floor is 3.3e-8 with correctly rounded inputs), sweep 516/516.

E-644 *src/osdi/osdiinit.c (osdi_select_paramset_member with an instance line's parameters; osdi_ paramset_family_head/_of; the collision warning skips a bound parameter; no bare i alias over a model's own i), src/spicelib/parser/inp2n.c (per-instance selection: the first n line binds the card, a later one needing another member gets a clone card), src/spicelib/parser/{inpgmod,inpmkmod}.c + include/ngspice/inpdefs.h (INPmodOsdiSel), include/ngspice/osdiitf.h, examples/paramsetinst_ examples/ (new)*

AN INSTANCE SELECTS ITS PARAMSET MEMBER, AND SETS THE PARAMSET'S OWN PARAMETERS (IHP HUNT C1); A BOUND PARAMETER'S CASE-TWIN AND A MODEL'S OWN **`i`** NO LONGER WARN (C2). With the compiler half making a paramset's own parameters instance parameters, `n1 a b rsil l=0.5u w=0.5u mm_ok=0` is accepted -- what every SPICE device library writes, refused before as 'a model parameter of this device' -- and LRM 6.4.2's selection runs per instance: the instance's values join the card's (an instance value replaces the card's for the same name; `m=`, `temp=` and other names no member declares take no part), the first n line binds the card to its member, and an instance that needs another member gets a clone of the card, `<card>.<member>`, made once and shared, with a note. The E-335 case-collision warning no longer fires for a parameter the paramset bound (PARA_FLAG_FIXED: PSP's SWSOA under the paramset's swsoa, sp_resistor's r alias under the paramset's R) -- nothing can be set through it -- nor for the loader's bare `i` alias over a model's own `i`, which is no longer synthesised then. VERIFIED: paramsetinst 14/14 per solver (7 fail on the E-643 binaries), the IHP resistors and PSP nmos run from one card per family with the geometry on the instance lines and give the gnucap values, sweep 517/517.

E-653 *src/frontend/commands.c, src/frontend/com_sweep.c, examples/exitcmd_examples/*

`exit` IS A SECOND NAME FOR **`quit`**. `exit` was no ngspice command: in a `.control` block it printed ”exit: no such command available in ngspice” and the block went on with its next line, in batch mode with everything after it. An `exit` entry now follows `quit` in the ngspice and the nutmeg command tables, bound to the same `com_quit`, so it takes what `quit` takes: an integer is the process's exit status (`exit 3`), `noask` skips the `askquit` question, and the shared library

returns through `controlled_exit(1000 + code)`. `oldhelp exit` and command completion come from the table entry. `sweep -analysis exit` is refused up front like `-analysis quit` (E-341). VERIFIED: exitcmd 10/10 per solver (9 fail on the E-652 binary), sweepanalysis/carddefault/probeblock unchanged, sweep 526/526.

E-654 *src/osdi/osdisetup.c (osdimc_corner_read/check/run, osdimc_apply_corner_type, osdimc_corner_restore, osdimc_cornered; the snapshot), src/osdi/osdiregistry.c, src/osdi/osdi.h, src/include/ngspice/osdiitf.h, src/frontend/spiceif.c, src/frontend/mcsave.c, examples/vacorner_examples/*

`.option corner=<name>` SELECTS THE PROCESS CORNER A VERILOG-A MODEL DECLARES. The corner entries a model exports (E-654, compiler side) are applied on every run-class command through the ordinary setter, the first run included (the pending path the draws use: nominals are resolvable only after one setup pass). A cornered parameter does not draw under `.option osdimc` -- the corner pins the process coordinate, mismatch on the other parameters goes on, a wcd/highsigma walk coordinate on it is consumed and ignored. A parameter naming other corners only sits at nominal; a model type without the name runs at nominal, said once; a name no loaded model declares refuses the run, naming the declared ones. `tt`/`nom` are the nominal, `set corner=` switches between runs, `unset corner` returns to the deck's option; `altermod` recentres a percentage or sigma corner; `showmod` prints the cornered value; `savemc` records a cornered run; `osdimc_verbose` says each write and how it was formed. VERIFIED: vacorner 18/18 per solver (16 fail on the E-653 binaries), osdimc/savemc/wcd/highsigma/mcfastpath/mcrecord/lhs unchanged, sweep 527/527.

E-655 *src/frontend/com_sweep.c (com_corners), src/frontend/commands.c, src/frontend/typesdef.c, src/frontend/mcsave.c (the corner column), src/osdi/osdisetup.c (OSDImcCornerNames, OSDImcCornerName), examples/cornerscmd_examples/*

THE **`corners`** COMMAND: THE ANALYSIS AT EVERY DECLARED PROCESS CORNER, AND **`-mc N`** FOR A MONTECARLO PER CORNER. `corners [-list c1,c2] [-nonominal] [-analysis cmd] [-output expr ...]` runs the analysis at the nominal `tt` and then at every corner the loaded Verilog-A models declare (E-654), in declaration order, and records each -output's last value into a `corners<n>` plot on a `corner` index scale, the names printed in a table and kept in $corners_names; a failed corner or an unresolved output is nan, an output that never resolves is not recorded, a listed name nobody declares, a destructive analysis and a circuit without corners are refused. `-mc N <montecarlo arguments>` runs a `montecarlo N ...` per corner instead (the corner pins the process parameters, the mismatch draws go on) and records yield, npass, nsamples, nfailed. A `.option savemc` file gains a `corner` column with the first cornered row (a file without one is unchanged; `writemc corner=` refused), and a nominal asked for by name tags its rows `tt`. The `corner` variable is put back afterwards. VERIFIED: cornerscmd 18/18 per solver (18 fail on the E-654 binaries), vacorner/savemc/writemc/plotorder/mcyield/osdimc/sweepanalysis/mcrecord unchanged, sweep 528/528.

E-656 *src/frontend/runcoms.c (the hook in dosim), src/frontend/com_sweep.c (autocorner_wanted, autocorner_run, ac_resample; the sw_running_cmd counter), src/frontend/com_optimize.c, src/frontend/typesdef.c, src/frontend/spiceif.c, examples/autocorner_examples/*

`.option autocorner`: A RUN-CLASS COMMAND AT EVERY DECLARED PROCESS CORNER, FOR DECKS WITHOUT A CONTROL SCRIPT. With the option set, every run-class command (op, tran, ac, dc, the batch-mode run, the shared library's) runs at the nominal `tt` and then at every corner the loaded Verilog-A models declare (E-654), as `set corner=` and the command would. The per-corner plots are kept, named with their corner, so batch .print/.plot cards print each corner (E-602); and for each plot the nominal run made a combined `autocorner<n>` plot is made current: the nominal's vectors under their names, each corner's as `<name>_<corner>` (`v(out_ss)`), resampled onto the nominal's scale (kept as it is, an ac plot's complex frequency included) so one scale serves them all -- the form a schematic host reads. $autocorner_plot/plots/names/n describe the run and a plain run clears them; a corner whose run failed is said and makes

no plot. Inert inside a loop command's own runs (sweep's dc handover, montecarlo, corners, wcd, highsigma, optimize -- each brackets its runs with a counter), for resume, and without a declared corner; the `corner` variable is put back. VERIFIED: autocorner 14/14 per solver (12 fail on the E-653 binaries), cornerscmd/vacorner/savemc/writemc/sweepanalysis/mcyield/multisave/osdimc/optimize/crashfix/plotorder unchanged, sweep 529/529.

E-657 *src/osdi/osdi.h (OsdiCornerParam.gated), src/osdi/osdiregistry.c (OSDI_CORNER_GATED read at load), src/osdi/osdisetup.c (the corner writer's gate, osdimc_say_corner_gated), examples/vacorner_examples/ (five checks added)*

A CORNER ON A **`$param_given`**-TESTED PARAMETER MOVES IT ONLY WHEN THE DECK GAVE IT (HUNT F4). A corner is a write through the setter, and the setter marks the parameter given; a model that derives a default with $param_given then took its "given" branch at the corner's value instead of moving -- a branch flip with no message (`(* corner="ss=2" *)` on `a`, `g = $param_given(a) ? a : 10`: the corner ran 2 where the nominal ran 10). The E-555 gate covered only parameters with statistics, and there it printed the draw's note. The corner writer now gates on the compiler's OSDI_CORNER_GATED bit as well as the statistics record: a tested, never-given parameter is left at its nominal with the corner's note once (give it on the card, or altermod it, for the corner to move it); given on the card, or altermoded, it moves as before; an untested neighbour is untouched; the corners loop and .option autocorner take the same path. An object from an older compiler carries no symbol and keeps the E-654 behaviour for a corner-only parameter. VERIFIED: vacorner 23/23 per solver (3 fail on the E-656 binaries), paramgiven/cornerscmd/autocorner/osdimc/savemc unchanged, sweep 529/529.

E-658 *src/osdi/osdisetup.c (osdimc_capture: the cornered parameters captured at every setup), examples/vacorner_examples/ (three checks added)*

THE FIRST CORNERED RUN AFTER A NOMINAL RUN UNDER **`.option osdimc`** CORNERS EVERY DECLARED PARAMETER (HUNT F1). The nominal table captured the statistical parameters at every setup under osdimc and the cornered ones only while a corner was on, so a nominal run under osdimc left the table populated and not stale with no entry for a corner-only parameter; `set corner=ss` then took OSDImcNewRun's direct path -- the draws, then the corner -- and the corner writer found no entry and returned in silence: the first cornered run had the statistical parameters at their corner and the others (an absolute, a percentage, an instance corner) at nominal, the second run all of them. The same through the corners loop (its ss row nominal, its ff row right), .option autocorner (its first ss pass) and a sweep as the first cornered run (its first point). The cornered parameters are now captured at every setup, corner on or off, as the statistical ones are under osdimc; a second capture is a no-op, savemc enumerates from the compiler's tables so nothing recorded changes, alter/altermod recentre the entry with the corner off too and E-614's stale marking covers it. VERIFIED: vacorner 26/26 per solver (the three new checks fail on the E-656 binaries), cornerscmd/autocorner/osdimc/savemc/sweepanalysis/paramgiven unchanged, sweep 529/529.

E-659 *src/frontend/com_sweep.c (com_corners: the -analysis, -output and -list parsers), examples/cornerscmd_examples/ (three checks added)*

QUOTED WORDS ON THE **`corners`** LINE ARE UNQUOTED (HUNT F2). The lexer keeps double quotes on a word and strips single ones, and every loop command strips the survivors with cp_unquote() -- except corners: `-analysis "dc v1 0 1 0.5"` (the handbook's own spelling) reached the command lookup with its quotes, every corner ran an unknown command and recorded nan; `-output "g=v(out) / v(in)"` made a column `"g` whose expression never resolved; `-list "ss ff"` split into `"ss` and `ff"` and was refused; an unquoted `-analysis tran 1u 3u` stopped at tran and refused 1u as an unknown option. Only the single-quoted form and the -mc forwarding worked. com_corners now follows the collect_until_flag rule of sweep and montecarlo: -analysis collects the words up to the next flag, each unquoted (a negative argument is not a flag; a command past the buffer is refused, not truncated; -analysis followed by a flag is refused), -output and

-list unquote each token; the -analysis before -mc is still forwarded re-quoted. VERIFIED: cornerscmd 21/21 per solver (the three new checks fail on the E-656 binaries), vacorner/autocorner/sweepanalysis/mcyield unchanged, sweep 529/529.

E-660 *src/osdi/osdicallbacks.c (the setup pass's messages held and superseded by the temperature pass's; osdi_display_setup_failed), src/osdi/osdidefs.h, src/osdi/osdisetup.c (the release on a failed setup), examples/display_examples/ (display_setup.va, six checks)*

A HOISTED DISPLAY OR SEVERITY TASK PRINTS ONCE PER ANALYSIS, AFTER THE DRAWS AND THE CORNER (HUNT F16). A $strobe, $info, $warning or $error whose arguments are solution-independent (a constant, a parameter, $temperature) is hoisted by the compiler into the model's setup code, which ngspice runs twice per analysis -- OSDIsetup's setup loops and then OSDItemp, which CKTdoJob runs after CKTsetup in every job -- so each such line printed twice, and the first copy was stale: the draws and the corner are applied at the end of OSDIsetup, so under .option corner=ss a banner printed rsh=100 and then rsh=115 for one operating point. The setup pass's messages are now held and dropped when the temperature pass re-enters (its copies print); released to the output when the setup failed for a card (what the code said before the failure must show) or at the first Newton iteration if no temperature pass came; $fatal and event-gated or analog-initial output are never held. A temperature pass alone (a .dc temp point, an altermod) prints once as before. The hoisting itself stays: the corpus relies on it (BSIM4's 338 parameter-check $strobes, BSIMSOI's 203, bsimbulk's 27 $error are all plain analog-block code), and a solution-dependent task still prints per accepted point. VERIFIED: display 30/30 per solver (five of the six new checks fail on the E-659 binaries), lrmsysio/lrmvoice/carddefault/dropguard/mcpolicy/scanbody/simctrl/osdimc unchanged, sweep 529/529.

E-662 *src/osdi/osdisetup.c (osdimc_corner_is_nominal_name; the declared-name collector leaves the nominal's spellings out), src/frontend/com_sweep.c (corners -list folds duplicates), examples/vacorner_examples/ and examples/cornerscmd_examples/ (a check each)*

THE CORNER LOOPS NEVER VISIT A DECLARED **`tt`**, AND A NAME LISTED TWICE RUNS ONCE (HUNT F3). `.option corner=tt`, `nom` and `nominal` are the nominal by E-654's rule, so a model that declared a corner of one of those names (the compiler accepted it until E-662) had an unreachable entry -- and the corners command's default set, tt first and then every declared name, became `tt tt nom ss nominal`: the nominal run three times under three names, autocorner's set the same. `corners -list TT,SS,ss` ran ss twice: folded to lower case, not folded together. The declared-name collector now leaves the nominal's spellings out (an object from an older compiler is looped over at its real corners: `tt ss`, and the `declared:` list of a refused name says the same), and -list folds a name listed twice after the case and nominal folds, saying so once per duplicate. VERIFIED: vacorner 27/27 and cornerscmd 22/22 per solver (the new check of each fails on the E-661 binaries), autocorner unchanged, sweep 529/529.

E-663 *src/osdi/osdisetup.c (OSDImcCornerPriority, OSDImcOptionSet; osdimc_enabled honours the priority), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c (com_corners standalone/nested, autocorner_run), examples/cornerscmd_examples/ and examples/autocorner_examples/ (checks added)*

A PLAIN CORNER LOOP TAKES PRIORITY OVER **`.option osdimc`**: THE AUTOMATIC MONTE CARLO IS DISABLED FOR THE LOOP, WITH A WARNING (HUNT F7). Under .option osdimc (automc) every corner run of the corners command and of .option autocorner was a fresh trial -- the cornered parameters pinned, the others drawn again -- so the corner-to-corner difference of any output carried a fresh mismatch draw, the tt row was a random sample after any prior run (the cornered parameter itself drawn there), and the table depended on how many runs came before; sweep and wcd hold one trial, the corner loops did not. By the user's rule corners win: a standalone plain corners loop disables the option for the loop with a warning and every corner runs at the nominal of the statistical parameters (the sequence restarts afterwards, as after unset osdimc); .option autocorner does the same for the pass, said once per circuit; corners -mc

N still draws per corner; nested inside a montecarlo sample the loop shares that sample (the E-535 hold) so the outer command measures the corner table's distribution. A single selected corner keeps the option (E-654's corner-plus-mismatch flow). VERIFIED: cornerscmd 24/24 and autocorner 15/15 per solver (the new checks fail on the E-661 binaries), vacorner/savemc/ mcyield unchanged, sweep 529/529.

Enhancement-666 *src/frontend/com_sweep.c (autocorner_run: the raw-file pass, the plot families, ac_ copy_name; com_corners), src/frontend/runcoms.c (raw_open_for_run), src/frontend/outitf.c (fileInit), src/include/ngspice/plot.h (pl_kind), src/frontend/postcoms.c, src/frontend/measure.c, src/ frontend/com_measure2.c, src/frontend/mcsave.c (MCSAVEappendPlot, com_writemc), src/osdi/ osdisetup.c (OSDImcCornerLeave), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.h, src/ frontend/mcsave.h, examples/autocorner_examples/ (six checks added)*

THE **autocorner** FOLLOW-ON GAPS: A RAW-FILE RUN KEEPS EVERY CORNER, **meas** AND **writemc** READ THE COMBINED PLOT, THE COPIES KEEP THEIR ACCESSOR READABLE, THE DEVICES FOLLOW THE CORNER VARIABLE (HUNT F8). Under .option autocorner a raw-file run wrote the file once per corner (the last corner survived) and said "made no plot" per corner; meas tran refused the combined plot as not a tran analysis; writemc had no row for it; the copies of `@rm[rsh]` and v1#branch were named `@rm[rsh]_ss` and v1#branch_ss, which the bare parser cannot read (i(v1_ss), the banner's own example, was not available); and the devices stayed at the last corner after the loop (showmod showed the ff value). Now the corners after the first open the raw file at its end (r+ and a seek, since an append-mode stream would misplace the point-count patch) and the file's Plotname carries the corner, so load reads one plot per corner; a plot gains pl_kind, the analysis a derived plot stands for, and the meas checks read it; writemc on the combined plot evaluates each item on every corner's own plot and puts it on that corner's row (MCSAVEappendPlot); the copy suffix goes on the node or device name (out_ss, v1_ss#branch, `@rm_ss[rsh]`); and OSDImcCornerLeave makes the writes the next run would have made when corners or the pass ends. A failed corner is said as failed. VERIFIED: autocorner 21/21 per solver (9 of 21 fail on the E-661 binaries); cornerscmd, savemc, writemc, vacorner, mcyield, lrmcorner, the meas and raw suites unchanged; sweep 530/530.

Enhancement-667 *src/osdi/osdisetup.c (osdimc_walk_count, osdimc_apply_type, OSDImcWalk-Cornered), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c (com_wcd), examples/wcd_ examples/ (wcdcorner.va, wcdheld.va, eight checks)*

A PARAMETER THE CORNER IN FORCE HOLDS IS NOT A **wcd** DIMENSION: THE COUNT IS OF THE FREE AXES AND THE HELD ONES ARE NAMED (HUNT F9). E-654 kept a cornered parameter in the walk's dimension count and had the applier consume and ignore its coordinate, so under .option osdimc corner=ss with three of four model-declared parameters cornered wcd announced 4 statistical dimensions, printed four MPFP coordinates three of which did nothing, called a metric of the held parameters alone unresponsive to any statistical parameter, and refused a deck whose every parameter was cornered as declaring no Gaussian statistics. A cornered entry is counted apart and takes no coordinate (the derived-default pass already skipped it, and the two agree now); OSDImcWalkCornered names the held ones owner:param; the banner reads 1 statistical dimension (0 netlist .param, 1 model-declared; 3 held by corner ss: mm:r mm:p mm:q), the flat-metric refusal says which are held and not searched, and the nothing-to-search refusal says the corner holds them all. The search's answer is unchanged (the free axis carried it); the mean-shift refinement shifts the free axes alone. VERIFIED: wcd 39/39 per solver (4 of 39 fail on the E-666 binaries); highsigma, osdidist, mcpolicy, vacorner, cornerscmd, silentloss, sweepguard, hunt12diag, agestate, osdireload, loopbar unchanged; sweep 530/530.

Enhancement-668 *src/osdi/osdisetup.c (osdi_oob_corner_note, handle_init_info), src/osdi/osdiinit.c (osdi_member_accepts), src/include/ngspice/osdiitf.h, examples/vacorner_examples/ (four checks added)*

A CORNER THAT MOVES A PARAMETER OUT OF ITS RANGE SAYS SO: THE CORNER, THE

NOMINAL, AND FOR A PARAMSET MEMBER THE FAMILY, THE MEMBER CHOSEN AT THE NOMINAL AND THE SIBLING THAT ACCEPTS THE VALUE (HUNT F10). Two rs paramsets over one module with l from (0:2u] and (2u:inf), a corner ss=+20% on l and an instance giving l=1.9u: E-644 binds the instance to the first member at the nominal, .option corner=ss writes 2.28u on it, and the temperature pass refused it with E-558's line alone -- the parameter, the value and the range, but not the corner that moved it nor the member -- and the corners command's ss row was just failed. The member cannot be re-judged at the corner (a member is a device type of its own, chosen when the card is materialised), so the failure gets a second line: the corner, the nominal it moved the parameter from and how (+20%), and for a paramset member the family, the member the instance was bound to at the nominal (LRM 6.4.2) and every sibling whose range accepts the value, with the way there; a plain module whose corner leaves its own range gets the corner line alone; no note inside the range or without a corner. VERIFIED: vacorner 31/31 per solver (3 of 31 fail on the E-666 binaries); paramsetoverload, cornerscmd, autocorner, hunt17diag, osdimc, hunt2diag, opvarmsg, paramgiven, wcd, lrmcorner unchanged; sweep 530/530.

Enhancement-669 *src/frontend/variable.c (cp_getvar's string read), src/frontend/plotting/graf.c, src/osdi/osdisetup.c (osdimc_corner 256 wide, osdimc_corner_read, osdimc_corner_collect_total, osdimc_corner_declared, osdimc_corner_check, osdimc_corner_say_missing, OSDImcCornerTotal, OSDImcCornerDeclared), src/include/ngspice/osdiitf.h, src/frontend/com_sweep.c (CO_MAXCORNERS 256, com_corners, autocorner_run), examples/vacorner_examples/ (four checks added)*

ANY NUMBER OF DECLARED CORNERS CAN BE SELECTED, THE LOOPS TAKE 255 AND SAY WHEN A CIRCUIT DECLARES MORE, AND A LONG CORNER NAME NO LONGER ABORTS NGSPICE (HUNT F14). A model declaring c0..c69: .option corner=c69 was refused as a corner no loaded model declares (the check collected the names into a 64-slot list and dropped the rest in silence, so the declared: list stopped at c63), and corners/autocorner ran 64 of the 70 without a word. A corner named with 80 characters or more aborted ngspice with a smashed stack: cp_getvar's string read kept rsize characters and copied them with the NUL, one byte past the buffer every caller sizes with sizeof buf. The reader keeps rsize-1 characters (ticchar passes its field's size); a selected corner is looked up directly in every corner table, the declared: lists capped at 256 and ending '... and N more'; CO_MAXCORNERS is 256, the loops take the first 255 beside tt and say 'the circuit declares 300 corners; this loop takes the first 255 (its limit) -- -list names any of them', -list judged by the direct lookup; the corner buffer holds 255 characters and a name the read had to cut is refused as longer than ngspice can select (the compiler refuses one longer than 79). VERIFIED: vacorner 35/35 per solver (the four new checks fail on the E-666 binaries); cornerscmd, autocorner, hunt17diag, lrmcorner, savemc, wcd, mcyield unchanged; sweep 530/530.

Enhancement-670 *src/frontend/variable.c (cp_off_pairs, cp_off_partners, cp_vset), src/frontend/inp.c (inp_opt_pair_prune, the option cards), src/include/ngspice/cpextern.h, examples/autoopts_examples/ (eight checks added), examples/savemc_examples/ (one check reordered)*

THE LATER SPELLING OF AN OPTION PAIR WINS: noautocorner TURNS autocorner OFF, noosdimc TURNS osdimc OFF, ON ONE CARD, ACROSS CARDS AND FROM THE CONTROL BLOCK (HUNT F17). E-572 registered the documented no spellings so they draw no unknown-option warning, but the readers that read the positive through cp_getvar never looked at them: .option autocorner noautocorner ran the loop, set noautocorner after a deck's .option autocorner ran it too, and .option osdimc noosdimc drew on trial 2 -- a known option that turned nothing off, accepted and ignored. A table of the registered pairs (autocorner; osdimc/automc; saveused; autobus; autoadapt; osdicache; dcpath/dcpathall; savemc/automc_save/osdimc_save; reusesetup, each with its no spellings) and cp_off_partners: cp_vset removes the other side before setting (cp_remvar searches the circuit's list too), and inp.c prunes the option cards card by card so the later word or card stands; the readers are untouched and find the positive gone, and the readers that judge the cards themselves (E-454's later-card-wins) already agreed. unset still works; a no

spelling alone is quiet. VERIFIED: autoopts 35/35 per solver (6 of the 8 new checks fail on the E-666 binaries); saveused, osdimc, autoadapt, autobus, dcpath, reusesetup, savemc, autocorner, vacorner, cornerscmd, mcyield unchanged; sweep 530/530.

Enhancement-671 *src/osdi/osdiload.c (absdelay_ensure_timepoints, absdelay_stamp_tran, last_crossing_stamp), examples/tranopdelay_examples/ (new, 11 checks per solver)*

A DELAY SURVIVES THE TRANSIENT OPERATING POINT: absdelay AND A DELAYED transition KEEP THEIR DELAY AFTER ngspice's TRANSIENT-OP FALLBACK (HUNT F1). When plain Newton, gmin stepping and source stepping fail and CKTop reaches optran.c's short frozen-source transient, every absdelay and every transition with a delay in every loaded OSDI model lost its delay for the whole transient that followed: 0 throughout as a variable, the input undelayed as a contribution, 0 under an idt -- whatever forced the rung (an ideal inductor, an idt without ic in any instance, an idt of a delayed signal). The fallback frees the accepted-timepoint list dctran.c had reset, the OSDI stamp re-creates it, OSDIaccept fills ~100 points with the fallback's times and the frozen operating point, and the real transient's MODEINITTRAN found a non-NULL list with the index at the fallback's last point and appended after it -- a non-monotonic timeline whose first microsecond was the operating point, which absdelay_lookup's binary search read. absdelay_ensure_timepoints(ckt, init_tran) now starts the timeline over at index 0 whenever MODEINITTRAN is set (both dctran.c re-raises are at t = 0). Gmin and source stepping alone and uic were never affected. VERIFIED: tranopdelay 11/11 per solver (5 of 11 fail on the E-670 binaries), sweep 531/531.

Enhancement-673 *src/spicelib/analysis/dctrcurv.c (DCTsweepTime, the swept value published before the solve, the abort on a raised $fatal), examples/lrmvoice_examples/ (lrmvoice_fatal.va and three checks, 31 per solver)*

A $fatal IN A DC SWEEP NAMES THE POINT BEING SOLVED, ONCE (HUNT F7). The swept value was published into CKTtime, which the LRM 9.7.3 ”(at sweep value ...)” label reads, only after a point converged; $error, $warning and $info are deferred to the accepted point (E-660) and named the right one, but $fatal is never deferred and named the PREVIOUS point (”over 0.5 (at sweep value 0)”). And the sweep took the E_PANIC a raised $fatal returns for a non-convergence and re-solved the point through CKTop's ladder, whose first rung evaluated every device again, so the line printed twice (CKTop itself stops on the flag, E-378). DCTsweepTime publishes the point before the solve as well, and a raised $fatal ends the sweep with its own abort line (”a Verilog-A device raised $fatal at sweep value 0.5; aborting. This is not a convergence failure”). $abstime is 0 outside a transient (E-434), so no model reads the earlier publish. VERIFIED: lrmvoice 31/31 per solver (2 of the 3 new checks fail on the E-670 binaries); sweep 531/531.

Enhancement-677 *src/osdi/osdiload.c (osdi_op_solve_capture at the MODEINITSMSIG load; OSDIfinalStep evaluates on it under AC and NOISE), examples/finalstep_examples/ (fsop and three checks, 26 per solver)*

THE FINAL-STEP EVALUATION OF AN ac OR noise ANALYSIS RUNS ON THE BIAS POINT -- last_crossing READ 0 AND V(a,b) THE SMALL-SIGNAL RESPONSE (HUNT F6). OSDIfinalStep (E-53) issues one evaluation per instance at CKTrhsOld, and after a frequency sweep that vector holds the small-signal solution at the last frequency: a node read the response (1 V for a unit AC source at a 0.2 V bias) and a last_crossing synthetic node read 0, a valid time, where LRM 4.5.10 requires a negative value before any crossing -- so a model's `last_crossing(...) < 0` test took the wrong branch under ac and noise. E-412 had snapshotted the INSTANCE around that evaluation (the opvars stayed right); the event body, which runs during it, still saw the wrong vector. The OSDI load copies the converged operating point at the MODEINITSMSIG pass every .ac and .noise job issues once before the first frequency, and OSDIfinalStep evaluates on that copy under MODEAC/MODEACNOISE; the instance snapshot stays. VERIFIED: finalstep 26/26 per solver (2 of the 3 new checks fail on the E-670 binaries), opvarac 19/19, tranopdelay 11/11, reusestate 32/32, sweep 531/531.

Enhancement-678 *src/osdi/osdiload.c (the private simparams $osditstep$ and $osdidelta$)*

THE TRANSIENT'S PRINT STEP AND THE STEP UNDER WAY ARE SERVED TO THE COMPILER'S idt RESET REALISATION (HUNT F9). Two private, namespaced simparam entries (the … convention E-215 uses for plusargs; not LRM names, and not the timestep/maxstep the table refuses as user-facing ones -- no model spells them): CKTstep and CKTdelta during a transient, 0 outside one. The compiler's realisation of idt(x, ic, assert) sizes its reset decay from them -- tau = tstep/1000, the gain capped at 2/h so the trapezoidal rule is deadbeat at the onset -- instead of a fixed 10 us that held a microsecond-scale reset at 1.32 instead of 0.5. reportguard's simparam drift check skips the private namespace. VERIFIED: idtassert 17/17 per solver (the 8 new checks fail on the E-670 binaries); reportguard 31/31; sweep 531/531.

Enhancement-679 *src/spicelib/analysis/dctran.c (the divergence guard at the accepted point, DCPATH_DIVERGE_V), src/maths/ni/niiter.c (the singular row recorded as the trouble node), examples/dcpath_examples/ (vdelay.va, vdelay2.va and five checks, 57 per solver)*

A NODE HELD ONLY BY THE DC-PATH gmin THAT DIVERGES IN THE TRANSIENT STOPS THE RUN AND SAYS WHY; A SINGULAR TRANSIENT NAMES ITS NODE (HUNT F8). A node no conductance or capacitance reaches is held by the installed gmin in every mode (E-575, E-595), by design; a delayed conductance stamps nothing, its present current being its past voltage over R, so two in series make the node's own past its present current and each delay multiplies the voltage by 1/(gmin*R) = 1e9: -1e18, 1e27, 1e45, 5.6e62 V over 73 accepted points, every one ”converged”, exit 0, no word after the setup warning. At each accepted point the always-held list is checked and a node past 1e15 V (a kiloampere into a node with no path) ends the run: ”TRAN: time = 2.5e-06: the solution is diverging -- node 'mid' has reached -1e+18 V”, with the cause and the remedy. The hierarchical spelling (the internal node carries the instance's '#' and is left out of the walk) was singular and its ”Timestep too small” line said ”cause unrecorded” after six ”check node” warnings: NIiter records the singular row as the trouble node, Sparse and KLU, and the line reads ”trouble with node n1#mid”. VERIFIED: dcpath 57/57 per solver (4 of the 5 new checks fail on the E-670 binaries); sweep 531/531.

Enhancement-680 *src/frontend/outitf.c (InterpFileAdd, InterpPlotAdd: a special vector read by its type), src/frontend/spiceif.c (if_is_option: interp), examples/opvar_examples/ (four checks, 15 per solver)*

AN INTEGER OPERATING-POINT VARIABLE SURVIVES .option interp (HUNT F10). The two interpolation routines the option switches in read every ”special” vector -- an opvar, an instance parameter, asked of the device per point as an IFvalue union -- through the union's double whatever its type, so an INTEGER opvar's four bytes sat under the previous double's upper bytes: a constant 3 printed as 1.48e-323 (the denormal whose bits are 3) and, with the previous column's value above them, as that value with a few low bits changed, which looked like a misaligned vector; the file path of a batch .print and the plot path of a control-block tran alike, while the ordinary paths had recorded integers as reals since E-32. The special is read once by its type in both routines; the integer then interpolates like any other vector. Taken along: `interp` joins the deck reader's known option words (E-438's list), so `.option interp` no longer draws ”unknown option ... ignored” in the run it takes effect in. VERIFIED: opvar 15/15 per solver (3 of the 4 new checks fail on the E-670 binaries), optknown unchanged; sweep 531/531.

Enhancement-681 *src/spicelib/analysis/cktmkvol.c (CKTmkSignal: the adoption branch reports a parse-time node a device line named), examples/internalnode_examples/ (five checks, 16 per solver)*

A NETLIST NODE SPELLED LIKE A DEVICE'S INTERNAL NODE IS REPORTED WHEN THE TWO BECOME ONE (HUNT N1). A device builds its internal nodes at setup under `<instance>#<node>` (n1#mid, q1#base, and a source's vin#branch), and E-608 made CKTmkSignal ADOPT a parse-time node of that name so that a .tf/.pz card may name the node ahead of the device. A DEVICE line that named the node landed in the same branch in silence: `vx n1#mid 0 dc 5` drove the model's internal node (v(out) = 2.5 from the forced 5 V, 11.5 mA through the

source), a BJT's q1#base drawn to 3 V put 1.2e35 A through the source, a resistor on vin#branch was wired into the source's current equation -- and nothing said so, though the spelling is the one every print, .save and .ic uses. The adoption now says so once when the node carries E-429's devRef mark (set for every node a device line names, clear for a node an analysis card invented; a .ic/.nodeset makes no node at all): ”node 'n1#mid' is named on a device line and is also the internal node 'mid' of instance n1; the two are one node -- whatever the netlist wires to 'n1#mid' is wired inside that model. Rename the netlist node unless that was intended”, with a branch-current wording for a `#branch` name; E-608's `adopted` mark keeps every later setup quiet (op; op; tran; sens prints it once). The connection itself is kept, so a deliberate reach into an internal node still works. VERIFIED: internalnode 16/16 per solver (3 of the 5 new checks fail on the E-680 binaries: no warning was printed), sensrestore/sensstate/senscplx/osdisens unchanged; sweep 531/531.

Enhancement-682 *src/spicelib/devices/isrc/isrc.c (r and td declared IP: settable, not askable), examples/sensrestore_examples/ (five checks, 21 per solver)*

sens OVER A CURRENT SOURCE PRINTS NO GET ERROR LINES (HUNT N2). Every sens in a deck with a current source printed ”GET ERROR: Isource:I:i1 -> param r (27)” and ”... td (28)” on stdout, three times per parameter per source per sweep (twenty-four lines for two sources and two sweeps). E-447 had declared the voltage-source-only pwl options r and td on the current source so the card could refuse them by name instead of ”unknown parameter”, but declared them IOP -- askable -- with no ask case behind them (nothing is ever stored, the card refuses the value), and the sensitivity walk admits every IF_SET

Enhancement-683 *src/osdi/osdiload.c (OSDIfinalStep: the bias-point capture used whenever valid, the E-412 snapshot only when there is none; osdi_job_name_flags shared with OSDIload), src/spicelib/analysis/tfanal.c, pzan.c, span.c, distoan.c (the final-step call), cktsens.c (the base operating point kept and put back before the call), examples/finalstep_examples/ (a counter module and nine checks, 35 per solver)*

@(final_step) FIRES AT THE END OF EVERY ANALYSIS, AND WHAT IT ASSIGNS STAYS (HUNT F2). LRM 5.10.2 places final_step on the last step of ANY analysis; OSDIfinalStep was called from acan.c, dcop.c, dctran.c, dctrcurv.c and noisean.c and nowhere else, so pz, tf, sens, disto and sp ended without the event, and under ac and noise it fired but E-412's snapshot -- written when the evaluation ran on the last frequency's small-signal solution -- restored the instance afterwards, so a counter assigned in the block read 0 where an op left 1. Since E-677 that evaluation runs at the captured bias point, so its results are the operating point's own plus what the event body assigned: the capture now carries a validity mark (set at the MODEINITSMSIG load, cleared by any DC or transient evaluation) and is used whenever valid, whatever CKTmode says at the end (.tf/.pz end in MODEDCOP, .sp in MODESP, .disto in MODEAC), and the snapshot is kept only for a small-signal evaluation with no bias point. tfanal.c, pzan.c, span.c and distoan.c call the final step once their result is out; cktsens.c keeps the base operating point from its CKTop (the AC sweep re-runs CKTsetup per frequency and NIacIter solves into CKTrhsOld), puts it back after the E-440 restore and calls it there, once. E-53's job consultation is one helper now, shared by OSDIload and OSDIfinalStep, so a job's final step carries the name its operating point carried. An op after an ac, at a new bias, sees the new bias. VERIFIED: finalstep 35/35 per solver (8 of the 9 new checks fail on the E-680 binaries), opvarac 19/19, lrmevents/analyses/evtnoise and the sens suites unchanged; sweep 531/531.

Enhancement-684 *src/osdi/osdiload.c (osdi_job_name_flags: SP, PZ, DISTO and an AC SENS name ”ac”; osdi_analysis_name reads the same helper), examples/analyses_examples/ (a module and nine checks, 29 per solver)*

THE OPERATING POINTS OF sp, pz, disto AND AN AC sens ANSWER analysis(”ac”) (HUNT F3). E-53 made the operating point of an ac job answer ”ac” -- the phase belongs to the owning analysis -- by consulting the job's name for AC and NOISE; the other small-signal jobs were not

on the list, so the one evaluation of the analog block they make said ”dc” (analysis(”ac”) = 0, analysis(”dc”) = 1, analysis_name ”dc”, initial_step(”ac”) silent), and a model that selects its small-signal formulation under analysis(”ac”) took the DC path in an S-parameter sweep, a pole-zero, a distortion and an AC sensitivity analysis. The helper E-683 made of that consultation, shared by OSDIload and OSDIfinalStep, names SP, PZ, DISTO and a SENS whose step_type is not SENS_DC ”ac”, and osdi_analysis_name reads it too, so the string, the flags and the qualified step events cannot disagree; the E-53 rule that drops the ”dc” bit under an owning name applies unchanged (static stays 1). tf and a DC sens are computed at zero frequency and stay ”dc”; the compiler accepts no other name. VERIFIED: analyses 29/29 per solver (6 of the 9 new checks fail on the E-680 binaries: four on the name, two on E-683's final step), finalstep/lrmevents/evtnoise/opvarac, the analysis_name suites and the sens suites unchanged; sweep 531/531.

Enhancement-685 *src/spicelib/analysis/cktsens.c (the perturbation loop: a byte snapshot of the instance being perturbed beside E-440's model snapshot, put back at both write-back sites), examples/sensrestore_examples/ (two OSDI modules and eight checks, 29 per solver)*

sens LEAVES EVERY INSTANCE AS IT FOUND IT (HUNT F1). A sensitivity analysis writes each perturbed instance parameter back through the device's own setter, which marks it GIVEN, and nothing un-marks it; E-440 restores the model structs byte for byte but the instance side was left as it was, and for an instance a given flag changes the circuit for the rest of the session: a built-in resistor got tce=0 given and lost its tc1 temperature dependence (1.00 V at 60 degC where a fresh deck gives 1.33, even after alter r2 temp=60), an OSDI instance got temp given at the circuit temperature without its dtemp and was pinned there (set temp, .option temp, dc temp and alter dtemp no longer reached it -- ”Instance temperature specified, dtemp ignored”), its $param_given() rules flipped (a derive-from-geometry-if-given resistor read 2 kOhm), an AC sweep left the resistor's ac alias given so alter r= followed by ac reported the old resistance and a second AC sens reported every resistor as insensitive, and the noise after a sens was the 27 degC resistor's; show and the @-vectors printed the original values throughout. The loop now snapshots the instance struct (DEVinstSize bytes: for an OSDI device the instance data and its given-flag table) before the perturbed value is set and puts it back after the original is written back, at both restore sites; the temperature update and the OSDI collapse check then run on the restored instance. VERIFIED: sensrestore 29/29 per solver (7 of the 8 new checks fail on the E-680 binaries), sensstate/senscplx/osdisens/analyses/finalstep unchanged; sweep 531/531.

Enhancement-687 *src/frontend/device.c (showmod: one block of instance defaults per card of the group), src/frontend/spiceif.c (altermod_instance_default: m refused with the card's reason), examples/carddefault_examples/ (four checks, 19 per solver)*

showmod LISTS EVERY CARD OF A GROUP, AND altermod m= IS REFUSED AS THE CARD REFUSES IT (HUNT F5). The hunt read a card's temp=60 and dtemp=10 as honoured but hidden from showmod while its _mfactor=2 was listed; the card path records all three (E-599's defaults list) and a single card lists them -- the probe was `showmod mt1 mt2 mt4`, and the listing printed the FIRST card's defaults only under the group's heading. E-599's listing is now called once per card through the group iterator, headed ”instance defaults on card :” for a group of several and ”instance defaults on this card:” (the wording paramsetinst pins) for one; a card without defaults gets no block. E-426 keeps m on a .model card ignored, audibly, so that a stray m= cannot silently multiply a deck; E-599's altermod path took `altermod mres m=3` and applied it to every follower -- the same thing through the other door. It is refused in the card's words (set it on the instance, or _mfactor on the card and altermod ... _mfactor=); _mfactor, temp, dtemp and dt keep working on both paths. VERIFIED: carddefault 19/19 per solver (2 of the 4 new checks fail on the E-680 binaries), paramsetinst unchanged; sweep 531/531.

Enhancement-688 *src/osdi/osdisetup.c (OSDIcollapsedNode: the node an internal node was collapsed into, from the instance's node mapping), src/include/ngspice/osdiitf.h, src/spicelib/analysis/cktsetnp.c (the deferred .ic/.nodeset placed on that node, with a Note), src/frontend/outitf.c (the save warning*

names it), examples/internalnode_examples/ (three checks, 19 per solver)

A .nodeset/.ic ON A COLLAPSED INTERNAL NODE IS APPLIED TO THE NODE IT COLLAPSED INTO, AND A .save OF ONE NAMES THAT NODE (HUNT F6). A model that collapses an internal node (`V(a, ai) <+ 0` with rs = 0) builds no node n1#ai at setup: ai IS a. E-608 defers a .ic/.nodeset on # to setup and places it once the device has built the node; a node the device did not build was refused as "Nodeset on non-existent node - n1#ai (n1 has no internal node 'ai')" -- the wrong reason -- and .save v(n1#ai) said "nothing of that name" with no pointer to v(a). OSDIcollapsedNode finds the instance, looks the suffix up among the descriptor's internal nodes and returns the global number of the node it was collapsed into (0 for ground or an unconnected node) from the instance's node mapping; the deferred entry is applied to that node at every setup with a Note once ("the model collapses n1's internal node 'ai' into 'a', so it is applied to 'a'"), a node collapsed into ground is reported and ignored, a name that is no internal node keeps the old refusal, and the save warning says "the model collapses n1's internal node 'ai' into 'a', so no such vector is produced; save v(a) instead". VERIFIED: internalnode 19/19 per solver (all 3 new checks fail on the E-680 binaries); sweep 531/531.

Enhancement-689 *src/osdi/osdiload.c (OSDIuicSeed: the transient's start vector seeded from the analysis("ic") potential contributions after the one load uic makes; the uic evaluation's flags kept for it), src/maths/ni/niiter.c (the hook after that load), src/include/ngspice/osdiitf.h, examples/analyses_examples/ (two modules, seven checks, 36 per solver)*

A MODEL'S analysis("ic") INITIAL CONDITION TAKES EFFECT UNDER tran ... uic (HUNT F7). LRM 4.6.1's idiom -- if (analysis("ic")) V(p,n) <+ ic; else I(p,n) <+ ddt(c*V(p,n)); -- applies its initial condition in the analysis that precedes a transient. With uic ngspice skips the operating point and loads every device once at the start vector so the built-in capacitor can seed its own state; the loader reported that evaluation as the ic analysis, the model took its branch, and the equation it stamped went into a matrix nothing solved: the parameter was dead (v(a) started at 3e-5 instead of 0.5) and only a .ic on the node worked. OSDIuicSeed now runs right after that load: every instance is evaluated with and without the flag, a potential row whose residual differs between the two is an initial condition, and it is met by the minimum-norm change of the free node voltages it involves (ground and the deck's .ic nodes fixed; the rows projected one at a time and swept to convergence, the circuit reloaded and the rows checked again), the capacitor's CAPgetic re-run so a built-in capacitor on a seeded node follows, one Note naming the seeded nodes, a row whose nodes are all fixed reported as not applied. An unconditional V(p,n) <+ vdc is identical with and without the flag and is left to the first step, as the built-in source is. VERIFIED: v(a) at the first point 0.500015, the same as the built-in ic=0.5 capacitor beside it (was 3.0e-5); a floating branch split into +0.25/-0.25 with the difference held; .ic v(a)=0.3 wins with the warning; the unconditional source and the run without uic unchanged; 5 of the 7 new checks fail on the E-680 binaries; the Newton-phase and uic suites unchanged; full sweep 531/531.

Enhancement-690 *src/spicelib/parser/inp2dot.c (inp_analysis_node: a declared OSDI internal node entered before the first setup, a collapsed one resolved after it, the hint for a built-in device's), src/spicelib/parser/inppas3.c (INPinternalNodeName exported), src/include/ngspice/inpdefs.h, src/osdi/osdisetup.c (OSDIdeclaredInternalNode; the analysis-invented twin of a collapsed node shorted to the node it collapsed into), src/include/ngspice/osdiitf.h, src/spicelib/analysis/cktsens.c and pzan.c (a phantom output refused), examples/internalnode_examples/ (seven checks, 26 per solver)*

AN INTERNAL NODE CAN BE THE OUTPUT OF sens, pz, tf AND noise TYPED AS THE FIRST ANALYSIS OF A SESSION, AND A COLLAPSED ONE IS BOUND TO THE NODE IT COLLAPSED INTO (HUNT F8). A device builds its internal nodes at setup and IFnewUid enters them in the parser's table then, so sens v(n1#mid), pz ... n1#mid ..., tf v(n1#mid) vin and noise v(n1#mid) ... ran after an op but, typed as the first analysis of a session, were refused by E-426's rule for a command ("no such node: n1#mid", the analysis aborted). Before the first setup a name whose instance is an OSDI device and whose suffix the module declares is now entered as a parse-

time node E-608's adoption makes the device's own; an undeclared suffix is refused with nothing created (a phantom would float for the rest of the session); a built-in device's node, declared nowhere the parser can see, keeps the refusal and the message says to run an analysis first or put the card in the deck. After a setup a collapsed internal node resolves to the node it collapsed into, with a Note (E-688's rule for .ic); at setup a parse-time node named after a collapsed internal node that only an analysis card or command referred to is shorted to that node by one of E-532's synthetic 0 V sources, said once, so the deck's .tf v(n1#ai) no longer floats and v(n1#ai) reads the merged node. sens and pz refuse a phantom output as tf and noise have since E-429 -- a mistyped .sens card printed a table of -0.0. VERIFIED: the four analyses on n1#mid as the first commands give the numbers they give after an op; a mistyped suffix as the first command is refused and the next command on the real node runs; the BJT's q1#base keeps the refusal with the hint and runs after an op; a collapsed n1#ai as the first command's tf output measures at 'a' (1000 ohm) with no floating-node warning, and typed after an op is resolved to 'a'; a mistyped .sens and .pz card are refused; all 7 new checks fail on the E-680 binaries; E-608's suites unchanged; full sweep 531/531.

Enhancement-691 *src/frontend/inpc_probe.c (the explicit i(), i(,k) and i(,) probes on an OSDI instance kept for inp_probe_osdi and spliced there with the model's terminal names; probe_splice_term, probe_osdi_line_terms), examples/probebus_examples/ (a module, five checks, 11 per solver)*

.probe i(n3) ON A MULTI-TERMINAL OSDI DEVICE NAMES EACH TERMINAL CURRENT AFTER THE MODEL'S TERMINAL (HUNT F9). The probe pass runs during the deck read, before pre_osdi has registered the modules, so an n line's terminals were all labelled nn: .probe i(n3) on a four-terminal device inserted four distinct sources and saved four vectors that all read n3: nn#branch, i(n3,2) gave one more, and i(n3,p) was refused ("Node p is not available for device n3"). E-628 had moved alli's OSDI lines to a second pass that runs once the modules are registered; the explicit forms had stayed behind. They are kept by their text now and spliced in that pass: i(n3) gives n3:p#branch, n3:n#branch, n3:cp#branch, n3:cn#branch; i(n3,2) the second; i(n3,p) and i(n3,cp) by name, case-insensitive; a bus line in autobus shorthand is written out against its ports first (i(n2,6) is n2:n_1_#branch); a two-terminal device keeps n3#branch and an unresolved model the generic nn. VERIFIED: the four currents 1 mA, -1 mA, 0, 0 under the terminal names with no nn; i(n3,2) alone; i(n3,p) and i(n3,cp) where the name was refused; the written-out bus line's -(k+1) mA per bit; all 5 new checks fail on the E-680 binaries; probeblock, probeshort and savecur unchanged; full sweep 531/531.

Enhancement-692 *src/osdi/osdisetup.c (the deck nodes a collapsed internal node may be bound to collected once per setup call and matched to the instances by name; one node fetched per match), src/osdi/osdiload.c (OSDIuicSeed: only the instance's residual entries cleared), examples/internalnode_examples/ (two checks, 28 per solver), examples/analyses_examples/ (one check, 37 per solver)*

THE SETUP NO LONGER WALKS THE NODE LIST FOR EVERY INTERNAL NODE OF EVERY OSDI INSTANCE. Reported by the user on version12 right after the E-691 sync: a photonic chip's wavelength sweep took 11.6 s to start where it took 2.4 s. E-690's collapsed-twin test fetched every internal node of every OSDI instance by number (CKTnum2nod walks the node list from its head) and, for a collapsed one, walked the list again by name -- instances x internal nodes x circuit nodes at every setup; a sample of the run put 85 percent of its time in CKTnum2nod under OSDIsetup. The candidate deck nodes (a '#' in the name, no device line naming them, or bound at an earlier setup) are now collected once per setup call, an empty set in almost every deck, and matched to instances by name with one node fetched per match; the bindings, the Note and the sens re-setup reuse are unchanged. E-689's OSDIuicSeed had the same shape -- two whole residual vectors cleared per instance per round -- and clears only the instance's own entries now. VERIFIED: the user's deck 11.6 s to 2.26 s with identical output (the E-680 build: 2.4 s); 1000 instances of a 40-internal-node module 1.7 s to 0.07 s and a 14x ratio between 250 and 1000 instances down to 2x (both new internalnode checks fail on the E-691 binaries); 3000 seeded instances under uic 0.37 s to 0.17 s, every node at 0.5; E-690's and E-689's own checks unchanged; full sweep 531/531.

Enhancement-698 *src/include/ngspice/osdiitf.h, src/osdi/osdidefs.h (OsdiTransitionInfo, OsdiSlewInfo, OsdiTransitionState, OsdiSlewState, osdi_wire_slots), src/osdi/osdiregistry.c (OSDI_TRANSITION_/* OSDI_SLEW_ *tables), src/osdi/osdisetup.c (the shared (z,y)/(z,z) entries, node marking, KLU rebind and switch), src/osdi/osdiload.c (transition_stamp, slew_stamp), src/osdi/osdiaccept.c (transition_accept, slew_accept), src/osdi/osditrunc.c (the slew corner), src/osdi/osdiacld.c, src/osdi/osdipzld.c*

THE transition AND slew OUTPUT ROWS ARE NGSPICE'S. Hunt F1 and F2 of 2026-09-21 (the compiler half is in the openvaf log). transition: the accept hook keeps LRM 4.5.8's schedule from the ACCEPTED input -- a change creates a ramp from the current output to the new value over the rise or fall time the operator holds at that point, started now (td = 0) with the accepted point entered at the head of the breakpoint table so the integrator restarts at order one there (a trapezoidal step across a corner that is not a breakpoint reads twice the slope and rings 2, 0, 2, 0 down the ramp) and the trailing corner set with CKTsetBreak, or queued for t + td with a breakpoint there; an active ramp is readjusted by the slope rule of Figures 4-7 to 4-12; a zero time is a negligible ramp of TSTEP/1000 with no trailing breakpoint; an input that changes at consecutive points (a sine through a bare transition) gets no breakpoints, because with them every timepoint bred a breakpoint td later and the transient never finished. The stamp is V(z) = ramp(t), a source to Newton. slew: V(z) = clamp(V(y), y_last - neg h, y_last + pos h) on the output accepted at the previous point, the truncation hook keeping the next step from growing past the corner. DC and MODEINITTRAN stamp the identity and seed the state from the operating point; AC is unity with the delay's phase (E-588), pz the zero-delay wire with absdelay's caveat for a delayed transition. The three slot families share the delay_jac arrays. VERIFIED: the hunt's tables exact in every row, ddt of a switched capacitor 1 mA on the edge and 0 on the plateau, the interrupted ramp per the LRM, evtedge's sine finishing; transedge 26/26, lrmfilters 14/14 and opargs 16/16 per solver, nineteen operator suites unchanged, sweep 531/531.

Enhancement-701 *src/frontend/cornersave.c, cornersave.h (new: the recorder), src/frontend/mcsave.c, mcsave.h (the format parsing, file naming, used-name registry, csv/txt cell writers and the workbook writer exported; writemc feeds both recorders), src/osdi/osdisetup.c, src/include/ngspice/osdiitf.h (OSDImcCornerSnapshot, OSDImcHasCorners; has_stats and held on the snapshot item), src/frontend/spiceif.c, com_sweep.c, runcoms2.c, inpcom.c, src/main.c, src/frontend/Makefile.am, examples/savecorner_examples/ (new, 29 checks per solver)*

`.option savecorner` -- EVERY CORNER RUN, ONE ROW PER CORNER, BESIDE THE NETLIST. The corner twin of `.option savemc` (E-610), keyed by corner where savemc keys by trial: for every corner run -- a plain run at the deck's corner (tt without one), each corner of an `.option autocorner` pass, each corner of the `corners` command -- one row of corner, analysis, status, then the value in force of every cornered parameter (each parameter carrying a corner attribute, read off the devices: the corner's value under a corner, the nominal at tt), then what the run computed: the `corners -output` values, writemc's (per corner on an autocorner combined plot), and under `corners -mc N` the montecarlo's yield, npass, nsamples and nfailed as one summary row per corner, a drawn parameter's cell left empty. A loop command's samples make no rows (montecarlo, sweep, optimize, wcd, highsigma), a dc that sweeps a cornered parameter leaves its cell empty (E-626's rule). csv (default), txt, excel (sheet `corners`, bold model headers, blue outputs, the savecorner_font/fontsize/model/instance/output options with the savemc ones as fallback), `savecorner=<name>.<ext>` with its directories made -- the same code as savemc, exported. nosavecorner last turns it off; a circuit without corners has nothing to record, said once. VERIFIED: savecorner 29/29 per solver (1/29 on the E-699 binaries), savemc, writemc, cornerscmd, autocorner, vacorner, osdimc and ten more unchanged, sweep 532/532.

Enhancement-703 *src/osdi/osdi.h (EVAL_RET_FLAG_FATAL_DEFERRED, LOG_FLAG_DEFER), src/osdi/osdicallbacks.c (a fatal-level message with LOG_FLAG_DEFER waits for the accepted iteration), src/osdi/osdiload.c (OSDI_REQ_FATAL in OSDIpendingRequests from the LAST evaluation's flags; OSDIdeferredFatal), src/osdi/osdisetup.c (raised during setup it is a rejection), src/include/ngspice/osdiitf.h, src/spicelib/analysis/dcop.c, dctran.c, dctrcurv.c, acan.c, noisean.c, tfanal.c (the check at*

each accepted-point boundary, before the point is output); compiler side in its own report; examples/langguard_examples/ (section 5, six checks, 125)

THE E EXTRAPOLATION ERROR OF $table_model IS JUDGED ON THE ACCEPTED SOLUTION (HUNT F2; BOTH TOOLS CHANGE). The compiler's 'E' check raised the immediate $fatal flag on the first Newton iterate outside the table -- the zero initial guess of every operating point -- so a table over 1..4 V could never start under 'E' and a point above the table was reported as below it. The check now raises EVAL_RET_FLAG_FATAL_DEFERRED with a LOG_LVL_FATAL message carrying LOG_FLAG_DEFER; OSDIload lets the evaluation complete with the clamped value, osdi_log holds the message with the iteration's other output, and OSDIdeferredFatal -- called by the six analyses where they act on a deferred $finish: after the operating point's converged reload (dcop), right after a transient point is accepted (dctran), before a dc sweep point is output (dctrcurv, E-673's $fatal path), after the operating-point load of ac and noise, after the transfer function's op -- reads the LAST evaluation's flag (not point_eval_flags, whose accumulation over iterates would re-create the defect), flushes the held message, sets CKTvaFatalRaised and returns E_PANIC before the point is output: "a Verilog-A device raised a fatal error at the accepted sweep value 5 / time point 3.001e-06 s / operating point; aborting". A gmin or source-stepping rung is not an accepted point and is not checked; a deferred fatal raised while setup evaluates a constant operand is a rejection like $fatal. VERIFIED: langguard 125/125 per solver (121/125 on the E-702 binaries), lrmvoice and the nine other fatal-pinning suites unchanged, sweep 532/532.

Enhancement-719 *src/spicelib/analysis/cktsetup.c (dcpath_check: the absent flag, the message, dcpath_silentports), src/osdi/osdisetup.c (OSDIdcpathEdges: the flag, the edges it withholds), src/include/ngspice/osdiitf.h; examples/dcpath_examples (E-575 suite, section [11], 12 checks, 69 per solver), examples/silentports_examples ([7] and [8] re-pinned), examples/groundports_examples ([9] re-pinned)*

AN OSDI TERMINAL THE INSTANCE LINE LEAVES OUT IS HELD BY THE DCPATH GMIN AND NAMED (CORRECTNESS CAMPAIGN F5). A port guarded by $port_connected -- the LRM idiom for an optional pin -- left off the instance line dangles (E-402), gets a node of its own, n1#c, and with the guarded branch off has an all-zero row: no DC path to ground, which E-575's walk exists to hold. The walk passed it by twice over: its OSDI edges come from the descriptor's Jacobian pattern, which carries the entry (c, b) whatever the guard says at run time, so c was "reached" through ground; and the node's name carries the '#' that marks a built-in device's internal node as reached. So every operating point went "singular matrix: check node n1#c" down the gmin and source-stepping ladders to the transient operating point, 277 iterations for a two-line deck. OSDIdcpathEdges now flags the node of every omitted terminal (never a group holding a connected terminal) and joins it to nothing; dcpath_check exempts it from the '#' rule, holds it in every mode (nothing outside the instance can carry it) and says what it is: no DC path from node n1#c to ground -- the terminal is not connected; gmin (1e-12 S) installed to provide one, in every mode. Three iterations. .option silentports (E-481) installs the hold without the line and outside the message cap; =ground (E-482) has no node to hold; dcpath=off is E-566's run; warn and error carry the same clause. Not grounded (E-402 stands): an unguarded resistor to the open port is held all the same and follows v(a). Observed, not changed: a 0 V short onto the open terminal in the else branch does not collapse the node (E-401 drops the short as redundant) -- held at 0 V now, singular before. dcpath 69/69 per solver (60/69 on the E-714 binaries), silentports 24/24 (22/24) and groundports 59/59 (56/59) with the gated thermal port re-pinned as held and named; portconnected, floatnode 16/16 per solver, singularname 12/12 per solver, solvercore, acgminhold 10/10 per solver, ctrlnode 37/37, oprobust 32/32 per solver, warmstart 5/5, failacct 9/9, linesearch 17/17 per solver, bsrcconv 5/5 per solver and vafcrash2 22/22 unchanged; sweep 532/532 alone.

Enhancement-720 *src/osdi/osdiaccept.c (transition_start: the reference ramp of a readjustment; transition_accept_slot: the churn of a change, step 4 -- the edge's corners), src/osdi/osdidefs.h (OsdiTransitionState: ref_active/ref_rising/ref_v_orig/ref_v_dest, corner_pending; OsdiTransition-*

Pending: churn), src/osdi/osdiload.c (the seeding); examples/transedge_examples (E-512 suite, section 6, 4 checks, 30 per solver)

A SOURCE'S EDGE FED STRAIGHT INTO TRANSITION IS ONE CHANGE OF THE INPUT, WHATEVER ITS WIDTH. Correctness campaign 2 (F3 of 2026-09-25): transition(V(b), 0, 20u, 30u) on a PULSE that rises at 5 us and falls at 15 us, the ramp at 0.5, fell at 2.1e4 V/s and reached 0 after 23.9 us -- 2.28e4 and 22.0 us behind a 10 us delay, 1.8e4 and 28.3 us for a 1 us edge -- where a comparator of the same source inside the model (E-698's pinned form) fell at LRM 4.5.8's (v3 - v2)/tf = 3.33e4 V/s and landed 15.0 us after, at reltol 1e-3 and 1e-6 alike. E-698's scheduler creates or readjusts a transition at every change of the accepted input, and the source's 1 ns edge is a dozen accepted points (0.99, 0.97, 0.935 ... 0.683, 0.459, 0.2295, 0), each a change readjusting against the ramp the previous point had made: a continuing direction keeps the origin (the edge's own level, so an uninterrupted edge landed exactly), a reversal takes the destination, which at the first point below the output was the previous point's value, 0.683 -- the ramp to 0 ran at 0.683/tf, and behind a delay the burst came due on other points, 0.63/tf. A change that follows a change (E-698's churn, which already placed no breakpoints, now carried on a queued transition too) readjusts against the ramp the run's FIRST change found, captured at every change that is not churn, so the edge's every point reverses against the same destination 1 and the ramp runs at (0 - 1)/tf from the point of interruption; the comparator's single change is untouched, and a continuous input keeps E-698's contract (idle reference: a fresh ramp per point, no breakpoints; the sine check reads 0.5063). At the first still point after a run the active ramp's destination time gets its trailing corner breakpoint (it landed on the print grid) and the last transition queued from the edge gets one at its due time (behind a delay the final value came due between accepted points and its ramp started 0.19 us late on a 1 us step). transedge 30/30 per solver (27/30 on the E-719 binaries: 0.325 at +25 us, y = 0.01 at 1.120 ms for 1.099, 1.130 for 1.109); lrmfilters 26/26, opargs 16/16, defaulttransition, tranopdelay 11/11, evtedge 14/14, hiername, domainwarn 28/28, lrmops 26/26, lrmfuncs 229/229, lrmevents 26/26, constguard 75/75, limguard 110/110, valguard 113/113 and saveguard 38/38 unchanged; sweep alone.

Enhancement-722 *src/osdi/osdiload.c (OSDIload: the analysis flags of the MODEINITSMSIG evaluation of an AC or NOISE job); examples/idtassert_examples (E-52 suite, idtac.va, section 5, 6 checks, 23 per solver), examples/lrmnoise_examples (E-528 suite, section [1], 3 checks, 28 in all)*

THE SMALL-SIGNAL LINEARISATION OF AN .AC OR .NOISE JOB CARRIES THE SWEEP'S FLAGS, ANALYSIS(”STATIC”) 0. Correctness campaign 2 (F1 of 2026-09-25): V(a,b) <+ idt(I(a, b), 0.0, analysis(”static”))/1e-9 -- LRM 4.5.5's own idiom for an integrator pinned at the bias point -- behind 1 kOhm read 1/R = 1 mA in .ac where 6.283e-6 A was due (a short), its dual I <+ 1e-3 idt(V, 0, analysis(”static”)) read 0 where 1.59e-7 A was due (an open), and if (analysis(”static”)) V <+ 0; else I <+ ddt(C V) linearised the short; the same asserted on ”dc” or ”ic” were right, every transient matched ngspice's capacitor, and $strobe printed static=1 dc=0 ac=1 for the .ac job. ngspice evaluates a small-signal job's model once after the operating point (MODEDCOP

Enhancement-723 *src/frontend/dotcards.c (e469_block_decision, ft_saveused), src/frontend/inp.c (inp_note_parse_time_sets); examples/autoopts_examples (E-572 suite, section E-723, 8 checks, 43 per solver); handbook 3.7*

SET SAVEUSED, SET NOSAVEUSED AND UNSET SAVEUSED IN THE CONTROL BLOCK COUNT; A SET OF AUTOBUS OR AUTOADAPT THERE SAYS IT COMES TOO LATE. Five-options dig (F7 of 2026-09-25): `set nosaveused` in the block under .option saveused still saved out alone, `set saveused` with no card saved everything, `set noautobus` left the line expanded, `set autobus` left it unexpanded, `set noautoadapt` left the node adapted -- nothing printed in any of them -- while `set autocorner` and `set osdimc` from the block act. E-670's heading (”from the control block”) and handbook 3.7 listed nosaveused, noautobus and noautoadapt among the pairs the block reaches; the rule holds for every option read when a run starts and never held for the three that act before the block runs (INP2N, INPadapt, ft_saveused). saveused is decided from

the block's TEXT before the block runs, so its lines count now: the block's last set saveused / set saveused=value / set nosaveused / unset saveused decides, the later line and the block over the cards (controls arrive reversed; the first match is the last line). autobus and autoadapt cannot be reached after the parse: every set, setcs or unset of autobus, noautobus, autoadapt, noautoadapt or adapter in the block draws "Note: `set noautobus` in the control block comes too late for this deck: autobus acts while the deck is parsed, before the block runs, and a deck card (.option autobus, .option noautobus) or .spiceinit decides it; the line changes nothing here", once per line; executing them ahead of the parse was not done (a set inside an if the block never reaches would act). The handbook says which pairs the block reaches. autoopts 43/43 per solver (37/43 on the E-722 binaries); saveused, saveforms, savenoise, savekw, savemiss, autobus, autobusopt, busmix, busmixed, subbus, autoadapt 27/27, adaptmsg, adaptlisted, adaptquiet, silentaccept, optknown, hunt12diag, autocorner 23/23, vacorner, cornerscmd, savecorner, osdimc, savemc unchanged; sweep alone.

Enhancement-724 *src/spicelib/parser/inp2n.c (adapt_name_taken, the naming loop in INPadapt), src/frontend/measure.c (do_measure: the corner header); examples/autoadapt_examples (E-463 suite, 1 check, 27 per solver), examples/autocorner_examples (E-656 suite, section [16], 2 checks, 23 per solver)*

THE INJECTED ADAPTER TAKES A NAME THE DECK DOES NOT USE; A BATCH .MEAS UNDER AUTOCORNER NAMES ITS CORNER. Five-options dig (F8 of 2026-09-25): INPadapt named its injected instance n_adapt_ from 1 without looking at the deck, so a deck with its own n_adapt1_ (an unrelated chan) beside a node the option split failed with "device already exists, bail out" on the USER'S line; and .option autocorner ran a batch .tran's .meas cards once per corner, printing three vmax/vend pairs in pass order with nothing to tell them apart, where E-602 names each corner's plot and corners -output each row. The counter now skips any name a line of the deck begins with (case-folded, whole token; the pass's own earlier adapters are in the deck too and it never returns to their numbers), the debug line naming the number chosen; do_measure prints "autocorner: measures at corner " once before the first result of each run made under the option (autocorner_corner_now, E-666's state), nothing on a plain run or in the autostop check, the result lines themselves and the .meas output file untouched. autoadapt 27/27 per solver (26/27 on the E-722 binaries: n_adapt2_, the hand-written values); autocorner 23/23 (22/23: three headers tt ss ff each before its values, none on a plain run); autoopts 43/43, adaptmsg, adaptlisted, adaptquiet, busmix, busmixed, subbus, vacorner, cornerscmd, savecorner, osdimc, savemc unchanged; sweep alone.

Enhancement-725 *src/frontend/dotcards.c (e469_strip_corner, e469_add_corner_bases; ft_saveused registers a corner copy's base), src/frontend/outitf.c (beginPlot, Pass 2: the two device-name save warnings, not for an inferred name); examples/autocorner_examples (E-656 suite, section [21], 3 checks, 26 per solver); handbook 3.7*

SAVEUSED BESIDE AUTOCORNER SAVES A CORNER COPY'S BASE. A block that read only a corner copy of the combined plot, `op` then `print v(out_ss)`, had `out_ss` saved -- a name no analysis produces -- and every run of the pass was refused, "no data saved for D.C. Operating point analysis; analysis not run", three times. When the loaded models declare corners, every collected name whose node or device part ends in an underscore and a declared corner also registers the name without it: `out_ss` gives `out`, `v(out_ss)` gives `v(out)`, `v1_ss#branch` gives `v1#branch`, the device accessor likewise. Not gated on the option; the extra name is inferred and warns about nothing. And a name the option inferred no longer draws E-418's "no such device" or "device has no parameter" save warnings (E-496's rule, which E-600 had applied to the third message only). Five-options dig F3.

Enhancement-726 *src/frontend/dotcards.c (e469_add_name: a bare name with a dot registers the whole and the part after the last dot; the bare-word, expression and dollar-ampersand scans use it), src/frontend/outitf.c (beginPlot: all_inferred, held_nothing; an inferred set that matches nothing keeps*

the plot whole beside E-603's path); examples/saveused_examples (E-469 suite, section E-726, 4 checks, 44 per solver); handbook 3.7

A PLOT-QUALIFIED NAME, AND AN INFERRED SET THAT NAMES NOTHING. `print tran1.out` -- ngspice's cross-plot spelling -- went into the save set whole; no analysis produces that name, the set matched nothing, and the `op` and the `tran` before the line were both refused "no data saved; analysis not run". The scan cannot tell a plot prefix from a subcircuit path, so it registers both spellings: `x1.out` for the node, `out` for the plot-qualified form. And a save set that `.option saveused` inferred and that matches nothing an analysis produces no longer refuses the analysis: when every applicable save is an inferred one and none matched, the analysis keeps every vector and says so once, `saveused: nothing the control block names is in the pole-zero analysis; everything of it is kept` -- a `pz` beside `print v(out)` had been refused, since no block names `pole(1)` by a form the scan reads. A hand-written save that matches nothing is refused as before. Five-options dig F4.

Enhancement-727 *src/frontend/dotcards.c (e469_scan_bare: expression tokens on every command, the meas words, the file after a redirect; e469_add_expr_names: dots and hashes inside a name; e469_scan_refs: dollar-ampersand names; e469_meas_cmds); examples/saveused_examples (section E-727, 9 checks, 44 per solver), examples/saveforms_examples ([5] and [7] re-pinned, 16 per solver); handbook 3.7*

THE SCAN READS A BARE VECTOR IN AN EXPRESSION, IN A MEAS, BEHIND DOLLAR-AMPERSAND, AND KEEPS A DOTTED NAME WHOLE. Under `.option saveused`, `print v(in) mag(out)` saved `in` alone and `out` was "not available"; `meas tran m find out at=0.5u` failed "no such vector as out" (E-496 had kept `meas` out of the bare-word scan); `echo mid is $&mid` said "no such variable" and an `if ($&mid > 0.4)` the same and a syntax error; `let y = x1.mid*2` in a subcircuit deck was "RHS invalid", the name split at its dot; `print out*2` alone worked by accident, nothing collected. A token with an operator character is now split into its names on an output command as on a `let`; a measure's words after the analysis and the result name are taken, either side of an `=`; `$&name` registers `name` on any line; a dot followed by an identifier character, and a hash, stay inside a name; the file after a `>` is skipped. Over-saving costs a slot and, being inferred, warns about nothing; `print out*2` alone now prunes to `out`. `saveforms` [5] and [7] re-pinned: their probe `print length(in)` named the node they declared unnamed. Five-options dig F5.

Enhancement-728 *src/frontend/com_sweep.c (ac_resample: the trunc argument, ac_missing; autocorner_run: failed[], the per-plot line, the autocorner_failed variable; ac_clear_results); examples/autocorner_examples (E-656 suite, section [22], 3 checks, and one in [9], 30 per solver); handbook 3.7*

A CORNER THAT STOPS PART WAY ENDS WHERE ITS DATA ENDS. A corner whose transient aborted on a timestep too small kept its partial plot, and the combined plot of `.option autocorner` resampled it onto the nominal's scale with the last computed value held flat to the end: a 44-point plot ending at 4.86 us became a 70-point copy to 10 us, flat, with nothing in the plot to say so, and a `find v(out_ss) at=9u` read the phantom. The copies of a corner whose run failed, made no plot or was interrupted now end where its data ends, nan from there (real, or nan + i nan), which every consumer treats as missing; one line per combined plot says where (`autocorner: corner ss: its tran2 ends at time = 4.86308e-06 of 1e-05, where the run stopped; its copies in 'autocorner1' are nan from there`); `$autocorner_failed` names the corners, unset when every corner ran. A corner that succeeded is resampled as before. Five-options dig F6.

Enhancement-729 *src/spicelib/parser/inp2n.c (INPadapt: the orientation of the adapter; adapt_member_name_len, adapt_forward_of, adapt_inst_is; adapt_listed reads the node part of a node:inst member); examples/autoadapt_examples (E-463 suite, adapt.va gains the asymmetric adapt2, section E-729, 9 checks, 36 per solver), examples/adaptlisted_examples (the chatter filter passes the note); handbook 3.7*

A TIE IN THE ADAPTER'S ORIENTATION GOES BY NAME, SAID, NOT BY DECK ORDER IN SILENCE. E-463 orients an injected adapter by port index, the higher forward, ”not deck order: a SPICE deck is order-independent”; when the shared node sat at the same port index on both devices -- two instances of one model, the ordinary shape -- the code fell back to deck order, with a warning under autoadapt=debug alone, so `N1 b a` / `N2 b c` and the same two lines the other way round put a different device on the adapter's p side, 0.2890173 against 0.2881844 with an asymmetric adapter. A tie now goes to the instance that sorts first by name, whatever the deck order, and is said in every mode with the way to make the other choice; `.adapt b:n2` names the forward device outright, tie or no tie (a device that is not one of the two is an error); the member still selects the node for E-467 and E-593. A colon, not =: numparam rewrites a name=value on a dot card. Five-options dig F1.

Enhancement-730 *src/spicelib/parser/inp2n.c (INP2N: E-490's port walk runs for fewer tokens than ports too; written_multi; a walk whose tokens ran out at a port boundary is accepted for the terminals it fed); examples/autobus_examples (E-444 suite, section E-730, 7 checks, 19 per solver); handbook 3.7*

FEWER TOKENS THAN PORTS: THE BUS IN SHORTHAND, THE TRAILING PORT ABSENT. Under `.option autobus`, `N1 a busdev` for a[0:4] and b -- the port_connected shape with the bus port in shorthand -- matched neither the port count nor the terminal count and fell through to positional binding: the bare `a` bound as the scalar node `a` onto the terminal a[0], the deck's a[0] drove nothing the device touched, and E-402 reported four bits of the very port the token names as absent. The walk now feeds the leading ports left to right and, when the tokens run out at a port boundary with no multi-bit port written out before, accepts the rewrite for the terminals it fed; the trailing terminals are absent, as on any short line, and E-402 names them alone (silentports=ground grounds them). A shorthand port, a written-out multi-bit port and then nothing stays E-490's refusal, since it reads exactly like a line whose next token was swallowed as a bit. Five-options dig F2.

Enhancement-731 *src/osdi/osdiinit.c (osdi_warn_case_collisions is handed the module's own rows; osdi_is_mfactor_alias)*

THE CASE-COLLISION CHECK READS THE MODULE'S OWN DECLARATIONS ONLY. The instance table starts with rows the loader writes (dt and dtemp, temp, and after the module's rows the terminal currents), and write_param_info spells the compiler's $mfactor as m as well; E-335's check ran over the whole table, so an operating-point variable named temp, m or dt drew, beside E-505's correct line naming the winner, a second one saying the instance parameter ”is declared more than once differing only in case” -- no such parameter is declared and the two spellings are identical. The check is handed the module's own rows and skips the m spelling of $mfactor; a variable named i_p or i draws nothing and reads back the model's value, as E-644 chose for i; GAIN beside gain is still reported. Evening hunt D1.

Enhancement-732 *src/frontend/breakp.c (stop_live_value, stop_operand; ft_bpcheck clears db_said at the first point), src/include/ngspice/ftedebug.h (db_said)*

A STOP CONDITION ON AN UNSAVED OPERATING-POINT VARIABLE READS IT LIVE, AND A MISSING NAME IS SAID ONCE. A stop-when on an accessor name -- an instance's operating-point variable, or a parameter -- that is not in the save set printed ”no such node” at every accepted point and never stopped, while print reads the live value of the same name; the condition now reads the plot's vector first and, failing that, the live value through if_getparam as print does, so the unsaved run stops where the saved one does. A name that is nowhere is reported once per run, with the stop's number, not once per point, and a fresh run reports it again. Evening hunt D2.

Enhancement-733 *src/frontend/device.c (alter_set, the model-wildcard branch), src/frontend/spiceif.c (if_hasparam_wildcard_instance_only), src/include/ngspice/fteext.h*

THE MODEL WILDCARD ALTER NAMES WHAT IT COULD NOT REACH. E-284's hint (no

loaded model has the parameter, but a loaded instance does -- use the instance wildcard) was printed only when no model took the value; ngspice's built-in resistor has a model parameter r (its default resistance), so beside a plain R card the same model-wildcard alter of r set that default, counted one, and said nothing while the OSDI instances whose r is an instance parameter kept theirs. Whenever the model wildcard set something, the instances of device types whose model lacks the parameter but whose instances carry it are counted, by type, and a warning names them and the instance wildcard; altermod through the same branch says the same; without the built-in the hint is unchanged. Evening hunt D3.

Enhancement-734 *src/spicelib/analysis/cktop.c (gillespie_src exits with CKTdiagGmin = CKTgshunt), cktsetup.c (the no-hold warning), tfanal.c (the output-node checks before CKTop), acan.c (comment)*

SOURCE STEPPING PUTS THE DIAGONAL GMIN BACK WHEN IT IS DONE. The default source stepping ended with CKTdiagGmin left at gmin -- the one rung that did; every other rung restores gshunt -- and NIiter factors with that value on every call, so after a source-stepped operating point the transient op, every transient time point, every later DC sweep point and the next job's plain Newton carried a gmin shunt on every node: a 1e11 ohm divider beside a diode clamp read 0.476 V for 0.5 V through the transient, a 1e14 ohm node beside an inductor across a source 990 V for 100 kV, on both solvers and silently; under Sparse the shunt also landed on the voltage-source branch rows that pivoting had given a diagonal, and a loop of sources whose values do not sum to zero "converged" with 1/gmin amperes where KLU refused it. The exit restores gshunt. Three decks that had reached a successful point only through the leak are honest now: a floating node with dcpath off is refused (E-575's explicit hold still reads I/gmin under the default), a device a model rejects in eval leaves a refused point and a cleanly aborted noise, an OSDI open gate's transient-op point moves by 0.7 mV; the E-566 warning on the no-hold path says nothing holds the node; and a .tf card naming a phantom node is refused before the operating point, as .noise already did. Solver-core hunt F1, with F2's symptom.

Enhancement-735 *src/maths/sparse/spfactor.c (the ordering loop of spOrderAndFactor, the searches, the Order* routines, the bucket index), spbuild.c (spcCreateFillin), spdefs.h, spalloc.c*

SPARSE'S ORDERING IS NO LONGER QUADRATIC IN THE NODE COUNT. The pivot search kept its element lists sorted by internal row and column and moved elements through them at every interchange and every fill-in -- for each element of the two rows and columns, a walk of its column from the top to the predecessor and on to the other position, through the entries of rows and columns eliminated long ago and through the active part; a fill-in walked its column and row from the start -- so a 300 x 300 resistor mesh spent 213 s of a 226 s run reordering under Sparse where KLU took 1.8 s (a sampler put 83 percent of the run in the exchanges and 11 percent in fill-in creation; the Markowitz scan the hunt blamed was 2 s). While spOrderAndFactor chooses pivots the lists now stay in an ordering layout: an entry of an eliminated row or column is finished, carries its external number and is never moved again, finished entries met by a walk are set aside once, the active part of a column is sorted by a key that travels with the row, an interchange only relabels the active entries of the two rows or columns, a fill-in is spliced at the place the merge found, and OrderExit rebuilds the sorted lists in two linear passes; the Markowitz scan is a bucket index over the products that finds exactly the diagonal the scan found (the smallest-product-of-zero case goes to the kept scan). Same pivots, same fill, same operations, same final lists: 86 matrices (meshes, random and MNA networks with branch rows, complex, singular, partial reorders, pivrel and pivtol variants, a right-hand side) dump bit for bit with the old and the new code, the hunt's 122 Sparse decks print the same numbers, and the mesh runs in 0.29, 3.2 and 15.6 s for 100, 200 and 300 (reorder 0.17, 1.9 and 9.3 s, two orderings per operating point) against 1.5, 17.8 and 225.8 s. Solver-core hunt N1.

Enhancement-736 *src/maths/sparse/spfactor.c (the record in SearchEntireMatrix), spalloc.c (spWhereSmallPivot), spdefs.h, spsmp.c; include/ngspice/spmatrix.h, smpdefs.h (SMPsmallPivot), cktdefs.h, optdefs.h (ERRP_PIVTOL); maths/KLU/klusmp.c (the U-diagonal check after klu_factor*

and klu_z_factor); maths/ni/niiter.c (NIsmallPivot), niaciter.c, niditer.c; spicelib/analysis/cktsopt.c

A PIVOT BELOW PIVTOL IS REPORTED, UNDER BOTH SOLVERS. .option pivtol, ”the absolute minimum value for a matrix entry to be accepted as a pivot”, reached both solvers and changed nothing anyone could see: Sparse honoured it while choosing among candidates and, when no candidate passed, took the largest element and returned a verdict (spSMALL_PIVOT) that ngspice maps to OK; KLU ignored the value outright; a matrix of 1e-14 S conductances solved under pivtol=1e-3, pivtol=1e30 and pivtol=1e-13 alike, silently, and E-475 had validated a value nothing read. Sparse now remembers the first pivot it took at or below the threshold (spWhereSmallPivot), KLU checks the diagonal of U after every full factorization with each pivot scaled back by its row's factor when the matrix was row-scaled, SMPreorder and SMPcReorder leave the record in the SMPmatrix (SMPsmallPivot), and NIiter, NIacIter and NIdIter print ”the pivot for node b is 3e-14, below pivtol (0.001)” -- the node whose unknown the pivot belongs to and, when it came from another node's equation, that node too; once per node and magnitude, six per run, all of them under set ngdebug, like the singular-matrix messages. A deck that sets pivtol is answered every time (an operating point, an AC, a transient); the default floor of 1e-13 is reported only under set ngdebug, because the suites cross it 125 times on decks that are fine (an OSDI branch carrying nothing, a probe source's row beside a 10 T-ohm path, a thermal node) and Spice3 chose silence there. Values never change; a factorization that reuses a pivot order does not check. Solver-core hunt F3.

Enhancement-737 *ngspice-46/src/spicelib/analysis/cktpzstr.c, examples/klupz_examples/verify_klupz.py, examples/klupz_examples/README.md, examples/pzeig_examples/verify_pzeig.py, examples/pzeig_examples/README.md*

THE POLE-ZERO SEARCH HOLDS AT THE DETERMINANT'S ROUNDING FLOOR. The same common-emitter stage gave 2 poles under KLU's default ordering (”iteration limit reached; giving up after 218 trials”), 3 with klu_btf=off or klu_scale=none, 5 with klu_scale=sum, colamd or Sparse, and 5 under KLU once three deck lines were moved (F4 of the second solver-core hunt). The determinant was not at fault, its rounding was the trigger: near the third pole the deflated determinant is at its floor (2^-88 against 2^-57 a decade away) and its sign flips at random, and the Muller driver replaced a sign-change bracket's endpoint by magnitude -- its own comment says ”really should check signs” -- so the one point of the other sign was dropped, the three same-sign points read as a magnitude minimum, and the search left the real axis for a conjugate pair that is not there until its iteration limit; which ordering's rounding did this first was the solver's and the knobs' only part. Three more of the same family on the way: Muller's two companions to a complex start sat at j1e8 and j1e12 whatever the circuit's scale (a twelve-decade RLC ladder started almost on its pair at j3.16e13 and wandered from that triple to the limit with 3 of 6 roots, both solvers); the outward march after the last root went on to +-1e21, where the rounding of the s*C entries against G makes the deflated determinant's variation noise, and a chance minimum there trapped the search (a bandpass under Sparse gave up after 231 trials with both roots found); and the hunt for a real-axis magnitude minimum refined values already equal to 1e-13 until two trials coincided to 1e-12 by chance (45 trials, or never). Now a bracket keeps its crossing and a repeated move splits inside it; the companions sit at half and twice the start's imaginary part when that came from the real-axis quadratic; the march stops once the deflated determinant is flat to 2^-20 over 1e16, which puts every remaining root beyond 1e22, the span at which the search gave up anyway; a minimum whose three magnitudes agree to 2^-33 is taken at once; the ”giving up” warning says how many roots it has and that .option pzeig does not iterate. The stage gives its five poles under every solver and knob, with the lines moved or not (61 trials against 208 for two); the ladder gives all six exact roots of its characteristic polynomial (a pair at -5e11 +- j3.16e13 and -1e15, which the dense pzeig method's infinity threshold discards, now said in its README); the bandpass no longer gives up. 40 random R/L/C/G networks against the exact roots of their characteristic polynomials: 77 of 80 solver runs complete against 70, one warning against eight, none worse; the 56 pole-zero decks of the hunt and the suites: 20 better, 33 the same, none worse. Unchanged: the coincidence test that

accepts a root, judged on the real part -- a pair whose real part is 3e-14 of its magnitude still stalls under KLU's rounding; judging it by the magnitude accepted creeping iterates elsewhere and was not kept -- the determinant path, NITER_LIM and pzeig. klupz gains checks 10 to 16 (21 in all; 10 of 21 on the E-736 binary); pzeig's check 8 is re-anchored on the ladder's far roots, its bandpass warning being gone.

Enhancement-738 *ngspice-46/src/maths/KLU/klusmp.c, ngspice-46/src/maths/sparse/spsmp.c, ngspice-46/src/include/ngspice/smpdefs.h, examples/solvercore_examples/verify_solvercore.py*

UNDER SPARSE THE DIAGONAL GMIN GOES TO THE STAMPED DIAGONALS, AS UNDER KLU. F2 of the second solver-core hunt: since E-734 an unsolvable loop of three voltage sources is refused under both solvers, but under Sparse its ladders still said "One successful gmin step" 22 times on a matrix singular by construction where KLU reported singular at every rung. The hunt read this as Sparse's pivoting creating a diagonal element for the branch row; that element exists (a fill-in) but is a side effect, and the exchange cited creates nothing. Sparse indexes Diag[] by internal row and the array follows the pivot order: the MNA preorder swaps a voltage source's column with a node's so that the source's +-1 twins sit on the diagonal, the Markowitz exchanges put whatever pivot was chosen there, and LoadGmin walked that array -- on the loop four of the six entries fed were +-1 KCL/KVL stamps, on a BJT stage with an inductor six of ten (the source and inductor twins), while the inductor's two nodes got no shunt at all; KLU's LoadGmin_CSC feeds the stamped (i,i) entries by external identity, never a branch row. Sparse now records the address of every stamped diagonal at its first preorder (before the column swaps, while internal and external numbering agree; SMPgminDiag in the SMPmatrix, SparseGminDiag in klusmp.c and spsmp.c) and LoadGmin feeds those whatever the pivot order has done since: the inconsistent loop, the rank-deficient loop and the loop under gmin=1e-6 climb no rung under Sparse (374 singular reports, 0 successful steps, KLU's numbers exactly), a BJT stage with noopiter climbs the ladder to the same point in the same 7 rungs and 23 iterations, a floating chain of sources under dcpath=off solves its rungs on the two stamped diagonals and is refused at the end alike under both solvers. No solvable deck moved: the 122 Sparse decks of the hunt's harnesses print the same values on the E-737 binary and this one, rshunt on the stage gives the same voltages to ten digits, and the twelve ladder-related suites (oprobust, solvercore, dcpath, floatnode, singularname, acgminhold, internalnode, explicitvalue, linesearch, warmstart, failacct, convhelp) pass under both solvers. Unchanged: KLU, Sparse's pivot choice, a node that stamped no diagonal (a floating pair joined by one source gets no shunt under either solver). solvercore gains section F2 of the second hunt (5 checks, 42 per solver; 39 of 42 under Sparse on the E-737 binary); E-734's write-up carries the correction.

Enhancement-739 *ngspice-46/src/osdi/osdidisto.c, examples/osdidisto_examples/verify_osdidisto.py, examples/osdidisto_examples/va/dst_qcube.va, examples/osdidisto_examples/va/dst_diodeq.va*

THE DISTORTION ANALYSIS OF COMPILED DEVICES INCLUDES THE NONLINEARITY OF THEIR CHARGES. F9 of the options-and-convergence hunt: E-359 builds its Volterra tensors by differencing the Jacobian arrays a compiled model writes back at perturbed voltages, in a resistive pass and a reactive pass rotated by j*omega, but the evaluations it perturbed ran under flags that mirror a DC operating point, which never compute the reactive Jacobian, so every probe step read the same stale reactive array, every finite difference of it was zero and the reactive pass rotated an empty tensor; a module whose only nonlinearity is a charge reported 0 + j0 for every harmonic where a tight transient shows 5.9 percent THD, and the benchmark diode's diffusion charge put HD2 a sixth low and HD3 two fifths low at 10 MHz against the transient fourier, which matches the built-in's .disto to three digits (the junction capacitance's share is 1 percent, which is the 1 percent that pair disagreed by; the resistive part was right to six digits all along); the driver's OsdiSimInfo now carries CALC_REACT_JACOBIAN and CALC_REACT_RESIDUAL, nothing else changes, the operating point after a .disto equals the one before to the bit; the compiled diode with junction and diffusion charge now matches the built-in's HD2, HD3, f1+f2, f1-f2 and IM3 at 1, 10 and 100 MHz to 6e-6 (the vt constant), a charge polynomial matches its closed form (the n-th order current source is* jomega_out times the

n-th order term of Q, solved at omega_out) to 1.4e-9, and matches the transient fourier within the transient's tolerance; osdidisto gains dst_qcube.va, dst_diodeq.va and checks 9 to 12 (12 in all; 8 of 12 on the E-738 binary), distoexact and limitdisto unchanged, sweep 532 of 532; unchanged: the resistive path, the built-in devices' distortion code, every other analysis, E-359's step rule.

Enhancement-740 *ngspice-46/src/spicelib/parser/inpdpar.c, ngspice-46/src/spicelib/parser/inp2n.c, ngspice-46/src/spicelib/parser/inpgmod.c, ngspice-46/src/spicelib/parser/inppas2.c, ngspice-46/src/osdi/osdisetup.c, examples/hunt12diag_examples/verify_hunt12diag.py*

WHAT A COMPILED DEVICE'S LINES SAY WHEN THE DECK SPEAKS THE BUILT-IN DIALECT. N1 of the options-and-convergence hunt, messages only: ic= on a compiled instance is named as the built-in devices' initial-condition list with the .ic v(node)= route, a bare off on a compiled model that declares no off parameter is named as the built-in start-off flag (a model that declares one keeps E-597's has-no-value text, which is then right), m= on an instance of a model that owns a parameter m (a diode's grading coefficient) says the instance multiplier is spelled _mfactor=, level= on a compiled model card is warned as ignored since the module is chosen by the type name (E-426 had left level silent because the built-ins consume it in pass 1), a compiled model under another device letter is named with its module and the n prefix after the letter's own message, in both halves the unused-model cull makes: a q, d, m, p or full y line keeps the card alive and fails on the type, a short y or a t line refers to the model as a node or a parameter so the cull has already commented the card out and pass 2 scans the deck's commented cards for the token and tests the card's type for a compiled module; and .option bypass with compiled devices is noted once per circuit as without effect; two of the hunt's notes were wrong and its page is corrected (the errors carried a third line the hunt's filter had cut; a constant-text message is hoisted by the compiler into the setup code and printed once per setup, not suppressed); hunt12diag gains six checks (26 in all; 20 of 26 on the E-739 binary), barevalue, guardspell, paramsetguard, numguard, binstale, acgminhold, cmcsweep, autobusopt unchanged, sweep 532 of 532; unchanged: a wrong prefix that parses as a valid line of its own kind (t1 a 0 dva fails at setup on its impedance), the ngspice-wide items (meas needing save, stop when re-firing on resume, maxord needing dynorder, dc over a compiled resistor), the repeat policy, and bypass itself.

E-741 *src/frontend/snp2va.c, examples/presnp_examples/**

PRE_SNP READS TOUCHSTONE 2, AND STOPS GUESSING. F1, F10 and F11 of the 2026-09-26 Touchstone-import hunt. The converter behind pre_snp (both backends share it) treated a bracketed Touchstone 2 keyword line as data: the numbers on [Version] 2.0, [Number of Ports] 2, [Number of Frequencies] 21 and [Reference] 50 50 entered the number stream ahead of the first frequency, every frame was misaligned, the frame count dropped its remainder in silence and the vector fit ran on garbage (16 poles, rms error 6e-2 and an S21 of -66.7 for a file whose v1 form fits to 2 poles and 4e-4); a v1 file with the noise-parameter rows the specification allows after the network data was fitted the same way (42 percent error, silently); a .y2p or .z2p written by wrsnp was read as a 1-port, because only an .sNp extension counted the ports and the divisor fallback answers 1 for every two-port frame of 9 numbers; and a v1 Y or Z file's data, which the specification, wrsnp and rdsnp normalize to R, was taken as absolute, fifty times too conductive (the Python original has the same lines). Now the parser reads the v2 keywords -- [Version], [Number of Ports], [Two-Port Data Order], [Number of Frequencies], [Number of Noise Frequencies], a per-port [Reference] continued over lines, [Matrix Format] Full/Lower/Upper with the triangle mirrored, [Begin Information]..[End Information] skipped, [Network Data], [Noise Data] and [End] ending the data -- refuses [Mixed-Mode Order], the G and H parameter types, an unknown keyword, a v2 two-port without its data order and a [Number of Ports] that disagrees with the extension by name, refuses a number count that is not a whole number of frames with the count, the frame size and the two usual causes named, checks [Number of Frequencies] against the frames, carries a per-port real reference into the S-to-Y conversion (S' = S scaled by sqrt(z_i/z_j), Y = G^-1 (I - S')(I + S')^-1, the old expression when every port is z0), de-normalizes a v1 file's Y and Z and takes a v2 file's as absolute, and counts ports from a .yNp or .zNp extension too; the fit, both emitters and

the native device are untouched. presnp gains ten checks (19 in all; 9 of 19 on the E-739 binary): a v2 two-port in 12_21 order with a split [Reference] and an information block, a per-port [Reference] 50 75, the 3-port star as a .ts file in Lower format, 21_12 with a [Noise Data] section, the v1 noise-row refusal with nothing written, .y2p and .z2p as wrsnp writes them, a v2 Y file, and the three refusals by name, each block matching the original network to 3e-7; nport_native, snpfuzz, lowrank, touchstone, nport and crashfix2 unchanged, sweep 532 of 532. Unchanged: rdsnp (a v1 reader still: the hunt's F3, F4, F8), the noise rows themselves (refused, not read), the fit-error report (no threshold), the Python snp2va.py, and the native device's terminal-warning flood, pz abort and missing thermal noise.

E-742 *src/frontend/mcsave.c, src/frontend/mcsave.h, src/frontend/commands.c, examples/savecorner_examples/**

WRITECORNER AND WRITECR, WRITEMC UNDER THE CORNER FILE'S NAME. Requested by the user. .option savecorner (E-701) is the corner twin of .option savemc, and writemc has put every value computed after a run onto both files' last rows since E-701 (on an autocorner combined plot onto every corner's own row, E-666), but there was no command named for the corner file; writecorner is now a second entry point that sets the command name and calls the same handler, and writecr a third, its short form -- the same values onto the same rows, the same refusals of the fixed columns, the same per-corner evaluation -- and the handler's fourteen messages, which carried a literal writemc prefix, print the name that was typed, so a refusal after writecorner corner=7 reads under writecorner; the usage line says savemc / savecorner row, and the nothing-recorded note, whose condition has been neither recorder set since E-701, names both options where it named savemc alone; the help entries say one command serves both recorders today and the names are kept separate so they can diverge; savecorner gains five checks (34 in all per solver; 29 of 34 on the E-741 binary, where the commands do not exist), writemc, savemc, cornerscmd and autocorner unchanged, sweep 532 of 532. Unchanged: which recorder a value lands on (both, for any of the three commands), the montecarlo and corners -writemc flags, the savemc_writemc font option.

E-743 *src/osdi/osdisetup.c, examples/dcpath_examples/**

A CURRENT CONTRIBUTION TO THE IMPLICIT GROUND IS A DC PATH. Requested by the user, whose behavioural models write their output stages as I(out) <+ (V(out) - y)/rout, the ordinary way to give a block a finite output resistance: E-575's walk reads an OSDI model's edges off its Jacobian pattern, a path needing the symmetric pair a conductance stamps and a diagonal joining nothing new, and a voltage contribution is recognised as a branch to ground through its flow unknown, but a current contribution to that ground has no flow unknown and no second node, so its conductance shows only as the diagonal (out, out) and a 1 S output stage driving a capacitor, nothing, or the next stage's probed input was named without a DC path and gmin-held (the point exact, the message on every instance, dcpath=error refusing the deck); the pattern does say where that current goes, since a branch to another device node that depended on V(out) would put an entry in that node's row, so a resistive diagonal on a voltage node whose column holds no other resistive entry is now read as the conductance to ground it must be and the node is joined to ground, in the reactive walk too (a ddt() to ground alone is a reactive path, held at DC only, where it was held in every mode); a controlled current with no V(out) term has no diagonal and stays held, a column with other entries stays with the symmetric rule, and E-719's guard for a terminal left off the instance line still wins; dcpath gains four modules and nine checks (84 per solver; 77 of 84 on the E-742 binary), floatnode, silentports, groundports, oprobust, solvercore, acgminhold, internalnode and singularname unchanged, sweep 532 of 532. Unchanged: a ground conductance beside a branch to a floating internal node (the pattern cannot split the diagonal), the topological reading of a zero-valued conductance, the built-in tables and the XSPICE port kinds, and the potential form, which was a path and still is.

E-744 *src/frontend/postcoms.c, src/frontend/commands.c, examples/touchstone_examples/*, .gitignore*

RDSNP READS TOUCHSTONE 2. F3 of the 2026-09-26 Touchstone-import hunt, with F4's message and F8. E-72's reader scanned every line that was not # or ! for numbers with sscanf, which stops at a bracket, so a Touchstone 2 keyword line was dropped whole: the frame count came out right, but [Two-Port Data Order] 12_21 was never seen and S12 and S21 came back swapped, [Reference] was lost and Rbase fell back to the option line's R or 50, and a v2 Y or Z file, which the v2 specification stores un-normalized, would have been divided or multiplied by Rbase as if v1; a v1 two-port carrying the noise-parameter rows the v1 specification allows was refused as a wrong port count, the right refusal with the wrong cause named; and a file with no option line was read as Hz S RI where the specification's default is GHz S MA R 50, which the converter applies (the 100 kHz point came back as 1e-4 Hz with magnitude and angle taken as real and imaginary parts). Now the reader is version-aware as E-741 made the converter: the keywords are read ([Version], [Number of Ports], [Two-Port Data Order], [Number of Frequencies], [Number of Noise Frequencies], a per-port [Reference] continued over lines and published as the vector Zref with Rbase port 1's and a note when the ports differ, [Matrix Format] Full/Lower/Upper with the triangle mirrored, [Begin Information]..[End Information] skipped, [Network Data], [Noise Data] and [End] ending the data), [Mixed-Mode Order], the G and H types, an unknown keyword, a v2 two-port without its data order and a [Number of Ports] that disagrees with the command or the extension are refused by name, a count that is not a whole number of frames names the count, the frame size and the two usual causes, [Number of Frequencies] is checked against the frames, a v1 file's Y and Z are de-normalized and a v2 file's taken as written, the port count comes from a .yNp or .zNp extension too, a trailing ! comment is stripped, and a file with no option line takes GHz S MA R 50 with the warning naming it; the plot keeps its shape (frequency in Hz, complex S_i_j, Rbase, Zref for a v2 file) and the summary line says Touchstone 2. touchstone gains seven checks in section 9 (24 per solver; 17 of 24 on the E-743 binary), snpfuzz, presnp, nport_native, rfstab, savenoise and pyplotsmith unchanged, sweep 532 of 532. Unchanged: the noise rows themselves (refused, not read, as in the converter), wrsnp as a v1 writer carrying one reference, mixed-mode and hybrid data (refused, not converted), the .sp analysis and rfstab.

E-745 *src/frontend/snp2va.c, src/frontend/snp2va.h, src/frontend/com_presnp.c, src/frontend/commands.c, examples/presnp_examples/**

PRE_SNP REFUSES A FIT THAT IS NOT THE DATA. F5 of the 2026-09-26 Touchstone-import hunt. The order selection reported its rms relative error (the worst element's relative rms error over the band) in the status line and emitted the model whatever the value -- the hunt's misread files were simulated at 6 and 42 percent error with that line as the only sign -- and the line reported the best fit's error even when the fit returned was the one before the knee; now a fit above the limit, 0.1 by default, is refused before anything is emitted, with the error, the pole count, the limit, the usual causes (a noisy or too coarse measurement, a delay-dominated block, a misread file, rdsnp named as the way to look) and the flags that accept it; pre_snp -maxerr raises the limit for that command and -force removes it, a fit accepted above the default says so, the limit returns to the default after every command, -maxerr without a positive number is refused naming the flag and the default, the status line reports the returned fit's own error, and the usage and the help entry carry the flags; presnp gains five checks (24 in all; 19 of 24 on the E-744 binary): a random unfittable file refused with the number and nothing written, -maxerr 2 and -force accepting it and saying so, the bad -maxerr values refused, and -maxerr 1e-9 refusing the clean resonator; lowrank, snpfuzz, nport_native and touchstone unchanged, sweep 532 of 532. Unchanged: the fit and both emitters, passivity (checked, not enforced), the Python snp2va.py, and a stable in-band fit of a delay-dominated block, which the limit accepts.

E-746 *src/include/ngspice/devdefs.h, src/spicelib/devices/nport/nportinit.c, src/spicelib/devices/nport/nportsetup.c, src/spicelib/parser/inp2n.c, examples/nport_native_examples/*, examples/crashfix2_examples/

A NATIVE NPORT LINE NO LONGER PRINTS 509 UNCONNECTED-TERMINAL WARNINGS. F6 of the 2026-09-26 Touchstone-import hunt. The native n-port device declares 512 terminals

as a maximum, its port count being known only from the model's fit file, and E-481's warning compares the line's node count with that declared count, so every N1 p1 p2 0 mm line listed terminals 4 to 512 as absent (510 lines on the suite's own one-port deck) with a closing sentence about $port_connected written for compiled modules, and .option silentports=ground bound the 509 phantoms to node 0; a device flag, DEV_VARTERMS, now says terms is a maximum, the native n-port carries it, and the dispatcher's E-481 warning and its =ground binding both skip a flagged device (the terminals stay unbound, as the device's setup expects), while an OSDI device is not flagged and keeps E-481 and E-719 as they were; the device's own refusal of a short line carries both counts (connects 1 node where the 1-port model needs 2, 1 port + the reference) where it said fewer; nport_native gains three checks (9 per solver; 7 of 9 on the E-745 binary), crashfix2's E-244 check reads the new wording, silentports, groundports, dcpath and presnp unchanged, sweep 532 of 532. Unchanged: every other device (none declares a maximum), a line with more nodes than the model's ports (bound and ignored, as before), and the 512-wide node array.

E-747 *src/spicelib/devices/nport/nportpzld.c (new), src/spicelib/devices/nport/nportinit.c, src/spicelib/devices/nport/nportext.h, src/spicelib/devices/nport/Makefile.am, examples/nport_native_examples/**

THE NATIVE NPORT HAS A POLE-ZERO LOAD. F7 of the 2026-09-26 Touchstone-import hunt. E-242's device table set a pz setup but no pz load, so the pole-zero analysis never held the block: on the hunt's RC two-port the run said pz simulation(s) aborted and nothing else, while the OSDI route through the same file found the RC pole; NPORTpzLoad is the AC load with the analysis's complex s in place of jw (NPORTadmittance evaluates Y at any complex s), the same four-corner admittance stamp into the real and imaginary slots the analysis reads, with the device's existing complex KLU binding serving it; nport_native gains three closed-form checks (12 per solver; 9 of 12 on the E-746 binary): the RC one-port's pole at -2e6 with the built-in twin agreeing, the RLC one-port's transfer poles at the roots of its quadratic with no zeros reported (its zeros sit at the fit's own poles, poles of the stamped admittance rather than roots of a determinant, which a direct stamp cannot report and the OSDI route's internal states can), and the Pi two-port's two poles at the roots of det(Y(s) + diag(Gs, Gl)), each to 1e-6, no run saying aborted; klupz, pzeig, pzklu, pzhb, crashfix2 and presnp unchanged, sweep 532 of 532. Unchanged: the fit's own far poles (reported, as through the OSDI route), zeros at the block's own poles (absent from the native answer), the device's missing noise, distortion and sensitivity loads, and the bare aborted message of any other device without a pz load.

E-748 *src/spicelib/analysis/nevalsrc.c, src/include/ngspice/noisedef.h, src/spicelib/devices/nport/nportnoise.c (new), src/spicelib/devices/nport/nportdefs.h, src/spicelib/devices/nport/nportinit.c, src/spicelib/devices/nport/nportext.h, src/spicelib/devices/nport/Makefile.am, examples/nport_native_examples/**

THE NATIVE NPORT HAS THERMAL NOISE. F9 of the 2026-09-26 Touchstone-import hunt, for the native backend. E-242's device had no noise entry, so a block imported from a measured filter, package or cable contributed nothing to .noise (3.4e-13 V/sqrt(Hz) through the hunt's RC two-port against the built-in's 1.09e-9) and the noise figure of .sp through it was the surrounding circuit's; a passive N-port at temperature T has the noise-current correlation matrix 4kT Re(Y(f)) between its ports (Bosma's theorem), fixed by the data the block carries; NevalSrcVec, a correlated multi-node source beside NevalSrc and NevalSrc2 (one process of density psd injecting amp[k] at node k against a reference, the adjoint transfer summed, the S-parameter noise path's per-port machinery applied to the summed source), and NPORTnoise, which forms 4kT sym(Re Y) from the fit at each frequency, decomposes it by cyclic Jacobi into eigen-sources and evaluates each positive one, with the resistor's bookkeeping and one summary source per instance (onoise_); nport_native gains six checks against built-in twins of the same admittance (18 per solver; 12 of 18 on the E-747 binary): the RC one-port's spectrum to 1e-6, the Pi two-port's with the series resistor's fully correlated port currents (the off-diagonal carried) to 1e-6, the RLC one-port's frequency-dependent Re Y to 1e-5, .temp 77 scaling the amplitude by sqrt(350.15/300.15), the summary vector onoise_n1, and the .sp

noise figure through the Pi block equal to the twin's 13.612351 dB; noise, noisecorr, noisefigure, rfanalyses, savenoise, noiseloop, lrmnoise, noisejw, modelnoise, noisetable and presnp unchanged, sweep 532 of 532. Unchanged: the -osdi route (its Verilog-A has no noise source and cannot express a frequency-shaped port-correlated 4kT Re(Y) with white_noise; a block whose noise matters is imported with pre_snp -native), an active block's noise (its noise parameters are the rows both readers still refuse), an instance temperature, and a non-passive fit's negative eigenvalues (dropped).

E-749 *src/frontend/snp2va.c, src/frontend/postcoms.c, examples/presnp_examples/*, examples/touchstone_examples/

BOTH TOUCHSTONE READERS READ THE NOISE-PARAMETER BLOCK. F2 and the second half of F4 of the 2026-09-26 Touchstone-import hunt. A Touchstone 1 two-port may carry noise parameters after its network data (rows of f, NFmin, the magnitude and angle of Gopt, Rn normalized to R) and a Touchstone 2 file carries them in a [Noise Data] section; E-741 and E-744 had both readers refuse such a file with the rows named, which left the network data behind them unusable without editing; now both parsers walk the v1 frames while the frequency rises and hand what follows the first fall to the noise block (the specification's rule), read a v2 [Noise Data] section to [End] instead of stopping at the keyword, and refuse only a block that is not a whole number of rows of five, naming it and its columns; rdsnp publishes the rows in a plot of their own (frequency in Hz, NFmin in dB, SOpt complex, Rn in ohms, a v1 file's de-normalized, under the names the .sp noise analysis uses) with the network plot staying current and a summary naming both, and pre_snp keeps the rows, fits the network as before and says on its status line how many were read and that the model does not use them; presnp replaces its refusal check by two reading checks and counts the v2 section's rows (26 in all; 23 of 26 on the E-748 binary), touchstone one by three (26 per solver; 23 of 26 on the E-748 binary), snpfuzz, nport_native, lowrank and rfanalyses unchanged, sweep 532 of 532. Unchanged: what the rows are used for (nothing yet: the native device's noise is the passive block's 4kT Re(Y), an amplifier's file still gives the wrong noise, and building the correlation matrix from NFmin, Gopt and Rn is the next step), wrsnp (writes no noise block), and a v1 block whose frequencies do not fall back (read as network data, refused by the frame count).

E-750 *src/frontend/snp2va.c, src/frontend/snp2va.h, src/frontend/com_presnp.c, src/frontend/commands.c, src/frontend/postcoms.c, examples/presnp_examples/*, examples/touchstone_examples/

THE FIT'S ERROR IS MEASURED AGAINST THE BLOCK, NOT ITS NEGLIGIBLE ELEMENTS; THE ORDER CLIMB GOES PAST 24 POLES. Found while timing the pre_snp backends on a 16-port bus of eight coupled lines: E-745 refused it at 23 percent, and the first reading, the order cap of 12 pole pairs, was wrong -- the error sat at exactly 0.2334 from 8 to 24 poles and rose above, because the worst element's rms error was taken relative to that element's own size, and the admittance between the bus's farthest channels is 4e-11 of its diagonal, so its tiny absolute error was 23 percent of it while the block was fitted to 1.5e-8 of its largest element; each element's error is now relative to the larger of its own size and a thousandth of the largest element's, the acceptance tolerance moves from 1e-3 to 1e-4 on that measure, the climb goes a pair at a time to 12 and then a fifth per step to a cap of 80 poles (never more than the frequency points support), the knee needs two flat steps in a row against the best so far (one flat step ended it and the bus gives the same error at 6 poles as at 4 before dropping three decades at 8) and returns the best fit, a fit that reaches the cap says so in its status and in E-745's refusal with -maxpoles named, pre_snp -maxpoles sets the cap and -order pins the pole count (a bad value refused naming the flag), PRE_SNP_DEBUG=1 traces one line per order, and both Touchstone readers read a line of any length into a growable buffer (a 16-port frame on one line is over 8 KB and the fixed 4 KB buffer cut a number in two); the bus now converges at 8 poles and 1.5e-5 and matches its built-in twin, a random file stops at 30 poles instead of climbing to the cap, every suite fit converges at the order it did; presnp gains five checks (31 in all; 26 of 31 on the E-749 binary) and touchstone one (27 per solver; 26 of 27),

lowrank, nport_native and snpfuzz unchanged, sweep 532 of 532. Unchanged: the fit's algorithm and both emitters, the measure's floor (a thousandth, no flag), the file's precision as the bound on a near-transparent block (eight digits fit the bus to 1e-3, ten to 1.5e-5, the S-to-Y conversion amplifying rounding by the inverse of I + S), and the Python snp2va.py.

E-751 *osdi/osdiparam.c, parser/inpdpar.c (comment), inputguard and barevalue suites*

M=0 ON A COMPILED INSTANCE IS ANNOUNCED. A compiled instance given m=0 vanished from the circuit without a word on every route (the instance line, _mfactor=0, a subcircuit called with m=0, a card's _mfactor=0 default, alter at run time): v(a) rose to the source, the instance current read 0, nothing named the device; E-426 had made m=0 the disable idiom and E-447 deliberately left it silent, so a multiplier a parameter expression evaluated to zero looked exactly like one written on purpose. The compiled layer's setter, where all five routes end, now prints one Note per instance naming it (Note: nr1: m=0 disables the instance -- it stays in the netlist and contributes nothing (a positive m re-enables it)), then applies the value as before; a subcircuit's instance is named by its flattened name; ten Notes at most, then one line saying the rest are not listed; a Note and not a Warning because the deck is not wrong. Unchanged: the instance stays disabled (instknobs reads 0 A), any positive m is silent, the built-ins keep E-426's silence (the parser-layer check is untouched), F6's two contradicting negative-m warnings, the fixed cap of ten with a per-process counter. inputguard 100 of 100 (94 of 100 on the E-750 binary), barevalue 21 of 21 per solver (20 of 21), guardspell and instknobs unchanged, sweep 532 of 532.

E-752 *frontend/trannoise/prbs.c (new), include/ngspice/prbs.h (new), vsrc/ and isrc/ (defs, table, par, load, acct, temp, del), prbs suite (new)*

BUILT-IN PRBS AND PAM4 SOURCES. Every serial-link deck built its data pattern outside ngspice (the eye example writes a PRBS as a PWL list from Python); the independent sources now take PRBS(v1 v2 tbit [td [tr [tf [order [seed]]]]]) and PAM4(v1 v2 tsym ...) on the voltage and the current source: a Fibonacci shift register with the standard maximal-length taps (ITU-T O.150's polynomials for 7, 9, 11, 15, 23 and 31, IEEE 802.3's x^13 + x^12 + x^2 + x + 1 for 13, XAPP052's for the other lengths 2..31), all ones as the default seed, bit k in [td + k tbit, td + (k+1) tbit) with a transition starting at the boundary over tr or tf (0 or omitted: the analysis's step, as PULSE), v1 held before td; PAM4 reads the same register two bits per symbol, Gray coded as 802.3 clause 120 defines PRBS13Q (8191 symbols per period, 2047/2048/2048/2048 per level) and PRBS31Q, the four levels evenly spaced from v1 to v2. Corners are breakpoints scheduled as PULSE schedules its edges and a run of equal symbols schedules none; the value is a pure function of time (the register after bit k cached with a 64-bit ring for backward moves), so a rejected step costs nothing and two runs agree; the register and the scheduled corner are re-armed per run beside E-498's PULSE re-arm, alter re-reads the list (and turns a prbs into a pam4); refusals by name, in the keyword's own words, for fewer than three values, a non-positive bit or symbol time, a negative delay, an edge longer than the bit, an order outside 2..31 and a zero seed; a dc value serves the operating point as with PULSE. PRBS31 at 10 Gb/s runs ten thousand bits in 0.6 s. prbs 42 of 42 per solver (new; 1 of 27 on the E-751 binary), guardspell, eye, trnoise, inputguard, pwlfix and srcnote unchanged, sweep 533 of 533. Unchanged: no explicit bit-pattern string, no PAM8 or precoding, no built-in jitter, the eye example's own PWL script.

E-753 *frontend/com_help.c, frontend/com_ahelp.c, frontend/commands.c, helpcmd suite*

HELP ALL LISTS EVERY COMMAND, ONE LINE EACH. Reported as a glitch: setscale's help was two literals joined by an embedded newline and an indent, so help all printed its second half as a stray indented line; deftype's text ended in a newline (a blank line mid-list), remzerovec and sysinfo had no separator, where no final period. Reading the whole listing found a larger defect: help all printed 137 commands where this build's table has 165, because com_help counted the table to the first entry without a HANDLER instead of to its terminator, the entry without a NAME; the control keywords (while, repeat, dowhile, foreach, if, else, end, break, continue, label, goto) have no handler, so nothing from while down was listed -- the eleven keywords and cdump,

mdump, mrdump, settype, strcmp, strstr, strslice, fopen, fread, fclose, linearize, cutout, devhelp, inventory, optran, wrnodev, check_ifparm -- while help with a name always found them; newhelp (com_ahelp) had the same loop; upstream ngspice's code. Both loops now run to the terminator (newhelp's also bounded by its array), and every help text in both tables has the one-line form name args : text. -- setscale joined with a comma, deftype's newline dropped, the separator added to remzerovec and sysinfo, mdump and mrdump spelled [outfile] : ..., leading and trailing spaces removed, final periods added. helpcmd gains fourteen checks (18 in all; 9 of 18 on the E-752 binary): the form of every line, the list sorted, the listed names exactly the table's names that help answers, the 28 and the eleven keywords present, help setscale on one line, help alone and an unknown name unchanged, newhelp at the advanced level; cornerscmd, exitcmd and lrmosdi unchanged, sweep 533 of 533. Unchanged: what every command does, the wording beyond the form, the grouping of the list.

E-754 *parser/inp2n.c, autoadapt suite*

AUTOADAPT'S NAME-ORDER NOTE UNDER NGDEBUG ONLY. Reported as a flood: E-729 announced the tie-break (the shared bus node at the same port index on both devices, the instance names deciding the forward side) in every mode, the quiet default included, and a generated deck whose channels are all N1 b a / N2 b c ties at every node, so six channels printed the same note six times. The note is now printed under set ngdebug or autoadapt=debug only; the choice, the .adapt b:n2 override, E-466's quiet default and the debug mode's split reports are unchanged. ngdebug must be set before the parse (a .spiceinit, or an interactive set before source); a set in the deck's own .control block runs after it, the rule the handbook states for autobus and autoadapt themselves. Measured: six notes in the quiet mode on E-753, none now; six under autoadapt=debug and under ngdebug. autoadapt 37 of 37 per solver (36 of 37 on the E-753 binary: the silence check; a .spiceinit route check added), adaptlisted, adaptmsg, adaptquiet, autobus, busmixed, autoopts, subbus, silentaccept, hunt12diag and autocorner unchanged, sweep 533 of 533. Unchanged: which device takes the forward side, the .control block's inability to reach the parse, the debug reports.

E-755 *include/ngspice/devdefs.h, osdi/osdiinit.c, osdi/osdiparam.c, parser/inpdpar.c, inputguard suite*

ONE WARNING FOR A COMPILED INSTANCE'S NEGATIVE MULTIPLIER. Options-and-convergence hunt F6 and workflows hunt F10: `nr1 a 0 nmod m=-1` printed the parser's built-in sentence ("the device's contribution is sign-inverted", E-426, true of a built-in that stamps the value) beside the OSDI setter's ("the value is ignored", E-529, which refuses it because the compiled noise factor is sqrt(m)), and the device ran at m=1 -- two warnings contradicting each other on one line, the first describing what does not happen. The device table gains a DEV_OSDI flag (devdefs.h, set by osdiinit.c on every compiled device), the parser's negative branch returns for a flagged device (its non-finite branch is unchanged for both), and the setter's warning names the instance and says what is done: "n1: multiplier m=-1 is negative; the value is ignored and the instance keeps the multiplier it had". One line on every route: the instance line, the _mfactor= spelling, a subcircuit called with m=-1 (naming n.x1.n1), a card's _mfactor=-1 default (one line per instance) and an alter of m at run time (the instance keeps m=2). A built-in keeps the parser's sentence and keeps inverting. inputguard 107 of 107 (102 of 107 on the E-754 binary), guardspell, lrmosdi, barevalue, instknobs, osdiparam, hunt12diag and guardpair unchanged, sweep 533 of 533.

E-756 *frontend/spiceif.c, spiceif.h, runcoms2.c, options.c, variable.c, inp.c, include/ngspice/ftedefs.h, fteext.h, sweeptemp and mcpolicy suites*

A SET TEMP SURVIVES RESET, AND UNSET GIVES THE DECK'S VALUE BACK. Plumbing hunt N1 and workflows hunt F14: `set temp=100` in a control block was honoured by the next op and dropped by every `reset`, the loop commands' internal ones included -- the variable still said 100 while the rebuilt circuit ran at the deck's temperature, so montecarlo, wcd and highsigma typed after a set temp sampled at the deck's and left the circuit there -- and `unset temp` re-applied the value being removed (40 stayed 40), printed a leftover "it's a US_SIMVAR!" and left the variable

set, or after a reset ran the next analysis at 0 C. cp_usrset records the simulator options the user sets onto the circuit; com_rset snapshots them before the circuit goes and re-sets them after the reload (through cp_vset, so a deck .temp card's reload does not win over the user's value); unset of an option rebuilds the default option block from scratch (application defaults, the deck's .options cards re-parsed by a helper factored out of the deck loader, its .temp, the other recorded variables) and cp_remvar removes the variable it found; any option is covered (temp, tnom, reltol measured). sweeptemp 29 of 29 (21 of 29 on E-755), mcpolicy 47 of 47 per solver (46 on E-755), autoopts, loopguard, aging, sensrestore, collapsestate, inputguard, autocorner, altermulti unchanged, sweep 533 of 533; unchanged: a set before any circuit is loaded still reaches none, on a source or a reset.

E-758 *osdi/osdiload.c, include/ngspice/osdiitf.h, frontend/com_chdir.c, lrmkernel suite*

THE COMPILED LOAD PATH NO LONGER READS THE WORKING DIRECTORY ON EVERY NEWTON ITERATION. F1 of the 2026-09-28 speed hunt: get_simparams filled the simparam table on every load call and the ”cwd” entry E-527 added was refreshed through getcwd() each time, which on macOS opens and stats the directory chain -- ~12 us per Newton iteration for any deck with a compiled model, independent of the instance and model count (one compiled resistor 12.08 us, a hundred 13.6 us, one built-in resistor 0.06 us), paid whether or not a model ever asks for cwd; a 1.2 M-iteration single-instance transient spent 12.8 of 14.4 s there, a 10 Gb/s PRBS through a compiled RC took 3.07 s where the built-in RC took 0.14 s. osdi_cwd() caches the string and OSDIcwdChanged(), called by the cd command (the only chdir in the tree), marks it stale; the model-visible answer is unchanged and still follows a cd between runs. Now 0.63 us per iteration for one instance, the PRBS run 0.28 s, a 500 MHz switching run 0.15 s (was 1.06 s). lrmkernel gains a cwd-strobing model and three checks (the first op reports the suite directory, an op after cd reports the new one, one instance loads under 6 us per iteration; 48 per solver, 47 of 48 on E-757), rangeguard 75 of 75, osdilimit, hunt12diag, instknobs unchanged, sweep 534 of 534; unchanged: the per-instance cost and the other per-iteration hooks (hunt O1, O2).

E-759 *osdi/osdiaccept.c, osdi/osdiload.c, osdi/osdidefs.h, osdi/osditrunc.c, spicelib/analysis/dctran.c, include/ngspice/cktdefs.h; OpenVAF hir_lower/src/stmt.rs, hir_lower/src/lib.rs, mir/src/serialize.rs; evtedge suite (section 9), simctrl check 6, constfold, hunt13slips check 9*

A CROSS/ABOVE EVENT LANDS THE TIMESTEP ON ITS CROSSING. F2 of the 2026-09-28 speed hunt: the crossing was resolved to the step grid -- a compiled switch on a 1 us ramp closed 31.2 ns late at a 100 ns step, the same as a plain `if` and as the built-in `sw`, `time_tol` was ignored, and a 500 MHz drive counted 1001/1001/2002 for 1000/1000/2000 crossings (the crossing's step was rejected by the truncation error the switching brought and E-587's evaluation edge saw the walk back). ngspice now counts accepted points (OSDIaccept) and serves the count as the `$osdi$point` simparam, so a compiled event tells a new point, a retry and the iterates of one attempt apart; each event keeps the sample at the last accepted point and, when the crossing lies between it and the current iterate, asks to land at the interpolated offset plus a margin (`time_tol` when given, else a thousandth of the step) by writing -(2 + t) into the bound-step slot; OSDItrunc makes that the attempt's step -- replacing the truncation estimate, which at a crossing measures the switching, not the smooth interval -- and forces the rejection (`CKTforceReject`, honoured past dctran's 0.9 rule); the landing attempt is accepted once converged (`CKTlanding`), its next step floored at an eighth, and a forced rejection stays out of E-128's order bucket. The compiled rule: a retry's first iterate (ngspice starts it from the rejected solution) neither fires nor asks and the next iterate is compared against the accepted sample, so the edge reappears at the landing where the body runs with the crossing's `$abstime`; an iterate that asks does not fire; a landing may refine once when it still overshoots by more than the margin its request used (a sine over a 60 ns step: 0.2 ns, then 0.04 ns); a module's attempt flags stop requests after any body fired in the attempt (a reset dropping the expression through a second threshold at the same instant, the idtassert oscillator) and stop later bodies once any event asked; a declined request (E-504's floor) fires at the next point; a landing that fell short is not chased. `$discontinuity(0)` in the body now restarts the integration after the event (E-55's eighth is the step after the landing) instead of redoing the approach. Mea-

sured: the switch closes 0.04 ns late (0.001 ns with `time_tol = 1p`), the bodies' `$abstime` and a `$strobe` in the body read the crossing time, the 500 MHz counts are exact with the same work (39 005 points, 4000 rejected, 88 008 iterations against 41 008, 4002, 84 016). Limits: a body's effect in a rejected attempt persists (E-678), two instances crossing in one step where one needs no landing can run the other's body twice, events inside conditionals degrade to E-587's rule, an ngspice without `$osdi$point` gets no requests. Found on the way: --dump-json wrote a non-finite float constant as a bare `inf` (every event module now has one, the bound-step slot's initial value) and the file did not parse; the serializer writes the string "inf". evtedge 24 of 24 per solver (18 of 24 on the E-757 toolchain), simctrl check 6 rewritten, hunt13slips 9 of 9, 23 event-related suites pass, sweep 534 of 534.

E-760 *osdi/osdicallbacks.c, osdi/osdiload.c, frontend/variable.c, include/ngspice/cpextern.h; OpenVAF osdi/stdlib.c (the compiled runtime); lrmkernel suite (section 7, optprobe.va)*

THE COMPILED LOAD PATH'S FIXED PER-ITERATION BOOKKEEPING IS HOISTED. O1 of the 2026-09-28 speed hunt, the remainder of E-758: one compiled instance still cost about 0.6 us of load per Newton iteration against 0.08 us for a built-in resistor, none of it model evaluation -- the repeated-message summary (E-543's F4) walked its 64-slot ring and freed two NULL pointers per slot on every iteration (about 60 % of the load samples), `noosdilim`, `osdilim_verbose` and `scale` were looked up by name on every load call (three cp_getvar walks with their cp_usrvar probe, about 25 %), the simulator version was re-parsed from its string, and the compiled runtime's `osdi_io_iter_begin` walked its 64 file slots with none open. Now the ring keeps a live-slot count and the summary returns at once when nothing is held; `variable.c` offers `cp_var_state_refresh()` (a generation counter advanced by every set and unset, plus the current circuit and plot and the heads of the four lists cp_getvar consults) and the load path re-reads its options only when the stamp moves, which never happens inside an analysis; the version is parsed once; the runtime's hook returns early when no stream is readable and nothing is deferred (`osdi_readable_any`, set at the four open paths and recomputed by the accepted-point flush; an older .osdi keeps its old hook). Measured: the one-instance 1.2 M-iteration transient's load 0.767 s -> 0.136 s (0.63 -> 0.11 us per iteration; total 2.12 -> 1.32 s against 1.20 s built in), the E-752 PRBS run 0.28 -> 0.155 s, the F2 hunt's 500 MHz switching run 0.13 -> 0.073 s. Nothing observable changes: a `set` or `.option` still reaches the next run, the summary still prints, file reads still replay across a rejected iteration. lrmkernel gains section [7] with optprobe.va (set scale between runs, .option scale, set osdilim_verbose between runs, the load ratio against a built-in resistor under 4x -- was 5x), 53 of 53 per solver (52 of 53 on the E-759 toolchain); osdilimit, display, rtdomain and the file suites unchanged; sweep 534 of 534. The same profile put 48 % of the transient, compiled or built in, in OUTpData's per-point free-memory query and clock() -- the hunt page's F3.

E-761 *frontend/outitf.c; progressbar suite (section 6), loopbar suite (its sweep deck)*

THE OUTPUT PATH NO LONGER QUERIES THE FREE MEMORY AND THE CLOCK ON EVERY ACCEPTED POINT. F3 of the 2026-09-28 speed hunt, found in the profile taken for O1: OUTpData ran getAvailableMemorySize() (a Mach host_statistics query plus a port trap on macOS), clock() (a getrusage system call, the quarter-second throttle of the "Reference value" line) and cp_getvar("no_mem_check") (a lookup by name) on every accepted point -- about a microsecond a point, 48 % of a 600 000-point transient, compiled or built in, hidden behind device work in large circuits exactly as F1's getcwd was. OUTpD_memory() now checks the free memory only when the run's output vectors are at a chunk boundary and about to grow (AddRealValueToVector extends by vlength2delta when v_length reaches v_alloc_length: a few times per run; the first point and every growth still check before the chunk is claimed), reads no_mem_check behind E-760's variable state stamp, and outp_ref_due() replaces the five clock() throttles with one wall-clock read through seconds() (a commpage clock_gettime, some 30 ns; the right clock for a display cadence anyway). Measured: the one-instance built-in transient 1.20 -> 0.54 s (its compiled twin 1.32 -> 0.65 s), the E-752 PRBS run 0.156 -> 0.087 s, the F2 500 MHz switching run 0.074 -> 0.042 s, the 300-stage RC ladder (2033 points) unchanged; the output path fell from 48 % of the samples

to 3 %. progressbar gains section [6] (the time outside load, factor, solve and trunc under 3x their sum and under 1.5 us per point -- was 4.6x and 1.8 us -- and the progress line drawn 2 to 12 times over a run of a second), 25 of 25 (23 of 25 on the E-760 binary); loopbar's sweep runs a 1.2 M-point inner transient so a point still spans the two refreshes its check [5] needs, 21 of 21; lifecycle, the raw-file and save suites, benchmark and abstolperf unchanged; sweep 534 of 534. Next down in the profile, noted as O7: ngspice's own statistics timers, six seconds() reads per Newton iteration, about 80 ns an iteration, a third of what is left of this trivial circuit's transient and nothing for a real one.

E-762 *frontend/com_optimize.c; optimize suite (section 19, a first README), reuseloops suite (its regex)*

OPTIMIZE SAYS WHY IT STOPPED. F1 of the 2026-09-29 optimize hunt: the report's last line read `optimize: converged, ...` whatever ended the search -- the iteration cap (Nelder-Mead after 5 evaluations at -maxiter 1, Levenberg-Marquardt after 4 with a sum-sq residual of 0.13, particle swarm after 11), a run in which no evaluation solved (objective 1e30), one whose objective never moved, a start inside a region the model refuses -- and only an interrupt (E-537) was told apart; E-499's and E-438's NOTEs followed and softened it, but the word was never qualified and nothing was published for a .control script to branch on (the knob values were, E-501; the outcome was not). Each method now records its stop where it breaks (OPT_ST_CONVERGED on its own criterion -- LM's 'no step lowers the cost' is a minimum to within the finite-difference accuracy and counts, OPT_ST_MAXITER when the loop runs out, OPT_ST_COMPLETED for the two whose stop is a schedule: annealing's cooling levels and NSGA-II's generations, OPT_ST_INTERRUPTED), the epilogue adds two verdicts (a best cost still at the penalty is NO SOLUTION, an objective equal at every evaluation is UNCHANGED, the interrupt outranks both), the line opens with the phrase for the scalar, least-squares and centering reports alike -- `converged` unchanged byte for byte, `stopped at -maxiter (N iterations) -- NOT converged` followed by a NOTE to raise -maxiter or loosen -tol, `cooling schedule complete (N levels)`, `NO SOLUTION -- no evaluation solved`, `unchanged -- nothing was optimised` -- and the outcome is published: `optimize_status` (a string variable: converged, maxiter, completed, nosolve, unchanged, interrupted), `optimize_converged` (1 on its own criterion or a completed schedule), `optimize_evals` and `optimize_cost` as vectors and variables; NSGA-II publishes the status and the count. optimize gains section [19] (the three methods at -maxiter 1, a converged line unchanged with the published numbers equal to it, NO SOLUTION, unchanged, the annealer's schedule), 54 of 54 (44 of 54 on the E-761 binary); reuseloops' regex accepts any stop phrase, 17 of 17 per solver; pareto and dcenter unchanged; sweep 534 of 534.

E-763 *frontend/com_optimize.c; optimize suite (section 20), loopguard check 11, reuseloops deck*

OPTIMIZE REFUSES WHAT IT CANNOT MEAN. F2 and F3 of the 2026-09-29 optimize hunt: E-499 gave the numeric options a strict parser and left the rest on a lenient one -- `-param R2 1k abc 10k` took abc as 0 and searched [0, 10k], an init outside [lo, hi] was clamped to a bound, a -target weight of 0 or -1 was accepted (0 fitted nothing and E-499's NOTE said so afterwards), `-target v(out) - v(in) 0.4` took - as a value of 0 and RAN the fit to it, a bare -method ran the default, `-maxiter5` was skipped and the run went on with the default cap, a second -minimize under a scalar method was dropped, a knob named twice was altered twice, a -swarmsize below a method's minimum was raised without a word, and `-minimize -v(out)` (the natural way to maximise) printed the usage line and, in batch mode, ngspice's 'incomplete or empty netlist'. Every number the command takes now goes through opt_strictnum (a SPICE suffix and a unit letter parse; text and trailing junk refuse the command naming the knob and the token), an init must lie in its box, a weight must be positive, a bare option flag, an unrecognised token, several objectives under a scalar method and a duplicate knob refuse the command with a message naming the cause, a raised population prints a NOTE, and an objective that begins with '-' gets the quoting hint -- the quoted form `-minimize "-v(out)"` already ran (the lexer keeps the quotes, the collector strips them), as does 0-v(out); the lenient optnum() is gone. The strict parser found two cases in the suites' own decks: reuseloops spelled -samples as -nsamples and had run its centering

check with the default 100 samples instead of 6 (corrected), and loopguard used a zero weight to provoke E-499's NOTE (it expects the refusal now). optimize gains section [20] (the ten refusals with no run following, suffixes and a negative target still parsing, the raised-population NOTE, the bare and quoted negated objective), 69 of 69 (47 of 69 on the E-761 binary); loopguard 49 of 49 and reuseloops 17 of 17 per solver; pareto, dcenter, agestate, autocorner, deopt, failacct, hierdev, mcpolicy, loopbar, opt100, psoopt, reusestate, sweepanalysis and saopt unchanged; sweep 534 of 534. Limits: a unit letter after a number is SPICE's and stays accepted (10o = 10), and an expression beginning with '-' still has to be quoted.

E-764 *frontend/com_optimize.c; a new optmethods suite (41 checks, a README), optimize suite (one expected message)*

CMA-ES, -POLISH AND -STARTS. Section 1 of the 2026-09-29 proposal on more optimizers (with its section 6): `-method cmaes` is Hansen's (mu/mu_w, lambda)-CMA-ES over the unit cube with the published constants -- lambda = -swarmsize (default 4 + floor(3 ln n), raised to 4 with a NOTE below it), mu = lambda/2, sigma_0 = 0.3, candidate 0 the start point, Gaussian draws on E-194's -seed stream, a cyclic Jacobi eigendecomposition per generation -- chosen first because it is rank-based (a failed evaluation is the worst rank: from the hunt's F6 start the model refuses it reaches g = 1m where Nelder-Mead and LM read NO SOLUTION, and a box refused entirely ends after three all-failed generations as NO SOLUTION), learns the valley's scale and orientation (a rotated ellipsoid of condition 1e6 to 0 after 1027 evaluations where the swarm 'converged' at 0.27, Rosenbrock in four knobs to 6e-18, O2's nine-decade box to rms 8e-8 at -tol 1e-10 against LM's 2.5e-5) and does not clamp onto a bound (O1's deck: R2 = 250 where the six-particle swarm sat at 10); stops on sigma times the longest axis below -tol or the published TolFun rule (the swarm's stall test on the all-time best fired while the distribution was still learning and was rejected), -verbose one line per generation; `-polish` runs the local method (LM for a -target fit, Nelder-Mead otherwise, a half-size first simplex) from a global method's best point and reports both costs -- O1's exact command with it: 0.036 on the bound to 1.3e-12 at 250, the bound NOTE gone; `-starts k` runs the method from the given point and k Latin-hypercube points, each with its share of -maxiter (a seed per start for the population methods, the population doubled per start for CMA-ES -- the IPOP rule), prints each start's verdict, names the winner, polishes it and publishes optimize_start (a double well: Nelder-Mead from 0.15 in the wrong basin at 6.8e-3, -starts 4 at 0.8); -polish under a local method is a NOTE and ignored, -polish/-starts under nsga2, a bare -starts, -starts 0 and -starts abc are refused; the six methods are byte for byte unchanged; optmethods 41 of 41 (5 of 41 on the E-763 binary), optimize 69 of 69, thirteen optimizer suites unchanged, sweep 535 of 535; the write-up, the change-log row and PDF, the catalog chip and counts (759), the README counter, the proposal's section markers, the hunt page's O1 and O2 notes, update notes in E-130, E-194, E-195, E-196, E-762 and E-763, the handbook subsection and the suite README; limits: -tol is a length in the cube for CMA-ES (the default on nine decades is a kilo-ohm; the log knob of the proposal's section 7 is the fix), a polish costs a few dozen evaluations when the global best is already at the optimum, and neither flag applies to NSGA-II

E-765 *frontend/com_optimize.c; optmethods suite (sections 11-19), optimize suite (one expected message)*

BAYESIAN OPTIMIZATION. Section 2 of the 2026-09-29 proposal on more optimizers: `-method bayes` fits a Gaussian-process surrogate (Matern 5/2, a length scale per knob between a twentieth of the box and a hundred boxes, a signal variance, a noise floor; the costs log-transformed when positive and spanning two decades, standardised over the solved points, a failed evaluation imputed at the worst solved value plus three sigma; Cholesky, the marginal likelihood maximised by a simplex on a callback) to every evaluation and chooses the next by expected improvement over 400 + 100n Latin-hypercube candidates and perturbations of the best, the three best refined, a point never evaluated twice; an initial design of 2n + 2 LHS points with the start among them, redrawn up to three times when nothing solved (then NO SOLUTION); -maxiter is the evaluation budget and the report says `stopped at -maxiter (N evaluations)`, -tol the smallest expected improvement worth an evaluation, -swarmsize a NOTE, -verbose one line per evaluation, the in-

terrupt polled before each; the hand-off: a Gaussian process locates the basin in ten to twenty evaluations and cannot resolve a cost that falls by orders of magnitude at the optimum, so when its criterion is met with budget left (expected improvement below -tol, or below a thousandth of the cost spread twice running past the design, or the best unmoved for 10 + 2n evaluations with little expected) the rest of the budget goes to E-764's polish -- LM for a -target fit, Nelder-Mead with a 0.02 simplex otherwise, capped at the iterations that remain -- and the polish's status is final; the divider fit at a budget of 25 hands off after 8 to 14 evaluations for seeds 1 to 5 and every seed ends at R2 = 9000 in 24 to 33 evaluations, the bowl's best is below 1e-5 after about twenty and the polish converges below 1e-12 in 91 to 97 against the simplex's 103 and the swarm's 1153, a budget of 30 ends NOT converged at the polish's share with the NOTE (the budget honoured); the surrogate line reports the predicted cost at the optimum with a one-sigma range, the transform, and a length scale per knob with `(no dependence seen)` past ten boxes -- E-762's unchanged case with a diagnosis; the seven other methods byte for byte unchanged; optmethods 61 of 61 (6 of 61 on E-763), optimize 69 of 69, thirteen optimizer suites unchanged, sweep 535 of 535; the write-up, the change-log row and PDF, the catalog chip and counts (760), the README counter, the proposal's section marker, the hunt page's F6 note, update notes in E-130, E-143, E-762, E-763 and E-764, the handbook subsection and the two suite READMEs; limits: the surrogate's resolution is a twentieth of the box and the digits are the local method's (a budget too small for the hand-off ends NOT converged), the polish's cap is in the local method's iterations so a run can report a few more evaluations than the budget, an ill-conditioned rotated valley and Rosenbrock are not its class, `(no dependence seen)` is the surrogate's finding and can be wrong where it fits poorly, -maxiter is capped at 2000

E-766 *frontend/com_optimize.c; optmethods suite (sections 20-27)*

CONSTRAINTS FOR OPTIMIZE. Section 3 of the 2026-09-29 proposal on more optimizers: `-constrain <expr> -max <hi>` and/or `-min <lo>` (up to 32, the expression one token like a -target's, the bounds strict finite numbers, a constraint belonging to the -analysis stage it follows, `-ctol` the feasibility tolerance at 1e-4 relative to each bound's own magnitude; -min keeps its -minimize meaning except right after a -constrain or a -spec) turn a fit into a design problem, solved by an augmented Lagrangian around whichever scalar method was chosen: each side is g <= 0 in units of its bound, opt_eval adds (rho/2) max(0, g + lambda/rho)^2 per side to the cost and gives LM the same as a residual after the targets, al_solve() minimises that with the method, reads the objective and the constraint values at the inner optimum, updates lambda <- max(0, lambda + rho g), raises rho tenfold when the violation did not fall by four (a hundredfold when it did not move), and stops feasible within -ctol or after ten rounds as INFEASIBLE, one line per round; the penalty starts balanced from one evaluation at the start point (2

E-767 *frontend/com_optimize.c; optimize suite (section 21)*

THE -DPARAM OPTIMUM IS THE DECK'S ON THE FAST PATH TOO. F7 of the 2026-09-29 optimize hunt: on a deck of 80 weighted devices or more E-322's fast path pushes every -dparam value in place and never touches the stored deck, and the final leave-at-the-optimum apply went the same way, so the fit reported rr = 9000, published optimize_rr, and the next reset -- the user's, or any command's internal one -- put the circuit back at the initial value in silence (a chain of 100 resistors fitted to v(out) = 0.9 read 0.5 after `reset`, and `listing param` showed rr = 1000), while a two-resistor deck, fitted through alterparam, kept its optimum: which of the two a user got depended on a device count they have no reason to know about; frontend/com_optimize.c: after the final apply, when the fast path was armed, one `alterparam <name>=<optimum>` per -dparam knob (no reset: the circuit holds the value, and the deck now agrees); the evaluations are untouched (the 100-resistor fit still takes 16) and the small-deck path is what it was; the optimize suite gains section [21] (the chain arming the fast path and converging in 16 evaluations, the output 0.9 after the fit and still 0.9 after a reset, the listing at 9000 equal to optimize_rr, the two-resistor deck as before), 73 of 73 (71 of 73 on the E-766 binary); optmethods, reuseloops, loopguard, opt100, paramfastsweep, mcfastpath, agestate, mcpolicy, dcenter and pareto unchanged; sweep

535 of 535; the write-up, the change-log row and PDF, the catalog chip and counts (762), the README counter, the hunt page's F7 note, update notes in E-144 and E-322, a handbook sentence and the suite README; limits: the -dparam kind only (a -param or -mparam optimum is still dropped by a user reset, the hunt's O3), NSGA-II leaves no single optimum to write, and the sweep command's own fast path is outside this change

E-768 *frontend/com_optimize.c; optmethods suite (sections 28-33, eight earlier expectations), optimize suite (one expected message)*

A TRUST REGION ON A QUADRATIC MODEL. Section 4 of the 2026-09-29 proposal on more optimizers: `-method tr` (aliases trust, trustregion, quad, quadratic) keeps 2n + 1 points, interpolates them with a quadratic whose Hessian changes least from one iteration to the next (the KKT system of the least-change update solved directly in coordinates scaled by the radius; a singular or non-interpolating solve rebuilds the axis set), and minimises the model inside a box of radius delta around the best point INTERSECTED with the knob bounds -- a bound-constrained quadratic program (a Newton step on the free coordinates with the active set grown one hit at a time, polished and cross-checked by exact coordinate minimisation), so a step runs along a wall instead of being clamped onto it; one evaluation per iteration, the ratio of actual to predicted decrease accepting the trial point and moving the radius, the trial point replacing the farthest one; converged when the model expects less than -tol of decrease with every point within two radii and the last trial step having delivered what it predicted, or after the radius has come down to -tol (tenfold drops with geometry steps, or rejected steps on a noisy objective); a failed evaluation is refused and the radius halved, a start that does not solve looks at its axis points; measured against Nelder-Mead, evaluations to converged: a bowl 9 against 103, a rotated ellipsoid of condition 1e6 33 against 134, Rosenbrock in four knobs 132 against 377, a coupled quadratic in ten knobs 146 against 996, an optimum on a wall 10 against 62 with that knob exactly on the bound, hunt O1's deck 13 against 47; E-764's -polish and E-765's hand-off run it for a scalar objective where they ran the simplex (LM stays the finisher for a -target fit, Nelder-Mead the -minimize default, byte for byte): O1's swarm is polished to 1.7e-14 in 11 evaluations instead of 38, the surrogate's hand-off on the bowl converges in 26 evaluations in all instead of 91 and inside a budget of 30 where it ended NOT converged, its hand-off on the ellipsoid reaches 4.8e-14 where the simplex sat flat in a box corner at 0.18, and the constrained divider ends at R1 = 1111.11 with the multiplier 1e-4 exactly in 107 evaluations instead of 179; the method lists name tr and -polish under it is a NOTE; the optmethods suite gains sections [28]-[33] and eight earlier checks expect the trust region in the polish lines, 97 of 97 (74 of 97 on E-766); optimize 73 of 73; sixteen related suites unchanged; sweep 535 of 535; the write-up, the change-log row and PDF, the catalog chip and counts (763), the README counter, the proposal's section 4 marker, update notes in E-130, E-762, E-763, E-764, E-765 and E-766, the handbook subsection and the suite READMEs; limits: a quadratic model wants a smooth objective (at the noise floor the radius collapses and the search stops; stepped objectives are pattern search's, section 5), the first model's 2n + 1 evaluations are outside a polish's iteration cap, the last digits follow -tol in cost rather than the simplex's shrink to the last bit, a start with no solving axis point still reads NO SOLUTION, and Nelder-Mead remains the -minimize default

E-770 *main.c (background input); frontend/runcoms.c, runcoms.h, runcoms2.c (ft_drop_empty_rawfile); spicelib/devices/dev.c (osdi_stage_reload_copy); frontend/inpcom.c (savestate keeps its case); frontend/inp.c, include/ngspice/ftedefs.h, frontend/spiceif.c (ci_serial, phys_seen); suites constguard, guardgaps, mcyield, reusesetup, pyplot, reduce, distoexact, argguard*

WHAT THE FIRST LINUX CI SWEEP FOUND IN NGSPICE. The sweep had only run on a Mac: no X, a terminal, a case-insensitive volume. On the CI machines (X builds, no display, no terminal, Linux case-sensitive and root in a container) five ngspice faults showed. An X build with no `DISPLAY` and no controlling terminal (input from a pipe: CI, cron, `ssh -T`) is in the background from its first line, and the read loop called `X11_Input` on the NULL display: every such run segfaulted before its first command (crashfix x3, exitcmd, pyplot, fftnorm, ...); the branch is now guarded by `dispdev->Input == X11_Input`, as the readline hook was. A run that writes nothing to its

rawfile unlinked it whatever it was: `-r /dev/null` deleted /dev/null as root and the linux-intel sweep died at suite 152; only a regular file is removed now. `pre_osdi -f`'s staged copy was named from the time in seconds and a per-process counter, so two processes reloading in the same second shared one file (osdireload beside carddefault failed 4 runs of 4); the name carries the pid. `saves-tate` was missing from the deck reader's case-keeping commands, so its path was lowercased (a missing directory on Linux, checkpoint 0 of 22). warn_physics' say-it-once memo reset whenever the CKTcircuit address changed, so montecarlo, which rebuilds per sample, printed a warning 2, 3 or 4 times in identical runs; each front-end circuit has a never-reused serial that a rebuild from its kept deck (mc_source, reset) keeps, and the memo keys on it: once, every time. Suites: a NaN compared as `nan` (glibc prints -nan on x86); pyplot uses `setcs` for a path (a deck lowercases a quoted `set`); reduce compiles rf_blocks.va itself instead of copying another suite's ignored build product; distoexact's HD3 bound is HB's noise floor (2e-4: arm64 4.0e-6, x86-64 4.9e-5); argguard's overflow check prints what it searched. Verified under Linux's conditions on macOS (an X build, no DISPLAY, no terminal, a case-sensitive volume under a path with capitals). Left for the next Linux run: argguard's overflow check and collapsestate's Sparse sens on Linux.

E-772 *maths/misc/randnumb.c (ng_srand, ng_rand), include/ngspice/randnumb.h, main.c, sharedspice.c, tclspice.c, maths/cmaths/cmath2.c (rnd); include/ngspice/ngspice.h (HUGE); spicelib/devices/dev.c (a reload that changed nothing); suites osdireload, lifecycle, pyplot, optpath, vafice, progressbar, lrmkernel, abstolperf; run_regression.py (serial set); .github/workflows/build-binaries.yml (Linux memory log)*

ONE RANDOM STREAM AND ONE HUGE ON EVERY PLATFORM. The second CI run with the sweep (linux-intel 519 of 536, macOS 534, Windows 474) showed two platform splits in ngspice itself. The generator's eight start states came from the C library's `rand()` after `srand(seed)`, and glibc (TYPE_3 additive), macOS (Park-Miller) and Windows (an LCG, RAND_MAX 32767) differ: one seed drew three different Monte Carlo runs, mcyield's 30 of 40 was 28 on Linux and 26 on Windows. `ng_rand()` is macOS's `rand()` in fixed-width integers (13 million draws matched against the system's), used by the seeding, the start-up seed and the control language's `rnd()`: macOS's streams unchanged, Linux and Windows draw the same. `HUGE` is PTeval's out-of-range flag and db(0)'s floor; macOS defines it as MAXFLOAT, glibc dropped it in 2.27 and the fallback made it infinity, so on Linux `1e300*1e300` in a B-source was refused and stopped the analysis where macOS evaluated it and warned (E-440); it is FLT_MAX everywhere. A forced reload the system loader answers with the object it already has reported "reloaded (0 devices)" and kept the old model; it is an error now. The Linux reload failures were the suites' `cp` over the mapped library (Linux shares its pages with the file); they replace it by rename, as a compiler does, in two `shell` lines (ngspice splits &&). Suites: pyplot runs its spaced interpreter through a script (a venv interpreter linked elsewhere loses its packages); optpath and vafice skip the unwritable-directory checks under root (the Linux container); progressbar's absolute us/point is reported, not asserted, on CI and its frame bound follows the run's length; lrmkernel keeps the ratio and drops an absolute cap; abstolperf's bound is 3.0 and it and progressbar run alone. CI: the Linux sweep logs memory every 10 s (the linux-arm runner was shut down two minutes in, twice).

E-773 *examples/run_regression.py (run_one, _group_procs, _memory_report, MEM_CAP_MB, the serial set)*

A SUITE THAT RUNS AWAY WITH THE MEMORY IS STOPPED AND REPORTED. The linux-arm CI job died at the same point of three runs, exit 137 and "the runner has received a shutdown signal", its log silent; E-772's memory log showed three suite scripts (analogloop, arraynodeprint, autocorner) growing by ~70 MB/s each -- children writing without end into captured pipes -- until the 16 GB runner went from 14.4 GB available to 250 MB and died, taking the sweep and its log. On Linux each suite now runs in its own session; once a second the sweep sums its process group's resident memory from /proc, and past 4096 MB (NG_SUITE_MEM_MB) stops the group, reports MEMORY and goes on, writing to _failures/.log each process's memory, working directory and command line, the deck each ngspice was running (copied beside it) and up to 16 kB of its output

read from its own pipe. A watchdog fault turns it off for that suite only; output and errors are one stream. Not the cause, which is still open (an ngspice built with -funsigned-char, aarch64 Linux's char, passes analogloop). plotname's flatness ratio (1.74 against 1.6 beside two suites on the 3-core macOS runner) runs alone.

E-774 *spicelib/devices/vsrc/vsrcask.c, isrc/isrcask.c (prbs, pam4); frontend/device.c (printvals, printvals_old); examples/_setup.py (PLATFORM_EXCLUDE), examples/run_regression.py; prbs suite [11]*

A SOURCE'S PRBS AND PAM4 ROWS WERE ERRORS WITH AN UNSET ROW COUNT. With E-773's watchdog the linux-arm sweep ran to its end: 28 suites stopped for memory, one timed out, the reports naming decks as small as a 1 kOhm resistor and a DC source under .op, and one catching the output: `<<NAN, error = 7>>` without end. E-752 added prbs and pam4 to the sources' parameter tables but VSRCask and ISRCask had no case for them (E_BADPARM), and printvals_old printed the error and returned a row count read from the IFvalue the failed query had not filled; its caller prints rows until that count runs out -- three on macOS (six error rows in every source table, unremarked), without end on aarch64 Linux at ~70 MB/s until the 16 GB runner had nothing left. printvals (`show`) read the unset vector outright. The sources answer both keywords, told apart by the register's level count, only the active one with its description (the E-447 rule); both printers zero the value first and make a failed query one row. examples/_setup.py PLATFORM_EXCLUDE holds a platform's known gaps out of the routine sweep by name with the reason (keyed by system and machine, normalised to aarch64/x86_64; --all runs them): on x86-64 arrayscale, cubic_table and table_model (huge-module code generation) and collapsestate (Sparse sens). prbs [11] (44 per solver): the .op tables carry no error row, `show` answers each keyword; the E-769 binaries fail both.

E-775 *frontend/com_dl.c, frontend/com_presnp.c (the compile command), frontend/plotting/pyplot.c (PYPLOT_DEFAULT_PYTHON), frontend/com_pyplot.c (the output path), spicelib/devices/dev.c (load_osdi, osdi_sweep_stale_copies) -- Windows branches*

NGSPICE ON WINDOWS. From the Windows CI sweep (490 of 532). A compile ngspice starts (`pre_osdi -va`, the osdicache rebuild, `pre_snp`) ran `system("\"openvaf-r\" \"src\" -o \"out\"")`, and cmd.exe strips the first and last quote of a line that starts with one and holds more: exit 1 every time; the line is wrapped in one more pair, as pyplot's launch has been since E-547. pyplot_python defaulted to `python3`, on Windows another installation or the Store stub (no numpy on the runner: five pyplot suites and mcrecord's render); it is `python` on Windows. pyplot's output path beside the deck was built with backslashes that the plot command's argument handling took as escapes (`C:\Users` became `C: sers`); `/` on Windows. Windows locks a loaded DLL, so a loaded model could be neither recompiled nor replaced and E-229's recompile-and-reload loop could not start; on Windows every load is of a staged copy in TMPDIR, and each session removes the copies earlier ones left. Compiled for Windows only; the Windows CI job is their first build.

E-777 *examples/_setup.py (NG_TRACE_DIR, exported_symbols, PLATFORM_EXCLUDE Windows entries), examples/run_regression.py (NG_TRACE_FAILED), .github/workflows/build-binaries.yml; suites constguard, osdidist, osdimc, paramgiven, savemc, writemc, cornerscmd, limguard, multimod, lrmdisc, numguard, winlink*

THE SWEEP TRACES WHAT THE SIMULATOR PRINTED, AND THE SUITES STOP ASSUMING POSIX. NG_TRACE_DIR makes _setup.py record every ngspice and openvaf-r run a suite makes (command, directory, deck, status, output tail; with NG_TRACE_LLDB on macOS an lldb backtrace of a run killed by a signal); NG_TRACE_FAILED makes run_regression.py re-run each failed suite once with it (at most 30), and both CI sweep steps set it and upload examples/_trace/ . Suites: exported_symbols() reads a DLL's export table in Python where Windows has no nm (osdidist, osdimc, paramgiven); constguard accepts `-nan(ind)`; savemc, writemc and cornerscmd close the workbook they open (Windows will not delete an open file); limguard reuses a directory a held file kept; multimod's emptied-PATH linker check and lrmdisc's dlopen harness stand

aside on Windows; numguard reports, not asserts, MinGW's large-argument sin/cos (1e-9 off at 1e12, wrong at 1e20); winlink accepts the x64 runtime's _fltused; PLATFORM_EXCLUDE holds helpcmd, syntaxhl and vafcolor out on Windows (a POSIX pseudo-terminal).

E-779 *frontend/parser/lexical.c (backslash in quotes on Windows); examples/_setup.py (cdb trace); suites pyplot, reusecache, vafice, hbconv, savemc, optpath, fgetc, ungetc, scanbody, lrmio*

A QUOTED WINDOWS PATH KEEPS ITS BACKSLASHES; A WINDOWS CRASH IS TRACED. The deck's command lexer drops a backslash; upstream already keeps it outside quotes on Windows (DIR_TERM), not inside, so `pre_osdi "C:\...\dir with space\va res.osdi"` reached the loader without separators (osdireload); inside quotes it is kept on Windows too. E-777's trace re-runs a run that died with an NTSTATUS code under cdb for the faulting stack -- every transition/slew/absdelay/crossing-event model crashes ngspice at once on Windows (0xC0000005: transedge, evtedge, rtdomain, opargs, defaulttransition, lrmops, portconnected, domainwarn), its tables exported and plain uint32_t, so the stack is what the next run brings. Suites: pyplot E-547c/d/f/g with a .bat interpreter, cmd.exe's status, `python -B`, '/' paths; reusecache's compiler copies carry .exe; vafice sets TMP/TEMP as well as TMPDIR; hbconv [3] accepts a clean convergence of the 'failure' deck (it converges on Windows) while a stall reported as a result still fails; savemc compares paths with separators normalised and [16] (rm -r of a directory holding an open file) stands aside on Windows; optpath's unwritable-root checks too; fgetc, ungetc, scanbody, lrmio write their inputs with LF.

E-781 *osdi/osdidefs.h (osdi_extra_data_off), osdi/osdiinit.c (instance block size), osdi/osdiregistry.c (osdi_extra_instance_data)*

THE PER-INSTANCE OSDI STATE IS ALIGNED. ngspice's OsdiExtraInstData (limiter history, absdelay/transition/slew slots, crossing times) is declared aligned to max_align_t -- 16 bytes on x86-64 -- but sat straight after the compiled model's instance data, whose size is only a multiple of 8, so it was 8 bytes off whenever that size was 8 mod 16. On macOS Intel all 43 crashing runs of benchmark, binstale, osdilimit, paramgiven, paramsetoverload, savecur and vafautodiff stopped at clang's aligned movapd store into lim_old in OSDIload (EXC_I386_GPFLT); an x86-64 -fsanitize=alignment build shows the same misalignment under exactly the transition/slew/event models the Windows job lost (tedge 130 reports, slewdemo 100, rtdom 103, dt_bare 103) and none under the ones it ran (tramp, constant slew rates). The offset is rounded up to MAX_ALIGN in one helper the block's size and its accessor share: 0 reports after. Apple Silicon's max_align_t is 8, so the arm64 layout is unchanged. Full sweep 536/536.

E-783 *examples/run_regression.py (_WinJob, _ps_group_procs, _win_procs, _timeout_report, NG_SWEEP_BUDGET_S, the trace phase), examples/_setup.py (a run announced before it starts), .github/workflows/build-binaries.yml (smoke step, sweep budget)*

A HUNG OR RUNAWAY SUITE CAN NO LONGER TAKE A CI RUNNER DOWN. Two runs in a row lost their Windows job in the regression step (one still running after 2.5 h, the next past its 180-minute limit until GitHub reported the runner lost, as it did the macOS Intel job of that run), and neither uploaded a log. A timed-out suite was stopped with p.kill(), which on Windows ends its Python alone and left a hung ngspice or openvaf-r running; E-773's memory cap read /proc (Linux only); the trace phase re-ran every failed suite with the full 20 minutes; and a run that never returned left no trace. Each Windows suite now runs in a job object (a stop ends the whole tree, committed memory capped at NG_SUITE_JOB_MEM_MB, 6 GB), macOS's watchdog reads the group with ps, a TIMEOUT lists the command line of every process still running, NG_SWEEP_BUDGET_S (9000 s on CI) stops the sweep in time for its summary and upload (NOT RUN past it), traced re-runs get 300 s, the trace writes each run's start, and a five-minute smoke step times one compile and one simulation before each sweep. Full sweep 536/536.

E-785 *examples/_setup.py (_install_ring, NG_RING_DIR), examples/run_regression.py (ring per sweep, attached to a failing suite's log), examples/lrmfuncs_examples/verify_lrmfuncs.py*

A SUITE THAT FAILS ONCE KEEPS THE OUTPUT OF ITS LAST RUNS. Four Windows suites failed once each over two sweeps (analyses, autobusopt, opargs, reuseloops), always on the first (Sparse) pass, and passed when E-777's trace re-ran them alone -- so the trace never held the failing run. With NG_RING_DIR, which run_regression.py sets for every suite, each process keeps its last 12 ngspice/openvaf-r runs in memory and writes them at exit, labelled with its solver pass; the sweep appends them to _failures/.log for a failed suite and deletes them otherwise. Checked with a stand-in ngspice: both passes' rings in the log; sweep time unchanged (378 s, 536/536). lrmfuncs writes its input file with \n (its $ftell was two bytes off on Windows, where Python wrote \r\n), as E-779 did for fgetc, ungetc, scanbody and lrmio.

E-786 *spicelib/devices/dev.c (osdi_pid_alive, osdi_sweep_stale_copies); examples osdireload*

ON WINDOWS ONE NGSPICE NO LONGER DELETES THE MODEL COPY ANOTHER HAS JUST STAGED. Since E-775 every Windows load is of a copy staged in TMPDIR, and each process's first load swept every ngspice_osdi_reload_*.osdi it found: a loaded copy refuses the delete, but one another ngspice had staged and not yet loaded did not, and that process's load failed ("The specified module could not be found"). Two to four random suites of every parallel Windows sweep, on either solver pass, passing when re-run -- E-785's rings showed the failing run in all four of the last sweep's. The copy's name carries its owner's pid (E-770); the sweep now removes only the copies of processes that are no longer running (OpenProcess / GetExitCodeProcess, access denied counted as running). osdireload: three rounds of eight ngspice processes loading one model at once, all 24 must simulate. Full sweep 536/536.

E-788 *frontend/com_optimize.c (com_optimize_help[]), frontend/com_optimize.h, frontend/com_help.c (help_details, help_detail), frontend/commands.c (optimize's line); docs ngspice_commands.md, handbook 03; examples helpcmd*

`help optimize` DESCRIBES EVERY OPTION. It printed the command table's one line, which had stayed at E-145's options: `[-method nm|lm]`, nothing of tr, pso, de, sa, cmaes, bayes or nsga2, of `-center`, `-constrain`, `-polish`, `-starts`, `-seed`, `-swarmsize`, the aliases, or the names a script reads afterwards. It now prints a 127-line description beneath that line -- usage, knobs, analyses, objectives, expression rules, constraints, the nine methods with every alias, the options with their defaults, the results and three examples on a deck the text names -- kept as an array of lines beside the parser in com_optimize.c and printed by a table of such descriptions in com_help.c (out_send, no printf conversion). `help all` stays one line per command; optimize's line names the nine methods and points at `help optimize`. The internals command table was regenerated. helpcmd [10], seven checks: the description follows the line; every flag and alias (39) and every -method name (31) the parser accepts is in it, scraped from the source; every published result is named; 79 columns, headers alone in column 0; `help all` keeps one line; the examples run as printed (R2 = 1k; is 1.13e-13 and n 1.21 fitted; R2 = 1.5k with the constraint met). helpcmd 27 of 27 (8 fail on the E-787 binaries); full sweep 536 of 536.

E-789 *osdi/osdisetup.c (OSDIholdInitialStep, the has_evaluated resets in setup and the temperature pass), include/ngspice/osdiitf.h, spicelib/analysis/cktsens.c (held from the base op to the end, released on both exits); examples finalstep*

`@(initial_step)` FIRES ONCE IN A SENSITIVITY ANALYSIS. E-7 fires it on an instance's first evaluation of an analysis, keyed on a mark that setup and the temperature pass clear; E-683 made `sens` fire `@(final_step)` once. But cktsens.c re-runs the device's setup and temperature pass per perturbed parameter, and CKTsetup/CKTtemp per AC frequency, and each pass cleared the mark: `sens v(a)` fired `@(initial_step)` 11 times for a two-parameter module and a three-frequency AC sensitivity 4 times, against one `@(final_step)`, so a model opening a file in one and closing it in the other, or resetting a counter or a seed there, saw the pair unbalanced. The analysis holds the mark from its base operating point to its end (OSDIholdInitialStep), released before the final step and on the error exit; the next analysis's setup clears it as before. finalstep [9], twelve checks: both sensitivity forms fire each event once, an op after a sens fires its own, op, dc, ac, tran, noise, tf,

pz, disto and sp fire each once; finalstep 47 of 47 per solver (3 per solver fail on the E-788 binaries); full sweep 536 of 536.

E-796 *frontend/spiceif.c (doset_user checks an integer write; doset rounds half away from zero); examples osdislips*

altermod AND **alter** GIVE AN INTEGER PARAMETER WHAT THE CARD GIVES IT. `altermod ipm n=3.7` rounded to 4 without a word, where the card warns; -2.5 became -2, where the card gives -3 (E-399, half away from zero); and `n=1e300` or `alter n1 k=3e9` stored the saturated 2147483647, which the card refuses (E-509) and which can pass a `from [0:2147483647]` check. The user-write wrapper now refuses a value whose rounding no `int` holds ("does not fit an integer; not applied", the old value kept) and warns for a non-integral one with the value and the result; `doset` rounds with `round()`, so the sweep path agrees. osdislips [1], six checks; osdislips 56 of 56 per solver (5 of the 6 in [1] fail on the E-795 binaries); full sweep 538 of 538.

E-797 *frontend/outitf.c (the save probe in beginPlot); examples osdislips, savemiss, scanfmt*

A SAVED OPERATING-POINT VARIABLE NO LONGER WARNS. `.save @n1[iop]` printed "'iop' has no value yet -- it is an operating-point variable and no analysis has computed one. It is recorded per point once an analysis runs" at every load: a warning about a correct deck (E-418's sentence, cut to one per item by E-600), where a built-in's `.save @r1[i]` in the same state never spoke. A name that resolves and only lacks a value is now accepted in silence; a missing device or parameter still warns. osdislips [2], three checks; savemiss [1] and scanfmt now check that nothing is said; osdislips 56 of 56 per solver (1 of the 3 in [2] fails on the E-795 binaries); full sweep 538 of 538.

E-798 *frontend/spiceif.c (if_print_string_params), frontend/postcoms.c (com_print), frontend/vectors.c (vec_get's string case), include/ngspice/fteext.h; examples osdislips, opvar*

print SHOWS A STRING PARAMETER. `print @t3m[mode]` died in vec_get, "can not handle string value of 'mode' in vec_get", then a checkvalid warning; `print @n1` printed the pair for every string parameter, and a string opvar was no better, while `showmod` listed them. `print` now prints a plain accessor's string parameters as text (`@t3m[mode] = lin`, a whole-device list included) and leaves out the words that named nothing else; an expression that meets one (`let x = @t3m[mode]`) is told it is a string parameter, its value, and that `print` shows it. osdislips [3], five checks; opvar [6] now expects the value; osdislips 56 of 56 per solver (all 5 in [3] fail on the E-795 binaries); full sweep 538 of 538.

E-799 *osdi/osdisetup.c (osdi_oob_member_note after E-558's range line); examples osdislips*

A VALUE OUT OF A PARAMSET MEMBER'S RANGE NAMES THE MEMBER THAT TAKES IT. After the netlist bound n1 to member rs (`l` in [1:10)), `altermod rsm l=20` or `alter n1 l=20` failed the next run with "Parameter l of 'n1' is out of bounds (value 20; range from [1:10))!" alone, though member rs__2 takes 20; E-668 gave the member line only when a corner moved the value. It follows any out-of-bounds real now: the member the instance was bound to when the netlist was read (LRM 6.4.2), that a member is not chosen again, which siblings accept the value and to write it in the netlist, or that no member accepts it (a `reset` reloads the netlist's own value, so it does not re-bind). osdislips [4], five checks; osdislips 56 of 56 per solver (3 of the 5 in [4] fail on the E-795 binaries); full sweep 538 of 538.

E-800 *frontend/inp.c (the .temp reader in inp_spsource); examples osdislips, sweepguard*

.temp 0 27 50 RUNS AT THE FIRST TEMPERATURE IT LISTS AND SAYS HOW TO RUN THEM ALL. SPICE2, HSPICE and LTspice run each listed temperature; ngspice read the list as a malformed number, "Could not set temperature to 0 27 50 / Set to default 27 C instead", and ran at 27 C, which the deck may not list at all (`.temp -40 125`). A list of numbers (spaces or commas) now runs at the first, with a warning naming the count and two ways to run them all: a `foreach t ... / set temp = $t / run` loop (checked to run each) or `.dc temp`. A value that is not a

list of numbers keeps the old message and 27 C. osdislips [5], seven checks; sweepguard's E-437 neighbour now expects `.temp 75 125` at 75 C; osdislips 56 of 56 per solver (3 of the 7 in [5] fail on the E-795 binaries); full sweep 538 of 538.

E-801 *spicelib/parser/inp2dot.c (dot_pz checks its two keywords); examples osdislips*

`pz` NAMES A MISSING OR WRONG KEYWORD, WITH THE SYNTAX. `pz 1 0 1 0 pol` (no `vol`/`cur`) handed an empty name to the analysis's parameter table and got "no such parameter on this device or parameter is missing"; so did a missing analysis, both missing, and `pole` for `pol`, and a third word was ignored. The two words are read and checked first: the missing transfer type (with what `vol` and `cur` mean), the missing analysis, both missing, a word that is not a keyword, two of one kind, a word after both, each ending `(pz in1 in2 out1 out2 vol|cur pol|zer|pz)`; either order stays accepted. osdislips [7], ten checks; osdislips 56 of 56 per solver (7 of the 10 in [7] fail on the E-795 binaries); full sweep 538 of 538.

E-802 *main.c (the batch epilogue), frontend/spiceif.c (if_deck_run_pending, ci_deck_ran), include/ngspice/ftedefs.h, include/ngspice/fteext.h, frontend/measure.c (meas_card_vector); examples osdislips*

A BATCH RUN WHOSE .control BLOCK RAN THE ANALYSES PRINTS EACH **`.meas`** ONCE, AND A **`.meas`** RESULT IS A VECTOR. The epilogue runs the deck whenever a `.print`/`.meas`/... card exists: with no analysis card that `run` had no job and re-printed every measurement, and with a `.tran` card the block had already `run` it simulated the deck a second time. It now skips when the block's simulations ran and nothing of the deck's is pending (`.print`/`.plot`/`.four` output still follows); a deck `.tran` beside a block `tran` still runs. And `print vmax` after the run said "not available": a card's result reached numparam only, where `meas` stores a vector. Each successful card measurement is now a vector of the plot it measured; a failed one drops a stale result (E-475); a name holding simulation data is not overwritten. osdislips [8], eight checks; osdislips 56 of 56 per solver (4 of the 8 in [8] fail on the E-795 binaries); full sweep 538 of 538.

E-803 *frontend/com_dl.c (com_osdi), spicelib/devices/dev.c (osdi_loaded_mods, osdi_list_loaded), spicelib/devices/dev.h, frontend/commands.c (osdi/pre_osdi take zero arguments); docs ngspice_commands.md; examples osdislips, helpcmd*

`osdi` ALONE LISTS THE LOADED LIBRARIES AND THEIR MODULES; AN UNKNOWN OPTION AND A **`-f`** WITH NO FILE ARE REFUSED. `osdi` answered "too few args" and nothing listed what was loaded; `osdi -l` opened a file named -l; `osdi -f` and `osdi -va` with no file did nothing in silence. With no argument the command now lists each library in load order with the modules it registered (a paramset family's members included), or says how to load one; an option other than -f/-force/-va is refused with the options named; flags with no file are refused; both count as load errors. osdislips [9], five checks; osdislips 56 of 56 per solver (4 of the 5 in [9] fail on the E-795 binaries); helpcmd 27 of 27; full sweep 538 of 538.

E-804 *spicelib/parser/inpgmod.c (inp_warn_dup_param; a repeated instance default replaces the listed value); examples osdislips*

A PARAMETER SET TWICE ON A **`.model`** CARD NAMES THE VALUE USED -- THE LAST, FOR AN INSTANCE-PARAMETER DEFAULT TOO. "only one value takes effect -- remove one" did not say which: E-395 left it out because a model parameter kept the last value and an instance-parameter default, pushed onto a list replayed in reverse, kept the first (`k=2 k=3` ran with 2; `showmod` listed both; `altermod` edited the entry the replay overwrote). A repeated default now replaces the listed value, so the last wins for both kinds and the message says so, naming the model; the alias pair stays E-517's error. osdislips [10], four checks; osdislips 56 of 56 per solver (3 of the 4 in [10] fail on the E-795 binaries); full sweep 538 of 538.

E-805 *spicelib/parser/inp2n.c (INP2N's node-count check); examples osdislips*

TOO MANY NODES ON AN OSDI INSTANCE LINE NAMES THE MODEL'S TERMINALS AND

THE NODES LEFT OVER. `n1 1 2 3 gm` on a two-terminal module answered ”too many nodes connected to instance”, naming neither the terminal count nor the terminals, while too few nodes (E-402) and a stray value are explained. It now reads ”too many nodes: n1 connects 3, but model gm (module gres) has 2 terminals (a, b); '3' is left over”. osdislips [11], three checks; osdislips 93 of 93 per solver (2 of the 3 in [11] fail on the E-795 binaries); full sweep 538 of 538.

E-806 *frontend/vectors.c (vec_fromplot: e806_alias, e806_short_form), frontend/outitf.c (name_eq), spicelib/analysis/cktsetnp.c (CKTapplyPendingNodPm); examples osdislips*

A VERILOG-A CHILD'S INTERNAL NODE ANSWERS TO **`n1#c1.mid`**, AND A VECTOR HOLDING A SUBCIRCUIT INSTANCE TO ITS SHORT FORM. A child instance's node is flattened as `n1#c1__mid` (`n1#g1__k1__mid` deeper) and only that spelling worked: `print v(n1#c1.mid)` was not available, `.ic` refused it as a non-existent node, `.save` matched nothing. And a noise contribution of a device in a subcircuit is `onoise_n.x1.n1_thermal`, while `print`, `alter` and `show` accept `x1.n1` (E-410). When a name is not found as written, the vector lookup, the save matching (`.save`, a `.meas` card's vector) and the `.ic`/`.nodeset` placement try the `.` after the `#` as `__`, and the vector lookup also matches a plot vector by its E-410 short form (only for names with an `x`-led segment). osdislips [12], seven checks; osdislips 93 of 93 per solver (5 of the 7 in [12] fail on the E-795 binaries); full sweep 538 of 538.

E-807 *frontend/spiceif.c (e807_altermod_bins, called by if_setparam); examples osdislips*

`altermod nch g=7m` REACHES EVERY BIN OF A BINNED MODEL. With cards `nch.1`, `nch.2` it answered ”no such device or model name nch”, with no word about the bins, and no spelling reached them together (`nch*`, `nch.*`, `@nch*[g]` failed; E-436's `@*:name[...]` covers subcircuit copies only). A model write naming nothing now goes to each `<name>.<digits>` card through the ordinary path (each bin's own messages and refusals), with a note naming the bins in order; it counts real writes, so a parameter no bin has says ”no bin took”. Built-in BSIM4 bins the same way. osdislips [13], five checks; osdislips 93 of 93 per solver (4 of the 5 in [13] fail on the E-795 binaries); full sweep 538 of 538.

E-808 *frontend/inp.c (inp_savecurrents writes .savecur), frontend/dotcards.c (ft_dotsaves marks them; ft_cktcoms), frontend/breakp2.c, include/ngspice/ftedefs.h (SAVE_AUTO_*), frontend/outitf.c (beginPlot leaves them out of frequency-domain plots); examples osdislips*

THE CURRENTS **`.options savecurrents`** ADDS STAY OUT OF AC, SP AND NOISE PLOTS. They are op/dc/tran quantities: in ac a built-in device's vector stayed 0 long (`print @c1[i] i(v1)` failed whole), an OSDI device's held the DC bias at every frequency (F19's route), and both noise plots carried every device's bias current per frequency. The saves savecurrents adds now carry their origin (`.savecur`, registered after the deck's own `.save` lines and marked), and beginPlot leaves them out of ac, noise, sp, disto, hb, pz and sens with one note per analysis, pointing an ac reader to `.probe i(<device>)`, checked to record the true ac current. op, dc and tran keep them; a `.save` the deck writes is kept everywhere. osdislips [14], six checks; osdislips 93 of 93 per solver (3 of the 6 in [14] fail on the E-795 binaries); full sweep 538 of 538.

E-809 *frontend/com_measure2.c (e809_window_cut after avg/rms/integ/min/max/pp; pp's echo); examples osdislips*

AN INTERVAL MEASUREMENT WHOSE TO LIES PAST THE END OF THE DATA SAYS THE WINDOW WAS CUT. A model's `$finish` ended a 1 us transient at 0.503 us and `.meas tran vavg avg v(1) from=0 to=1u` averaged over [0, 0.503 us] with only the corrected echo (E-485) to show it; `pp` even echoed the requested to=1e-06. A warning now says the data end at 5.028e-07, before TO=1e-06, and what the measurement covers, for avg, rms, integ, min, max and pp on any analysis scale; pp echoes the window used. Nothing during autostop's checks, nothing for a window inside the data. osdislips [15], seven checks; osdislips 93 of 93 per solver (6 of the 7 in [15] fail on the E-795 binaries); full sweep 538 of 538.

E-810 *osdi/osdiparam.c (OSDIparamUnits), include/ngspice/osdiitf.h, frontend/outitf.c (guess_type: e810_osdi_units_type); examples osdislips*

A SAVED OSDI PARAMETER OR OPVAR IS TYPED BY THE **units** IT DECLARES. The type of a saved `@dev[param]` vector was guessed from the name (`[g` admittance, `[c` capacitance, `[i` current, `[q` charge, `[p]` power, else voltage), so `(* units="W" *) real pw` was voltage, Ohm and no units were voltage, and an opvar was current only if its name began with i. For an OSDI instance's parameter or opvar the declared units now decide: A current, V voltage, W power, Ohm impedance, S/mho admittance, F capacitance, C charge, s time, Hz frequency, dB, rad, the two densities; none or unknown notype. Terminal currents, temp and built-in devices keep the heuristics. osdislips [16], ten checks; osdislips 93 of 93 per solver (5 of the 10 in [16] fail on the E-795 binaries); full sweep 538 of 538.

E-811 *frontend/postcoms.c (com_print sizes its line buffer to the name; the column header is a bounded snprintf); examples nameovf*

print OF A VECTOR WITH A LONG NAME NO LONGER OVERRUNS A HEAP BUFFER. `com_print` copied the vector's name into a 512-byte buffer (`strcpy`, or `sprintf` of `plot.name`), so a 512-character instance, node or Verilog-A parameter name -- a flattener's or an extractor's -- corrupted the heap: a crash in 2 of 12 plain runs, a fault in every one under macOS Guard Malloc (base ngspice; a built-in resistor reproduces it). The buffer is grown to the name before it is written, its capacity tracked across the width reallocations, and the column form's plot-name line is a bounded snprintf; names print whole. nameovf, eight checks under Guard Malloc where it exists: 600- and 3000-character instance and node names in line, column and `print all` form, and a 600-character Verilog-A parameter name; nameovf 15 of 15 (7 of the 8 fail on the E-810 binaries); full sweep 539 of 539.

E-812 *frontend/plotting/pvec.c (pvec builds its line in a DSTRING); examples nameovf*

display, **load** AND A NOISE PLOT'S LISTING OF A LONG VECTOR NAME NO LONGER OVERFLOW A STACK BUFFER. `pvec` sprintf'd `name : type, real, N long` into a 512-byte stack buffer and strcat'd the colour, the scale's name and the dimensions after it, so `display` of a ~470-character node aborted every time (fortified), a noise analysis's `onoise_<device>_thermal` of a long OSDI instance did the same, and `load` of a raw file holding a 600-character name aborted while listing it. The line is built in a growing string, otherwise byte for byte the same. nameovf, three checks under Guard Malloc where it exists: display, a noise plot's contribution and its value, write then load; nameovf 15 of 15 (all 3 fail on the E-810 binaries); full sweep 539 of 539.

E-813 *osdi/osdiload.c (osdi_solution_len: SMPmatSize + 1, for the E-677 capture and OSDIfinalStep's check); examples guardmalloc*

OSDIload'S BIAS-POINT CAPTURE READS WITHIN THE SOLUTION VECTOR. E-677's capture of the operating point (for a final step at the bias) runs at the MODEINITSMSIG load, which DCop issues at the end of every `op`, and copied `CKTmaxEqNum + 1` doubles out of `CKTrhsOld` -- allocated with `SMPmatSize + 1`, which is `CKTmaxEqNum`: one double past the end at every `op` and every ac or noise bias point of a circuit with an OSDI device. The value was never used, but the read went past the block whenever the vector's length filled its allocation slot (the parity of the unknown count): a fault under macOS Guard Malloc, first met with an OSDI internal node, then beside built-in R, C and D. The capture copies the vector's real length, as E-689's uic code already did. New suite guardmalloc, eleven checks per solver under Guard Malloc where it exists: an internal-node module through op, dc, ac, noise, tran, tf, sens; the D-slip decks; one to eight extra nodes; E-677's final step still at the bias; 7 of the 11 fail on the E-810 binaries; full sweep 539 of 539.

E-815 *spicelib/parser/inpdpar.c (INPtempParamKind, INPtempBelowZero; the E-467 line guard by parameter id), frontend/spiceif.c (doset_user; if_user_write_refusals) and spiceif.h, frontend/com_sweep.c (a refused point is NaN), spicelib/analysis/dctrcurv.c (a .dc sweep of a temp|dtemp knob,*

judged at its first and last point), osdi/osdisetup.c (osdi_instance_temp for OSDIsetup, OSDItemp and the osdimc re-derivation), osdi/osdiparam.c (OSDItempParamKind), osdi/osdidefs.h, include/ngspice/inpdefs.h and osdiitf.h; examples tempguard

NO ROUTE HANDS A MODEL A TEMPERATURE AT OR BELOW ABSOLUTE ZERO. Only OSDIsetup guarded the composed instance temperature; OSDItemp, which runs for every later analysis and sweep point, did not, so `alter n1 dtemp=-400` printed "the offset is ignored" and evaluated the model at -99.85 K (a resistor model's current reversed), and the note for `temp=-300` named temp=27 C, dtemp=0. Wider: `dt=-400` on the line, the knob's other spelling, passed E-467's keyword test; `dc @n1[dtemp] -400 0 200` ran its first point below 0 K without a word; a built-in device took such values through alter and .dc; and a later ambient change made a valid offset unphysical. One rule, by parameter id, now judges a written value on the line, in `alter` (not applied, the previous value kept) and in a .dc sweep (refused when its first or last point reaches 0 K, as .dc temp is), for OSDI and built-in devices alike; the `sweep` command records a point whose value alter refused as NaN instead of solving it at the previous value; one guarded composition serves OSDIsetup, OSDItemp and the osdimc re-derivation, running the device at the circuit temperature with one warning naming the offset, the circuit and the device temperature. Suite tempguard 18 per solver (13 fail on E-813).

E-817 *include/ngspice/inpdefs.h (struct card xgiven), frontend/inpcom.c (inp_fix_inst_calls_for_numparam records it; insert_new_line), frontend/subckt.c (the three deck copies carry it), frontend/inp.c (line_free_x; com_alterparam); examples subcktedit [1]*

`alterparam cell rr=100` ON A PARAMETER OF THE **`.subckt`** LINE CHANGES THE DEFAULT, NOT THE INSTANCES THAT GAVE THEIR OWN VALUE. By the time alterparam edits the stored deck, inpcom has rewritten each call to carry a value for every parameter in order -- the instance's own or a copy of the default -- and the inner `.params` share the `params:` list; alterparam replaced the value on every call, which is right for an inner `.param` (the documented form) and overrode `xb ... rr=500` along with the instances that took the default: 4 mA became 30 mA (built-in R and an OSDI card alike). The rewrite now records on the X card the names the instance gave itself (`xgiven`, carried by inp_deckcopy/_oc/_ln through every reset); alterparam leaves those calls alone, moves the `.subckt` line's default, and a Note names who kept their own value. Suite subcktedit [1] (5 checks, all fail on E-816): 4 -> 13 mA, inner `.param` still everywhere, nested call, OSDI card.

E-818 *frontend/spiceif.c (if_hasmodel_toplevel), include/ngspice/fteext.h, frontend/device.c (altermod's note); examples subcktedit [2]*

`altermod gm` WITH **`gm`** ONLY INSIDE A SUBCIRCUIT NO LONGER CLAIMS A TOP-LEVEL CARD WAS CHANGED. E-436's note counted every model carrying the leaf name and took one for the top-level card: with copies x1:gm, x2:gm, x3:gm and no top-level card, `altermod gm g=7m` failed ("no such device or model name gm") and the note said "the top-level card and 2 flattened subcircuit copies; only the top-level one was changed". Whether a top-level card exists is now asked: with one the note is as before; without, it says none exists, that nothing changed, and names a copy. Suite subcktedit [2] (4 layouts).

E-819 *spicelib/parser/inppas3.c (INPinternalNodeCanon; .ic/.nodeset defer it), include/ngspice/inpdefs.h, spicelib/parser/inp2dot.c (an analysis command's node retried by it), frontend/outitf.c (.save takes E-410's reconstruction when it finds the device); examples subcktedit [3]*

`.ic`, **`.nodeset`**, **`.save @...`** AND AN EARLY ANALYSIS COMMAND TAKE THE **`x1.n1`** SPELLING OF A DEVICE INSIDE A SUBCIRCUIT. `n1` in `x1` is flattened to `n.x1.n1`; E-410 lets print, alter and show write `x1.n1`, but `.ic v(x1.n1#mid)` and `.nodeset` (E-608's deferral asked for an instance named `x1.n1`) were "on non-existent node -- ignored", a built-in BJT's `x1.q1#base` too; `tf v(x1.n1#mid) v1` as the first command aborted (E-690, the same question); and `.save @x1.n1[pw]` of an operating-point variable was "no such device" because the save

path kept E-410's reconstruction only when the parameter read back a value, which an opvar has not before an analysis (E-476). The internal node's name is now taken as the circuit spells it (exact first, then `<type>.<path>`), composing with E-806's `x1.n1#c1.mid`; the save path takes the reconstruction whenever it finds the device. Suite subcktedit [3] (7 checks, 6 fail on E-816): .ic 1.5 V, .nodeset accepted, .save 4 mW / 2 mA, tf 0.5 before and after op, child node 1.7 V, BJT accepted, typo still refused.

E-820 *include/ngspice/osdiitf.h (OSDIdcSweep and its phases), src/osdi/osdisetup.c (the phase; setup and the temperature pass mark an instance sweep_continues at a later point), src/osdi/osdidefs.h (the mark, EVAL_FLAG_SWEEP_CONTINUES), src/osdi/osdiload.c (both evaluation paths pass it with the initial step), spicelib/analysis/dctrcurv.c (the sweep sets the phase; OSDIfinalStep before the restore); examples sweepstate [1]. With the compiler half (openvaf log)*

A **`.dc`** SWEEP OF A TEMPERATURE OR AN OSDI PARAMETER CARRIES THE MODEL'S VARIABLES FROM POINT TO POINT, AND **`@(final_step)`** SEES THE LAST POINT. Each temperature or parameter point re-runs the temperature pass, which re-fires `@(initial_step)` (kept: BSIM4 does its whole temperature and parameter preprocessing there, LRM 5.2.1), but every variable was initialised with it -- a counter read 1 at every point where a source sweep carried it, against LRM 4.6.2 -- and the final step ran after the sweep had restored the temperature or parameter (`dc temp 0 100 50` saw 300.15 K, `dc @n1[p] 1 3 1` p = 1, a mid-sweep $finish the same). The sweep now tells the OSDI layer its phase; a later point's initial step carries EVAL_FLAG_SWEEP_CONTINUES and the compiled model keeps its variables; the final step fires before the restore. Suite sweepstate [1] (9 checks, 7 fail on E-819): counter 11, 12, 13, conductance follows T, final at 373.15 K / p = 3 / q = 3, nested, op after starts again, $finish at 323.15 K; BSIM4 identical to ten digits.

E-821 *spicelib/analysis/dctrcurv.c (DCTrestoreSwept, the restore loop taken out of the sweep's end; dct_abort, the exit every abort now takes); examples sweepstate [2]*

A **`.dc`** SWEEP STOPPED BY A **`$fatal`** PUTS THE SWEPT SOURCE OR PARAMETER BACK. The sweep restored at its end, after $finish and after a refused point (E-427), but a $fatal in the solve (E-673), a fatal judged on the accepted point (E-703), a non-converged point, a refused rewind and an output-setup error all returned with the knob at the failing value: `dc v1 0 3 1` stopped at 2 V left `@v1[dc]` = 2 and the next op raised the same $fatal; `dc @fpm[g] 1m 4m 1m` left 3m. An aborted run cannot be resumed (only E_PAUSE can). Every abort now goes through one exit that restores every level (no final step, no end of plot, as before); $stop and a pause keep their values for resume. Suite sweepstate [2] (3 checks, 2 fail on E-819).

E-822 *src/osdi/osdicallbacks.c (osdi_severity_when labels a message raised outside a Newton iteration from the sweep phase; a setup pass holds its $fatal like its other messages), src/osdi/osdisetup.c (setup and the temperature pass publish the circuit), spicelib/analysis/dctrcurv.c (DCTpointAhead: the point in CKTtime before it is applied); examples sweepstate [3]*

A VERILOG-A MESSAGE RAISED IN A SETUP PASS NAMES THE SWEEP POINT BEING APPLIED, OR SAYS "(during setup)", AND A SETUP **`$fatal`** PRINTS ONCE. Code hoisted into setup runs in the setup and temperature passes, and its severity tasks took their LRM 9.7.3 place from the last analysis's CKTmode/CKTtime: a parameter sweep's $fatal at 3m said "(at sweep value 0.002)", a temperature sweep's warning at 50 C "(at sweep value 0)", an op after a dc "(at sweep value 0)" twice, an altermod after a tran "(at t = 2e-09)", a plain op printed the $fatal twice with no place (E-660 held every setup message but $fatal). The sweep publishes the point before applying it; outside a sweep the place is "(during setup)"; the setup pass's $fatal is held and superseded by the temperature pass's. Suite sweepstate [3] (6 checks, 5 fail on E-819).

E-823 *frontend/inpcom.c (inp_osdi_paths_at: a relative pre_osdi/osdi argument in an included file that names a file beside it is rewritten, relative to the deck when beneath it, else absolute; inp_top_dir; the reader calls it at call_depth > 0); examples libsens [1]*

pre_osdi IN AN INCLUDED LIBRARY NAMES ITS FILES BESIDE THE LIBRARY, NOT BESIDE THE TOP DECK. The pre_ commands resolve a relative name against inputdir, the top deck's directory, and nothing remembered which file a control line came from, so `.include sub/inc.lib` holding `pre_osdi gres.osdi` gave "Error opening osdi lib" from any directory, and `pre_osdi -va gres.va` "no such Verilog-A source"; a nested .include resolves against its including file, a pre_osdi in it did not. The reader now rewrites such a name when the file exists beside the included file (a .include at any depth, a .lib section): relative to the deck when beneath it, so the -va object keeps E-573's name `osdi/sub_gres.osdi`; absolute otherwise and for the plain osdi command. Flags, absolute names and names not found are left as written. Suite libsens [1] (8 checks, 6 fail on E-822).

E-824 *spicelib/parser/inplkmod.c (INPlookMod asks modtabhash), xspice/mif/mifgetmod.c (MIFgetMod starts at the hashed model); examples libsens [2]*

A **.model** CARD INSIDE A **.subckt** NO LONGER MAKES THE PARSE QUADRATIC IN THE INSTANCE COUNT. The card is copied per instance (`x1:gm`, `x2:gm`, ...) and each instance looked its copy up with a linear strcmp walk of the model list: 32 000 wrappers took 7.9 s with a built-in card inside against 0.13 s at top level, 2.8 s with an OSDI card, an XSPICE card 4.2 s at 16 000. INPmakeMod already kept a hash of the same models beside the list (swapped with it per circuit, used by INPgetMod); INPlookMod and MIFgetMod now use it. Card inside 0.23 s at 32 000, linear to 1.1 s at 128 000. Suite libsens [2] (4 checks, 3 fail on E-822). Left: under Sparse, XSPICE models sharing one node stay quadratic in spGetElement wherever their card is.

E-825 *spicelib/analysis/cktsens.c (each perturbation named for the OSDI layer; the perturbed load's E_PANIC, the $fatal count and new $finish$/stop requests read; abort or end once the parameter is back; the error exit restores CKTbypass), src/osdi/osdicallbacks.c (OSDIsensPerturbing labels, OSDIfatalsReported counts), include/ngspice/osdiitf.h; examples libsens [3]*

A VERILOG-A **$fatal** RAISED WHILE **sens** PERTURBS A PARAMETER ABORTS THE ANALYSIS, AND **$finish** AND **$stop** END IT. The perturbation loop discarded what its device calls returned -- `(void) sens_temp`, a bare `sens_load` whose E_PANIC carried the $fatal -- and never read the requests, so `if (g > lim) $fatal` at g = lim printed "(at the operating point)" and every sensitivity followed, and $finish$/stop vanished, where op/dc/tran/ac/noise act on all three. The message now says "(while sens perturbed n1:g to 0.001000001)"; a $fatal (the load's flag, a count at the log callback that catches an AC sens's setup-pass fatal, or a deferred fatal) aborts with an error naming the perturbation; $finish$/stop end sens at its last completed point with a Note ($stop cannot pause there). The error exit also switches bypass back on, which a failed sens left off for the session. Suite libsens [3] (6 checks, 4 fail on E-822).

E-826 *frontend/device.c (alter l=/w= re-bins any instance on a bin, not only an m-device; a size no bin covers is not applied, counted for `sweep`), frontend/spiceif.c (if_instance_binned; if_setparam_model returns 1 when no bin covers and prints INPgetModBin's message naming the bins), include/ngspice/fteext.h, main.c stub; examples rebinfinal [1]*

alter OF A BINNED INSTANCE'S SIZE MOVES IT TO THE BIN THAT COVERS THE SIZE, AND REFUSES A SIZE NO BIN COVERS. ngspice re-binds an instance on alter w/l (if_setparam_model) but only for a name starting with m, so an OSDI instance on `nch.1` (l < 1u) altered to l=5u kept nch.1's parameters at a size only nch.2 covers -- 1 mA where 3 mA was due -- in silence; and on the m-device path a size no bin covered printed "no model available" and was applied anyway (a BSIM4 instance left on nch.2 at 50 um). Now any instance whose card is `<base>.<digits>` is re-binned with the existing "Notice: model has changed", and an uncovered size is refused with E-600's message naming the bins. Suite rebinfinal [1] (6 checks, 4 fail on E-825).

E-827 *frontend/com_presnp.c (osdi_report_compile_failure; osdi_resolve_on_path moved here from com_dl.c and exported), frontend/com_dl.c (va_compile reports through it), include/ngspice/osdiitf.h; examples rebinfinal [2]*

A FAILED VERILOG-A COMPILE SAYS WHETHER THE COMPILER COULD NOT RUN OR THE SOURCE HAS AN ERROR. `pre_osdi -va` answered every failure with advice on where to put openvaf-r -- including a compiler that ran and printed the source's error just above (exit 65) -- and `pre_snp` the same, with the raw wait status ("exit 32512"). One report now decodes the status and says: could not run the compiler (no such file, not on PATH, not executable, the shell's 127) with the advice; exit 101 an internal error; any other exit "could not compile X (exit N); its messages above say why"; a signal by name. Suite rebinfinal [2] (5 checks, all fail on E-825); vacompile [22]/[23], which pinned the advice on a source error, now pin the new report.

E-828 *spicelib/analysis/dcpss.c (DCpss fires the final step once the period is confirmed, before its small-signal reports), frontend/com_hb.c (the steady state at t = T into CKTrhsOld, then the final step), src/osdi/osdiload.c (OSDIforgetBiasPoint), include/ngspice/osdiitf.h; examples rebinfinal [3]*

`pss` AND **`hb`** FIRE **`@(final_step)`**, AT THE END OF THE PERIOD, ON THE STEADY STATE. E-683 made every analysis fire it once; pss and hb end in their own code and never did (initial_step x1, final_step x0), so a model closing a file or reporting a tracked peak there never got to. pss fires it while CKTrhsOld holds the end of the period (a relaunched run fires in its own confirmed branch); hb puts the sum of each node's two-sided harmonics (t = T) into CKTrhsOld and drops E-677's bias-point capture, which HB's per-sample small-signal loads had retaken -- without that the step read -0.14895 V where the steady state is -0.14515. Suite rebinfinal [3] (4 checks, 2 fail on E-825; tran and qpss controls).

E-829 *src/osdi/osdiload.c (OSDIfinalStep keeps what its evaluations raised; OSDIfinalStepVerdict reports it), include/ngspice/osdiitf.h, spicelib/analysis/cktdojob.c (the verdict after each analysis, sens included), frontend/com_hb.c (hb asks too, sets sim_status); examples limfinal [1]*

A VERILOG-A **`$fatal`** INSIDE **`@(final_step)`** FAILS THE RUN, AND **`$finish`** OR **`$stop`** THERE IS NOTED. OSDIfinalStep (E-53) discarded what its evaluations returned and every analysis ignored its return value, so `@(final_step) $fatal(...)` printed its message and the analysis was not marked aborted, the exit status was 0 and the script ran on, while a $fatal anywhere else aborts; $finish and $stop there were dropped in silence. The flags are now kept, and after the analysis returns CKTdoJob (or hb) asks: a $fatal prints an error and returns E_PANIC -- "simulation(s) aborted", exit status 1, the results kept -- and $finish$/stop get a Note that the analysis had already ended. Suite limfinal [1] (7 checks, 6 fail on E-828).

E-830 *src/osdi/osdiload.c (osdi_lim_apply: a BJT-family module without a polarity parameter `type` gets no simulator-side limiting; osdilim_verbose says why); examples limfinal [2]*

E-543'S BJT LIMITING NEEDS A POLARITY PARAMETER, NOT ONLY TERMINALS NAMED c, b, e. E-543 recognized a BJT by its terminal names, so a linear three-terminal resistor network `rnet(c, b, e)` got DEVpnjlim crawling across a 100 V bias: 577 Newton iterations and dynamic gmin stepping for an operating point that needs 3, and 40 % more iterations in a transient. A transistor model names its polarity (MEXTRAM TYPE, HICUM type, opamp741's bjt741 type, which the limiter already reads); without one a module is no longer limited. The MOSFET family is unchanged (its linear twin cost 6 against 3). Suite limfinal [2] (4 checks, 3 fail on E-828); osdilimit and opamp741 unchanged.

E-831 *frontend/dotcards.c (ft_saveused standing aside still registers `save all` and the @-references it collected); examples limfinal [3]*

`.option saveused` SAVES THE **`@dev[param]`** VECTORS A DECK'S **`.meas`** CARDS READ, ALSO WHEN IT OTHERWISE STANDS ASIDE. E-572 collected the references of the deck's .meas/.print/.plot/.four cards, `@dev[param]` included, but the option acts only when the control block has an output command; a block that only runs `tran` made it stand aside, leaving ngspice's default save set, which never holds an `@device[param]` vector -- `.meas tran p_max max @n2[pw]` failed with "holds 1 point(s) but the analysis produced 2018". Standing aside now registers `save`

`all` and those vectors (marked as the option's own), as the error message itself advises. Suite limfinal [3] (3 checks, 1 fails on E-828).

E-832 *src/osdi/osdiload.c (the private simparam `$osdi$seed`; OSDIanalysisSeed), include/ngspice/osdiitf.h, spicelib/analysis/cktdojob.c (seeded at a job's start, before its setup, and for each later analysis; not a resume), frontend/com_hb.c (hb seeds), maths/misc/randnumb.c and include/ngspice/randnumb.h (ng_seed_generation); examples seedac [1]. With the compiler half (openvaf log)*

AN UNSEEDED VERILOG-A **`$random`** OR **`$arandom`** DRAWS A NEW VALUE IN EACH ANALYSIS AND EACH INSTANCE, AND **`setseed`** REPRODUCES THEM. The unseeded forms drew with the seed 0, so a draw was a function of its call site alone: four ops around `setseed 1`, `setseed 1`, `setseed 2` all printed 1366254664, two instances drew identical values, and a `repeat N op` loop drew one value N times. ngspice now publishes a per-analysis seed, a function of the generator's seed (setseed, rndseed, 1 at startup) and of the analyses started since it was set, taking nothing from the generator's own stream; the compiled model mixes it with the instance's name. Constant through an analysis, so the draw stays pure (E-10). Seeded draws unchanged. Suite seedac [1] (5 checks, 3 fail on E-831).

E-833 *src/osdi/osdisetup.c (OSDIdcpathEdges marks the Kirchhoff nodes an instance created for itself -- not a flow, not an implicit equation), include/ngspice/osdiitf.h, spicelib/analysis/cktsetup.c (the dcpath walk checks such a node instead of passing it by); examples seedac [2], dcpath [10]*

AN OSDI MODULE'S INTERNAL NODE WITH NO DC PATH GETS THE DC-PATH GMIN, AS A BUILT-IN DEVICE'S TWIN DOES. E-575's walk passes by every node named with a `#`, taking it for a built-in device's internal node joined to its terminals by series elements, and an OSDI module's internal nodes are named so too (`n1#mid`). One reached only through ddt() went `singular matrix: check node n1#mid` down every homotopy, 277 iterations, and the transient operating point left it at 0.5 V where the built-in twin was held at 0 at once. The module's pattern says what joins such a node, and the walk now checks it: 3 iterations, the dc-path message, held at DC only and released in a transient (E-595). The compiler's implicit equations (absdelay's) are not KCL nodes and stay unmarked. E-679's delay chain around a child's internal node is now held and stops as diverging, as at the top level (dcpath [10] revised). Suite seedac [2] (4 checks, 1 fails on E-831); dcpath 84/84.

E-834 *src/osdi/osdisetup.c (osdimc_def_hash, osdimc_def_name, osdimc_walk_slot and the bin table; the keys of osdimc_apply_type, osdimc_apply_derived and the importance weight; osdimc_walk_count); examples seedac [3]*

UNDER **`osdimc`** THE BINS OF ONE BINNED MODEL DRAW ONE PROCESS DEVIATE. A process parameter's draw was keyed on the model card, so `nch.1` and `nch.2`, one device type at two sizes, drew independently (g 0.930m against 0.869m, then 1.100m against 1.113m) and a transistor's process shift jumped at a bin boundary. The key is now the definition, the name less a bin's `.<digits>` (E-826's rule): the same z in a plain trial, the same -lhs stratum, the same walk coordinate (wcd counts one dimension, was two), one factor of the highsigma weight (two bins give one card's estimate), one -inflate scope. Other names key as before. Suite seedac [3] (4 checks, 3 fail on E-831).

E-835 *src/osdi/osdiparam.c (osdi_ask_ac_current: in ac and sp, the Jacobian's rows of the terminal's collapse group times the complex solution, plus an active ac_stim), include/ngspice/osdiitf.h (osdi_ask_complex), frontend/outitf.c (getSpecial records the answer as complex; savecurrents keeps an OSDI device's currents in ac and sp); examples seedac [4], osdislips [14]*

IN **`ac`** AND **`sp`** AN OSDI DEVICE'S TERMINAL CURRENTS ARE ITS SMALL-SIGNAL CURRENTS, NOT THE BIAS CURRENT. E-394's read sums the resistive residual the bias point left in the instance, so a saved `@n1[i_a]` in ac held 1 mA + 0j at 1 kHz and at 1 MHz where the true current was 0.5 mA and 0.954 + 0.145j mA (sp the same), complex vectors that looked like ac

data and were not; E-808 had only kept `.options savecurrents` from adding them. The ask now forms (G + jwC) times the solution over the group's rows, from write_jacobian_array_* (the loads' own values, m included), plus an ac_stim source, and the plot records it complex; savecurrents keeps them in ac and sp, built-in devices' still left out. Suite seedac [4] (7 checks, 5 fail on E-831); osdislips 93/93.

E-836 *src/osdi/osdisetup.c (OSDIsetup: the internal rows take their name, kind and nodeset from the node each stands for, row_node); examples osdirows [1]*

AN OSDI INSTANCE'S INTERNAL ROWS CARRY THE NAME, THE KIND AND THE NODESET OF THE NODE THEY STAND FOR. collapse_nodes compacts an instance's rows, and the loop that created them read `descr->nodes[i]` with i the compacted index, another node once anything below had collapsed (the upstream `TODO handle currents correctly`). With `V(p, a1) <+ 0` the midpoint row was named n1#a1 and held a2's 0.5 V, and terminal t's 0 V branch was a VOLTAGE node n1#a2: vectors misnamed, a branch current tested against vntol, E-45's nodeset on the wrong row, and every singular-matrix and dc-path message naming the wrong node (BSIM-BULK's thermal branch read `check node n1#si`, which sent the campaign's F1 lead into its source network). Each row now takes the smallest descriptor node of its collapse group, the survivor collapse_nodes keeps. F1 of the 2026-10-10 robustness and correctness campaign.

E-837 *src/osdi/osdisetup.c (OSDIsetupPass, osdi_dup_shorts; collapse_nodes drops the duplicates), src/osdi/osdidefs.h (dup_short_mask), src/include/ngspice/osdiitf.h, src/spicelib/analysis/cktsetup.c (the pass around the device loop); VA_TEST/correctness_campaign.py and .md; examples osdirows [2]*

A MODEL'S TERMINAL-TO-GROUND SHORT THAT ANOTHER INSTANCE ALREADY MAKES IS DROPPED, AND ONE BESIDE A VOLTAGE SOURCE IS NAMED. Self-heating models with the heating off write `Temp(t) <+ 0` (BSIM-BULK, -CMG, -IMG) or `V(br_sht) <+ 0` (HICUM), a 0 V branch to ground that E-401 drops only when the terminal is ground. Two instances on one thermal net, or a deck's `vt t 0 0`, put two ideal sources in parallel: a singular matrix the leaked diagonal gmin hid until E-734, and 11 of the corpus's 14 F1 models (the harness holds spare terminals with 0 V sources). A pass-scoped table of pinned nodes now drops the second short as the same equation (sens reuses the per-instance decision), and a voltage source on the node is named in a warning: its value is the deck's, so that loop stays refused. The corpus harness connects such a terminal to ground and is back to 90 of 92.

E-838 *src/spicelib/analysis/cktsetup.c (CKTdeadHold; its mode from .option dcpath; the hold in CKTdcpathStamp), src/include/ngspice/cktdefs.h (the list; CKTnode notKcl), src/osdi/osdisetup.c (an implicit equation is marked notKcl), src/maths/ni/niiter.c (before each reorder without diagonal gmin); examples osdirows [3], floatnode and paramrange [4] revised*

A VOLTAGE NODE WHOSE ROW NO VALUE REACHES IS HELD WITH THE DC-PATH CONDUCTANCE AT THE REORDER. HiSIM-SOI's current release writes its body nodes db and sb only when COBCNODE = 1; in the 4-terminal mode the pattern still holds their entries and every value is 0, so E-116 (no entry), E-575's walk (reads the pattern) and E-571 (ac only) all passed them by and every factorization of the op failed, hidden by the leaked gmin until E-734. Before a reorder with no diagonal gmin, an all-zero row or column (E-571's SMPzeroLines) is held with gmin or the dcpath value from then on, in every mode, and named; dcpath=warn and error name it, off and gshunt leave it. Only a matrix that cannot be factored changes; the cost is one pass per reorder. An OSDI implicit equation (an idt state, absdelay's) is no Kirchhoff node and goes to the ladder as before: `idt(1.0)` reads 0 = 1 at dc. hisimsoi_n4 and _n5 converge to the version11 answer; an OSDI BSIM4 open gate is v(g) = 0 as its built-in twin (it was optran's ramp-dependent 0.4324 V), and a rejected HiSIM-SOI's noise completes again.

E-840 *src/spicelib/analysis/cktdelt.c (CKTdelTask clears CKTcurJob when it frees the job it names, after DCtran_step_quit), src/frontend/runcoms2.c (com_remcirc deletes the tasks before it frees the cir-*

cuit); examples jobreset

A REFUSED ANALYSIS COMMAND NO LONGER LEAVES THE CIRCUIT POINTING AT A FREED JOB. CKTcurJob stays on the last job run, and an interactive analysis command deletes the previous command's task before parsing its own card; a card refused there (`sens v(nosuch)`, `ac dec 0 1 1`, `tran 1u`, `tf v(nosuch) v1`, `noise v(nosuch) ...`) never reaches CKTdoJob, so the next `reset` read the freed job's JOBtype in DCtran_step_quit: SIGSEGV under Guard Malloc, silent without it (campaign 2026-10-10 F3, found by the deck fuzz). CKTdelTask now closes a stepped transient's plot and clears the pointer when it frees that job, and com_remcirc, which freed the circuit before its tasks, deletes them first. jobreset 14/14 under Guard Malloc, 13 fail on the E-838 binaries.

E-841 *src/spicelib/analysis/cktsens.c (sens_rebind: CKTunsetup, CKTsetup, CKTtemp once the perturbations are done; the error path puts the matrix and right-hand sides back); examples jobreset [6]*

`sens` BINDS THE DEVICES IT PERTURBED BACK TO THE CIRCUIT'S MATRIX. Each perturbation re-runs a model's DEVsetup into sens's own delta_Y, binding its instances' matrix pointers there; the ac path re-set up per frequency but the last, the dc path never, and delta_Y was freed. A second `.sens` in one deck (or an `.sp` after one, which runs in the same task without a new setup) loaded through freed memory: singular matrix on a resistor divider, a fault in RESload under Guard Malloc (campaign 2026-10-10 F4). An error inside the loop also left CKTmatrix, CKTrhs and CKTirhs on sens's arrays. A re-setup after the last perturbation (and on the error path, after the matrix is back) binds every device to the circuit's matrix. jobreset 21/21; 4 of [6]'s 5 checks fail on E-840.

E-842 *src/frontend/com_envelope.c (the check after the settling transient); examples jobreset [7]*

`envelope` CHECKS THAT ITS SETTLING TRANSIENT REACHED ITS END. E-502 asked only for CKTmatrix, which exists before any device setup; a device refused at setup (a lossless line with z0 = 0, the fuzz's `tran 0 0` read as a line named tran) stops CKTsetup before NIreinit, and EFanalysis copied from a NULL CKTrhsOld, a SIGSEGV every time (campaign 2026-10-10 F5). The check now asks for the solution vector, the state vectors when the circuit has states (a divider with no charges has none, and still runs), and CKTtime at Tsettle, so a setup refusal and a transient that gave up part-way are both refused by name. jobreset 21/21, sensstate 17/17.

E-843 *src/frontend/inp.c (com_source counts its nesting and refuses past 50 levels; inp_source_prompt runs the prompt of a failed source on its own control level), src/frontend/signal_handler.c (an interrupt resets the count), src/include/ngspice/fteext.h, src/frontend/parser/lexical.c (the lexer reports EOF), src/main.c (without readline the main loop quits at EOF); examples sourcenest (new)*

`source` NESTING STOPS AT 50 LEVELS, AND THE PROMPT A FAILED **`source`** DROPS TO ENDS AT END OF INPUT. A deck that sources itself, two that source each other, or a .spiceinit that sources itself recursed com_source -> inp_spsource -> the control block -> com_source until the stack overflowed: SIGSEGV after 3 932 levels on an 8 MB stack (campaign 2026-10-10 F6). `.include` has stopped at 50 since E-212; `source` now refuses a 51st level by name and fails as a missing file does: exit 1 in batch, a prompt under `set interactive`. That prompt ran away at the end of its input -- cp_lexer had no case for EOF, pushed it onto the word and read EOF again until a 2 GB realloc failed -- and, once it could end, ran every command typed there a second time, because it recorded them on the control level of the block that ran `source`. The lexer now ends a partial line at EOF and returns NULL with nothing read; the prompt runs on a level of its own; a build without readline, whose main loop is a bare cp_evloop(NULL), quits at EOF as the readline loop does. VERIFIED: sourcenest 9/9 (7 fail on E-840: 5 SIGSEGV, a 51-level chain run, the realloc failure); a scratch build with --with-readline=no quits at EOF.

E-846 *src/spicelib/analysis/dctran.c (the first step's truncation check; the t = 0 solution kept for a retry of the first step, on a rejection or a non-convergence); examples firststep (new), loopguard [20], reusestate [20], scaleguard [5]*

THE FIRST TRANSIENT STEP IS CHECKED AGAINST THE TRUNCATION ERROR. dctran accepted the first step unchecked ("no check on first time point": the estimate needs a point before t = 0), at a fixed fraction of the run whatever the time constants and tolerances: a sine into a 1 ns RC took a 1 ns backward-Euler step, 32 % off at reltol = 1e-7, a `uic` charge 21 %, and the bad point stayed in every later estimate's divided differences -- a 7-element linear L-C network collapsed "Timestep too small" right after it at reltol <= 1e-6 (campaign 2 of 2026-10-10 F2, F3). After an operating point the circuit was at rest before t = 0, which dctran already copies into states 2 and 3: the first step is now checked with that history at its own spacing and retried from the kept t = 0 solution (the non-convergence retry now restores it too). A kink at t = 0, a jump or a moving `uic` state makes the estimate scale with the step; when a retry does not double newdelta/delta the first step goes back to SPICE's size, since a shorter first step leaves trap's unresolved stiff modes ringing to the end of the run. A floor of a thousandth of the first step and ten rejections bound it; trap, Gear and Adams. An estimate asking for less than a billionth of SPICE's first step is distrusted the same way (at abstol = 1e-15, chgtol = 1e-20 such steps reach the estimate's own rounding). A `uic` state already moving looks like a kink, so `uic` is only partly fixed: 26 of Family D's uic runs keep a first-step error over the limit (42 on E-844). VERIFIED: firststep 9/9 per solver (6 fail on E-844), Family D: 2 of 292 tight runs collapse at abstol = 1e-15, chgtol = 1e-20 (7 on E-844), none at reltol = 1e-7 with default abstol and chgtol (none on E-844 either: there it is F3's reduced network that collapses); loopguard [20], reusestate [20], scaleguard [5] rewritten to their intent (each pinned the old grid), integmethod (adams = trap) and collapsecur unchanged.

E-847 *src/spicelib/analysis/cktterr.c (the estimate's absolute floor is a parameter: CKTterr keeps abstol, the new CKTterrFlux uses vntol), src/spicelib/devices/ind/indtrunc.c, src/include/ngspice/cktdefs.h; examples firststep [3]*

AN INDUCTOR'S TRUNCATION-ERROR FLOOR IS A VOLTAGE TOLERANCE. CKTterr bounds a state's error by abstol + reltol*

E-848 *src/frontend/com_measure2.c (measure_minMaxAvg and measure_rms_integral: the value at `from` is a point of its own, ahead of the first sample in the window); examples measstart (new)*

AVG, INTEG AND RMS WITH FROM= KEEP THE FIRST SAMPLE IN THE WINDOW. E-302 made AVG start at exactly `from` by interpolating there, but it overwrote the first in-window sample's value and time with the interpolated point, so that sample never entered the sum and the trapezoid ran from `from` straight to the second sample; measure_rms_integral did the same for INTEG and RMS. On a triangle whose corner is that sample, AVG and INTEG were 3e-4 low; 96 of Family C's windowed AVGs were off, by up to 4.8 % on a sampled sine (campaign 2 of 2026-10-10 F4). AVG now opens its sum with the trapezoid from `from` to the sample, and INTEG and RMS put the point before it. VERIFIED: measstart [1]..[3] under both solvers (3 checks fail on E-844); Family C's AVGs all match, and its remaining INTEG and RMS differences match a model of the quadrature (F11).

E-849 *src/frontend/com_measure2.c (com_measure_when: a window whose first sample comes after sample 1 opens at its TD, FROM or TO; helpers when_value and when_scale); examples measstart*

A WHEN WINDOW OPENS AT ITS TD, FROM OR TO. com_measure_when spent a window's first sample on remembering a value and its second on classifying the side of the threshold, and evaluated from the third, which E-418 made deliberate for the interval from the operating point. After TD both samples are real, and a crossing between them was lost: `when v(1)=0.55 rise=1 td=0.45u` failed out of interval, and with several crossings the next one was reported (campaign 2 of 2026-10-10 F5). FIND..WHEN and TRIG/TARG with TD, WHEN with FROM, and ac and dc sweeps entering through FROM or TO lost it too. The boundary's value is now interpolated, the side classified there, and the interval to the first sample evaluated at once; the interval from sample 0 and the restart of a nested dc sweep are as before. VERIFIED: measstart [4]..[11] under both solvers (11 checks fail on E-844; 3 are controls).

E-850 *src/include/ngspice/noisedef.h (N_MINGAIN 1e-20 -> 1e-200; Ndata counts the frequencies that reach it), src/spicelib/analysis/noisean.c (NgainSqInv and NlowGainNote; NOISEan uses them), src/spicelib/analysis/dcpss.c (pnoise and the swept qpnoise count and report); examples noisegain (new)*

THE INPUT-REFERRED NOISE DIVIDES BY THE GAIN THE CIRCUIT HAS. noisean.c computed GainSqInv as 1/MAX(gain^2, N_MINGAIN) with N_MINGAIN = 1e-20, so a gain below 1e-10 was taken as 1e-10 without a word. A real circuit reaches it: 1 fF into 1 kOhm has 6.3e-12 at 1 Hz, and inoise_spectrum came out 40.7 V/sqrt(Hz) where onoise/gain is 648, 16 times low (campaign 2 of 2026-10-10 F6; 3 of Family E's 300 networks). The floor is now 1e-200 on the gain squared, which only an output the input cannot reach meets, and the frequencies that meet it are reported once per analysis, naming the source; pnoise and qpnoise share it. VERIFIED: noisegain [1]..[4] under both solvers (2 fail on E-844); the 16 noise suites unchanged.

E-851 *src/frontend/measure.c (com_meas stores its result, and substitutes a vector after =, with %.17g; meas_card_vector too, for E-802's card results); examples meascmd (new)*

THE MEAS COMMAND KEEPS EVERY DIGIT. com_meas stored its result with `let name = %e`, 7 significant digits, where the same .meas card's result had 16 (E-802), and put a vector named after val=, at=, from= or to= back into the line with %e too. Two crossings 1.8 fs apart read the same time and their difference was 0, and `find v(1) at=t2` read 0.5 where the crossing t2 names is at 0. 50000001 (campaign 2 of 2026-10-10 F7). All three now print 17 significant digits, which identify a double, and both readers are correctly rounded since E-643, so the vector holds the measured double. VERIFIED: measmd [1]..[4] under both solvers (3 fail on E-844; [4] is a control).

E-852 *src/spicelib/analysis/dctran.c (uic_t0_point, called where the first accepted point is output); examples uicstart (new), firststep [2]*

A UIC TRANSIENT HAS ITS T = 0 POINT. uic loads the circuit once and solves nothing, so CKTrhsOld holds the .ic values and zeros, and dctran wrote a point under uic only for CKTtime > 0: the run began at the first step, meas and fourier with it, and `find v(2) at=0` was out of interval (campaign 2 of 2026-10-10 F8). The t = 0 point is now solved -- each capacitor at its initial voltage, each inductor at its initial current, the sources at t = 0 -- as a backward-Euler step a millionth of the first; where a source makes a capacitor jump (a supply node starting at 0 V, junctions at their IC= values) that solve carries the jump's impulse, C*dV/h, so a second solve from the point found, its charges as the history, gives the finite currents at t = 0+. The solution, states 0 and 1, mode, step, step history, order and solver state are then put back, so the run after t = 0 is unchanged. Not with tstart > 0 or XSPICE event instances; no point if a solve fails. VERIFIED: uicstart [1]..[8] under both solvers (6 fail on E-844); five uic circuits under trap, Gear, KLU and Sparse are bit for bit HEAD's after t = 0.

E-853 *src/spicelib/analysis/dctran.c (a note at the start of a transient when maxord is above 2 and has no effect; order_ceiling bounds dynorder's order, its hold and ordfix's pin at 2 under trap); examples maxordnote (new)*

MAXORD SAYS WHEN IT HAS NO EFFECT, AND TRAP STAYS AT ORDER 2. The stock order control only toggles Gear between orders 1 and 2, and dynorder (E-128) and ordfix (E-181) are the ways above; maxord was accepted up to 6 and range-checked, and `method=gear maxord=6` was the run of maxord=2 to the last digit without a word (campaign 2 of 2026-10-10 F9). A note now says so, for Gear without dynorder or ordfix, and for trap, whose order is 2 at most. The dig found trap with dynorder or ordfix taken to order 3: NIcomCof refuses it with E_ORDER, which dctran does not check, so the step ran on the previous step's coefficients -- 19 rejected steps and a 2e-6 shift with dynorder maxord=3, and ordfix=3 ran past 300 s -- and maxord also set dynorder's order hold. order_ceiling is 2 under trap: every trap maxord and ordfix above 2 is now the run of 2, bit for bit. Gear is unchanged. VERIFIED: maxordnote [1]..[6] under both solvers (4 fail on E-852; [2] and [6] are controls).

E-854 *src/osdi/osdiload.c (nonfinite_count before and after each instance's load, nonfinite_report), src/*

osdi/osdidefs.h (nonfinite_told), src/include/ngspice/cktdefs.h (CKTtroubleNonfinite; cleared in cktdojob.c and cktinit.c), src/spicelib/analysis/ckttroub.c (names it first); examples osdinonfinite (new)

AN OSDI DEVICE'S NON-FINITE LOAD IS NAMED. `V(o) <+ ln(V(x))` at Newton's zero start loads -inf, every homotopy restarts from the same point, and the operating point ended "Timestep too small; cause unrecorded" -- "trouble with node o", the singular row, under KLU -- naming neither the device nor the value (campaign 2 of 2026-10-10 F10). The non-finite entries of each instance's right-hand side rows and Jacobian entries are counted before and after it loads, so another device's value is not charged to it; when they rise, the instance becomes the trouble element, which CKTtrouble names before a singular node ("trouble with qm-instance n1, which loaded a value that is not finite"), and a warning, once per instance, gives the value and the terminal voltages. VERIFIED: osdinonfinite [1]..[5] under both solvers (4 fail on E-852; [5], a guarded model, is the control).

E-855 *src/frontend/com_measure2.c (measure_quad, the integral INTEG, RMS and AVG share; measure_minMaxAvg collects AVG's points and calls it); examples measquad (new)*

AVG AND INTEG INTEGRATE BY ONE RULE. AVG integrated by trapezoids and INTEG and RMS by Simpson's 3/8 or 1/3 rule on runs of equal steps, so INTEG/(to-from) and AVG over one window differed -- 2e-3 on a sampled sine, 4e-4 over a whole run, 7e-5 on a dc sweep -- where E-302 calls them the same quantity (campaign 2 of 2026-10-10 F11). The rule is now one function; AVG collects the window's points -- the value at from, the samples, the value at to, as INTEG's -- and divides their integral by the span, a falling dc sweep integrated upward as INTEG walks it (E-304). The sine's AVG is 1.4e-4 from the analytic average, against 1.9e-3. VERIFIED: measquad [1]. .[6] under both solvers (5 fail on E-852; [2] and [6] are controls); Family C's AVGs all match the quadrature model.

E-856 *src/spicelib/analysis/cktpzstr.c (root_factor, the analytic deflation factor; a repeated complex point tested against the modulus; pz_deflated, pz_row_scales, pz_c_dominates and pz_is_zero, which tell a zero from rounding noise beyond the last root), src/maths/KLU/klusmp.c and src/maths/sparse/spsmp.c (SMProwMaxReal, declared in src/include/ngspice/smpdefs.h); examples pzdeflate (new)*

THE POLE-ZERO SEARCH DIVIDES OUT ITS ROOTS ANALYTICALLY. The search divides each found root out of det(G + sC) before it looks for the next, and the factor was |p - s|^2 whenever p - s was complex: the pair's factor at a real trial, and not analytic at a complex one, where a real root was divided out by its squared distance. Muller wandered to the iteration limit around a pair it had started next to, and the plot held a partial set -- 4 of 252 random networks (campaign 3 of 2026-10-10 F1), 18 of 1 718 runs of 859 more, 2 of those without a warning -- whose rational function missed ngspice's own ac response by 240 % to 450 %. A pair on the jw axis was found to the last digit and never declared: the test of a repeated point asked for equal imaginary parts and a real part within 1e-12 of |Re s| = 0. The factor is now (p - s), or (p - s)(conj(p) - s) for a pair (bitwise the old one on the real axis), and a complex point that repeats to 1e-12 of |s| is where Muller has converged. A converging search can stall on the noise beyond its last root, where s*C swamps G (|s| ~ 1e21); there, where some row has |s| max|C| >= max|G|, a point is a root only if the function grows around it as at a zero, 2^7 from 1e-9 to 1e-6 of |s| on both sides. VERIFIED: pzdeflate [1]..[9] under both solvers (6 fail on E-855 under Sparse, 7 under KLU; [8] and [9] are controls); Family C's 252 networks all match the exact roots under both solvers; 859 more: no wrong set but one zero at 1.7e-6 reported as 0, and none of the 11 remaining warnings with a wrong set; the pz suites pass.

E-857 *src/frontend/numparam/xpressn.c (unary_operand_end; formula evaluates a sign after an operator as a unary one), src/frontend/parse-bison.y (a unary plus for let; %expect 3); examples unaryparam (new)*

A SIGN AFTER AN OPERATOR IN A .PARAM IS UNARY, AS IN THE B-SOURCE. numparam read a minus after a binary operator by setting a flag that only a number consumed: before a name,

a parenthesis or a function the formula was refused and the netlist not loaded ({2*-p}, {1/-(2)}, {2*-sin(1)}: 114 of 4 000 random expressions, campaign 3 of 2026-10-10 F2); a second minus set the flag again, losing a sign without a word ({2*--2} was -4, {2---2} 4); the flag negated the number before ^ saw it ({2*-3^2} was 18, the B-source -18); and a plus there was a misplaced operator. A sign after an operator now takes the power chain after it -- sign* primary (^ sign* primary)*, as the B-source's grammar and let's give it -- evaluated as an atom; a sign at the start of a formula keeps its reading, 0 - x. let takes a unary plus, the mirror of its unary minus. VERIFIED: unaryparam [1]..[10] under both solvers (9 fail on E-855; [9] is a control); Family B's 114 refusals are gone and nothing new is flagged; 50 expressions with unary signs agree between the .param and the B-source.

E-858 *src/frontend/numparam/xpressn.c (fetchnumber reads mil and folds a mantissa's own exponent with the suffix's through decimal_parts; double_to_string writes 17 significant digits); examples milparam (new)*

A .PARAM READS A SUFFIXED NUMBER AS THE NETLIST DOES, AND PASSES IT ON WITH EVERY DIGIT. numparam's suffix table (E-643) had no mil: it read m and let "il" go, so {1mil} was 1e-3 where r1 1 0 1mil, a B-source and let give 25.4e-6 (the manual's table), 39 times apart in one netlist without a word (campaign 3 of 2026-10-10 F3); a subcircuit's wid=4mil the same. The dig found two more drifts: a mantissa with its own exponent was multiplied by the suffix, a second rounding ({66.5e4u} 0.6649999999999999 where the netlist reads 0.665; 37 of 300 spellings an ulp off), and the value went back into the deck with 16 significant digits ({0.1+0.2} reached the element as 0.3, {1mil} as 2.54e-05). mil is 25.4 times 1e-6 as in INPevaluate; the exponent and the suffix's power are added and the mantissa read once with the sum; the placeholder takes 17 digits in its 25 characters. VERIFIED: milparam [1]..[6] under both solvers (5 fail on E-857; [6] is a control); 316 suffixed spellings give one double through an element value, a .param, a B-source and let.

E-859 *src/frontend/parser/numparse.c (scale_factor, taken out of ft_numparse, follows an exponent too); examples letsuffix (new)*

THE FRONT END READS A SCALE FACTOR AFTER AN EXPONENT. ft_numparse took "the scale factor or the exponent (can't have both)": let x = 1.5e2u was 150 where an element value 1.5e2u is 1.5e-4, 1e3meg 1000, 1e-3k 0.001, 3.3e2mil 330, without a word (campaign 3 of 2026-10-10 F4) -- INPevaluate and numparam apply both. The scale-factor table is its own function and is consulted after an exponent too; the powers add, mil's 25.4 with them, and INPdecimal reads the sum as one exact decimal (E-643), so the value is INPevaluate's to the bit. An e with no digits after it is still trailing text (1em is 1, E-426). VERIFIED: letsuffix [1]..[6] under both solvers (4 fail on E-857; [5] and [6] are controls); 300 random spellings with an exponent and a suffix give the element value to the bit.

E-860 *src/maths/ni/niaciter.c (ac_refine after the solve in NIacIter and NIspSolve), src/maths/KLU/klusmp.c and src/maths/sparse/spsmp.c (SMPcCopyLoaded, Sparse's loaded values before the factorization with the element list kept per matrix; SMPcResidual, from the copy or from the matrix KLU keeps), src/include/ngspice/smpdefs.h; examples acrefine (new)*

THE AC SOLVE REFINES ITS ANSWER. The ac factorization takes a pivot down to pivrel (1e-3) of its column's largest entry, and Sparse orders the matrix at the first frequency only; on inductive networks at a few hertz -- a 1 nH branch row carries jwL ~ 1e-8 beside entries of order 1 -- that cost up to 6 digits on matrices of condition 5: 22 of 300 networks under Sparse (3.4e-6), 5 under KLU (2e-7), where any backward-stable solve gives 1e-15 (campaign 3 of 2026-10-10 F5). After each ac solve the residual b - Ax gives the componentwise backward error max |r_i| / (|A||x| + |b|)_i; where it is above 64 eps, A d = r is solved with the same factors and x + d kept if better (two steps at most). A row below DBL_MIN/eps is not measured; a point solved well keeps its bits. Cost, one pass over the matrix per frequency (and the Sparse copy): +6 to +25 % on circuits whose devices load almost nothing. VERIFIED: acrefine [1]..[4] under both solvers (3 fail on E-859 under Sparse, 2 under KLU; [4] a control); Family A's 300 low-frequency networks 22/5 flagged -> 1/1 (both solvers alike, not the solve), its 300 ordinary ones 4 -> 0.

E-861 *src/spicelib/analysis/cktsens.c (sens_sens: a central difference, the load at p - delta in the nominal load's place, where the parameter is not zero); examples senscentral (new)*

SENSITIVITIES BY A CENTRAL DIFFERENCE. sens divided the change in a device's stamp at p + delta (delta = 1e-6 of p) by delta: a gain or a source enters linearly and came out exact, a resistance enters as 1/R and (1/(R+d) - 1/R)/d is low by 1/(1 + 1e-6), so every sensitivity to a resistance was low by exactly that, in dc and ac (campaign 3 of 2026-10-10 F6). Where the parameter is not zero the step is central, p + delta against p - delta over 2 delta, with the same number of loads; the lower side goes through E-417's collapse check, E-825's $fatal count and the model snapshot as the upper one. A parameter at zero keeps the one-sided Sens_Abs_Delta. A diode's isr, given only once written, now reports the slope of the recombination branch it switches on (-1.8e9) where it reported the switch (-1.8e15). VERIFIED: senscentral [1]..[5] under both solvers (3 fail on E-859; [4] and [5] controls); Family C's 6 267 flags at 1e-6 -> 79, all values zero at the circuit's scale but one gain at 2.6e-9.

E-863 *src/maths/cmaths/cmath1.c (d_tan: the result's finiteness checked, not tan(x) != 0); examples lettan (new)*

LET TAKES TAN(0). d_tan, the front end's tangent of a real vector, checked rcheck(tan(x) != 0) before taking the tangent, which refused a valid result: let x = tan(0) failed "argument out of range for tan" while tan(0.5) was fine, and so did any vector with a 0 in it and tan(0) under units=degrees -- tan(0) is 0 in a .param and a B-source, and the complex tan took 0 already (campaign 3 of 2026-10-10 F8). The pole the check meant to guard is never exactly met by a double (tan of pi/2 rounded is 1.6e16, which it let through). The tangent is taken first and checked to be finite: only a NaN or infinite argument is refused. VERIFIED: lettan [1]..[5] under both solvers (3 fail on E-861; [4] and [5] controls).

E-864 *src/maths/cmaths/cmath2.c (cx_mod: a divisor with no imaginary part takes both parts modulo itself); examples cplxmod (new)*

COMPLEX % TAKES A REAL DIVISOR. E-480 made complex % a real modulo part by part, and it checked that both of the divisor's parts were nonzero; a real divisor promoted to complex has an imaginary part of 0, so let a = v(1) % 2 after an ac failed "argument out of range for mod" while v(1) % (2+j(1)) evaluated (campaign 3 of 2026-10-10 F9). A divisor with no imaginary part now takes both parts modulo itself: (a + jb) % d is fmod(a, d) + j fmod(b, d). A divisor of 0, or a complex one with a zero real part, is still refused; a complex divisor keeps the part-by-part rule. VERIFIED: cplxmod [1]..[5] under both solvers (2 fail on E-863; [3]..[5] controls).

E-865 *src/frontend/parse-bison.y ('^' %left); examples powassoc (new)*

^ ASSOCIATES LEFT TO RIGHT IN LET, AS EVERYWHERE ELSE. The manual (2.11.5) says equal-precedence binary operators evaluate left to right; the .param and the B-source take 2^3^2 as (2^3)^2 = 64, and so does Verilog-A's LRM (openvaf-r: 64); let's grammar had '^' %right and gave 512, and 2^0.5^2 = 1.189 where the others give 2 (campaign 3 of 2026-10-10 F10; the page's "as Verilog-A" was wrong). '^' is %left now; 2^-3^2 stays 2^-9 everywhere. The manual's precedence table puts unary minus above ** -- Verilog-A's rule, -2**2 = 4 -- while all three ngspice parsers give -2^2 = -4 (NEG below '^'); the table is what is wrong, and the handbook gives the order the parsers use. VERIFIED: powassoc [1]..[5] under both solvers (2 fail on E-863; [3]..[5] controls).

E-866 *src/frontend/rawfile.c (raw_write leaves a name of the form f(...) unwrapped), src/frontend/evaluate.c (ft_evaluate: a lone vector found by all/allv/alli/ally/alle keeps its name); examples pzwrite (new)*

A PZ PLOT'S VECTORS KEEP THEIR NAMES IN WRITE AND PRINT ALL. write wraps a voltage-typed vector in v(...) unless it is a node voltage or the scale; a pz plot's roots are voltage-typed, so zero(1), pole(2) were written v(zero(1)), v(pole(2)) and loaded back under those names. And all, on a plot holding one vector, was that vector renamed after the expression: print all

printed all = -1.5e+06,0, and write wrote the pole twice, the second time as v(all) (campaign 3 of 2026-10-10 F11). A name already of the form f(...) is written as it is (op plots still write v(1), i(v1)), and a lone vector found by an all-word keeps its own name. A script that read a lone root as all = ... reads pole(1) = ... now; six suites that did (analyses, klupz, osdislips, pzhb, saveguard, stdaudit) read both. VERIFIED: pzwrite [1]..[6] under both solvers (4 fail on E-863; [5] and [6] controls).

E-867 *src/maths/ni/niiter.c (ni_refine after the solve in NIiter, ni_residual, ni_scratch), src/maths/KLU/klusmp.c and src/maths/sparse/spsmp.c (SMPluFac and SMPreorder copy Sparse's loaded values into SMPloadCopy between the diagonal gmin and the factorization; SMPresidual from the copy or from the matrix KLU keeps), src/include/ngspice/smpdefs.h; examples realrefine (new)*

THE DC AND TRANSIENT SOLVE REFINES ITS ANSWER. Under Sparse a follower built from a VCVS of gain A, its feedback through a 0 V source, lost about A times the rounding of a double: 1.8e-5 at the operating point at a gain of 1e12 after 126 iterations of gmin stepping, 2.4e-4 through a transient, 25 % at 1e15; and a dc sweep of a resistance kept the pivot order of its first point, Sparse's refactor testing only for an exactly zero pivot, so an ammeter beside the swept load read up to 0.5 % wrong (Sparse and KLU campaign of 2026-10-11, F1 and F2). KLU, which scales its rows, was exact on both. After every solve but the junction guess's, the residual now gives the backward error, each row against the larger of |A||x| + |b| and |A| times the largest unknown of each kind, so a row whose unknowns are all at rounding does not count; where that is above 64 eps, A d = r is solved with the same factors and x + d kept while it improves, at most twice. The diagonal gmin and the trust region's shunt are in the copy, so they stay in the solve. A good solve keeps its bits; the cost is one pass over the matrix per Newton iteration (+16 to +22 % on a 20 000-node RC ladder transient, +3 to +7 % on transistor circuits and a mesh). The campaign's F3 deck (teraohm dead ends) is exact too. VERIFIED: realrefine [1]..[9] under both solvers (7 fail under Sparse on E-866, none under KLU; [8] and [9] controls); high-gain family 25 networks wrong under Sparse, 5 left, all of condition 7e16 or more.

E-868 *src/maths/ni/niiter.c (ni_residual: a row counts unless its terms are at rounding, NI_NOISE_ROW; ni_refine: a step weighed against x on common denominators; ni_kind_scale); examples realrefine [10] [11]*

THE REFINEMENT MEASURES EVERY ROW THAT CARRIES MORE THAN ROUNDING. E-867 measured each row against the larger of its own terms and the largest unknown of each kind, which left out the KCL row of a 6e-8 A sensor beside a 4e5 A inductor current: it read 2e-16 and the sensor stayed 1.5e-3 off under Sparse, 1e-7 under KLU (Sparse and KLU campaign of 2026-10-11, F3). A row now counts unless its own terms are below 1024 eps of that scale, and is measured componentwise; a candidate x + d is weighed against x row by row over the larger of the two rows' terms, so a row whose true value is 0, whose terms shrink with its residual, no longer refuses a step that helps it. The sensor is within 1.3e-15 under Sparse and 1.1e-15 under KLU; no high-gain network of the campaign needs gmin stepping under either solver (3 and 2 after E-867); the remaining wrong ones are of condition 7e16 or more under both. VERIFIED: realrefine [10] [11] under both solvers ([10] fails under both on E-867).

E-869 *src/maths/ni/niiter.c (the E-256 guard's row exemption gets a floor of NI_GUARD_ROUND times |G| s, s the largest unknown of each kind); examples guardgain (new)*

THE FALSE-CONVERGENCE GUARD ALLOWS WHAT A HIGH GAIN MAKES OF ROUNDING. An inverting amplifier of VCVS gain 1e12 went through 126 iterations of dynamic gmin stepping under KLU, 3 at 1e11: the control voltage v(out)/A is rounded at the circuit's volts and the gain multiplies that into its branch row, 2.5e-5 in 2.5 of terms, where E-568 exempted a row only below one part per million of its terms (Sparse and KLU campaign of 2026-10-11, N1). A row is now also exempt within 1024 eps of |G| s, the line below which E-868's refinement takes a row for rounding, 2e-13 of the row's own scale; E-256's false point (0.4 A on a 0.6 V divider) is still declined, to the bit. The inverter takes 3 iterations at gains up to 1e15 under both solvers. VERIFIED: guardgain [1]..[5] under both solvers ([1] and [3] fail under KLU on E-867; [4] and [5] controls);

oprobust, linesearch, bsrcconv, explicitvalue, failacct, warmstart pass.